

```

1  import { _decorator, assetManager, CCInteger, director, Director, Enum, Node, NodeEventType, SpriteAtlas,
2  SpriteFrame, UITransform, UIOpacity, UIRenderer, Vec2 } from 'cc';
3  import { EDITOR, JSB } from 'cc/env';
4  enum SizeMode { CUSTOM, TRIMMED, RAW }
5  const { ccclass, property } = _decorator;
6  @ccclass('Corner')
7  class Corner {
8      @property({ displayName: '↙ 左下' })
9      leftBottom: boolean = true;
10     @property({ displayName: '↘ 右下' })
11     rightBottom: boolean = true;
12     @property({ displayName: '↖ 左上' })
13     leftTop: boolean = true;
14     @property({ displayName: '↗ 右上' })
15     rightTop: boolean = true;
16     visible: boolean[] = null;
17 }
18 @ccclass('RoundBox')
19 export class RoundBox extends UIRenderer {
20     @property({ displayName: '图集', type: SpriteAtlas, readonly: true, editorOnly: true, serializable: false })
21     private atlas: SpriteAtlas = null;
22     @property
23     private _spriteFrame: SpriteFrame = null;
24     @property({ displayName: 'Sprite Frame', type: SpriteFrame })
25     private get spriteFrame() { return this._spriteFrame; }
26     private set spriteFrame(val) {
27         this._spriteFrame = val;
28         this.updateSpriteFrame();
29         this.updateUv();
30         if (this._renderData?.chunk) {
31             this.markForUpdateRenderData();
32         }
33     }
34     /**
35      * 运行时换图（皮肤等），与编辑器「Sprite Frame」一致。
36      * 节点未启用或非激活子节点上时尚无 _renderData，只写入 _spriteFrame，等 onEnable 里会
37     updateUv。
38      */
39     public setDisplaySpriteFrame(val: SpriteFrame | null): void {
40         this._spriteFrame = val;
41         this.updateSpriteFrame();
42         if (this._renderData?.chunk) {
43             this.updateUv();
44             this.markForUpdateRenderData();
45         }
46     }
47     @property
48     private _sizeMode: SizeMode = SizeMode.TRIMMED;
49     @property({ displayName: '尺寸模式', type: Enum(SizeMode) })
50     private get sizeMode() { return this._sizeMode; }
51     private set sizeMode(val) {
52         this._sizeMode = val;
53         this.updateSizeMode();
54     }
55     @property({ displayName: '顶点数 / 三角形数', readonly: true, editorOnly: true, serializable: false })
56     private vertexTriangle: Vec2 = new Vec2(0, 0);
57     @property
58     private _segment: number = 5;

```

```

59     @property({ displayName: '..... 线段数量', type: CCInteger })
60     private get segment() { return this._segment; }
61     private set segment(val) {
62         this._segment = Math.max(val, 1);
63         this.createData();
64         this.updateLocal();
65         this.updateUv();
66         this.updateColor();
67         this.markForUpdateRenderData();
68     }
69     @property
70     private _radius: number = 100;
71     @property({ displayName: '..... 圆角半径' })
72     private get radius() { return this._radius; }
73     private set radius(val) {
74         this._radius = Math.max(val, 0);
75         this.updateLocal();
76         this.updateUv();
77         this.markForUpdateRenderData();
78     }
79     @property
80     private _corner: Corner = new Corner();
81     @property({ displayName: '..... 圆角可见性' })
82     private get corner() { return this._corner; }
83     private set corner(val) {
84         this._corner = val;
85         this.updateCorner();
86         this.createData();
87         this.updateLocal();
88         this.updateUv();
89         this.updateColor();
90         this.markForUpdateRenderData();
91     }
92     private uiTrans: UITransform = null;    //当前节点的 UITransform 对象
93     private left: number = 0;              //左边缘本地坐标
94     private bottom: number = 0;            //下边缘本地坐标
95     private locals: number[][] = [];       //顶点本地坐标
96     /** 上一帧链式 UIOpacity 乘积; 父节点 tween 时引擎不会每帧调 updateColor, 需自行对齐顶点
97     alpha */
98     private _lastCascadeOpacityMul = -1;
99     /**
100     * `MenuOverlayWindow` 等下一帧 `forceRefreshRender` 时, 可能早于本组件 `__preload`, `uiTrans` 尚
101     未赋值。
102     */
103     private ensureUiTransform(): boolean {
104         if (!this.node?.isValid) {
105             return false;
106         }
107         if (!this.uiTrans || !this.uiTrans.isValid) {
108             this.uiTrans = this.node.getComponent(UITransform) ?? this.node.addComponent(UITransform);
109         }
110         return !!this.uiTrans && this.uiTrans.isValid;
111     }
112     __preload(): void {
113         super.__preload();
114         this._assembler = {                //定制 assembler
115             updateColor: this.updateColor.bind(this),
116             updateRenderData: this.updateRenderData.bind(this),

```

```

117         fillBuffers: this.fillBuffer.bind(this),
118     };
119     this.uiTrans = this.node.getComponent(UITransform) ?? this.node.addComponent(UITransform);
120     this._useVertexOpacity = true;
121     this['updateMaterial']();
122     this.updateSpriteFrame();
123     this.updateCorner();
124     this.updateLocal();
125 }
126 onEnable(): void {
127     super.onEnable();
128     this._lastCascadeOpacityMul = -1;
129     this.createData();
130     this.updateUv();
131     JSB ? director.once(Director.EVENT_AFTER_DRAW, this.updateColor, this) : this.updateColor();
132     this.node.on(NodeEventType.SIZE_CHANGED, this.onSizeChanged, this);
133     this.node.on(NodeEventType.ANCHOR_CHANGED, this.onAnchorChanged, this);
134 }
135 /**
136  * 整棵 UI 曾处于 inactive、首次被打开时，部分环境下首帧 `_spriteFrame.texture` /
137  * `renderData.chunk`
138  * 尚未就绪，`updateUv` 会空跑，圆角底图不显示；下一帧再算一次即可（由 MenuOverlayWindow 等
139 调用）。
140  */
141 forceRefreshRender(): void {
142     if (!this.isValid || !this.ensureUiTransform()) {
143         return;
144     }
145     /** MenuOverlayWindow 等下一帧回调时，可能早于 __preload 里对 corner.visible 的赋值 */
146     this.updateCorner();
147     this.createData();
148     this.updateLocal();
149     this.updateUv();
150     this.updateXy();
151     this.updateColor();
152     this.markForUpdateRenderData();
153 }
154 onDisable(): void {
155     super.onDisable();
156     this.node.off(NodeEventType.SIZE_CHANGED, this.onSizeChanged, this);
157     this.node.off(NodeEventType.ANCHOR_CHANGED, this.onAnchorChanged, this);
158 }
159 //修改节点尺寸后，更新顶点数据，并根据 sizeMode 设置图片宽高
160 private onSizeChanged(): void {
161     if (!this._spriteFrame) return;
162     if (!this.ensureUiTransform()) return;
163     this.updateLocal();
164     this.updateXy();
165     let cw = this.uiTrans.width, ch = this.uiTrans.height;
166     let size = this._spriteFrame['_originalSize'], rect = this._spriteFrame['_rect'];
167     switch (this._sizeMode) {
168         case SizeMode.TRIMMED: if (cw === rect.width && ch === rect.height) return; break;
169         case SizeMode.RAW: if (cw === size.width && ch === size.height) return; break;
170     }
171     this._sizeMode = SizeMode.CUSTOM;
172 }
173 private onAnchorChanged(): void {
174     this.updateLocal();

```

```

175         this.updateXy();
176     }
177     //可以传入单张图片 Texture2D, 或 Atlas 图集帧 (支持合批, 推荐)
178     private updateSpriteFrame(): void {
179         let spriteFrame = this._spriteFrame;
180         if (!spriteFrame) { this.atlas = null; return; }
181         this._renderData && (this._renderData.textureDirty = true);
182         this.updateSizeMode();
183         if (EDITOR) {
184             if (!spriteFrame['_atlasUuid']) { this.atlas = null; return; }
185             assetManager.loadAny(spriteFrame['_atlasUuid'], (err: Error, asset: SpriteAtlas) => {
186                 if (err) { this.atlas = null; return; }
187                 this.atlas = asset;
188             });
189         }
190     }
191     private updateCorner(): void {
192         if (!this._corner) {
193             this._corner = new Corner();
194         }
195         const corner = this._corner;
196         corner.visible = [corner.leftBottom, corner.rightBottom, corner.rightTop, corner.leftTop];
197     }
198     //设置顶点数和三角形数
199     private createData(): void {
200         if (!this._corner?.visible || this._corner.visible.length < 4) {
201             this.updateCorner();
202         }
203         const renderData = (this._renderData = this.requestRenderData());
204         if (!renderData) {
205             return;
206         }
207         let vertexTriangle = [4, 2];
208         let cornerCnt = 0;
209         const visible = this._corner.visible;
210         for (let i = 0; i < 4; visible[i++] && ++cornerCnt);
211         vertexTriangle = [12 + (this._segment - 1) * cornerCnt, 6 + this._segment * cornerCnt];
212         renderData.dataLength = vertexTriangle[0];
213         renderData.resize(vertexTriangle[0], 3 * vertexTriangle[1]);
214         EDITOR && ([this.vertexTriangle.x, this.vertexTriangle.y] = vertexTriangle);
215         this.updateIndices();
216     }
217     //计算顶点索引
218     private updateIndices(): void {
219         const renderData = this._renderData;
220         if (!renderData?.chunk) {
221             return;
222         }
223         let indices = new Uint16Array(renderData.chunk.indexCount);
224         const ROUND_IB = [0, 9, 11, 0, 11, 1, 2, 8, 10, 2, 4, 8, 3, 5, 7, 3, 7, 6];
225         for (let i = ROUND_IB.length - 1; i > -1; indices[i] = ROUND_IB[i--]);
226         for (let i = 0, offset = ROUND_IB.length, id = 36, visible = this._corner.visible; i < 4; ++i) {
227             if (!visible[i]) continue;
228             let o = 3 * i;
229             let a = o + 1;
230             let b = id / 3;
231             for (let j = 0, len = this._segment - 1; j < len; ++j) {
232                 indices[offset++] = o;

```

```

233             indices[offset++] = a;
234             indices[offset++] = b;
235             a = b++;
236             id += 3;
237         }
238         indices[offset++] = o;
239         indices[offset++] = a;
240         indices[offset++] = o + 2;
241     }
242     JSB ? renderData.chunk.setIndexBuffer(indices) : (renderData.indices = indices);
243 }
244 //计算本地坐标
245 updateLocal(): void {
246     if (!this.ensureUiTransform()) {
247         return;
248     }
249     if (!this._corner?.visible || this._corner.visible.length < 4) {
250         this.updateCorner();
251     }
252     let ut = this.uiTrans, cw = ut.width, ch = ut.height, ax = ut.anchorX, ay = ut.anchorY;
253     let l = this.left = -cw * ax, b = this.bottom = -ch * ay, r = cw * (1 - ax), t = ch * (1 - ay);
254     let locals = this.locals = [];
255     let radius = Math.min(this._radius, Math.min(cw, ch) / 2);
256     let lo = l + radius, bo = b + radius, ro = r - radius, to = t - radius;
257     let corner = this._corner;
258     locals[0] = [lo, corner.leftBottom ? bo : b];
259     locals[1] = [l, locals[0][1]];
260     locals[2] = [lo, b];
261     locals[3] = [ro, corner.rightBottom ? bo : b];
262     locals[4] = [ro, b];
263     locals[5] = [r, locals[3][1]];
264     locals[6] = [ro, corner.rightTop ? to : t];
265     locals[7] = [r, locals[6][1]];
266     locals[8] = [ro, t];
267     locals[9] = [lo, corner.leftTop ? to : t];
268     locals[10] = [lo, t];
269     locals[11] = [l, locals[9][1]];
270     let radian = Math.PI / (this._segment << 1);
271     let sin = Math.sin(radian), cos = Math.cos(radian);
272     for (let i = 0, offset = 12, visible = corner.visible; i < 4; ++i) {
273         if (!visible[i]) continue;
274         let id = i * 3;
275         let ox = locals[id][0], oy = locals[id][1];
276         let delTX = locals[id + 1][0] - ox, delTY = locals[id + 1][1] - oy;
277         for (let j = 0, len = this._segment - 1; j < len; ++j) {
278             locals[offset] = [ox + delTX * cos - delTY * sin, oy + delTY * cos + delTX * sin];
279             delTX = locals[offset][0] - ox;
280             delTY = locals[offset][1] - oy;
281             ++offset;
282         }
283     }
284 }
285 //根据本地坐标，计算 XY 坐标
286 updateXy(): void {
287     let renderData = this._renderData, data = renderData.data, locals = this.locals;
288     for (let i = locals.length - 1; i > -1; --i) {
289         let local = this.locals[i];
290         data[i].x = local[0];

```

```

291         data[i].y = local[1];
292     }
293     !JSB && (renderData.vertDirty = true);
294 }
295 updateUv(): void {
296     let spriteFrame = this._spriteFrame;
297     if (!spriteFrame) return;
298     if (!this.ensureUITransform()) return;
299     let renderData = this._renderData;
300     if (!renderData?.chunk) return;
301     let vb = renderData.chunk.vb, locals = this.locals;
302     let ut = this.uiTrans, cw = ut.width, ch = ut.height, l = this.left, b = this.bottom;
303     for (let i = 3, len = vb.length, step = renderData.floatStride, id = 0; i < len; i += step, ++id) {
304         let local = locals[id];
305         vb[i] = (local[0] - l) / cw;
306         vb[i + 1] = (local[1] - b) / ch;
307     }
308     let uv = spriteFrame.uv;
309     if (spriteFrame['_rotated']) {
310         let uvL = uv[0], uvB = uv[1], uvW = uv[4] - uvL, uvH = uv[3] - uvB;
311         for (let i = 3, len = vb.length, step = renderData.floatStride; i < len; i += step) {
312             let tmp = vb[i];
313             vb[i] = uvL + vb[i + 1] * uvW;
314             vb[i + 1] = uvB + tmp * uvH;
315         }
316     } else {
317         let uvL = uv[0], uvB = uv[1], uvW = uv[2] - uvL, uvH = uv[5] - uvB;
318         for (let i = 3, len = vb.length, step = renderData.floatStride; i < len; i += step) {
319             vb[i] = uvL + vb[i] * uvW;
320             vb[i + 1] = uvB + vb[i + 1] * uvH;
321         }
322     }
323 }
324 /**
325  * VERTEX 填充色类型下, Batch2D 不会对顶点再乘 updateOpacity (见引擎 _handleUIRenderer 里
326  * VERTEX 分支)。
327  * 因此必须把节点链上 UIOpacity 的乘积并入顶点 alpha, 否则父节点 tween UIOpacity(如 Toast 根)
328  * 时底图不会变淡。
329  */
330 private cascadeUiOpacityMultiplier(): number {
331     let m = 1;
332     for (let n: Node | null = this.node; n; n = n.parent) {
333         const ui = n.getComponent(UIOpacity);
334         if (ui && ui.enabledInHierarchy) {
335             m *= ui.opacity / 255;
336         }
337     }
338     return m;
339 }
340 //计算顶点颜色
341 updateColor(): void {
342     let renderData = this._renderData;
343     if (!renderData?.chunk) return;
344     let vb = renderData.chunk.vb, color = this._color;
345     const opacityMul = this.cascadeUiOpacityMultiplier();
346     this._lastCascadeOpacityMul = opacityMul;
347     let r = color.r / 255, g = color.g / 255, b = color.b / 255, a = (color.a / 255) * opacityMul;
348     for (let i = 5, len = vb.length, step = renderData.floatStride; i < len; i += step) {

```

```

349         vb[i] = r;
350         vb[i + 1] = g;
351         vb[i + 2] = b;
352         vb[i + 3] = a;
353     }
354     renderData.chunk.meshBuffer?.setDirty();
355 }
356 protected lateUpdate(): void {
357     if (!this._renderData?.chunk || !this._spriteFrame?.texture) {
358         return;
359     }
360     const m = this.cascadeUiOpacityMultiplier();
361     if (Math.abs(m - this._lastCascadeOpacityMul) < 1 / 2550) {
362         return;
363     }
364     this.updateColor();
365     this.markForUpdateRenderData();
366 }
367 //根据尺寸模式，修改节点尺寸
368 updateSizeMode(): void {
369     if (!this._spriteFrame) return;
370     if (!this.ensureUiTransform()) return;
371     switch (this._sizeMode) {
372         case SizeMode.TRIMMED: this.uiTrans.setContentSize(this._spriteFrame['_rect'].size); break;
373         case SizeMode.RAW: this.uiTrans.setContentSize(this._spriteFrame['_originalSize']); break;
374     }
375 }
376 //调用 markForUpdateRenderData 后该函数会被触发
377 updateRenderData(): void {
378     if (!this._renderData || !this._spriteFrame?.texture) return;
379     this.updateXy();
380     this._renderData.updateRenderData(this, this._spriteFrame);
381 }
382 protected _render(render: any): void {
383     render.commitComp(this, this._renderData, this._spriteFrame, this._assembler, null);
384 }
385 protected _canRender(): boolean {
386     return super._canRender() && !(this._spriteFrame?.texture);
387 }
388 //Web 平台，将 renderData 的数据提交给 GPU 渲染
389 //原生平台并不会执行该函数，引擎另外实现了渲染函数
390 fillBuffer(): void {
391     let renderData = this._renderData;
392     if (!renderData) return;
393     let chunk = renderData.chunk;
394     if (this.node.hasChangedFlags || renderData.vertDirty) {
395         let data = renderData.data, vb = renderData.chunk.vb, m = this.node.worldMatrix;
396         for (let step = renderData.floatStride, len = vb.length, i = 0, id = 0; i < len; i += step, ++id) {
397             let x = data[id].x, y = data[id].y, rhw = m.m03 * x + m.m07 * y + m.m15;
398             rhw = rhw ? 1 / rhw : 1;
399             vb[i] = (m.m00 * x + m.m04 * y + m.m12) * rhw;
400             vb[i + 1] = (m.m01 * x + m.m05 * y + m.m13) * rhw;
401         }
402         renderData.vertDirty = false;
403     }
404     let vid = chunk.vertexOffset;
405     let meshBuffer = chunk.meshBuffer;
406     let iData = meshBuffer.iData;

```

```

407         let indexOffset = meshBuffer.indexOffset;
408         let indices = renderData.indices;
409         for (let i = 0; i < indices.length; iData[indexOffset++] = vid + indices[i++]);
410         meshBuffer.indexOffset += indices.length;
411     }
412 }
413 declare global {
414     module gi {
415         class RoundBox extends UIRenderer {
416             static SizeMode: typeof SizeMode;           //尺寸模式枚举
417             spriteFrame: SpriteFrame;                   //纹理，支持单张图片和 Atlas 图集帧
418             sizeMode: SizeMode;                         //尺寸模式
419             roundSegment: number;                       //线段数量
420             roundRadius: number;                       //圆角半径
421             roundCorner: Corner;                       //圆角可见性
422         }
423     }
424 }
425 ((globalThis as any).gi || {}).RoundBox ||= Object.assign(RoundBox, { SizeMode: SizeMode });
426 // ----- assets/Scripts/Component/BrownianSquareFieldView.ts -----
427 import {
428     _decorator,
429     Color,
430     Component,
431     Graphics,
432     Node,
433     UITransform,
434 } from 'cc';
435 import { clamp } from '../Utils/Utils';
436 const { ccclass } = _decorator;
437 /**#region 参数（只改这里；勿用 @property，避免场景序列化旧值覆盖脚本）
438  ** 小正方形数量 n */
439 const SQUARE_COUNT = 25;
440 /** 边长基准 x（设计像素） */
441 const SIDE_BASE = 60;
442 /** 边长随机幅度 y：实际边长  $\in [x-y, x+y]$  */
443 const SIDE_JITTER = 20;
444 /** 边长目标值随机游走（设计像素/秒），在  $[x-y, x+y]$  内缩放 */
445 const SIDE_SCALE_TARGET_WANDER = 10;
446 /** 边长向目标靠拢速度（越大缩放响应越快） */
447 const SIDE_SCALE_LERP = 4;
448 /** 边框厚度占边长比例 a（0.08 = 8%） */
449 const BORDER_THICKNESS_RATIO = 0;
450 /** 圆角半径占边长比例 b（0.12 = 12%） */
451 const CORNER_RADIUS_RATIO = 0.1;
452 /** 边框不透明度基准 d（0~255） */
453 const OPACITY_BASE = 10;
454 /** 不透明度随机幅度 c（0.2 =  $\pm 20\%$  相对 d） */
455 const OPACITY_JITTER_RATIO = 0.2;
456 /** 边框相对填充的 RGB 深度（0.55 = 边框更暗；1 则与填充同色） */
457 const STROKE_DARKEN_RATIO = 0.55;
458 /** 游走速度基准 v（设计像素/秒） */
459 const SPEED_BASE = 20;
460 /** 速度随机幅度 u：实际速度  $\in [v-u, v+u]$  */
461 const SPEED_JITTER = 5;
462 /** 布朗运动：每秒速度方向最大抖动（弧度） */
463 const BROWNIAN_ANGLE_JITTER = 2.8;
464 /** 布朗运动：每秒速度标量随机游走强度 */

```



```

465 const BROWNIAN_SPEED_WANDER = 28;
466 /** 初始撒点：在网格单元内随机偏移幅度（0.7 = 单元 70% 范围内抖动） */
467 const SPAWN_CELL_JITTER = 0.7;
468 /** 运行时互斥强度（越大越均匀；0 关闭） */
469 const SEPARATION_STRENGTH = 1;
470 /** 互斥迭代次数（每帧，2~3 通常够用） */
471 const SEPARATION_ITERATIONS = 2;
472 /** 中心最小间距 = (两边长之和)/2 × 该系数（<1 允许轻贴，>1 留缝） */
473 const SEPARATION_GAP_RATIO = 1.4;
474 /** 子节点容器名 */
475 const PARTICLE_LAYER_NAME = 'BrownianSquareLayer';
476 //endregion
477 interface IBrownianSquareParticle {
478     node: Node;
479     graphics: Graphics;
480     side: number;
481     /** 边长缩放目标（布朗游走，限制在  $x \pm y$ ） */
482     sideTarget: number;
483     opacity: number;
484     borderRatio: number;
485     cornerRatio: number;
486     speed: number;
487     angle: number;
488 }
489 /**
490  * 挂载在任意带 UITransform 的节点上：按节点尺寸生成 n 个圆角小正方形（填充与边框同色），
491  * 在范围内布朗运动游走。改参数请编辑本文件顶部常量区。
492  */
493 @ccclass('BrownianSquareFieldView')
494 export class BrownianSquareFieldView extends Component {
495     private _layerNode: Node | null = null;
496     private _particles: IBrownianSquareParticle[] = [];
497     private _boundsLeft = 0;
498     private _boundsRight = 0;
499     private _boundsBottom = 0;
500     private _boundsTop = 0;
501     private _lastFieldWidth = -1;
502     private _lastFieldHeight = -1;
503     protected onLoad(): void {
504         this._ensureFieldTransform();
505         this._rebuildField();
506     }
507     protected onEnable(): void {
508         this._rebuildField();
509     }
510     protected onDisable(): void {
511         this._clearParticles();
512     }
513     protected update(dt: number): void {
514         if (this._particles.length === 0) {
515             return;
516         }
517         if (this._syncBoundsIfResized()) {
518             this._rebuildField();
519             return;
520         }
521         this._stepBrownianMotion(dt);
522     }

```

```
523  /** 尺寸变更后手动重建 */
524  rebuild(): void {
525      this._rebuildField();
526  }
527  //region 构建
528  private _ensureFieldTransform(): UITransform | null {
529      const ut = this.getComponent(UITransform) ?? this.addComponent(UITransform);
530      return ut;
531  }
532  private _ensureLayerNode(): Node {
533      let layer = this.node.getChildByName(PARTICLE_LAYER_NAME);
534      if (!layer?.isValid) {
535          layer = new Node(PARTICLE_LAYER_NAME);
536          layer.setParent(this.node);
537      }
538      layer.setPosition(0, 0, 0);
539      if (!layer.getComponent(UITransform)) {
540          layer.addComponent(UITransform);
541      }
542      this._layerNode = layer;
543      return layer;
544  }
545  private _syncBoundsIfResized(): boolean {
546      const ut = this._ensureFieldTransform();
547      if (!ut) {
548          return false;
549      }
550      const w = ut.width;
551      const h = ut.height;
552      if (w === this._lastFieldWidth && h === this._lastFieldHeight) {
553          return false;
554      }
555      return true;
556  }
557  private _updateBounds(): void {
558      const ut = this._ensureFieldTransform();
559      if (!ut) {
560          return;
561      }
562      const w = ut.width;
563      const h = ut.height;
564      this._lastFieldWidth = w;
565      this._lastFieldHeight = h;
566      this._boundsLeft = -ut.anchorX * w;
567      this._boundsRight = this._boundsLeft + w;
568      this._boundsBottom = -ut.anchorY * h;
569      this._boundsTop = this._boundsBottom + h;
570  }
571  private _rebuildField(): void {
572      this._updateBounds();
573      this._clearParticles();
574      const layer = this._ensureLayerNode();
575      const count = Math.max(0, Math.floor(SQUARE_COUNT));
576      for (let i = 0; i < count; i++) {
577          this._particles.push(this._createParticle(layer, i, count));
578      }
579      if (SEPARATION_STRENGTH > 0) {
580          this._applySpatialSeparation(SEPARATION_ITERATIONS * 3);
```

```

581     }
582 }
583 private _createParticle(layer: Node, index: number, total: number): IBrownianSquareParticle {
584     const side = this._randomSide();
585     const opacity = this._randomOpacity();
586     const speed = this._randomSpeed();
587     const angle = Math.random() * Math.PI * 2;
588     const half = side * 0.5;
589     const { x, y } = this._sampleGridSpawnPosition(index, total, half);
590     const node = new Node(`BrownianSq_${index}`);
591     node.setParent(layer);
592     node.setPosition(x, y, 0);
593     const ut = node.addComponent(UITransform);
594     ut.setContentSize(side, side);
595     ut.setAnchorPoint(0.5, 0.5);
596     const graphics = node.addComponent(Graphics);
597     const particle: IBrownianSquareParticle = {
598         node,
599         graphics,
600         side,
601         sideTarget: side,
602         opacity,
603         borderRatio: BORDER_THICKNESS_RATIO,
604         cornerRatio: CORNER_RADIUS_RATIO,
605         speed,
606         angle,
607     };
608     this._drawSquareStroke(particle);
609     return particle;
610 }
611 private _drawSquareStroke(p: IBrownianSquareParticle): void {
612     const g = p.graphics;
613     if (!g?.isValid) {
614         return;
615     }
616     const side = p.side;
617     const half = side * 0.5;
618     const lineWidth = Math.max(1, side * p.borderRatio);
619     const radius = clamp(side * p.cornerRatio, 0, half);
620     g.clear();
621     g.lineWidth = lineWidth;
622     g.lineJoin = Graphics.LineJoin.ROUND;
623     g.lineCap = Graphics.LineCap.ROUND;
624     const alpha = clamp(Math.round(p.opacity), 0, 255);
625     const strokeRgb = clamp(Math.round(255 * STROKE_DARKEN_RATIO), 0, 255);
626     g.fillColor = new Color(255, 255, 255, alpha);
627     g.strokeColor = new Color(strokeRgb, strokeRgb, strokeRgb, alpha);
628     g.roundRect(-half, -half, side, side, radius);
629     g.fill();
630     g.stroke();
631 }
632 private _clearParticles(): void {
633     for (const p of this._particles) {
634         if (p.node?.isValid) {
635             p.node.destroy();
636         }
637     }
638     this._particles.length = 0;

```

```

639     }
640     //endregion
641     //region 空间平衡
642     /** 按网格分区生成初始位置，避免纯随机留下大面积空白 */
643     private _sampleGridSpawnPosition(index: number, total: number, half: number): { x: number; y: number } {
644         const minX = this._boundsLeft + half;
645         const maxX = this._boundsRight - half;
646         const minY = this._boundsBottom + half;
647         const maxY = this._boundsTop - half;
648         const spanX = maxX - minX;
649         const spanY = maxY - minY;
650         const centerX = (minX + maxX) * 0.5;
651         const centerY = (minY + maxY) * 0.5;
652         if (spanX <= 0 || spanY <= 0 || total <= 0) {
653             return { x: centerX, y: centerY };
654         }
655         const aspect = spanX / Math.max(spanY, 1);
656         const cols = Math.max(1, Math.ceil(Math.sqrt(total * aspect)));
657         const rows = Math.max(1, Math.ceil(total / cols));
658         const col = index % cols;
659         const row = Math.floor(index / cols);
660         const cellW = spanX / cols;
661         const cellH = spanY / rows;
662         const jitter = clamp(SPAWN_CELL_JITTER, 0, 1);
663         const x = clamp(
664             minX + (col + 0.5) * cellW + (Math.random() - 0.5) * cellW * jitter,
665             minX,
666             maxX,
667         );
668         const y = clamp(
669             minY + (row + 0.5) * cellH + (Math.random() - 0.5) * cellH * jitter,
670             minY,
671             maxY,
672         );
673         return { x, y };
674     }
675     /** 运行时轻量互斥：过近则推开，减轻聚堆与大块空白 */
676     private _applySpatialSeparation(iterations: number): void {
677         if (SEPARATION_STRENGTH <= 0 || this._particles.length < 2) {
678             return;
679         }
680         const strength = clamp(SEPARATION_STRENGTH, 0, 1);
681         const gapRatio = Math.max(0.1, SEPARATION_GAP_RATIO);
682         for (let pass = 0; pass < iterations; pass++) {
683             for (let i = 0; i < this._particles.length; i++) {
684                 const a = this._particles[i];
685                 if (!a.node?.isValid) {
686                     continue;
687                 }
688                 const posA = a.node.position;
689                 let ax = posA.x;
690                 let ay = posA.y;
691                 for (let j = i + 1; j < this._particles.length; j++) {
692                     const b = this._particles[j];
693                     if (!b.node?.isValid) {
694                         continue;
695                     }
696                     const posB = b.node.position;

```

```

697         let bx = posB.x;
698         let by = posB.y;
699         const dx = bx - ax;
700         const dy = by - ay;
701         const distSq = dx * dx + dy * dy;
702         const minDist = (a.side + b.side) * 0.5 * gapRatio;
703         if (minDist <= 0) {
704             continue;
705         }
706         if (distSq >= minDist * minDist) {
707             continue;
708         }
709         let dist = Math.sqrt(distSq);
710         if (dist < 1e-4) {
711             const angle = Math.random() * Math.PI * 2;
712             const push = minDist * 0.5 * strength;
713             ax -= Math.cos(angle) * push;
714             ay -= Math.sin(angle) * push;
715             bx += Math.cos(angle) * push;
716             by += Math.sin(angle) * push;
717         } else {
718             const overlap = (minDist - dist) * 0.5 * strength;
719             const nx = dx / dist;
720             const ny = dy / dist;
721             ax -= nx * overlap;
722             ay -= ny * overlap;
723             bx += nx * overlap;
724             by += ny * overlap;
725         }
726         const aHalf = a.side * 0.5;
727         const bHalf = b.side * 0.5;
728         ax = clamp(ax, this._boundsLeft + aHalf, this._boundsRight - aHalf);
729         ay = clamp(ay, this._boundsBottom + aHalf, this._boundsTop - aHalf);
730         bx = clamp(bx, this._boundsLeft + bHalf, this._boundsRight - bHalf);
731         by = clamp(by, this._boundsBottom + bHalf, this._boundsTop - bHalf);
732         a.node.setPosition(ax, ay, 0);
733         b.node.setPosition(bx, by, 0);
734     }
735 }
736 }
737 }
738 private _clampParticleToBounds(p: IBrownianSquareParticle, x: number, y: number): { x: number; y: number }
739 {
740     const half = p.side * 0.5;
741     return {
742         x: clamp(x, this._boundsLeft + half, this._boundsRight - half),
743         y: clamp(y, this._boundsBottom + half, this._boundsTop - half),
744     };
745 }
746 //endregion
747 //region 布朗运动
748 private _stepBrownianMotion(dt: number): void {
749     const safeDt = clamp(dt, 0, 0.05);
750     for (const p of this._particles) {
751         if (!p.node?.isValid) {
752             continue;
753         }
754         this._stepSideScale(p, safeDt);

```

```

755         p.angle += (Math.random() - 0.5) * 2 * BROWNIAN_ANGLE_JITTER * safeDt;
756         p.speed = clamp(
757             p.speed + (Math.random() - 0.5) * 2 * BROWNIAN_SPEED_WANDER * safeDt,
758             this._speedMin(),
759             this._speedMax(),
760         );
761         let vx = Math.cos(p.angle) * p.speed;
762         let vy = Math.sin(p.angle) * p.speed;
763         const pos = p.node.position;
764         let x = pos.x + vx * safeDt;
765         let y = pos.y + vy * safeDt;
766         const half = p.side * 0.5;
767         const minX = this._boundsLeft + half;
768         const maxX = this._boundsRight - half;
769         const minY = this._boundsBottom + half;
770         const maxY = this._boundsTop - half;
771         if (x < minX) {
772             x = minX;
773             vx = Math.abs(vx);
774         } else if (x > maxX) {
775             x = maxX;
776             vx = -Math.abs(vx);
777         }
778         if (y < minY) {
779             y = minY;
780             vy = Math.abs(vy);
781         } else if (y > maxY) {
782             y = maxY;
783             vy = -Math.abs(vy);
784         }
785         p.angle = Math.atan2(vy, vx);
786         const clamped = this._clampParticleToBounds(p, x, y);
787         p.node.setPosition(clamped.x, clamped.y, 0);
788     }
789     this._applySpatialSeparation(SEPARATION_ITERATIONS);
790 }
791 //endregion
792 //region 随机参数
793 private _sideMin(): number {
794     return Math.max(4, SIDE_BASE - Math.max(0, SIDE_JITTER));
795 }
796 private _sideMax(): number {
797     return SIDE_BASE + Math.max(0, SIDE_JITTER);
798 }
799 private _randomSide(): number {
800     const min = this._sideMin();
801     const max = this._sideMax();
802     return min + Math.random() * (max - min);
803 }
804 /** 边长在 x±y 范围内随机缩放, 并同步重绘与边界 */
805 private _stepSideScale(p: IBrownianSquareParticle, dt: number): void {
806     const minSide = this._sideMin();
807     const maxSide = this._sideMax();
808     p.sideTarget += (Math.random() - 0.5) * 2 * SIDE_SCALE_TARGET_WANDER * dt;
809     p.sideTarget = clamp(p.sideTarget, minSide, maxSide);
810     const prevSide = p.side;
811     const lerpT = clamp(SIDE_SCALE_LERP * dt, 0, 1);
812     p.side = prevSide + (p.sideTarget - prevSide) * lerpT;

```

```

813         if (Math.abs(p.side - prevSide) < 0.2) {
814             return;
815         }
816         const ut = p.node.getComponent(UITransform);
817         if (ut) {
818             ut.setContentSize(p.side, p.side);
819         }
820         this._drawSquareStroke(p);
821         const pos = p.node.position;
822         const clamped = this._clampParticleToBounds(p, pos.x, pos.y);
823         if (clamped.x !== pos.x || clamped.y !== pos.y) {
824             p.node.setPosition(clamped.x, clamped.y, 0);
825         }
826     }
827     private _randomOpacity(): number {
828         const ratio = Math.max(0, OPACITY_JITTER_RATIO);
829         const factor = 1 + (Math.random() * 2 - 1) * ratio;
830         return clamp(OPACITY_BASE * factor, 0, 255);
831     }
832     private _randomSpeed(): number {
833         const jitter = Math.max(0, SPEED_JITTER);
834         return clamp(SPEED_BASE + (Math.random() * 2 - 1) * jitter, 0, this._speedMax());
835     }
836     private _speedMin(): number {
837         return Math.max(0, SPEED_BASE - Math.max(0, SPEED_JITTER));
838     }
839     private _speedMax(): number {
840         return Math.max(this._speedMin(), SPEED_BASE + Math.max(0, SPEED_JITTER));
841     }
842     //endregion
843 }
844 // ----- assets/Scripts/Config/DebugMockSave.ts -----
845 /**
846  * true: 仅在进程内第一次 DataManager.init 时把 MOCK_PLAYER_DATA 写入本地再 restore（避免切场景
847 第二次 init 覆盖你在 Menu 的修改）。
848  * false: 完全读本地已有存档，不写 mock。
849  * 发版前请改为 false。
850  */
851 export const USE_DEBUG MOCK_SAVE = true;
852 /** 可只写关心的字段；其余在 restore 时与默认存档合并。仅当 USE_DEBUG MOCK_SAVE 为 true 时写
853 入本地 */
854 export const MOCK_PLAYER_DATA: {
855     /** 菜单「开始游戏」进入的关卡 / 上次退出关卡（与解锁无关，可任意已解锁关） */
856     opticalCurrentLevelId?: number;
857     /** 已解锁 / 通关记录: [levelId, steps, ...]; steps=999999 表示仅解锁未通；解锁以出现过的 levelId
858 为准（支持跳关） */
859     opticalLevelClears?: number[];
860     bgmOn?: boolean;
861     sfxOn?: boolean;
862 } = {
863     /** 继续游戏目标关（须在 clears 已解锁的 id 中） */
864     opticalCurrentLevelId: 35,
865     /** 1/2/3/5 已通；4/6 未在表中 → 选关锁定；无 999999 占位也可，有真实步数即视为已解锁 */
866     opticalLevelClears: [1, 999999, 2, 999999, 3, 999999, 4, 999999, 5, 999999, 6, 999999, 7, 999999, 8, 999999,
867 9, 999999, 10, 999999, 11, 999999, 12, 999999, 13, 999999, 14, 999999, 15, 999999, 16, 999999, 17, 999999, 18,
868 999999, 19, 999999, 20, 999999, 21, 999999, 22, 999999, 23, 999999, 24, 999999, 25, 999999, 26, 999999, 27,
869 999999, 28, 999999, 29, 999999, 30, 999999, 31, 999999, 32, 999999, 33, 999999, 34, 999999, 35, 999999, 36,
870 999999, 37, 999999, 38, 999999, 39, 999999, 40, 999999, 41, 999999, 42, 999999, 43, 999999, 44, 999999, 45,

```

```

871 999999, 46, 999999, 47, 999999, 48, 999999, 49, 999999, 50, 999999, 51, 999999, 52, 999999, 53, 999999, 54,
872 999999, 55, 999999, 56, 999999, 57, 999999, 58, 999999, 59, 999999, 60, 999999, 61, 999999, 62, 999999, 63,
873 999999, 64, 999999, 65, 999999, 66, 999999, 67, 999999, 68, 999999, 69, 999999, 70, 999999, 71, 999999, 72,
874 999999, 73, 999999, 74, 999999, 75, 999999, 76, 999999, 77, 999999, 78, 999999, 79, 999999, 80, 999999, 81,
875 999999, 82, 999999, 83, 999999, 84, 999999, 85, 999999, 86, 999999, 87, 999999, 88, 999999, 89, 999999, 90,
876 999999, 91, 999999, 92, 999999, 93, 999999, 94, 999999, 95, 999999, 96, 999999, 97, 999999, 98, 999999, 99,
877 999999, 100, 999999],
878 };
879 // ----- assets/Scripts/Games/OpticalPuzzle/Application/OpticalPuzzleSession.ts -----
880 import { DEV_LEVEL_MINIMAL, type IOpticalLevelConfig } from '../Config/OpticalPuzzleLevelSchema';
881 import { OpticalPuzzleCore, type OpticalPlayStateSnapshot } from '../Core/OpticalPuzzleCore';
882 import type { OpticalBeamSnapshot } from '../Core/OpticalPuzzleCore';
883 import { Direction, MoveAttemptResult, type OpticalBoardSnapshot } from '../Core/OpticalPuzzleTypes';
884 /** OPTICAL_SNAPSHOT_CHANGED 载荷: 棋盘快照 + 本局移动步数 + 触发原因 */
885 export interface OpticalSnapshotNotify {
886     snapshot: OpticalBoardSnapshot;
887     moveCount: number;
888     notifyReason?: OpticalSessionNotifyReason;
889 }
890 import { AUDIO_EFFECT_ENUM, EVENT_ENUM } from '../Utils/Enum';
891 import { PLAY_AUDIO, SHOW_TOAST } from '../Utils/Event';
892 import { OpticalGameFlowState } from './OpticalPuzzleStateMachine';
893 const UNDO_STEPS = 1;
894 /** 撤回键图标横向耗尽阶段: 0 满 → 3 空 (每按一次 +1, 各减 33% 右缘填充) */
895 export const UNDO_ICON_FILL_STAGES = 3;
896 export type OpticalSessionNotifyReason = 'load' | 'move' | 'face' | 'push' | 'undo' | 'reset' | 'complete';
897 /** 已在初始状态时撤回/重开的 Toast 参数 */
898 const INITIAL_STATE_TOAST = {
899     message: '已在初始状态',
900     bgWidth: 360,
901     localY: -250,
902 } as const;
903 /**
904  * 关卡会话: 加载配置、维护历史栈、撤回、重置。
905  * 不依赖场景节点。
906  */
907 export class OpticalPuzzleSession {
908     readonly core = new OpticalPuzzleCore();
909     private _level: IOpticalLevelConfig | null = null;
910     private _history: OpticalPlayStateSnapshot[] = [];
911     private _flow: OpticalGameFlowState = OpticalGameFlowState.BOOT;
912     /** 撤回键图标填充阶段 0~UNDO_ICON_FILL_STAGES */
913     private _undoFillStage = 0;
914     /** 本局有效移动步数 (成功移动/推箱 +1, 撤回 -1, 重开/开局 0) */
915     private _moveCount = 0;
916     /** 本局已观看激励视频解锁参考解的关卡 id (换关清零; 重开本关不清) */
917     private _answerUnlockedLevelId = -1;
918     /** Presentation 订阅: 参数用于区分移动 / 撤回 / 重置等 (音效与埋点) */
919     onStateChanged: ((reason: OpticalSessionNotifyReason) => void) | null = null;
920     get flowState(): OpticalGameFlowState {
921         return this._flow;
922     }
923     /** 撤回键图标填充阶段 (0 全填, 3 不填) */
924     get undoFillStage(): number {
925         return this._undoFillStage;
926     }
927     /** 本局已消耗移动步数 */
928     get moveCount(): number {

```



```

929         return this._moveCount;
930     }
931     /** 是否存在可撤回的有效操作（玩家或元件相对上一检查点有变化） */
932     canUndo(): boolean {
933         return this._flow === OpticalGameFlowState.RUNNING && this._history.length > 1;
934     }
935     /** 局内进行中且相对开局无任何操作记录 */
936     isAtInitialPlayState(): boolean {
937         return this._flow === OpticalGameFlowState.RUNNING && this._history.length <= 1;
938     }
939     private _emitInitialStateToast(): void {
940         SHOW_TOAST.emit(EVENT_ENUM.SHOW_TOAST, { ...INITIAL_STATE_TOAST });
941     }
942     /** 撤回键按下：图标右缘按 33% 递进清空（仅在实际撤回成功时调用） */
943     registerUndoButtonPress(): void {
944         if (this._undoFillStage < UNDO_ICON_FILL_STAGES) {
945             this._undoFillStage += 1;
946         }
947     }
948     /** 图标填充已耗尽（stage=3），再按撤回应走激励广告 */
949     isUndoIconFillExhausted(): boolean {
950         return this._undoFillStage >= UNDO_ICON_FILL_STAGES;
951     }
952     /** 激励广告完整观看后：恢复撤回图标满填，可再按三次 */
953     restoreUndoFillFromRewardedAd(): void {
954         this._resetUndoFillStage();
955     }
956     private _resetUndoFillStage(): void {
957         this._undoFillStage = 0;
958     }
959     /** 使用内置开发关启动（可改为 loadLevel(cfg)） */
960     startWithDevLevel(): void {
961         this.loadLevel(DEV_LEVEL_MINIMAL);
962     }
963     loadLevel(level: IOpticalLevelConfig): void {
964         const prevLevelId = this._level?.levelId ?? -1;
965         this._level = level;
966         if (prevLevelId !== level.levelId) {
967             this._answerUnlockedLevelId = -1;
968         }
969         this.core.reset(level);
970         this._history.length = 0;
971         this._resetUndoFillStage();
972         this._moveCount = 0;
973         this._pushHistory();
974         this._flow = OpticalGameFlowState.RUNNING;
975         this._emit('load');
976     }
977     getSnapshot(): OpticalBoardSnapshot {
978         return this.core.getSnapshot();
979     }
980     getBeamSnapshot(): OpticalBeamSnapshot {
981         return this.core.getBeamSnapshot();
982     }
983     /** 本关参考解是否已在本局通过激励视频解锁 */
984     isAnswerUnlocked(): boolean {
985         return this._level != null && this._answerUnlockedLevelId === this._level.levelId;
986     }

```

```

987      /** 激励视频完整观看后解锁本关参考解（仅本局有效） */
988      unlockAnswerForCurrentLevel(): void {
989          this._answerUnlockedLevelId = this._level?.levelId ?? -1;
990      }
991      /** 四向按钮 / 键盘入口；非 RUNNING 时返回 null */
992      applyDirection(dir: Direction): MoveAttemptResult | null {
993          if (this._flow !== OpticalGameFlowState.RUNNING) {
994              return null;
995          }
996          this.core.setPlayerFacing(dir);
997          const prevTargetLit = this.core.getSnapshot().targets.map((t) => t.lit);
998          const r = this.core.tryMove(dir);
999          if (r === MoveAttemptResult.Blocked) {
1000              // 与 BoardView 「><」 阻拦表情（reason=face）同一条件：凡 Blocked 即失败反馈
1001              PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.OPTICAL_MOVE_FAIL);
1002              this._emit('face');
1003              return r;
1004          }
1005          if (r === MoveAttemptResult.PlayerMoved || r === MoveAttemptResult.PiecePushed) {
1006              this._moveCount += 1;
1007              this._pushHistory();
1008              this._emitMoveSuccessSounds(prevTargetLit);
1009              if (this.core.isAllTargetsLit()) {
1010                  this._flow = OpticalGameFlowState.SETTLEMENT;
1011                  this._emit('complete');
1012              } else if (r === MoveAttemptResult.PiecePushed) {
1013                  this._emit('push');
1014              } else {
1015                  this._emit('move');
1016              }
1017          }
1018          return r;
1019      }
1020      /** 移动成功音；若本步有新亮灯则与点亮音同帧叠播（无先后顺序） */
1021      private _emitMoveSuccessSounds(prevLit: boolean[]): void {
1022          const sfx: AUDIO_EFFECT_ENUM[] = [AUDIO_EFFECT_ENUM.OPTICAL_MOVE_SUCCESS];
1023          if (this._hasNewlyLitTarget(prevLit)) {
1024              sfx.push(AUDIO_EFFECT_ENUM.OPTICAL_TARGET_LIT);
1025          }
1026          PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, sfx);
1027      }
1028      private _hasNewlyLitTarget(prevLit: boolean[]): boolean {
1029          const targets = this.core.getSnapshot().targets;
1030          for (let i = 0; i < targets.length; i++) {
1031              if (targets[i].lit && !prevLit[i]) {
1032                  return true;
1033              }
1034          }
1035          return false;
1036      }
1037      /** 撤回：撤销 UNDO_STEPS 步有效操作（当前为 1 步）；无可撤回步时弹 Toast 并返回 false */
1038      undoBatch(): boolean {
1039          if (!this.canUndo()) {
1040              if (this.isAtInitialPlayState()) {
1041                  this._emitInitialStateToast();
1042              }
1043              return false;
1044          }

```

```

1045         const pops = Math.min(UNDO_STEPS, this._history.length - 1);
1046         for (let i = 0; i < pops; i++) {
1047             this._history.pop();
1048         }
1049         const snap = this._history[this._history.length - 1];
1050         this.core.restorePlayState(snap);
1051         this._moveCount = Math.max(0, this._moveCount - pops);
1052         this._emit('undo');
1053         return true;
1054     }
1055     resetLevel(): void {
1056         if (!this._level) {
1057             return;
1058         }
1059         if (this.isAtInitialPlayState()) {
1060             this._emitInitialStateToast();
1061             return;
1062         }
1063         this.core.reset(this._level);
1064         this._history.length = 0;
1065         this._resetUndoFillStage();
1066         this._moveCount = 0;
1067         this._pushHistory();
1068         this._flow = OpticalGameFlowState.RUNNING;
1069         this._emit('reset');
1070     }
1071     private _pushHistory(): void {
1072         this._history.push(this.core.clonePlayState());
1073     }
1074     private _emit(reason: OpticalSessionNotifyReason): void {
1075         this.onStateChanged?.(reason);
1076     }
1077 }
1078 // ----- assets/Scripts/Games/OpticalPuzzle/Application/OpticalPuzzleStateMachine.ts -----
1079 /**
1080  * 与《小游戏设计文档》及 project-rules 统一的最小状态枚举。
1081  * 具体流转由 OpticalPuzzleSession 驱动。
1082  */
1083 export enum OpticalGameFlowState {
1084     BOOT = 'BOOT',
1085     READY = 'READY',
1086     RUNNING = 'RUNNING',
1087     PAUSED = 'PAUSED',
1088     SETTLEMENT = 'SETTLEMENT',
1089     FINISHED = 'FINISHED',
1090 }
1091 // ----- assets/Scripts/Games/OpticalPuzzle/Config/OpticalPuzzleLevels.ts -----
1092 import type { IOpticalLevelConfig, IOpticalLevelLayeredSource } from './OpticalPuzzleLevelSchema';
1093 import { parseLayeredGridsToLevelConfig } from './OpticalPuzzleLevelSchema';
1094 /**
1095  * 关卡源数据：每关须完整配置四层同尺寸网格（`staticLayout` / `objects` / `colors` / `directions`）。
1096  * 含义见 `OpticalPuzzleLevelSchema.ts`；层 3/4 仅在光源、目标、元件处标注，其余为 `.`。
1097  * 元件：层 2 写数字 `0`~`4`（通道类型）；层 4 `w/a/s/d` 旋转朝向。
1098  */
1099 export const OPTICAL_LEVEL_LAYERED_SOURCES: IOpticalLevelLayeredSource[] = [
1100     /*{
1101         levelId: 1,
1102         levelName: '点亮目标',

```

```

1103     perfectSteps: 2,
1104     threeStarSteps: 4,
1105     twoStarSteps: 10,
1106     oneStarSteps: 20,
1107     height: 5,
1108     width: 9,
1109     staticLayout: [
1110         '#####',
1111         '#..#..#',
1112         '#.....#',
1113         '#..#..#',
1114         '#####',
1115     ],
1116     objects: [
1117         '#####',
1118         '#..#..#',
1119         '#S..O..E#',
1120         '#..#@#..#',
1121         '#####',
1122     ],
1123     colors: [
1124         '#####',
1125         '#..#..#',
1126         '#W.W.W.W#',
1127         '#..#..#',
1128         '#####',
1129     ],
1130     directions: [
1131         '#####',
1132         '#..#..#',
1133         '#d..w...#',
1134         '#..#..#',
1135         '#####',
1136     ],
1137 },*/
1138 {
1139     levelId: 1,//第 1 关
1140     levelName: '关卡 1（难度 19）',
1141     perfectSteps: 1,
1142     threeStarSteps: 1,
1143     twoStarSteps: 3,
1144     oneStarSteps: 5,
1145     bestSolution: ['w'],
1146     height: 7,
1147     width: 9,
1148     staticLayout: [
1149         '#####',
1150         '#.###..#',
1151         '#.....#',
1152         '#.....#',
1153         '#.....#',
1154         '###.###.##',
1155         '#####',
1156     ],
1157     objects: [
1158         '#####',
1159         '#.S###E.#',
1160         '#.....#',

```

```

1161         '#.1.....#',
1162         '#.....1.#',
1163         '##.###@##',
1164         '#####',
1165     ],
1166     colors: [
1167         '#####',
1168         '#.W###W.#',
1169         '#.....#',
1170         '#.W....#',
1171         '#....W.#',
1172         '##.###.##',
1173         '#####',
1174     ],
1175     directions: [
1176         '#####',
1177         '#.s###..#',
1178         '#.....#',
1179         '#.w....#',
1180         '#.....a.#',
1181         '##.###.##',
1182         '#####',
1183     ],
1184 },
1185 {
1186     levelId: 2,//第 2 关
1187     levelName: '关卡 2（难度 18）',
1188     perfectSteps: 1,
1189     threeStarSteps: 1,
1190     twoStarSteps: 3,
1191     oneStarSteps: 5,
1192     bestSolution: ['a'],
1193     height: 6,
1194     width: 9,
1195     staticLayout: [
1196         '#####',
1197         '#.....#',
1198         '##.....##',
1199         '##.....##',
1200         '#.....#',
1201         '#####',
1202     ],
1203     objects: [
1204         '#####',
1205         '#E....1@#',
1206         '##.....##',
1207         '##.....##',
1208         '#S...1..#',
1209         '#####',
1210     ],
1211     colors: [
1212         '#####',
1213         '#W...W.#',
1214         '##.....##',
1215         '##.....##',
1216         '#W...W..#',
1217         '#####',
1218     ],

```

```

1219         directions: [
1220             '#####',
1221             '#....s.#',
1222             '##....##',
1223             '##....##',
1224             '#d...a..#',
1225             '#####',
1226         ],
1227     },
1228     {
1229         levelId: 3,//第 3 关
1230         levelName: '关卡 3（难度 23）',
1231         perfectSteps: 7,
1232         threeStarSteps: 9,
1233         twoStarSteps: 15,
1234         oneStarSteps: 30,
1235         bestSolution: ['s', 'd', 'd', 'd', 's', 's', 'a'],
1236         height: 6,
1237         width: 9,
1238         staticLayout: [
1239             '#####',
1240             '#.....#',
1241             '#.....#',
1242             '#.....#',
1243             '#.....#',
1244             '#####',
1245         ],
1246         objects: [
1247             '#####',
1248             '#..S@E..#',
1249             '#.....#',
1250             '#.....#',
1251             '#..1..1.#',
1252             '#####',
1253         ],
1254         colors: [
1255             '#####',
1256             '#..W.W..#',
1257             '#.....#',
1258             '#.....#',
1259             '#..W.W.#',
1260             '#####',
1261         ],
1262         directions: [
1263             '#####',
1264             '#..s....#',
1265             '#.....#',
1266             '#.....#',
1267             '#..w..a.#',
1268             '#####',
1269         ],
1270     },
1271     {
1272         levelId: 4,//第 4 关
1273         levelName: '关卡 4（难度 27）',
1274         perfectSteps: 6,
1275         threeStarSteps: 7,
1276         twoStarSteps: 9,

```

```

1277     oneStarSteps: 15,
1278     bestSolution: ['d', 'd', 'w', 'd', 'w', 'a'],
1279     height: 6,
1280     width: 9,
1281     staticLayout: [
1282         '#####',
1283         '#.....#',
1284         '#.....#',
1285         '#.....#',
1286         '#...#...#',
1287         '#####',
1288     ],
1289     objects: [
1290         '#####',
1291         '#..1....#',
1292         '#.....1.#',
1293         '#...@...#',
1294         '#S.1#1.E#',
1295         '#####',
1296     ],
1297     colors: [
1298         '#####',
1299         '#.W...#',
1300         '#....W.#',
1301         '#.....#',
1302         '#W.W#W.W#',
1303         '#####',
1304     ],
1305     directions: [
1306         '#####',
1307         '#.d....#',
1308         '#.....s.#',
1309         '#.....#',
1310         '#d.a#w..#',
1311         '#####',
1312     ],
1313 },
1314 {
1315     levelId: 5, //第 5 关
1316     levelName: '关卡 5（难度 25）',
1317     perfectSteps: 13,
1318     threeStarSteps: 15,
1319     twoStarSteps: 20,
1320     oneStarSteps: 30,
1321     bestSolution: [
1322         'd', 'd', 'w', 'd', 's', 'd', 's', 'a', 'a', 'a', 'a', 'a', 'w',
1323     ],
1324     height: 6,
1325     width: 10,
1326     staticLayout: [
1327         '#####',
1328         '#.....#',
1329         '#...#...#',
1330         '#.....#',
1331         '#.....#',
1332         '#####',
1333     ],
1334     objects: [

```

```

1335         '#####',
1336         '#S.....#',
1337         '#...#...#',
1338         '#..@.1...#',
1339         '#.....E#',
1340         '#####',
1341     ],
1342     colors: [
1343         '#####',
1344         '#W.....#',
1345         '#...#...#',
1346         '#...W...#',
1347         '#.....W#',
1348         '#####',
1349     ],
1350     directions: [
1351         '#####',
1352         '#s.....#',
1353         '#...#...#',
1354         '#...w...#',
1355         '#.....#',
1356         '#####',
1357     ],
1358 },
1359 {
1360     levelId: 6,//第 6 关
1361     levelName: '关卡 6（难度 31）',
1362     perfectSteps: 20,
1363     threeStarSteps: 23,
1364     twoStarSteps: 30,
1365     oneStarSteps: 40,
1366     bestSolution: [
1367         's', 's', 'd', 's', 's', 'a', 'a', 'w', 'd', 's', 'd', 'w', 'a', 'w', 'd', 'd', 'd', 'a', 'a', 's',
1368     ],
1369     height: 7,
1370     width: 10,
1371     staticLayout: [
1372         '#####',
1373         '#.....#',
1374         '#.###...#',
1375         '#.....#',
1376         '#...##...#',
1377         '#....#..#',
1378         '#####',
1379     ],
1380     objects: [
1381         '#####',
1382         '#.@...E..#',
1383         '#.1###...#',
1384         '#S.....#',
1385         '#...##...#',
1386         '#....#..#',
1387         '#####',
1388     ],
1389     colors: [
1390         '#####',
1391         '#....W..#',
1392         '#.W###...#',

```



```

1393         '#W.....#',
1394         '#...##...#',
1395         '#.....#.#',
1396         '#####',
1397     ],
1398     directions: [
1399         '#####',
1400         '#.....#',
1401         '#.a###...#',
1402         '#d.....#',
1403         '#...##...#',
1404         '#.....#.#',
1405         '#####',
1406     ],
1407 },
1408 {
1409     levelId: 7,//第 7 关
1410     levelName: '关卡 7（难度 33）',
1411     perfectSteps: 18,
1412     threeStarSteps: 20,
1413     twoStarSteps: 30,
1414     oneStarSteps: 50,
1415     bestSolution: [
1416         'd','s','a','w','a','s','s','s','s','s','d','s','a','d','d','d','d','w',
1417     ],
1418     height: 9,
1419     width: 9,
1420     staticLayout: [
1421         '#####',
1422         '#.....#',
1423         '#.....#',
1424         '#...#...#',
1425         '#.###..#',
1426         '#...#...#',
1427         '#.....#',
1428         '#.....#',
1429         '#####',
1430     ],
1431     objects: [
1432         '#####',
1433         '#S.@...E#',
1434         '#.1....#',
1435         '#...#...#',
1436         '#.###..#',
1437         '#...#...#',
1438         '#.....#',
1439         '#.....1.#',
1440         '#####',
1441     ],
1442     colors: [
1443         '#####',
1444         '#W....W#',
1445         '#.W...#',
1446         '#...#...#',
1447         '#.###..#',
1448         '#...#...#',
1449         '#.....#',
1450         '#.....W#',

```

```

1451         '#####',
1452     ],
1453     directions: [
1454         '#####',
1455         '#s.....#',
1456         '#.w....#',
1457         '#...#...#',
1458         '#.###..#',
1459         '#...#...#',
1460         '#.....#',
1461         '#....a.#',
1462         '#####',
1463     ],
1464 },
1465 {
1466     levelId: 8,//第 8 关
1467     levelName: '关卡 8（难度 36）',
1468     perfectSteps: 19,
1469     threeStarSteps: 21,
1470     twoStarSteps: 30,
1471     oneStarSteps: 50,
1472     bestSolution: [
1473         's', 's', 'd', 'd', 'd', 'd', 'w', 'w', 'a', 'a', 'a', 's', 'd', 's', 'a', 'a', 'a', 'a', 'w',
1474     ],
1475     height: 6,
1476     width: 11,
1477     staticLayout: [
1478         '#####',
1479         '#.....#',
1480         '#...#...#',
1481         '#.....#',
1482         '#...#...#',
1483         '#####',
1484     ],
1485     objects: [
1486         '#####',
1487         '#S..@....E#',
1488         '#..#1#..#',
1489         '#...1...#',
1490         '#...#...#',
1491         '#####',
1492     ],
1493     colors: [
1494         '#####',
1495         '#W.....R#',
1496         '#..#W#..#',
1497         '#...R...#',
1498         '#...#...#',
1499         '#####',
1500     ],
1501     directions: [
1502         '#####',
1503         '#s.....#',
1504         '#..#w#..#',
1505         '#...a...#',
1506         '#...#...#',
1507         '#####',
1508     ],

```

```

1509     },
1510     {
1511         levelId: 9,//第 9 关
1512         levelName: '关卡 9（难度 36）',
1513         perfectSteps: 11,
1514         threeStarSteps: 15,
1515         twoStarSteps: 24,
1516         oneStarSteps: 36,
1517         bestSolution: [
1518             'w', 'a', 'a', 'w', 'a', 's', 's', 'd', 's', 'a', 'w',
1519         ],
1520         height: 7,
1521         width: 9,
1522         staticLayout: [
1523             '#####',
1524             '#.....#',
1525             '#...#...#',
1526             '#.....#',
1527             '#...#...#',
1528             '#.....#',
1529             '#####',
1530         ],
1531         objects: [
1532             '#####',
1533             '#1.....E#',
1534             '#...#...#',
1535             '#3..1..S#',
1536             '#...#@..#',
1537             '#.....E#',
1538             '#####',
1539         ],
1540         colors: [
1541             '#####',
1542             '#R.....R#',
1543             '#...#...#',
1544             '#W.W.W.W#',
1545             '#...#...#',
1546             '#.....W#',
1547             '#####',
1548         ],
1549         directions: [
1550             '#####',
1551             '#d.....#',
1552             '#...#...#',
1553             '#d..w..a#',
1554             '#...#...#',
1555             '#.....#',
1556             '#####',
1557         ],
1558     },
1559     {
1560         levelId: 10,//第 10 关
1561         levelName: '关卡 10（难度 37）',
1562         perfectSteps: 18,
1563         threeStarSteps: 24,
1564         twoStarSteps: 30,
1565         oneStarSteps: 50,
1566         bestSolution: [

```

```

1567         'a', 'w', 'w', 's', 'a', 'a', 'a', 'w', 'a', 's', 'd', 'd', 'w', 'w', 'd', 'd', 'd', 's',
1568     ],
1569     height: 6,
1570     width: 9,
1571     staticLayout: [
1572         '#####',
1573         '#.....#',
1574         '#...#...#',
1575         '#.....#',
1576         '#.....#',
1577         '#####',
1578     ],
1579     objects: [
1580         '#####',
1581         '#1.....#',
1582         '#...#...#',
1583         '#.1.1..#',
1584         '#...E.@S#',
1585         '#####',
1586     ],
1587     colors: [
1588         '#####',
1589         '#R.....#',
1590         '#...#...#',
1591         '#.WW..#',
1592         '#...R.W#',
1593         '#####',
1594     ],
1595     directions: [
1596         '#####',
1597         '#d.....#',
1598         '#...#...#',
1599         '#.w.s..#',
1600         '#.....w#',
1601         '#####',
1602     ],
1603 },
1604 {
1605     levelId: 11,//第 11 关
1606     levelName: '关卡 11（难度 55）',
1607     perfectSteps: 38,
1608     threeStarSteps: 44,
1609     twoStarSteps: 50,
1610     oneStarSteps: 70,
1611     bestSolution: [
1612         'd', 's', 'd', 'w', 'd', 's', 's', 's', 's', 'd', 's', 'a', 'a', 'a', 'a', 'a', 'w', 'a', 's',
1613         'd', 'w', 'w', 'w', 'w', 'd', 'd', 'd', 'w', 'd', 's', 's', 's', 's', 's', 'a', 's', 'd', 'w',
1614     ],
1615     height: 9,
1616     width: 9,
1617     staticLayout: [
1618         '#####',
1619         '#.....#',
1620         '#.....#',
1621         '#...#...#',
1622         '#.###..#',
1623         '#...#...#',
1624         '#.....#',

```

```

1625         '#.....#',
1626         '#####',
1627     ],
1628     objects: [
1629         '#####',
1630         '#S.@...E#',
1631         '#.1.1..#',
1632         '#...#...#',
1633         '#.###..#',
1634         '#...#...#',
1635         '#.....#',
1636         '#...2...#',
1637         '#####',
1638     ],
1639     colors: [
1640         '#####',
1641         '#W....R#',
1642         '#.WW..#',
1643         '#...#...#',
1644         '#.###..#',
1645         '#...#...#',
1646         '#.....#',
1647         '#...R...#',
1648         '#####',
1649     ],
1650     directions: [
1651         '#####',
1652         '#s.....#',
1653         '#.a.w..#',
1654         '#...#...#',
1655         '#.###..#',
1656         '#...#...#',
1657         '#.....#',
1658         '#...d...#',
1659         '#####',
1660     ],
1661 },
1662 {
1663     levelId: 12,//第 12 关
1664     levelName: '关卡 12（难度 52）',
1665     perfectSteps: 32,
1666     threeStarSteps: 40,
1667     twoStarSteps: 48,
1668     oneStarSteps: 60,
1669     bestSolution: [
1670         'w', 'w', 'w', 's', 'd', 'd', 'w', 'w', 'a', 'a', 'a', 'a', 'd', 's', 's', 's', 'a', 'a',
1671         'w', 'd', 'd', 'd', 'w', 's', 'd', 's', 'd', 'w', 'w', 's', 'a',
1672     ],
1673     height: 7,
1674     width: 8,
1675     staticLayout: [
1676         '#####',
1677         '#.....#',
1678         '#.#.#.#',
1679         '#.....#',
1680         '#.....#',
1681         '#.#.#.#',
1682         '#####',

```

```

1683     ],
1684     objects: [
1685         '#####',
1686         '#.....#',
1687         '##.3#.#',
1688         '#.11...#',
1689         '#.....#',
1690         '#E#@S#E#',
1691         '#####',
1692     ],
1693     colors: [
1694         '#####',
1695         '#.....#',
1696         '##.W#.#',
1697         '##.RW...#',
1698         '#.....#',
1699         '#W#.W#R#',
1700         '#####',
1701     ],
1702     directions: [
1703         '#####',
1704         '#.....#',
1705         '##.s#.#',
1706         '##.sd...#',
1707         '#.....#',
1708         '##.w#.#',
1709         '#####',
1710     ],
1711 },
1712 {
1713     levelId: 13,//第 13 关
1714     levelName: '关卡 13（难度 54）',
1715     perfectSteps: 11,
1716     threeStarSteps: 12,
1717     twoStarSteps: 16,
1718     oneStarSteps: 30,
1719     bestSolution: [
1720         'w', 'w', 'a', 's', 'a', 'w', 'd', 'w', 'd', 'd', 'w',
1721     ],
1722     height: 7,
1723     width: 9,
1724     staticLayout: [
1725         '#####',
1726         '#.....#',
1727         '#.....#',
1728         '#.....#',
1729         '#.....#',
1730         '#.....#',
1731         '#####',
1732     ],
1733     objects: [
1734         '#####',
1735         '#.E...E#',
1736         '#.....#',
1737         '##.33...#',
1738         '#.....#',
1739         '#.S.@.E#',
1740         '#####',

```

```

1741     ],
1742     colors: [
1743         '#####',
1744         '#.R...P.#',
1745         '#.....#',
1746         '#..RB...#',
1747         '#.....#',
1748         '#.W...P.#',
1749         '#####',
1750     ],
1751     directions: [
1752         '#####',
1753         '#.....#',
1754         '#.....#',
1755         '#..da...#',
1756         '#.....#',
1757         '#.w....#',
1758         '#####',
1759     ],
1760 },
1761 {
1762     levelId: 14,//第 14 关
1763     levelName: '关卡 14（难度 37）',
1764     perfectSteps: 9,
1765     threeStarSteps: 14,
1766     twoStarSteps: 20,
1767     oneStarSteps: 30,
1768     bestSolution: [
1769         'd','w','a','w','a','s','d','s','a',
1770     ],
1771     height: 6,
1772     width: 7,
1773     staticLayout: [
1774         '#####',
1775         '###.##',
1776         '#.....',
1777         '#.....',
1778         '#.....',
1779         '#####',
1780     ],
1781     objects: [
1782         '#####',
1783         '###.##',
1784         '#..1.S#',
1785         '#...1.#',
1786         '#E..@.#',
1787         '#####',
1788     ],
1789     colors: [
1790         '#####',
1791         '###.##',
1792         '#..B..#',
1793         '#...R.#',
1794         '#P...#',
1795         '#####',
1796     ],
1797     directions: [
1798         '#####',

```

```

1799         '###.##',
1800         '#.d.a#',
1801         '#...a.#',
1802         '#.....#',
1803         '#####',
1804     ],
1805 },
1806 {
1807     levelId: 15,//第 15 关
1808     levelName: '关卡 15（难度 49）',
1809     perfectSteps: 15,
1810     threeStarSteps: 19,
1811     twoStarSteps: 23,
1812     oneStarSteps: 30,
1813     bestSolution: [
1814         's','a','a','w','d','w','d','d','w','d','s','a','s','a','w',
1815     ],
1816     height: 7,
1817     width: 9,
1818     staticLayout: [
1819         '#####',
1820         '#.....#',
1821         '#.....#',
1822         '#.....#',
1823         '#.....#',
1824         '#.....#',
1825         '#####',
1826     ],
1827     objects: [
1828         '#####',
1829         '#.....E#',
1830         '#.1.2...#',
1831         '#.3@...S#',
1832         '#.....#',
1833         '#.E.....#',
1834         '#####',
1835     ],
1836     colors: [
1837         '#####',
1838         '#.....P#',
1839         '#.B.R...#',
1840         '#.W...W#',
1841         '#.....#',
1842         '#.R.....#',
1843         '#####',
1844     ],
1845     directions: [
1846         '#####',
1847         '#.....#',
1848         '#.d.d...#',
1849         '#.d....a#',
1850         '#.....#',
1851         '#.....#',
1852         '#####',
1853     ],
1854 },
1855 {
1856     levelId: 16,//第 16 关

```



```

1857     levelName: '关卡 16（难度 64）',
1858     perfectSteps: 43,
1859     threeStarSteps: 46,
1860     twoStarSteps: 60,
1861     oneStarSteps: 80,
1862     bestSolution: [
1863         'w', 'w', 'd', 's', 's', 'a', 'a', 's', 'a', 'a', 'w', 'd', 'd', 's', 'd', 'd', 'w', 'w', 'w',
1864         'a', 'a', 's', 'd', 's', 'a', 's', 'a', 'a', 'w', 'd', 'd', 'w', 'd', 'd', 's', 's', 'a', 'a',
1865         'w', 'w', 'd', 's', 'd',
1866     ],
1867     height: 7,
1868     width: 8,
1869     staticLayout: [
1870         '#####',
1871         '###...##',
1872         '###...##',
1873         '##....#',
1874         '#.....#',
1875         '#.....#',
1876         '#####',
1877     ],
1878     objects: [
1879         '#####',
1880         '###.S.##',
1881         '###...##',
1882         '##E3.1E#',
1883         '#.1.@..#',
1884         '#.....#',
1885         '#####',
1886     ],
1887     colors: [
1888         '#####',
1889         '###.W.##',
1890         '###...##',
1891         '##PR.BP#',
1892         '#.B....#',
1893         '#.....#',
1894         '#####',
1895     ],
1896     directions: [
1897         '#####',
1898         '###.s.##',
1899         '###...##',
1900         '##.w.a.#',
1901         '#.w....#',
1902         '#.....#',
1903         '#####',
1904     ],
1905 },
1906 {
1907     levelId: 17, //第 17 关
1908     levelName: '关卡 17（难度 48）',
1909     perfectSteps: 19,
1910     threeStarSteps: 29,
1911     twoStarSteps: 40,
1912     oneStarSteps: 56,
1913     bestSolution: [
1914         's', 'a', 'w', 'a', 'a', 's', 's', 'd', 'w', 'a', 'w', 'd', 'a', 's', 'a', 'a', 'w', 'd', 's',

```

```

1915     ],
1916     height: 6,
1917     width: 9,
1918     staticLayout: [
1919         '#####',
1920         '#.....#',
1921         '#.....#',
1922         '#.....#',
1923         '###...###',
1924         '#####',
1925     ],
1926     objects: [
1927         '#####',
1928         '#E...3.@#',
1929         '#...1.1.#',
1930         '#....E.#',
1931         '###..S###',
1932         '#####',
1933     ],
1934     colors: [
1935         '#####',
1936         '#R...W..#',
1937         '#...W.B.#',
1938         '#....P.#',
1939         '###..R###',
1940         '#####',
1941     ],
1942     directions: [
1943         '#####',
1944         '#....s..#',
1945         '#...w.s.#',
1946         '#.....#',
1947         '###..a###',
1948         '#####',
1949     ],
1950 },
1951 {
1952     levelId: 18,//第 18 关
1953     levelName: '关卡 18（难度 71）',
1954     perfectSteps: 44,
1955     threeStarSteps: 48,
1956     twoStarSteps: 60,
1957     oneStarSteps: 90,
1958     bestSolution: [
1959         'w', 'a', 'w', 'a', 'a', 'a', 'a', 'w', 'd', 's', 'd', 'd', 'd', 's', 's', 'a', 'w', 'd', 'w', 'a',
1960         'w', 'w', 'd', 's', 's', 'a', 'a', 'a', 'w', 'd', 's', 'd', 'd', 'w', 'w', 'a', 's', 'd', 's', 'a',
1961         's', 's', 'd', 'w',
1962     ],
1963     height: 7,
1964     width: 8,
1965     staticLayout: [
1966         '#####',
1967         '#.#...##',
1968         '#.....##',
1969         '#.....#',
1970         '####...#',
1971         '####...#',
1972         '#####',

```

```

1973     ],
1974     objects: [
1975         '#####',
1976         '#S#E..##',
1977         '#.3...##',
1978         '#....2E#',
1979         '####.1.#',
1980         '####.@#',
1981         '#####',
1982     ],
1983     colors: [
1984         '#####',
1985         '#W#R..##',
1986         '#.R...##',
1987         '#....BP#',
1988         '####.W.#',
1989         '####...#',
1990         '#####',
1991     ],
1992     directions: [
1993         '#####',
1994         '#s#...##',
1995         '#.w...##',
1996         '#....d.#',
1997         '####.w.#',
1998         '####...#',
1999         '#####',
2000     ],
2001 },
2002 {
2003     levelId: 19,//第 19 关
2004     levelName: '关卡 19（难度 58）',
2005     perfectSteps: 15,
2006     threeStarSteps: 19,
2007     twoStarSteps: 30,
2008     oneStarSteps: 40,
2009     bestSolution: [
2010         'w', 'd', 'w', 'w', 'w', 'd', 'd', 's', 's', 'a', 's', 'd', 's', 'd', 'w',
2011     ],
2012     height: 7,
2013     width: 8,
2014     staticLayout: [
2015         '#####',
2016         '##....##',
2017         '##.#.##',
2018         '#.....##',
2019         '#.....#',
2020         '#.....#',
2021         '#####',
2022     ],
2023     objects: [
2024         '#####',
2025         '##...E##',
2026         '##.#1.##',
2027         '#S.3..##',
2028         '#.....#',
2029         '#@1...E#',
2030         '#####',

```

```

2031     ],
2032     colors: [
2033         '#####',
2034         '##...P##',
2035         '##.B.##',
2036         '#W.R..##',
2037         '#.....#',
2038         '#.G...Y#',
2039         '#####',
2040     ],
2041     directions: [
2042         '#####',
2043         '##....##',
2044         '##.#a.##',
2045         '#d.s..##',
2046         '#.....#',
2047         '#.w....#',
2048         '#####',
2049     ],
2050 },
2051 {
2052     levelId: 20,//第 20 关
2053     levelName: '关卡 20（难度 64）',
2054     perfectSteps: 11,
2055     threeStarSteps: 17,
2056     twoStarSteps: 30,
2057     oneStarSteps: 45,
2058     bestSolution: [
2059         's', 'a', 's', 'd', 'a', 'a', 'a', 'w', 'd', 'd', 's',
2060     ],
2061     height: 7,
2062     width: 9,
2063     staticLayout: [
2064         '#####',
2065         '#####..##',
2066         '#####...#',
2067         '#.....#',
2068         '##....##',
2069         '##..#..##',
2070         '#####',
2071     ],
2072     objects: [
2073         '#####',
2074         '#####SS##',
2075         '#####@..#',
2076         '#E..32..#',
2077         '##....##',
2078         '##..EE##',
2079         '#####',
2080     ],
2081     colors: [
2082         '#####',
2083         '#####RR##',
2084         '#####..#',
2085         '#Y..GB..#',
2086         '##....##',
2087         '##..YP##',
2088         '#####',

```

```

2089     ],
2090     directions: [
2091         '#####',
2092         '#####ss##',
2093         '#####...#',
2094         '#...aw..#',
2095         '##.....##',
2096         '##..#..##',
2097         '#####',
2098     ],
2099 },
2100 {
2101     levelId: 21,//第 21 关
2102     levelName: '关卡 21（难度 60）',
2103     perfectSteps: 34,
2104     threeStarSteps: 43,
2105     twoStarSteps: 55,
2106     oneStarSteps: 70,
2107     bestSolution: [
2108         'w','w','a','w','a','a','a','s','s','d','d','w','d','d','s','s','a','w','d',
2109         'w','a','w','a','a','d','d','s','s','a','w','d','w','a','s',
2110     ],
2111     height: 6,
2112     width: 7,
2113     staticLayout: [
2114         '#####',
2115         '#.....#',
2116         '#.....#',
2117         '#.....#',
2118         '###..#',
2119         '#####',
2120     ],
2121     objects: [
2122         '#####',
2123         '#...S#',
2124         '#.E1..#',
2125         '#..3..#',
2126         '#E##.@#',
2127         '#####',
2128     ],
2129     colors: [
2130         '#####',
2131         '#...W#',
2132         '#.RB..#',
2133         '#..R..#',
2134         '#P##..#',
2135         '#####',
2136     ],
2137     directions: [
2138         '#####',
2139         '#...a#',
2140         '#..d..#',
2141         '#..s..#',
2142         '###..#',
2143         '#####',
2144     ],
2145 },
2146 {

```

```

2147     levelId: 22,//第 22 关
2148     levelName: '关卡 22（难度 80）',
2149     perfectSteps: 31,
2150     threeStarSteps: 39,
2151     twoStarSteps: 50,
2152     oneStarSteps: 65,
2153     bestSolution: [
2154         's', 'a', 's', 'a', 'a', 'w', 'd', 'd', 'd', 'w', 'd', 'd', 's', 'a', 's', 'a', 'a', 'w', 'd',
2155         'a', 'a', 'a', 's', 's', 'd', 'w', 'a', 'w', 'd', 'd', 's',
2156     ],
2157     height: 7,
2158     width: 9,
2159     staticLayout: [
2160         '#####',
2161         '#####',
2162         '#####',
2163         '#.....#',
2164         '##.....#',
2165         '##..#..#',
2166         '#####',
2167     ],
2168     objects: [
2169         '#####',
2170         '#####SS##',
2171         '#####@..#',
2172         '#E..23..#',
2173         '##.....#',
2174         '##..#EE##',
2175         '#####',
2176     ],
2177     colors: [
2178         '#####',
2179         '#####RR##',
2180         '#####',
2181         '#Y..BG..#',
2182         '##.....#',
2183         '##..#YP##',
2184         '#####',
2185     ],
2186     directions: [
2187         '#####',
2188         '#####ss##',
2189         '#####',
2190         '#...wa..#',
2191         '##.....#',
2192         '##..#..#',
2193         '#####',
2194     ],
2195 },
2196 {
2197     levelId: 23,//第 23 关
2198     levelName: '关卡 23（难度 70）',
2199     perfectSteps: 24,
2200     threeStarSteps: 30,
2201     twoStarSteps: 36,
2202     oneStarSteps: 48,
2203     bestSolution: [
2204         'd', 's', 'a', 'a', 'a', 's', 'a', 'a', 'w', 'd', 'a', 'w', 'a', 'a', 's', 'd', 'd', 'a', 'a', 's', 'd', 'd', 'd', 'a',

```

```

2205     ],
2206     height: 6,
2207     width: 12,
2208     staticLayout: [
2209         '#####',
2210         '#.....#',
2211         '#.....#',
2212         '#.....#',
2213         '#####.####',
2214         '#####',
2215     ],
2216     objects: [
2217         '#####',
2218         '#....SS.@.#',
2219         '#...2...2.#',
2220         '#...2...2..#',
2221         '#####EE####',
2222         '#####',
2223     ],
2224     colors: [
2225         '#####',
2226         '#....WW...#',
2227         '#...G....B.#',
2228         '#...R...R..#',
2229         '#####GP####',
2230         '#####',
2231     ],
2232     directions: [
2233         '#####',
2234         '#....ss...#',
2235         '#...w...w.#',
2236         '#...w...w..#',
2237         '#####.####',
2238         '#####',
2239     ],
2240 },
2241 {
2242     levelId: 24,//第 24 关
2243     levelName: '关卡 24（难度 108）',
2244     perfectSteps: 53,
2245     threeStarSteps: 67,
2246     twoStarSteps: 80,
2247     oneStarSteps: 100,
2248     bestSolution: [
2249         's', 'a', 's', 'a', 'a', 'w', 'd', 'd', 'd', 'w', 'd', 'd', 's', 'a', 's', 'a', 'a', 'd', 'd', 'w',
2250         'w', 'a', 's', 'd', 's', 'a', 'w', 'a', 'a', 'a', 's', 's', 'd', 'w', 'a', 'w', 'd', 'd', 'd', 's',
2251         'a', 'w', 'a', 'a', 's', 's', 'd', 'w', 'a', 'w', 'd', 'd', 's',
2252     ],
2253     height: 7,
2254     width: 10,
2255     staticLayout: [
2256         '#####',
2257         '#####.####',
2258         '#####...##',
2259         '#.....#',
2260         '##.....##',
2261         '##..#...##',
2262         '#####',

```

```

2263     ],
2264     objects: [
2265         '#####',
2266         '#####SS###',
2267         '#####@..##',
2268         '#E..33..E#',
2269         '##.....###',
2270         '##..#EE###',
2271         '#####',
2272     ],
2273     colors: [
2274         '#####',
2275         '#####RR###',
2276         '#####...##',
2277         '#Y..GB..P#',
2278         '##.....###',
2279         '##..#YP###',
2280         '#####',
2281     ],
2282     directions: [
2283         '#####',
2284         '#####ss###',
2285         '#####...##',
2286         '#...ad...#',
2287         '##.....###',
2288         '##..#...###',
2289         '#####',
2290     ],
2291 },
2292 {
2293     levelId: 25, //第 25 关
2294     levelName: '关卡 25 (难度 78) ',
2295     perfectSteps: 58,
2296     threeStarSteps: 73,
2297     twoStarSteps: 87,
2298     oneStarSteps: 116,
2299     bestSolution: [
2300         'w', 'w', 'a', 'w', 'a', 'a', 'a', 's', 's', 'd', 'd', 'a', 'a', 'w', 'w', 'd', 'd', 's', 'd', 'd',
2301         's', 's', 'a', 'w', 'd', 'w', 'a', 'w', 'a', 'a', 'a', 's', 's', 'd', 'd', 'w', 'd', 'd', 's', 's',
2302         'a', 'w', 'd', 'w', 'a', 'w', 'a', 'a', 'd', 'd', 's', 's', 'a', 'w', 'd', 'w', 'a', 's',
2303     ],
2304     height: 6,
2305     width: 7,
2306     staticLayout: [
2307         '#####',
2308         '#.....#',
2309         '#.....#',
2310         '#.....#',
2311         '###..#',
2312         '#####',
2313     ],
2314     objects: [
2315         '#####',
2316         '#...S#',
2317         '#.E3..#',
2318         '#..1..#',
2319         '#E##.@#',
2320         '#####',

```



```

2321     ],
2322     colors: [
2323         '#####',
2324         '#...W#',
2325         '#.RR..#',
2326         '#..B..#',
2327         '#P##..#',
2328         '#####',
2329     ],
2330     directions: [
2331         '#####',
2332         '#...a#',
2333         '#..s..#',
2334         '#..d..#',
2335         '#.##..#',
2336         '#####',
2337     ],
2338 },
2339 {
2340     levelId: 26,//第 26 关
2341     levelName: '关卡 26（难度 73）',
2342     perfectSteps: 54,
2343     threeStarSteps: 68,
2344     twoStarSteps: 81,
2345     oneStarSteps: 108,
2346     bestSolution: [
2347         'a','a','a','a','s','a','a','w','d','d','w','w','a','a','a','a','s','s','d','d',
2348         'd','a','a','a','w','w','d','d','d','d','s','s','d','s','d','d','w','a','a','s',
2349         'a','w','a','a','a','a','w','w','d','d','d','d','s','s',
2350     ],
2351     height: 6,
2352     width: 11,
2353     staticLayout: [
2354         '#####',
2355         '#....#...#',
2356         '#.###.#####',
2357         '#.....#',
2358         '##.....#',
2359         '#####',
2360     ],
2361     objects: [
2362         '#####',
2363         '#....#...#',
2364         '#.###.#####',
2365         '#.2.2....@#',
2366         '##S.....E#',
2367         '#####',
2368     ],
2369     colors: [
2370         '#####',
2371         '#....#...#',
2372         '#.###.#####',
2373         '#.G.B....#',
2374         '##R.....W#',
2375         '#####',
2376     ],
2377     directions: [
2378         '#####',

```

```

2379         '#.....#..#',
2380         '#####',
2381         '#.d.d....#',
2382         '##d.....#',
2383         '#####',
2384     ],
2385 },
2386 {
2387     levelId: 27,//第 27 关
2388     levelName: '关卡 27（难度 77）',
2389     perfectSteps: 32,
2390     threeStarSteps: 40,
2391     twoStarSteps: 48,
2392     oneStarSteps: 64,
2393     bestSolution: [
2394         'w', 'a', 'w', 'd', 'w', 'd', 'd', 'w', 'd', 'd', 's', 'a', 'a', 's', 'a', 'a', 's', 's', 'd', 'd',
2395         'w', 'd', 'w', 'a', 's', 's', 'a', 'a', 'w', 'w', 'd', 'd',
2396     ],
2397     height: 7,
2398     width: 9,
2399     staticLayout: [
2400         '#####',
2401         '#.....#',
2402         '#.....#',
2403         '#.....#',
2404         '##..#..##',
2405         '#.....#',
2406         '#####',
2407     ],
2408     objects: [
2409         '#####',
2410         '#...S...#',
2411         '#...3...#',
2412         '#.1...1.#',
2413         '##.2#2.##',
2414         '#.E@..E.#',
2415         '#####',
2416     ],
2417     colors: [
2418         '#####',
2419         '#...W...#',
2420         '#...B...#',
2421         '#.W...W.#',
2422         '##.G#R.##',
2423         '#.C...P.#',
2424         '#####',
2425     ],
2426     directions: [
2427         '#####',
2428         '#...s...#',
2429         '#...w...#',
2430         '#.d...s.#',
2431         '##.w#w.##',
2432         '#.....#',
2433         '#####',
2434     ],
2435 },
2436 {

```

```

2437     levelId: 28,//第 28 关
2438     levelName: '关卡 28（难度 70）',
2439     perfectSteps: 49,
2440     threeStarSteps: 62,
2441     twoStarSteps: 74,
2442     oneStarSteps: 98,
2443     bestSolution: [
2444         'a','a','a','w','a','w','w','d','s','a','s','s','d','d','d','w','w','a','a','w',
2445         'a','s','d','d','d','s','s','a','a','w','s','d','d','w','w','a','a','a','a','d',
2446         'd','s','s','a','w','d','w','a','w',
2447     ],
2448     height: 6,
2449     width: 10,
2450     staticLayout: [
2451         '#####',
2452         '###.###',
2453         '#.....#',
2454         '#....#.#',
2455         '#.....#',
2456         '#####',
2457     ],
2458     objects: [
2459         '#####',
2460         '###.###',
2461         'S...11.S#',
2462         '#....#.#',
2463         '#.EE...@#',
2464         '#####',
2465     ],
2466     colors: [
2467         '#####',
2468         '###.###',
2469         '#R...WW.B#',
2470         '#....#.#',
2471         '#.RB....#',
2472         '#####',
2473     ],
2474     directions: [
2475         '#####',
2476         '###.###',
2477         '#d...sd.a#',
2478         '#....#.#',
2479         '#.....#',
2480         '#####',
2481     ],
2482 },
2483 {
2484     levelId: 29,//第 29 关
2485     levelName: '关卡 29（难度 70）',
2486     perfectSteps: 49,
2487     threeStarSteps: 62,
2488     twoStarSteps: 74,
2489     oneStarSteps: 98,
2490     bestSolution: [
2491         'a','a','a','w','a','w','w','d','s','a','s','s','d','d','d','w','w','a','a','w',
2492         'a','s','d','d','d','s','s','a','a','w','s','d','d','w','w','a','a','a','a','d',
2493         'd','s','s','a','w','d','w','a','w',
2494     ],

```

```

2495     height: 6,
2496     width: 10,
2497     staticLayout: [
2498         '#####',
2499         '###.###',
2500         '#.....#',
2501         '#....#.#',
2502         '#.....#',
2503         '#####',
2504     ],
2505     objects: [
2506         '#####',
2507         '###.###',
2508         '#S...11.S#',
2509         '#....#.#',
2510         '#.EE...@#',
2511         '#####',
2512     ],
2513     colors: [
2514         '#####',
2515         '###.###',
2516         '#R...WW.B#',
2517         '#....#.#',
2518         '#.RB....#',
2519         '#####',
2520     ],
2521     directions: [
2522         '#####',
2523         '###.###',
2524         '#d...sd.a#',
2525         '#....#.#',
2526         '#.....#',
2527         '#####',
2528     ],
2529 },
2530 {
2531     levelId: 30,//第 30 关
2532     levelName: '关卡 30（难度 67）',
2533     perfectSteps: 37,
2534     threeStarSteps: 47,
2535     twoStarSteps: 56,
2536     oneStarSteps: 74,
2537     bestSolution: [
2538         'w', 'd', 'd', 'w', 'w', 'a', 's', 'w', 'a', 'w', 'a', 'a', 's', 'd', 's', 's', 'd', 'd',
2539         'w', 'w', 'a', 'w', 'a', 's', 's', 'w', 'd', 'd', 's', 's', 'a', 'a', 'd', 's', 'a', 'w', 'w',
2540     ],
2541     height: 7,
2542     width: 9,
2543     staticLayout: [
2544         '#####',
2545         '##...###',
2546         '##.....#',
2547         '##..#.#',
2548         '##.....#',
2549         '#....###',
2550         '#####',
2551     ],
2552     objects: [

```

```

2553         '#####',
2554         '##...####',
2555         '##.3...##',
2556         '##E3#2.##',
2557         '##.....E#',
2558         '#S..@S###',
2559         '#####',
2560     ],
2561     colors: [
2562         '#####',
2563         '##...####',
2564         '##.W...##',
2565         '##PW#G.##',
2566         '##.....W#',
2567         '#B...R###',
2568         '#####',
2569     ],
2570     directions: [
2571         '#####',
2572         '##...####',
2573         '##.d...##',
2574         '##.w#d.##',
2575         '##.....#',
2576         '#d...a###',
2577         '#####',
2578     ],
2579 },
2580 {
2581     levelId: 31,//第 31 关
2582     levelName: '关卡 31（难度 82）',
2583     perfectSteps: 68,
2584     threeStarSteps: 85,
2585     twoStarSteps: 102,
2586     oneStarSteps: 136,
2587     bestSolution: [
2588         'a','a','a','a','w','w','d','d','d','d','s','w','a','a','a','a','s','s','a','s',
2589         's','d','w','w','w','s','d','d','d','s','d','d','w','a','a','a','a','d','d','d',
2590         'w','w','a','a','a','w','a','a','s','d','s','s','a','s','s','d','w','w','d','d',
2591         'd','d','w','w','a','a','a','w',
2592     ],
2593     height: 8,
2594     width: 9,
2595     staticLayout: [
2596         '#####',
2597         '##...####',
2598         '##.....#',
2599         '##...##.##',
2600         '##.....#',
2601         '##...##.##',
2602         '##...####',
2603         '#####',
2604     ],
2605     objects: [
2606         '#####',
2607         '##...####',
2608         '##.....E#',
2609         '#S.S##3##',
2610         '##.....@.#',

```

```

2611         '#.1##...#',
2612         '#.#####',
2613         '#####',
2614     ],
2615     colors: [
2616         '#####',
2617         '#...####',
2618         '#.....P#',
2619         '#R.B##W##',
2620         '#.....#',
2621         '#.W##...#',
2622         '#.#####',
2623         '#####',
2624     ],
2625     directions: [
2626         '#####',
2627         '#...####',
2628         '#.....#',
2629         '#d.a##w##',
2630         '#.....#',
2631         '#.d##...#',
2632         '#.#####',
2633         '#####',
2634     ],
2635 },
2636 {
2637     levelId: 32, //第 32 关
2638     levelName: '关卡 32（难度 36）',
2639     perfectSteps: 11,
2640     threeStarSteps: 14,
2641     twoStarSteps: 17,
2642     oneStarSteps: 22,
2643     bestSolution: [
2644         'd', 'w', 'w', 'a', 'w', 's', 'd', 'd', 'w', 'a', 's',
2645     ],
2646     height: 7,
2647     width: 8,
2648     staticLayout: [
2649         '#####',
2650         '#...####',
2651         '#.....#',
2652         '#.....##',
2653         '##...###',
2654         '##...###',
2655         '#####',
2656     ],
2657     objects: [
2658         '#####',
2659         '#...####',
2660         '#.S0..S#',
2661         '#.....##',
2662         '##.E3###',
2663         '##.@.###',
2664         '#####',
2665     ],
2666     colors: [
2667         '#####',
2668         '#...####',

```

```

2669         '#.RW..B#',
2670         '#.....##',
2671         '##.PW###',
2672         '##...###',
2673         '#####',
2674     ],
2675     directions: [
2676         '#####',
2677         '#...###',
2678         '#.dw..a#',
2679         '#.....##',
2680         '##..s###',
2681         '##...###',
2682         '#####',
2683     ],
2684 },
2685 {
2686     levelId: 33,//第 33 关
2687     levelName: '关卡 33（难度 76）',
2688     perfectSteps: 58,
2689     threeStarSteps: 68,
2690     twoStarSteps: 88,
2691     oneStarSteps: 120,
2692     bestSolution: [
2693         'a','a','a','a','w','w','d','d','s','w','a','a','s','s','d','s','s','d','d','w',
2694         'w','d','w','a','s','s','s','a','a','w','w','d','a','a','w','w','d','d','s','d',
2695         's','a','a','s','s','d','d','w','s','a','a','w','w','d','w','d','d','s',
2696     ],
2697     height: 7,
2698     width: 8,
2699     staticLayout: [
2700         '#####',
2701         '#....###',
2702         '##...###',
2703         '#.....#',
2704         '#..###',
2705         '#....###',
2706         '#####',
2707     ],
2708     objects: [
2709         '#####',
2710         '#...S###',
2711         '#.#31.##',
2712         '#....@S#',
2713         '#..###',
2714         '#E...###',
2715         '#####',
2716     ],
2717     colors: [
2718         '#####',
2719         '#...R###',
2720         '#.WB.##',
2721         '#....G#',
2722         '#..###',
2723         '#P...###',
2724         '#####',
2725     ],
2726     directions: [

```

```

2727         '#####',
2728         '#...s###',
2729         '#. #wd. ##',
2730         '#.....a#',
2731         '#..#.###',
2732         '#....###',
2733         '#####',
2734     ],
2735 },
2736 {
2737     levelId: 34,//第 34 关
2738     levelName: '关卡 34（难度 44）',
2739     perfectSteps: 11,
2740     threeStarSteps: 18,
2741     twoStarSteps: 26,
2742     oneStarSteps: 40,
2743     bestSolution: [
2744         'd','s','a','s','a','w','d','d','d','s','a',
2745     ],
2746     height: 6,
2747     width: 7,
2748     staticLayout: [
2749         '#####',
2750         '#.....#',
2751         '#.....#',
2752         '#.....#',
2753         '#.....#',
2754         '#####',
2755     ],
2756     objects: [
2757         '#####',
2758         '#S.E.S#',
2759         '#..@..#',
2760         '#..3..#',
2761         '#.1.1.#',
2762         '#####',
2763     ],
2764     colors: [
2765         '#####',
2766         '#W.Y.W#',
2767         '#.....#',
2768         '#..W..#',
2769         '#.R.G.#',
2770         '#####',
2771     ],
2772     directions: [
2773         '#####',
2774         '#s...s#',
2775         '#.....#',
2776         '#..w..#',
2777         '#.w.a.#',
2778         '#####',
2779     ],
2780 },
2781 {
2782     levelId: 35,//第 35 关
2783     levelName: '关卡 35（难度 101）',
2784     perfectSteps: 70,

```



```

2785     threeStarSteps: 89,
2786     twoStarSteps: 105,
2787     oneStarSteps: 140,
2788     bestSolution: [
2789         'w', 'w', 'w', 'a', 'w', 'a', 'a', 'a', 's', 's', 'd', 'd', 'd', 'w', 'w', 'a', 's', 'd', 's', 'a',
2790         'd', 'w', 'd', 'd', 's', 'a', 's', 's', 'a', 'a', 'w', 'w', 'd', 'w', 'w', 'a', 'a', 'a', 's', 's',
2791         'd', 'd', 's', 's', 'd', 'd', 'w', 'w', 'd', 'w', 'a', 's', 's', 's', 'a', 'a', 'w', 'w', 'd', 'a',
2792         'a', 'a', 'w', 'w', 'd', 'd', 's', 'w', 'd', 's',
2793     ],
2794     height: 8,
2795     width: 9,
2796     staticLayout: [
2797         '#####',
2798         '#####',
2799         '##....##',
2800         '##.....#',
2801         '#.....#',
2802         '####...##',
2803         '####...##',
2804         '#####',
2805     ],
2806     objects: [
2807         '#####',
2808         '#####S###',
2809         '##....S##',
2810         '##.S1...#',
2811         '#S..4...#',
2812         '####OE.##',
2813         '####..@##',
2814         '#####',
2815     ],
2816     colors: [
2817         '#####',
2818         '#####G###',
2819         '##....W##',
2820         '##.GB...#',
2821         '#G..R...#',
2822         '####WP.##',
2823         '####...##',
2824         '#####',
2825     ],
2826     directions: [
2827         '#####',
2828         '#####s###',
2829         '##....s##',
2830         '##.da...#',
2831         '#d..s...#',
2832         '####w..##',
2833         '####...##',
2834         '#####',
2835     ],
2836 },
2837 {
2838     // 测试关: 左 11 列原 L99 (十字网+纵向锯齿+L28) + col11 中间墙 + 右 9 列 L31
2839     levelId: 99,
2840     levelName: '测试光路 (十字网+L31+L28)',
2841     height: 14,
2842     width: 21,

```

```

2843 staticLayout: [
2844     '#####',
2845     '#.....####...###',
2846     '#.....#.....##',
2847     '#.....#..#..##',
2848     '#.....##.....',
2849     '#.....###....###',
2850     '####.#####',
2851     '####.#####....###',
2852     '#####.#####',
2853     '####.#####',
2854     '#.....#####',
2855     '#....#..#####',
2856     '#.....#####',
2857     '#####',
2858 ],
2859 objects: [
2860     '#####',
2861     '#E.S.S.S.####...####',
2862     '#.....#.....##',
2863     '#S.....S#.##E..##',
2864     '#.4.4.4....##3..2.E#',
2865     '#.....###S3..S##',
2866     '####.#####',
2867     '####.#####S.2.S##',
2868     '#####.#####',
2869     '####.#####',
2870     '#S...11.S#####',
2871     '#....#..#####',
2872     '#.EE...@#####',
2873     '#####',
2874 ],
2875 colors: [
2876     '#####',
2877     '#W.R.G.B.####...####',
2878     '#.....#.....##',
2879     '#R.....B#.##P.#.##',
2880     '#.W.W.W....##W.G.W#',
2881     '#.....###BW..R##',
2882     '####.#####',
2883     '####.#####W.B.R##',
2884     '#####.#####',
2885     '####.#####',
2886     '#R...WW.B#####',
2887     '#....#..#####',
2888     '#.RB....#####',
2889     '#####',
2890 ],
2891 directions: [
2892     '#####',
2893     '#.s.s.s.####...####',
2894     '#.....#.....##',
2895     '#d.....a#.##..##',
2896     '#.w.w.w....##d..d..#',
2897     '#.....###dw..a##',
2898     '####.#####',
2899     '####.#####d.d.a##',
2900     '#####',

```

```

2901         '####.#####',
2902         '#d...sd.a#####',
2903         '#.....#####',
2904         '#.....#####',
2905         '#####',
2906     ],
2907 },
2908 /*{
2909     levelId: 7,
2910     levelName: '双目标',
2911     perfectSteps: 18,
2912     threeStarSteps: 25,
2913     twoStarSteps: 35,
2914     oneStarSteps: 50,
2915     height: 8,
2916     width: 10,
2917     staticLayout: [
2918         '#####',
2919         '#.....#',
2920         '#.....#',
2921         '#.....#',
2922         '#.....#',
2923         '#.....#',
2924         '#.....#',
2925         '#####',
2926     ],
2927     objects: [
2928         '#####',
2929         '#...S...#',
2930         '#.....#',
2931         '#.....#',
2932         '#@.....#',
2933         '#.....#',
2934         '#.E...E.#',
2935         '#####',
2936     ],
2937     colors: [
2938         '#####',
2939         '#...W...#',
2940         '#.....#',
2941         '#.....#',
2942         '#.....#',
2943         '#.....#',
2944         '#.W...W.#',
2945         '#####',
2946     ],
2947     directions: [
2948         '#####',
2949         '#...S...#',
2950         '#.....#',
2951         '#.....#',
2952         '#.....#',
2953         '#.....#',
2954         '#.....#',
2955         '#####',
2956     ],
2957 },
2958 {

```

```

2959     levelId: 8,
2960     levelName: '地形测试',
2961     perfectSteps: 24,
2962     threeStarSteps: 32,
2963     twoStarSteps: 45,
2964     oneStarSteps: 60,
2965     height: 15,
2966     width: 19,
2967     staticLayout: [
2968         '#####',
2969         '#.....#',
2970         '#.##...#.##...#',
2971         '#.#.....##...#',
2972         '#.#....###.##...#',
2973         '#.....#.##...#',
2974         '#.....###.##...#',
2975         '#.....#',
2976         '#.....#####',
2977         '#.....#...#...#',
2978         '#.....#...#...#',
2979         '#.....#####',
2980         '#.###.....##...#',
2981         '#.#.....##...#',
2982         '#####',
2983     ],
2984     objects: [
2985         '#####',
2986         '#S.....E#',
2987         '#.##...#.##...#',
2988         '#.#...E...##...#',
2989         '#.#...4###.##...#',
2990         '#.{...2#.##...#',
2991         '#.....}.###.##...#',
2992         '#....S.../.....#',
2993         '#.....#####',
2994         '#.....#1.3#.....#',
2995         '#.....#@.#.....#',
2996         '#.....#####',
2997         '#.###...E...##...#',
2998         '#.#...E...##.E...#',
2999         '#####',
3000     ],
3001     colors: [
3002         '#####',
3003         '#W.....R#',
3004         '#.##...#.##...#',
3005         '#.#...G...##...#',
3006         '#.#...B###.##...#',
3007         '#.W...G#.##...#',
3008         '#.....W.###.##...#',
3009         '#....B...W.....#',
3010         '#.....#####',
3011         '#.....#R.W#.....#',
3012         '#.....#...#...#',
3013         '#.....#####',
3014         '#.###...Y...##...#',
3015         '#.#...C...##.P...#',
3016         '#####',

```

```

3017     ],
3018     directions: [
3019         '#####',
3020         '#s.....#',
3021         '#.##...#.##...#',
3022         '#.#.....##...#',
3023         '#.#....d###.##...#',
3024         '#.w...a#.##...#',
3025         '#.....w###.##...#',
3026         '#....a....d.....#',
3027         '#.....#####',
3028         '#.....#d.s#.....',
3029         '#.....#...#.....',
3030         '#.....#####',
3031         '#.###.....##...#',
3032         '#.#.....##...#',
3033         '#####',
3034     ],
3035 },*/
3036 ];
3037 /** 正式手工关卡 */
3038 const ALL_LAYERED_SOURCES: IOpticalLevelLayeredSource[] = [
3039     ...OPTICAL_LEVEL_LAYERED_SOURCES,
3040 ];
3041 export const OPTICAL_LEVELS: IOpticalLevelConfig[] = ALL_LAYERED_SOURCES.map((src) =>
3042     parseLayeredGridsToLevelConfig(src),
3043 );
3044 const _byId = new Map<number, IOpticalLevelConfig>(OPTICAL_LEVELS.map((lv) => [lv.levelId, lv]));
3045 export function getOpticalLevelById(levelId: number): IOpticalLevelConfig | undefined {
3046     return _byId.get(levelId);
3047 }
3048 /** 关卡表中最小 levelId (通常为第 1 关) */
3049 export function getFirstOpticalLevelId(): number {
3050     let first: number | null = null;
3051     for (const lv of OPTICAL_LEVELS) {
3052         if (first === null || lv.levelId < first) {
3053             first = lv.levelId;
3054         }
3055     }
3056     return first ?? 1;
3057 }
3058 /** 大于当前 id 的最小关卡 id; 无后续关卡时返回 null */
3059 export function getNextOpticalLevelId(currentLevelId: number): number | null {
3060     let next: number | null = null;
3061     for (const lv of OPTICAL_LEVELS) {
3062         if (lv.levelId <= currentLevelId) {
3063             continue;
3064         }
3065         if (next === null || lv.levelId < next) {
3066             next = lv.levelId;
3067         }
3068     }
3069     return next;
3070 }
3071 // ----- assets/Scripts/Games/OpticalPuzzle/Config/OpticalPuzzleLevelSchema.ts -----
3072 import type { ColorMode, Direction } from '../Core/OpticalPuzzleTypes';
3073 import type { PieceConnectivity } from '../Core/OpticalPieceConnectivity';
3074 import { parseConnectivityChar } from '../Core/OpticalPieceConnectivity';

```

```

3075 import { ColorMode as ColorModeEnum, Direction as DirEnum, PieceType, TerrainKind } from
3076 '../Core/OpticalPuzzleTypes';
3077 export interface IOpticalLightSource {
3078     x: number;
3079     y: number;
3080     direction: Direction;
3081     colorKey?: string;
3082 }
3083 export interface IOpticalTarget {
3084     x: number;
3085     y: number;
3086     colorKey?: string;
3087 }
3088 export interface IOpticalPiece {
3089     id: string;
3090     /** 统一通道类型 0~4 (旧 `type` 字段保留兼容, 新逻辑以本字段为准) */
3091     connectivity: PieceConnectivity;
3092     type: string;
3093     colorMode: ColorMode;
3094     x: number;
3095     y: number;
3096     /** 层 4 朝向: w 默认, d/s/a 为相对默认右转 90° × 1/2/3 (同 w) */
3097     direction: Direction;
3098     portalGroup?: string;
3099 }
3100 /**
3101  * 关卡源数据: 四张同尺寸二维字符表 ( `[y][x]` 单字符), 解析见 `parseLayeredGridsToLevelConfig`。
3102  *
3103  * —— 层 1 `staticLayout` (静态地形, 局内不可推动) ——
3104  * - `#` 墙 / 石块 (统一挡光、不可进入, 不再单独区分 stone)
3105  * - `.` 或空格 可走地板
3106  *
3107  * —— 层 2 `objects` (玩家、光源、目标、可推动光学元件) ——
3108  * - `#` 与层 1 对齐 (墙格写 `#`, 空地写 `.`)
3109  * - `@` 玩家出生 (全关唯一)
3110  * - `S` 光源 (颜色见层 3, 朝向见层 4)
3111  * - `E` 目标 (期望颜色见层 3, 层 4 一般为 `.`)
3112  * - 元件通道 (层 2 推荐数字 `0`~`4`, 层 4 `w/a/s/d` 旋转; 同 `w`):
3113  *
3114  * 默认朝向通道图 ( `O` 为格心留空光路, 线为有开口的通道臂):
3115  *
3116  * 0 挡光 (无通道; 层 4 仍写 `w/a/s/d` 或 `.` , 同 `w` , 朝向不影响光路与占位绘制)
3117  *
3118  *      1 上+右      2 上+下      3 上+左+右      4 四面
3119  *      ┌───┐      ┌───┐      ┌───┐      ┌───┐      ┌───┐
3120  *      │■■■│      │ ○— │      │ ○  │      │ —○— │      │
3121  *  O  │■■■│      │   │      │   │      │   │      │
3122  *      │■■■│      │   │      │   │      │   │      │   │
3123  *      └───┐      └───┐      └───┐      └───┐      └───┐
3124  *
3125  *
3126  * 传播约定: 向下传播 = 从上方进入 (见 `propagationToEntrySide`)。
3127  * 混色: 见 `OpticalColorMix.ts` (红绿蓝白 + 黄青紫二次色)。
3128  *
3129  * 旧字符兼容: ` / `→1, ` | `→2, ` T `→3, ` + `→4, ` o `→4, ` - `→2 (默认 `d` 横向)。
3130  *
3131  * —— 层 3 `colors` (光色, 仅标注有物件的格子) ——
3132  * - `#` 墙; 无物件的地板 / 玩家格为 `.` (不要整图铺 `w`)

```

```

3133 * - `S`/`E`: `W`/`R`/`G`/`B`/`Y`/`C`/`P` (白/红/绿/蓝/黄/青/紫); `` 表示默认 `W`
3134 * - 光学元件: 仅 `W`/`R`/`G`/`B` (`` 同 `W`)
3135 * - 关卡数据须四层齐全; `overlayLayersFromObjects` 仅作编辑辅助, 不替代正式配置
3136 *
3137 * —— 层 4 `directions` (朝向, 仅标注需要朝向的物件) ——
3138 * - `#` 墙: 无物件 / 玩家 / 目标为 ``
3139 * - 光源 `S`: `` 默认朝下; 定向元件与 `/` 镜: `w` 默认朝向, `` 同 `w`, `d`/`s`/`a` 为相对默认旋转 90°
3140 /180° /270°
3141 */
3142 export interface IOpticalLevelLayeredData {
3143     height: number;
3144     width: number;
3145     staticLayout: readonly string[];
3146     objects: readonly string[];
3147     colors: readonly string[];
3148     directions: readonly string[];
3149 }
3150 /** 通关步数评星阈值 (步数越少越好) */
3151 export interface IOpticalLevelStarThresholds {
3152     /** 完美步数: 三颗星均发光 */
3153     perfectSteps: number;
3154     /** 三星步数: 三颗星涂满 */
3155     threeStarSteps: number;
3156     /** 二星步数: 左+中涂满, 右不发光 */
3157     twoStarSteps: number;
3158     /** 一星步数: 仅左涂满, 中+右不发光 */
3159     oneStarSteps: number;
3160 }
3161 export type IOpticalLevelLayeredSource = IOpticalLevelLayeredData & {
3162     levelId: number;
3163     levelName: string;
3164     perfectSteps?: number;
3165     threeStarSteps?: number;
3166     twoStarSteps?: number;
3167     oneStarSteps?: number;
3168     /**
3169      * BFS 最佳解法: 方向键 `w/a/s/d` (与层 4 朝向一致; 回放时由 Core 判定移动或推块)。
3170      */
3171     bestSolution?: readonly string[];
3172 };
3173 /** 运行时关卡 (terrain 行优先:  $i = y * width + x$ ) */
3174 export interface IOpticalLevelConfig {
3175     levelId: number;
3176     levelName: string;
3177     width: number;
3178     height: number;
3179     terrain: TerrainKind[];
3180     player: { x: number; y: number };
3181     pieces: IOpticalPiece[];
3182     sources: IOpticalLightSource[];
3183     targets: IOpticalTarget[];
3184     starThresholds: IOpticalLevelStarThresholds;
3185     /** 见 `IOpticalLevelLayeredSource.bestSolution` */
3186     bestSolution?: readonly string[];
3187 }
3188 /** 未在关卡源数据中填写时的默认评星步数 */
3189 export function defaultStarThresholdsForLevel(levelId: number): IOpticalLevelStarThresholds {
3190     const base = Math.max(4, 5 + levelId * 2);

```

```

3191         return {
3192             perfectSteps: base,
3193             threeStarSteps: base + 3,
3194             twoStarSteps: base + 8,
3195             oneStarSteps: base + 15,
3196         };
3197     }
3198     function resolveStarThresholds(src: IOpticalLevelLayeredSource): IOpticalLevelStarThresholds {
3199         const fallback = defaultStarThresholdsForLevel(src.levelId);
3200         return {
3201             perfectSteps: src.perfectSteps ?? fallback.perfectSteps,
3202             threeStarSteps: src.threeStarSteps ?? fallback.threeStarSteps,
3203             twoStarSteps: src.twoStarSteps ?? fallback.twoStarSteps,
3204             oneStarSteps: src.oneStarSteps ?? fallback.oneStarSteps,
3205         };
3206     }
3207     /** 旧层 2 字符 → 通道类型 (`` 须在层 4 写 `d` 表横向) */
3208     const LEGACY_OBJ_CONNECTIVITY: Readonly<Record<string, PieceConnectivity>> = {
3209         '/': 1,
3210         '|': 2,
3211         '-': 2,
3212         o: 4,
3213         T: 3,
3214         '+': 4,
3215     };
3216     /** 层 2: 几何类型 → PieceType (遗留; 新关卡请用数字 0~4) */
3217     const OBJECT_PIECE_CHAR: Readonly<Record<string, PieceType>> = {
3218         o: PieceType.Glass,
3219         '-': PieceType.GlassH,
3220         '|': PieceType.GlassV,
3221         '/': PieceType.MirrorSlash,
3222         T: PieceType.SplitterT,
3223         '+': PieceType.SplitterCross,
3224     };
3225     const DIR_CHAR: Readonly<Record<string, DirEnum>> = {
3226         d: DirEnum.Right,
3227         w: DirEnum.Up,
3228         a: DirEnum.Left,
3229         s: DirEnum.Down,
3230     };
3231     /** 层 3 光学元件光色 (RGBW; `` 为默认 `W`) */
3232     export type Layer3PieceColorChar = 'W' | 'R' | 'G' | 'B';
3233     /** 层 3 光源/目标光色 (含二次色 Y/C/P) */
3234     export type Layer3SourceTargetColorChar = Layer3PieceColorChar | 'Y' | 'C' | 'P';
3235     /**
3236      * 由静态层生成层 3/4 全占位 (墙 `#`, 其余 ``)。仅适合无 objects 信息时; 有关卡物件请用
3237      * `overlayLayersFromObjects`。
3238      */
3239     export function blankOverlayLayersFromStatic(staticLayout: readonly string[]): {
3240         colors: string[];
3241         directions: string[];
3242     } {
3243         const pad = staticLayout.map((row) =>
3244             row
3245                 .split("")
3246                 .map((ch) => (ch === '#' ? '#' : ''))
3247                 .join(""),
3248         );

```



```

3249     return { colors: pad, directions: pad.map((row) => row) };
3250 }
3251 /** 层 4 需要写明或默认解析朝向的物件 */
3252 function objectUsesDirection(obj: string): boolean {
3253     if (obj === 'S') {
3254         return true;
3255     }
3256     if (parseConnectivityChar(obj) !== null) {
3257         return true;
3258     }
3259     return obj === '/' || obj === '-' || obj === '|' || obj === 'T' || obj === '+';
3260 }
3261 /** 元件层 4: `.` 与 `w` 同为默认朝向 (Up) */
3262 function parsePieceRotation(ch: string, levelId: number, x: number, y: number): DirEnum {
3263     return parseDirectionCell(ch, levelId, x, y, DirEnum.Up);
3264 }
3265 /**
3266  * `/' 镜基件 + 旋转 → 通道 1 (遗留解析: 新关直接写数字 `1')。
3267  */
3268 function legacyMirrorConnectivity(_rot: DirEnum): PieceConnectivity {
3269     return 1;
3270 }
3271 /**
3272  * 将 `staticLayout` 中 `#` 的格同步到另一层 (层 3/4 须与静态墙对齐, 否则解析报错)。
3273  */
3274 export function applyStaticWallsToLayer(
3275     staticLayout: readonly string[],
3276     layer: readonly string[],
3277 ): string[] {
3278     return staticLayout.map((stRow, y) => {
3279         const row = layer[y] ?? '';
3280         let out = '';
3281         for (let x = 0; x < stRow.length; x++) {
3282             out += stRow.charAt(x) === '#' ? '#' : row.charAt(x) || '.';
3283         }
3284         return out;
3285     });
3286 }
3287 /**
3288  * 按层 2 `objects` 生成稀疏的层 3/4 草稿; **须再** `applyStaticWallsToLayer` 对齐内部墙 `#`。
3289  */
3290 export function overlayLayersFromObjects(objects: readonly string[]): {
3291     colors: string[];
3292     directions: string[];
3293 } {
3294     const colors: string[] = [];
3295     const directions: string[] = [];
3296     for (const row of objects) {
3297         let c = '';
3298         let d = '';
3299         for (const obj of row) {
3300             if (obj === '#') {
3301                 c += '#';
3302                 d += '#';
3303             } else if (obj === 'S' || obj === 'E' || parseConnectivityChar(obj) !== null ||
3304 OBJECT_PIECE_CHAR[obj] || obj === '/') {
3305                 c += 'W';
3306                 if (obj === 'S') {

```

```

3307             d += 's';
3308         } else if (obj === '-') {
3309             d += 'd';
3310         } else if (obj === '/' || parseConnectivityChar(obj) !== null || objectUsesDirection(obj)) {
3311             d += 'w';
3312         } else {
3313             d += '.';
3314         }
3315     } else {
3316         c += '.';
3317         d += '.';
3318     }
3319 }
3320 colors.push(c);
3321 directions.push(d);
3322 }
3323 return { colors, directions };
3324 }
3325 function assertLayer3Padding(
3326     ch: string,
3327     levelId: number,
3328     x: number,
3329     y: number,
3330     label: string,
3331 ): void {
3332     if (!isEmptyChar(ch)) {
3333         throw new Error(
3334             `[parseLayeredGrids] id=${levelId}: ${label} (${x},${y}) 层 3 应为 ., 当前 '${ch}',
3335         );
3336     }
3337 }
3338 function normalizeLayer3ColorChar(ch: string): string {
3339     if (ch === 'w') {
3340         return 'W';
3341     }
3342     if (ch.length === 1 && ch >= 'a' && ch <= 'z') {
3343         return ch.toUpperCase();
3344     }
3345     return ch;
3346 }
3347 /** 解析层 3 元件色 (仅 W/R/G/B) */
3348 function resolveLayer3PieceColor(
3349     ch: string,
3350     levelId: number,
3351     x: number,
3352     y: number,
3353 ): Layer3PieceColorChar {
3354     const upper = normalizeLayer3ColorChar(ch);
3355     if (isEmptyChar(ch) || upper === 'W') {
3356         return 'W';
3357     }
3358     if (upper === 'R' || upper === 'G' || upper === 'B') {
3359         return upper;
3360     }
3361     throw new Error(
3362         `[parseLayeredGrids] id=${levelId}: 元件层 3 (${x},${y}) 须为 W/R/G/B 或 ., 当前 '${ch}',
3363     );
3364 }

```

```

3365  /** 解析层 3 光源/目标色 (W/R/G/B/Y/C/P) */
3366  function resolveLayer3SourceTargetColor(
3367      ch: string,
3368      levelId: number,
3369      x: number,
3370      y: number,
3371  ): Layer3SourceTargetColorChar {
3372      const upper = normalizeLayer3ColorChar(ch);
3373      if (isEmptyChar(ch) || upper === 'W') {
3374          return 'W';
3375      }
3376      if (upper === 'R' || upper === 'G' || upper === 'B' || upper === 'Y' || upper === 'C' || upper === 'P') {
3377          return upper;
3378      }
3379      throw new Error(
3380          `[parseLayeredGrids] id=${levelId}: 光源/目标层 3(${x},${y})须为 W/R/G/B/Y/C/P 或 ., 当前 '${ch}'`,
3381      );
3382  }
3383  function layer3ToColorMode(c: Layer3PieceColorChar): ColorModeEnum {
3384      switch (c) {
3385          case 'W':
3386              return ColorModeEnum.Through;
3387          case 'R':
3388              return ColorModeEnum.FilterRed;
3389          case 'G':
3390              return ColorModeEnum.FilterGreen;
3391          case 'B':
3392              return ColorModeEnum.FilterBlue;
3393      }
3394  }
3395  function layer3ToColorKey(c: Layer3SourceTargetColorChar): string {
3396      switch (c) {
3397          case 'W':
3398              return 'white';
3399          case 'R':
3400              return 'red';
3401          case 'G':
3402              return 'green';
3403          case 'B':
3404              return 'blue';
3405          case 'Y':
3406              return 'yellow';
3407          case 'C':
3408              return 'cyan';
3409          case 'P':
3410              return 'purple';
3411      }
3412  }
3413  function parseColorCell(ch: string, levelId: number, x: number, y: number): ColorModeEnum {
3414      return layer3ToColorMode(resolveLayer3PieceColor(ch, levelId, x, y));
3415  }
3416  function parseColorKey(ch: string, levelId: number, x: number, y: number): string {
3417      return layer3ToColorKey(resolveLayer3SourceTargetColor(ch, levelId, x, y));
3418  }
3419  function parseDirectionCell(
3420      ch: string,
3421      levelId: number,
3422      x: number,

```

```

3423     y: number,
3424     defaultDir: DirEnum,
3425 ): DirEnum {
3426     if (ch === '.' || ch === ' ') {
3427         return defaultDir;
3428     }
3429     const dir = DIR_CHAR[ch];
3430     if (dir === undefined) {
3431         throw new Error(
3432             `[parseLayeredGrids] id=${levelId}: 层 4 非法方向 '${ch}' (${x},${y}), 应为 wasd 或 .`,
3433         );
3434     }
3435     return dir;
3436 }
3437 function assertLayerGrid(
3438     name: string,
3439     rows: readonly string[],
3440     levelId: number,
3441     height: number,
3442     width: number,
3443 ): void {
3444     if (rows.length !== height) {
3445         throw new Error(`[parseLayeredGrids] id=${levelId}: ${name} 行数 ${rows.length} ≠ height
3446 ${height}`);
3447     }
3448     for (let y = 0; y < height; y++) {
3449         if (rows[y].length !== width) {
3450             throw new Error(
3451                 `[parseLayeredGrids] id=${levelId}: ${name} 第 ${y} 行列数 ${rows[y].length} ≠ width
3452 ${width}`,
3453             );
3454         }
3455     }
3456 }
3457 function isWallChar(ch: string): boolean {
3458     return ch === '#';
3459 }
3460 function isEmptyChar(ch: string): boolean {
3461     return ch === '.' || ch === ' ';
3462 }
3463 export function parseLayeredGridsToLevelConfig(src: IOpticalLevelLayeredSource): IOpticalLevelConfig {
3464     const { levelId, levelName, height, width, staticLayout, objects, colors, directions } = src;
3465     assertLayerGrid('staticLayout', staticLayout, levelId, height, width);
3466     assertLayerGrid('objects', objects, levelId, height, width);
3467     assertLayerGrid('colors', colors, levelId, height, width);
3468     assertLayerGrid('directions', directions, levelId, height, width);
3469     const terrain: TerrainKind[] = [];
3470     const sources: IOpticalLightSource[] = [];
3471     const targets: IOpticalTarget[] = [];
3472     const pieces: IOpticalPiece[] = [];
3473     let px = 0;
3474     let py = 0;
3475     let playerFound = false;
3476     const portalStack: number[] = [];
3477     let portalGroupSeq = 0;
3478     let pieceSeq = 0;
3479     const pushPiece = (
3480         connectivity: PieceConnectivity,

```

```

3481     pieceType: PieceType,
3482     colorMode: ColorModeEnum,
3483     x: number,
3484     y: number,
3485     pieceDirection: DirEnum,
3486     portalGroup?: string,
3487   ): void => {
3488     terrain.push(TerrainKind.Floor);
3489     pieces.push({
3490       id: `piece_${levelId}_${pieceSeq++}`,
3491       connectivity,
3492       type: pieceType,
3493       colorMode,
3494       x,
3495       y,
3496       direction: pieceDirection,
3497       portalGroup,
3498     });
3499   };
3500   for (let y = 0; y < height; y++) {
3501     for (let x = 0; x < width; x++) {
3502       const st = staticLayout[y].charAt(x);
3503       const obj = objects[y].charAt(x);
3504       const col = colors[y].charAt(x);
3505       const dir = directions[y].charAt(x);
3506       if (isWallChar(st)) {
3507         if (!isWallChar(obj) || !isWallChar(col) || !isWallChar(dir)) {
3508           throw new Error(
3509             `[parseLayeredGrids] id=${levelId}: 静态墙（${x},${y}）在
3510 static/objects/colors/directions 四层须均为 #`,
3511           );
3512         }
3513         terrain.push(TerrainKind.Wall);
3514         continue;
3515       }
3516       if (!isEmptyChar(st)) {
3517         throw new Error(
3518           `[parseLayeredGrids] id=${levelId}: 层 1 未定义字符 '${st}'（${x},${y}）`,
3519         );
3520       }
3521       if (isWallChar(obj)) {
3522         throw new Error(
3523           `[parseLayeredGrids] id=${levelId}: 层 1 为地板但层 2 为墙（${x},${y}）`,
3524         );
3525       }
3526       if (isWallChar(col) || isWallChar(dir)) {
3527         throw new Error(
3528           `[parseLayeredGrids] id=${levelId}: 层 3/4 地板格（${x},${y}）不应为 #`,
3529         );
3530       }
3531       switch (obj) {
3532         case '.':
3533         case ' ':
3534           terrain.push(TerrainKind.Floor);
3535           assertLayer3Padding(col, levelId, x, y, '空地');
3536           if (!isEmptyChar(dir)) {
3537             throw new Error(
3538               `[parseLayeredGrids] id=${levelId}: 空地（${x},${y}）层 4 应为 .`,

```

```

3539         );
3540     }
3541     break;
3542     case '@':
3543         terrain.push(TerrainKind.Floor);
3544         if (playerFound) {
3545             throw new Error(`[parseLayeredGrids] id=${levelId}: 重复的 @`);
3546         }
3547         playerFound = true;
3548         px = x;
3549         py = y;
3550         assertLayer3Padding(col, levelId, x, y, '玩家');
3551         if (!isEmptyChar(dir)) {
3552             throw new Error(
3553                 `[parseLayeredGrids] id=${levelId}: 玩家格 (${x},${y}) 层 4 应为 .`,
3554             );
3555         }
3556         break;
3557     case 'S':
3558         terrain.push(TerrainKind.Source);
3559         sources.push({
3560             x,
3561             y,
3562             direction: parseDirectionCell(dir, levelId, x, y, DirEnum.Down),
3563             colorKey: parseColorKey(col, levelId, x, y),
3564         });
3565         break;
3566     case 'E':
3567         terrain.push(TerrainKind.Target);
3568         targets.push({
3569             x,
3570             y,
3571             colorKey: parseColorKey(col, levelId, x, y),
3572         });
3573         if (!isEmptyChar(dir)) {
3574             throw new Error(
3575                 `[parseLayeredGrids] id=${levelId}: 目标格 (${x},${y}) 层 4 应为 .`,
3576             );
3577         }
3578         break;
3579     case '!': {
3580         const g = portalGroupSeq++;
3581         portalStack.push(g);
3582         pushPiece(
3583             0,
3584             PieceType.PortalA,
3585             parseColorCell(col, levelId, x, y),
3586             x,
3587             y,
3588             parsePieceRotation(dir, levelId, x, y),
3589             String(g),
3590         );
3591         break;
3592     }
3593     case '}': {
3594         if (portalStack.length === 0) {
3595             throw new Error(`[parseLayeredGrids] id=${levelId}: 多余的 } (${x},${y})`);
3596         }

```

```

3597         const g = portalStack.pop()!;
3598         pushPiece(
3599             0,
3600             PieceType.PortalB,
3601             parseColorCell(col, levelId, x, y),
3602             x,
3603             y,
3604             parsePieceRotation(dir, levelId, x, y),
3605             String(g),
3606         );
3607         break;
3608     }
3609     case '/': {
3610         const rot = parsePieceRotation(dir, levelId, x, y);
3611         pushPiece(
3612             legacyMirrorConnectivity(rot),
3613             PieceType.Glass,
3614             parseColorCell(col, levelId, x, y),
3615             x,
3616             y,
3617             rot,
3618         );
3619         break;
3620     }
3621     default: {
3622         const connDigit = parseConnectivityChar(obj);
3623         if (connDigit !== null) {
3624             pushPiece(
3625                 connDigit,
3626                 PieceType.Glass,
3627                 parseColorCell(col, levelId, x, y),
3628                 x,
3629                 y,
3630                 parsePieceRotation(dir, levelId, x, y),
3631             );
3632             break;
3633         }
3634         const legacyConn = LEGACY_OBJ_CONNECTIVITY[obj];
3635         const pieceType = OBJECT_PIECE_CHAR[obj];
3636         if (legacyConn !== undefined && pieceType) {
3637             const rot =
3638                 obj === '-'
3639                     ? parseDirectionCell(dir, levelId, x, y, DirEnum.Right)
3640                     : parsePieceRotation(dir, levelId, x, y);
3641             pushPiece(
3642                 legacyConn,
3643                 pieceType,
3644                 parseColorCell(col, levelId, x, y),
3645                 x,
3646                 y,
3647                 rot,
3648             );
3649             break;
3650         }
3651         throw new Error(
3652             `[parseLayeredGrids] id=${levelId}: 层 2 未定义字符 '${obj}' (${x},${y})`,
3653         );
3654     }

```

```

3655         }
3656     }
3657 }
3658 if (!playerFound) {
3659     throw new Error(`[parseLayeredGrids] id=${levelId}: 缺少 @`);
3660 }
3661 if (portalStack.length > 0) {
3662     throw new Error(`[parseLayeredGrids] id=${levelId}: 未闭合的 { 传送门 (缺) `);
3663 }
3664 return {
3665     levelId,
3666     levelName,
3667     width,
3668     height,
3669     terrain,
3670     player: { x: px, y: py },
3671     pieces,
3672     sources,
3673     targets,
3674     starThresholds: resolveStarThresholds(src),
3675     bestSolution: src.bestSolution?.length ? [...src.bestSolution] : undefined,
3676 };
3677 }
3678 const DEV_STATIC = [
3679     '#####',
3680     '#.....#',
3681     '#.....#',
3682     '#.....#',
3683     '#.....#',
3684     '#.....#',
3685     '#####',
3686 ];
3687 const DEV_OBJECTS = [
3688     '#####',
3689     '#.....#',
3690     '#.....#',
3691     '#..@..#',
3692     '#.....#',
3693     '#.....#',
3694     '#####',
3695 ];
3696 const { colors: DEV_COLORS, directions: DEV_DIRECTIONS } =
3697     overlayLayersFromObjects(DEV_OBJECTS);
3698 export const DEV_LEVEL_MINIMAL: IOpticalLevelConfig = parseLayeredGridsToLevelConfig({
3699     levelId: 0,
3700     levelName: '开发最小',
3701     height: 7,
3702     width: 7,
3703     staticLayout: DEV_STATIC,
3704     objects: DEV_OBJECTS,
3705     colors: DEV_COLORS,
3706     directions: DEV_DIRECTIONS,
3707 });
3708 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalBeamContactResolve.ts -----
3709 import type { OpticalBeamBlockContact, OpticalBeamSegment } from './OpticalBeamTypes';
3710 import { coerceBeamColorKey, mixLightColors } from './OpticalColorMix';
3711 import { resolveBeamColorKey } from './OpticalLightColor';
3712 import { Direction, normalizeDirection } from './OpticalPuzzleTypes';

```



```

3713 const ENDPOINT_EPS = 2e-3;
3714 function oppositeDirection(dir: Direction): Direction {
3715     return ((dir + 2) % 4) as Direction;
3716 }
3717 /** 光段 x0→x1 的传播方向（与光追 DIR 一致） */
3718 function segmentPropagationDir(seg: OpticalBeamSegment): Direction {
3719     const dx = seg.x1 - seg.x0;
3720     const dy = seg.y1 - seg.y0;
3721     if (Math.abs(dx) >= Math.abs(dy)) {
3722         return dx >= 0 ? Direction.Right : Direction.Left;
3723     }
3724     return dy <= 0 ? Direction.Up : Direction.Down;
3725 }
3726 function pointsNear(ax: number, ay: number, bx: number, by: number): boolean {
3727     return Math.abs(ax - bx) <= ENDPOINT_EPS && Math.abs(ay - by) <= ENDPOINT_EPS;
3728 }
3729 /**
3730  * 合并后光段端点顺序可能反转（y0>y1 等），按端点与传播方向匹配接触点。
3731  * @returns 严格方向匹配 > 端点重合兜底 > null
3732  */
3733 function matchSegmentAtContact(
3734     contact: OpticalBeamBlockContact,
3735     seg: OpticalBeamSegment,
3736     dir: Direction,
3737 ): 'strict' | 'endpoint' | null {
3738     const atEnd = pointsNear(seg.x1, seg.y1, contact.x, contact.y);
3739     const atStart = pointsNear(seg.x0, seg.y0, contact.x, contact.y);
3740     if (!atEnd && !atStart) {
3741         return null;
3742     }
3743     const segDir = segmentPropagationDir(seg);
3744     const beamDirAtEnd = atEnd ? segDir : oppositeDirection(segDir);
3745     if (beamDirAtEnd === dir) {
3746         return 'strict';
3747     }
3748     return 'endpoint';
3749 }
3750 /**
3751  * 阻挡接触色键：以合并后光路段为准（与 BeamView 一致）。
3752  * blockContact.colorKey 在重叠混色后可能仍为旧值（如 white）。
3753  */
3754 export function resolveBlockContactColorFromSegments(
3755     contact: OpticalBeamBlockContact,
3756     segments: readonly OpticalBeamSegment[],
3757 ): string {
3758     const dir = normalizeDirection(contact.dir, Direction.Right);
3759     let endpointFallback: string | null = null;
3760     for (let i = segments.length - 1; i >= 0; i--) {
3761         const seg = segments[i];
3762         const match = matchSegmentAtContact(contact, seg, dir);
3763         if (match === 'strict') {
3764             return resolveBeamColorKey(seg.colorKey);
3765         }
3766         if (match === 'endpoint' && endpointFallback === null) {
3767             endpointFallback = resolveBeamColorKey(seg.colorKey);
3768         }
3769     }
3770     if (endpointFallback !== null) {

```

```

3771         return endpointFallback;
3772     }
3773     // 同位置多光段叠色兜底（十字/共线混色后 contact 仍可能落后）
3774     const atContact: string[] = [];
3775     for (const seg of segments) {
3776         if (
3777             pointsNear(seg.x1, seg.y1, contact.x, contact.y)
3778             || pointsNear(seg.x0, seg.y0, contact.x, contact.y)
3779         ) {
3780             atContact.push(resolveBeamColorKey(seg.colorKey));
3781         }
3782     }
3783     if (atContact.length > 0) {
3784         return mixLightColors(atContact);
3785     }
3786     return coerceBeamColorKey(contact.colorKey);
3787 }
3788 /** 合并光路后，将 blockContacts 色键与对应光段终点对齐 */
3789 export function alignBlockContactsWithSegments(
3790     contacts: readonly OpticalBeamBlockContact[],
3791     segments: readonly OpticalBeamSegment[],
3792 ): OpticalBeamBlockContact[] {
3793     return contacts.map((contact) => ({
3794         ...contact,
3795         colorKey: resolveBlockContactColorFromSegments(contact, segments),
3796     }));
3797 }
3798 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalBeamCycleResolve.ts -----
3799 import type { IOpticalLightSource, IOpticalPiece } from '../Config/OpticalPuzzleLevelSchema';
3800 import type { OpticalBeamSegment } from './OpticalBeamTypes';
3801 import { mixLightColors, type MixedLightColorKey } from './OpticalColorMix';
3802 import { colorModeToKey, resolveBeamColorKey } from './OpticalLightColor';
3803 import { terrainKindAt } from './OpticalRuntimeCoerce';
3804 import {
3805     canEnterPieceWithPropagation,
3806     entrySideToPropagation,
3807     openDirectionsForPiece,
3808     propagationToEntrySide,
3809 } from './OpticalPieceConnectivity';
3810 import type { OpticalBeamTraceInput } from './OpticalBeamTracer';
3811 import { Direction, normalizeDirection, TerrainKind } from './OpticalPuzzleTypes';
3812 const DIR_DX: ReadonlyArray<number> = [1, 0, -1, 0];
3813 const DIR_DY: ReadonlyArray<number> = [0, -1, 0, 1];
3814 const EPS = 1e-4;
3815 export interface BeamCycleGroup {
3816     id: number;
3817     /** 稳态环上的光传播状态 (x,y,dir) */
3818     stateKeys: ReadonlySet<string>;
3819     /** 由状态推导的格（光实际经过） */
3820     cellIndices: ReadonlySet<number>;
3821     /** 参与本稳态环的光源下标（`sources` 数组索引） */
3822     sourceIndices: readonly number[];
3823     /** 环内光源层 3 色 */
3824     sourceColorKeys: readonly string[];
3825     /** 环上滤光元件层 3 色（R/G/B，每格至多一项） */
3826     pieceFilterColorKeys: readonly string[];
3827     /** 组内是否存在 SCC 闭合环（有则地板态也强制稳态色；纯源间连通则否） */
3828     hasClosedCycle: boolean;

```

```

3829 }
3830 export interface BeamCycleContext {
3831     groups: readonly BeamCycleGroup[];
3832     /** 光状态 → 循环组 id (按方向连通, 非按格) */
3833     stateToGroupId: ReadonlyMap<string, number>;
3834     /** 仅稳态环上的光学元件格 → 组 id (后处理着色用) */
3835     cellToGroupId: ReadonlyMap<number, number>;
3836 }
3837 function cellIndex(w: number, x: number, y: number): number {
3838     return y * w + x;
3839 }
3840 function inBounds(x: number, y: number, w: number, h: number): boolean {
3841     return x >= 0 && y >= 0 && x < w && y < h;
3842 }
3843 function isBlockedCell(
3844     x: number,
3845     y: number,
3846     w: number,
3847     terrain: TerrainKind[],
3848     player: { x: number; y: number },
3849 ): boolean {
3850     if (!inBounds(x, y, w, terrain.length / w)) {
3851         return true;
3852     }
3853     if (player.x === x && player.y === y) {
3854         return true;
3855     }
3856     const t = terrainKindAt(terrain, cellIndex(w, x, y));
3857     return t === TerrainKind.Wall || t === TerrainKind.Source || t === TerrainKind.Target;
3858 }
3859 function parseStateKey(sk: string): { x: number; y: number; dir: Direction } {
3860     const parts = sk.split(',');
3861     return { x: Number(parts[0]), y: Number(parts[1]), dir: Number(parts[2]) as Direction };
3862 }
3863 function statesToCellIndices(states: ReadonlySet<string>, w: number): Set<number> {
3864     const cells = new Set<number>();
3865     for (const sk of states) {
3866         const { x, y } = parseStateKey(sk);
3867         cells.add(cellIndex(w, x, y));
3868     }
3869     return cells;
3870 }
3871 function stateKey(x: number, y: number, dir: Direction): string {
3872     return `${x},${y},${dir}`;
3873 }
3874 function transitionsFromState(
3875     x: number,
3876     y: number,
3877     dir: Direction,
3878     w: number,
3879     h: number,
3880     terrain: TerrainKind[],
3881     player: { x: number; y: number },
3882     pieceAt: ReadonlyMap<number, IOpticalPiece>,
3883 ): Array<{ x: number; y: number; dir: Direction }> {
3884     if (isBlockedCell(x, y, w, terrain, player)) {
3885         return [];
3886     }

```

```

3887     const piece = pieceAt.get(cellIndex(w, x, y));
3888     if (!piece || piece.connectivity === 0) {
3889         if (piece) {
3890             return [];
3891         }
3892         const nx = x + DIR_DX[dir];
3893         const ny = y + DIR_DY[dir];
3894         if (!inBounds(nx, ny, w, h) || isBlockedCell(nx, ny, w, terrain, player)) {
3895             return [];
3896         }
3897         const nextPiece = pieceAt.get(cellIndex(w, nx, ny));
3898         if (
3899             nextPiece
3900             && nextPiece.connectivity !== 0
3901             && !canEnterPieceWithPropagation(dir, nextPiece)
3902         ) {
3903             return [];
3904         }
3905         return [{ x: nx, y: ny, dir }];
3906     }
3907     const openDirs = openDirectionsForPiece(piece.connectivity, piece.direction);
3908     const entrySide = propagationToEntrySide(dir);
3909     if (!openDirs.includes(entrySide)) {
3910         return [];
3911     }
3912     const out: Array<{ x: number; y: number; dir: Direction }> = [];
3913     for (const outSide of openDirs) {
3914         if (outSide === entrySide) {
3915             continue;
3916         }
3917         const outDir = entrySideToPropagation(outSide);
3918         const nx = x + DIR_DX[outDir];
3919         const ny = y + DIR_DY[outDir];
3920         if (!inBounds(nx, ny, w, h) || isBlockedCell(nx, ny, w, terrain, player)) {
3921             continue;
3922         }
3923         out.push({ x: nx, y: ny, dir: outDir });
3924     }
3925     return out;
3926 }
3927 function buildTransitionEdges(
3928     w: number,
3929     h: number,
3930     terrain: TerrainKind[],
3931     player: { x: number; y: number },
3932     pieceAt: ReadonlyMap<number, IOpticalPiece>,
3933 ): Map<string, string[]> {
3934     const edges = new Map<string, string[]>();
3935     for (let y = 0; y < h; y++) {
3936         for (let x = 0; x < w; x++) {
3937             if (isBlockedCell(x, y, w, terrain, player)) {
3938                 continue;
3939             }
3940             for (let dir = 0; dir < 4; dir++) {
3941                 const d = dir as Direction;
3942                 const piece = pieceAt.get(cellIndex(w, x, y));
3943                 if (
3944                     piece

```

```

3945         && piece.connectivity !== 0
3946         && !canEnterPieceWithPropagation(d, piece)
3947     ) {
3948         continue;
3949     }
3950     const from = stateKey(x, y, d);
3951     const nexts = transitionsFromState(x, y, d, w, h, terrain, player, pieceAt);
3952     edges.set(from, nexts.map((n) => stateKey(n.x, n.y, n.dir)));
3953 }
3954 }
3955 }
3956 return edges;
3957 }
3958 function buildReverseEdges(forward: ReadonlyMap<string, string[]>): Map<string, string[]> {
3959     const reverse = new Map<string, string[]>();
3960     for (const [from, tos] of forward) {
3961         for (const to of tos) {
3962             let preds = reverse.get(to);
3963             if (!preds) {
3964                 preds = [];
3965                 reverse.set(to, preds);
3966             }
3967             preds.push(from);
3968         }
3969     }
3970     return reverse;
3971 }
3972 /** 从 seed 状态沿正向边 BFS */
3973 function forwardReachableFrom(
3974     seeds: ReadonlySet<string>,
3975     forwardEdges: ReadonlyMap<string, string[]>,
3976 ): Set<string> {
3977     const out = new Set<string>();
3978     const queue: string[] = [];
3979     for (const sk of seeds) {
3980         if (!out.has(sk)) {
3981             out.add(sk);
3982             queue.push(sk);
3983         }
3984     }
3985     while (queue.length > 0) {
3986         const sk = queue.shift()!;
3987         for (const next of forwardEdges.get(sk) ?? []) {
3988             if (!out.has(next)) {
3989                 out.add(next);
3990                 queue.push(next);
3991             }
3992         }
3993     }
3994     return out;
3995 }
3996 function collectSourceEmissionStateKeys(
3997     sources: readonly IOpticalLightSource[],
3998     w: number,
3999     h: number,
4000     terrain: TerrainKind[],
4001     player: { x: number; y: number },
4002 ): Set<string> {

```

```

4003     const out = new Set<string>();
4004     for (let i = 0; i < sources.length; i++) {
4005         const sk = sourceEmissionStateKey(sources[i], i, w, h, terrain, player);
4006         if (sk) {
4007             out.add(sk);
4008         }
4009     }
4010     return out;
4011 }
4012 function collectSourceEmissionStateKeyByIndex(
4013     sources: readonly IOpticalLightSource[],
4014     w: number,
4015     h: number,
4016     terrain: TerrainKind[],
4017     player: { x: number; y: number },
4018 ): Map<number, string> {
4019     const out = new Map<number, string>();
4020     for (let i = 0; i < sources.length; i++) {
4021         const sk = sourceEmissionStateKey(sources[i], i, w, h, terrain, player);
4022         if (sk) {
4023             out.set(i, sk);
4024         }
4025     }
4026     return out;
4027 }
4028 function sourceEmissionStateKey(
4029     src: IOpticalLightSource,
4030     _index: number,
4031     w: number,
4032     h: number,
4033     terrain: TerrainKind[],
4034     player: { x: number; y: number },
4035 ): string | null {
4036     const dir = normalizeDirection(src.direction, Direction.Down);
4037     const startX = src.x + DIR_DX[dir];
4038     const startY = src.y + DIR_DY[dir];
4039     if (!inBounds(startX, startY, w, h) || isBlockedCell(startX, startY, w, terrain, player)) {
4040         return null;
4041     }
4042     return stateKey(startX, startY, dir);
4043 }
4044 /** 在状态子集内找 SCC 闭合环，其状态作为「环锚点」（环外射后须能回到此处） */
4045 function findCycleAnchorStatesInSet(
4046     states: ReadonlySet<string>,
4047     forwardEdges: ReadonlyMap<string, string[]>,
4048 ): Set<string> {
4049     const anchors = new Set<string>();
4050     if (states.size === 0) {
4051         return anchors;
4052     }
4053     const edgesFrom = (v: string): string[] =>
4054         (forwardEdges.get(v) ?? []).filter((to) => states.has(to));
4055     const nodes = [...states];
4056     const index = new Map<string, number>();
4057     const lowlink = new Map<string, number>();
4058     const onStack = new Set<string>();
4059     const stack: string[] = [];
4060     let nextIndex = 0;

```

```

4061     const sccs: string[][] = [];
4062     function strongConnect(v: string): void {
4063         index.set(v, nextIndex);
4064         lowlink.set(v, nextIndex);
4065         nextIndex++;
4066         stack.push(v);
4067         onStack.add(v);
4068         for (const wKey of edgesFrom(v)) {
4069             if (!index.has(wKey)) {
4070                 strongConnect(wKey);
4071                 lowlink.set(v, Math.min(lowlink.get(v)!, lowlink.get(wKey)!));
4072             } else if (onStack.has(wKey)) {
4073                 lowlink.set(v, Math.min(lowlink.get(v)!, index.get(wKey)!));
4074             }
4075         }
4076         if (lowlink.get(v) === index.get(v)) {
4077             const comp: string[] = [];
4078             let wNode: string;
4079             do {
4080                 wNode = stack.pop()!;
4081                 onStack.delete(wNode);
4082                 comp.push(wNode);
4083             } while (wNode !== v);
4084             sccs.push(comp);
4085         }
4086     }
4087     for (const n of nodes) {
4088         if (!index.has(n)) {
4089             strongConnect(n);
4090         }
4091     }
4092     for (const comp of sccs) {
4093         if (comp.length <= 1) {
4094             const only = comp[0];
4095             if (edgesFrom(only).includes(only)) {
4096                 for (const sk of comp) {
4097                     anchors.add(sk);
4098                 }
4099             }
4100         } else if (comp.length >= 3) {
4101             for (const sk of comp) {
4102                 anchors.add(sk);
4103             }
4104         }
4105     }
4106     return anchors;
4107 }
4108 function buildAnchorStatesForBundle(
4109     bundle: SteadyStateBundle,
4110     forwardEdges: ReadonlyMap<string, string[]>,
4111     emissionBySourceIndex: ReadonlyMap<number, string>,
4112 ): Set<string> {
4113     const anchors = new Set<string>();
4114     for (const i of bundle.sourceIndices) {
4115         const es = emissionBySourceIndex.get(i);
4116         if (es) {
4117             anchors.add(es);
4118         }
4119     }

```

```

4119     }
4120     for (const sk of findCycleAnchorStatesInSet(bundle.states, forwardEdges)) {
4121         anchors.add(sk);
4122     }
4123     return anchors;
4124 }
4125 function canReachAnyAnchorState(
4126     fromSk: string,
4127     forwardEdges: ReadonlyMap<string, string[]>,
4128     anchorStates: ReadonlySet<string>,
4129 ): boolean {
4130     if (anchorStates.size === 0) {
4131         return false;
4132     }
4133     const reachable = forwardReachableFrom(new Set([fromSk]), forwardEdges);
4134     for (const anchor of anchorStates) {
4135         if (reachable.has(anchor)) {
4136             return true;
4137         }
4138     }
4139     return false;
4140 }
4141 /**
4142  * 剔除元件格上不能回到稳态锚点的状态（锚点 = 环上 SCC 态 ∪ 组内光源出射态）；地板态保留。
4143  */
4144 function filterStatesByPieceReturnToAnchors(
4145     states: ReadonlySet<string>,
4146     forwardEdges: ReadonlyMap<string, string[]>,
4147     anchorStates: ReadonlySet<string>,
4148     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4149     w: number,
4150 ): Set<string> {
4151     const out = new Set<string>();
4152     for (const sk of states) {
4153         const { x, y, dir } = parseStateKey(sk);
4154         const piece = pieceAt.get(cellIndex(w, x, y));
4155         if (!piece || piece.connectivity === 0) {
4156             out.add(sk);
4157             continue;
4158         }
4159         if (!canEnterPieceWithPropagation(dir, piece)) {
4160             continue;
4161         }
4162         if (canReachAnyAnchorState(sk, forwardEdges, anchorStates)) {
4163             out.add(sk);
4164         }
4165     }
4166     return out;
4167 }
4168 /** 从 seed 状态沿反向边 BFS，得到「能到达 seed 中某一态」的全部状态 */
4169 function reverseReachableFrom(
4170     seeds: ReadonlySet<string>,
4171     reverseEdges: ReadonlyMap<string, string[]>,
4172 ): Set<string> {
4173     const out = new Set<string>();
4174     const queue: string[] = [];
4175     for (const sk of seeds) {
4176         if (!out.has(sk)) {

```



```

4177         out.add(sk);
4178         queue.push(sk);
4179     }
4180 }
4181 while (queue.length > 0) {
4182     const sk = queue.shift();
4183     for (const pred of reverseEdges.get(sk) ?? []) {
4184         if (!out.has(pred)) {
4185             out.add(pred);
4186             queue.push(pred);
4187         }
4188     }
4189 }
4190 return out;
4191 }
4192 /**
4193  * 光源 A 可达态  $\cap$  「能到达光源 B 可达态」 = 位于  $A \rightarrow B$  光路上的状态（含地板与元件）。
4194  * 不含 A 可达但到不了 B 的岔路/末梢。
4195  */
4196 function statesOnPathBetweenReachSets(
4197     reachA: ReadonlySet<string>,
4198     reachB: ReadonlySet<string>,
4199     reverseEdges: ReadonlyMap<string, string[]>,
4200 ): Set<string> {
4201     const canReachB = reverseReachableFrom(reachB, reverseEdges);
4202     const out = new Set<string>();
4203     for (const sk of reachA) {
4204         if (canReachB.has(sk)) {
4205             out.add(sk);
4206         }
4207     }
4208     return out;
4209 }
4210 function findCyclicStateGroups(
4211     forwardEdges: ReadonlyMap<string, string[]>,
4212 ): ReadonlySet<string>[] {
4213     const nodes: string[] = [];
4214     for (const from of forwardEdges.keys()) {
4215         nodes.push(from);
4216     }
4217     const index = new Map<string, number>();
4218     const lowlink = new Map<string, number>();
4219     const onStack = new Set<string>();
4220     const stack: string[] = [];
4221     let nextIndex = 0;
4222     const sccs: string[][] = [];
4223     function strongConnect(v: string): void {
4224         index.set(v, nextIndex);
4225         lowlink.set(v, nextIndex);
4226         nextIndex++;
4227         stack.push(v);
4228         onStack.add(v);
4229         for (const wKey of forwardEdges.get(v) ?? []) {
4230             if (!index.has(wKey)) {
4231                 strongConnect(wKey);
4232                 lowlink.set(v, Math.min(lowlink.get(v)!, lowlink.get(wKey)!));
4233             } else if (onStack.has(wKey)) {
4234                 lowlink.set(v, Math.min(lowlink.get(v)!, index.get(wKey)!));

```

```

4235         }
4236     }
4237     if (lowlink.get(v) === index.get(v)) {
4238         const comp: string[] = [];
4239         let wNode: string;
4240         do {
4241             wNode = stack.pop()!;
4242             onStack.delete(wNode);
4243             comp.push(wNode);
4244         } while (wNode !== v);
4245         sccs.push(comp);
4246     }
4247 }
4248 for (const n of nodes) {
4249     if (!index.has(n)) {
4250         strongConnect(n);
4251     }
4252 }
4253 const cyclicStateSets: ReadonlySet<string>[] = [];
4254 for (const comp of sccs) {
4255     if (comp.length <= 1) {
4256         const only = comp[0];
4257         const outs = forwardEdges.get(only) ?? [];
4258         if (!outs.includes(only)) {
4259             continue;
4260         }
4261     }
4262     if (comp.length >= 3) {
4263         cyclicStateSets.push(new Set(comp));
4264     }
4265 }
4266 return cyclicStateSets;
4267 }
4268 function reachableStatesFromSourceEmission(
4269     src: IOpticalLightSource,
4270     w: number,
4271     h: number,
4272     terrain: TerrainKind[],
4273     player: { x: number; y: number },
4274     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4275 ): Set<string> {
4276     const states = new Set<string>();
4277     const dir = normalizeDirection(src.direction, Direction.Down);
4278     const startX = src.x + DIR_DX[dir];
4279     const startY = src.y + DIR_DY[dir];
4280     if (!inBounds(startX, startY, w, h) || isBlockedCell(startX, startY, w, terrain, player)) {
4281         return states;
4282     }
4283     const visited = new Set<string>();
4284     const queue: Array<{ x: number; y: number; dir: Direction }> = [{ x: startX, y: startY, dir }];
4285     while (queue.length > 0) {
4286         const { x, y, dir: beamDir } = queue.shift()!;
4287         const sk = stateKey(x, y, beamDir);
4288         if (visited.has(sk)) {
4289             continue;
4290         }
4291         visited.add(sk);
4292         states.add(sk);

```

```

4293         const nexts = transitionsFromState(x, y, beamDir, w, h, terrain, player, pieceAt);
4294         for (const n of nexts) {
4295             queue.push(n);
4296         }
4297     }
4298     return states;
4299 }
4300 /** 两束光路是否在光学元件格上以合法入射态交汇（按 connectivity 通道，非按格 footprint） */
4301 function stateSetsConnectOnPieces(
4302     a: ReadonlySet<string>,
4303     b: ReadonlySet<string>,
4304     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4305     w: number,
4306 ): boolean {
4307     const pieceCellsB = new Set<number>();
4308     for (const sk of b) {
4309         const { x, y, dir } = parseStateKey(sk);
4310         const idx = cellIndex(w, x, y);
4311         const piece = pieceAt.get(idx);
4312         if (
4313             piece
4314             && piece.connectivity !== 0
4315             && canEnterPieceWithPropagation(dir, piece)
4316         ) {
4317             pieceCellsB.add(idx);
4318         }
4319     }
4320     for (const sk of a) {
4321         const { x, y, dir } = parseStateKey(sk);
4322         const idx = cellIndex(w, x, y);
4323         const piece = pieceAt.get(idx);
4324         if (
4325             piece
4326             && piece.connectivity !== 0
4327             && pieceCellsB.has(idx)
4328             && canEnterPieceWithPropagation(dir, piece)
4329         ) {
4330             return true;
4331         }
4332     }
4333     return false;
4334 }
4335 interface SteadyStateBundle {
4336     states: Set<string>;
4337     sourceIndices: number[];
4338 }
4339 function sourceIndicesReachingStateSet(
4340     targetStates: ReadonlySet<string>,
4341     w: number,
4342     h: number,
4343     terrain: TerrainKind[],
4344     player: { x: number; y: number },
4345     sources: readonly IOpticalLightSource[],
4346     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4347 ): number[] {
4348     const indices: number[] = [];
4349     for (let i = 0; i < sources.length; i++) {
4350         const reach = reachableStatesFromSourceEmission(

```

```

4351         sources[i],
4352         w,
4353         h,
4354         terrain,
4355         player,
4356         pieceAt,
4357     );
4358     for (const sk of targetStates) {
4359         if (reach.has(sk)) {
4360             indices.push(i);
4361             break;
4362         }
4363     }
4364 }
4365 return indices;
4366 }
4367 function sourceColorKeysForIndices(
4368     sources: readonly IOpticalLightSource[],
4369     indices: readonly number[],
4370 ): string[] {
4371     const keys: string[] = [];
4372     const seen = new Set<string>();
4373     for (const i of indices) {
4374         const key = resolveBeamColorKey(sources[i].colorKey);
4375         if (!seen.has(key)) {
4376             seen.add(key);
4377             keys.push(key);
4378         }
4379     }
4380     return keys;
4381 }
4382 function mergeOverlappingStateSets(
4383     bundles: SteadyStateBundle[],
4384     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4385     w: number,
4386 ): SteadyStateBundle[] {
4387     const merged: SteadyStateBundle[] = bundles.map((b) => ({
4388         states: new Set(b.states),
4389         sourceIndices: [...b.sourceIndices],
4390     }));
4391     let changed = true;
4392     while (changed) {
4393         changed = false;
4394         for (let i = 0; i < merged.length; i++) {
4395             for (let j = i + 1; j < merged.length; j++) {
4396                 const a = merged[i];
4397                 const b = merged[j];
4398                 if (!stateSetsConnectOnPieces(a.states, b.states, pieceAt, w)) {
4399                     continue;
4400                 }
4401                 for (const sk of b.states) {
4402                     a.states.add(sk);
4403                 }
4404                 for (const sid of b.sourceIndices) {
4405                     if (!a.sourceIndices.includes(sid)) {
4406                         a.sourceIndices.push(sid);
4407                     }
4408                 }

```

```

4409         merged.splice(j, 1);
4410         changed = true;
4411         break;
4412     }
4413     if (changed) {
4414         break;
4415     }
4416 }
4417 }
4418 return merged;
4419 }
4420 /**
4421  * 多光源经光学元件连通（共享元件格过光）时视为同一抽象稳态网；
4422  * 状态集为源间光路；元件是否参与稳态在合并后按锚点统一过滤。
4423  * 不含纯地板对射合并（避免无元件交汇时误染整段光路）。
4424  */
4425 function findMultiSourceConnectedStateGroups(
4426     input: OpticalBeamTraceInput,
4427     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4428     forwardEdges: ReadonlyMap<string, string[]>,
4429     reverseEdges: ReadonlyMap<string, string[]>,
4430 ): SteadyStateBundle[] {
4431     const { width: w, height: h, terrain, player, sources } = input;
4432     if (sources.length < 2) {
4433         return [];
4434     }
4435     const reachSets = sources.map((src) =>
4436         reachableStatesFromSourceEmission(src, w, h, terrain, player, pieceAt),
4437     );
4438     const parent = sources.map((_, i) => i);
4439     function find(i: number): number {
4440         while (parent[i] !== i) {
4441             parent[i] = parent[parent[i]];
4442             i = parent[i];
4443         }
4444         return i;
4445     }
4446     function union(a: number, b: number): void {
4447         const ra = find(a);
4448         const rb = find(b);
4449         if (ra !== rb) {
4450             parent[rb] = ra;
4451         }
4452     }
4453     for (let i = 0; i < sources.length; i++) {
4454         for (let j = i + 1; j < sources.length; j++) {
4455             if (stateSetsConnectOnPieces(reachSets[i], reachSets[j], pieceAt, w)) {
4456                 union(i, j);
4457             }
4458         }
4459     }
4460     const clusters = new Map<number, number[]>();
4461     for (let i = 0; i < sources.length; i++) {
4462         const root = find(i);
4463         let list = clusters.get(root);
4464         if (!list) {
4465             list = [];
4466             clusters.set(root, list);

```

```

4467     }
4468     list.push(i);
4469 }
4470 const out: SteadyStateBundle[] = [];
4471 for (const indices of clusters.values()) {
4472     if (indices.length < 2) {
4473         continue;
4474     }
4475     const merged = new Set<string>();
4476     for (let a = 0; a < indices.length; a++) {
4477         for (let b = a + 1; b < indices.length; b++) {
4478             const i = indices[a];
4479             const j = indices[b];
4480             for (const sk of statesOnPathBetweenReachSets(
4481                 reachSets[i],
4482                 reachSets[j],
4483                 reverseEdges,
4484             )) {
4485                 merged.add(sk);
4486             }
4487             for (const sk of statesOnPathBetweenReachSets(
4488                 reachSets[j],
4489                 reachSets[i],
4490                 reverseEdges,
4491             )) {
4492                 merged.add(sk);
4493             }
4494         }
4495     }
4496     if (merged.size === 0) {
4497         continue;
4498     }
4499     out.push({ states: merged, sourceIndices: [...indices] });
4500 }
4501 return out;
4502 }
4503 /** 稳态滤光元件：合法入射且至少一态能回到组内锚点（环 SCC 态 ∪ 光源出射态） */
4504 function coloredFilterKeysOnCycleStates(
4505     stateKeys: ReadonlySet<string>,
4506     w: number,
4507     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4508     forwardEdges?: ReadonlyMap<string, string[]>,
4509     anchorStates?: ReadonlySet<string>,
4510 ): string[] {
4511     const keys: string[] = [];
4512     const seenKeys = new Set<string>();
4513     const seenCells = new Set<number>();
4514     const statesByCell = new Map<number, string[]>();
4515     for (const sk of stateKeys) {
4516         const { x, y, dir } = parseStateKey(sk);
4517         const idx = cellIndex(w, x, y);
4518         const piece = pieceAt.get(idx);
4519         if (!piece || piece.connectivity === 0) {
4520             continue;
4521         }
4522         if (!canEnterPieceWithPropagation(dir, piece)) {
4523             continue;
4524         }

```

```

4525         let list = statesByCell.get(idx);
4526         if (!list) {
4527             list = [];
4528             statesByCell.set(idx, list);
4529         }
4530         list.push(sk);
4531     }
4532     for (const [idx, cellStates] of statesByCell) {
4533         if (seenCells.has(idx)) {
4534             continue;
4535         }
4536         if (
4537             forwardEdges
4538             && anchorStates
4539             && anchorStates.size > 0
4540             && !cellStates.some((sk) => canReachAnyAnchorState(sk, forwardEdges, anchorStates))
4541         ) {
4542             continue;
4543         }
4544         seenCells.add(idx);
4545         const piece = pieceAt.get(idx)!;
4546         const key = colorModeToKey(piece.colorMode);
4547         if (key !== 'white' && !seenKeys.has(key)) {
4548             seenKeys.add(key);
4549             keys.push(key);
4550         }
4551     }
4552     return keys;
4553 }
4554 /**
4555  * 稳态上下文：
4556  * 1. SCC 闭合环；
4557  * 2. 多光源经元件连通；
4558  * 有元件交集可合并。参与稳态的元件须能沿正向光路回到组内锚点（环 SCC 态或光源出射态），
4559  * 环外射、只单向收光且不回落到环/源的元件不计入。
4560  */
4561 export function buildBeamCycleContext(
4562     input: OpticalBeamTraceInput,
4563     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4564 ): BeamCycleContext {
4565     const { width: w, height: h, terrain, player, sources } = input;
4566     const forwardEdges = buildTransitionEdges(w, h, terrain, player, pieceAt);
4567     const reverseEdges = buildReverseEdges(forwardEdges);
4568     const emissionBySourceIndex = collectSourceEmissionStateKeyByIndex(
4569         sources,
4570         w,
4571         h,
4572         terrain,
4573         player,
4574     );
4575     const cyclicStateSets = findCyclicStateGroups(forwardEdges);
4576     const cyclicBundles: SteadyStateBundle[] = cyclicStateSets.map((states) => ({
4577         states: new Set(states),
4578         sourceIndices: sourceIndicesReachingStateSet(
4579             states,
4580             w,
4581             h,
4582             terrain,

```

```

4583         player,
4584         sources,
4585         pieceAt,
4586     ),
4587 });
4588 const connectedBundles = findMultiSourceConnectedStateGroups(
4589     input,
4590     pieceAt,
4591     forwardEdges,
4592     reverseEdges,
4593 );
4594 const mergedBundles = mergeOverlappingStateSets(
4595     [...cyclicBundles, ...connectedBundles],
4596     pieceAt,
4597     w,
4598 );
4599 const filteredBundles: SteadyStateBundle[] = [];
4600 for (const bundle of mergedBundles) {
4601     const anchors = buildAnchorStatesForBundle(bundle, forwardEdges, emissionBySourceIndex);
4602     const filtered = filterStatesByPieceReturnToAnchors(
4603         bundle.states,
4604         forwardEdges,
4605         anchors,
4606         pieceAt,
4607         w,
4608     );
4609     if (filtered.size === 0) {
4610         continue;
4611     }
4612     filteredBundles.push({ states: filtered, sourceIndices: [...bundle.sourceIndices] });
4613 }
4614 const groups: BeamCycleGroup[] = filteredBundles.map((bundle, id) => {
4615     const stateKeys = bundle.states;
4616     const cellIndices = statesToCellIndices(stateKeys, w);
4617     const sourceIndices = bundle.sourceIndices;
4618     const anchorStates = buildAnchorStatesForBundle(bundle, forwardEdges, emissionBySourceIndex);
4619     const hasClosedCycle = findCycleAnchorStatesInSet(stateKeys, forwardEdges).size > 0;
4620     return {
4621         id,
4622         stateKeys,
4623         cellIndices,
4624         sourceIndices,
4625         sourceColorKeys: sourceColorKeysForIndices(sources, sourceIndices),
4626         pieceFilterColorKeys: coloredFilterKeysOnCycleStates(
4627             stateKeys,
4628             w,
4629             pieceAt,
4630             forwardEdges,
4631             anchorStates,
4632         ),
4633         hasClosedCycle,
4634     };
4635 });
4636 const stateToGroupId = new Map<string, number>();
4637 const cellToGroupId = new Map<number, number>();
4638 for (const g of groups) {
4639     for (const sk of g.stateKeys) {
4640         stateToGroupId.set(sk, g.id);

```



```

4641         const { x, y } = parseStateKey(sk);
4642         const idx = cellIndex(w, x, y);
4643         const piece = pieceAt.get(idx);
4644         if (piece !== null && piece.connectivity !== 0) {
4645             cellToGroupId.set(idx, g.id);
4646         }
4647     }
4648 }
4649 return { groups, stateToGroupId, cellToGroupId };
4650 }
4651 export function createCycleIncomingMap(ctx: BeamCycleContext): Map<number, string[]> {
4652     const incoming = new Map<number, string[]>();
4653     for (const g of ctx.groups) {
4654         incoming.set(g.id, []);
4655     }
4656     return incoming;
4657 }
4658 /**
4659  * 记录射入循环光路的外来光（在元件混色之前、从环外格进入时调用）。
4660  * 每束入射单独 push，供后续与环上滤光元件色一起累加混色。
4661  */
4662 export function recordCycleIncomingBeam(
4663     cx: number,
4664     cy: number,
4665     dir: Direction,
4666     colorKey: string,
4667     ctx: BeamCycleContext,
4668     w: number,
4669     incoming: Map<number, string[]>,
4670     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4671     sourceIds: ReadonlySet<number>,
4672 ): void {
4673     const sk = stateKey(cx, cy, dir);
4674     const gid = ctx.stateToGroupId.get(sk);
4675     if (gid === undefined) {
4676         return;
4677     }
4678     const px = cx - DIR_DX[dir];
4679     const py = cy - DIR_DY[dir];
4680     if (ctx.stateToGroupId.has(stateKey(px, py, dir))) {
4681         return;
4682     }
4683     const group = ctx.groups.find((g) => g.id === gid);
4684     if (!group) {
4685         return;
4686     }
4687     // 本环光源射入已计入 sourceColorKeys，不算环外入射
4688     if (beamBelongsToSteadyGroup(group, sourceIds)) {
4689         return;
4690     }
4691     const piece = pieceAt.get(cellIndex(w, cx, cy));
4692     const isPieceCell = piece !== null && piece.connectivity !== 0;
4693     // 异源光仅垂直/水平穿过环内地板不算环外入射（虚空十字交叉）
4694     if (!isPieceCell && !beamBelongsToSteadyGroup(group, sourceIds)) {
4695         return;
4696     }
4697     incoming.get(gid)!.push(resolveBeamColorKey(colorKey));
4698 }

```

```

4699  /** 组内光源色 + 环外入射 + 滤光元件色 → 稳态色 */
4700  export function computeCycleUniformColors(
4701      ctx: BeamCycleContext,
4702      incomingByGroup: ReadonlyMap<number, string[]>,
4703  ): Map<number, MixedLightColorKey> {
4704      const out = new Map<number, MixedLightColorKey>();
4705      for (const g of ctx.groups) {
4706          const inputs = incomingByGroup.get(g.id) ?? [];
4707          const filterExtras = g.pieceFilterColorKeys.filter(
4708              (fk) => !g.sourceColorKeys.includes(fk),
4709          );
4710          const mixKeys = [...g.sourceColorKeys, ...inputs, ...filterExtras];
4711          out.set(g.id, mixKeys.length > 0 ? mixLightColors(mixKeys) : 'white');
4712      }
4713      return out;
4714  }
4715  /** 稳态环内传播/着色用的统一色；纯源间连通时仅强制稳态网络元件，地板保留滤光结果 */
4716  export function steadyColorAtCell(
4717      cx: number,
4718      cy: number,
4719      dir: Direction,
4720      w: number,
4721      colorKey: string,
4722      sourceIds: ReadonlySet<number>,
4723      ctx: BeamCycleContext,
4724      steadyByGroup?: ReadonlyMap<number, MixedLightColorKey>,
4725  ): string {
4726      if (!steadyByGroup) {
4727          return colorKey;
4728      }
4729      const sk = stateKey(cx, cy, dir);
4730      const gid = ctx.stateToGroupId.get(sk);
4731      if (gid === undefined) {
4732          return colorKey;
4733      }
4734      const group = ctx.groups.find((g) => g.id === gid);
4735      if (!group || !beamBelongsToSteadyGroup(group, sourceIds)) {
4736          return colorKey;
4737      }
4738      const onSteadyPiece = ctx.cellToGroupId.has(cellIndex(w, cx, cy));
4739      if (!onSteadyPiece && !group.hasClosedCycle) {
4740          return colorKey;
4741      }
4742      return steadyByGroup.get(gid) ?? colorKey;
4743  }
4744  /** 稳态环内光源的出射初始色（非环内光源保持本色） */
4745  export function steadyEmissionColorForSource(
4746      src: IOpticalLightSource,
4747      sourceIndex: number,
4748      w: number,
4749      ctx: BeamCycleContext,
4750      steadyByGroup?: ReadonlyMap<number, MixedLightColorKey>,
4751  ): string {
4752      const base = resolveBeamColorKey(src.colorKey);
4753      if (!steadyByGroup) {
4754          return base;
4755      }
4756      for (const g of ctx.groups) {

```

```

4757         if (g.sourceIndices.some((sid) => Number(sid) === Number(sourceIndex))) {
4758             const steady = steadyByGroup.get(g.id);
4759             if (steady !== undefined) {
4760                 return steady;
4761             }
4762         }
4763     }
4764     return base;
4765 }
4766 function beamBelongsToSteadyGroup(group: BeamCycleGroup, sourceIds: ReadonlySet<number>): boolean {
4767     for (const sid of Array.from(sourceIds)) {
4768         if (group.sourceIndices.includes(Number(sid))) {
4769             return true;
4770         }
4771     }
4772     return false;
4773 }
4774 function cellsOverlap1D(lo: number, hi: number, cellStart: number): boolean {
4775     const cellLo = cellStart;
4776     const cellHi = cellStart + 1;
4777     return Math.max(lo, cellLo) < Math.min(hi, cellHi) - EPS;
4778 }
4779 /** 光段穿过的格：水平段取 floor(y)，垂直段取 floor(x) */
4780 function cellsUnderSegment(seg: OpticalBeamSegment): Array<{ x: number; y: number }> {
4781     const dx = seg.x1 - seg.x0;
4782     const dy = seg.y1 - seg.y0;
4783     const adx = Math.abs(dx);
4784     const ady = Math.abs(dy);
4785     if (adx >= ady) {
4786         const y = Math.floor((seg.y0 + seg.y1) * 0.5);
4787         const lo = Math.min(seg.x0, seg.x1);
4788         const hi = Math.max(seg.x0, seg.x1);
4789         const cells: Array<{ x: number; y: number }> = [];
4790         const cxStart = Math.floor(lo);
4791         const cxEnd = Math.ceil(hi) - 1;
4792         for (let x = cxStart; x <= cxEnd; x++) {
4793             if (cellsOverlap1D(lo, hi, x)) {
4794                 cells.push({ x, y });
4795             }
4796         }
4797         return cells;
4798     }
4799     const x = Math.floor((seg.x0 + seg.x1) * 0.5);
4800     const lo = Math.min(seg.y0, seg.y1);
4801     const hi = Math.max(seg.y0, seg.y1);
4802     const cells: Array<{ x: number; y: number }> = [];
4803     const cyStart = Math.floor(lo);
4804     const cyEnd = Math.ceil(hi) - 1;
4805     for (let y = cyStart; y <= cyEnd; y++) {
4806         if (cellsOverlap1D(lo, hi, y)) {
4807             cells.push({ x, y });
4808         }
4809     }
4810     return cells;
4811 }
4812 function clipSegmentToCellSpan(
4813     seg: OpticalBeamSegment,
4814     minCellX: number,

```

```

4815     maxCellX: number,
4816     minCellY: number,
4817     maxCellY: number,
4818 ): OpticalBeamSegment | null {
4819     const dx = seg.x1 - seg.x0;
4820     const dy = seg.y1 - seg.y0;
4821     const adx = Math.abs(dx);
4822     const ady = Math.abs(dy);
4823     if (adx >= ady) {
4824         const y = seg.y0;
4825         const lo = Math.min(seg.x0, seg.x1);
4826         const hi = Math.max(seg.x0, seg.x1);
4827         const clipLo = Math.max(lo, minCellX);
4828         const clipHi = Math.min(hi, maxCellX + 1);
4829         if (clipHi - clipLo < EPS) {
4830             return null;
4831         }
4832         if (seg.x0 <= seg.x1) {
4833             return { x0: clipLo, y0: y, x1: clipHi, y1: y, colorKey: seg.colorKey };
4834         }
4835         return { x0: clipHi, y0: y, x1: clipLo, y1: y, colorKey: seg.colorKey };
4836     }
4837     const x = seg.x0;
4838     const lo = Math.min(seg.y0, seg.y1);
4839     const hi = Math.max(seg.y0, seg.y1);
4840     const clipLo = Math.max(lo, minCellY);
4841     const clipHi = Math.min(hi, maxCellY + 1);
4842     if (clipHi - clipLo < EPS) {
4843         return null;
4844     }
4845     if (seg.y0 <= seg.y1) {
4846         return { x0: x, y0: clipLo, x1: x, y1: clipHi, colorKey: seg.colorKey };
4847     }
4848     return { x0: x, y0: clipHi, x1: x, y1: clipLo, colorKey: seg.colorKey };
4849 }
4850 function pieceGroupIdAtCell(
4851     x: number,
4852     y: number,
4853     w: number,
4854     ctx: BeamCycleContext,
4855     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4856 ): number | undefined {
4857     const idx = cellIndex(w, x, y);
4858     const piece = pieceAt.get(idx);
4859     if (!piece || piece.connectivity === 0) {
4860         return undefined;
4861     }
4862     return ctx.cellToGroupId.get(idx);
4863 }
4864 function splitSegmentForCycleColor(
4865     seg: OpticalBeamSegment,
4866     ctx: BeamCycleContext,
4867     colorsByGroup: ReadonlyMap<number, MixedLightColorKey>,
4868     w: number,
4869     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4870 ): OpticalBeamSegment[] {
4871     const cells = cellsUnderSegment(seg);
4872     if (cells.length === 0) {

```

```

4873         return [{ ...seg }];
4874     }
4875     const out: OpticalBeamSegment[] = [];
4876     let runGid: number | undefined | null = null;
4877     let runMinX = 0;
4878     let runMaxX = 0;
4879     let runMinY = 0;
4880     let runMaxY = 0;
4881     const flushRun = (): void => {
4882         if (runGid === null) {
4883             return;
4884         }
4885         const clipped = clipSegmentToCellSpan(seg, runMinX, runMaxX, runMinY, runMaxY);
4886         if (!clipped) {
4887             runGid = null;
4888             return;
4889         }
4890         if (runGid === undefined) {
4891             out.push({ ...clipped });
4892         } else {
4893             const colorKey = colorsByGroup.get(runGid) ?? clipped.colorKey;
4894             out.push({ ...clipped, colorKey });
4895         }
4896         runGid = null;
4897     };
4898     for (const c of cells) {
4899         const gid = pieceGroupIdAtCell(c.x, c.y, w, ctx, pieceAt);
4900         if (runGid === null) {
4901             runGid = gid;
4902             runMinX = c.x;
4903             runMaxX = c.x;
4904             runMinY = c.y;
4905             runMaxY = c.y;
4906             continue;
4907         }
4908         if (gid === runGid) {
4909             runMinX = Math.min(runMinX, c.x);
4910             runMaxX = Math.max(runMaxX, c.x);
4911             runMinY = Math.min(runMinY, c.y);
4912             runMaxY = Math.max(runMaxY, c.y);
4913             continue;
4914         }
4915         flushRun();
4916         runGid = gid;
4917         runMinX = c.x;
4918         runMaxX = c.x;
4919         runMinY = c.y;
4920         runMaxY = c.y;
4921     }
4922     flushRun();
4923     if (out.length === 0) {
4924         return [{ ...seg }];
4925     }
4926     return out;
4927 }
4928 /** 将循环光路覆盖段统一为计算色（仅元件格；地板虚空交叉保留原色） */
4929 export function applyCycleUniformColors(
4930     segments: readonly OpticalBeamSegment[],

```

```

4931     ctx: BeamCycleContext,
4932     colorsByGroup: ReadonlyMap<number, MixedLightColorKey>,
4933     w: number,
4934     pieceAt: ReadonlyMap<number, IOpticalPiece>,
4935 ): OpticalBeamSegment[] {
4936     if (ctx.groups.length === 0) {
4937         return segments.map((s) => ({ ...s }));
4938     }
4939     const out: OpticalBeamSegment[] = [];
4940     for (const seg of segments) {
4941         out.push(...splitSegmentForCycleColor(seg, ctx, colorsByGroup, w, pieceAt));
4942     }
4943     return out;
4944 }
4945 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalBeamOverlapMerge.ts -----
4946 import type { IOpticalTarget } from '../Config/OpticalPuzzleLevelSchema';
4947 import { mixLightColors } from './OpticalColorMix';
4948 import type { OpticalBeamSegment } from './OpticalBeamTypes';
4949 import { lightMatchesTarget } from './OpticalLightColor';
4950 const EPS = 1e-3;
4951 interface AxisInterval {
4952     axis: 'h' | 'v';
4953     /** 水平段固定 y; 垂直段固定 x */
4954     fixed: number;
4955     start: number;
4956     end: number;
4957     colorKey: string;
4958 }
4959 /** 十字交叉叠色: 仅交点处窄条混色 (由表现层按光路宽度绘制) */
4960 export interface OpticalCrossBeamOverlay {
4961     x: number;
4962     y: number;
4963     /** 玩法混色结果 */
4964     colorKey: string;
4965     /** 参与叠色的各光路本色 (供分层渐变混绘) */
4966     colorKeys: string[];
4967 }
4968 function toAxisInterval(seg: OpticalBeamSegment): AxisInterval | null {
4969     const dx = seg.x1 - seg.x0;
4970     const dy = seg.y1 - seg.y0;
4971     const adx = Math.abs(dx);
4972     const ady = Math.abs(dy);
4973     const span = Math.max(adx, ady);
4974     if (span < EPS) {
4975         return null;
4976     }
4977     // 主方向归类: 避免微信等环境 dy 微非零导致无法进 bucket、sweep 后又丢段
4978     if (adx >= ady) {
4979         return {
4980             axis: 'h',
4981             fixed: (seg.y0 + seg.y1) * 0.5,
4982             start: Math.min(seg.x0, seg.x1),
4983             end: Math.max(seg.x0, seg.x1),
4984             colorKey: seg.colorKey,
4985         };
4986     }
4987     return {
4988         axis: 'v',

```

```

4989         fixed: (seg.x0 + seg.x1) * 0.5,
4990         start: Math.min(seg.y0, seg.y1),
4991         end: Math.max(seg.y0, seg.y1),
4992         colorKey: seg.colorKey,
4993     };
4994 }
4995 function axisIntervalToSegment(interval: AxisInterval): OpticalBeamSegment {
4996     if (interval.axis === 'h') {
4997         return {
4998             x0: interval.start,
4999             y0: interval.fixed,
5000             x1: interval.end,
5001             y1: interval.fixed,
5002             colorKey: interval.colorKey,
5003         };
5004     }
5005     return {
5006         x0: interval.fixed,
5007         y0: interval.start,
5008         x1: interval.fixed,
5009         y1: interval.end,
5010         colorKey: interval.colorKey,
5011     };
5012 }
5013 function sweepMergeIntervals(list: readonly AxisInterval[]): AxisInterval[] {
5014     if (list.length === 0) {
5015         return [];
5016     }
5017     const points = new Set<number>();
5018     for (const s of list) {
5019         points.add(s.start);
5020         points.add(s.end);
5021     }
5022     const sorted = Array.from(points).sort((a, b) => a - b);
5023     const out: AxisInterval[] = [];
5024     for (let i = 0; i < sorted.length - 1; i++) {
5025         const a = sorted[i];
5026         const b = sorted[i + 1];
5027         if (b - a < EPS) {
5028             continue;
5029         }
5030         const mid = (a + b) * 0.5;
5031         const colors: string[] = [];
5032         for (const s of list) {
5033             if (mid >= s.start - EPS && mid <= s.end + EPS) {
5034                 colors.push(s.colorKey);
5035             }
5036         }
5037         if (colors.length === 0) {
5038             continue;
5039         }
5040         out.push({
5041             axis: list[0].axis,
5042             fixed: list[0].fixed,
5043             start: a,
5044             end: b,
5045             colorKey: mixLightColors(colors),
5046         });

```

```

5047     }
5048     const merged: AxisInterval[] = [];
5049     for (const seg of out) {
5050         const last = merged[merged.length - 1];
5051         if (last && last.colorKey === seg.colorKey && Math.abs(last.end - seg.start) < EPS) {
5052             last.end = seg.end;
5053         } else {
5054             merged.push({ ...seg });
5055         }
5056     }
5057     if (merged.length === 0 && list.length > 0) {
5058         return list.map((s) => ({ ...s }));
5059     }
5060     return merged;
5061 }
5062 function isHorizontal(seg: OpticalBeamSegment): boolean {
5063     return Math.abs(seg.y1 - seg.y0) < EPS && Math.abs(seg.x1 - seg.x0) > EPS;
5064 }
5065 function isVertical(seg: OpticalBeamSegment): boolean {
5066     return Math.abs(seg.x1 - seg.x0) < EPS && Math.abs(seg.y1 - seg.y0) > EPS;
5067 }
5068 function pointOnSegment(x: number, y: number, seg: OpticalBeamSegment): boolean {
5069     if (isHorizontal(seg)) {
5070         const lo = Math.min(seg.x0, seg.x1);
5071         const hi = Math.max(seg.x0, seg.x1);
5072         return Math.abs(y - seg.y0) < EPS && x >= lo - EPS && x <= hi + EPS;
5073     }
5074     if (isVertical(seg)) {
5075         const lo = Math.min(seg.y0, seg.y1);
5076         const hi = Math.max(seg.y0, seg.y1);
5077         return Math.abs(x - seg.x0) < EPS && y >= lo - EPS && y <= hi + EPS;
5078     }
5079     return false;
5080 }
5081 function crossPointKey(x: number, y: number): string {
5082     return `${x.toFixed(4)},${y.toFixed(4)}`;
5083 }
5084 /**
5085  * 横竖光路交点：仅当经过该点的光色不止一种时返回叠色（供表现层画窄条）。
5086  */
5087 export function computeCrossBeamOverlays(segments: readonly OpticalBeamSegment[]):
5088     OpticalCrossBeamOverlay[] {
5089     const colorSets = new Map<string, Set<string>>();
5090     const points = new Map<string, { x: number; y: number }>();
5091     const horizontals = segments.filter(isHorizontal);
5092     const verticals = segments.filter(isVertical);
5093     for (const h of horizontals) {
5094         for (const v of verticals) {
5095             const x = v.x0;
5096             const y = h.y0;
5097             if (!pointOnSegment(x, y, h) || !pointOnSegment(x, y, v)) {
5098                 continue;
5099             }
5100             const key = crossPointKey(x, y);
5101             points.set(key, { x, y });
5102             let set = colorSets.get(key);
5103             if (!set) {
5104                 set = new Set<string>();

```



```

5105         colorSets.set(key, set);
5106     }
5107     set.add(h.colorKey);
5108     set.add(v.colorKey);
5109 }
5110 }
5111 const overlays: OpticalCrossBeamOverlay[] = [];
5112 for (const [key, colors] of Array.from(colorSets.entries())) {
5113     if (colors.size < 2) {
5114         continue;
5115     }
5116     const pt = points.get(key);
5117     if (!pt) {
5118         continue;
5119     }
5120     const colorList = Array.from(colors);
5121     overlays.push({
5122         x: pt.x,
5123         y: pt.y,
5124         colorKey: mixLightColors(colorList),
5125         colorKeys: colorList,
5126     });
5127 }
5128 return overlays;
5129 }
5130 function pointsNear(ax: number, ay: number, bx: number, by: number): boolean {
5131     return Math.abs(ax - bx) < EPS && Math.abs(ay - by) < EPS;
5132 }
5133 function segmentUnitDir(seg: OpticalBeamSegment): { dx: number; dy: number } | null {
5134     const dx = seg.x1 - seg.x0;
5135     const dy = seg.y1 - seg.y0;
5136     const len = Math.hypot(dx, dy);
5137     if (len < EPS) {
5138         return null;
5139     }
5140     return { dx: dx / len, dy: dy / len };
5141 }
5142 function dirsParallel(
5143     a: { dx: number; dy: number },
5144     b: { dx: number; dy: number },
5145 ): boolean {
5146     return Math.abs(a.dx - b.dx) < EPS && Math.abs(a.dy - b.dy) < EPS;
5147 }
5148 /** 合并同色、共线、首尾相接的片段 */
5149 export function mergeCollinearBeamSegments(segments: readonly OpticalBeamSegment[]):
5150 OpticalBeamSegment[] {
5151     let list = segments.map((s) => ({ ...s }));
5152     let changed = true;
5153     while (changed) {
5154         changed = false;
5155         outer: for (let i = 0; i < list.length; i++) {
5156             const a = list[i];
5157             const da = segmentUnitDir(a);
5158             if (!da) {
5159                 continue;
5160             }
5161             for (let j = i + 1; j < list.length; j++) {
5162                 const b = list[j];

```

```

5163         if (a.colorKey !== b.colorKey) {
5164             continue;
5165         }
5166         const db = segmentUnitDir(b);
5167         if (!db) {
5168             continue;
5169         }
5170         if (pointsNear(a.x1, a.y1, b.x0, b.y0) && dirsParallel(da, db)) {
5171             list[i] = { ...a, x1: b.x1, y1: b.y1 };
5172             list.splice(j, 1);
5173             changed = true;
5174             break outer;
5175         }
5176         if (pointsNear(a.x1, a.y1, b.x1, b.y1) && dirsParallel(da, { dx: -db.dx, dy: -db.dy })) {
5177             list[i] = { ...a, x1: b.x0, y1: b.y0 };
5178             list.splice(j, 1);
5179             changed = true;
5180             break outer;
5181         }
5182         if (pointsNear(a.x0, a.y0, b.x0, b.y0) && dirsParallel(da, { dx: -db.dx, dy: -db.dy })) {
5183             list[i] = { x0: a.x1, y0: a.y1, x1: b.x1, y1: b.y1, colorKey: a.colorKey };
5184             list.splice(j, 1);
5185             changed = true;
5186             break outer;
5187         }
5188         if (pointsNear(a.x0, a.y0, b.x1, b.y1) && dirsParallel(da, db)) {
5189             list[i] = { x0: b.x0, y0: b.y0, x1: a.x1, y1: a.y1, colorKey: a.colorKey };
5190             list.splice(j, 1);
5191             changed = true;
5192             break outer;
5193         }
5194     }
5195 }
5196 }
5197 return list;
5198 }
5199 /**
5200  * 共线重叠区间混色（同轴光路叠加）；十字交叉不在此改色，由表现层在交点窄条叠绘。
5201  */
5202 export function mergeOverlappingBeamSegments(segments: readonly OpticalBeamSegment[]):
5203 OpticalBeamSegment[] {
5204     const axisBuckets = new Map<string, AxisInterval[]>();
5205     const passthrough: OpticalBeamSegment[] = [];
5206     for (const seg of segments) {
5207         const axis = toAxisInterval(seg);
5208         if (!axis) {
5209             passthrough.push({ ...seg });
5210             continue;
5211         }
5212         const key = `${axis.axis}:${axis.fixed.toFixed(4)}`;
5213         const bucket = axisBuckets.get(key);
5214         if (bucket) {
5215             bucket.push(axis);
5216         } else {
5217             axisBuckets.set(key, [axis]);
5218         }
5219     }
5220     const merged: OpticalBeamSegment[] = [...passthrough];

```

```

5221     for (const bucket of Array.from(axisBuckets.values())) {
5222         for (const interval of sweepMergeIntervals(bucket)) {
5223             merged.push(axisIntervalToSegment(interval));
5224         }
5225     }
5226     const collinear = mergeCollinearBeamSegments(merged);
5227     if (collinear.length === 0 && segments.length > 0) {
5228         return mergeCollinearBeamSegments(segments.map((s) => ({ ...s })));
5229     }
5230     return collinear;
5231 }
5232 function pointOnTargetEdge(x: number, y: number, tx: number, ty: number): boolean {
5233     const onWest = Math.abs(x - tx) < EPS && y >= ty - EPS && y <= ty + 1 + EPS;
5234     const onEast = Math.abs(x - (tx + 1)) < EPS && y >= ty - EPS && y <= ty + 1 + EPS;
5235     const onNorth = Math.abs(y - ty) < EPS && x >= tx - EPS && x <= tx + 1 + EPS;
5236     const onSouth = Math.abs(y - (ty + 1)) < EPS && x >= tx - EPS && x <= tx + 1 + EPS;
5237     return onWest || onEast || onNorth || onSouth;
5238 }
5239 function segmentEndsOnTargetEdge(seg: OpticalBeamSegment, tx: number, ty: number): boolean {
5240     return (
5241         pointOnTargetEdge(seg.x0, seg.y0, tx, ty) ||
5242         pointOnTargetEdge(seg.x1, seg.y1, tx, ty)
5243     );
5244 }
5245 /** 按合并后的光段重算目标点亮（重叠光路先混色再判定） */
5246 export function recomputeTargetLitFromSegments(
5247     segments: readonly OpticalBeamSegment[],
5248     targets: readonly IOpticalTarget[],
5249 ): boolean[] {
5250     const colorsAtTarget: string[][] = targets.map(() => []);
5251     for (const seg of segments) {
5252         for (let i = 0; i < targets.length; i++) {
5253             const t = targets[i];
5254             if (segmentEndsOnTargetEdge(seg, t.x, t.y)) {
5255                 colorsAtTarget[i].push(seg.colorKey);
5256             }
5257         }
5258     }
5259     return targets.map((t, i) => {
5260         const colors = colorsAtTarget[i];
5261         if (colors.length === 0) {
5262             return false;
5263         }
5264         return lightMatchesTarget(mixLightColors(colors), t.colorKey);
5265     });
5266 }
5267 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalBeamTracer.ts -----
5268 import type { IOpticalLightSource, IOpticalPiece, IOpticalTarget } from '../Config/OpticalPuzzleLevelSchema';
5269 import {
5270     mergeOverlappingBeamSegments,
5271     recomputeTargetLitFromSegments,
5272 } from './OpticalBeamOverlapMerge';
5273 import { alignBlockContactsWithSegments } from './OpticalBeamContactResolve';
5274 import type { OpticalBeamBlockContact, OpticalBeamSegment } from './OpticalBeamTypes';
5275 export type { OpticalBeamBlockContact, OpticalBeamSegment } from './OpticalBeamTypes';
5276 import {
5277     coerceBeamColorKey,
5278     collectColorKeyStrings,

```

```

5279     mixLightColors,
5280     type MixedLightColorKey,
5281 } from './OpticalColorMix';
5282 import type { BeamCycleContext } from './OpticalBeamCycleResolve';
5283 import {
5284     applyCycleUniformColors,
5285     buildBeamCycleContext,
5286     computeCycleUniformColors,
5287     createCycleIncomingMap,
5288     recordCycleIncomingBeam,
5289     steadyColorAtCell,
5290     steadyEmissionColorForSource,
5291 } from './OpticalBeamCycleResolve';
5292 import { colorModeToKey, lightMatchesTarget, resolveBeamColorKey } from './OpticalLightColor';
5293 import type { PieceConnectivity } from './OpticalPieceConnectivity';
5294 import {
5295     entrySideToPropagation,
5296     openDirectionsForPiece,
5297     propagationToEntrySide,
5298 } from './OpticalPieceConnectivity';
5299 import {
5300     normalizeGridCoord,
5301     normalizeGridSize,
5302     normalizePieceConnectivity,
5303     normalizeTerrainKind,
5304     opticalDirDelta,
5305     terrainKindAt,
5306 } from './OpticalRuntimeCoerce';
5307 import { Direction, normalizeDirection, TerrainKind } from './OpticalPuzzleTypes';
5308 const DIR_DX: ReadonlyArray<number> = [1, 0, -1, 0];
5309 const DIR_DY: ReadonlyArray<number> = [0, -1, 0, 1];
5310 export interface OpticalBeamTraceInput {
5311     width: number;
5312     height: number;
5313     terrain: TerrainKind[];
5314     player: { x: number; y: number };
5315     pieces: readonly IOpticalPiece[];
5316     sources: readonly IOpticalLightSource[];
5317     targets: readonly IOpticalTarget[];
5318 }
5319 export interface OpticalBeamTraceResult {
5320     segments: OpticalBeamSegment[];
5321     blockContacts: OpticalBeamBlockContact[];
5322     targetLit: boolean[];
5323 }
5324 interface BeamRay {
5325     ax: number;
5326     ay: number;
5327     cx: number;
5328     cy: number;
5329     dir: Direction;
5330     colorKey: string;
5331     /** 该射线归属的光源下标（`sources` 数组索引） */
5332     sourceIds: Set<number>;
5333 }
5334 function copySourceIds(sourceIds: ReadonlySet<number>): Set<number> {
5335     return new Set(sourceIds);
5336 }

```

```
5337 function mergeSourceIds(target: Set<number>, from: ReadonlySet<number>): void {
5338     for (const id of from) {
5339         target.add(id);
5340     }
5341 }
5342 function cellIndex(w: number, x: number, y: number): number {
5343     return y * w + x;
5344 }
5345 function inBounds(x: number, y: number, w: number, h: number): boolean {
5346     return x >= 0 && y >= 0 && x < w && y < h;
5347 }
5348 function isWall(x: number, y: number, w: number, terrain: TerrainKind[]): boolean {
5349     return terrainKindAt(terrain, cellIndex(w, x, y)) === TerrainKind.Wall;
5350 }
5351 function isPlayerCell(x: number, y: number, player: { x: number; y: number }): boolean {
5352     return player.x === x && player.y === y;
5353 }
5354 function pushSegment(
5355     segments: OpticalBeamSegment[],
5356     x0: number,
5357     y0: number,
5358     x1: number,
5359     y1: number,
5360     colorKey: string,
5361 ): void {
5362     const eps = 1e-4;
5363     if (Math.abs(x0 - x1) < eps && Math.abs(y0 - y1) < eps) {
5364         return;
5365     }
5366     segments.push({ x0, y0, x1, y1, colorKey });
5367 }
5368 function blockContactAtCellFace(
5369     cx: number,
5370     cy: number,
5371     dir: Direction,
5372 ): { x: number; y: number } {
5373     return {
5374         x: cx + 0.5 - DIR_DX[dir] * 0.5,
5375         y: cy + 0.5 - DIR_DY[dir] * 0.5,
5376     };
5377 }
5378 /** 光线在地图边界被截断时的接触点（当前格朝传播方向的外侧面） */
5379 function blockContactAtMapEdge(
5380     cx: number,
5381     cy: number,
5382     dir: Direction,
5383 ): { x: number; y: number } {
5384     return {
5385         x: cx + 0.5 + DIR_DX[dir] * 0.5,
5386         y: cy + 0.5 + DIR_DY[dir] * 0.5,
5387     };
5388 }
5389 function pushBlockContact(
5390     contacts: OpticalBeamBlockContact[],
5391     cx: number,
5392     cy: number,
5393     dir: Direction,
5394     colorKey: string,
```

```

5395         atMapEdge = false,
5396     ): void {
5397         const { x, y } = atMapEdge
5398             ? blockContactAtMapEdge(cx, cy, dir)
5399             : blockContactAtCellFace(cx, cy, dir);
5400         contacts.push({ x, y, dir, colorKey });
5401     }
5402     function dedupeBlockContacts(contacts: readonly OpticalBeamBlockContact[]): OpticalBeamBlockContact[] {
5403         const grouped = new Map<string, { x: number; y: number; dir: Direction; colorKeys: Set<string> }>();
5404         for (const contact of contacts) {
5405             const key = `${Math.round(contact.x * 2000)},${Math.round(contact.y * 2000)},${contact.dir}`;
5406             let entry = grouped.get(key);
5407             if (!entry) {
5408                 entry = { x: contact.x, y: contact.y, dir: contact.dir as Direction, colorKeys: new Set() };
5409                 grouped.set(key, entry);
5410             }
5411             for (const colorKey of collectColorKeyStrings(contact.colorKey)) {
5412                 entry.colorKeys.add(resolveBeamColorKey(colorKey));
5413             }
5414         }
5415         const out: OpticalBeamBlockContact[] = [];
5416         for (const entry of grouped.values()) {
5417             const keys = [...entry.colorKeys];
5418             out.push({
5419                 x: entry.x,
5420                 y: entry.y,
5421                 dir: entry.dir,
5422                 colorKey: coerceBeamColorKey(keys),
5423             });
5424         }
5425         return out;
5426     }
5427     function tryLightTarget(
5428         x: number,
5429         y: number,
5430         w: number,
5431         colorKey: string,
5432         targetAt: ReadonlyMap<number, number>,
5433         targets: readonly IOpticalTarget[],
5434         targetLit: boolean[],
5435     ): void {
5436         const targetIdx = targetAt.get(cellIndex(w, x, y));
5437         if (targetIdx === undefined) {
5438             return;
5439         }
5440         const tgt = targets[targetIdx];
5441         if (lightMatchesTarget(colorKey, tgt.colorKey)) {
5442             targetLit[targetIdx] = true;
5443         }
5444     }
5445     function pieceCellKey(cx: number, cy: number): string {
5446         return `${cx},${cy}`;
5447     }
5448     interface TraceAllBeamsOptions {
5449         /** 为 false 时不记录环外入射（第二遍光追用） */
5450         recordCycleIncoming?: boolean;
5451         /** 已算出的环稳态色；环上元件出射强制使用该色 */
5452         cycleUniformByGroup?: ReadonlyMap<number, MixedLightColorKey>;

```

```

5453 }
5454 function cycleGroupIdForPieceEmission(
5455     w: number,
5456     cx: number,
5457     cy: number,
5458     cycleCtx: BeamCycleContext,
5459 ): number | undefined {
5460     return cycleCtx.cellToGroupId.get(cellIndex(w, cx, cy));
5461 }
5462 function pieceOutColor(
5463     w: number,
5464     cx: number,
5465     cy: number,
5466     piece: IOpticalPiece,
5467     incomingColors: readonly string[],
5468     sourceIds: ReadonlySet<number>,
5469     cycleCtx: BeamCycleContext,
5470     cycleUniformByGroup?: ReadonlyMap<number, MixedLightColorKey>,
5471 ): MixedLightColorKey {
5472     const gid = cycleGroupIdForPieceEmission(w, cx, cy, cycleCtx);
5473     if (gid !== undefined && cycleUniformByGroup) {
5474         const group = cycleCtx.groups.find((g) => g.id === gid);
5475         if (group && group.sourceIndices.some((sid) => sourceIds.has(sid))) {
5476             const uniform = cycleUniformByGroup.get(gid);
5477             if (uniform !== undefined) {
5478                 return uniform;
5479             }
5480         }
5481     }
5482     const mergedIncoming = mixLightColors(incomingColors);
5483     return mixLightColors([mergedIncoming, colorModeToKey(piece.colorMode)]);
5484 }
5485 function processOnePieceCell(
5486     w: number,
5487     pieceAt: ReadonlyMap<number, IOpticalPiece>,
5488     cellKey: string,
5489     pieceIncoming: Map<string, string[]>,
5490     pieceEntryDirs: Map<string, Direction[]>,
5491     pieceSourceIds: Map<string, Set<number>>,
5492     outSegments: OpticalBeamSegment[],
5493     queue: BeamRay[],
5494     cycleCtx: BeamCycleContext,
5495     cycleUniformByGroup?: ReadonlyMap<number, MixedLightColorKey>,
5496 ): void {
5497     const colors = pieceIncoming.get(cellKey);
5498     const entryDirs = pieceEntryDirs.get(cellKey);
5499     const sourceIds = pieceSourceIds.get(cellKey);
5500     if (!colors || colors.length === 0 || !entryDirs || entryDirs.length === 0 || !sourceIds) {
5501         return;
5502     }
5503     const parts = cellKey.split(',');
5504     const cx = Number(parts[0]);
5505     const cy = Number(parts[1]);
5506     const piece = pieceAt.get(cellIndex(w, cx, cy));
5507     if (!piece) {
5508         return;
5509     }
5510     const openDirs = openDirectionsForPiece(piece.connectivity, piece.direction);

```

```

5511     const outColor = pieceOutColor(w, cx, cy, piece, colors, sourceIds, cycleCtx, cycleUniformByGroup);
5512     const outputSides = new Set<Direction>();
5513     for (const entryDir of entryDirs) {
5514         const entrySide = propagationToEntrySide(entryDir);
5515         for (const side of openDirs) {
5516             if (side !== entrySide) {
5517                 outputSides.add(side);
5518             }
5519         }
5520     }
5521     const mx = cx + 0.5;
5522     const my = cy + 0.5;
5523     for (const outSide of outputSides) {
5524         const outDir = entrySideToPropagation(outSide);
5525         const exitX = mx + DIR_DX[outDir] * 0.5;
5526         const exitY = my + DIR_DY[outDir] * 0.5;
5527         pushSegment(outSegments, mx, my, exitX, exitY, outColor);
5528         queue.push({
5529             ax: exitX,
5530             ay: exitY,
5531             cx: cx + DIR_DX[outDir],
5532             cy: cy + DIR_DY[outDir],
5533             dir: outDir,
5534             colorKey: outColor,
5535             sourceIds: copySourceIds(sourceIds),
5536         });
5537     }
5538 }
5539 function processWavePendingPieces(
5540     w: number,
5541     pieceAt: ReadonlyMap<number, IOpticalPiece>,
5542     pendingPieceCells: Set<string>,
5543     pieceIncoming: Map<string, string[]>,
5544     pieceEntryDirs: Map<string, Direction[]>,
5545     pieceSourceIds: Map<string, Set<number>>,
5546     outSegments: OpticalBeamSegment[],
5547     nextWave: BeamRay[],
5548     cycleCtx: BeamCycleContext,
5549     cycleUniformByGroup?: ReadonlyMap<number, MixedLightColorKey>,
5550 ): void {
5551     const cells = sortPieceCellKeys(Array.from(pendingPieceCells));
5552     pendingPieceCells.clear();
5553     for (const cellKey of cells) {
5554         processOnePieceCell(
5555             w,
5556             pieceAt,
5557             cellKey,
5558             pieceIncoming,
5559             pieceEntryDirs,
5560             pieceSourceIds,
5561             outSegments,
5562             nextWave,
5563             cycleCtx,
5564             cycleUniformByGroup,
5565         );
5566         pieceIncoming.delete(cellKey);
5567         pieceEntryDirs.delete(cellKey);
5568         pieceSourceIds.delete(cellKey);

```



```

5569     }
5570 }
5571 function advanceBeamRay(
5572     input: OpticalBeamTraceInput,
5573     beam: BeamRay,
5574     pieceAt: ReadonlyMap<number, IOpticalPiece>,
5575     targetAt: ReadonlyMap<number, number>,
5576     targetLit: boolean[],
5577     visited: Set<string>,
5578     pieceIncoming: Map<string, string[]>,
5579     pieceEntryDirs: Map<string, Direction[]>,
5580     pieceSourceIds: Map<string, Set<number>>,
5581     pendingPieceCells: Set<string>,
5582     outSegments: OpticalBeamSegment[],
5583     outBlockContacts: OpticalBeamBlockContact[],
5584     nextWave: BeamRay[],
5585     cycleCtx: BeamCycleContext,
5586     incomingByCycle: Map<number, string[]>,
5587     traceOptions: TraceAllBeamsOptions = {},
5588 ): void {
5589     const { recordCycleIncoming = true, cycleUniformByGroup } = traceOptions;
5590     const { width: w, height: h, terrain, player, targets } = input;
5591     const { cx, cy, dir } = beam;
5592     let { ax, ay } = beam;
5593     const colorKey = steadyColorAtCell(
5594         cx,
5595         cy,
5596         dir,
5597         w,
5598         beam.colorKey,
5599         beam.sourceIds,
5600         cycleCtx,
5601         cycleUniformByGroup,
5602     );
5603     if (!inBounds(cx, cy, w, h)) {
5604         return;
5605     }
5606     const piece = pieceAt.get(cellIndex(w, cx, cy));
5607     const isPieceCell = piece != null && piece.connectivity !== 0;
5608     const visitKey = isPieceCell
5609         ? `${cx},${cy},${dir},${colorKey},p`
5610         : `${cx},${cy},${dir},${colorKey}`;
5611     if (visited.has(visitKey)) {
5612         return;
5613     }
5614     visited.add(visitKey);
5615     if (recordCycleIncoming) {
5616         recordCycleIncomingBeam(
5617             cx,
5618             cy,
5619             dir,
5620             colorKey,
5621             cycleCtx,
5622             w,
5623             incomingByCycle,
5624             pieceAt,
5625             beam.sourceIds,
5626         );

```

```
5627     }
5628     if (isWall(cx, cy, w, terrain) || isPlayerCell(cx, cy, player)) {
5629         pushSegment(
5630             outSegments,
5631             ax,
5632             ay,
5633             cx + 0.5 - DIR_DX[dir] * 0.5,
5634             cy + 0.5 - DIR_DY[dir] * 0.5,
5635             colorKey,
5636         );
5637         pushBlockContact(outBlockContacts, cx, cy, dir, colorKey);
5638         return;
5639     }
5640     if (terrainKindAt(terrain, cellIndex(w, cx, cy)) === TerrainKind.Target) {
5641         pushSegment(
5642             outSegments,
5643             ax,
5644             ay,
5645             cx + 0.5 - DIR_DX[dir] * 0.5,
5646             cy + 0.5 - DIR_DY[dir] * 0.5,
5647             colorKey,
5648         );
5649         tryLightTarget(cx, cy, w, colorKey, targetAt, targets, targetLit);
5650         return;
5651     }
5652     if (terrainKindAt(terrain, cellIndex(w, cx, cy)) === TerrainKind.Source) {
5653         pushSegment(
5654             outSegments,
5655             ax,
5656             ay,
5657             cx + 0.5 - DIR_DX[dir] * 0.5,
5658             cy + 0.5 - DIR_DY[dir] * 0.5,
5659             colorKey,
5660         );
5661         return;
5662     }
5663     if (!piece || piece.connectivity === 0) {
5664         if (piece) {
5665             pushSegment(
5666                 outSegments,
5667                 ax,
5668                 ay,
5669                 cx + 0.5 - DIR_DX[dir] * 0.5,
5670                 cy + 0.5 - DIR_DY[dir] * 0.5,
5671                 colorKey,
5672             );
5673             pushBlockContact(outBlockContacts, cx, cy, dir, colorKey);
5674             return;
5675         }
5676         const nx = cx + DIR_DX[dir];
5677         const ny = cy + DIR_DY[dir];
5678         if (!inBounds(nx, ny, w, h)) {
5679             pushSegment(
5680                 outSegments,
5681                 ax,
5682                 ay,
5683                 cx + 0.5 + DIR_DX[dir] * 0.5,
5684                 cy + 0.5 + DIR_DY[dir] * 0.5,
```

```

5685         colorKey,
5686     );
5687     pushBlockContact(outBlockContacts, cx, cy, dir, colorKey, true);
5688     return;
5689 }
5690 pushSegment(
5691     outSegments,
5692     ax,
5693     ay,
5694     cx + 0.5 + DIR_DX[dir] * 0.5,
5695     cy + 0.5 + DIR_DY[dir] * 0.5,
5696     colorKey,
5697 );
5698 ax = cx + 0.5 + DIR_DX[dir] * 0.5;
5699 ay = cy + 0.5 + DIR_DY[dir] * 0.5;
5700 nextWave.push({
5701     ax,
5702     ay,
5703     cx: nx,
5704     cy: ny,
5705     dir,
5706     colorKey,
5707     sourceIds: copySourceIds(beam.sourceIds),
5708 });
5709 return;
5710 }
5711 const openDirs = openDirectionsForPiece(piece.connectivity, piece.direction);
5712 const entrySide = propagationToEntrySide(dir);
5713 if (!openDirs.includes(entrySide)) {
5714     pushSegment(
5715         outSegments,
5716         ax,
5717         ay,
5718         cx + 0.5 - DIR_DX[dir] * 0.5,
5719         cy + 0.5 - DIR_DY[dir] * 0.5,
5720         colorKey,
5721     );
5722     pushBlockContact(outBlockContacts, cx, cy, dir, colorKey);
5723     return;
5724 }
5725 const cellKey = pieceCellKey(cx, cy);
5726 let incomingList = pieceIncoming.get(cellKey);
5727 if (!incomingList) {
5728     incomingList = [];
5729     pieceIncoming.set(cellKey, incomingList);
5730 }
5731 incomingList.push(resolveBeamColorKey(colorKey));
5732 let entryDirs = pieceEntryDirs.get(cellKey);
5733 if (!entryDirs) {
5734     entryDirs = [];
5735     pieceEntryDirs.set(cellKey, entryDirs);
5736 }
5737 entryDirs.push(dir);
5738 pendingPieceCells.add(cellKey);
5739 let sourceIdSet = pieceSourceIds.get(cellKey);
5740 if (!sourceIdSet) {
5741     sourceIdSet = new Set<number>();
5742     pieceSourceIds.set(cellKey, sourceIdSet);

```

```

5743     }
5744     mergeSourceIds(sourceIdSet, beam.sourceIds);
5745     const mx = cx + 0.5;
5746     const my = cy + 0.5;
5747     const entryX = mx - DIR_DX[dir] * 0.5;
5748     const entryY = my - DIR_DY[dir] * 0.5;
5749     pushSegment(outSegments, ax, ay, entryX, entryY, colorKey);
5750     pushSegment(outSegments, entryX, entryY, mx, my, colorKey);
5751 }
5752 function sortPieceCellKeys(keys: readonly string[]): string[] {
5753     return [...keys].sort((a, b) => {
5754         const pa = a.split(',').map(Number);
5755         const pb = b.split(',').map(Number);
5756         return pa[1] - pb[1] || pa[0] - pb[0];
5757     });
5758 }
5759 function beamRayStateKey(beam: BeamRay): string {
5760     return `${beam.cx},${beam.cy},${beam.dir},${beam.colorKey}`;
5761 }
5762 /** 同波层内合并相同 (格, 方向, 色) 的射线, 避免分路指数膨胀 */
5763 function dedupeWaveRays(beams: readonly BeamRay[]): BeamRay[] {
5764     const map = new Map<string, BeamRay>();
5765     for (const beam of beams) {
5766         const key = beamRayStateKey(beam);
5767         const existing = map.get(key);
5768         if (!existing) {
5769             map.set(key, { ...beam, sourceIds: copySourceIds(beam.sourceIds) });
5770         } else {
5771             mergeSourceIds(existing.sourceIds, beam.sourceIds);
5772         }
5773     }
5774     return Array.from(map.values());
5775 }
5776 function traceAllBeams(
5777     input: OpticalBeamTraceInput,
5778     sources: readonly IOpticalLightSource[],
5779     pieceAt: ReadonlyMap<number, IOpticalPiece>,
5780     targetAt: ReadonlyMap<number, number>,
5781     targetLit: boolean[],
5782     outSegments: OpticalBeamSegment[],
5783     outBlockContacts: OpticalBeamBlockContact[],
5784     cycleCtx: BeamCycleContext,
5785     incomingByCycle: Map<number, string[]>,
5786     traceOptions: TraceAllBeamsOptions = {},
5787 ): void {
5788     const { width: w, height: h } = input;
5789     let currentWave: BeamRay[] = [];
5790     for (let si = 0; si < sources.length; si++) {
5791         const src = sources[si];
5792         const delta = opticalDirDelta(src.direction);
5793         if (!delta) {
5794             continue;
5795         }
5796         const { dx: dx0, dy: dy0, dir: srcDir } = delta;
5797         const sx = normalizeGridCoord(src.x);
5798         const sy = normalizeGridCoord(src.y);
5799         currentWave.push({
5800             ax: sx + 0.5 + dx0 * 0.5,

```

```

5801         ay: sy + 0.5 + dy0 * 0.5,
5802         cx: sx + dx0,
5803         cy: sy + dy0,
5804         dir: srcDir,
5805         colorKey: steadyEmissionColorForSource(
5806             src,
5807             si,
5808             w,
5809             cycleCtx,
5810             traceOptions.cycleUniformByGroup,
5811         ),
5812         sourceIds: new Set([si]),
5813     });
5814 }
5815 /** 空地/目标等：按色分开防环；元件格用 `p` 后缀允许多色入射累积 */
5816 const visited = new Set<string>();
5817 const maxWaves = w * h * 8 * Math.max(1, sources.length);
5818 for (let wave = 0; wave < maxWaves && currentWave.length > 0; wave++) {
5819     currentWave = dedupeWaveRays(currentWave);
5820     if (currentWave.length === 0) {
5821         break;
5822     }
5823     const nextWave: BeamRay[] = [];
5824     const pieceIncoming = new Map<string, string[]>();
5825     const pieceEntryDirs = new Map<string, Direction[]>();
5826     const pieceSourceIds = new Map<string, Set<number>>();
5827     const pendingPieceCells = new Set<string>();
5828     for (const beam of currentWave) {
5829         advanceBeamRay(
5830             input,
5831             beam,
5832             pieceAt,
5833             targetAt,
5834             targetLit,
5835             visited,
5836             pieceIncoming,
5837             pieceEntryDirs,
5838             pieceSourceIds,
5839             pendingPieceCells,
5840             outSegments,
5841             outBlockContacts,
5842             nextWave,
5843             cycleCtx,
5844             incomingByCycle,
5845             traceOptions,
5846         );
5847     }
5848     processWavePendingPieces(
5849         w,
5850         pieceAt,
5851         pendingPieceCells,
5852         pieceIncoming,
5853         pieceEntryDirs,
5854         pieceSourceIds,
5855         outSegments,
5856         nextWave,
5857         cycleCtx,
5858         traceOptions.cycleUniformByGroup,

```

```

5859         );
5860         currentWave = nextWave;
5861     }
5862 }
5863 /** 通道 0~4 统一光追（波层同步多光源 + 稳态环后处理） */
5864 export function traceBeams(input: OpticalBeamTraceInput): OpticalBeamTraceResult {
5865     const w = normalizeGridSize(input.width, input.width);
5866     const h = normalizeGridSize(input.height, input.height);
5867     const normalizedInput: OpticalBeamTraceInput = {
5868         ...input,
5869         width: w,
5870         height: h,
5871         terrain: input.terrain.map((t) => normalizeTerrainKind(t) as TerrainKind),
5872         player: {
5873             x: normalizeGridCoord(input.player.x),
5874             y: normalizeGridCoord(input.player.y),
5875         },
5876         pieces: input.pieces.map((p) => ({
5877             ...p,
5878             x: normalizeGridCoord(p.x),
5879             y: normalizeGridCoord(p.y),
5880             direction: normalizeDirection(p.direction, Direction.Up),
5881             connectivity: normalizePieceConnectivity(p.connectivity) as PieceConnectivity,
5882         })),
5883         sources: input.sources.map((s) => ({
5884             ...s,
5885             x: normalizeGridCoord(s.x),
5886             y: normalizeGridCoord(s.y),
5887             direction: normalizeDirection(s.direction, Direction.Down),
5888         })),
5889         targets: input.targets.map((t) => ({
5890             ...t,
5891             x: normalizeGridCoord(t.x),
5892             y: normalizeGridCoord(t.y),
5893         })),
5894     };
5895     const { pieces, sources, targets } = normalizedInput;
5896     const pieceAt = new Map<number, IOpticalPiece>();
5897     for (const p of pieces) {
5898         pieceAt.set(cellIndex(w, p.x, p.y), p);
5899     }
5900     const targetAt = new Map<number, number>();
5901     targets.forEach((t, i) => {
5902         targetAt.set(cellIndex(w, t.x, t.y), i);
5903     });
5904     const cycleCtx = buildBeamCycleContext(normalizedInput, pieceAt);
5905     const incomingByCycle = createCycleIncomingMap(cycleCtx);
5906     /** 第一遍：收集环外入射（不产出最终光段） */
5907     const incomingProbeLit = targets.map(() => false);
5908     traceAllBeams(
5909         normalizedInput,
5910         sources,
5911         pieceAt,
5912         targetAt,
5913         incomingProbeLit,
5914         [],
5915         [],
5916         cycleCtx,

```

```

5917         incomingByCycle,
5918         { recordCycleIncoming: true },
5919     );
5920     const cycleColors = computeCycleUniformColors(cycleCtx, incomingByCycle);
5921     /** 第二遍：环上元件出射使用环稳态色 */
5922     const rawSegments: OpticalBeamSegment[] = [];
5923     const rawBlockContacts: OpticalBeamBlockContact[] = [];
5924     const scratchTargetLit = targets.map(() => false);
5925     traceAllBeams(
5926         normalizedInput,
5927         sources,
5928         pieceAt,
5929         targetAt,
5930         scratchTargetLit,
5931         rawSegments,
5932         rawBlockContacts,
5933         cycleCtx,
5934         incomingByCycle,
5935         { recordCycleIncoming: false, cycleUniformByGroup: cycleColors },
5936     );
5937     const cycleResolvedSegments = applyCycleUniformColors(rawSegments, cycleCtx, cycleColors, w, pieceAt);
5938     const segments = mergeOverlappingBeamSegments(cycleResolvedSegments);
5939     const blockContacts = alignBlockContactsWithSegments(
5940         dedupeBlockContacts(rawBlockContacts),
5941         segments,
5942     );
5943     const targetLit = recomputeTargetLitFromSegments(segments, targets);
5944     return { segments, blockContacts, targetLit };
5945 }
5946 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalBeamTypes.ts -----
5947 /** 光路段（格坐标，可含 0.5 半格）；独立文件避免 Tracer ↔ OverlapMerge 循环依赖 */
5948 export interface OpticalBeamSegment {
5949     x0: number;
5950     y0: number;
5951     x1: number;
5952     y1: number;
5953     colorKey: string;
5954 }
5955 /** 光线被阻挡时的接触点（格坐标，可含 0.5 半格；位于阻挡格朝向光路的接触面） */
5956 export interface OpticalBeamBlockContact {
5957     x: number;
5958     y: number;
5959     /** 光线传播方向（与光追 DIR_DX/DY 一致） */
5960     dir: number;
5961     colorKey: string;
5962 }
5963 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalColorMix.ts -----
5964 import type { LightColorKey } from './OpticalLightColor';
5965 import { resolveBeamColorKey } from './OpticalLightColor';
5966 /** 运行时混色结果（含二次色） */
5967 export type MixedLightColorKey =
5968     | LightColorKey
5969     | 'yellow'
5970     | 'cyan'
5971     | 'purple';
5972 export interface RgbScores {
5973     r: number;
5974     g: number;

```

```

5975     b: number;
5976 }
5977 const BASE_KEYS: readonly LightColorKey[] = ['red', 'green', 'blue', 'white'];
5978 const MIXED_KEYS: readonly MixedLightColorKey[] = [
5979     'red',
5980     'green',
5981     'blue',
5982     'white',
5983     'yellow',
5984     'cyan',
5985     'purple',
5986 ];
5987 export function isMixedLightColorKey(key: string): key is MixedLightColorKey {
5988     return (MIXED_KEYS as readonly string[]).includes(key);
5989 }
5990 /** 第一步：单色分解为 R/G/B 得分（可累加） */
5991 export function decomposeToRgbScores(colorKey: string): RgbScores {
5992     switch (colorKey) {
5993         case 'red':
5994             return { r: 1, g: 0, b: 0 };
5995         case 'green':
5996             return { r: 0, g: 1, b: 0 };
5997         case 'blue':
5998             return { r: 0, g: 0, b: 1 };
5999         case 'yellow':
6000             return { r: 1, g: 1, b: 0 };
6001         case 'cyan':
6002             return { r: 0, g: 1, b: 1 };
6003         case 'purple':
6004             return { r: 1, g: 0, b: 1 };
6005         case 'white':
6006             default:
6007                 return { r: 1, g: 1, b: 1 };
6008     }
6009 }
6010 export function addRgbScores(a: RgbScores, b: RgbScores): RgbScores {
6011     return { r: a.r + b.r, g: a.g + b.g, b: a.b + b.b };
6012 }
6013 export function sumRgbScores(colors: readonly string[]): RgbScores {
6014     let acc: RgbScores = { r: 0, g: 0, b: 0 };
6015     for (const c of colors) {
6016         acc = addRgbScores(acc, decomposeToRgbScores(c));
6017     }
6018     return acc;
6019 }
6020 /**
6021  * 三步混色（入射光 + 元件本色均参与第一步累加）：
6022  * 1. 分解累加 → 2. 若 R/G/B 得分均 ≥1 则各减最小值 → 3. 两色合成 / 单色直出 / 无则白
6023  */
6024 export function mixLightColors(colors: readonly string[]): MixedLightColorKey {
6025     let scores = sumRgbScores(colors);
6026     if (scores.r >= 1 && scores.g >= 1 && scores.b >= 1) {
6027         const min = Math.min(scores.r, scores.g, scores.b);
6028         scores = {
6029             r: scores.r - min,
6030             g: scores.g - min,
6031             b: scores.b - min,
6032         };

```



```
6033     }
6034     const present: LightColorKey[] = [];
6035     if (scores.r > 0) {
6036         present.push('red');
6037     }
6038     if (scores.g > 0) {
6039         present.push('green');
6040     }
6041     if (scores.b > 0) {
6042         present.push('blue');
6043     }
6044     if (present.length === 0) {
6045         return 'white';
6046     }
6047     if (present.length === 1) {
6048         return present[0];
6049     }
6050     if (present.length === 2) {
6051         const has = (c: LightColorKey) => present.includes(c);
6052         if (has('red') && has('green')) {
6053             return 'yellow';
6054         }
6055         if (has('red') && has('blue')) {
6056             return 'purple';
6057         }
6058         if (has('green') && has('blue')) {
6059             return 'cyan';
6060         }
6061     }
6062     return 'white';
6063 }
6064 function isSetLike(raw: unknown): raw is { forEach: (cb: (value: unknown) => void) => void } {
6065     return raw != null
6066         && typeof raw === 'object'
6067         && typeof (raw as { forEach?: unknown }).forEach === 'function';
6068 }
6069 /** 从 runtime 各种形态（含微信包 Set duck-type）提取色键字符串 */
6070 export function collectColorKeyStrings(raw: unknown): string[] {
6071     if (typeof raw === 'string') {
6072         return [raw];
6073     }
6074     if (raw == null) {
6075         return [];
6076     }
6077     if (Array.isArray(raw)) {
6078         return raw.filter((k): k is string => typeof k === 'string');
6079     }
6080     if (isSetLike(raw)) {
6081         const out: string[] = [];
6082         raw.forEach((value) => {
6083             if (typeof value === 'string') {
6084                 out.push(value);
6085             }
6086         });
6087         return out;
6088     }
6089     if (typeof raw === 'object') {
6090         const boxed = Object.prototype.toString.call(raw);
```

```

6091         if (boxed === '[object String]') {
6092             return [String(raw)];
6093         }
6094         const nested = (raw as { colorKey?: unknown; colorKeys?: unknown }).colorKey;
6095         if (nested != null && nested !== raw) {
6096             return collectColorKeyStrings(nested);
6097         }
6098         const nestedKeys = (raw as { colorKeys?: unknown }).colorKeys;
6099         if (nestedKeys != null && nestedKeys !== raw) {
6100             return collectColorKeyStrings(nestedKeys);
6101         }
6102     }
6103     return [];
6104 }
6105 /**
6106  * 将阻挡接触 / 光路 snapshot 中的 colorKey 规范为 BeamColorKey。
6107  * 微信 Rollup 后偶发 Set、类数组 object 等，避免 instanceof Set 失效。
6108  */
6109 export function coerceBeamColorKey(raw: unknown): MixedLightColorKey {
6110     const keys = collectColorKeyStrings(raw);
6111     if (keys.length === 0) {
6112         return 'white';
6113     }
6114     if (keys.length === 1) {
6115         return resolveBeamColorKey(keys[0]);
6116     }
6117     return mixLightColors(keys);
6118 }
6119 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalLightColor.ts -----
6120 /** 光色键：基础四色 + 混色二次色（层 3 `Y`/`C`/`P` 或混色输出） */
6121 export type LightColorKey = 'white' | 'red' | 'green' | 'blue';
6122 export type BeamColorKey = LightColorKey | 'yellow' | 'cyan' | 'purple';
6123 import type { ColorMode } from './OpticalPuzzleTypes';
6124 import { ColorMode as ColorModeEnum } from './OpticalPuzzleTypes';
6125 const BASE_KEYS: ReadonlySet<string> = new Set(['white', 'red', 'green', 'blue']);
6126 const MIXED_KEYS: ReadonlySet<string> = new Set(['yellow', 'cyan', 'purple']);
6127 export function isBeamColorKey(key: string): key is BeamColorKey {
6128     return BASE_KEYS.has(key) || MIXED_KEYS.has(key);
6129 }
6130 /** 基础四色（元件层 3 等）；未知回落为 white */
6131 export function normalizeLightColorKey(key?: string): LightColorKey {
6132     if (key && BASE_KEYS.has(key)) {
6133         return key as LightColorKey;
6134     }
6135     return 'white';
6136 }
6137 /** 光追 / S/E 用色键：保留黄/青/紫，未知则回落为 white */
6138 export function resolveBeamColorKey(key?: string): BeamColorKey {
6139     if (key && isBeamColorKey(key)) {
6140         return key as BeamColorKey;
6141     }
6142     return 'white';
6143 }
6144 /** 入射光色是否满足目标期望色（严格同色，含黄/青/紫混色） */
6145 export function lightMatchesTarget(beamKey: string | undefined, targetKey: string | undefined): boolean {
6146     return resolveBeamColorKey(beamKey) === resolveBeamColorKey(targetKey);
6147 }
6148 /** 元件 `colorMode` → 混色/显示用 colorKey */

```

```

6149 export function colorModeToKey(mode: ColorMode): string {
6150     switch (mode) {
6151         case ColorModeEnum.FilterRed:
6152             return 'red';
6153         case ColorModeEnum.FilterGreen:
6154             return 'green';
6155         case ColorModeEnum.FilterBlue:
6156             return 'blue';
6157         default:
6158             return 'white';
6159     }
6160 }
6161 /**
6162  * 单束光经元件后的出射色（§ 3.4 透传 / 滤光）。
6163  * `through` 保持入射色；滤光格将白光染成对应单色、同色直过，异色返回 `null`（阻挡）。
6164  */
6165 export function applyPieceColorMode(incomingKey: string, colorMode: ColorMode): string | null {
6166     const inKey = resolveBeamColorKey(incomingKey);
6167     if (colorMode === ColorModeEnum.Through) {
6168         return inKey;
6169     }
6170     const filterKey = colorModeToKey(colorMode);
6171     if (inKey === 'white') {
6172         return filterKey;
6173     }
6174     if (inKey === filterKey) {
6175         return inKey;
6176     }
6177     return null;
6178 }
6179 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalMirrorReflection.ts -----
6180 import type { IOpticalPiece } from '../Config/OpticalPuzzleLevelSchema';
6181 import { Direction, PieceType } from './OpticalPuzzleTypes';
6182 /**
6183  * 查表下标 = 光线**射入该格时的传播方向**（与 `Direction` 枚举一致）：
6184  * - 光从**上方**来 → 传播方向 **Down**
6185  * - 光从**下方**来 → **Up**；从**左侧**来 → **Right**；从**右侧**来 → **Left**
6186  */
6187 /**
6188  * ` / ` 镜：上→左、左→上、右→下、下→右（指光的来向与去向）。
6189  */
6190 const REFLECT_MIRROR_SLASH: ReadonlyArray<Direction> = [
6191     Direction.Up, // 自左 (Right) → 向上
6192     Direction.Right, // 自下 (Up) → 向右
6193     Direction.Down, // 自右 (Left) → 向下
6194     Direction.Left, // 自上 (Down) → 向左
6195 ];
6196 /**
6197  * ` \ ` 镜：上→右、右→上、下→左、左→下。
6198  */
6199 const REFLECT_MIRROR_BACKSLASH: ReadonlyArray<Direction> = [
6200     Direction.Down, // 自左 (Right) → 向下
6201     Direction.Left, // 自下 (Up) → 向左
6202     Direction.Up, // 自右 (Left) → 向上
6203     Direction.Right, // 自上 (Down) → 向右
6204 ];
6205 export function isMirrorPiece(piece: IOpticalPiece): boolean {
6206     return (

```

```

6207         piece.type === PieceType.MirrorSlash || piece.type === PieceType.MirrorBackslash
6208     );
6209 }
6210 export function reflectAtPiece(piece: IOpticalPiece, incoming: Direction): Direction | null {
6211     if (!isMirrorPiece(piece)) {
6212         return null;
6213     }
6214     if (piece.type === PieceType.MirrorSlash) {
6215         return REFLECT_MIRROR_SLASH[incoming];
6216     }
6217     return REFLECT_MIRROR_BACKSLASH[incoming];
6218 }
6219 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalPieceConnectivity.ts -----
6220 import { Direction } from './OpticalPuzzleTypes';
6221 /** 统一光学元件通道类型（默认朝向下的开口集合） */
6222 export type PieceConnectivity = 0 | 1 | 2 | 3 | 4;
6223 /**
6224  * 默认朝向（层 4 `w`）下各类型的开口方向。
6225  * 0 无口；1 上+右；2 上+下；3 上+左+右；4 四面。
6226  */
6227 export const CONNECTIVITY_OPEN_DIRS: ReadonlyArray<readonly Direction[]> = [
6228     [],
6229     [Direction.Up, Direction.Right],
6230     [Direction.Up, Direction.Down],
6231     [Direction.Up, Direction.Left, Direction.Right],
6232     [Direction.Up, Direction.Right, Direction.Down, Direction.Left],
6233 ];
6234 const DIR_STEPS_CW: Readonly<Record<Direction, number>> = {
6235     [Direction.Up]: 0,
6236     [Direction.Right]: 1,
6237     [Direction.Down]: 2,
6238     [Direction.Left]: 3,
6239 };
6240 /** 顺时针旋转开口方向（`w`=0, `d`=1, `s`=2, `a`=3） */
6241 export function rotateOpenDirections(
6242     dirs: readonly Direction[],
6243     pieceDirection: Direction,
6244 ): Direction[] {
6245     const steps = DIR_STEPS_CW[pieceDirection];
6246     if (steps === 0) {
6247         return [...dirs];
6248     }
6249     return dirs.map((d) => ((d + 3 * steps) % 4) as Direction);
6250 }
6251 export function openDirectionsForPiece(
6252     connectivity: PieceConnectivity,
6253     pieceDirection: Direction,
6254 ): readonly Direction[] {
6255     const base = CONNECTIVITY_OPEN_DIRS[connectivity] ?? [];
6256     return rotateOpenDirections(base, pieceDirection);
6257 }
6258 /** 光以 propagation 方向进入元件格时，入射侧是否为该 connectivity 的通道开口 */
6259 export function canEnterPieceWithPropagation(
6260     propagation: Direction,
6261     piece: { connectivity: PieceConnectivity; direction: Direction },
6262 ): boolean {
6263     if (piece.connectivity === 0) {
6264         return false;

```

```

6265     }
6266     const openDirs = openDirectionsForPiece(piece.connectivity, piece.direction);
6267     return openDirs.includes(propagationToEntrySide(propagation));
6268 }
6269 /** 传播方向 → 从哪一侧进入该格（向下传播 = 从上方进入 = Up） */
6270 export function propagationToEntrySide(propagation: Direction): Direction {
6271     return ((propagation + 2) % 4) as Direction;
6272 }
6273 /** 从某开口侧出射时的传播方向（向上开口 = 向上传播） */
6274 export function entrySideToPropagation(exitSide: Direction): Direction {
6275     return exitSide;
6276 }
6277 export function parseConnectivityChar(ch: string): PieceConnectivity | null {
6278     if (ch >= '0' && ch <= '4') {
6279         return Number(ch) as PieceConnectivity;
6280     }
6281     return null;
6282 }
6283 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalPuzzleCore.ts -----
6284 import type {
6285     IOpticalLevelConfig,
6286     IOpticalLightSource,
6287     IOpticalPiece,
6288     IOpticalTarget,
6289 } from './Config/OpticalPuzzleLevelSchema';
6290 import { colorModeToKey, resolveBeamColorKey } from './OpticalLightColor';
6291 import type { PieceConnectivity } from './OpticalPieceConnectivity';
6292 import {
6293     normalizeGridCoord,
6294     normalizeGridSize,
6295     normalizePieceConnectivity,
6296     normalizeTerrainKind,
6297 } from './OpticalRuntimeCoerce';
6298 import {
6299     type OpticalBeamBlockContact,
6300     type OpticalBeamSegment,
6301     type OpticalBeamTraceInput,
6302     traceBeams,
6303 } from './OpticalBeamTracer';
6304 import {
6305     Direction,
6306     MoveAttemptResult,
6307     normalizeDirection,
6308     type OpticalBoardSnapshot,
6309     type OpticalPieceSnapshot,
6310     type OpticalSourceSnapshot,
6311     type OpticalTargetSnapshot,
6312     TerrainKind,
6313 } from './OpticalPuzzleTypes';
6314 const DIR_DX: ReadonlyArray<number> = [1, 0, -1, 0];
6315 const DIR_DY: ReadonlyArray<number> = [0, -1, 0, 1];
6316 export interface OpticalBeamSnapshot {
6317     width: number;
6318     height: number;
6319     segments: OpticalBeamSegment[];
6320     blockContacts: OpticalBeamBlockContact[];
6321 }
6322 /** 撤回栈条目：玩家位置 + 元件布局（地形本关不变） */

```

```

6323 export interface OpticalPlayStateSnapshot {
6324     player: { x: number; y: number };
6325     playerFacing: Direction;
6326     pieces: IOpticalPiece[];
6327 }
6328 /**
6329  * 纯规则层：不引用 Node / Prefab。
6330  * 行走、推箱子式推动、光追、通关判定。
6331  */
6332 export class OpticalPuzzleCore {
6333     private _levelId = 0;
6334     private _levelName = "";
6335     private _w = 0;
6336     private _h = 0;
6337     private _terrain: TerrainKind[] = [];
6338     private _px = 0;
6339     private _py = 0;
6340     private _playerFacing: Direction = Direction.Left;
6341     private _sources: IOpticalLightSource[] = [];
6342     private _targets: IOpticalTarget[] = [];
6343     private _pieces: IOpticalPiece[] = [];
6344     private _beamSegments: OpticalBeamSegment[] = [];
6345     private _beamBlockContacts: OpticalBeamBlockContact[] = [];
6346     private _targetLit: boolean[] = [];
6347     reset(level: IOpticalLevelConfig): void {
6348         const n = level.width * level.height;
6349         if (level.terrain.length !== n) {
6350             throw new Error(
6351                 `[OpticalPuzzleCore] terrain length ${level.terrain.length} != ${n}`,
6352             );
6353         }
6354         this._levelId = level.levelId;
6355         this._levelName = level.levelName;
6356         this._w = normalizeGridSize(level.width, level.width);
6357         this._h = normalizeGridSize(level.height, level.height);
6358         this._terrain = level.terrain.map((t) => normalizeTerrainKind(t) as TerrainKind);
6359         this._px = normalizeGridCoord(level.player.x);
6360         this._py = normalizeGridCoord(level.player.y);
6361         this._playerFacing = Direction.Left;
6362         this._sources = level.sources.map((s) => ({
6363             ...s,
6364             x: normalizeGridCoord(s.x),
6365             y: normalizeGridCoord(s.y),
6366             direction: normalizeDirection(s.direction, Direction.Down),
6367         }));
6368         this._targets = level.targets.map((t) => ({
6369             ...t,
6370             x: normalizeGridCoord(t.x),
6371             y: normalizeGridCoord(t.y),
6372         }));
6373         this._pieces = level.pieces.map((p) => ({
6374             ...p,
6375             x: normalizeGridCoord(p.x),
6376             y: normalizeGridCoord(p.y),
6377             direction: normalizeDirection(p.direction, Direction.Up),
6378             connectivity: normalizePieceConnectivity(p.connectivity) as PieceConnectivity,
6379         }));
6380         this._recomputeLighting();

```

```

6381     }
6382     getSnapshot(): OpticalBoardSnapshot {
6383         return {
6384             levelId: this._levelId,
6385             levelName: this._levelName,
6386             width: this._w,
6387             height: this._h,
6388             terrain: this._terrain.slice(),
6389             player: { x: this._px, y: this._py },
6390             playerFacing: this._playerFacing,
6391             sources: this._buildSourceSnapshots(),
6392             targets: this._buildTargetSnapshots(),
6393             pieces: this._buildPieceSnapshots(),
6394             allTargetsLit: this.isAllTargetsLit(),
6395         };
6396     }
6397     getBeamSnapshot(): OpticalBeamSnapshot {
6398         return {
6399             width: this._w,
6400             height: this._h,
6401             segments: this._beamSegments.map((s) => ({ ...s })),
6402             blockContacts: this._beamBlockContacts.map((c) => ({ ...c })),
6403         };
6404     }
6405     isAllTargetsLit(): boolean {
6406         if (this._targets.length === 0) {
6407             return false;
6408         }
6409         return this._targetLit.every(Boolean);
6410     }
6411     /** 更新朝向（与是否移动成功无关，由输入层在 tryMove 前调用） */
6412     setPlayerFacing(dir: Direction): void {
6413         this._playerFacing = dir;
6414     }
6415     /**
6416      * 推动方向若连续两格都有元件则推不动。
6417      */
6418     tryMove(dir: Direction): MoveAttemptResult {
6419         const dx = DIR_DX[dir];
6420         const dy = DIR_DY[dir];
6421         const nx = this._px + dx;
6422         const ny = this._py + dy;
6423         if (!this._inBounds(nx, ny)) {
6424             return MoveAttemptResult.Blocked;
6425         }
6426         const piece = this._pieceAt(nx, ny);
6427         if (!piece) {
6428             if (!this._canEnterFloor(nx, ny)) {
6429                 return MoveAttemptResult.Blocked;
6430             }
6431             this._px = nx;
6432             this._py = ny;
6433             this._recomputeLighting();
6434             return MoveAttemptResult.PlayerMoved;
6435         }
6436         const bx = nx + dx;
6437         const by = ny + dy;
6438         if (!this._canPushPieceTo(bx, by)) {

```

```

6439         return MoveAttemptResult.Blocked;
6440     }
6441     piece.x = bx;
6442     piece.y = by;
6443     this._px = nx;
6444     this._py = ny;
6445     this._recomputeLighting();
6446     return MoveAttemptResult.PiecePushed;
6447 }
6448 clonePlayState(): OpticalPlayStateSnapshot {
6449     return {
6450         player: { x: this._px, y: this._py },
6451         playerFacing: this._playerFacing,
6452         pieces: this._pieces.map((p) => ({ ...p })),
6453     };
6454 }
6455 restorePlayState(data: OpticalPlayStateSnapshot, options?: { deferLighting?: boolean }): void {
6456     this._px = normalizeGridCoord(data.player.x);
6457     this._py = normalizeGridCoord(data.player.y);
6458     this._playerFacing = normalizeDirection(data.playerFacing, Direction.Left);
6459     this._pieces = data.pieces.map((p) => ({
6460         ...p,
6461         x: normalizeGridCoord(p.x),
6462         y: normalizeGridCoord(p.y),
6463         direction: normalizeDirection(p.direction, Direction.Up),
6464         connectivity: normalizePieceConnectivity(p.connectivity) as PieceConnectivity,
6465     }));
6466     if (!options?.deferLighting) {
6467         this._recomputeLighting();
6468     }
6469 }
6470 /** 目标点亮位掩码（求解器剪枝用，低位对应 targets 顺序） */
6471 getTargetLitMask(): number {
6472     let mask = 0;
6473     for (let i = 0; i < this._targetLit.length; i++) {
6474         if (this._targetLit[i]) {
6475             mask |= 1 << i;
6476         }
6477     }
6478     return mask;
6479 }
6480 get targetCount(): number {
6481     return this._targets.length;
6482 }
6483 private _inBounds(x: number, y: number): boolean {
6484     return x >= 0 && y >= 0 && x < this._w && y < this._h;
6485 }
6486 private _pieceAt(x: number, y: number): IOpticalPiece | undefined {
6487     return this._pieces.find((p) => p.x === x && p.y === y);
6488 }
6489 /** 主角走入：须为地板且非光源/目标格 */
6490 private _canEnterFloor(x: number, y: number): boolean {
6491     if (!this._inBounds(x, y)) {
6492         return false;
6493     }
6494     return this._terrain[y * this._w + x] === TerrainKind.Floor && !this._pieceAt(x, y);
6495 }
6496 /** 元件被推到：须为地板、无其他元件、非光源/目标 */

```



```

6497     private _canPushPieceTo(x: number, y: number): boolean {
6498         if (!this._inBounds(x, y)) {
6499             return false;
6500         }
6501         if (this._terrain[y * this._w + x] !== TerrainKind.Floor) {
6502             return false;
6503         }
6504         if (this._pieceAt(x, y)) {
6505             return false;
6506         }
6507         return true;
6508     }
6509     private _buildSourceSnapshots(): OpticalSourceSnapshot[] {
6510         return this._sources.map((s) => ({
6511             x: s.x,
6512             y: s.y,
6513             colorKey: resolveBeamColorKey(s.colorKey),
6514             direction: s.direction,
6515         }));
6516     }
6517     private _buildPieceSnapshots(): OpticalPieceSnapshot[] {
6518         return this._pieces.map((p) => ({
6519             x: p.x,
6520             y: p.y,
6521             connectivity: p.connectivity,
6522             direction: p.direction,
6523             colorKey: colorModeToKey(p.colorMode),
6524         }));
6525     }
6526     private _buildTargetSnapshots(): OpticalTargetSnapshot[] {
6527         return this._targets.map((t, i) => ({
6528             x: t.x,
6529             y: t.y,
6530             colorKey: resolveBeamColorKey(t.colorKey),
6531             lit: this._targetLit[i] ?? false,
6532         }));
6533     }
6534     private _recomputeLighting(): void {
6535         const input: OpticalBeamTraceInput = {
6536             width: this._w,
6537             height: this._h,
6538             terrain: this._terrain,
6539             player: { x: this._px, y: this._py },
6540             pieces: this._pieces,
6541             sources: this._sources,
6542             targets: this._targets,
6543         };
6544         const result = traceBeams(input);
6545         this._beamSegments = result.segments;
6546         this._beamBlockContacts = result.blockContacts;
6547         this._targetLit = result.targetLit;
6548     }
6549 }
6550 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalPuzzleStarRating.ts -----
6551 import type { IOpticalLevelStarThresholds } from '../Config/OpticalPuzzleLevelSchema';
6552 /** 五角星显示状态 */
6553 export enum OpticalStarVisualState {
6554     /** 不发光: 键帽底色填充 */

```

```

6555     Dim = 1,
6556     /** 涂满：纯白填充，无边缘光晕 */
6557     Filled = 2,
6558     /** 发光：纯白填充 + 边缘光晕 */
6559     Glow = 3,
6560 }
6561 /** 三颗星槽位：左 / 上 / 右 */
6562 export type OpticalStarSlotStates = readonly [
6563     OpticalStarVisualState,
6564     OpticalStarVisualState,
6565     OpticalStarVisualState,
6566 ];
6567 /** 是否完美通关（步数 ≤ perfectSteps） */
6568 export function isPerfectClear(
6569     moveCount: number,
6570     thresholds: IOpticalLevelStarThresholds,
6571 ): boolean {
6572     return Math.max(0, Math.floor(moveCount)) <= thresholds.perfectSteps;
6573 }
6574 /**
6575  * 按通关步数与关卡阈值计算三颗星状态。
6576  * 左 = 一星线，中 = 二星线，右 = 三星线（完美时三颗均发光）。
6577  */
6578 export function resolveStarSlotStates(
6579     moveCount: number,
6580     thresholds: IOpticalLevelStarThresholds,
6581 ): OpticalStarSlotStates {
6582     const steps = Math.max(0, Math.floor(moveCount));
6583     if (isPerfectClear(steps, thresholds)) {
6584         return [OpticalStarVisualState.Glow, OpticalStarVisualState.Glow, OpticalStarVisualState.Glow];
6585     }
6586     if (steps <= thresholds.threeStarSteps) {
6587         return [OpticalStarVisualState.Filled, OpticalStarVisualState.Filled, OpticalStarVisualState.Filled];
6588     }
6589     if (steps <= thresholds.twoStarSteps) {
6590         return [OpticalStarVisualState.Filled, OpticalStarVisualState.Filled, OpticalStarVisualState.Dim];
6591     }
6592     if (steps <= thresholds.oneStarSteps) {
6593         return [OpticalStarVisualState.Filled, OpticalStarVisualState.Dim, OpticalStarVisualState.Dim];
6594     }
6595     return [OpticalStarVisualState.Dim, OpticalStarVisualState.Dim, OpticalStarVisualState.Dim];
6596 }
6597 /** 未通关前默认三颗星均为不发光 */
6598 export function defaultDimStarStates(): OpticalStarSlotStates {
6599     return [OpticalStarVisualState.Dim, OpticalStarVisualState.Dim, OpticalStarVisualState.Dim];
6600 }
6601 // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalPuzzleTypes.ts -----
6602 /** 网格四向，与文档一致：0 右 1 上 2 左 3 下（可按实现统一调整，全项目一致即可） */
6603 import type { PieceConnectivity } from './OpticalPieceConnectivity';
6604 export enum Direction {
6605     Right = 0,
6606     Up = 1,
6607     Left = 2,
6608     Down = 3,
6609 }
6610 const DIR_CHAR_TO_DIRECTION: Readonly<Record<string, Direction>> = {
6611     d: Direction.Right,
6612     w: Direction.Up,

```

```

6613     a: Direction.Left,
6614     s: Direction.Down,
6615 };
6616 /** 字面量 fallback, 避免微信包 enum 初始化顺序导致 Direction.Down 为 undefined */
6617 const DIR_DOWN_LITERAL = 3 as Direction;
6618 /**
6619  * 将关卡/存档中的朝向归一为 0~3 数值枚举。
6620  * 微信包 Rollup 后偶发 string (wasd、数字串或枚举名), 会导致 DIR_DX[dir] 为 undefined → 光追零
6621  * 段。
6622  */
6623 export function normalizeDirection(dir: unknown, fallback: Direction = DIR_DOWN_LITERAL): Direction {
6624     if (typeof dir === 'number' && Number.isFinite(dir)) {
6625         const n = Math.floor(dir);
6626         if (n >= 0 && n <= 3) {
6627             return n as Direction;
6628         }
6629     }
6630     if (typeof dir === 'string') {
6631         const trimmed = dir.trim();
6632         const fromChar = DIR_CHAR_TO_DIRECTION[trimmed] ??
6633         DIR_CHAR_TO_DIRECTION[trimmed.toLowerCase()];
6634         if (fromChar !== undefined) {
6635             return fromChar;
6636         }
6637         const asNum = Number(trimmed);
6638         if (Number.isFinite(asNum)) {
6639             const n = Math.floor(asNum);
6640             if (n >= 0 && n <= 3) {
6641                 return n as Direction;
6642             }
6643         }
6644         const fromEnum = Direction[trimmed as keyof typeof Direction];
6645         if (typeof fromEnum === 'number' && fromEnum >= 0 && fromEnum <= 3) {
6646             return fromEnum as Direction;
6647         }
6648     }
6649     const fb = typeof fallback === 'number' && fallback >= 0 && fallback <= 3 ? fallback : DIR_DOWN_LITERAL;
6650     return fb as Direction;
6651 }
6652 /** 地形 (与《小游戏设计文档》§ 3.2) */
6653 export enum TerrainKind {
6654     Floor = 0,
6655     Wall = 1,
6656     Source = 2,
6657     Target = 3,
6658 }
6659 /** 光学元件 type (§ 3.3) */
6660 export enum PieceType {
6661     Glass = 'glass',
6662     GlassH = 'glass-h',
6663     GlassV = 'glass-v',
6664     MirrorSlash = 'mirror-slash',
6665     MirrorBackslash = 'mirror-backslash',
6666     SplitterT = 'splitter-t',
6667     SplitterCross = 'splitter-cross',
6668     PortalA = 'portal-a',
6669     PortalB = 'portal-b',
6670 }

```

```

6671  /** 颜色模式 (§ 3.4) */
6672  export enum ColorMode {
6673      Through = 'through',
6674      FilterRed = 'filterRed',
6675      FilterGreen = 'filterGreen',
6676      FilterBlue = 'filterBlue',
6677  }
6678  /** 光源格 (与关卡 sources 顺序一致) */
6679  export interface OpticalSourceSnapshot {
6680      x: number;
6681      y: number;
6682      colorKey: string;
6683      direction: Direction;
6684  }
6685  /** 目标格点亮状态 (与关卡 targets 顺序一致) */
6686  export interface OpticalTargetSnapshot {
6687      x: number;
6688      y: number;
6689      colorKey: string;
6690      lit: boolean;
6691  }
6692  /** 光学元件 (通道 0~4+ 朝向 + 层 3 颜色, 用于棋盘绘制) */
6693  export interface OpticalPieceSnapshot {
6694      x: number;
6695      y: number;
6696      connectivity: PieceConnectivity;
6697      direction: Direction;
6698      colorKey: string;
6699  }
6700  /** Application / Presentation 使用的只读棋盘快照 */
6701  export interface OpticalBoardSnapshot {
6702      levelId: number;
6703      levelName: string;
6704      width: number;
6705      height: number;
6706      terrain: TerrainKind[];
6707      player: { x: number; y: number };
6708      /** 主角朝向 (决定双眼在格内的偏移, 默认左) */
6709      playerFacing: Direction;
6710      sources: OpticalSourceSnapshot[];
6711      targets: OpticalTargetSnapshot[];
6712      pieces: OpticalPieceSnapshot[];
6713      /** 全部目标已按对应颜色点亮 */
6714      allTargetsLit: boolean;
6715  }
6716  export enum MoveAttemptResult {
6717      /** 主角走入空地 */
6718      PlayerMoved = 'player_moved',
6719      /** 主角推动一块光学元件成功 (仅推动一格, 不可推两格连体) */
6720      PiecePushed = 'piece_pushed',
6721      /** 被墙、边界、光源/目标格或推不动挡住 */
6722      Blocked = 'blocked',
6723  }
6724  // ----- assets/Scripts/Games/OpticalPuzzle/Core/OpticalRuntimeCoerce.ts -----
6725  import { Direction, normalizeDirection, TerrainKind } from './OpticalPuzzleTypes';
6726  export const OPTICAL_DIR_DX: ReadonlyArray<number> = [1, 0, -1, 0];
6727  export const OPTICAL_DIR_DY: ReadonlyArray<number> = [0, -1, 0, 1];
6728  const DIR_DOWN = 3 as Direction;

```

```

6729  /** 微信包关卡坐标偶发 string, 与 number 相加会变成字符串拼接 → 光追 inBounds 失败 */
6730  export function normalizeGridCoord(value: unknown, fallback = 0): number {
6731      if (typeof value === 'number' && Number.isFinite(value)) {
6732          return Math.floor(value);
6733      }
6734      if (typeof value === 'string') {
6735          const n = Number(value.trim());
6736          if (Number.isFinite(n)) {
6737              return Math.floor(n);
6738          }
6739      }
6740      return fallback;
6741  }
6742  export function normalizeGridSize(value: unknown, fallback: number): number {
6743      const n = normalizeGridCoord(value, fallback);
6744      return n > 0 ? n : fallback;
6745  }
6746  export function opticalDirDelta(dir: unknown): { dx: number; dy: number; dir: Direction } | null {
6747      const normalized = normalizeDirection(dir, DIR_DOWN);
6748      const dx = OPTICAL_DIR_DX[normalized];
6749      const dy = OPTICAL_DIR_DY[normalized];
6750      if (dx === undefined || dy === undefined) {
6751          return null;
6752      }
6753      return { dx, dy, dir: normalized };
6754  }
6755  export function normalizeTerrainKind(value: unknown): number {
6756      if (typeof value === 'number' && Number.isFinite(value)) {
6757          return Math.floor(value);
6758      }
6759      if (typeof value === 'string') {
6760          const trimmed = value.trim();
6761          const asNum = Number(trimmed);
6762          if (Number.isFinite(asNum)) {
6763              return Math.floor(asNum);
6764          }
6765          const fromEnum = TerrainKind[trimmed as keyof typeof TerrainKind];
6766          if (typeof fromEnum === 'number') {
6767              return fromEnum;
6768          }
6769      }
6770      return -1;
6771  }
6772  export function terrainKindAt(terrain: readonly unknown[], index: number): number {
6773      if (index < 0 || index >= terrain.length) {
6774          return -1;
6775      }
6776      return normalizeTerrainKind(terrain[index]);
6777  }
6778  export function normalizePieceConnectivity(value: unknown): number {
6779      if (typeof value === 'number' && Number.isFinite(value)) {
6780          return Math.floor(value);
6781      }
6782      if (typeof value === 'string') {
6783          const n = Number(value.trim());
6784          if (Number.isFinite(n)) {
6785              return Math.floor(n);
6786          }

```

```

6787     }
6788     return 0;
6789 }
6790 // ----- assets/Scripts/Games/OpticalPuzzle/Infrastructure/OpticalPlayerPersist.ts -----
6791 import { getNextOpticalLevelId } from '../Config/OpticalPuzzleLevels';
6792 /**
6793  * 已解锁 / 已通关关卡记录（紧凑存储）。
6794  * 扁平数组 `[levelId, steps, levelId, steps, ...]`，JSON 里全是 number，无重复字段名。
6795  * - 真实通关：`steps` 为本关最少步数。
6796  * - 仅解锁未通：`steps === OPTICAL_LEVEL_UNLOCK_PLACEHOLDER_STEPS`（通关上一关时写入下一关，支
6797 持跳关）。
6798  * 某 levelId 未出现在数组中 = 未解锁；后续新增关卡 id 无需迁移旧档。
6799 */
6800 export type OpticalLevelClearsFlat = number[];
6801 /** 仅解锁、尚未真实通关时的占位步数（不参与评星与最佳步数展示） */
6802 export const OPTICAL_LEVEL_UNLOCK_PLACEHOLDER_STEPS = 999999;
6803 export function isOpticalUnlockPlaceholderSteps(steps: number): boolean {
6804     return steps >= OPTICAL_LEVEL_UNLOCK_PLACEHOLDER_STEPS;
6805 }
6806 function toFiniteNumber(raw: unknown): number | null {
6807     if (typeof raw === 'number' && Number.isFinite(raw)) {
6808         return raw;
6809     }
6810     if (typeof raw === 'string') {
6811         const trimmed = raw.trim();
6812         if (!trimmed) {
6813             return null;
6814         }
6815         const n = Number(trimmed);
6816         if (Number.isFinite(n)) {
6817             return n;
6818         }
6819     }
6820     if (raw != null && typeof raw === 'object') {
6821         const valueOf = (raw as { valueOf?: () => unknown }).valueOf;
6822         if (typeof valueOf === 'function') {
6823             const n = Number(valueOf.call(raw));
6824             if (Number.isFinite(n)) {
6825                 return n;
6826             }
6827         }
6828     }
6829     return null;
6830 }
6831 function normalizeLevelId(raw: unknown): number | null {
6832     const n = toFiniteNumber(raw);
6833     if (n == null) {
6834         return null;
6835     }
6836     const id = Math.trunc(n);
6837     return id > 0 ? id : null;
6838 }
6839 function normalizeBestSteps(raw: unknown): number | null {
6840     if (raw && typeof raw === 'object' && !Array.isArray(raw)) {
6841         const steps = (raw as Record<string, unknown>).bestSteps;
6842         const nested = normalizeBestSteps(steps);
6843         if (nested != null) {
6844             return nested;

```

```

6845     }
6846   }
6847   const n = toFiniteNumber(raw);
6848   if (n == null) {
6849     return null;
6850   }
6851   return Math.max(0, Math.floor(n));
6852 }
6853 /** 微信端勿依赖 Map（部分基础库实现异常）；用 plain object 聚合 */
6854 type LevelClearPairMap = Record<number, number>;
6855 function putLevelClearPair(pairMap: LevelClearPairMap, levelId: number, steps: number): void {
6856   if (levelId <= 0) {
6857     return;
6858   }
6859   const prev = pairMap[levelId];
6860   if (prev === undefined || steps < prev) {
6861     pairMap[levelId] = steps;
6862   }
6863 }
6864 function buildSortedFlatPairs(pairMap: LevelClearPairMap): OpticalLevelClearsFlat {
6865   const sortedIds = Object.keys(pairMap)
6866     .map((k) => Number.parseInt(k, 10))
6867     .filter((id) => Number.isFinite(id) && id > 0)
6868     .sort((a, b) => a - b);
6869   const out: OpticalLevelClearsFlat = [];
6870   for (const id of sortedIds) {
6871     const steps = pairMap[id];
6872     if (steps != null && Number.isFinite(steps)) {
6873       out.push(id, steps);
6874     }
6875   }
6876   return out;
6877 }
6878 /**
6879  * 从 JSON 还原 flat 数组；兼容 Array、类数组（含 length）、数字键对象。
6880  * 微信端 JSON.parse 后的数组偶发 `Array.isArray === false` 或仅有 length 可枚举。
6881  */
6882 function extractFlatPairValues(raw: unknown): unknown[] | null {
6883   if (raw == null) {
6884     return null;
6885   }
6886   if (Array.isArray(raw)) {
6887     return Array.from(raw);
6888   }
6889   if (typeof raw !== 'object') {
6890     return null;
6891   }
6892   const obj = raw as Record<string | number, unknown> & { length?: unknown };
6893   const lengthValue = toFiniteNumber(obj.length);
6894   if (lengthValue != null && lengthValue >= 0) {
6895     const len = Math.floor(lengthValue);
6896     if (len > 0 && len % 2 === 0) {
6897       const fromLength: unknown[] = [];
6898       let hasValue = false;
6899       for (let i = 0; i < len; i++) {
6900         const v = obj[i];
6901         fromLength.push(v);
6902         if (v !== undefined && v !== null) {

```

```

6903             hasValue = true;
6904         }
6905     }
6906     if (hasValue) {
6907         return fromLength;
6908     }
6909 }
6910 }
6911 const numericKeys = Object.keys(obj)
6912     .filter((k) => /^d+$/ .test(k))
6913     .sort((a, b) => Number(a) - Number(b));
6914 if (numericKeys.length > 0 && numericKeys[0] === '0') {
6915     return numericKeys.map((k) => obj[k]);
6916 }
6917 return null;
6918 }
6919 function ingestLegacyLevelMap(raw: Record<string, unknown>, pairMap: LevelClearPairMap): void {
6920     for (const [key, value] of Object.entries(raw)) {
6921         if (key === 'length') {
6922             continue;
6923         }
6924         const levelId = Number.parseInt(key, 10);
6925         const steps = normalizeBestSteps(value);
6926         if (Number.isFinite(levelId) && levelId > 0 && steps != null) {
6927             putLevelClearPair(pairMap, levelId, steps);
6928         }
6929     }
6930 }
6931 function ingestFlatPairs(values: unknown[], pairMap: LevelClearPairMap): void {
6932     for (let i = 0; i + 1 < values.length; i += 2) {
6933         const levelId = normalizeLevelId(values[i]);
6934         const steps = normalizeBestSteps(values[i + 1]);
6935         if (levelId != null && steps != null) {
6936             putLevelClearPair(pairMap, levelId, steps);
6937         }
6938     }
6939 }
6940 /** 已是合法 flat number 数组时直接拷贝，避免二次聚合丢数据 */
6941 function tryCopyValidFlatClears(raw: unknown): OpticalLevelClearsFlat | null {
6942     const values = extractFlatPairValues(raw);
6943     if (!values || values.length === 0 || values.length % 2 !== 0) {
6944         return null;
6945     }
6946     const out: OpticalLevelClearsFlat = [];
6947     for (let i = 0; i + 1 < values.length; i += 2) {
6948         const levelId = normalizeLevelId(values[i]);
6949         const steps = normalizeBestSteps(values[i + 1]);
6950         if (levelId == null || steps == null) {
6951             return null;
6952         }
6953         out.push(levelId, steps);
6954     }
6955     return out;
6956 }
6957 /** 读档合并：支持 flat 数组、JSON 字符串数组、旧版 `{"1": { bestSteps }}`、过渡版 `{"1": 4}` */
6958 export function mergeOpticalLevelClears(raw: unknown): OpticalLevelClearsFlat {
6959     if (raw == null) {
6960         return [];

```



```

6961     }
6962     if (typeof raw === 'string') {
6963         const trimmed = raw.trim().replace(/\^\uFEFF/, "");
6964         if (!trimmed) {
6965             return [];
6966         }
6967         try {
6968             return mergeOpticalLevelClears(JSON.parse(trimmed));
6969         } catch (e) {
6970             console.warn('[OpticalPlayerPersist] opticalLevelClears 字符串解析失败', trimmed.slice(0, 120),
6971 e);
6972             return [];
6973         }
6974     }
6975     const quick = tryCopyValidFlatClears(raw);
6976     if (quick && quick.length > 0) {
6977         return quick;
6978     }
6979     const pairMap: LevelClearPairMap = {};
6980     const flatValues = extractFlatPairValues(raw);
6981     if (flatValues) {
6982         ingestFlatPairs(flatValues, pairMap);
6983     } else if (typeof raw === 'object') {
6984         ingestLegacyLevelMap(raw as Record<string, unknown>, pairMap);
6985     }
6986     const built = buildSortedFlatPairs(pairMap);
6987     if (built.length > 0) {
6988         return built;
6989     }
6990     // 微信端：parse 后的 Array 可能 length 正确但下标读不到，stringify 再 parse 可恢复
6991     try {
6992         const roundTrip = JSON.stringify(raw);
6993         if (roundTrip.startsWith('[') {
6994             const viaJson = mergeOpticalLevelClears(roundTrip);
6995             if (viaJson.length > 0) {
6996                 return viaJson;
6997             }
6998         }
6999     } catch {
7000         /* ignore */
7001     }
7002     return built;
7003 }
7004 /** 从完整存档 JSON 文本中提取 opticalLevelClears（parse 后数组异常时的兜底） */
7005 export function mergeOpticalLevelClearsFromPlayerJsonText(jsonText: string): OpticalLevelClearsFlat {
7006     const match = jsonText.match(/"opticalLevelClears"\s*:\s*(\[([^\d\s,]*)\])/);
7007     if (!match) {
7008         return [];
7009     }
7010     try {
7011         return mergeOpticalLevelClears(match[1]);
7012     } catch {
7013         return [];
7014     }
7015 }
7016 export function getOpticalLevelRawStepsFromFlat(
7017     clears: OpticalLevelClearsFlat,
7018     levelId: number,

```

```
7019 ): number | null {
7020     const id = Math.trunc(levelId);
7021     const normalized = mergeOpticalLevelClears(clears);
7022     for (let i = 0; i + 1 < normalized.length; i += 2) {
7023         if (normalized[i] === id) {
7024             return normalized[i + 1];
7025         }
7026     }
7027     return null;
7028 }
7029 /** 是否在 clears 中有该关记录（含仅解锁占位） */
7030 export function hasOpticalLevelRecordInFlat(clears: OpticalLevelClearsFlat, levelId: number): boolean {
7031     return getOpticalLevelRawStepsFromFlat(clears, levelId) != null;
7032 }
7033 export function isOpticalLevelUnlockedInFlat(clears: OpticalLevelClearsFlat, levelId: number): boolean {
7034     return hasOpticalLevelRecordInFlat(clears, levelId);
7035 }
7036 /** 真实通关最少步数；仅解锁占位返回 null */
7037 export function getOpticalLevelBestStepsFromFlat(
7038     clears: OpticalLevelClearsFlat,
7039     levelId: number,
7040 ): number | null {
7041     const raw = getOpticalLevelRawStepsFromFlat(clears, levelId);
7042     if (raw == null || isOpticalUnlockPlaceholderSteps(raw)) {
7043         return null;
7044     }
7045     return raw;
7046 }
7047 export function isOpticalLevelClearedInFlat(clears: OpticalLevelClearsFlat, levelId: number): boolean {
7048     return getOpticalLevelBestStepsFromFlat(clears, levelId) != null;
7049 }
7050 /** 写入/刷新最少步数（更少才更新）；始终返回规范化后的新 flat 数组，避免污染原数组 */
7051 export function upsertOpticalLevelBestSteps(
7052     clears: OpticalLevelClearsFlat,
7053     levelId: number,
7054     steps: number,
7055 ): { changed: boolean; clears: OpticalLevelClearsFlat } {
7056     const id = Math.trunc(levelId);
7057     const bestSteps = Math.max(0, Math.floor(steps));
7058     if (id <= 0) {
7059         return { changed: false, clears: mergeOpticalLevelClears(clears) };
7060     }
7061     const next = mergeOpticalLevelClears(clears);
7062     for (let i = 0; i + 1 < next.length; i += 2) {
7063         if (next[i] !== id) {
7064             continue;
7065         }
7066         if (bestSteps >= next[i + 1]) {
7067             return { changed: false, clears: next };
7068         }
7069         next[i + 1] = bestSteps;
7070         return { changed: true, clears: next };
7071     }
7072     next.push(id, bestSteps);
7073     return { changed: true, clears: next };
7074 }
7075 /** 该关尚无记录时写入解锁占位；已有记录（含真实步数）则不覆盖 */
7076 export function ensureOpticalLevelUnlockInFlat(
```

```

7077     clears: OpticalLevelClearsFlat,
7078     levelId: number,
7079 ): { changed: boolean; clears: OpticalLevelClearsFlat } {
7080     const id = Math.trunc(levelId);
7081     if (id <= 0) {
7082         return { changed: false, clears: mergeOpticalLevelClears(clears) };
7083     }
7084     if (hasOpticalLevelRecordInFlat(clears, id)) {
7085         return { changed: false, clears: mergeOpticalLevelClears(clears) };
7086     }
7087     const next = mergeOpticalLevelClears(clears);
7088     next.push(id, OPTICAL_LEVEL_UNLOCK_PLACEHOLDER_STEPS);
7089     return { changed: true, clears: next };
7090 }
7091 /** 通关后: 若存在下一关且下一关尚无记录, 则写入 (nextId, 999999) */
7092 export function markNextOpticalLevelUnlockedIfAbsent(
7093     clears: OpticalLevelClearsFlat,
7094     clearedLevelId: number,
7095 ): { changed: boolean; clears: OpticalLevelClearsFlat } {
7096     const nextId = getNextOpticalLevelId(Math.trunc(clearedLevelId));
7097     if (nextId == null) {
7098         return { changed: false, clears: mergeOpticalLevelClears(clears) };
7099     }
7100     return ensureOpticalLevelUnlockInFlat(clears, nextId);
7101 }
7102 /** 新档 / 旧档无第 1 关记录时, 保证第 1 关可玩 */
7103 export function ensureDefaultFirstLevelUnlock(clears: OpticalLevelClearsFlat): OpticalLevelClearsFlat {
7104     const { changed, clears: next } = ensureOpticalLevelUnlockInFlat(clears, 1);
7105     return changed ? next : mergeOpticalLevelClears(clears);
7106 }
7107 /** @deprecated 仅诊断用: clears 中出现过的最大 levelId; 选关解锁请用 isOpticalLevelUnlockedInFlat */
7108 export function resolveOpticalMaxUnlockedLevelId(
7109     opticalCurrentLevelId: number,
7110     clears: OpticalLevelClearsFlat,
7111 ): number {
7112     const normalized = mergeOpticalLevelClears(clears);
7113     let maxId = normalized.length === 0 ? Math.max(1, Math.trunc(opticalCurrentLevelId)) : 1;
7114     for (let i = 0; i + 1 < normalized.length; i += 2) {
7115         const id = normalized[i];
7116         if (Number.isFinite(id) && id > 0) {
7117             maxId = Math.max(maxId, id);
7118         }
7119     }
7120     return maxId;
7121 }
7122 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/GameSubtitlePathSegs.generated.ts -----
7123 // AUTO-GENERATED by tools/gen-game-title-paths.py (subtitle) — do not edit by hand
7124 export interface GameSubtitlePathSeg {
7125     readonly x: number;
7126     readonly y: number;
7127 }
7128 /** 归一化 path 包围盒宽高 (按字高归一化; 运行时 fit 进 UITransform) */
7129 export const GAME_SUBTITLE_NORM_W = 2.975543;
7130 export const GAME_SUBTITLE_NORM_H = 1.000000;
7131 /** 全部轮廓 (1px 描边; 含孔洞边缘) */
7132 export const GAME_SUBTITLE_OUTLINE_PATHS: ReadonlyArray<
7133     ReadonlyArray<Readonly<GameSubtitlePathSeg>>
7134 > = [

```

```
7135      [
7136          { x: 1.199728, y: 0.016304 },
7137          { x: 1.197011, y: 0.013587 },
7138          { x: 1.186141, y: 0.013587 },
7139          { x: 1.183424, y: 0.010870 },
7140          { x: 1.169837, y: 0.010870 },
7141          { x: 1.148098, y: -0.000000 },
7142          { x: 1.142663, y: -0.000000 },
7143          { x: 1.131793, y: -0.008152 },
7144          { x: 1.123641, y: -0.008152 },
7145          { x: 1.115489, y: -0.016304 },
7146          { x: 1.104620, y: -0.021739 },
7147          { x: 1.104620, y: -0.024457 },
7148          { x: 1.099185, y: -0.027174 },
7149          { x: 1.091033, y: -0.035326 },
7150          { x: 1.085598, y: -0.046196 },
7151          { x: 1.085598, y: -0.076087 },
7152          { x: 1.093750, y: -0.086957 },
7153          { x: 1.093750, y: -0.089674 },
7154          { x: 1.104620, y: -0.100543 },
7155          { x: 1.110054, y: -0.100543 },
7156          { x: 1.112772, y: -0.103261 },
7157          { x: 1.118207, y: -0.103261 },
7158          { x: 1.129076, y: -0.108696 },
7159          { x: 1.148098, y: -0.108696 },
7160          { x: 1.150815, y: -0.105978 },
7161          { x: 1.161685, y: -0.105978 },
7162          { x: 1.164402, y: -0.103261 },
7163          { x: 1.175272, y: -0.103261 },
7164          { x: 1.177989, y: -0.100543 },
7165          { x: 1.191576, y: -0.100543 },
7166          { x: 1.205163, y: -0.092391 },
7167          { x: 1.210598, y: -0.092391 },
7168          { x: 1.216033, y: -0.089674 },
7169          { x: 1.226902, y: -0.081522 },
7170          { x: 1.235054, y: -0.081522 },
7171          { x: 1.235054, y: -0.078804 },
7172          { x: 1.240489, y: -0.076087 },
7173          { x: 1.259511, y: -0.057065 },
7174          { x: 1.262228, y: -0.051630 },
7175          { x: 1.262228, y: -0.046196 },
7176          { x: 1.264946, y: -0.043478 },
7177          { x: 1.264946, y: -0.019022 },
7178          { x: 1.262228, y: -0.016304 },
7179          { x: 1.262228, y: -0.010870 },
7180          { x: 1.254076, y: -0.002717 },
7181          { x: 1.254076, y: -0.000000 },
7182          { x: 1.243207, y: 0.010870 },
7183          { x: 1.235054, y: 0.010870 },
7184          { x: 1.232337, y: 0.013587 },
7185          { x: 1.221467, y: 0.013587 },
7186          { x: 1.218750, y: 0.016304 },
7187      ],
7188      [
7189          { x: -1.158967, y: 0.070652 },
7190          { x: -1.177989, y: 0.067935 },
7191          { x: -1.216033, y: 0.048913 },
7192          { x: -1.224185, y: 0.048913 },
```

```
7193      { x: -1.248641, y: 0.032609 },
7194      { x: -1.259511, y: 0.029891 },
7195      { x: -1.278533, y: 0.013587 },
7196      { x: -1.289402, y: 0.010870 },
7197      { x: -1.330163, y: -0.027174 },
7198      { x: -1.335598, y: -0.027174 },
7199      { x: -1.373641, y: -0.065217 },
7200      { x: -1.403533, y: -0.103261 },
7201      { x: -1.444293, y: -0.173913 },
7202      { x: -1.482337, y: -0.274457 },
7203      { x: -1.482337, y: -0.290761 },
7204      { x: -1.487772, y: -0.304348 },
7205      { x: -1.487772, y: -0.342391 },
7206      { x: -1.476902, y: -0.366848 },
7207      { x: -1.447011, y: -0.383152 },
7208      { x: -1.425272, y: -0.383152 },
7209      { x: -1.395380, y: -0.366848 },
7210      { x: -1.379076, y: -0.336957 },
7211      { x: -1.368207, y: -0.304348 },
7212      { x: -1.368207, y: -0.290761 },
7213      { x: -1.349185, y: -0.247283 },
7214      { x: -1.343750, y: -0.222826 },
7215      { x: -1.327446, y: -0.190217 },
7216      { x: -1.313859, y: -0.173913 },
7217      { x: -1.311141, y: -0.163043 },
7218      { x: -1.289402, y: -0.133152 },
7219      { x: -1.273098, y: -0.119565 },
7220      { x: -1.264946, y: -0.105978 },
7221      { x: -1.210598, y: -0.062500 },
7222      { x: -1.172554, y: -0.043478 },
7223      { x: -1.118207, y: -0.024457 },
7224      { x: -1.101902, y: -0.010870 },
7225      { x: -1.088315, y: 0.021739 },
7226      { x: -1.091033, y: 0.040761 },
7227      { x: -1.099185, y: 0.057065 },
7228      { x: -1.120924, y: 0.070652 },
7229      ],
7230      [
7231      { x: -0.729620, y: 0.116848 },
7232      { x: -0.751359, y: 0.105978 },
7233      { x: -0.759511, y: 0.105978 },
7234      { x: -0.778533, y: 0.084239 },
7235      { x: -0.783967, y: 0.070652 },
7236      { x: -0.783967, y: 0.051630 },
7237      { x: -0.778533, y: 0.040761 },
7238      { x: -0.721467, y: -0.013587 },
7239      { x: -0.694293, y: -0.046196 },
7240      { x: -0.691576, y: -0.057065 },
7241      { x: -0.677989, y: -0.073370 },
7242      { x: -0.653533, y: -0.122283 },
7243      { x: -0.631793, y: -0.187500 },
7244      { x: -0.612772, y: -0.211957 },
7245      { x: -0.591033, y: -0.220109 },
7246      { x: -0.572011, y: -0.220109 },
7247      { x: -0.555707, y: -0.214674 },
7248      { x: -0.533967, y: -0.195652 },
7249      { x: -0.523098, y: -0.176630 },
7250      { x: -0.520380, y: -0.154891 },
```

```
7251      { x: -0.531250, y: -0.114130 },
7252      { x: -0.539402, y: -0.100543 },
7253      { x: -0.539402, y: -0.089674 },
7254      { x: -0.558424, y: -0.057065 },
7255      { x: -0.558424, y: -0.048913 },
7256      { x: -0.618207, y: 0.040761 },
7257      { x: -0.661685, y: 0.086957 },
7258      { x: -0.697011, y: 0.111413 },
7259      ],
7260      [
7261      { x: -0.283967, y: 0.269022 },
7262      { x: -0.343750, y: 0.250000 },
7263      { x: -0.381793, y: 0.225543 },
7264      { x: -0.408967, y: 0.198370 },
7265      { x: -0.436141, y: 0.157609 },
7266      { x: -0.444293, y: 0.130435 },
7267      { x: -0.449728, y: 0.125000 },
7268      { x: -0.460598, y: 0.089674 },
7269      { x: -0.466033, y: 0.046196 },
7270      { x: -0.466033, y: -0.021739 },
7271      { x: -0.460598, y: -0.040761 },
7272      { x: -0.460598, y: -0.062500 },
7273      { x: -0.447011, y: -0.097826 },
7274      { x: -0.438859, y: -0.108696 },
7275      { x: -0.438859, y: -0.116848 },
7276      { x: -0.400815, y: -0.157609 },
7277      { x: -0.379076, y: -0.173913 },
7278      { x: -0.343750, y: -0.187500 },
7279      { x: -0.305707, y: -0.195652 },
7280      { x: -0.262228, y: -0.192935 },
7281      { x: -0.221467, y: -0.176630 },
7282      { x: -0.210598, y: -0.176630 },
7283      { x: -0.186141, y: -0.157609 },
7284      { x: -0.180707, y: -0.157609 },
7285      { x: -0.153533, y: -0.130435 },
7286      { x: -0.118207, y: -0.081522 },
7287      { x: -0.115489, y: -0.067935 },
7288      { x: -0.101902, y: -0.043478 },
7289      { x: -0.088315, y: 0.016304 },
7290      { x: -0.085598, y: 0.048913 },
7291      { x: -0.085598, y: 0.086957 },
7292      { x: -0.096467, y: 0.154891 },
7293      { x: -0.115489, y: 0.187500 },
7294      { x: -0.115489, y: 0.195652 },
7295      { x: -0.134511, y: 0.220109 },
7296      { x: -0.172554, y: 0.250000 },
7297      { x: -0.213315, y: 0.266304 },
7298      ],
7299      [
7300      { x: -0.270380, y: 0.168478 },
7301      { x: -0.251359, y: 0.171196 },
7302      { x: -0.226902, y: 0.165761 },
7303      { x: -0.199728, y: 0.141304 },
7304      { x: -0.188859, y: 0.122283 },
7305      { x: -0.183424, y: 0.092391 },
7306      { x: -0.183424, y: 0.032609 },
7307      { x: -0.188859, y: 0.016304 },
7308      { x: -0.188859, y: -0.002717 },
```

```
7309      { x: -0.216033, y: -0.057065 },
7310      { x: -0.248641, y: -0.084239 },
7311      { x: -0.273098, y: -0.092391 },
7312      { x: -0.305707, y: -0.092391 },
7313      { x: -0.327446, y: -0.084239 },
7314      { x: -0.346467, y: -0.065217 },
7315      { x: -0.357337, y: -0.040761 },
7316      { x: -0.365489, y: -0.000000 },
7317      { x: -0.362772, y: 0.043478 },
7318      { x: -0.346467, y: 0.105978 },
7319      { x: -0.327446, y: 0.133152 },
7320      { x: -0.327446, y: 0.138587 },
7321      { x: -0.302989, y: 0.160326 },
7322      ],
7323      [
7324      { x: 0.849185, y: 0.277174 },
7325      { x: 0.816576, y: 0.271739 },
7326      { x: 0.759511, y: 0.250000 },
7327      { x: 0.716033, y: 0.220109 },
7328      { x: 0.677989, y: 0.182065 },
7329      { x: 0.677989, y: 0.176630 },
7330      { x: 0.664402, y: 0.163043 },
7331      { x: 0.658967, y: 0.146739 },
7332      { x: 0.642663, y: 0.125000 },
7333      { x: 0.639946, y: 0.111413 },
7334      { x: 0.623641, y: 0.084239 },
7335      { x: 0.604620, y: 0.027174 },
7336      { x: 0.601902, y: -0.002717 },
7337      { x: 0.593750, y: -0.029891 },
7338      { x: 0.588315, y: -0.081522 },
7339      { x: 0.591033, y: -0.152174 },
7340      { x: 0.601902, y: -0.211957 },
7341      { x: 0.620924, y: -0.255435 },
7342      { x: 0.645380, y: -0.290761 },
7343      { x: 0.677989, y: -0.317935 },
7344      { x: 0.697011, y: -0.328804 },
7345      { x: 0.756793, y: -0.347826 },
7346      { x: 0.808424, y: -0.347826 },
7347      { x: 0.849185, y: -0.336957 },
7348      { x: 0.903533, y: -0.309783 },
7349      { x: 0.947011, y: -0.271739 },
7350      { x: 0.979620, y: -0.228261 },
7351      { x: 1.004076, y: -0.179348 },
7352      { x: 1.023098, y: -0.119565 },
7353      { x: 1.023098, y: -0.103261 },
7354      { x: 1.042120, y: -0.019022 },
7355      { x: 1.042120, y: 0.111413 },
7356      { x: 1.023098, y: 0.173913 },
7357      { x: 1.004076, y: 0.206522 },
7358      { x: 0.985054, y: 0.225543 },
7359      { x: 0.982337, y: 0.233696 },
7360      { x: 0.947011, y: 0.258152 },
7361      { x: 0.903533, y: 0.274457 },
7362      ],
7363      [
7364      { x: 0.841033, y: 0.176630 },
7365      { x: 0.873641, y: 0.176630 },
7366      { x: 0.903533, y: 0.163043 },
```

```

7367      { x: 0.930707, y: 0.127717 },
7368      { x: 0.941576, y: 0.078804 },
7369      { x: 0.941576, y: 0.013587 },
7370      { x: 0.930707, y: -0.051630 },
7371      { x: 0.930707, y: -0.076087 },
7372      { x: 0.911685, y: -0.127717 },
7373      { x: 0.911685, y: -0.138587 },
7374      { x: 0.879076, y: -0.192935 },
7375      { x: 0.849185, y: -0.222826 },
7376      { x: 0.827446, y: -0.236413 },
7377      { x: 0.802989, y: -0.244565 },
7378      { x: 0.773098, y: -0.247283 },
7379      { x: 0.737772, y: -0.236413 },
7380      { x: 0.702446, y: -0.195652 },
7381      { x: 0.688859, y: -0.144022 },
7382      { x: 0.691576, y: -0.089674 },
7383      { x: 0.697011, y: -0.076087 },
7384      { x: 0.707880, y: -0.065217 },
7385      { x: 0.735054, y: -0.062500 },
7386      { x: 0.737772, y: -0.081522 },
7387      { x: 0.732337, y: -0.108696 },
7388      { x: 0.737772, y: -0.127717 },
7389      { x: 0.748641, y: -0.138587 },
7390      { x: 0.759511, y: -0.141304 },
7391      { x: 0.792120, y: -0.138587 },
7392      { x: 0.830163, y: -0.100543 },
7393      { x: 0.843750, y: -0.059783 },
7394      { x: 0.841033, y: -0.027174 },
7395      { x: 0.827446, y: 0.002717 },
7396      { x: 0.792120, y: 0.027174 },
7397      { x: 0.756793, y: 0.029891 },
7398      { x: 0.718750, y: 0.019022 },
7399      { x: 0.710598, y: 0.027174 },
7400      { x: 0.710598, y: 0.032609 },
7401      { x: 0.718750, y: 0.059783 },
7402      { x: 0.735054, y: 0.089674 },
7403      { x: 0.773098, y: 0.141304 },
7404      { x: 0.802989, y: 0.165761 },
7405      { x: 0.811141, y: 0.165761 },
7406      { x: 0.830163, y: 0.176630 },
7407      ],
7408      [
7409      { x: -0.014946, y: 0.361413 },
7410      { x: -0.042120, y: 0.347826 },
7411      { x: -0.058424, y: 0.326087 },
7412      { x: -0.061141, y: 0.307065 },
7413      { x: -0.052989, y: 0.279891 },
7414      { x: 0.009511, y: 0.165761 },
7415      { x: 0.031250, y: 0.146739 },
7416      { x: 0.077446, y: 0.146739 },
7417      { x: 0.096467, y: 0.163043 },
7418      { x: 0.104620, y: 0.182065 },
7419      { x: 0.104620, y: 0.201087 },
7420      { x: 0.096467, y: 0.225543 },
7421      { x: 0.039402, y: 0.334239 },
7422      { x: 0.017663, y: 0.355978 },
7423      { x: 0.006793, y: 0.361413 },
7424      ],

```



```
7425 [
7426     { x: 0.427989, y: 0.432065 },
7427     { x: 0.414402, y: 0.426630 },
7428     { x: 0.392663, y: 0.404891 },
7429     { x: 0.389946, y: 0.394022 },
7430     { x: 0.384511, y: 0.391304 },
7431     { x: 0.351902, y: 0.342391 },
7432     { x: 0.351902, y: 0.336957 },
7433     { x: 0.332880, y: 0.315217 },
7434     { x: 0.332880, y: 0.307065 },
7435     { x: 0.313859, y: 0.277174 },
7436     { x: 0.313859, y: 0.230978 },
7437     { x: 0.332880, y: 0.217391 },
7438     { x: 0.349185, y: 0.214674 },
7439     { x: 0.379076, y: 0.222826 },
7440     { x: 0.403533, y: 0.241848 },
7441     { x: 0.463315, y: 0.317935 },
7442     { x: 0.466033, y: 0.328804 },
7443     { x: 0.482337, y: 0.353261 },
7444     { x: 0.487772, y: 0.372283 },
7445     { x: 0.487772, y: 0.399457 },
7446     { x: 0.482337, y: 0.413043 },
7447     { x: 0.463315, y: 0.429348 },
7448 ],
7449 [
7450     { x: -1.001359, y: 0.453804 },
7451     { x: -1.023098, y: 0.445652 },
7452     { x: -1.039402, y: 0.429348 },
7453     { x: -1.044837, y: 0.394022 },
7454     { x: -1.039402, y: 0.364130 },
7455     { x: -1.017663, y: 0.298913 },
7456     { x: -1.017663, y: 0.279891 },
7457     { x: -0.998641, y: 0.190217 },
7458     { x: -0.993207, y: 0.130435 },
7459     { x: -0.990489, y: -0.035326 },
7460     { x: -0.998641, y: -0.086957 },
7461     { x: -0.998641, y: -0.116848 },
7462     { x: -1.017663, y: -0.187500 },
7463     { x: -1.020380, y: -0.211957 },
7464     { x: -1.061141, y: -0.298913 },
7465     { x: -1.088315, y: -0.331522 },
7466     { x: -1.137228, y: -0.369565 },
7467     { x: -1.150815, y: -0.375000 },
7468     { x: -1.172554, y: -0.396739 },
7469     { x: -1.177989, y: -0.413043 },
7470     { x: -1.172554, y: -0.440217 },
7471     { x: -1.137228, y: -0.461957 },
7472     { x: -1.082880, y: -0.459239 },
7473     { x: -1.055707, y: -0.442935 },
7474     { x: -1.047554, y: -0.442935 },
7475     { x: -1.001359, y: -0.402174 },
7476     { x: -0.968750, y: -0.361413 },
7477     { x: -0.944293, y: -0.317935 },
7478     { x: -0.944293, y: -0.309783 },
7479     { x: -0.925272, y: -0.271739 },
7480     { x: -0.925272, y: -0.258152 },
7481     { x: -0.906250, y: -0.203804 },
7482     { x: -0.906250, y: -0.187500 },
```

```
7483      { x: -0.887228, y: -0.100543 },
7484      { x: -0.881793, y: 0.029891 },
7485      { x: -0.887228, y: 0.173913 },
7486      { x: -0.908967, y: 0.315217 },
7487      { x: -0.925272, y: 0.369565 },
7488      { x: -0.925272, y: 0.383152 },
7489      { x: -0.944293, y: 0.421196 },
7490      { x: -0.944293, y: 0.429348 },
7491      { x: -0.966033, y: 0.448370 },
7492  ],
7493  [
7494      { x: 0.851902, y: 0.467391 },
7495      { x: 0.699728, y: 0.391304 },
7496      { x: 0.683424, y: 0.366848 },
7497      { x: 0.683424, y: 0.342391 },
7498      { x: 0.699728, y: 0.320652 },
7499      { x: 0.718750, y: 0.312500 },
7500      { x: 0.748641, y: 0.312500 },
7501      { x: 0.759511, y: 0.317935 },
7502      { x: 0.773098, y: 0.317935 },
7503      { x: 0.797554, y: 0.331522 },
7504      { x: 0.819293, y: 0.336957 },
7505      { x: 0.851902, y: 0.355978 },
7506      { x: 0.860054, y: 0.355978 },
7507      { x: 0.889946, y: 0.375000 },
7508      { x: 0.908967, y: 0.394022 },
7509      { x: 0.914402, y: 0.413043 },
7510      { x: 0.914402, y: 0.434783 },
7511      { x: 0.908967, y: 0.448370 },
7512      { x: 0.889946, y: 0.464674 },
7513  ],
7514  [
7515      { x: 1.226902, y: 0.470109 },
7516      { x: 1.213315, y: 0.467391 },
7517      { x: 1.199728, y: 0.453804 },
7518      { x: 1.186141, y: 0.426630 },
7519      { x: 1.180707, y: 0.396739 },
7520      { x: 1.161685, y: 0.350543 },
7521      { x: 1.161685, y: 0.339674 },
7522      { x: 1.148098, y: 0.315217 },
7523      { x: 1.120924, y: 0.233696 },
7524      { x: 1.107337, y: 0.209239 },
7525      { x: 1.085598, y: 0.152174 },
7526      { x: 1.088315, y: 0.119565 },
7527      { x: 1.110054, y: 0.103261 },
7528      { x: 1.137228, y: 0.103261 },
7529      { x: 1.142663, y: 0.108696 },
7530      { x: 1.156250, y: 0.111413 },
7531      { x: 1.177989, y: 0.133152 },
7532      { x: 1.216033, y: 0.184783 },
7533      { x: 1.232337, y: 0.190217 },
7534      { x: 1.281250, y: 0.190217 },
7535      { x: 1.316576, y: 0.184783 },
7536      { x: 1.349185, y: 0.165761 },
7537      { x: 1.368207, y: 0.138587 },
7538      { x: 1.379076, y: 0.086957 },
7539      { x: 1.379076, y: 0.032609 },
7540      { x: 1.373641, y: 0.008152 },
```

```
7541      { x: 1.373641, y: -0.019022 },
7542      { x: 1.354620, y: -0.070652 },
7543      { x: 1.354620, y: -0.081522 },
7544      { x: 1.335598, y: -0.122283 },
7545      { x: 1.335598, y: -0.130435 },
7546      { x: 1.313859, y: -0.168478 },
7547      { x: 1.281250, y: -0.211957 },
7548      { x: 1.237772, y: -0.255435 },
7549      { x: 1.167120, y: -0.312500 },
7550      { x: 1.150815, y: -0.336957 },
7551      { x: 1.150815, y: -0.361413 },
7552      { x: 1.161685, y: -0.380435 },
7553      { x: 1.167120, y: -0.385870 },
7554      { x: 1.205163, y: -0.399457 },
7555      { x: 1.235054, y: -0.394022 },
7556      { x: 1.278533, y: -0.366848 },
7557      { x: 1.316576, y: -0.328804 },
7558      { x: 1.322011, y: -0.328804 },
7559      { x: 1.330163, y: -0.320652 },
7560      { x: 1.332880, y: -0.312500 },
7561      { x: 1.351902, y: -0.296196 },
7562      { x: 1.379076, y: -0.258152 },
7563      { x: 1.387228, y: -0.252717 },
7564      { x: 1.389946, y: -0.241848 },
7565      { x: 1.408967, y: -0.217391 },
7566      { x: 1.444293, y: -0.149457 },
7567      { x: 1.449728, y: -0.125000 },
7568      { x: 1.468750, y: -0.078804 },
7569      { x: 1.468750, y: -0.062500 },
7570      { x: 1.487772, y: 0.013587 },
7571      { x: 1.487772, y: 0.122283 },
7572      { x: 1.463315, y: 0.198370 },
7573      { x: 1.433424, y: 0.239130 },
7574      { x: 1.389946, y: 0.269022 },
7575      { x: 1.335598, y: 0.282609 },
7576      { x: 1.262228, y: 0.282609 },
7577      { x: 1.254076, y: 0.290761 },
7578      { x: 1.294837, y: 0.394022 },
7579      { x: 1.294837, y: 0.434783 },
7580      { x: 1.273098, y: 0.461957 },
7581      { x: 1.262228, y: 0.467391 },
7582      ],
7583      [
7584      { x: 0.197011, y: 0.500000 },
7585      { x: 0.164402, y: 0.483696 },
7586      { x: 0.153533, y: 0.451087 },
7587      { x: 0.153533, y: 0.119565 },
7588      { x: 0.137228, y: 0.105978 },
7589      { x: 0.120924, y: 0.105978 },
7590      { x: 0.052989, y: 0.086957 },
7591      { x: 0.031250, y: 0.086957 },
7592      { x: -0.033967, y: 0.067935 },
7593      { x: -0.052989, y: 0.043478 },
7594      { x: -0.052989, y: 0.021739 },
7595      { x: -0.042120, y: -0.005435 },
7596      { x: -0.017663, y: -0.024457 },
7597      { x: 0.031250, y: -0.024457 },
7598      { x: 0.123641, y: -0.000000 },
```

7599	{ x: 0.134511, y: -0.000000 },
7600	{ x: 0.139946, y: -0.005435 },
7601	{ x: 0.139946, y: -0.024457 },
7602	{ x: 0.118207, y: -0.043478 },
7603	{ x: 0.107337, y: -0.046196 },
7604	{ x: 0.085598, y: -0.065217 },
7605	{ x: 0.080163, y: -0.065217 },
7606	{ x: 0.055707, y: -0.086957 },
7607	{ x: -0.033967, y: -0.179348 },
7608	{ x: -0.033967, y: -0.184783 },
7609	{ x: -0.074728, y: -0.222826 },
7610	{ x: -0.080163, y: -0.233696 },
7611	{ x: -0.145380, y: -0.293478 },
7612	{ x: -0.150815, y: -0.293478 },
7613	{ x: -0.188859, y: -0.331522 },
7614	{ x: -0.205163, y: -0.361413 },
7615	{ x: -0.202446, y: -0.380435 },
7616	{ x: -0.186141, y: -0.404891 },
7617	{ x: -0.161685, y: -0.415761 },
7618	{ x: -0.145380, y: -0.415761 },
7619	{ x: -0.126359, y: -0.404891 },
7620	{ x: -0.118207, y: -0.404891 },
7621	{ x: -0.036685, y: -0.336957 },
7622	{ x: 0.033967, y: -0.266304 },
7623	{ x: 0.096467, y: -0.190217 },
7624	{ x: 0.118207, y: -0.171196 },
7625	{ x: 0.126359, y: -0.179348 },
7626	{ x: 0.120924, y: -0.244565 },
7627	{ x: 0.099185, y: -0.358696 },
7628	{ x: 0.099185, y: -0.383152 },
7629	{ x: 0.082880, y: -0.442935 },
7630	{ x: 0.082880, y: -0.467391 },
7631	{ x: 0.088315, y: -0.480978 },
7632	{ x: 0.115489, y: -0.500000 },
7633	{ x: 0.142663, y: -0.500000 },
7634	{ x: 0.153533, y: -0.494565 },
7635	{ x: 0.175272, y: -0.475543 },
7636	{ x: 0.194293, y: -0.437500 },
7637	{ x: 0.194293, y: -0.421196 },
7638	{ x: 0.213315, y: -0.345109 },
7639	{ x: 0.213315, y: -0.315217 },
7640	{ x: 0.232337, y: -0.201087 },
7641	{ x: 0.235054, y: -0.138587 },
7642	{ x: 0.243207, y: -0.119565 },
7643	{ x: 0.251359, y: -0.119565 },
7644	{ x: 0.256793, y: -0.125000 },
7645	{ x: 0.273098, y: -0.144022 },
7646	{ x: 0.278533, y: -0.157609 },
7647	{ x: 0.286685, y: -0.163043 },
7648	{ x: 0.292120, y: -0.176630 },
7649	{ x: 0.370924, y: -0.269022 },
7650	{ x: 0.389946, y: -0.285326 },
7651	{ x: 0.389946, y: -0.290761 },
7652	{ x: 0.406250, y: -0.301630 },
7653	{ x: 0.427989, y: -0.326087 },
7654	{ x: 0.444293, y: -0.334239 },
7655	{ x: 0.452446, y: -0.345109 },
7656	{ x: 0.479620, y: -0.364130 },

```

7657      { x: 0.506793, y: -0.375000 },
7658      { x: 0.539402, y: -0.372283 },
7659      { x: 0.561141, y: -0.358696 },
7660      { x: 0.574728, y: -0.326087 },
7661      { x: 0.561141, y: -0.293478 },
7662      { x: 0.528533, y: -0.269022 },
7663      { x: 0.517663, y: -0.255435 },
7664      { x: 0.512228, y: -0.255435 },
7665      { x: 0.444293, y: -0.187500 },
7666      { x: 0.441576, y: -0.179348 },
7667      { x: 0.425272, y: -0.165761 },
7668      { x: 0.425272, y: -0.160326 },
7669      { x: 0.387228, y: -0.119565 },
7670      { x: 0.387228, y: -0.114130 },
7671      { x: 0.370924, y: -0.097826 },
7672      { x: 0.365489, y: -0.084239 },
7673      { x: 0.330163, y: -0.046196 },
7674      { x: 0.305707, y: -0.027174 },
7675      { x: 0.259511, y: -0.016304 },
7676      { x: 0.251359, y: -0.010870 },
7677      { x: 0.251359, y: 0.021739 },
7678      { x: 0.262228, y: 0.029891 },
7679      { x: 0.292120, y: 0.032609 },
7680      { x: 0.362772, y: 0.051630 },
7681      { x: 0.381793, y: 0.051630 },
7682      { x: 0.460598, y: 0.078804 },
7683      { x: 0.482337, y: 0.097826 },
7684      { x: 0.490489, y: 0.135870 },
7685      { x: 0.479620, y: 0.163043 },
7686      { x: 0.457880, y: 0.176630 },
7687      { x: 0.417120, y: 0.173913 },
7688      { x: 0.387228, y: 0.163043 },
7689      { x: 0.351902, y: 0.157609 },
7690      { x: 0.324728, y: 0.146739 },
7691      { x: 0.300272, y: 0.144022 },
7692      { x: 0.273098, y: 0.133152 },
7693      { x: 0.262228, y: 0.133152 },
7694      { x: 0.254076, y: 0.141304 },
7695      { x: 0.256793, y: 0.456522 },
7696      { x: 0.251359, y: 0.478261 },
7697      { x: 0.243207, y: 0.486413 },
7698      { x: 0.224185, y: 0.497283 },
7699      ],
7700 ];
7701 /**
7702  * 挖孔三角化填充（归一化坐标；每 6 个数一个三角形 x0,y0,x1,y1,x2,y2）。
7703  * 孔洞区域不在三角网内，运行时只 fill 三角形。
7704  */
7705 export const GAME_SUBTITLE_FILL_TRI_FLAT: readonly number[] = [
7706     1.221467, 0.013587, 1.218750, 0.016304, 1.199728, 0.016304,
7707     1.197011, 0.013587, 1.186141, 0.013587, 1.183424, 0.010870,
7708     1.183424, 0.010870, 1.169837, 0.010870, 1.148098, -0.000000,
7709     1.148098, -0.000000, 1.142663, -0.000000, 1.131793, -0.008152,
7710     1.131793, -0.008152, 1.123641, -0.008152, 1.115489, -0.016304,
7711     1.115489, -0.016304, 1.104620, -0.021739, 1.104620, -0.024457,
7712     1.104620, -0.024457, 1.099185, -0.027174, 1.091033, -0.035326,
7713     1.091033, -0.035326, 1.085598, -0.046196, 1.085598, -0.076087,
7714     1.093750, -0.086957, 1.093750, -0.089674, 1.104620, -0.100543,

```

7715 1.110054, -0.100543, 1.112772, -0.103261, 1.118207, -0.103261,
7716 1.118207, -0.103261, 1.129076, -0.108696, 1.148098, -0.108696,
7717 1.150815, -0.105978, 1.161685, -0.105978, 1.164402, -0.103261,
7718 1.164402, -0.103261, 1.175272, -0.103261, 1.177989, -0.100543,
7719 1.177989, -0.100543, 1.191576, -0.100543, 1.205163, -0.092391,
7720 1.205163, -0.092391, 1.210598, -0.092391, 1.216033, -0.089674,
7721 1.226902, -0.081522, 1.235054, -0.081522, 1.235054, -0.078804,
7722 1.235054, -0.078804, 1.240489, -0.076087, 1.259511, -0.057065,
7723 1.259511, -0.057065, 1.262228, -0.051630, 1.262228, -0.046196,
7724 1.262228, -0.046196, 1.264946, -0.043478, 1.264946, -0.019022,
7725 1.262228, -0.016304, 1.262228, -0.010870, 1.254076, -0.002717,
7726 1.254076, -0.002717, 1.254076, -0.000000, 1.243207, 0.010870,
7727 1.235054, 0.010870, 1.232337, 0.013587, 1.221467, 0.013587,
7728 1.221467, 0.013587, 1.199728, 0.016304, 1.197011, 0.013587,
7729 1.197011, 0.013587, 1.183424, 0.010870, 1.148098, -0.000000,
7730 1.131793, -0.008152, 1.115489, -0.016304, 1.104620, -0.024457,
7731 1.104620, -0.024457, 1.091033, -0.035326, 1.085598, -0.076087,
7732 1.085598, -0.076087, 1.093750, -0.086957, 1.104620, -0.100543,
7733 1.110054, -0.100543, 1.118207, -0.103261, 1.148098, -0.108696,
7734 1.164402, -0.103261, 1.177989, -0.100543, 1.205163, -0.092391,
7735 1.205163, -0.092391, 1.216033, -0.089674, 1.226902, -0.081522,
7736 1.226902, -0.081522, 1.235054, -0.078804, 1.259511, -0.057065,
7737 1.259511, -0.057065, 1.262228, -0.046196, 1.264946, -0.019022,
7738 1.262228, -0.016304, 1.254076, -0.002717, 1.243207, 0.010870,
7739 1.235054, 0.010870, 1.221467, 0.013587, 1.197011, 0.013587,
7740 1.197011, 0.013587, 1.148098, -0.000000, 1.131793, -0.008152,
7741 1.131793, -0.008152, 1.104620, -0.024457, 1.085598, -0.076087,
7742 1.085598, -0.076087, 1.104620, -0.100543, 1.110054, -0.100543,
7743 1.110054, -0.100543, 1.148098, -0.108696, 1.150815, -0.105978,
7744 1.150815, -0.105978, 1.164402, -0.103261, 1.205163, -0.092391,
7745 1.205163, -0.092391, 1.226902, -0.081522, 1.259511, -0.057065,
7746 1.259511, -0.057065, 1.264946, -0.019022, 1.262228, -0.016304,
7747 1.262228, -0.016304, 1.243207, 0.010870, 1.235054, 0.010870,
7748 1.235054, 0.010870, 1.197011, 0.013587, 1.131793, -0.008152,
7749 1.131793, -0.008152, 1.085598, -0.076087, 1.110054, -0.100543,
7750 1.110054, -0.100543, 1.150815, -0.105978, 1.205163, -0.092391,
7751 1.205163, -0.092391, 1.259511, -0.057065, 1.262228, -0.016304,
7752 1.262228, -0.016304, 1.235054, 0.010870, 1.131793, -0.008152,
7753 1.131793, -0.008152, 1.110054, -0.100543, 1.205163, -0.092391,
7754 1.205163, -0.092391, 1.262228, -0.016304, 1.131793, -0.008152,
7755 -1.099185, 0.057065, -1.120924, 0.070652, -1.158967, 0.070652,
7756 -1.158967, 0.070652, -1.177989, 0.067935, -1.216033, 0.048913,
7757 -1.216033, 0.048913, -1.224185, 0.048913, -1.248641, 0.032609,
7758 -1.248641, 0.032609, -1.259511, 0.029891, -1.278533, 0.013587,
7759 -1.278533, 0.013587, -1.289402, 0.010870, -1.330163, -0.027174,
7760 -1.330163, -0.027174, -1.335598, -0.027174, -1.373641, -0.065217,
7761 -1.373641, -0.065217, -1.403533, -0.103261, -1.444293, -0.173913,
7762 -1.444293, -0.173913, -1.482337, -0.274457, -1.482337, -0.290761,
7763 -1.482337, -0.290761, -1.487772, -0.304348, -1.487772, -0.342391,
7764 -1.487772, -0.342391, -1.476902, -0.366848, -1.447011, -0.383152,
7765 -1.447011, -0.383152, -1.425272, -0.383152, -1.395380, -0.366848,
7766 -1.395380, -0.366848, -1.379076, -0.336957, -1.368207, -0.304348,
7767 -1.368207, -0.290761, -1.349185, -0.247283, -1.343750, -0.222826,
7768 -1.327446, -0.190217, -1.313859, -0.173913, -1.311141, -0.163043,
7769 -1.289402, -0.133152, -1.273098, -0.119565, -1.264946, -0.105978,
7770 -1.172554, -0.043478, -1.118207, -0.024457, -1.101902, -0.010870,
7771 -1.101902, -0.010870, -1.088315, 0.021739, -1.091033, 0.040761,
7772 -1.091033, 0.040761, -1.099185, 0.057065, -1.158967, 0.070652,

7773 -1.158967, 0.070652, -1.216033, 0.048913, -1.248641, 0.032609,
7774 -1.248641, 0.032609, -1.278533, 0.013587, -1.330163, -0.027174,
7775 -1.330163, -0.027174, -1.373641, -0.065217, -1.444293, -0.173913,
7776 -1.444293, -0.173913, -1.482337, -0.290761, -1.487772, -0.342391,
7777 -1.487772, -0.342391, -1.447011, -0.383152, -1.395380, -0.366848,
7778 -1.395380, -0.366848, -1.368207, -0.304348, -1.368207, -0.290761,
7779 -1.172554, -0.043478, -1.101902, -0.010870, -1.091033, 0.040761,
7780 -1.091033, 0.040761, -1.158967, 0.070652, -1.248641, 0.032609,
7781 -1.248641, 0.032609, -1.330163, -0.027174, -1.444293, -0.173913,
7782 -1.444293, -0.173913, -1.487772, -0.342391, -1.395380, -0.366848,
7783 -1.210598, -0.062500, -1.172554, -0.043478, -1.091033, 0.040761,
7784 -1.091033, 0.040761, -1.248641, 0.032609, -1.444293, -0.173913,
7785 -1.444293, -0.173913, -1.395380, -0.366848, -1.368207, -0.290761,
7786 -1.264946, -0.105978, -1.210598, -0.062500, -1.091033, 0.040761,
7787 -1.444293, -0.173913, -1.368207, -0.290761, -1.343750, -0.222826,
7788 -1.264946, -0.105978, -1.091033, 0.040761, -1.444293, -0.173913,
7789 -1.444293, -0.173913, -1.343750, -0.222826, -1.327446, -0.190217,
7790 -1.289402, -0.133152, -1.264946, -0.105978, -1.444293, -0.173913,
7791 -1.444293, -0.173913, -1.327446, -0.190217, -1.311141, -0.163043,
7792 -1.311141, -0.163043, -1.289402, -0.133152, -1.444293, -0.173913,
7793 -0.661685, 0.086957, -0.697011, 0.111413, -0.729620, 0.116848,
7794 -0.751359, 0.105978, -0.759511, 0.105978, -0.778533, 0.084239,
7795 -0.778533, 0.084239, -0.783967, 0.070652, -0.783967, 0.051630,
7796 -0.783967, 0.051630, -0.778533, 0.040761, -0.721467, -0.013587,
7797 -0.694293, -0.046196, -0.691576, -0.057065, -0.677989, -0.073370,
7798 -0.653533, -0.122283, -0.631793, -0.187500, -0.612772, -0.211957,
7799 -0.612772, -0.211957, -0.591033, -0.220109, -0.572011, -0.220109,
7800 -0.572011, -0.220109, -0.555707, -0.214674, -0.533967, -0.195652,
7801 -0.533967, -0.195652, -0.523098, -0.176630, -0.520380, -0.154891,
7802 -0.520380, -0.154891, -0.531250, -0.114130, -0.539402, -0.100543,
7803 -0.539402, -0.100543, -0.539402, -0.089674, -0.558424, -0.057065,
7804 -0.558424, -0.057065, -0.558424, -0.048913, -0.618207, 0.040761,
7805 -0.618207, 0.040761, -0.661685, 0.086957, -0.729620, 0.116848,
7806 -0.729620, 0.116848, -0.751359, 0.105978, -0.778533, 0.084239,
7807 -0.778533, 0.084239, -0.783967, 0.051630, -0.721467, -0.013587,
7808 -0.653533, -0.122283, -0.612772, -0.211957, -0.572011, -0.220109,
7809 -0.572011, -0.220109, -0.533967, -0.195652, -0.520380, -0.154891,
7810 -0.520380, -0.154891, -0.539402, -0.100543, -0.558424, -0.057065,
7811 -0.558424, -0.057065, -0.618207, 0.040761, -0.729620, 0.116848,
7812 -0.729620, 0.116848, -0.778533, 0.084239, -0.721467, -0.013587,
7813 -0.677989, -0.073370, -0.653533, -0.122283, -0.572011, -0.220109,
7814 -0.572011, -0.220109, -0.520380, -0.154891, -0.558424, -0.057065,
7815 -0.558424, -0.057065, -0.729620, 0.116848, -0.721467, -0.013587,
7816 -0.694293, -0.046196, -0.677989, -0.073370, -0.572011, -0.220109,
7817 -0.558424, -0.057065, -0.721467, -0.013587, -0.694293, -0.046196,
7818 -0.694293, -0.046196, -0.572011, -0.220109, -0.558424, -0.057065,
7819 -0.466033, -0.021739, -0.365489, -0.000000, -0.362772, 0.043478,
7820 -0.346467, 0.105978, -0.327446, 0.133152, -0.327446, 0.138587,
7821 -0.183424, 0.032609, -0.188859, 0.016304, -0.188859, -0.002717,
7822 -0.357337, -0.040761, -0.365489, -0.000000, -0.466033, -0.021739,
7823 -0.460598, -0.040761, -0.460598, -0.062500, -0.447011, -0.097826,
7824 -0.438859, -0.108696, -0.438859, -0.116848, -0.400815, -0.157609,
7825 -0.400815, -0.157609, -0.379076, -0.173913, -0.343750, -0.187500,
7826 -0.343750, -0.187500, -0.305707, -0.195652, -0.262228, -0.192935,
7827 -0.221467, -0.176630, -0.210598, -0.176630, -0.186141, -0.157609,
7828 -0.186141, -0.157609, -0.180707, -0.157609, -0.153533, -0.130435,
7829 -0.153533, -0.130435, -0.118207, -0.081522, -0.115489, -0.067935,
7830 -0.115489, -0.067935, -0.101902, -0.043478, -0.088315, 0.016304,

7831 -0.088315, 0.016304, -0.085598, 0.048913, -0.085598, 0.086957,
7832 -0.085598, 0.086957, -0.096467, 0.154891, -0.115489, 0.187500,
7833 -0.115489, 0.187500, -0.115489, 0.195652, -0.134511, 0.220109,
7834 -0.134511, 0.220109, -0.172554, 0.250000, -0.213315, 0.266304,
7835 -0.213315, 0.266304, -0.283967, 0.269022, -0.343750, 0.250000,
7836 -0.343750, 0.250000, -0.381793, 0.225543, -0.408967, 0.198370,
7837 -0.408967, 0.198370, -0.436141, 0.157609, -0.444293, 0.130435,
7838 -0.444293, 0.130435, -0.449728, 0.125000, -0.460598, 0.089674,
7839 -0.460598, 0.089674, -0.466033, 0.046196, -0.466033, -0.021739,
7840 -0.466033, -0.021739, -0.362772, 0.043478, -0.346467, 0.105978,
7841 -0.346467, -0.065217, -0.357337, -0.040761, -0.466033, -0.021739,
7842 -0.460598, -0.040761, -0.447011, -0.097826, -0.438859, -0.108696,
7843 -0.438859, -0.108696, -0.400815, -0.157609, -0.343750, -0.187500,
7844 -0.343750, -0.187500, -0.262228, -0.192935, -0.221467, -0.176630,
7845 -0.221467, -0.176630, -0.186141, -0.157609, -0.153533, -0.130435,
7846 -0.153533, -0.130435, -0.115489, -0.067935, -0.088315, 0.016304,
7847 -0.088315, 0.016304, -0.085598, 0.086957, -0.115489, 0.187500,
7848 -0.115489, 0.187500, -0.134511, 0.220109, -0.213315, 0.266304,
7849 -0.213315, 0.266304, -0.343750, 0.250000, -0.408967, 0.198370,
7850 -0.408967, 0.198370, -0.444293, 0.130435, -0.460598, 0.089674,
7851 -0.460598, 0.089674, -0.466033, -0.021739, -0.346467, 0.105978,
7852 -0.327446, -0.084239, -0.346467, -0.065217, -0.466033, -0.021739,
7853 -0.466033, -0.021739, -0.460598, -0.040761, -0.438859, -0.108696,
7854 -0.438859, -0.108696, -0.343750, -0.187500, -0.221467, -0.176630,
7855 -0.221467, -0.176630, -0.153533, -0.130435, -0.088315, 0.016304,
7856 -0.088315, 0.016304, -0.115489, 0.187500, -0.213315, 0.266304,
7857 -0.213315, 0.266304, -0.408967, 0.198370, -0.460598, 0.089674,
7858 -0.460598, 0.089674, -0.346467, 0.105978, -0.327446, 0.138587,
7859 -0.327446, -0.084239, -0.466033, -0.021739, -0.438859, -0.108696,
7860 -0.213315, 0.266304, -0.460598, 0.089674, -0.327446, 0.138587,
7861 -0.305707, -0.092391, -0.327446, -0.084239, -0.438859, -0.108696,
7862 -0.213315, 0.266304, -0.327446, 0.138587, -0.302989, 0.160326,
7863 -0.273098, -0.092391, -0.305707, -0.092391, -0.438859, -0.108696,
7864 -0.213315, 0.266304, -0.302989, 0.160326, -0.270380, 0.168478,
7865 -0.273098, -0.092391, -0.438859, -0.108696, -0.221467, -0.176630,
7866 -0.213315, 0.266304, -0.270380, 0.168478, -0.251359, 0.171196,
7867 -0.248641, -0.084239, -0.273098, -0.092391, -0.221467, -0.176630,
7868 -0.213315, 0.266304, -0.251359, 0.171196, -0.226902, 0.165761,
7869 -0.216033, -0.057065, -0.248641, -0.084239, -0.221467, -0.176630,
7870 -0.213315, 0.266304, -0.226902, 0.165761, -0.199728, 0.141304,
7871 -0.188859, -0.002717, -0.216033, -0.057065, -0.221467, -0.176630,
7872 -0.088315, 0.016304, -0.213315, 0.266304, -0.199728, 0.141304,
7873 -0.188859, -0.002717, -0.221467, -0.176630, -0.088315, 0.016304,
7874 -0.088315, 0.016304, -0.199728, 0.141304, -0.188859, 0.122283,
7875 -0.183424, 0.032609, -0.188859, -0.002717, -0.088315, 0.016304,
7876 -0.088315, 0.016304, -0.188859, 0.122283, -0.183424, 0.092391,
7877 -0.183424, 0.092391, -0.183424, 0.032609, -0.088315, 0.016304,
7878 0.588315, -0.081522, 0.688859, -0.144022, 0.691576, -0.089674,
7879 0.737772, -0.081522, 0.732337, -0.108696, 0.737772, -0.127717,
7880 0.737772, -0.127717, 0.748641, -0.138587, 0.759511, -0.141304,
7881 0.759511, -0.141304, 0.792120, -0.138587, 0.830163, -0.100543,
7882 0.830163, -0.100543, 0.843750, -0.059783, 0.841033, -0.027174,
7883 0.841033, -0.027174, 0.827446, 0.002717, 0.792120, 0.027174,
7884 0.792120, 0.027174, 0.756793, 0.029891, 0.718750, 0.019022,
7885 0.802989, 0.165761, 0.811141, 0.165761, 0.830163, 0.176630,
7886 0.941576, 0.013587, 0.930707, -0.051630, 0.930707, -0.076087,
7887 0.930707, -0.076087, 0.911685, -0.127717, 0.911685, -0.138587,
7888 0.702446, -0.195652, 0.688859, -0.144022, 0.588315, -0.081522,

7889 0.588315, -0.081522, 0.591033, -0.152174, 0.601902, -0.211957,
7890 0.601902, -0.211957, 0.620924, -0.255435, 0.645380, -0.290761,
7891 0.645380, -0.290761, 0.677989, -0.317935, 0.697011, -0.328804,
7892 0.697011, -0.328804, 0.756793, -0.347826, 0.808424, -0.347826,
7893 0.808424, -0.347826, 0.849185, -0.336957, 0.903533, -0.309783,
7894 0.903533, -0.309783, 0.947011, -0.271739, 0.979620, -0.228261,
7895 0.979620, -0.228261, 1.004076, -0.179348, 1.023098, -0.119565,
7896 1.023098, -0.103261, 1.042120, -0.019022, 1.042120, 0.111413,
7897 1.042120, 0.111413, 1.023098, 0.173913, 1.004076, 0.206522,
7898 0.985054, 0.225543, 0.982337, 0.233696, 0.947011, 0.258152,
7899 0.947011, 0.258152, 0.903533, 0.274457, 0.849185, 0.277174,
7900 0.849185, 0.277174, 0.816576, 0.271739, 0.759511, 0.250000,
7901 0.759511, 0.250000, 0.716033, 0.220109, 0.677989, 0.182065,
7902 0.677989, 0.176630, 0.664402, 0.163043, 0.658967, 0.146739,
7903 0.658967, 0.146739, 0.642663, 0.125000, 0.639946, 0.111413,
7904 0.639946, 0.111413, 0.623641, 0.084239, 0.604620, 0.027174,
7905 0.601902, -0.002717, 0.593750, -0.029891, 0.588315, -0.081522,
7906 0.588315, -0.081522, 0.691576, -0.089674, 0.697011, -0.076087,
7907 0.737772, -0.081522, 0.737772, -0.127717, 0.759511, -0.141304,
7908 0.759511, -0.141304, 0.830163, -0.100543, 0.841033, -0.027174,
7909 0.841033, -0.027174, 0.792120, 0.027174, 0.718750, 0.019022,
7910 0.737772, -0.236413, 0.702446, -0.195652, 0.588315, -0.081522,
7911 0.588315, -0.081522, 0.601902, -0.211957, 0.645380, -0.290761,
7912 0.645380, -0.290761, 0.697011, -0.328804, 0.808424, -0.347826,
7913 0.808424, -0.347826, 0.903533, -0.309783, 0.979620, -0.228261,
7914 0.979620, -0.228261, 1.023098, -0.119565, 1.023098, -0.103261,
7915 1.023098, -0.103261, 1.042120, 0.111413, 1.004076, 0.206522,
7916 1.004076, 0.206522, 0.985054, 0.225543, 0.947011, 0.258152,
7917 0.947011, 0.258152, 0.849185, 0.277174, 0.759511, 0.250000,
7918 0.759511, 0.250000, 0.677989, 0.182065, 0.677989, 0.176630,
7919 0.677989, 0.176630, 0.658967, 0.146739, 0.639946, 0.111413,
7920 0.639946, 0.111413, 0.604620, 0.027174, 0.601902, -0.002717,
7921 0.601902, -0.002717, 0.588315, -0.081522, 0.697011, -0.076087,
7922 0.735054, -0.062500, 0.737772, -0.081522, 0.759511, -0.141304,
7923 0.759511, -0.141304, 0.841033, -0.027174, 0.718750, 0.019022,
7924 0.737772, -0.236413, 0.588315, -0.081522, 0.645380, -0.290761,
7925 0.645380, -0.290761, 0.808424, -0.347826, 0.979620, -0.228261,
7926 0.979620, -0.228261, 1.023098, -0.103261, 1.004076, 0.206522,
7927 1.004076, 0.206522, 0.947011, 0.258152, 0.759511, 0.250000,
7928 0.759511, 0.250000, 0.677989, 0.176630, 0.639946, 0.111413,
7929 0.639946, 0.111413, 0.601902, -0.002717, 0.697011, -0.076087,
7930 0.735054, -0.062500, 0.759511, -0.141304, 0.718750, 0.019022,
7931 0.773098, -0.247283, 0.737772, -0.236413, 0.645380, -0.290761,
7932 0.639946, 0.111413, 0.697011, -0.076087, 0.707880, -0.065217,
7933 0.707880, -0.065217, 0.735054, -0.062500, 0.718750, 0.019022,
7934 0.802989, -0.244565, 0.773098, -0.247283, 0.645380, -0.290761,
7935 0.707880, -0.065217, 0.718750, 0.019022, 0.710598, 0.027174,
7936 0.802989, -0.244565, 0.645380, -0.290761, 0.979620, -0.228261,
7937 0.639946, 0.111413, 0.707880, -0.065217, 0.710598, 0.027174,
7938 0.827446, -0.236413, 0.802989, -0.244565, 0.979620, -0.228261,
7939 0.639946, 0.111413, 0.710598, 0.027174, 0.710598, 0.032609,
7940 0.849185, -0.222826, 0.827446, -0.236413, 0.979620, -0.228261,
7941 0.759511, 0.250000, 0.639946, 0.111413, 0.710598, 0.032609,
7942 0.879076, -0.192935, 0.849185, -0.222826, 0.979620, -0.228261,
7943 0.759511, 0.250000, 0.710598, 0.032609, 0.718750, 0.059783,
7944 0.911685, -0.138587, 0.879076, -0.192935, 0.979620, -0.228261,
7945 0.759511, 0.250000, 0.718750, 0.059783, 0.735054, 0.089674,
7946 0.930707, -0.076087, 0.911685, -0.138587, 0.979620, -0.228261,

7947 0.759511, 0.250000, 0.735054, 0.089674, 0.773098, 0.141304,
7948 0.941576, 0.013587, 0.930707, -0.076087, 0.979620, -0.228261,
7949 0.759511, 0.250000, 0.773098, 0.141304, 0.802989, 0.165761,
7950 0.941576, 0.078804, 0.941576, 0.013587, 0.979620, -0.228261,
7951 0.759511, 0.250000, 0.802989, 0.165761, 0.830163, 0.176630,
7952 0.941576, 0.078804, 0.979620, -0.228261, 1.004076, 0.206522,
7953 1.004076, 0.206522, 0.759511, 0.250000, 0.830163, 0.176630,
7954 0.930707, 0.127717, 0.941576, 0.078804, 1.004076, 0.206522,
7955 1.004076, 0.206522, 0.830163, 0.176630, 0.841033, 0.176630,
7956 0.903533, 0.163043, 0.930707, 0.127717, 1.004076, 0.206522,
7957 1.004076, 0.206522, 0.841033, 0.176630, 0.873641, 0.176630,
7958 0.873641, 0.176630, 0.903533, 0.163043, 1.004076, 0.206522,
7959 0.017663, 0.355978, 0.006793, 0.361413, -0.014946, 0.361413,
7960 -0.014946, 0.361413, -0.042120, 0.347826, -0.058424, 0.326087,
7961 -0.058424, 0.326087, -0.061141, 0.307065, -0.052989, 0.279891,
7962 -0.052989, 0.279891, 0.009511, 0.165761, 0.031250, 0.146739,
7963 0.031250, 0.146739, 0.077446, 0.146739, 0.096467, 0.163043,
7964 0.096467, 0.163043, 0.104620, 0.182065, 0.104620, 0.201087,
7965 0.104620, 0.201087, 0.096467, 0.225543, 0.039402, 0.334239,
7966 0.039402, 0.334239, 0.017663, 0.355978, -0.014946, 0.361413,
7967 -0.014946, 0.361413, -0.058424, 0.326087, -0.052989, 0.279891,
7968 -0.052989, 0.279891, 0.031250, 0.146739, 0.096467, 0.163043,
7969 0.096467, 0.163043, 0.104620, 0.201087, 0.039402, 0.334239,
7970 0.039402, 0.334239, -0.014946, 0.361413, -0.052989, 0.279891,
7971 -0.052989, 0.279891, 0.096467, 0.163043, 0.039402, 0.334239,
7972 0.482337, 0.413043, 0.463315, 0.429348, 0.427989, 0.432065,
7973 0.427989, 0.432065, 0.414402, 0.426630, 0.392663, 0.404891,
7974 0.389946, 0.394022, 0.384511, 0.391304, 0.351902, 0.342391,
7975 0.351902, 0.336957, 0.332880, 0.315217, 0.332880, 0.307065,
7976 0.332880, 0.307065, 0.313859, 0.277174, 0.313859, 0.230978,
7977 0.313859, 0.230978, 0.332880, 0.217391, 0.349185, 0.214674,
7978 0.349185, 0.214674, 0.379076, 0.222826, 0.403533, 0.241848,
7979 0.403533, 0.241848, 0.463315, 0.317935, 0.466033, 0.328804,
7980 0.466033, 0.328804, 0.482337, 0.353261, 0.487772, 0.372283,
7981 0.487772, 0.372283, 0.487772, 0.399457, 0.482337, 0.413043,
7982 0.482337, 0.413043, 0.427989, 0.432065, 0.392663, 0.404891,
7983 0.389946, 0.394022, 0.351902, 0.342391, 0.351902, 0.336957,
7984 0.351902, 0.336957, 0.332880, 0.307065, 0.313859, 0.230978,
7985 0.313859, 0.230978, 0.349185, 0.214674, 0.403533, 0.241848,
7986 0.403533, 0.241848, 0.466033, 0.328804, 0.487772, 0.372283,
7987 0.487772, 0.372283, 0.482337, 0.413043, 0.392663, 0.404891,
7988 0.389946, 0.394022, 0.351902, 0.336957, 0.313859, 0.230978,
7989 0.313859, 0.230978, 0.403533, 0.241848, 0.487772, 0.372283,
7990 0.487772, 0.372283, 0.392663, 0.404891, 0.389946, 0.394022,
7991 0.389946, 0.394022, 0.313859, 0.230978, 0.487772, 0.372283,
7992 -0.944293, 0.429348, -0.966033, 0.448370, -1.001359, 0.453804,
7993 -1.001359, 0.453804, -1.023098, 0.445652, -1.039402, 0.429348,
7994 -1.039402, 0.429348, -1.044837, 0.394022, -1.039402, 0.364130,
7995 -1.017663, 0.298913, -1.017663, 0.279891, -0.998641, 0.190217,
7996 -0.990489, -0.035326, -0.998641, -0.086957, -0.998641, -0.116848,
7997 -0.998641, -0.116848, -1.017663, -0.187500, -1.020380, -0.211957,
7998 -1.137228, -0.369565, -1.150815, -0.375000, -1.172554, -0.396739,
7999 -1.172554, -0.396739, -1.177989, -0.413043, -1.172554, -0.440217,
8000 -1.172554, -0.440217, -1.137228, -0.461957, -1.082880, -0.459239,
8001 -1.055707, -0.442935, -1.047554, -0.442935, -1.001359, -0.402174,
8002 -1.001359, -0.402174, -0.968750, -0.361413, -0.944293, -0.317935,
8003 -0.944293, -0.309783, -0.925272, -0.271739, -0.925272, -0.258152,
8004 -0.925272, -0.258152, -0.906250, -0.203804, -0.906250, -0.187500,

8005 -0.906250, -0.187500, -0.887228, -0.100543, -0.881793, 0.029891,
8006 -0.881793, 0.029891, -0.887228, 0.173913, -0.908967, 0.315217,
8007 -0.925272, 0.369565, -0.925272, 0.383152, -0.944293, 0.421196,
8008 -0.944293, 0.421196, -0.944293, 0.429348, -1.001359, 0.453804,
8009 -1.001359, 0.453804, -1.039402, 0.429348, -1.039402, 0.364130,
8010 -1.137228, -0.369565, -1.172554, -0.396739, -1.172554, -0.440217,
8011 -1.172554, -0.440217, -1.082880, -0.459239, -1.055707, -0.442935,
8012 -1.055707, -0.442935, -1.001359, -0.402174, -0.944293, -0.317935,
8013 -0.944293, -0.309783, -0.925272, -0.258152, -0.906250, -0.187500,
8014 -0.906250, -0.187500, -0.881793, 0.029891, -0.908967, 0.315217,
8015 -0.908967, 0.315217, -0.925272, 0.369565, -0.944293, 0.421196,
8016 -0.944293, 0.421196, -1.001359, 0.453804, -1.039402, 0.364130,
8017 -1.088315, -0.331522, -1.137228, -0.369565, -1.172554, -0.440217,
8018 -1.172554, -0.440217, -1.055707, -0.442935, -0.944293, -0.317935,
8019 -0.944293, -0.309783, -0.906250, -0.187500, -0.908967, 0.315217,
8020 -0.908967, 0.315217, -0.944293, 0.421196, -1.039402, 0.364130,
8021 -1.061141, -0.298913, -1.088315, -0.331522, -1.172554, -0.440217,
8022 -1.172554, -0.440217, -0.944293, -0.317935, -0.944293, -0.309783,
8023 -0.908967, 0.315217, -1.039402, 0.364130, -1.017663, 0.298913,
8024 -1.061141, -0.298913, -1.172554, -0.440217, -0.944293, -0.309783,
8025 -0.908967, 0.315217, -1.017663, 0.298913, -0.998641, 0.190217,
8026 -1.020380, -0.211957, -1.061141, -0.298913, -0.944293, -0.309783,
8027 -0.944293, -0.309783, -0.908967, 0.315217, -0.998641, 0.190217,
8028 -0.998641, -0.116848, -1.020380, -0.211957, -0.944293, -0.309783,
8029 -0.944293, -0.309783, -0.998641, 0.190217, -0.993207, 0.130435,
8030 -0.990489, -0.035326, -0.998641, -0.116848, -0.944293, -0.309783,
8031 -0.944293, -0.309783, -0.993207, 0.130435, -0.990489, -0.035326,
8032 0.908967, 0.448370, 0.889946, 0.464674, 0.851902, 0.467391,
8033 0.851902, 0.467391, 0.699728, 0.391304, 0.683424, 0.366848,
8034 0.683424, 0.366848, 0.683424, 0.342391, 0.699728, 0.320652,
8035 0.699728, 0.320652, 0.718750, 0.312500, 0.748641, 0.312500,
8036 0.759511, 0.317935, 0.773098, 0.317935, 0.797554, 0.331522,
8037 0.797554, 0.331522, 0.819293, 0.336957, 0.851902, 0.355978,
8038 0.851902, 0.355978, 0.860054, 0.355978, 0.889946, 0.375000,
8039 0.889946, 0.375000, 0.908967, 0.394022, 0.914402, 0.413043,
8040 0.914402, 0.413043, 0.914402, 0.434783, 0.908967, 0.448370,
8041 0.908967, 0.448370, 0.851902, 0.467391, 0.683424, 0.366848,
8042 0.683424, 0.366848, 0.699728, 0.320652, 0.748641, 0.312500,
8043 0.759511, 0.317935, 0.797554, 0.331522, 0.851902, 0.355978,
8044 0.851902, 0.355978, 0.889946, 0.375000, 0.914402, 0.413043,
8045 0.914402, 0.413043, 0.908967, 0.448370, 0.683424, 0.366848,
8046 0.683424, 0.366848, 0.748641, 0.312500, 0.759511, 0.317935,
8047 0.759511, 0.317935, 0.851902, 0.355978, 0.914402, 0.413043,
8048 0.914402, 0.413043, 0.683424, 0.366848, 0.759511, 0.317935,
8049 1.273098, 0.461957, 1.262228, 0.467391, 1.226902, 0.470109,
8050 1.226902, 0.470109, 1.213315, 0.467391, 1.199728, 0.453804,
8051 1.199728, 0.453804, 1.186141, 0.426630, 1.180707, 0.396739,
8052 1.180707, 0.396739, 1.161685, 0.350543, 1.161685, 0.339674,
8053 1.161685, 0.339674, 1.148098, 0.315217, 1.120924, 0.233696,
8054 1.120924, 0.233696, 1.107337, 0.209239, 1.085598, 0.152174,
8055 1.085598, 0.152174, 1.088315, 0.119565, 1.110054, 0.103261,
8056 1.110054, 0.103261, 1.137228, 0.103261, 1.142663, 0.108696,
8057 1.142663, 0.108696, 1.156250, 0.111413, 1.177989, 0.133152,
8058 1.379076, 0.032609, 1.373641, 0.008152, 1.373641, -0.019022,
8059 1.373641, -0.019022, 1.354620, -0.070652, 1.354620, -0.081522,
8060 1.354620, -0.081522, 1.335598, -0.122283, 1.335598, -0.130435,
8061 1.237772, -0.255435, 1.167120, -0.312500, 1.150815, -0.336957,
8062 1.150815, -0.336957, 1.150815, -0.361413, 1.161685, -0.380435,

8063 1.161685, -0.380435, 1.167120, -0.385870, 1.205163, -0.399457,
8064 1.205163, -0.399457, 1.235054, -0.394022, 1.278533, -0.366848,
8065 1.316576, -0.328804, 1.322011, -0.328804, 1.330163, -0.320652,
8066 1.332880, -0.312500, 1.351902, -0.296196, 1.379076, -0.258152,
8067 1.379076, -0.258152, 1.387228, -0.252717, 1.389946, -0.241848,
8068 1.389946, -0.241848, 1.408967, -0.217391, 1.444293, -0.149457,
8069 1.449728, -0.125000, 1.468750, -0.078804, 1.468750, -0.062500,
8070 1.468750, -0.062500, 1.487772, 0.013587, 1.487772, 0.122283,
8071 1.487772, 0.122283, 1.463315, 0.198370, 1.433424, 0.239130,
8072 1.433424, 0.239130, 1.389946, 0.269022, 1.335598, 0.282609,
8073 1.254076, 0.290761, 1.294837, 0.394022, 1.294837, 0.434783,
8074 1.294837, 0.434783, 1.273098, 0.461957, 1.226902, 0.470109,
8075 1.226902, 0.470109, 1.199728, 0.453804, 1.180707, 0.396739,
8076 1.120924, 0.233696, 1.085598, 0.152174, 1.110054, 0.103261,
8077 1.110054, 0.103261, 1.142663, 0.108696, 1.177989, 0.133152,
8078 1.237772, -0.255435, 1.150815, -0.336957, 1.161685, -0.380435,
8079 1.161685, -0.380435, 1.205163, -0.399457, 1.278533, -0.366848,
8080 1.316576, -0.328804, 1.330163, -0.320652, 1.332880, -0.312500,
8081 1.332880, -0.312500, 1.379076, -0.258152, 1.389946, -0.241848,
8082 1.389946, -0.241848, 1.444293, -0.149457, 1.449728, -0.125000,
8083 1.449728, -0.125000, 1.468750, -0.062500, 1.487772, 0.122283,
8084 1.487772, 0.122283, 1.433424, 0.239130, 1.335598, 0.282609,
8085 1.254076, 0.290761, 1.294837, 0.434783, 1.226902, 0.470109,
8086 1.226902, 0.470109, 1.180707, 0.396739, 1.161685, 0.339674,
8087 1.161685, 0.339674, 1.120924, 0.233696, 1.110054, 0.103261,
8088 1.110054, 0.103261, 1.177989, 0.133152, 1.216033, 0.184783,
8089 1.281250, -0.211957, 1.237772, -0.255435, 1.161685, -0.380435,
8090 1.161685, -0.380435, 1.278533, -0.366848, 1.316576, -0.328804,
8091 1.316576, -0.328804, 1.332880, -0.312500, 1.389946, -0.241848,
8092 1.389946, -0.241848, 1.449728, -0.125000, 1.487772, 0.122283,
8093 1.487772, 0.122283, 1.335598, 0.282609, 1.262228, 0.282609,
8094 1.254076, 0.290761, 1.226902, 0.470109, 1.161685, 0.339674,
8095 1.161685, 0.339674, 1.110054, 0.103261, 1.216033, 0.184783,
8096 1.313859, -0.168478, 1.281250, -0.211957, 1.161685, -0.380435,
8097 1.161685, -0.380435, 1.316576, -0.328804, 1.389946, -0.241848,
8098 1.262228, 0.282609, 1.254076, 0.290761, 1.161685, 0.339674,
8099 1.161685, 0.339674, 1.216033, 0.184783, 1.232337, 0.190217,
8100 1.313859, -0.168478, 1.161685, -0.380435, 1.389946, -0.241848,
8101 1.487772, 0.122283, 1.262228, 0.282609, 1.161685, 0.339674,
8102 1.161685, 0.339674, 1.232337, 0.190217, 1.281250, 0.190217,
8103 1.335598, -0.130435, 1.313859, -0.168478, 1.389946, -0.241848,
8104 1.161685, 0.339674, 1.281250, 0.190217, 1.316576, 0.184783,
8105 1.354620, -0.081522, 1.335598, -0.130435, 1.389946, -0.241848,
8106 1.487772, 0.122283, 1.161685, 0.339674, 1.316576, 0.184783,
8107 1.373641, -0.019022, 1.354620, -0.081522, 1.389946, -0.241848,
8108 1.487772, 0.122283, 1.316576, 0.184783, 1.349185, 0.165761,
8109 1.379076, 0.032609, 1.373641, -0.019022, 1.389946, -0.241848,
8110 1.487772, 0.122283, 1.349185, 0.165761, 1.368207, 0.138587,
8111 1.379076, 0.086957, 1.379076, 0.032609, 1.389946, -0.241848,
8112 1.487772, 0.122283, 1.368207, 0.138587, 1.379076, 0.086957,
8113 1.379076, 0.086957, 1.389946, -0.241848, 1.487772, 0.122283,
8114 0.243207, 0.486413, 0.224185, 0.497283, 0.197011, 0.500000,
8115 0.197011, 0.500000, 0.164402, 0.483696, 0.153533, 0.451087,
8116 0.137228, 0.105978, 0.120924, 0.105978, 0.052989, 0.086957,
8117 0.052989, 0.086957, 0.031250, 0.086957, -0.033967, 0.067935,
8118 -0.033967, 0.067935, -0.052989, 0.043478, -0.052989, 0.021739,
8119 -0.052989, 0.021739, -0.042120, -0.005435, -0.017663, -0.024457,
8120 -0.017663, -0.024457, 0.031250, -0.024457, 0.123641, -0.000000,

8121 0.118207, -0.043478, 0.107337, -0.046196, 0.085598, -0.065217,
8122 0.085598, -0.065217, 0.080163, -0.065217, 0.055707, -0.086957,
8123 0.055707, -0.086957, -0.033967, -0.179348, -0.033967, -0.184783,
8124 -0.033967, -0.184783, -0.074728, -0.222826, -0.080163, -0.233696,
8125 -0.145380, -0.293478, -0.150815, -0.293478, -0.188859, -0.331522,
8126 -0.188859, -0.331522, -0.205163, -0.361413, -0.202446, -0.380435,
8127 -0.202446, -0.380435, -0.186141, -0.404891, -0.161685, -0.415761,
8128 -0.161685, -0.415761, -0.145380, -0.415761, -0.126359, -0.404891,
8129 -0.126359, -0.404891, -0.118207, -0.404891, -0.036685, -0.336957,
8130 -0.036685, -0.336957, 0.033967, -0.266304, 0.096467, -0.190217,
8131 0.120924, -0.244565, 0.099185, -0.358696, 0.099185, -0.383152,
8132 0.099185, -0.383152, 0.082880, -0.442935, 0.082880, -0.467391,
8133 0.082880, -0.467391, 0.088315, -0.480978, 0.115489, -0.500000,
8134 0.115489, -0.500000, 0.142663, -0.500000, 0.153533, -0.494565,
8135 0.153533, -0.494565, 0.175272, -0.475543, 0.194293, -0.437500,
8136 0.194293, -0.421196, 0.213315, -0.345109, 0.213315, -0.315217,
8137 0.213315, -0.315217, 0.232337, -0.201087, 0.235054, -0.138587,
8138 0.273098, -0.144022, 0.278533, -0.157609, 0.286685, -0.163043,
8139 0.286685, -0.163043, 0.292120, -0.176630, 0.370924, -0.269022,
8140 0.389946, -0.285326, 0.389946, -0.290761, 0.406250, -0.301630,
8141 0.406250, -0.301630, 0.427989, -0.326087, 0.444293, -0.334239,
8142 0.444293, -0.334239, 0.452446, -0.345109, 0.479620, -0.364130,
8143 0.479620, -0.364130, 0.506793, -0.375000, 0.539402, -0.372283,
8144 0.539402, -0.372283, 0.561141, -0.358696, 0.574728, -0.326087,
8145 0.574728, -0.326087, 0.561141, -0.293478, 0.528533, -0.269022,
8146 0.528533, -0.269022, 0.517663, -0.255435, 0.512228, -0.255435,
8147 0.444293, -0.187500, 0.441576, -0.179348, 0.425272, -0.165761,
8148 0.425272, -0.165761, 0.425272, -0.160326, 0.387228, -0.119565,
8149 0.387228, -0.119565, 0.387228, -0.114130, 0.370924, -0.097826,
8150 0.370924, -0.097826, 0.365489, -0.084239, 0.330163, -0.046196,
8151 0.330163, -0.046196, 0.305707, -0.027174, 0.259511, -0.016304,
8152 0.262228, 0.029891, 0.292120, 0.032609, 0.362772, 0.051630,
8153 0.362772, 0.051630, 0.381793, 0.051630, 0.460598, 0.078804,
8154 0.460598, 0.078804, 0.482337, 0.097826, 0.490489, 0.135870,
8155 0.490489, 0.135870, 0.479620, 0.163043, 0.457880, 0.176630,
8156 0.457880, 0.176630, 0.417120, 0.173913, 0.387228, 0.163043,
8157 0.387228, 0.163043, 0.351902, 0.157609, 0.324728, 0.146739,
8158 0.324728, 0.146739, 0.300272, 0.144022, 0.273098, 0.133152,
8159 0.254076, 0.141304, 0.256793, 0.456522, 0.251359, 0.478261,
8160 0.251359, 0.478261, 0.243207, 0.486413, 0.197011, 0.500000,
8161 0.197011, 0.500000, 0.153533, 0.451087, 0.153533, 0.119565,
8162 0.052989, 0.086957, -0.033967, 0.067935, -0.052989, 0.021739,
8163 -0.052989, 0.021739, -0.017663, -0.024457, 0.123641, -0.000000,
8164 0.118207, -0.043478, 0.085598, -0.065217, 0.055707, -0.086957,
8165 -0.145380, -0.293478, -0.188859, -0.331522, -0.202446, -0.380435,
8166 -0.202446, -0.380435, -0.161685, -0.415761, -0.126359, -0.404891,
8167 -0.126359, -0.404891, -0.036685, -0.336957, 0.096467, -0.190217,
8168 0.099185, -0.383152, 0.082880, -0.467391, 0.115489, -0.500000,
8169 0.115489, -0.500000, 0.153533, -0.494565, 0.194293, -0.437500,
8170 0.194293, -0.421196, 0.213315, -0.315217, 0.235054, -0.138587,
8171 0.273098, -0.144022, 0.286685, -0.163043, 0.370924, -0.269022,
8172 0.389946, -0.285326, 0.406250, -0.301630, 0.444293, -0.334239,
8173 0.444293, -0.334239, 0.479620, -0.364130, 0.539402, -0.372283,
8174 0.539402, -0.372283, 0.574728, -0.326087, 0.528533, -0.269022,
8175 0.387228, -0.119565, 0.370924, -0.097826, 0.330163, -0.046196,
8176 0.262228, 0.029891, 0.362772, 0.051630, 0.460598, 0.078804,
8177 0.460598, 0.078804, 0.490489, 0.135870, 0.457880, 0.176630,
8178 0.457880, 0.176630, 0.387228, 0.163043, 0.324728, 0.146739,

```

8179      0.254076, 0.141304, 0.251359, 0.478261, 0.197011, 0.500000,
8180      0.137228, 0.105978, 0.052989, 0.086957, -0.052989, 0.021739,
8181      -0.052989, 0.021739, 0.123641, -0.000000, 0.134511, -0.000000,
8182      0.118207, -0.043478, 0.055707, -0.086957, -0.033967, -0.184783,
8183      -0.080163, -0.233696, -0.145380, -0.293478, -0.202446, -0.380435,
8184      -0.202446, -0.380435, -0.126359, -0.404891, 0.096467, -0.190217,
8185      0.120924, -0.244565, 0.099185, -0.383152, 0.115489, -0.500000,
8186      0.115489, -0.500000, 0.194293, -0.437500, 0.194293, -0.421196,
8187      0.273098, -0.144022, 0.370924, -0.269022, 0.389946, -0.285326,
8188      0.389946, -0.285326, 0.444293, -0.334239, 0.539402, -0.372283,
8189      0.539402, -0.372283, 0.528533, -0.269022, 0.512228, -0.255435,
8190      0.387228, -0.119565, 0.330163, -0.046196, 0.259511, -0.016304,
8191      0.262228, 0.029891, 0.460598, 0.078804, 0.457880, 0.176630,
8192      0.457880, 0.176630, 0.324728, 0.146739, 0.273098, 0.133152,
8193      0.254076, 0.141304, 0.197011, 0.500000, 0.153533, 0.119565,
8194      0.137228, 0.105978, -0.052989, 0.021739, 0.134511, -0.000000,
8195      0.139946, -0.024457, 0.118207, -0.043478, -0.033967, -0.184783,
8196      -0.033967, -0.184783, -0.080163, -0.233696, -0.202446, -0.380435,
8197      -0.202446, -0.380435, 0.096467, -0.190217, 0.118207, -0.171196,
8198      0.126359, -0.179348, 0.120924, -0.244565, 0.115489, -0.500000,
8199      0.115489, -0.500000, 0.194293, -0.421196, 0.235054, -0.138587,
8200      0.273098, -0.144022, 0.389946, -0.285326, 0.539402, -0.372283,
8201      0.539402, -0.372283, 0.512228, -0.255435, 0.444293, -0.187500,
8202      0.425272, -0.165761, 0.387228, -0.119565, 0.259511, -0.016304,
8203      0.251359, 0.021739, 0.262228, 0.029891, 0.457880, 0.176630,
8204      0.262228, 0.133152, 0.254076, 0.141304, 0.153533, 0.119565,
8205      0.153533, 0.119565, 0.137228, 0.105978, 0.134511, -0.000000,
8206      0.139946, -0.024457, -0.033967, -0.184783, -0.202446, -0.380435,
8207      0.126359, -0.179348, 0.115489, -0.500000, 0.235054, -0.138587,
8208      0.256793, -0.125000, 0.273098, -0.144022, 0.539402, -0.372283,
8209      0.539402, -0.372283, 0.444293, -0.187500, 0.425272, -0.165761,
8210      0.425272, -0.165761, 0.259511, -0.016304, 0.251359, -0.010870,
8211      0.251359, 0.021739, 0.457880, 0.176630, 0.273098, 0.133152,
8212      0.273098, 0.133152, 0.262228, 0.133152, 0.153533, 0.119565,
8213      0.153533, 0.119565, 0.134511, -0.000000, 0.139946, -0.005435,
8214      0.139946, -0.024457, -0.202446, -0.380435, 0.118207, -0.171196,
8215      0.118207, -0.171196, 0.126359, -0.179348, 0.235054, -0.138587,
8216      0.251359, -0.119565, 0.256793, -0.125000, 0.539402, -0.372283,
8217      0.539402, -0.372283, 0.425272, -0.165761, 0.251359, -0.010870,
8218      0.251359, 0.021739, 0.273098, 0.133152, 0.153533, 0.119565,
8219      0.153533, 0.119565, 0.139946, -0.005435, 0.139946, -0.024457,
8220      0.139946, -0.024457, 0.118207, -0.171196, 0.235054, -0.138587,
8221      0.251359, -0.119565, 0.539402, -0.372283, 0.251359, -0.010870,
8222      0.251359, -0.010870, 0.251359, 0.021739, 0.153533, 0.119565,
8223      0.153533, 0.119565, 0.139946, -0.024457, 0.235054, -0.138587,
8224      0.243207, -0.119565, 0.251359, -0.119565, 0.251359, -0.010870,
8225      0.251359, -0.010870, 0.153533, 0.119565, 0.235054, -0.138587,
8226      0.235054, -0.138587, 0.243207, -0.119565, 0.251359, -0.010870,
8227 ];
8228 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/GameTitlePathSegs.generated.ts -----
8229 // AUTO-GENERATED by tools/gen-game-title-paths.py (title) — do not edit by hand
8230 export interface GameTitlePathSeg {
8231     readonly x: number;
8232     readonly y: number;
8233 }
8234 /** 归一化 path 包围盒宽高（按字高归一化；运行时 fit 进 UITransform） */
8235 export const GAME_TITLE_NORM_W = 3.413043;
8236 export const GAME_TITLE_NORM_H = 1.000000;

```

```

8237 /** 全部轮廓（1px 描边：含孔洞边缘） */
8238 export const GAME_TITLE_OUTLINE_PATHS: ReadonlyArray<
8239     ReadonlyArray<Readonly<GameTitlePathSeg>>
8240 > = [
8241     [
8242         { x: 1.063665, y: -0.130435 },
8243         { x: 1.048137, y: -0.136646 },
8244         { x: 1.032609, y: -0.152174 },
8245         { x: 1.020186, y: -0.183230 },
8246         { x: 1.020186, y: -0.425466 },
8247         { x: 1.029503, y: -0.450311 },
8248         { x: 1.048137, y: -0.472050 },
8249         { x: 1.069876, y: -0.481366 },
8250         { x: 1.616460, y: -0.481366 },
8251         { x: 1.638199, y: -0.468944 },
8252         { x: 1.656832, y: -0.447205 },
8253         { x: 1.663043, y: -0.425466 },
8254         { x: 1.663043, y: -0.177019 },
8255         { x: 1.650621, y: -0.149068 },
8256         { x: 1.635093, y: -0.136646 },
8257         { x: 1.619565, y: -0.130435 },
8258     ],
8259     [
8260         { x: 1.125776, y: -0.208075 },
8261         { x: 1.560559, y: -0.208075 },
8262         { x: 1.579193, y: -0.223602 },
8263         { x: 1.579193, y: -0.388199 },
8264         { x: 1.554348, y: -0.406832 },
8265         { x: 1.119565, y: -0.403727 },
8266         { x: 1.100932, y: -0.385093 },
8267         { x: 1.100932, y: -0.226708 },
8268         { x: 1.113354, y: -0.211180 },
8269     ],
8270     [
8271         { x: 0.687888, y: 0.024845 },
8272         { x: 0.675466, y: 0.012422 },
8273         { x: 0.669255, y: -0.006211 },
8274         { x: 0.690994, y: -0.108696 },
8275         { x: 0.728261, y: -0.248447 },
8276         { x: 0.743789, y: -0.288820 },
8277         { x: 0.756211, y: -0.301242 },
8278         { x: 0.768634, y: -0.307453 },
8279         { x: 0.793478, y: -0.307453 },
8280         { x: 0.815217, y: -0.288820 },
8281         { x: 0.818323, y: -0.267081 },
8282         { x: 0.790373, y: -0.189441 },
8283         { x: 0.768634, y: -0.090062 },
8284         { x: 0.759317, y: -0.068323 },
8285         { x: 0.746894, y: 0.003106 },
8286         { x: 0.740683, y: 0.015528 },
8287         { x: 0.728261, y: 0.024845 },
8288     ],
8289     [
8290         { x: 0.405280, y: 0.027950 },
8291         { x: 0.383540, y: 0.012422 },
8292         { x: 0.377329, y: -0.003106 },
8293         { x: 0.343168, y: -0.180124 },
8294         { x: 0.309006, y: -0.276398 },

```

```
8295      { x: 0.315217, y: -0.295031 },
8296      { x: 0.340062, y: -0.310559 },
8297      { x: 0.364907, y: -0.307453 },
8298      { x: 0.383540, y: -0.288820 },
8299      { x: 0.395963, y: -0.260870 },
8300      { x: 0.423913, y: -0.161491 },
8301      { x: 0.454969, y: -0.012422 },
8302      { x: 0.451863, y: 0.009317 },
8303      { x: 0.442547, y: 0.021739 },
8304      { x: 0.427019, y: 0.027950 },
8305      ],
8306      [
8307      { x: -0.631988, y: 0.133540 },
8308      { x: -0.647516, y: 0.111801 },
8309      { x: -0.647516, y: 0.083851 },
8310      { x: -0.631988, y: 0.062112 },
8311      { x: -0.364907, y: 0.062112 },
8312      { x: -0.340062, y: 0.046584 },
8313      { x: -0.340062, y: 0.027950 },
8314      { x: -0.423913, y: -0.052795 },
8315      { x: -0.436335, y: -0.071429 },
8316      { x: -0.439441, y: -0.108696 },
8317      { x: -0.458075, y: -0.124224 },
8318      { x: -0.762422, y: -0.124224 },
8319      { x: -0.787267, y: -0.136646 },
8320      { x: -0.796584, y: -0.164596 },
8321      { x: -0.793478, y: -0.180124 },
8322      { x: -0.777950, y: -0.198758 },
8323      { x: -0.759317, y: -0.204969 },
8324      { x: -0.451863, y: -0.208075 },
8325      { x: -0.439441, y: -0.220497 },
8326      { x: -0.442547, y: -0.385093 },
8327      { x: -0.458075, y: -0.400621 },
8328      { x: -0.473602, y: -0.406832 },
8329      { x: -0.585404, y: -0.406832 },
8330      { x: -0.604037, y: -0.422360 },
8331      { x: -0.610248, y: -0.434783 },
8332      { x: -0.610248, y: -0.456522 },
8333      { x: -0.600932, y: -0.475155 },
8334      { x: -0.582298, y: -0.484472 },
8335      { x: -0.411491, y: -0.481366 },
8336      { x: -0.383540, y: -0.462733 },
8337      { x: -0.358696, y: -0.416149 },
8338      { x: -0.355590, y: -0.220497 },
8339      { x: -0.343168, y: -0.208075 },
8340      { x: -0.327640, y: -0.201863 },
8341      { x: -0.097826, y: -0.201863 },
8342      { x: -0.082298, y: -0.189441 },
8343      { x: -0.076087, y: -0.177019 },
8344      { x: -0.076087, y: -0.145963 },
8345      { x: -0.091615, y: -0.127329 },
8346      { x: -0.104037, y: -0.124224 },
8347      { x: -0.336957, y: -0.124224 },
8348      { x: -0.355590, y: -0.108696 },
8349      { x: -0.355590, y: -0.093168 },
8350      { x: -0.277950, y: -0.015528 },
8351      { x: -0.234472, y: 0.040373 },
8352      { x: -0.228261, y: 0.062112 },
```



```
8353      { x: -0.228261, y: 0.096273 },
8354      { x: -0.234472, y: 0.114907 },
8355      { x: -0.250000, y: 0.130435 },
8356      ],
8357      [
8358      { x: 0.097826, y: 0.183230 },
8359      { x: 0.079193, y: 0.161491 },
8360      { x: 0.076087, y: 0.142857 },
8361      { x: 0.082298, y: 0.127329 },
8362      { x: 0.088509, y: 0.118012 },
8363      { x: 0.100932, y: 0.111801 },
8364      { x: 0.135093, y: 0.111801 },
8365      { x: 0.156832, y: 0.093168 },
8366      { x: 0.156832, y: -0.332298 },
8367      { x: 0.147516, y: -0.360248 },
8368      { x: 0.094720, y: -0.416149 },
8369      { x: 0.069876, y: -0.450311 },
8370      { x: 0.072981, y: -0.478261 },
8371      { x: 0.088509, y: -0.496894 },
8372      { x: 0.122671, y: -0.500000 },
8373      { x: 0.190994, y: -0.431677 },
8374      { x: 0.206522, y: -0.425466 },
8375      { x: 0.228261, y: -0.434783 },
8376      { x: 0.268634, y: -0.462733 },
8377      { x: 0.318323, y: -0.481366 },
8378      { x: 0.812112, y: -0.481366 },
8379      { x: 0.827640, y: -0.472050 },
8380      { x: 0.836957, y: -0.456522 },
8381      { x: 0.840062, y: -0.434783 },
8382      { x: 0.827640, y: -0.409938 },
8383      { x: 0.815217, y: -0.403727 },
8384      { x: 0.349379, y: -0.403727 },
8385      { x: 0.309006, y: -0.391304 },
8386      { x: 0.256211, y: -0.357143 },
8387      { x: 0.240683, y: -0.341615 },
8388      { x: 0.234472, y: -0.326087 },
8389      { x: 0.234472, y: 0.149068 },
8390      { x: 0.222050, y: 0.170807 },
8391      { x: 0.203416, y: 0.183230 },
8392      ],
8393      [
8394      { x: 1.147516, y: 0.223602 },
8395      { x: 1.128882, y: 0.217391 },
8396      { x: 1.113354, y: 0.201863 },
8397      { x: 1.100932, y: 0.164596 },
8398      { x: 1.100932, y: -0.000000 },
8399      { x: 1.113354, y: -0.034161 },
8400      { x: 1.128882, y: -0.049689 },
8401      { x: 1.147516, y: -0.055901 },
8402      { x: 1.520186, y: -0.055901 },
8403      { x: 1.541925, y: -0.046584 },
8404      { x: 1.557453, y: -0.027950 },
8405      { x: 1.563665, y: -0.012422 },
8406      { x: 1.563665, y: 0.180124 },
8407      { x: 1.551242, y: 0.204969 },
8408      { x: 1.532609, y: 0.220497 },
8409      ],
8410      [
```

```
8411      { x: 1.206522, y: 0.152174 },
8412      { x: 1.470497, y: 0.152174 },
8413      { x: 1.486025, y: 0.136646 },
8414      { x: 1.486025, y: 0.034161 },
8415      { x: 1.461180, y: 0.015528 },
8416      { x: 1.197205, y: 0.018634 },
8417      { x: 1.181677, y: 0.031056 },
8418      { x: 1.181677, y: 0.136646 },
8419      { x: 1.197205, y: 0.152174 },
8420      ],
8421      [
8422      { x: -1.072981, y: 0.437888 },
8423      { x: -1.088509, y: 0.428571 },
8424      { x: -1.104037, y: 0.397516 },
8425      { x: -1.131988, y: 0.319876 },
8426      { x: -1.187888, y: 0.201863 },
8427      { x: -1.194099, y: 0.170807 },
8428      { x: -1.187888, y: 0.158385 },
8429      { x: -1.172360, y: 0.145963 },
8430      { x: -1.141304, y: 0.145963 },
8431      { x: -1.119565, y: 0.167702 },
8432      { x: -1.020186, y: 0.394410 },
8433      { x: -1.023292, y: 0.422360 },
8434      { x: -1.035714, y: 0.434783 },
8435      ],
8436      [
8437      { x: -1.628882, y: 0.437888 },
8438      { x: -1.644410, y: 0.428571 },
8439      { x: -1.653727, y: 0.403727 },
8440      { x: -1.594720, y: 0.276398 },
8441      { x: -1.560559, y: 0.186335 },
8442      { x: -1.538820, y: 0.149068 },
8443      { x: -1.523292, y: 0.142857 },
8444      { x: -1.495342, y: 0.145963 },
8445      { x: -1.476708, y: 0.164596 },
8446      { x: -1.473602, y: 0.189441 },
8447      { x: -1.541925, y: 0.344720 },
8448      { x: -1.560559, y: 0.397516 },
8449      { x: -1.582298, y: 0.431677 },
8450      { x: -1.594720, y: 0.437888 },
8451      ],
8452      [
8453      { x: 0.750000, y: 0.453416 },
8454      { x: 0.740683, y: 0.447205 },
8455      { x: 0.740683, y: 0.444099 },
8456      { x: 0.734472, y: 0.437888 },
8457      { x: 0.725155, y: 0.419255 },
8458      { x: 0.725155, y: 0.413043 },
8459      { x: 0.722050, y: 0.409938 },
8460      { x: 0.718944, y: 0.394410 },
8461      { x: 0.712733, y: 0.385093 },
8462      { x: 0.712733, y: 0.378882 },
8463      { x: 0.706522, y: 0.369565 },
8464      { x: 0.703416, y: 0.354037 },
8465      { x: 0.694099, y: 0.338509 },
8466      { x: 0.694099, y: 0.332298 },
8467      { x: 0.684783, y: 0.316770 },
8468      { x: 0.684783, y: 0.310559 },
```

```
8469      { x: 0.675466, y: 0.295031 },
8470      { x: 0.675466, y: 0.288820 },
8471      { x: 0.669255, y: 0.276398 },
8472      { x: 0.669255, y: 0.260870 },
8473      { x: 0.672360, y: 0.254658 },
8474      { x: 0.681677, y: 0.245342 },
8475      { x: 0.687888, y: 0.242236 },
8476      { x: 0.697205, y: 0.242236 },
8477      { x: 0.700311, y: 0.239130 },
8478      { x: 0.715839, y: 0.239130 },
8479      { x: 0.728261, y: 0.248447 },
8480      { x: 0.731366, y: 0.248447 },
8481      { x: 0.731366, y: 0.251553 },
8482      { x: 0.737578, y: 0.257764 },
8483      { x: 0.756211, y: 0.295031 },
8484      { x: 0.756211, y: 0.301242 },
8485      { x: 0.762422, y: 0.310559 },
8486      { x: 0.762422, y: 0.316770 },
8487      { x: 0.771739, y: 0.329193 },
8488      { x: 0.771739, y: 0.335404 },
8489      { x: 0.784161, y: 0.360248 },
8490      { x: 0.787267, y: 0.375776 },
8491      { x: 0.796584, y: 0.391304 },
8492      { x: 0.796584, y: 0.397516 },
8493      { x: 0.805901, y: 0.416149 },
8494      { x: 0.805901, y: 0.434783 },
8495      { x: 0.793478, y: 0.450311 },
8496      { x: 0.787267, y: 0.453416 },
8497      ],
8498      [
8499      { x: 0.346273, y: 0.453416 },
8500      { x: 0.343168, y: 0.450311 },
8501      { x: 0.333851, y: 0.450311 },
8502      { x: 0.330745, y: 0.447205 },
8503      { x: 0.324534, y: 0.447205 },
8504      { x: 0.318323, y: 0.440994 },
8505      { x: 0.312112, y: 0.428571 },
8506      { x: 0.312112, y: 0.413043 },
8507      { x: 0.315217, y: 0.409938 },
8508      { x: 0.318323, y: 0.394410 },
8509      { x: 0.324534, y: 0.385093 },
8510      { x: 0.324534, y: 0.378882 },
8511      { x: 0.327640, y: 0.372671 },
8512      { x: 0.333851, y: 0.366460 },
8513      { x: 0.333851, y: 0.360248 },
8514      { x: 0.343168, y: 0.344720 },
8515      { x: 0.343168, y: 0.338509 },
8516      { x: 0.349379, y: 0.329193 },
8517      { x: 0.349379, y: 0.322981 },
8518      { x: 0.355590, y: 0.313665 },
8519      { x: 0.355590, y: 0.307453 },
8520      { x: 0.361801, y: 0.298137 },
8521      { x: 0.361801, y: 0.291925 },
8522      { x: 0.368012, y: 0.282609 },
8523      { x: 0.368012, y: 0.276398 },
8524      { x: 0.380435, y: 0.251553 },
8525      { x: 0.389752, y: 0.242236 },
8526      { x: 0.399068, y: 0.242236 },
```

```
8527      { x: 0.402174, y: 0.239130 },
8528      { x: 0.414596, y: 0.239130 },
8529      { x: 0.417702, y: 0.242236 },
8530      { x: 0.423913, y: 0.242236 },
8531      { x: 0.439441, y: 0.251553 },
8532      { x: 0.445652, y: 0.263975 },
8533      { x: 0.445652, y: 0.270186 },
8534      { x: 0.448758, y: 0.273292 },
8535      { x: 0.445652, y: 0.276398 },
8536      { x: 0.445652, y: 0.282609 },
8537      { x: 0.442547, y: 0.285714 },
8538      { x: 0.439441, y: 0.301242 },
8539      { x: 0.430124, y: 0.316770 },
8540      { x: 0.430124, y: 0.322981 },
8541      { x: 0.414596, y: 0.354037 },
8542      { x: 0.414596, y: 0.360248 },
8543      { x: 0.405280, y: 0.378882 },
8544      { x: 0.405280, y: 0.385093 },
8545      { x: 0.395963, y: 0.400621 },
8546      { x: 0.395963, y: 0.406832 },
8547      { x: 0.380435, y: 0.437888 },
8548      { x: 0.371118, y: 0.447205 },
8549      { x: 0.364907, y: 0.450311 },
8550      { x: 0.358696, y: 0.450311 },
8551      { x: 0.355590, y: 0.453416 },
8552      ],
8553      [
8554      { x: 0.131988, y: 0.456522 },
8555      { x: 0.128882, y: 0.453416 },
8556      { x: 0.119565, y: 0.453416 },
8557      { x: 0.104037, y: 0.444099 },
8558      { x: 0.097826, y: 0.431677 },
8559      { x: 0.097826, y: 0.416149 },
8560      { x: 0.107143, y: 0.397516 },
8561      { x: 0.113354, y: 0.391304 },
8562      { x: 0.122671, y: 0.372671 },
8563      { x: 0.128882, y: 0.366460 },
8564      { x: 0.128882, y: 0.360248 },
8565      { x: 0.135093, y: 0.354037 },
8566      { x: 0.163043, y: 0.298137 },
8567      { x: 0.178571, y: 0.282609 },
8568      { x: 0.184783, y: 0.279503 },
8569      { x: 0.209627, y: 0.279503 },
8570      { x: 0.222050, y: 0.285714 },
8571      { x: 0.231366, y: 0.298137 },
8572      { x: 0.231366, y: 0.304348 },
8573      { x: 0.234472, y: 0.307453 },
8574      { x: 0.234472, y: 0.319876 },
8575      { x: 0.231366, y: 0.322981 },
8576      { x: 0.231366, y: 0.329193 },
8577      { x: 0.222050, y: 0.347826 },
8578      { x: 0.215839, y: 0.354037 },
8579      { x: 0.203416, y: 0.381988 },
8580      { x: 0.197205, y: 0.388199 },
8581      { x: 0.190994, y: 0.403727 },
8582      { x: 0.184783, y: 0.409938 },
8583      { x: 0.178571, y: 0.425466 },
8584      { x: 0.172360, y: 0.431677 },
```

```
8585      { x: 0.166149, y: 0.444099 },
8586      { x: 0.153727, y: 0.453416 },
8587  ],
8588  [
8589      { x: -0.703416, y: 0.484472 },
8590      { x: -0.715839, y: 0.478261 },
8591      { x: -0.725155, y: 0.462733 },
8592      { x: -0.725155, y: 0.450311 },
8593      { x: -0.715839, y: 0.431677 },
8594      { x: -0.706522, y: 0.422360 },
8595      { x: -0.706522, y: 0.419255 },
8596      { x: -0.697205, y: 0.409938 },
8597      { x: -0.697205, y: 0.406832 },
8598      { x: -0.687888, y: 0.397516 },
8599      { x: -0.687888, y: 0.394410 },
8600      { x: -0.663043, y: 0.369565 },
8601      { x: -0.656832, y: 0.366460 },
8602      { x: -0.625776, y: 0.366460 },
8603      { x: -0.619565, y: 0.369565 },
8604      { x: -0.610248, y: 0.378882 },
8605      { x: -0.604037, y: 0.391304 },
8606      { x: -0.604037, y: 0.397516 },
8607      { x: -0.607143, y: 0.400621 },
8608      { x: -0.607143, y: 0.406832 },
8609      { x: -0.610248, y: 0.413043 },
8610      { x: -0.616460, y: 0.419255 },
8611      { x: -0.622671, y: 0.431677 },
8612      { x: -0.635093, y: 0.444099 },
8613      { x: -0.635093, y: 0.447205 },
8614      { x: -0.644410, y: 0.456522 },
8615      { x: -0.644410, y: 0.459627 },
8616      { x: -0.659938, y: 0.478261 },
8617      { x: -0.672360, y: 0.484472 },
8618  ],
8619  [
8620      { x: 0.554348, y: 0.496894 },
8621      { x: 0.532609, y: 0.487578 },
8622      { x: 0.523292, y: 0.468944 },
8623      { x: 0.520186, y: 0.186335 },
8624      { x: 0.495342, y: 0.170807 },
8625      { x: 0.330745, y: 0.170807 },
8626      { x: 0.312112, y: 0.152174 },
8627      { x: 0.309006, y: 0.124224 },
8628      { x: 0.318323, y: 0.105590 },
8629      { x: 0.336957, y: 0.096273 },
8630      { x: 0.501553, y: 0.096273 },
8631      { x: 0.520186, y: 0.080745 },
8632      { x: 0.520186, y: -0.301242 },
8633      { x: 0.526398, y: -0.319876 },
8634      { x: 0.538820, y: -0.332298 },
8635      { x: 0.569876, y: -0.338509 },
8636      { x: 0.591615, y: -0.329193 },
8637      { x: 0.604037, y: -0.304348 },
8638      { x: 0.607143, y: 0.083851 },
8639      { x: 0.625776, y: 0.096273 },
8640      { x: 0.787267, y: 0.096273 },
8641      { x: 0.805901, y: 0.105590 },
8642      { x: 0.818323, y: 0.139752 },
```

```
8643      { x: 0.812112, y: 0.158385 },
8644      { x: 0.799689, y: 0.170807 },
8645      { x: 0.628882, y: 0.170807 },
8646      { x: 0.607143, y: 0.183230 },
8647      { x: 0.604037, y: 0.468944 },
8648      { x: 0.585404, y: 0.493789 },
8649      ],
8650      [
8651      { x: -0.197205, y: 0.496894 },
8652      { x: -0.215839, y: 0.481366 },
8653      { x: -0.305901, y: 0.335404 },
8654      { x: -0.327640, y: 0.316770 },
8655      { x: -0.756211, y: 0.313665 },
8656      { x: -0.771739, y: 0.307453 },
8657      { x: -0.781056, y: 0.301242 },
8658      { x: -0.796584, y: 0.270186 },
8659      { x: -0.793478, y: 0.127329 },
8660      { x: -0.774845, y: 0.111801 },
8661      { x: -0.734472, y: 0.111801 },
8662      { x: -0.722050, y: 0.121118 },
8663      { x: -0.715839, y: 0.136646 },
8664      { x: -0.715839, y: 0.220497 },
8665      { x: -0.694099, y: 0.239130 },
8666      { x: -0.178571, y: 0.239130 },
8667      { x: -0.166149, y: 0.232919 },
8668      { x: -0.156832, y: 0.220497 },
8669      { x: -0.156832, y: 0.142857 },
8670      { x: -0.150621, y: 0.127329 },
8671      { x: -0.128882, y: 0.114907 },
8672      { x: -0.091615, y: 0.118012 },
8673      { x: -0.076087, y: 0.136646 },
8674      { x: -0.076087, y: 0.270186 },
8675      { x: -0.085404, y: 0.291925 },
8676      { x: -0.097826, y: 0.304348 },
8677      { x: -0.113354, y: 0.310559 },
8678      { x: -0.197205, y: 0.313665 },
8679      { x: -0.215839, y: 0.326087 },
8680      { x: -0.215839, y: 0.344720 },
8681      { x: -0.141304, y: 0.462733 },
8682      { x: -0.141304, y: 0.475155 },
8683      { x: -0.156832, y: 0.493789 },
8684      ],
8685      [
8686      { x: 1.296584, y: 0.500000 },
8687      { x: 1.274845, y: 0.490683 },
8688      { x: 1.265528, y: 0.475155 },
8689      { x: 1.268634, y: 0.453416 },
8690      { x: 1.284161, y: 0.425466 },
8691      { x: 1.284161, y: 0.413043 },
8692      { x: 1.265528, y: 0.397516 },
8693      { x: 1.004658, y: 0.394410 },
8694      { x: 0.989130, y: 0.385093 },
8695      { x: 0.976708, y: 0.372671 },
8696      { x: 0.967391, y: 0.347826 },
8697      { x: 0.970497, y: 0.217391 },
8698      { x: 0.992236, y: 0.201863 },
8699      { x: 1.029503, y: 0.204969 },
8700      { x: 1.038820, y: 0.214286 },
```

```
8701      { x: 1.048137, y: 0.239130 },
8702      { x: 1.048137, y: 0.304348 },
8703      { x: 1.072981, y: 0.319876 },
8704      { x: 1.600932, y: 0.319876 },
8705      { x: 1.613354, y: 0.313665 },
8706      { x: 1.622671, y: 0.301242 },
8707      { x: 1.622671, y: 0.239130 },
8708      { x: 1.628882, y: 0.217391 },
8709      { x: 1.641304, y: 0.204969 },
8710      { x: 1.659938, y: 0.198758 },
8711      { x: 1.684783, y: 0.201863 },
8712      { x: 1.700311, y: 0.214286 },
8713      { x: 1.706522, y: 0.226708 },
8714      { x: 1.706522, y: 0.344720 },
8715      { x: 1.697205, y: 0.369565 },
8716      { x: 1.678571, y: 0.388199 },
8717      { x: 1.644410, y: 0.397516 },
8718      { x: 1.389752, y: 0.397516 },
8719      { x: 1.364907, y: 0.416149 },
8720      { x: 1.358696, y: 0.453416 },
8721      { x: 1.346273, y: 0.481366 },
8722      { x: 1.324534, y: 0.496894 },
8723      ],
8724      [
8725      { x: -0.482919, y: 0.500000 },
8726      { x: -0.486025, y: 0.496894 },
8727      { x: -0.495342, y: 0.496894 },
8728      { x: -0.501553, y: 0.493789 },
8729      { x: -0.507764, y: 0.487578 },
8730      { x: -0.513975, y: 0.475155 },
8731      { x: -0.513975, y: 0.462733 },
8732      { x: -0.510870, y: 0.459627 },
8733      { x: -0.510870, y: 0.453416 },
8734      { x: -0.501553, y: 0.434783 },
8735      { x: -0.489130, y: 0.419255 },
8736      { x: -0.486025, y: 0.409938 },
8737      { x: -0.479814, y: 0.403727 },
8738      { x: -0.479814, y: 0.400621 },
8739      { x: -0.473602, y: 0.394410 },
8740      { x: -0.467391, y: 0.381988 },
8741      { x: -0.454969, y: 0.369565 },
8742      { x: -0.448758, y: 0.369565 },
8743      { x: -0.445652, y: 0.366460 },
8744      { x: -0.427019, y: 0.366460 },
8745      { x: -0.423913, y: 0.369565 },
8746      { x: -0.417702, y: 0.369565 },
8747      { x: -0.411491, y: 0.372671 },
8748      { x: -0.402174, y: 0.381988 },
8749      { x: -0.399068, y: 0.388199 },
8750      { x: -0.399068, y: 0.394410 },
8751      { x: -0.395963, y: 0.397516 },
8752      { x: -0.399068, y: 0.400621 },
8753      { x: -0.399068, y: 0.406832 },
8754      { x: -0.408385, y: 0.425466 },
8755      { x: -0.414596, y: 0.431677 },
8756      { x: -0.427019, y: 0.456522 },
8757      { x: -0.439441, y: 0.472050 },
8758      { x: -0.442547, y: 0.481366 },
```

```
8759      { x: -0.454969, y: 0.493789 },
8760      { x: -0.461180, y: 0.496894 },
8761      { x: -0.467391, y: 0.496894 },
8762      { x: -0.470497, y: 0.500000 },
8763      ],
8764      [
8765      { x: -1.336957, y: 0.500000 },
8766      { x: -1.358696, y: 0.493789 },
8767      { x: -1.374224, y: 0.462733 },
8768      { x: -1.374224, y: 0.086957 },
8769      { x: -1.389752, y: 0.071429 },
8770      { x: -1.399068, y: 0.068323 },
8771      { x: -1.684783, y: 0.068323 },
8772      { x: -1.697205, y: 0.059006 },
8773      { x: -1.706522, y: 0.037267 },
8774      { x: -1.700311, y: 0.006211 },
8775      { x: -1.678571, y: -0.009317 },
8776      { x: -1.526398, y: -0.009317 },
8777      { x: -1.513975, y: -0.015528 },
8778      { x: -1.504658, y: -0.027950 },
8779      { x: -1.510870, y: -0.133540 },
8780      { x: -1.523292, y: -0.192547 },
8781      { x: -1.545031, y: -0.257764 },
8782      { x: -1.585404, y: -0.338509 },
8783      { x: -1.631988, y: -0.400621 },
8784      { x: -1.659938, y: -0.428571 },
8785      { x: -1.675466, y: -0.456522 },
8786      { x: -1.669255, y: -0.484472 },
8787      { x: -1.650621, y: -0.500000 },
8788      { x: -1.622671, y: -0.500000 },
8789      { x: -1.604037, y: -0.487578 },
8790      { x: -1.548137, y: -0.425466 },
8791      { x: -1.504658, y: -0.360248 },
8792      { x: -1.464286, y: -0.276398 },
8793      { x: -1.445652, y: -0.220497 },
8794      { x: -1.423913, y: -0.121118 },
8795      { x: -1.417702, y: -0.024845 },
8796      { x: -1.395963, y: -0.009317 },
8797      { x: -1.277950, y: -0.009317 },
8798      { x: -1.268634, y: -0.012422 },
8799      { x: -1.253106, y: -0.031056 },
8800      { x: -1.253106, y: -0.406832 },
8801      { x: -1.240683, y: -0.440994 },
8802      { x: -1.218944, y: -0.465839 },
8803      { x: -1.194099, y: -0.478261 },
8804      { x: -1.166149, y: -0.484472 },
8805      { x: -0.986025, y: -0.484472 },
8806      { x: -0.961180, y: -0.468944 },
8807      { x: -0.951863, y: -0.447205 },
8808      { x: -0.954969, y: -0.425466 },
8809      { x: -0.970497, y: -0.406832 },
8810      { x: -1.138199, y: -0.403727 },
8811      { x: -1.147516, y: -0.400621 },
8812      { x: -1.169255, y: -0.378882 },
8813      { x: -1.166149, y: -0.021739 },
8814      { x: -1.147516, y: -0.009317 },
8815      { x: -0.992236, y: -0.009317 },
8816      { x: -0.967391, y: 0.009317 },
```



```
8817         { x: -0.961180, y: 0.031056 },
8818         { x: -0.967391, y: 0.052795 },
8819         { x: -0.982919, y: 0.068323 },
8820         { x: -1.262422, y: 0.068323 },
8821         { x: -1.287267, y: 0.083851 },
8822         { x: -1.290373, y: 0.475155 },
8823         { x: -1.309006, y: 0.496894 },
8824     ],
8825 ];
8826 /**
8827  * 挖孔三角化填充（归一化坐标；每 6 个数一个三角形 x0,y0,x1,y1,x2,y2）。
8828  * 孔洞区域不在三角网内，运行时只 fill 三角形。
8829  */
8830 export const GAME_TITLE_FILL_TRI_FLAT: readonly number[] = [
8831     1.020186, -0.425466, 1.100932, -0.385093, 1.100932, -0.226708,
8832     1.119565, -0.403727, 1.100932, -0.385093, 1.020186, -0.425466,
8833     1.020186, -0.425466, 1.029503, -0.450311, 1.048137, -0.472050,
8834     1.048137, -0.472050, 1.069876, -0.481366, 1.616460, -0.481366,
8835     1.616460, -0.481366, 1.638199, -0.468944, 1.656832, -0.447205,
8836     1.656832, -0.447205, 1.663043, -0.425466, 1.663043, -0.177019,
8837     1.663043, -0.177019, 1.650621, -0.149068, 1.635093, -0.136646,
8838     1.635093, -0.136646, 1.619565, -0.130435, 1.063665, -0.130435,
8839     1.063665, -0.130435, 1.048137, -0.136646, 1.032609, -0.152174,
8840     1.032609, -0.152174, 1.020186, -0.183230, 1.020186, -0.425466,
8841     1.554348, -0.406832, 1.119565, -0.403727, 1.020186, -0.425466,
8842     1.020186, -0.425466, 1.048137, -0.472050, 1.616460, -0.481366,
8843     1.616460, -0.481366, 1.656832, -0.447205, 1.663043, -0.177019,
8844     1.663043, -0.177019, 1.635093, -0.136646, 1.063665, -0.130435,
8845     1.063665, -0.130435, 1.032609, -0.152174, 1.020186, -0.425466,
8846     1.554348, -0.406832, 1.020186, -0.425466, 1.616460, -0.481366,
8847     1.063665, -0.130435, 1.020186, -0.425466, 1.100932, -0.226708,
8848     1.579193, -0.388199, 1.554348, -0.406832, 1.616460, -0.481366,
8849     1.063665, -0.130435, 1.100932, -0.226708, 1.113354, -0.211180,
8850     1.579193, -0.223602, 1.579193, -0.388199, 1.616460, -0.481366,
8851     1.063665, -0.130435, 1.113354, -0.211180, 1.125776, -0.208075,
8852     1.579193, -0.223602, 1.616460, -0.481366, 1.663043, -0.177019,
8853     1.663043, -0.177019, 1.063665, -0.130435, 1.125776, -0.208075,
8854     1.560559, -0.208075, 1.579193, -0.223602, 1.663043, -0.177019,
8855     1.663043, -0.177019, 1.125776, -0.208075, 1.560559, -0.208075,
8856     0.740683, 0.015528, 0.728261, 0.024845, 0.687888, 0.024845,
8857     0.687888, 0.024845, 0.675466, 0.012422, 0.669255, -0.006211,
8858     0.669255, -0.006211, 0.690994, -0.108696, 0.728261, -0.248447,
8859     0.728261, -0.248447, 0.743789, -0.288820, 0.756211, -0.301242,
8860     0.756211, -0.301242, 0.768634, -0.307453, 0.793478, -0.307453,
8861     0.793478, -0.307453, 0.815217, -0.288820, 0.818323, -0.267081,
8862     0.790373, -0.189441, 0.768634, -0.090062, 0.759317, -0.068323,
8863     0.759317, -0.068323, 0.746894, 0.003106, 0.740683, 0.015528,
8864     0.740683, 0.015528, 0.687888, 0.024845, 0.669255, -0.006211,
8865     0.669255, -0.006211, 0.728261, -0.248447, 0.756211, -0.301242,
8866     0.756211, -0.301242, 0.793478, -0.307453, 0.818323, -0.267081,
8867     0.759317, -0.068323, 0.740683, 0.015528, 0.669255, -0.006211,
8868     0.669255, -0.006211, 0.756211, -0.301242, 0.818323, -0.267081,
8869     0.790373, -0.189441, 0.759317, -0.068323, 0.669255, -0.006211,
8870     0.669255, -0.006211, 0.818323, -0.267081, 0.790373, -0.189441,
8871     0.442547, 0.021739, 0.427019, 0.027950, 0.405280, 0.027950,
8872     0.405280, 0.027950, 0.383540, 0.012422, 0.377329, -0.003106,
8873     0.343168, -0.180124, 0.309006, -0.276398, 0.315217, -0.295031,
8874     0.315217, -0.295031, 0.340062, -0.310559, 0.364907, -0.307453,
```

8875 0.364907, -0.307453, 0.383540, -0.288820, 0.395963, -0.260870,
8876 0.395963, -0.260870, 0.423913, -0.161491, 0.454969, -0.012422,
8877 0.454969, -0.012422, 0.451863, 0.009317, 0.442547, 0.021739,
8878 0.442547, 0.021739, 0.405280, 0.027950, 0.377329, -0.003106,
8879 0.343168, -0.180124, 0.315217, -0.295031, 0.364907, -0.307453,
8880 0.364907, -0.307453, 0.395963, -0.260870, 0.454969, -0.012422,
8881 0.454969, -0.012422, 0.442547, 0.021739, 0.377329, -0.003106,
8882 0.377329, -0.003106, 0.343168, -0.180124, 0.364907, -0.307453,
8883 0.364907, -0.307453, 0.454969, -0.012422, 0.377329, -0.003106,
8884 -0.234472, 0.114907, -0.250000, 0.130435, -0.631988, 0.133540,
8885 -0.631988, 0.133540, -0.647516, 0.111801, -0.647516, 0.083851,
8886 -0.647516, 0.083851, -0.631988, 0.062112, -0.364907, 0.062112,
8887 -0.340062, 0.027950, -0.423913, -0.052795, -0.436335, -0.071429,
8888 -0.458075, -0.124224, -0.762422, -0.124224, -0.787267, -0.136646,
8889 -0.787267, -0.136646, -0.796584, -0.164596, -0.793478, -0.180124,
8890 -0.793478, -0.180124, -0.777950, -0.198758, -0.759317, -0.204969,
8891 -0.473602, -0.406832, -0.585404, -0.406832, -0.604037, -0.422360,
8892 -0.604037, -0.422360, -0.610248, -0.434783, -0.610248, -0.456522,
8893 -0.610248, -0.456522, -0.600932, -0.475155, -0.582298, -0.484472,
8894 -0.582298, -0.484472, -0.411491, -0.481366, -0.383540, -0.462733,
8895 -0.383540, -0.462733, -0.358696, -0.416149, -0.355590, -0.220497,
8896 -0.327640, -0.201863, -0.097826, -0.201863, -0.082298, -0.189441,
8897 -0.082298, -0.189441, -0.076087, -0.177019, -0.076087, -0.145963,
8898 -0.076087, -0.145963, -0.091615, -0.127329, -0.104037, -0.124224,
8899 -0.355590, -0.093168, -0.277950, -0.015528, -0.234472, 0.040373,
8900 -0.234472, 0.040373, -0.228261, 0.062112, -0.228261, 0.096273,
8901 -0.228261, 0.096273, -0.234472, 0.114907, -0.631988, 0.133540,
8902 -0.631988, 0.133540, -0.647516, 0.083851, -0.364907, 0.062112,
8903 -0.340062, 0.027950, -0.436335, -0.071429, -0.439441, -0.108696,
8904 -0.458075, -0.124224, -0.787267, -0.136646, -0.793478, -0.180124,
8905 -0.793478, -0.180124, -0.759317, -0.204969, -0.451863, -0.208075,
8906 -0.473602, -0.406832, -0.604037, -0.422360, -0.610248, -0.456522,
8907 -0.610248, -0.456522, -0.582298, -0.484472, -0.383540, -0.462733,
8908 -0.327640, -0.201863, -0.082298, -0.189441, -0.076087, -0.145963,
8909 -0.076087, -0.145963, -0.104037, -0.124224, -0.336957, -0.124224,
8910 -0.355590, -0.093168, -0.234472, 0.040373, -0.228261, 0.096273,
8911 -0.228261, 0.096273, -0.631988, 0.133540, -0.364907, 0.062112,
8912 -0.458075, -0.124224, -0.793478, -0.180124, -0.451863, -0.208075,
8913 -0.473602, -0.406832, -0.610248, -0.456522, -0.383540, -0.462733,
8914 -0.327640, -0.201863, -0.076087, -0.145963, -0.336957, -0.124224,
8915 -0.228261, 0.096273, -0.364907, 0.062112, -0.340062, 0.046584,
8916 -0.439441, -0.108696, -0.458075, -0.124224, -0.451863, -0.208075,
8917 -0.458075, -0.400621, -0.473602, -0.406832, -0.383540, -0.462733,
8918 -0.343168, -0.208075, -0.327640, -0.201863, -0.336957, -0.124224,
8919 -0.228261, 0.096273, -0.340062, 0.046584, -0.340062, 0.027950,
8920 -0.340062, 0.027950, -0.439441, -0.108696, -0.451863, -0.208075,
8921 -0.442547, -0.385093, -0.458075, -0.400621, -0.383540, -0.462733,
8922 -0.355590, -0.220497, -0.343168, -0.208075, -0.336957, -0.124224,
8923 -0.355590, -0.093168, -0.228261, 0.096273, -0.340062, 0.027950,
8924 -0.340062, 0.027950, -0.451863, -0.208075, -0.439441, -0.220497,
8925 -0.439441, -0.220497, -0.442547, -0.385093, -0.383540, -0.462733,
8926 -0.355590, -0.220497, -0.336957, -0.124224, -0.355590, -0.108696,
8927 -0.355590, -0.093168, -0.340062, 0.027950, -0.439441, -0.220497,
8928 -0.439441, -0.220497, -0.383540, -0.462733, -0.355590, -0.220497,
8929 -0.355590, -0.108696, -0.355590, -0.093168, -0.439441, -0.220497,
8930 -0.439441, -0.220497, -0.355590, -0.220497, -0.355590, -0.108696,
8931 0.222050, 0.170807, 0.203416, 0.183230, 0.097826, 0.183230,
8932 0.097826, 0.183230, 0.079193, 0.161491, 0.076087, 0.142857,

8933 0.076087, 0.142857, 0.082298, 0.127329, 0.088509, 0.118012,
8934 0.088509, 0.118012, 0.100932, 0.111801, 0.135093, 0.111801,
8935 0.147516, -0.360248, 0.094720, -0.416149, 0.069876, -0.450311,
8936 0.069876, -0.450311, 0.072981, -0.478261, 0.088509, -0.496894,
8937 0.088509, -0.496894, 0.122671, -0.500000, 0.190994, -0.431677,
8938 0.228261, -0.434783, 0.268634, -0.462733, 0.318323, -0.481366,
8939 0.318323, -0.481366, 0.812112, -0.481366, 0.827640, -0.472050,
8940 0.827640, -0.472050, 0.836957, -0.456522, 0.840062, -0.434783,
8941 0.840062, -0.434783, 0.827640, -0.409938, 0.815217, -0.403727,
8942 0.234472, -0.326087, 0.234472, 0.149068, 0.222050, 0.170807,
8943 0.222050, 0.170807, 0.097826, 0.183230, 0.076087, 0.142857,
8944 0.076087, 0.142857, 0.088509, 0.118012, 0.135093, 0.111801,
8945 0.147516, -0.360248, 0.069876, -0.450311, 0.088509, -0.496894,
8946 0.228261, -0.434783, 0.318323, -0.481366, 0.827640, -0.472050,
8947 0.827640, -0.472050, 0.840062, -0.434783, 0.815217, -0.403727,
8948 0.222050, 0.170807, 0.076087, 0.142857, 0.135093, 0.111801,
8949 0.147516, -0.360248, 0.088509, -0.496894, 0.190994, -0.431677,
8950 0.206522, -0.425466, 0.228261, -0.434783, 0.827640, -0.472050,
8951 0.827640, -0.472050, 0.815217, -0.403727, 0.349379, -0.403727,
8952 0.222050, 0.170807, 0.135093, 0.111801, 0.156832, 0.093168,
8953 0.156832, -0.332298, 0.147516, -0.360248, 0.190994, -0.431677,
8954 0.206522, -0.425466, 0.827640, -0.472050, 0.349379, -0.403727,
8955 0.234472, -0.326087, 0.222050, 0.170807, 0.156832, 0.093168,
8956 0.156832, 0.093168, 0.156832, -0.332298, 0.190994, -0.431677,
8957 0.206522, -0.425466, 0.349379, -0.403727, 0.309006, -0.391304,
8958 0.234472, -0.326087, 0.156832, 0.093168, 0.190994, -0.431677,
8959 0.206522, -0.425466, 0.309006, -0.391304, 0.256211, -0.357143,
8960 0.240683, -0.341615, 0.234472, -0.326087, 0.190994, -0.431677,
8961 0.190994, -0.431677, 0.206522, -0.425466, 0.256211, -0.357143,
8962 0.256211, -0.357143, 0.240683, -0.341615, 0.190994, -0.431677,
8963 1.100932, -0.000000, 1.181677, 0.031056, 1.181677, 0.136646,
8964 1.197205, 0.018634, 1.181677, 0.031056, 1.100932, -0.000000,
8965 1.100932, -0.000000, 1.113354, -0.034161, 1.128882, -0.049689,
8966 1.128882, -0.049689, 1.147516, -0.055901, 1.520186, -0.055901,
8967 1.520186, -0.055901, 1.541925, -0.046584, 1.557453, -0.027950,
8968 1.557453, -0.027950, 1.563665, -0.012422, 1.563665, 0.180124,
8969 1.563665, 0.180124, 1.551242, 0.204969, 1.532609, 0.220497,
8970 1.532609, 0.220497, 1.147516, 0.223602, 1.128882, 0.217391,
8971 1.128882, 0.217391, 1.113354, 0.201863, 1.100932, 0.164596,
8972 1.100932, 0.164596, 1.100932, -0.000000, 1.181677, 0.136646,
8973 1.461180, 0.015528, 1.197205, 0.018634, 1.100932, -0.000000,
8974 1.100932, -0.000000, 1.128882, -0.049689, 1.520186, -0.055901,
8975 1.520186, -0.055901, 1.557453, -0.027950, 1.563665, 0.180124,
8976 1.563665, 0.180124, 1.532609, 0.220497, 1.128882, 0.217391,
8977 1.128882, 0.217391, 1.100932, 0.164596, 1.181677, 0.136646,
8978 1.461180, 0.015528, 1.100932, -0.000000, 1.520186, -0.055901,
8979 1.128882, 0.217391, 1.181677, 0.136646, 1.197205, 0.152174,
8980 1.486025, 0.034161, 1.461180, 0.015528, 1.520186, -0.055901,
8981 1.563665, 0.180124, 1.128882, 0.217391, 1.197205, 0.152174,
8982 1.486025, 0.136646, 1.486025, 0.034161, 1.520186, -0.055901,
8983 1.563665, 0.180124, 1.197205, 0.152174, 1.206522, 0.152174,
8984 1.486025, 0.136646, 1.520186, -0.055901, 1.563665, 0.180124,
8985 1.563665, 0.180124, 1.206522, 0.152174, 1.470497, 0.152174,
8986 1.470497, 0.152174, 1.486025, 0.136646, 1.563665, 0.180124,
8987 -1.023292, 0.422360, -1.035714, 0.434783, -1.072981, 0.437888,
8988 -1.072981, 0.437888, -1.088509, 0.428571, -1.104037, 0.397516,
8989 -1.131988, 0.319876, -1.187888, 0.201863, -1.194099, 0.170807,
8990 -1.194099, 0.170807, -1.187888, 0.158385, -1.172360, 0.145963,

8991 -1.172360, 0.145963, -1.141304, 0.145963, -1.119565, 0.167702,
8992 -1.119565, 0.167702, -1.020186, 0.394410, -1.023292, 0.422360,
8993 -1.023292, 0.422360, -1.072981, 0.437888, -1.104037, 0.397516,
8994 -1.131988, 0.319876, -1.194099, 0.170807, -1.172360, 0.145963,
8995 -1.172360, 0.145963, -1.119565, 0.167702, -1.023292, 0.422360,
8996 -1.023292, 0.422360, -1.104037, 0.397516, -1.131988, 0.319876,
8997 -1.131988, 0.319876, -1.172360, 0.145963, -1.023292, 0.422360,
8998 -1.582298, 0.431677, -1.594720, 0.437888, -1.628882, 0.437888,
8999 -1.628882, 0.437888, -1.644410, 0.428571, -1.653727, 0.403727,
9000 -1.594720, 0.276398, -1.560559, 0.186335, -1.538820, 0.149068,
9001 -1.538820, 0.149068, -1.523292, 0.142857, -1.495342, 0.145963,
9002 -1.495342, 0.145963, -1.476708, 0.164596, -1.473602, 0.189441,
9003 -1.541925, 0.344720, -1.560559, 0.397516, -1.582298, 0.431677,
9004 -1.582298, 0.431677, -1.628882, 0.437888, -1.653727, 0.403727,
9005 -1.594720, 0.276398, -1.538820, 0.149068, -1.495342, 0.145963,
9006 -1.495342, 0.145963, -1.473602, 0.189441, -1.541925, 0.344720,
9007 -1.541925, 0.344720, -1.582298, 0.431677, -1.653727, 0.403727,
9008 -1.653727, 0.403727, -1.594720, 0.276398, -1.495342, 0.145963,
9009 -1.495342, 0.145963, -1.541925, 0.344720, -1.653727, 0.403727,
9010 0.793478, 0.450311, 0.787267, 0.453416, 0.750000, 0.453416,
9011 0.750000, 0.453416, 0.740683, 0.447205, 0.740683, 0.444099,
9012 0.740683, 0.444099, 0.734472, 0.437888, 0.725155, 0.419255,
9013 0.725155, 0.413043, 0.722050, 0.409938, 0.718944, 0.394410,
9014 0.718944, 0.394410, 0.712733, 0.385093, 0.712733, 0.378882,
9015 0.712733, 0.378882, 0.706522, 0.369565, 0.703416, 0.354037,
9016 0.703416, 0.354037, 0.694099, 0.338509, 0.694099, 0.332298,
9017 0.694099, 0.332298, 0.684783, 0.316770, 0.684783, 0.310559,
9018 0.684783, 0.310559, 0.675466, 0.295031, 0.675466, 0.288820,
9019 0.675466, 0.288820, 0.669255, 0.276398, 0.669255, 0.260870,
9020 0.669255, 0.260870, 0.672360, 0.254658, 0.681677, 0.245342,
9021 0.681677, 0.245342, 0.687888, 0.242236, 0.697205, 0.242236,
9022 0.697205, 0.242236, 0.700311, 0.239130, 0.715839, 0.239130,
9023 0.728261, 0.248447, 0.731366, 0.248447, 0.731366, 0.251553,
9024 0.731366, 0.251553, 0.737578, 0.257764, 0.756211, 0.295031,
9025 0.756211, 0.301242, 0.762422, 0.310559, 0.762422, 0.316770,
9026 0.762422, 0.316770, 0.771739, 0.329193, 0.771739, 0.335404,
9027 0.771739, 0.335404, 0.784161, 0.360248, 0.787267, 0.375776,
9028 0.787267, 0.375776, 0.796584, 0.391304, 0.796584, 0.397516,
9029 0.796584, 0.397516, 0.805901, 0.416149, 0.805901, 0.434783,
9030 0.805901, 0.434783, 0.793478, 0.450311, 0.750000, 0.453416,
9031 0.750000, 0.453416, 0.740683, 0.444099, 0.725155, 0.419255,
9032 0.718944, 0.394410, 0.712733, 0.378882, 0.703416, 0.354037,
9033 0.703416, 0.354037, 0.694099, 0.332298, 0.684783, 0.310559,
9034 0.684783, 0.310559, 0.675466, 0.288820, 0.669255, 0.260870,
9035 0.669255, 0.260870, 0.681677, 0.245342, 0.697205, 0.242236,
9036 0.697205, 0.242236, 0.715839, 0.239130, 0.728261, 0.248447,
9037 0.728261, 0.248447, 0.731366, 0.251553, 0.756211, 0.295031,
9038 0.762422, 0.316770, 0.771739, 0.335404, 0.787267, 0.375776,
9039 0.787267, 0.375776, 0.796584, 0.397516, 0.805901, 0.434783,
9040 0.805901, 0.434783, 0.750000, 0.453416, 0.725155, 0.419255,
9041 0.703416, 0.354037, 0.684783, 0.310559, 0.669255, 0.260870,
9042 0.669255, 0.260870, 0.697205, 0.242236, 0.728261, 0.248447,
9043 0.728261, 0.248447, 0.756211, 0.295031, 0.756211, 0.301242,
9044 0.762422, 0.316770, 0.787267, 0.375776, 0.805901, 0.434783,
9045 0.805901, 0.434783, 0.725155, 0.419255, 0.725155, 0.413043,
9046 0.718944, 0.394410, 0.703416, 0.354037, 0.669255, 0.260870,
9047 0.669255, 0.260870, 0.728261, 0.248447, 0.756211, 0.301242,
9048 0.756211, 0.301242, 0.762422, 0.316770, 0.805901, 0.434783,

9049 0.805901, 0.434783, 0.725155, 0.413043, 0.718944, 0.394410,
9050 0.718944, 0.394410, 0.669255, 0.260870, 0.756211, 0.301242,
9051 0.756211, 0.301242, 0.805901, 0.434783, 0.718944, 0.394410,
9052 0.358696, 0.450311, 0.355590, 0.453416, 0.346273, 0.453416,
9053 0.343168, 0.450311, 0.333851, 0.450311, 0.330745, 0.447205,
9054 0.330745, 0.447205, 0.324534, 0.447205, 0.318323, 0.440994,
9055 0.318323, 0.440994, 0.312112, 0.428571, 0.312112, 0.413043,
9056 0.315217, 0.409938, 0.318323, 0.394410, 0.324534, 0.385093,
9057 0.324534, 0.385093, 0.324534, 0.378882, 0.327640, 0.372671,
9058 0.333851, 0.366460, 0.333851, 0.360248, 0.343168, 0.344720,
9059 0.343168, 0.344720, 0.343168, 0.338509, 0.349379, 0.329193,
9060 0.349379, 0.329193, 0.349379, 0.322981, 0.355590, 0.313665,
9061 0.355590, 0.313665, 0.355590, 0.307453, 0.361801, 0.298137,
9062 0.361801, 0.298137, 0.361801, 0.291925, 0.368012, 0.282609,
9063 0.368012, 0.282609, 0.368012, 0.276398, 0.380435, 0.251553,
9064 0.380435, 0.251553, 0.389752, 0.242236, 0.399068, 0.242236,
9065 0.399068, 0.242236, 0.402174, 0.239130, 0.414596, 0.239130,
9066 0.417702, 0.242236, 0.423913, 0.242236, 0.439441, 0.251553,
9067 0.439441, 0.251553, 0.445652, 0.263975, 0.445652, 0.270186,
9068 0.445652, 0.270186, 0.448758, 0.273292, 0.445652, 0.276398,
9069 0.445652, 0.276398, 0.445652, 0.282609, 0.442547, 0.285714,
9070 0.442547, 0.285714, 0.439441, 0.301242, 0.430124, 0.316770,
9071 0.430124, 0.316770, 0.430124, 0.322981, 0.414596, 0.354037,
9072 0.414596, 0.354037, 0.414596, 0.360248, 0.405280, 0.378882,
9073 0.405280, 0.378882, 0.405280, 0.385093, 0.395963, 0.400621,
9074 0.395963, 0.400621, 0.395963, 0.406832, 0.380435, 0.437888,
9075 0.380435, 0.437888, 0.371118, 0.447205, 0.364907, 0.450311,
9076 0.358696, 0.450311, 0.346273, 0.453416, 0.343168, 0.450311,
9077 0.343168, 0.450311, 0.330745, 0.447205, 0.318323, 0.440994,
9078 0.318323, 0.440994, 0.312112, 0.413043, 0.315217, 0.409938,
9079 0.324534, 0.385093, 0.327640, 0.372671, 0.333851, 0.366460,
9080 0.349379, 0.329193, 0.355590, 0.313665, 0.361801, 0.298137,
9081 0.361801, 0.298137, 0.368012, 0.282609, 0.380435, 0.251553,
9082 0.380435, 0.251553, 0.399068, 0.242236, 0.414596, 0.239130,
9083 0.417702, 0.242236, 0.439441, 0.251553, 0.445652, 0.270186,
9084 0.445652, 0.270186, 0.445652, 0.276398, 0.442547, 0.285714,
9085 0.442547, 0.285714, 0.430124, 0.316770, 0.414596, 0.354037,
9086 0.414596, 0.354037, 0.405280, 0.378882, 0.395963, 0.400621,
9087 0.395963, 0.400621, 0.380435, 0.437888, 0.364907, 0.450311,
9088 0.358696, 0.450311, 0.343168, 0.450311, 0.318323, 0.440994,
9089 0.318323, 0.440994, 0.315217, 0.409938, 0.324534, 0.385093,
9090 0.349379, 0.329193, 0.361801, 0.298137, 0.380435, 0.251553,
9091 0.380435, 0.251553, 0.414596, 0.239130, 0.417702, 0.242236,
9092 0.417702, 0.242236, 0.445652, 0.270186, 0.442547, 0.285714,
9093 0.414596, 0.354037, 0.395963, 0.400621, 0.364907, 0.450311,
9094 0.364907, 0.450311, 0.358696, 0.450311, 0.318323, 0.440994,
9095 0.318323, 0.440994, 0.324534, 0.385093, 0.333851, 0.366460,
9096 0.349379, 0.329193, 0.380435, 0.251553, 0.417702, 0.242236,
9097 0.417702, 0.242236, 0.442547, 0.285714, 0.414596, 0.354037,
9098 0.414596, 0.354037, 0.364907, 0.450311, 0.318323, 0.440994,
9099 0.318323, 0.440994, 0.333851, 0.366460, 0.343168, 0.344720,
9100 0.343168, 0.344720, 0.349379, 0.329193, 0.417702, 0.242236,
9101 0.417702, 0.242236, 0.414596, 0.354037, 0.318323, 0.440994,
9102 0.318323, 0.440994, 0.343168, 0.344720, 0.417702, 0.242236,
9103 0.166149, 0.444099, 0.153727, 0.453416, 0.131988, 0.456522,
9104 0.128882, 0.453416, 0.119565, 0.453416, 0.104037, 0.444099,
9105 0.104037, 0.444099, 0.097826, 0.431677, 0.097826, 0.416149,
9106 0.097826, 0.416149, 0.107143, 0.397516, 0.113354, 0.391304,

9107 0.113354, 0.391304, 0.122671, 0.372671, 0.128882, 0.366460,
9108 0.128882, 0.366460, 0.128882, 0.360248, 0.135093, 0.354037,
9109 0.135093, 0.354037, 0.163043, 0.298137, 0.178571, 0.282609,
9110 0.178571, 0.282609, 0.184783, 0.279503, 0.209627, 0.279503,
9111 0.209627, 0.279503, 0.222050, 0.285714, 0.231366, 0.298137,
9112 0.231366, 0.304348, 0.234472, 0.307453, 0.234472, 0.319876,
9113 0.231366, 0.322981, 0.231366, 0.329193, 0.222050, 0.347826,
9114 0.215839, 0.354037, 0.203416, 0.381988, 0.197205, 0.388199,
9115 0.197205, 0.388199, 0.190994, 0.403727, 0.184783, 0.409938,
9116 0.184783, 0.409938, 0.178571, 0.425466, 0.172360, 0.431677,
9117 0.172360, 0.431677, 0.166149, 0.444099, 0.131988, 0.456522,
9118 0.128882, 0.453416, 0.104037, 0.444099, 0.097826, 0.416149,
9119 0.128882, 0.366460, 0.135093, 0.354037, 0.178571, 0.282609,
9120 0.178571, 0.282609, 0.209627, 0.279503, 0.231366, 0.298137,
9121 0.231366, 0.304348, 0.234472, 0.319876, 0.231366, 0.322981,
9122 0.231366, 0.322981, 0.222050, 0.347826, 0.215839, 0.354037,
9123 0.215839, 0.354037, 0.197205, 0.388199, 0.184783, 0.409938,
9124 0.184783, 0.409938, 0.172360, 0.431677, 0.131988, 0.456522,
9125 0.131988, 0.456522, 0.128882, 0.453416, 0.097826, 0.416149,
9126 0.128882, 0.366460, 0.178571, 0.282609, 0.231366, 0.298137,
9127 0.231366, 0.304348, 0.231366, 0.322981, 0.215839, 0.354037,
9128 0.215839, 0.354037, 0.184783, 0.409938, 0.131988, 0.456522,
9129 0.131988, 0.456522, 0.097826, 0.416149, 0.113354, 0.391304,
9130 0.113354, 0.391304, 0.128882, 0.366460, 0.231366, 0.298137,
9131 0.231366, 0.298137, 0.231366, 0.304348, 0.215839, 0.354037,
9132 0.215839, 0.354037, 0.131988, 0.456522, 0.113354, 0.391304,
9133 0.113354, 0.391304, 0.231366, 0.298137, 0.215839, 0.354037,
9134 -0.659938, 0.478261, -0.672360, 0.484472, -0.703416, 0.484472,
9135 -0.703416, 0.484472, -0.715839, 0.478261, -0.725155, 0.462733,
9136 -0.725155, 0.462733, -0.725155, 0.450311, -0.715839, 0.431677,
9137 -0.706522, 0.422360, -0.706522, 0.419255, -0.697205, 0.409938,
9138 -0.697205, 0.409938, -0.697205, 0.406832, -0.687888, 0.397516,
9139 -0.687888, 0.397516, -0.687888, 0.394410, -0.663043, 0.369565,
9140 -0.663043, 0.369565, -0.656832, 0.366460, -0.625776, 0.366460,
9141 -0.625776, 0.366460, -0.619565, 0.369565, -0.610248, 0.378882,
9142 -0.610248, 0.378882, -0.604037, 0.391304, -0.604037, 0.397516,
9143 -0.607143, 0.400621, -0.607143, 0.406832, -0.610248, 0.413043,
9144 -0.616460, 0.419255, -0.622671, 0.431677, -0.635093, 0.444099,
9145 -0.635093, 0.444099, -0.635093, 0.447205, -0.644410, 0.456522,
9146 -0.644410, 0.456522, -0.644410, 0.459627, -0.659938, 0.478261,
9147 -0.659938, 0.478261, -0.703416, 0.484472, -0.725155, 0.462733,
9148 -0.725155, 0.462733, -0.715839, 0.431677, -0.706522, 0.422360,
9149 -0.697205, 0.409938, -0.687888, 0.397516, -0.663043, 0.369565,
9150 -0.663043, 0.369565, -0.625776, 0.366460, -0.610248, 0.378882,
9151 -0.610248, 0.378882, -0.604037, 0.397516, -0.607143, 0.400621,
9152 -0.607143, 0.400621, -0.610248, 0.413043, -0.616460, 0.419255,
9153 -0.616460, 0.419255, -0.635093, 0.444099, -0.644410, 0.456522,
9154 -0.644410, 0.456522, -0.659938, 0.478261, -0.725155, 0.462733,
9155 -0.725155, 0.462733, -0.706522, 0.422360, -0.697205, 0.409938,
9156 -0.697205, 0.409938, -0.663043, 0.369565, -0.610248, 0.378882,
9157 -0.610248, 0.378882, -0.607143, 0.400621, -0.616460, 0.419255,
9158 -0.616460, 0.419255, -0.644410, 0.456522, -0.725155, 0.462733,
9159 -0.725155, 0.462733, -0.697205, 0.409938, -0.610248, 0.378882,
9160 -0.610248, 0.378882, -0.616460, 0.419255, -0.725155, 0.462733,
9161 0.604037, 0.468944, 0.585404, 0.493789, 0.554348, 0.496894,
9162 0.554348, 0.496894, 0.532609, 0.487578, 0.523292, 0.468944,
9163 0.495342, 0.170807, 0.330745, 0.170807, 0.312112, 0.152174,
9164 0.312112, 0.152174, 0.309006, 0.124224, 0.318323, 0.105590,

9165 0.318323, 0.105590, 0.336957, 0.096273, 0.501553, 0.096273,
9166 0.520186, 0.080745, 0.520186, -0.301242, 0.526398, -0.319876,
9167 0.526398, -0.319876, 0.538820, -0.332298, 0.569876, -0.338509,
9168 0.569876, -0.338509, 0.591615, -0.329193, 0.604037, -0.304348,
9169 0.625776, 0.096273, 0.787267, 0.096273, 0.805901, 0.105590,
9170 0.805901, 0.105590, 0.818323, 0.139752, 0.812112, 0.158385,
9171 0.812112, 0.158385, 0.799689, 0.170807, 0.628882, 0.170807,
9172 0.607143, 0.183230, 0.604037, 0.468944, 0.554348, 0.496894,
9173 0.554348, 0.496894, 0.523292, 0.468944, 0.520186, 0.186335,
9174 0.495342, 0.170807, 0.312112, 0.152174, 0.318323, 0.105590,
9175 0.520186, 0.080745, 0.526398, -0.319876, 0.569876, -0.338509,
9176 0.569876, -0.338509, 0.604037, -0.304348, 0.607143, 0.083851,
9177 0.625776, 0.096273, 0.805901, 0.105590, 0.812112, 0.158385,
9178 0.607143, 0.183230, 0.554348, 0.496894, 0.520186, 0.186335,
9179 0.495342, 0.170807, 0.318323, 0.105590, 0.501553, 0.096273,
9180 0.520186, 0.080745, 0.569876, -0.338509, 0.607143, 0.083851,
9181 0.625776, 0.096273, 0.812112, 0.158385, 0.628882, 0.170807,
9182 0.628882, 0.170807, 0.607143, 0.183230, 0.520186, 0.186335,
9183 0.520186, 0.186335, 0.495342, 0.170807, 0.501553, 0.096273,
9184 0.501553, 0.096273, 0.520186, 0.080745, 0.607143, 0.083851,
9185 0.607143, 0.083851, 0.625776, 0.096273, 0.628882, 0.170807,
9186 0.628882, 0.170807, 0.520186, 0.186335, 0.501553, 0.096273,
9187 0.501553, 0.096273, 0.607143, 0.083851, 0.628882, 0.170807,
9188 -0.141304, 0.475155, -0.156832, 0.493789, -0.197205, 0.496894,
9189 -0.197205, 0.496894, -0.215839, 0.481366, -0.305901, 0.335404,
9190 -0.327640, 0.316770, -0.756211, 0.313665, -0.771739, 0.307453,
9191 -0.771739, 0.307453, -0.781056, 0.301242, -0.796584, 0.270186,
9192 -0.796584, 0.270186, -0.793478, 0.127329, -0.774845, 0.111801,
9193 -0.774845, 0.111801, -0.734472, 0.111801, -0.722050, 0.121118,
9194 -0.722050, 0.121118, -0.715839, 0.136646, -0.715839, 0.220497,
9195 -0.156832, 0.220497, -0.156832, 0.142857, -0.150621, 0.127329,
9196 -0.150621, 0.127329, -0.128882, 0.114907, -0.091615, 0.118012,
9197 -0.091615, 0.118012, -0.076087, 0.136646, -0.076087, 0.270186,
9198 -0.076087, 0.270186, -0.085404, 0.291925, -0.097826, 0.304348,
9199 -0.097826, 0.304348, -0.113354, 0.310559, -0.197205, 0.313665,
9200 -0.215839, 0.344720, -0.141304, 0.462733, -0.141304, 0.475155,
9201 -0.141304, 0.475155, -0.197205, 0.496894, -0.305901, 0.335404,
9202 -0.327640, 0.316770, -0.771739, 0.307453, -0.796584, 0.270186,
9203 -0.796584, 0.270186, -0.774845, 0.111801, -0.722050, 0.121118,
9204 -0.156832, 0.220497, -0.150621, 0.127329, -0.091615, 0.118012,
9205 -0.091615, 0.118012, -0.076087, 0.270186, -0.097826, 0.304348,
9206 -0.215839, 0.344720, -0.141304, 0.475155, -0.305901, 0.335404,
9207 -0.796584, 0.270186, -0.722050, 0.121118, -0.715839, 0.220497,
9208 -0.156832, 0.220497, -0.091615, 0.118012, -0.097826, 0.304348,
9209 -0.215839, 0.326087, -0.215839, 0.344720, -0.305901, 0.335404,
9210 -0.796584, 0.270186, -0.715839, 0.220497, -0.694099, 0.239130,
9211 -0.166149, 0.232919, -0.156832, 0.220497, -0.097826, 0.304348,
9212 -0.197205, 0.313665, -0.215839, 0.326087, -0.305901, 0.335404,
9213 -0.327640, 0.316770, -0.796584, 0.270186, -0.694099, 0.239130,
9214 -0.178571, 0.239130, -0.166149, 0.232919, -0.097826, 0.304348,
9215 -0.197205, 0.313665, -0.305901, 0.335404, -0.327640, 0.316770,
9216 -0.327640, 0.316770, -0.694099, 0.239130, -0.178571, 0.239130,
9217 -0.178571, 0.239130, -0.097826, 0.304348, -0.197205, 0.313665,
9218 -0.197205, 0.313665, -0.327640, 0.316770, -0.178571, 0.239130,
9219 1.346273, 0.481366, 1.324534, 0.496894, 1.296584, 0.500000,
9220 1.296584, 0.500000, 1.274845, 0.490683, 1.265528, 0.475155,
9221 1.265528, 0.475155, 1.268634, 0.453416, 1.284161, 0.425466,
9222 1.265528, 0.397516, 1.004658, 0.394410, 0.989130, 0.385093,

9223 0.989130, 0.385093, 0.976708, 0.372671, 0.967391, 0.347826,
9224 0.967391, 0.347826, 0.970497, 0.217391, 0.992236, 0.201863,
9225 0.992236, 0.201863, 1.029503, 0.204969, 1.038820, 0.214286,
9226 1.038820, 0.214286, 1.048137, 0.239130, 1.048137, 0.304348,
9227 1.622671, 0.301242, 1.622671, 0.239130, 1.628882, 0.217391,
9228 1.628882, 0.217391, 1.641304, 0.204969, 1.659938, 0.198758,
9229 1.659938, 0.198758, 1.684783, 0.201863, 1.700311, 0.214286,
9230 1.700311, 0.214286, 1.706522, 0.226708, 1.706522, 0.344720,
9231 1.706522, 0.344720, 1.697205, 0.369565, 1.678571, 0.388199,
9232 1.678571, 0.388199, 1.644410, 0.397516, 1.389752, 0.397516,
9233 1.364907, 0.416149, 1.358696, 0.453416, 1.346273, 0.481366,
9234 1.346273, 0.481366, 1.296584, 0.500000, 1.265528, 0.475155,
9235 1.265528, 0.397516, 0.989130, 0.385093, 0.967391, 0.347826,
9236 0.967391, 0.347826, 0.992236, 0.201863, 1.038820, 0.214286,
9237 1.622671, 0.301242, 1.628882, 0.217391, 1.659938, 0.198758,
9238 1.659938, 0.198758, 1.700311, 0.214286, 1.706522, 0.344720,
9239 1.706522, 0.344720, 1.678571, 0.388199, 1.389752, 0.397516,
9240 1.364907, 0.416149, 1.346273, 0.481366, 1.265528, 0.475155,
9241 0.967391, 0.347826, 1.038820, 0.214286, 1.048137, 0.304348,
9242 1.622671, 0.301242, 1.659938, 0.198758, 1.706522, 0.344720,
9243 1.389752, 0.397516, 1.364907, 0.416149, 1.265528, 0.475155,
9244 0.967391, 0.347826, 1.048137, 0.304348, 1.072981, 0.319876,
9245 1.613354, 0.313665, 1.622671, 0.301242, 1.706522, 0.344720,
9246 1.389752, 0.397516, 1.265528, 0.475155, 1.284161, 0.425466,
9247 1.265528, 0.397516, 0.967391, 0.347826, 1.072981, 0.319876,
9248 1.600932, 0.319876, 1.613354, 0.313665, 1.706522, 0.344720,
9249 1.389752, 0.397516, 1.284161, 0.425466, 1.284161, 0.413043,
9250 1.265528, 0.397516, 1.072981, 0.319876, 1.600932, 0.319876,
9251 1.600932, 0.319876, 1.706522, 0.344720, 1.389752, 0.397516,
9252 1.389752, 0.397516, 1.284161, 0.413043, 1.265528, 0.397516,
9253 1.265528, 0.397516, 1.600932, 0.319876, 1.389752, 0.397516,
9254 -0.467391, 0.496894, -0.470497, 0.500000, -0.482919, 0.500000,
9255 -0.486025, 0.496894, -0.495342, 0.496894, -0.501553, 0.493789,
9256 -0.501553, 0.493789, -0.507764, 0.487578, -0.513975, 0.475155,
9257 -0.513975, 0.475155, -0.513975, 0.462733, -0.510870, 0.459627,
9258 -0.510870, 0.459627, -0.510870, 0.453416, -0.501553, 0.434783,
9259 -0.489130, 0.419255, -0.486025, 0.409938, -0.479814, 0.403727,
9260 -0.479814, 0.403727, -0.479814, 0.400621, -0.473602, 0.394410,
9261 -0.473602, 0.394410, -0.467391, 0.381988, -0.454969, 0.369565,
9262 -0.448758, 0.369565, -0.445652, 0.366460, -0.427019, 0.366460,
9263 -0.423913, 0.369565, -0.417702, 0.369565, -0.411491, 0.372671,
9264 -0.411491, 0.372671, -0.402174, 0.381988, -0.399068, 0.388199,
9265 -0.399068, 0.394410, -0.395963, 0.397516, -0.399068, 0.400621,
9266 -0.399068, 0.400621, -0.399068, 0.406832, -0.408385, 0.425466,
9267 -0.414596, 0.431677, -0.427019, 0.456522, -0.439441, 0.472050,
9268 -0.439441, 0.472050, -0.442547, 0.481366, -0.454969, 0.493789,
9269 -0.454969, 0.493789, -0.461180, 0.496894, -0.467391, 0.496894,
9270 -0.467391, 0.496894, -0.482919, 0.500000, -0.486025, 0.496894,
9271 -0.486025, 0.496894, -0.501553, 0.493789, -0.513975, 0.475155,
9272 -0.513975, 0.475155, -0.510870, 0.459627, -0.501553, 0.434783,
9273 -0.489130, 0.419255, -0.479814, 0.403727, -0.473602, 0.394410,
9274 -0.473602, 0.394410, -0.454969, 0.369565, -0.448758, 0.369565,
9275 -0.448758, 0.369565, -0.427019, 0.366460, -0.423913, 0.369565,
9276 -0.423913, 0.369565, -0.411491, 0.372671, -0.399068, 0.388199,
9277 -0.399068, 0.394410, -0.399068, 0.400621, -0.408385, 0.425466,
9278 -0.414596, 0.431677, -0.439441, 0.472050, -0.454969, 0.493789,
9279 -0.454969, 0.493789, -0.467391, 0.496894, -0.486025, 0.496894,
9280 -0.486025, 0.496894, -0.513975, 0.475155, -0.501553, 0.434783,

9281 -0.489130, 0.419255, -0.473602, 0.394410, -0.448758, 0.369565,
9282 -0.448758, 0.369565, -0.423913, 0.369565, -0.399068, 0.388199,
9283 -0.399068, 0.388199, -0.399068, 0.394410, -0.408385, 0.425466,
9284 -0.414596, 0.431677, -0.454969, 0.493789, -0.486025, 0.496894,
9285 -0.486025, 0.496894, -0.501553, 0.434783, -0.489130, 0.419255,
9286 -0.489130, 0.419255, -0.448758, 0.369565, -0.399068, 0.388199,
9287 -0.399068, 0.388199, -0.408385, 0.425466, -0.414596, 0.431677,
9288 -0.414596, 0.431677, -0.486025, 0.496894, -0.489130, 0.419255,
9289 -0.489130, 0.419255, -0.399068, 0.388199, -0.414596, 0.431677,
9290 -1.290373, 0.475155, -1.309006, 0.496894, -1.336957, 0.500000,
9291 -1.336957, 0.500000, -1.358696, 0.493789, -1.374224, 0.462733,
9292 -1.399068, 0.068323, -1.684783, 0.068323, -1.697205, 0.059006,
9293 -1.697205, 0.059006, -1.706522, 0.037267, -1.700311, 0.006211,
9294 -1.700311, 0.006211, -1.678571, -0.009317, -1.526398, -0.009317,
9295 -1.631988, -0.400621, -1.659938, -0.428571, -1.675466, -0.456522,
9296 -1.675466, -0.456522, -1.669255, -0.484472, -1.650621, -0.500000,
9297 -1.650621, -0.500000, -1.622671, -0.500000, -1.604037, -0.487578,
9298 -1.604037, -0.487578, -1.548137, -0.425466, -1.504658, -0.360248,
9299 -1.504658, -0.360248, -1.464286, -0.276398, -1.445652, -0.220497,
9300 -1.445652, -0.220497, -1.423913, -0.121118, -1.417702, -0.024845,
9301 -1.253106, -0.031056, -1.253106, -0.406832, -1.240683, -0.440994,
9302 -1.240683, -0.440994, -1.218944, -0.465839, -1.194099, -0.478261,
9303 -1.194099, -0.478261, -1.166149, -0.484472, -0.986025, -0.484472,
9304 -0.986025, -0.484472, -0.961180, -0.468944, -0.951863, -0.447205,
9305 -0.951863, -0.447205, -0.954969, -0.425466, -0.970497, -0.406832,
9306 -1.147516, -0.009317, -0.992236, -0.009317, -0.967391, 0.009317,
9307 -0.967391, 0.009317, -0.961180, 0.031056, -0.967391, 0.052795,
9308 -0.967391, 0.052795, -0.982919, 0.068323, -1.262422, 0.068323,
9309 -1.287267, 0.083851, -1.290373, 0.475155, -1.336957, 0.500000,
9310 -1.336957, 0.500000, -1.374224, 0.462733, -1.374224, 0.086957,
9311 -1.399068, 0.068323, -1.697205, 0.059006, -1.700311, 0.006211,
9312 -1.631988, -0.400621, -1.675466, -0.456522, -1.650621, -0.500000,
9313 -1.650621, -0.500000, -1.604037, -0.487578, -1.504658, -0.360248,
9314 -1.504658, -0.360248, -1.445652, -0.220497, -1.417702, -0.024845,
9315 -1.253106, -0.031056, -1.240683, -0.440994, -1.194099, -0.478261,
9316 -1.194099, -0.478261, -0.986025, -0.484472, -0.951863, -0.447205,
9317 -0.951863, -0.447205, -0.970497, -0.406832, -1.138199, -0.403727,
9318 -1.147516, -0.009317, -0.967391, 0.009317, -0.967391, 0.052795,
9319 -1.287267, 0.083851, -1.336957, 0.500000, -1.374224, 0.086957,
9320 -1.399068, 0.068323, -1.700311, 0.006211, -1.526398, -0.009317,
9321 -1.585404, -0.338509, -1.631988, -0.400621, -1.650621, -0.500000,
9322 -1.650621, -0.500000, -1.504658, -0.360248, -1.417702, -0.024845,
9323 -1.194099, -0.478261, -0.951863, -0.447205, -1.138199, -0.403727,
9324 -1.147516, -0.009317, -0.967391, 0.052795, -1.262422, 0.068323,
9325 -1.262422, 0.068323, -1.287267, 0.083851, -1.374224, 0.086957,
9326 -1.389752, 0.071429, -1.399068, 0.068323, -1.526398, -0.009317,
9327 -1.545031, -0.257764, -1.585404, -0.338509, -1.650621, -0.500000,
9328 -1.194099, -0.478261, -1.138199, -0.403727, -1.147516, -0.400621,
9329 -1.166149, -0.021739, -1.147516, -0.009317, -1.262422, 0.068323,
9330 -1.262422, 0.068323, -1.374224, 0.086957, -1.389752, 0.071429,
9331 -1.389752, 0.071429, -1.526398, -0.009317, -1.513975, -0.015528,
9332 -1.545031, -0.257764, -1.650621, -0.500000, -1.417702, -0.024845,
9333 -1.194099, -0.478261, -1.147516, -0.400621, -1.169255, -0.378882,
9334 -1.169255, -0.378882, -1.166149, -0.021739, -1.262422, 0.068323,
9335 -1.262422, 0.068323, -1.389752, 0.071429, -1.513975, -0.015528,
9336 -1.523292, -0.192547, -1.545031, -0.257764, -1.417702, -0.024845,
9337 -1.253106, -0.031056, -1.194099, -0.478261, -1.169255, -0.378882,
9338 -1.262422, 0.068323, -1.513975, -0.015528, -1.504658, -0.027950,

```

9339     -1.510870, -0.133540, -1.523292, -0.192547, -1.417702, -0.024845,
9340     -1.253106, -0.031056, -1.169255, -0.378882, -1.262422, 0.068323,
9341     -1.504658, -0.027950, -1.510870, -0.133540, -1.417702, -0.024845,
9342     -1.268634, -0.012422, -1.253106, -0.031056, -1.262422, 0.068323,
9343     -1.504658, -0.027950, -1.417702, -0.024845, -1.395963, -0.009317,
9344     -1.277950, -0.009317, -1.268634, -0.012422, -1.262422, 0.068323,
9345     -1.262422, 0.068323, -1.504658, -0.027950, -1.395963, -0.009317,
9346     -1.395963, -0.009317, -1.277950, -0.009317, -1.262422, 0.068323,
9347 ];
9348 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuAboutButtonGlyph.ts -----
9349 import { Color, Graphics } from 'cc';
9350 import { unlitHudIconColor } from './OpticalPuzzleHudButtonCommon';
9351 import {
9352     drawMenuCircleButtonChrome,
9353     MENU_START_ICON_BORDER_PX,
9354     strokeMenuButtonIconGlow,
9355 } from './MenuStartButtonGlyph';
9356 import {
9357     MENU_ABOUT_BODY_SVG_SEGS,
9358     MENU_ABOUT_HEAD_SVG_SEGS,
9359 } from './MenuAboutIconSegs.generated';
9360 /** 菜单关于键 icon 占键帽边长比例 */
9361 export const HUD_MENU_ABOUT_ICON_SIZE_RATIO = 0.45;
9362 /** icon 视觉中心相对键帽中心向上微调（相对 icon 高度） */
9363 const HUD_MENU_ABOUT_ICON_OFFSET_Y_NORM = 0.04;
9364 const MENU_ICON_BORDER = new Color(255, 255, 255, 255);
9365 function iconHeight(buttonSize: number): number {
9366     return buttonSize * HUD_MENU_ABOUT_ICON_SIZE_RATIO;
9367 }
9368 function iconCenterY(cy: number, iconH: number): number {
9369     return cy + iconH * HUD_MENU_ABOUT_ICON_OFFSET_Y_NORM;
9370 }
9371 function traceSvgSegs(
9372     g: Graphics,
9373     segs: ReadonlyArray<Readonly<{ x: number; y: number }>>,
9374     cx: number,
9375     cy: number,
9376     iconH: number,
9377 ): void {
9378     for (let i = 0; i < segs.length; i++) {
9379         const p = segs[i];
9380         const x = cx + p.x * iconH;
9381         const y = cy + p.y * iconH;
9382         if (i === 0) {
9383             g.moveTo(x, y);
9384         } else {
9385             g.lineTo(x, y);
9386         }
9387     }
9388     g.close();
9389 }
9390 function traceAboutHead(g: Graphics, cx: number, cy: number, iconH: number): void {
9391     traceSvgSegs(g, MENU_ABOUT_HEAD_SVG_SEGS, cx, cy, iconH);
9392 }
9393 function traceAboutBody(g: Graphics, cx: number, cy: number, iconH: number): void {
9394     traceSvgSegs(g, MENU_ABOUT_BODY_SVG_SEGS, cx, cy, iconH);
9395 }
9396 function traceAboutIcon(g: Graphics, cx: number, cy: number, iconH: number): void {

```

```

9397     traceAboutHead(g, cx, cy, iconH);
9398     traceAboutBody(g, cx, cy, iconH);
9399 }
9400 function fillAboutIcon(
9401     g: Graphics,
9402     cx: number,
9403     cy: number,
9404     iconH: number,
9405     fillColor: Color,
9406 ): void {
9407     g.fillColor = fillColor;
9408     traceAboutHead(g, cx, cy, iconH);
9409     g.fill();
9410     traceAboutBody(g, cx, cy, iconH);
9411     g.fill();
9412 }
9413 function strokeAboutIconOutline(
9414     g: Graphics,
9415     cx: number,
9416     cy: number,
9417     iconH: number,
9418     lineWidth: number,
9419 ): void {
9420     g.strokeColor = MENU_ICON_BORDER;
9421     g.lineWidth = lineWidth;
9422     g.lineJoin = Graphics.LineJoin.ROUND;
9423     g.lineCap = Graphics.LineCap.ROUND;
9424     traceAboutIcon(g, cx, cy, iconH);
9425     g.stroke();
9426 }
9427 function drawMenuAboutIcon(
9428     g: Graphics,
9429     size: number,
9430     cx: number,
9431     cy: number,
9432     pressed: boolean,
9433 ): void {
9434     const iconH = iconHeight(size);
9435     const borderW = MENU_START_ICON_BORDER_PX;
9436     const iconCy = iconCenterY(cy, iconH);
9437     if (pressed) {
9438         fillAboutIcon(g, cx, iconCy, iconH, MENU_ICON_BORDER);
9439         strokeMenuButtonIconGlow(g, size, () => {
9440             traceAboutIcon(g, cx, iconCy, iconH);
9441         });
9442         return;
9443     }
9444     fillAboutIcon(g, cx, iconCy, iconH, unlitHudIconColor());
9445     strokeAboutIconOutline(g, cx, iconCy, iconH, borderW);
9446 }
9447 /** 圆形键帽 + 用户/about icon (SVG subpath 2+3, 120×120 等正方尺寸) */
9448 export function drawMenuAboutButtonGlyph(
9449     g: Graphics,
9450     left: number,
9451     bottom: number,
9452     width: number,
9453     height: number,
9454     pressed = false,

```

```

9455 ): void {
9456     const size = Math.min(width, height);
9457     const cx = left + width * 0.5;
9458     const cy = bottom + height * 0.5;
9459     drawMenuCircleButtonChrome(g, cx, cy, size, pressed);
9460     drawMenuAboutIcon(g, size, cx, cy, pressed);
9461 }
9462 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuAboutIconSegs.generated.ts -----
9463 // AUTO-GENERATED by tools/gen-menu-about-segs.py — do not edit by hand
9464 /** About icon: SVG subpath 2 头内圆 + subpath 3 身体外轮廓; y 已翻转, 按组合 bbox 高度归一化 */
9465 export const MENU_ABOUT_HEAD_SVG_SEGS: ReadonlyArray<Readonly<{ x: number; y: number }>> = [
9466     { x: -0.00001382, y: 0.50000000 },
9467     { x: -0.01071325, y: 0.49972632 },
9468     { x: -0.02127301, y: 0.49891422 },
9469     { x: -0.03167970, y: 0.49757710 },
9470     { x: -0.04191990, y: 0.49572836 },
9471     { x: -0.05198021, y: 0.49338141 },
9472     { x: -0.06184724, y: 0.49054965 },
9473     { x: -0.07150757, y: 0.48724649 },
9474     { x: -0.08094781, y: 0.48348532 },
9475     { x: -0.09015456, y: 0.47927955 },
9476     { x: -0.09911440, y: 0.47464258 },
9477     { x: -0.10781393, y: 0.46958783 },
9478     { x: -0.11623976, y: 0.46412868 },
9479     { x: -0.12437847, y: 0.45827854 },
9480     { x: -0.13221668, y: 0.45205082 },
9481     { x: -0.13974096, y: 0.44545893 },
9482     { x: -0.14693792, y: 0.43851625 },
9483     { x: -0.15379416, y: 0.43123620 },
9484     { x: -0.16029627, y: 0.42363218 },
9485     { x: -0.16643085, y: 0.41571760 },
9486     { x: -0.17218449, y: 0.40750585 },
9487     { x: -0.17754380, y: 0.39901034 },
9488     { x: -0.18249537, y: 0.39024447 },
9489     { x: -0.18702579, y: 0.38122165 },
9490     { x: -0.19112166, y: 0.37195527 },
9491     { x: -0.19476959, y: 0.36245875 },
9492     { x: -0.19795616, y: 0.35274549 },
9493     { x: -0.20066797, y: 0.34282888 },
9494     { x: -0.20289163, y: 0.33272234 },
9495     { x: -0.20461371, y: 0.32243926 },
9496     { x: -0.20582084, y: 0.31199305 },
9497     { x: -0.20649959, y: 0.30139712 },
9498     { x: -0.20663664, y: 0.29067275 },
9499     { x: -0.20622710, y: 0.28000697 },
9500     { x: -0.20528248, y: 0.26948422 },
9501     { x: -0.20381620, y: 0.25911790 },
9502     { x: -0.20184166, y: 0.24892142 },
9503     { x: -0.19937225, y: 0.23890817 },
9504     { x: -0.19642138, y: 0.22909156 },
9505     { x: -0.19300246, y: 0.21948499 },
9506     { x: -0.18912888, y: 0.21010187 },
9507     { x: -0.18481406, y: 0.20095560 },
9508     { x: -0.18007139, y: 0.19205958 },
9509     { x: -0.17491428, y: 0.18342721 },
9510     { x: -0.16935612, y: 0.17507190 },
9511     { x: -0.16341034, y: 0.16700706 },
9512     { x: -0.15709032, y: 0.15924608 },

```

9513 { x: -0.15040947, y: 0.15180237 },
9514 { x: -0.14338119, y: 0.14468934 },
9515 { x: -0.13601889, y: 0.13792037 },
9516 { x: -0.12833598, y: 0.13150889 },
9517 { x: -0.12034584, y: 0.12546829 },
9518 { x: -0.11206190, y: 0.11981197 },
9519 { x: -0.10349754, y: 0.11455334 },
9520 { x: -0.09466618, y: 0.10970581 },
9521 { x: -0.08558121, y: 0.10528276 },
9522 { x: -0.07625604, y: 0.10129762 },
9523 { x: -0.06670408, y: 0.09776378 },
9524 { x: -0.05693872, y: 0.09469464 },
9525 { x: -0.04697338, y: 0.09210361 },
9526 { x: -0.03682144, y: 0.09000409 },
9527 { x: -0.02649633, y: 0.08840949 },
9528 { x: -0.01601143, y: 0.08733320 },
9529 { x: -0.00538016, y: 0.08678863 },
9530 { x: 0.00535252, y: 0.08678863 },
9531 { x: 0.01598380, y: 0.08733320 },
9532 { x: 0.02646870, y: 0.08840949 },
9533 { x: 0.03679381, y: 0.09000409 },
9534 { x: 0.04694575, y: 0.09210361 },
9535 { x: 0.05691109, y: 0.09469464 },
9536 { x: 0.06667645, y: 0.09776378 },
9537 { x: 0.07622841, y: 0.10129762 },
9538 { x: 0.08555358, y: 0.10528276 },
9539 { x: 0.09463855, y: 0.10970581 },
9540 { x: 0.10346991, y: 0.11455334 },
9541 { x: 0.11203427, y: 0.11981197 },
9542 { x: 0.12031821, y: 0.12546829 },
9543 { x: 0.12830835, y: 0.13150889 },
9544 { x: 0.13599126, y: 0.13792037 },
9545 { x: 0.14335356, y: 0.14468934 },
9546 { x: 0.15038184, y: 0.15180237 },
9547 { x: 0.15706269, y: 0.15924608 },
9548 { x: 0.16338271, y: 0.16700706 },
9549 { x: 0.16932849, y: 0.17507190 },
9550 { x: 0.17488664, y: 0.18342721 },
9551 { x: 0.18004376, y: 0.19205958 },
9552 { x: 0.18478643, y: 0.20095560 },
9553 { x: 0.18910125, y: 0.21010187 },
9554 { x: 0.19297483, y: 0.21948499 },
9555 { x: 0.19639375, y: 0.22909156 },
9556 { x: 0.19934462, y: 0.23890817 },
9557 { x: 0.20181403, y: 0.24892142 },
9558 { x: 0.20378857, y: 0.25911790 },
9559 { x: 0.20525485, y: 0.26948422 },
9560 { x: 0.20619947, y: 0.28000697 },
9561 { x: 0.20660901, y: 0.29067275 },
9562 { x: 0.20647196, y: 0.30139712 },
9563 { x: 0.20579321, y: 0.31199305 },
9564 { x: 0.20458608, y: 0.32243926 },
9565 { x: 0.20286399, y: 0.33272234 },
9566 { x: 0.20064034, y: 0.34282888 },
9567 { x: 0.19792853, y: 0.35274549 },
9568 { x: 0.19474196, y: 0.36245875 },
9569 { x: 0.19109403, y: 0.37195527 },
9570 { x: 0.18699816, y: 0.38122165 },

```

9571      { x: 0.18246773, y: 0.39024447 },
9572      { x: 0.17751617, y: 0.39901034 },
9573      { x: 0.17215686, y: 0.40750585 },
9574      { x: 0.16640322, y: 0.41571760 },
9575      { x: 0.16026864, y: 0.42363218 },
9576      { x: 0.15376653, y: 0.43123620 },
9577      { x: 0.14691029, y: 0.43851625 },
9578      { x: 0.13971333, y: 0.44545893 },
9579      { x: 0.13218904, y: 0.45205082 },
9580      { x: 0.12435084, y: 0.45827854 },
9581      { x: 0.11621213, y: 0.46412868 },
9582      { x: 0.10778630, y: 0.46958783 },
9583      { x: 0.09908676, y: 0.47464258 },
9584      { x: 0.09012692, y: 0.47927955 },
9585      { x: 0.08092018, y: 0.48348532 },
9586      { x: 0.07147994, y: 0.48724649 },
9587      { x: 0.06181961, y: 0.49054965 },
9588      { x: 0.05195258, y: 0.49338141 },
9589      { x: 0.04189227, y: 0.49572836 },
9590      { x: 0.03165207, y: 0.49757710 },
9591      { x: 0.02124538, y: 0.49891422 },
9592      { x: 0.01068562, y: 0.49972632 },
9593 ];
9594 export const MENU_ABOUT_BODY_SVG_SEGS: ReadonlyArray<Readonly<{ x: number; y: number }>> = [
9595     { x: 0.38282129, y: -0.50000000 },
9596     { x: 0.36274905, y: -0.50000000 },
9597     { x: 0.34267681, y: -0.50000000 },
9598     { x: 0.32260458, y: -0.50000000 },
9599     { x: 0.30253234, y: -0.50000000 },
9600     { x: 0.28246011, y: -0.50000000 },
9601     { x: 0.26238787, y: -0.50000000 },
9602     { x: 0.24231563, y: -0.50000000 },
9603     { x: 0.22224340, y: -0.50000000 },
9604     { x: 0.20217116, y: -0.50000000 },
9605     { x: 0.18209893, y: -0.50000000 },
9606     { x: 0.16202669, y: -0.50000000 },
9607     { x: 0.14195445, y: -0.50000000 },
9608     { x: 0.12188222, y: -0.50000000 },
9609     { x: 0.10180998, y: -0.50000000 },
9610     { x: 0.08173775, y: -0.50000000 },
9611     { x: 0.06166551, y: -0.50000000 },
9612     { x: 0.04159327, y: -0.50000000 },
9613     { x: 0.02152104, y: -0.50000000 },
9614     { x: 0.00144880, y: -0.50000000 },
9615     { x: -0.01862343, y: -0.50000000 },
9616     { x: -0.03869567, y: -0.50000000 },
9617     { x: -0.05876791, y: -0.50000000 },
9618     { x: -0.07884014, y: -0.50000000 },
9619     { x: -0.09891238, y: -0.50000000 },
9620     { x: -0.11898461, y: -0.50000000 },
9621     { x: -0.13905685, y: -0.50000000 },
9622     { x: -0.15912909, y: -0.50000000 },
9623     { x: -0.17920132, y: -0.50000000 },
9624     { x: -0.19927356, y: -0.50000000 },
9625     { x: -0.21934579, y: -0.50000000 },
9626     { x: -0.23941803, y: -0.50000000 },
9627     { x: -0.25949027, y: -0.50000000 },
9628     { x: -0.27956250, y: -0.50000000 },

```

9629 { x: -0.29963474, y: -0.50000000 },
9630 { x: -0.31970697, y: -0.50000000 },
9631 { x: -0.33977921, y: -0.50000000 },
9632 { x: -0.35985145, y: -0.50000000 },
9633 { x: -0.37992368, y: -0.50000000 },
9634 { x: -0.39993277, y: -0.49872214 },
9635 { x: -0.41943323, y: -0.49407087 },
9636 { x: -0.43784967, y: -0.48615023 },
9637 { x: -0.45463847, y: -0.47519402 },
9638 { x: -0.46930402, y: -0.46152567 },
9639 { x: -0.48141340, y: -0.44554866 },
9640 { x: -0.49060916, y: -0.42773462 },
9641 { x: -0.49661984, y: -0.40860940 },
9642 { x: -0.49926801, y: -0.38873758 },
9643 { x: -0.49906996, y: -0.36783265 },
9644 { x: -0.49761010, y: -0.34690446 },
9645 { x: -0.49500504, y: -0.32632430 },
9646 { x: -0.49128730, y: -0.30612408 },
9647 { x: -0.48648937, y: -0.28633566 },
9648 { x: -0.48064373, y: -0.26699095 },
9649 { x: -0.47378288, y: -0.24812183 },
9650 { x: -0.46593932, y: -0.22976020 },
9651 { x: -0.45714554, y: -0.21193793 },
9652 { x: -0.44743403, y: -0.19468691 },
9653 { x: -0.43683730, y: -0.17803905 },
9654 { x: -0.42538783, y: -0.16202621 },
9655 { x: -0.41311811, y: -0.14668030 },
9656 { x: -0.40006065, y: -0.13203320 },
9657 { x: -0.38624793, y: -0.11811680 },
9658 { x: -0.37171245, y: -0.10496298 },
9659 { x: -0.35648671, y: -0.09260365 },
9660 { x: -0.34060319, y: -0.08107067 },
9661 { x: -0.32409440, y: -0.07039595 },
9662 { x: -0.30699282, y: -0.06061137 },
9663 { x: -0.28933096, y: -0.05174882 },
9664 { x: -0.27114130, y: -0.04384019 },
9665 { x: -0.25245633, y: -0.03691737 },
9666 { x: -0.23330856, y: -0.03101224 },
9667 { x: -0.21373048, y: -0.02615669 },
9668 { x: -0.19375458, y: -0.02238262 },
9669 { x: -0.17341336, y: -0.01972191 },
9670 { x: -0.15273930, y: -0.01820645 },
9671 { x: -0.13198395, y: -0.01783992 },
9672 { x: -0.11191172, y: -0.01783992 },
9673 { x: -0.09183948, y: -0.01783992 },
9674 { x: -0.07176725, y: -0.01783992 },
9675 { x: -0.05169501, y: -0.01783992 },
9676 { x: -0.03162277, y: -0.01783992 },
9677 { x: -0.01155054, y: -0.01783992 },
9678 { x: 0.00852170, y: -0.01783992 },
9679 { x: 0.02859393, y: -0.01783992 },
9680 { x: 0.04866617, y: -0.01783992 },
9681 { x: 0.06873841, y: -0.01783992 },
9682 { x: 0.08881064, y: -0.01783992 },
9683 { x: 0.10888288, y: -0.01783992 },
9684 { x: 0.12895511, y: -0.01783992 },
9685 { x: 0.14902444, y: -0.01807956 },
9686 { x: 0.16905312, y: -0.01936196 },

```

9687     { x: 0.18898050, y: -0.02174766 },
9688     { x: 0.20874587, y: -0.02522940 },
9689     { x: 0.22828899, y: -0.02979656 },
9690     { x: 0.24755029, y: -0.03543523 },
9691     { x: 0.26647108, y: -0.04212822 },
9692     { x: 0.28499369, y: -0.04985514 },
9693     { x: 0.30306167, y: -0.05859243 },
9694     { x: 0.32061996, y: -0.06831347 },
9695     { x: 0.33761506, y: -0.07898864 },
9696     { x: 0.35399516, y: -0.09058539 },
9697     { x: 0.36971034, y: -0.10306839 },
9698     { x: 0.38471272, y: -0.11639959 },
9699     { x: 0.39895657, y: -0.13053837 },
9700     { x: 0.41239848, y: -0.14544164 },
9701     { x: 0.42499749, y: -0.16106397 },
9702     { x: 0.43671520, y: -0.17735776 },
9703     { x: 0.44751590, y: -0.19427335 },
9704     { x: 0.45736667, y: -0.21175919 },
9705     { x: 0.46623749, y: -0.22976199 },
9706     { x: 0.47410134, y: -0.24822689 },
9707     { x: 0.48093423, y: -0.26709760 },
9708     { x: 0.48671535, y: -0.28631663 },
9709     { x: 0.49142708, y: -0.30582539 },
9710     { x: 0.49505507, y: -0.32556444 },
9711     { x: 0.49758824, y: -0.34547362 },
9712     { x: 0.49901889, y: -0.36549225 },
9713     { x: 0.49926801, y: -0.38591143 },
9714     { x: 0.49666964, y: -0.40678556 },
9715     { x: 0.49076923, y: -0.42628193 },
9716     { x: 0.48187407, y: -0.44411269 },
9717     { x: 0.47029144, y: -0.45999000 },
9718     { x: 0.45632864, y: -0.47362600 },
9719     { x: 0.44029294, y: -0.48473285 },
9720     { x: 0.42249162, y: -0.49302270 },
9721     { x: 0.40323198, y: -0.49820770 },
9722 ];
9723 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuGameSubtitleGlyph.ts -----
9724 import { Color, Graphics } from 'cc';
9725 import {
9726     GAME_SUBTITLE_FILL_TRI_FLAT,
9727     GAME_SUBTITLE_NORM_H,
9728     GAME_SUBTITLE_NORM_W,
9729     GAME_SUBTITLE_OUTLINE_PATHS,
9730 } from './GameSubtitlePathSegs.generated';
9731 /** 副标题外轮廓线宽（设计 px，不随节点缩放） */
9732 export const GAME_SUBTITLE_STROKE_PX = 1;
9733 const SUBTITLE_FILL = new Color(255, 255, 255, 255);
9734 const SUBTITLE_STROKE = new Color(255, 255, 255, 255);
9735 /** 外轮廓 + 内孔边缘发光（整体收窄，减轻叠光发糊） */
9736 const SUBTITLE_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
9737     { width: 8, alpha: 20 },
9738     { width: 5, alpha: 40 },
9739     { width: 3, alpha: 60 },
9740 ];
9741 function traceOutlinePath(
9742     g: Graphics,
9743     segs: ReadonlyArray<Readonly<{ x: number; y: number }>>,
9744     cx: number,

```



```

9745     cy: number,
9746     scale: number,
9747 ): void {
9748     for (let i = 0; i < segs.length; i++) {
9749         const p = segs[i];
9750         const x = cx + p.x * scale;
9751         const y = cy + p.y * scale;
9752         if (i === 0) {
9753             g.moveTo(x, y);
9754         } else {
9755             g.lineTo(x, y);
9756         }
9757     }
9758     g.close();
9759 }
9760 function strokePaths(
9761     g: Graphics,
9762     paths: ReadonlyArray<ReadonlyArray<Readonly<{ x: number; y: number }>>>,
9763     cx: number,
9764     cy: number,
9765     drawScale: number,
9766     lineWidth: number,
9767     color: Color,
9768     roundJoin: boolean,
9769 ): void {
9770     g.strokeColor = color;
9771     g.lineWidth = lineWidth;
9772     g.lineJoin = roundJoin ? Graphics.LineJoin.ROUND : Graphics.LineJoin.MITER;
9773     g.lineCap = roundJoin ? Graphics.LineCap.ROUND : Graphics.LineCap.BUTT;
9774     if (!roundJoin) {
9775         g.miterLimit = 4;
9776     }
9777     for (const path of paths) {
9778         if (path.length < 2) {
9779             continue;
9780         }
9781         traceOutlinePath(g, path, cx, cy, drawScale);
9782         g.stroke();
9783     }
9784 }
9785 function strokeSubtitleGlow(g: Graphics, cx: number, cy: number, drawScale: number): void {
9786     for (const layer of SUBTITLE_GLOW_LAYERS) {
9787         strokePaths(
9788             g,
9789             GAME_SUBTITLE_OUTLINE_PATHS,
9790             cx,
9791             cy,
9792             drawScale,
9793             layer.width,
9794             new Color(255, 255, 255, layer.alpha),
9795             true,
9796         );
9797     }
9798 }
9799 function fillTriangulatedSubtitle(
9800     g: Graphics,
9801     cx: number,
9802     cy: number,

```

```

9803         drawScale: number,
9804     ): void {
9805         const flat = GAME_SUBTITLE_FILL_TRI_FLAT;
9806         g.fillColor = SUBTITLE_FILL;
9807         for (let i = 0; i < flat.length; i += 6) {
9808             g.moveTo(cx + flat[i] * drawScale, cy + flat[i + 1] * drawScale);
9809             g.lineTo(cx + flat[i + 2] * drawScale, cy + flat[i + 3] * drawScale);
9810             g.lineTo(cx + flat[i + 4] * drawScale, cy + flat[i + 5] * drawScale);
9811             g.close();
9812             g.fill();
9813         }
9814     }
9815     /** 纯白填充 + 外轮廓/内孔发光 + 1px 描边 */
9816     export function drawGameSubtitleOutline(
9817         g: Graphics,
9818         left: number,
9819         bottom: number,
9820         width: number,
9821         height: number,
9822     ): void {
9823         const cx = left + width * 0.5;
9824         const cy = bottom + height * 0.5;
9825         const drawScale = Math.min(width / GAME_SUBTITLE_NORM_W, height / GAME_SUBTITLE_NORM_H);
9826         strokeSubtitleGlow(g, cx, cy, drawScale);
9827         fillTriangulatedSubtitle(g, cx, cy, drawScale);
9828         strokePaths(
9829             g,
9830             GAME_SUBTITLE_OUTLINE_PATHS,
9831             cx,
9832             cy,
9833             drawScale,
9834             GAME_SUBTITLE_STROKE_PX,
9835             SUBTITLE_STROKE,
9836             true,
9837         );
9838     }
9839     // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuGameTitleGlyph.ts -----
9840     import { Color, Graphics } from 'cc';
9841     import {
9842         GAME_TITLE_FILL_TRI_FLAT,
9843         GAME_TITLE_NORM_H,
9844         GAME_TITLE_NORM_W,
9845         GAME_TITLE_OUTLINE_PATHS,
9846     } from './GameTitlePathSegs.generated';
9847     /** 标题外轮廓线宽（设计 px，不随节点缩放） */
9848     export const GAME_TITLE_STROKE_PX = 1;
9849     const TITLE_FILL = new Color(255, 255, 255, 255);
9850     const TITLE_STROKE = new Color(255, 255, 255, 255);
9851     /** 外轮廓 + 内孔边缘发光（整体收窄，减轻叠光发糊） */
9852     const TITLE_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
9853         { width: 8, alpha: 20 },
9854         { width: 5, alpha: 40 },
9855         { width: 3, alpha: 60 },
9856     ];
9857     function traceOutlinePath(
9858         g: Graphics,
9859         segs: ReadonlyArray<Readonly<{ x: number; y: number }>>,
9860         cx: number,

```

```
9861     cy: number,
9862     scale: number,
9863 ): void {
9864     for (let i = 0; i < segs.length; i++) {
9865         const p = segs[i];
9866         const x = cx + p.x * scale;
9867         const y = cy + p.y * scale;
9868         if (i === 0) {
9869             g.moveTo(x, y);
9870         } else {
9871             g.lineTo(x, y);
9872         }
9873     }
9874     g.close();
9875 }
9876 function strokePaths(
9877     g: Graphics,
9878     paths: ReadonlyArray<ReadonlyArray<Readonly<{ x: number; y: number }>>>,
9879     cx: number,
9880     cy: number,
9881     drawScale: number,
9882     lineWidth: number,
9883     color: Color,
9884     roundJoin: boolean,
9885 ): void {
9886     g.strokeColor = color;
9887     g.lineWidth = lineWidth;
9888     g.lineJoin = roundJoin ? Graphics.LineJoin.ROUND : Graphics.LineJoin.MITER;
9889     g.lineCap = roundJoin ? Graphics.LineCap.ROUND : Graphics.LineCap.BUTT;
9890     if (!roundJoin) {
9891         g.miterLimit = 4;
9892     }
9893     for (const path of paths) {
9894         if (path.length < 2) {
9895             continue;
9896         }
9897         traceOutlinePath(g, path, cx, cy, drawScale);
9898         g.stroke();
9899     }
9900 }
9901 function strokeTitleGlow(g: Graphics, cx: number, cy: number, drawScale: number): void {
9902     for (const layer of TITLE_GLOW_LAYERS) {
9903         strokePaths(
9904             g,
9905             GAME_TITLE_OUTLINE_PATHS,
9906             cx,
9907             cy,
9908             drawScale,
9909             layer.width,
9910             new Color(255, 255, 255, layer.alpha),
9911             true,
9912         );
9913     }
9914 }
9915 function fillTriangulatedTitle(
9916     g: Graphics,
9917     cx: number,
9918     cy: number,
```

```

9919         drawScale: number,
9920     ): void {
9921         const flat = GAME_TITLE_FILL_TRI_FLAT;
9922         g.fillColor = TITLE_FILL;
9923         for (let i = 0; i < flat.length; i += 6) {
9924             g.moveTo(cx + flat[i] * drawScale, cy + flat[i + 1] * drawScale);
9925             g.lineTo(cx + flat[i + 2] * drawScale, cy + flat[i + 3] * drawScale);
9926             g.lineTo(cx + flat[i + 4] * drawScale, cy + flat[i + 5] * drawScale);
9927             g.close();
9928             g.fill();
9929         }
9930     }
9931     /** 纯白填充 + 外轮廓/内孔发光 + 1px 描边 */
9932     export function drawGameTitleOutline(
9933         g: Graphics,
9934         left: number,
9935         bottom: number,
9936         width: number,
9937         height: number,
9938     ): void {
9939         const cx = left + width * 0.5;
9940         const cy = bottom + height * 0.5;
9941         const drawScale = Math.min(width / GAME_TITLE_NORM_W, height / GAME_TITLE_NORM_H);
9942         strokeTitleGlow(g, cx, cy, drawScale);
9943         fillTriangulatedTitle(g, cx, cy, drawScale);
9944         strokePaths(
9945             g,
9946             GAME_TITLE_OUTLINE_PATHS,
9947             cx,
9948             cy,
9949             drawScale,
9950             GAME_TITLE_STROKE_PX,
9951             TITLE_STROKE,
9952             true,
9953         );
9954     }
9955     // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuGameTitleIcon2Glyph.ts -----
9956     import { Graphics, Node, ParticleSystem2D, UITransform, Vec2, Vec3 } from 'cc';
9957     import { Direction } from '../Core/OpticalPuzzleTypes';
9958     import { configureBeamSparkParticle } from './OpticalPuzzleBeamImpactView';
9959     import { strokeBeamSegmentGradient } from './OpticalPuzzleBeamGradient';
9960     import { beamColorFromKey } from './OpticalPuzzleColorUtil';
9961     import { drawSourceEmitter, sourceMuzzleScreenPoint } from './OpticalPuzzleSourceGlyph';
9962     /** 菜单标题区白色光源向右发射的光束最大长度（设计 px） */
9963     export const MENU_TITLE_SOURCE_BEAM_LENGTH_PX = 1000;
9964     /** 光束右端相对屏幕边界内缩（设计 px） */
9965     const MENU_TITLE_BEAM_EDGE_INSET_PX = 4;
9966     /** Menu 屏右缘火花：比局内阻挡点更大 */
9967     export function configureMenuTitleBeamSparkParticle(ps: ParticleSystem2D): void {
9968         configureBeamSparkParticle(ps, Direction.Right, 'white');
9969         ps.startSize = 4;
9970         ps.startSizeVar = 2;
9971         ps.endSize = 2;
9972         ps.endSizeVar = 1;
9973         ps.emissionRate = 85;
9974         ps.totalParticles = 120;
9975         ps.posVar = new Vec2(2, 10);
9976         ps.speed = 65;

```

```

9977     ps.speedVar = 15;
9978 }
9979 export interface MenuTitleRightBeamLayout {
9980     readonly muzzleX: number;
9981     readonly muzzleY: number;
9982     readonly endX: number;
9983     readonly endY: number;
9984     /** 是否在最大长度前碰到右边界（用于显示火花） */
9985     readonly hitsScreenEdge: boolean;
9986 }
9987 /** 在 icon 节点本地坐标系下计算向右光束终点（碰 Canvas 右缘则截断） */
9988 export function computeMenuTitleRightBeamLayout(
9989     iconUt: UITransform,
9990     left: number,
9991     bottom: number,
9992     width: number,
9993     height: number,
9994     boundsNode: Node | null,
9995     maxLength = MENU_TITLE_SOURCE_BEAM_LENGTH_PX,
10000 ): MenuTitleRightBeamLayout {
10001     const size = Math.min(width, height);
10002     const cellLeft = left + (width - size) * 0.5;
10003     const cellBottom = bottom + (height - size) * 0.5;
10004     const muzzle = sourceMuzzleScreenPoint(cellLeft, cellBottom, size, Direction.Right);
10005     const maxEndX = muzzle.x + maxLength;
10006     let endX = maxEndX;
10007     let hitsScreenEdge = false;
10008     const boundsUt = boundsNode?.isValid ? boundsNode.getComponent(UITransform) : null;
10009     if (boundsUt) {
10010         const muzzleWorld = new Vec3();
10011         iconUt.convertToWorldSpaceAR(new Vec3(muzzle.x, muzzle.y, 0), muzzleWorld);
10012         const muzzleInBounds = new Vec3();
10013         boundsUt.convertToNodeSpaceAR(muzzleWorld, muzzleInBounds);
10014         const rightLocalX =
10015             boundsUt.width * (1 - boundsUt.anchorX) - MENU_TITLE_BEAM_EDGE_INSET_PX;
10016         const rightInBounds = new Vec3(rightLocalX, muzzleInBounds.y, 0);
10017         const rightWorld = new Vec3();
10018         boundsUt.convertToWorldSpaceAR(rightInBounds, rightWorld);
10019         const rightInIcon = new Vec3();
10020         iconUt.convertToNodeSpaceAR(rightWorld, rightInIcon);
10021         if (rightInIcon.x > muzzle.x) {
10022             endX = Math.min(maxEndX, rightInIcon.x);
10023             hitsScreenEdge = endX < maxEndX;
10024         }
10025     }
10026     return {
10027         muzzleX: muzzle.x,
10028         muzzleY: muzzle.y,
10029         endX,
10030         endY: muzzle.y,
10031         hitsScreenEdge,
10032     };
10033 }
10034 /** 白色光源发射器朝右 + 光束（终点由 layout 截断至屏幕边缘） */
10035 export function drawMenuTitleWhiteSourceRightBeam(
10036     g: Graphics,
10037     left: number,
10038     bottom: number,

```

```

10035     width: number,
10036     height: number,
10037     layout: MenuTitleRightBeamLayout,
10038 ): void {
10039     const size = Math.min(width, height);
10040     const cellLeft = left + (width - size) * 0.5;
10041     const cellBottom = bottom + (height - size) * 0.5;
10042     const direction = Direction.Right;
10043     const beamColor = beamColorFromKey('white');
10044     strokeBeamSegmentGradient(
10045         g,
10046         layout.muzzleX,
10047         layout.muzzleY,
10048         layout.endX,
10049         layout.endY,
10050         beamColor,
10051     );
10052     drawSourceEmitter(g, cellLeft, cellBottom, size, 'white', direction);
10053 }
10054 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuLevelSelectButtonGlyph.ts -----
10055 import { Graphics } from 'cc';
10056 import { traceLevelSelectGridIconForButton } from './OpticalPuzzleLevelSelectButtonGlyph';
10057 import { drawMenuButtonIcon, drawMenuCircleButtonChrome } from './MenuStartButtonGlyph';
10058 /** 菜单选关键四宫格 icon 占键帽边长比例（与开始键箭头 0.42 对齐） */
10059 export const HUD_MENU_LEVEL_SELECT_ICON_SIZE_RATIO = 0.42;
10060 /** 菜单选关键：圆形键帽 + Game 同款四宫格 icon（120×120 等正方尺寸） */
10061 export function drawMenuLevelSelectButtonGlyph(
10062     g: Graphics,
10063     left: number,
10064     bottom: number,
10065     width: number,
10066     height: number,
10067     pressed = false,
10068 ): void {
10069     const size = Math.min(width, height);
10070     const cx = left + width * 0.5;
10071     const cy = bottom + height * 0.5;
10072     drawMenuCircleButtonChrome(g, cx, cy, size, pressed);
10073     drawMenuButtonIcon(g, size, pressed, () => {
10074         traceLevelSelectGridIconForButton(g, cx, cy, size, HUD_MENU_LEVEL_SELECT_ICON_SIZE_RATIO);
10075     });
10076 }
10077 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuShareButtonGlyph.ts -----
10078 import { Color, Graphics } from 'cc';
10079 import { unlitHudIconColor } from './OpticalPuzzleHudButtonCommon';
10080 import {
10081     drawMenuCircleButtonChrome,
10082     MENU_START_ICON_BORDER_PX,
10083     strokeMenuButtonIconGlow,
10084 } from './MenuStartButtonGlyph';
10085 /** 菜单分享键 icon 占键帽边长比例 */
10086 export const HUD_MENU_SHARE_ICON_SIZE_RATIO = 0.45;
10087 /** 三圆心（SVG viewBox 1024 归一化，y 已翻转） */
10088 const SHARE_NODES: ReadonlyArray<Readonly<{ x: number; y: number }>> = [
10089     { x: 0.18228432, y: 0.39084138 },
10090     { x: -0.29232185, y: -0.05988111 },
10091     { x: 0.29232185, y: -0.39084138 },
10092 ];

```

```

10093  /** 圆半径 / icon 高度（相对 SVG 略放大，菜单 120 键更易辨认） */
10094  const SHARE_CIRCLE_R_NORM = 0.15;
10095  /** 连线：右上 → 左 → 右下 */
10096  const SHARE_LINE_PAIRS: ReadonlyArray<readonly [number, number]> = [
10097      [0, 1],
10098      [1, 2],
10099  ];
10100  const MENU_ICON_BORDER = new Color(255, 255, 255, 255);
10101  function shareIconHeight(buttonSize: number): number {
10102      return buttonSize * HUD_MENU_SHARE_ICON_SIZE_RATIO;
10103  }
10104  function traceShareCircles(g: Graphics, cx: number, cy: number, iconH: number): void {
10105      const r = iconH * SHARE_CIRCLE_R_NORM;
10106      for (const p of SHARE_NODES) {
10107          g.circle(cx + p.x * iconH, cy + p.y * iconH, r);
10108      }
10109  }
10110  function traceShareLines(g: Graphics, cx: number, cy: number, iconH: number): void {
10111      const r = iconH * SHARE_CIRCLE_R_NORM;
10112      for (let i = 0; i < SHARE_LINE_PAIRS.length; i++) {
10113          const [a, b] = SHARE_LINE_PAIRS[i];
10114          const na = SHARE_NODES[a];
10115          const nb = SHARE_NODES[b];
10116          const ax = cx + na.x * iconH;
10117          const ay = cy + na.y * iconH;
10118          const bx = cx + nb.x * iconH;
10119          const by = cy + nb.y * iconH;
10120          const dx = bx - ax;
10121          const dy = by - ay;
10122          const len = Math.hypot(dx, dy);
10123          if (len < 1e-6) {
10124              continue;
10125          }
10126          const ux = dx / len;
10127          const uy = dy / len;
10128          const x0 = ax + ux * r;
10129          const y0 = ay + uy * r;
10130          const x1 = bx - ux * r;
10131          const y1 = by - uy * r;
10132          if (i === 0) {
10133              g.moveTo(x0, y0);
10134          } else {
10135              g.moveTo(x0, y0);
10136          }
10137          g.lineTo(x1, y1);
10138      }
10139  }
10140  function traceShareIconStroke(g: Graphics, cx: number, cy: number, iconH: number): void {
10141      traceShareLines(g, cx, cy, iconH);
10142      traceShareCircles(g, cx, cy, iconH);
10143  }
10144  function drawMenuShareIcon(
10145      g: Graphics,
10146      size: number,
10147      cx: number,
10148      cy: number,
10149      pressed: boolean,
10150  ): void {

```

```

10151     const iconH = shareIconHeight(size);
10152     const borderW = MENU_START_ICON_BORDER_PX;
10153     if (pressed) {
10154         g.fillColor = MENU_ICON_BORDER;
10155         traceShareCircles(g, cx, cy, iconH);
10156         g.fill();
10157         strokeMenuButtonIconGlow(g, size, () => {
10158             traceShareIconStroke(g, cx, cy, iconH);
10159         });
10160         return;
10161     }
10162     g.fillColor = unlitHudIconColor();
10163     traceShareCircles(g, cx, cy, iconH);
10164     g.fill();
10165     g.strokeColor = MENU_ICON_BORDER;
10166     g.lineWidth = borderW;
10167     g.lineJoin = Graphics.LineJoin.ROUND;
10168     g.lineCap = Graphics.LineCap.ROUND;
10169     traceShareIconStroke(g, cx, cy, iconH);
10170     g.stroke();
10171 }
10172 /** 圆形键帽 + 三圆分享 icon (120×120 等正方尺寸) */
10173 export function drawMenuShareButtonGlyph(
10174     g: Graphics,
10175     left: number,
10176     bottom: number,
10177     width: number,
10178     height: number,
10179     pressed = false,
10180 ): void {
10181     const size = Math.min(width, height);
10182     const cx = left + width * 0.5;
10183     const cy = bottom + height * 0.5;
10184     drawMenuCircleButtonChrome(g, cx, cy, size, pressed);
10185     drawMenuShareIcon(g, size, cx, cy, pressed);
10186 }
10187 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/MenuStartButtonGlyph.ts -----
10188 import { Color, Graphics } from 'cc';
10189 import { Direction } from '../Core/OpticalPuzzleTypes';
10190 import {
10191     drawHudSvgArrowIcon,
10192     HUD_MENU_START_ARROW_SIZE_RATIO,
10193     traceHudSvgDirectionArrow,
10194 } from './OpticalPuzzleHudArrowGlyph';
10195 import { targetDimFillColor } from './OpticalPuzzleColorUtil';
10196 /** 菜单开始键圆形外框线宽 (设计 px, 不随节点缩放) */
10197 export const MENU_START_BORDER_PX = 3;
10198 /** 中间右箭头 icon 外描边 (设计 px, 不随节点缩放) */
10199 export const MENU_START_ICON_BORDER_PX = 4;
10200 const KEY_FILL = new Color(22, 28, 38, 255);
10201 const MENU_START_BORDER = new Color(255, 255, 255, 255);
10202 const PRESSED_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
10203     { width: 9, alpha: 48 },
10204     { width: 6, alpha: 120 },
10205     { width: 3, alpha: 255 },
10206 ];
10207 const HUD_BUTTON_DESIGN_SIZE = 80;
10208 function scaleHudDesign(size: number, design: number): number {

```



```

10209         return design * (size / HUD_BUTTON_DESIGN_SIZE);
10210     }
10211     function menuStartIconFillColor(): Color {
10212         const dim = targetDimFillColor(undefined);
10213         return new Color(
10214             Math.floor(dim.r * 0.76),
10215             Math.floor(dim.g * 0.76),
10216             Math.floor(dim.b * 0.76),
10217             255,
10218         );
10219     }
10220     /** 菜单圆形键内 icon: 与开始键箭头同描边 (4px 固定、MITER/BUTT) */
10221     export function drawMenuButtonIcon(
10222         g: Graphics,
10223         size: number,
10224         pressed: boolean,
10225         traceIcon: () => void,
10226     ): void {
10227         if (pressed) {
10228             g.fillColor = MENU_START_BORDER;
10229             traceIcon();
10230             g.fill();
10231             strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, traceIcon);
10232             return;
10233         }
10234         g.fillColor = menuStartIconFillColor();
10235         traceIcon();
10236         g.fill();
10237         g.strokeColor = MENU_START_BORDER;
10238         g.lineWidth = MENU_START_ICON_BORDER_PX;
10239         g.lineJoin = Graphics.LineJoin.MITER;
10240         g.lineCap = Graphics.LineCap.BUTT;
10241         traceIcon();
10242         g.stroke();
10243     }
10244     function strokeGlowLayers(
10245         g: Graphics,
10246         size: number,
10247         layers: ReadonlyArray<{ width: number; alpha: number }>,
10248         tracePath: () => void,
10249     ): void {
10250         g.lineJoin = Graphics.LineJoin.ROUND;
10251         g.lineCap = Graphics.LineCap.ROUND;
10252         for (const layer of layers) {
10253             g.strokeColor = new Color(255, 255, 255, layer.alpha);
10254             g.lineWidth = Math.max(1, scaleHudDesign(size, layer.width));
10255             tracePath();
10256             g.stroke();
10257         }
10258     }
10259     /** 菜单 icon 按下光晕 (分享键等多段路径复用) */
10260     export function strokeMenuButtonIconGlow(
10261         g: Graphics,
10262         size: number,
10263         tracePath: () => void,
10264     ): void {
10265         strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, tracePath);
10266     }

```

```

10267 /** 菜单圆形键帽底 + 白描边（开始键 / 选关键共用） */
10268 export function drawMenuCircleButtonChrome(
10269     g: Graphics,
10270     cx: number,
10271     cy: number,
10272     size: number,
10273     pressed: boolean,
10274 ): void {
10275     const radius = size * 0.5 - MENU_START_BORDER_PX * 0.5;
10276     const traceCircle = (): void => {
10277         g.circle(cx, cy, radius);
10278     };
10279     g.fillColor = KEY_FILL;
10280     traceCircle();
10281     g.fill();
10282     g.lineJoin = Graphics.LineJoin.ROUND;
10283     g.lineCap = Graphics.LineCap.ROUND;
10284     if (pressed) {
10285         strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, traceCircle);
10286     } else {
10287         g.strokeColor = MENU_START_BORDER;
10288         g.lineWidth = MENU_START_BORDER_PX;
10289         traceCircle();
10290         g.stroke();
10291     }
10292 }
10293 function drawMenuStartRightArrowIcon(
10294     g: Graphics,
10295     size: number,
10296     cx: number,
10297     cy: number,
10298     pressed: boolean,
10299 ): void {
10300     const traceArrow = (): void => {
10301         traceHudSvgDirectionArrow(g, cx, cy, size, Direction.Right, HUD_MENU_START_ARROW_SIZE_RATIO);
10302     };
10303     if (pressed) {
10304         drawMenuButtonIcon(g, size, true, traceArrow);
10305         return;
10306     }
10307     drawHudSvgArrowIcon(
10308         g,
10309         cx,
10310         cy,
10311         size,
10312         Direction.Right,
10313         menuStartIconFillColor(),
10314         MENU_START_BORDER,
10315         MENU_START_ICON_BORDER_PX,
10316         HUD_MENU_START_ARROW_SIZE_RATIO,
10317     );
10318 }
10319 /** 圆形键帽 + 右箭头 icon（120×120 等正方尺寸） */
10320 export function drawMenuStartButtonGlyph(
10321     g: Graphics,
10322     left: number,
10323     bottom: number,
10324     width: number,

```

```

10325         height: number,
10326         pressed = false,
10327     ): void {
10328         const size = Math.min(width, height);
10329         const cx = left + width * 0.5;
10330         const cy = bottom + height * 0.5;
10331         drawMenuCircleButtonChrome(g, cx, cy, size, pressed);
10332         drawMenuStartRightArrowIcon(g, size, cx, cy, pressed);
10333     }
10334 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleAboutMePanel.ts -----
10335 import {
10336     _decorator,
10337     BlockInputEvents,
10338     Button,
10339     Component,
10340     EventTouch,
10341     Graphics,
10342     Node,
10343     Sprite,
10344     UITransform,
10345 } from 'cc';
10346 import { AUDIO_EFFECT_ENUM, EVENT_ENUM } from '../Utils/Enum';
10347 import { PLAY_AUDIO } from '../Utils/Event';
10348 import {
10349     drawLevelSelectCloseGlyph,
10350     drawLevelSelectPanelChrome,
10351     drawLevelSelectTitleBar,
10352     LEVEL_SELECT_CLOSE_DESIGN_SIZE,
10353 } from './OpticalPuzzleLevelSelectPanelGlyph';
10354 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
10355 const { ccclass } = _decorator;
10356 /** 与 levelSelectPanel 占位布局一致（设计 px） */
10357 const PANEL_WIDTH = 650;
10358 const PANEL_HEIGHT = 1200;
10359 const PANEL_OFFSET_Y = -25;
10360 const TITLE_WIDTH = 250;
10361 const TITLE_HEIGHT = 80;
10362 const TITLE_OFFSET_Y = 575;
10363 const CLOSE_SIZE = LEVEL_SELECT_CLOSE_DESIGN_SIZE;
10364 const CLOSE_OFFSET_X = 315;
10365 const CLOSE_OFFSET_Y = 570;
10366 const GFX_CHILD_NAME = 'gfx';
10367 /**
10368  * 关于弹层：panel / titlebar / closebtn 绘制与关闭逻辑（参考 levelSelectPanel，无 scroll）。
10369  * 挂到 layerOverlay/aboutMePanel；子节点尺寸与位置由场景控制。
10370  */
10371 @ccclass('OpticalPuzzleAboutMePanel')
10372 export class OpticalPuzzleAboutMePanel extends Component {
10373     private _backdropNode: Node | null = null;
10374     private _panelNode: Node | null = null;
10375     private _titleNode: Node | null = null;
10376     private _closeNode: Node | null = null;
10377     private _panelGraphics: Graphics | null = null;
10378     private _titleGraphics: Graphics | null = null;
10379     private _closeGraphics: Graphics | null = null;
10380     private _closePressed = false;
10381     private _closePressCtrl: HudButtonPressController | null = null;
10382     protected onLoad(): void {

```

```

10383         this._hidePlaceholderSplashes();
10384         this._ensureRootLayout();
10385         this._buildUITree();
10386         this._bindCloseButton();
10387         this._redrawStaticChrome();
10388     }
10389     protected onEnable(): void {
10390         this._closePressCtrl?.bind();
10391     }
10392     protected onDisable(): void {
10393         this._closePressCtrl?.unbind();
10394         this._closePressed = false;
10395     }
10396     protected onDestroy(): void {
10397         this._closePressCtrl?.unbind();
10398         if (this._closeNode?.isValid) {
10399             this._closeNode.off(Button.EventType.CLICK, this._onCloseClick, this);
10400         }
10401         if (this._backdropNode?.isValid) {
10402             this._backdropNode.off(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
10403         }
10404     }
10405     /** 关闭关于面板 */
10406     close(): void {
10407         if (this.node?.isValid) {
10408             this.node.active = false;
10409         }
10410     }
10411     private _ensureRootLayout(): void {
10412         if (!this.getComponent(UITransform)) {
10413             const ut = this.addComponent(UITransform);
10414             ut.setAnchorPoint(0.5, 0.5);
10415         }
10416     }
10417     private _buildUITree(): void {
10418         this._backdropNode = this._ensureNamedChild('bg');
10419         this._ensureBackdrop(this._backdropNode);
10420         this._panelNode = this._ensureNamedChild('panel');
10421         this._ensurePanelBody(this._panelNode);
10422         this._titleNode = this._ensureNamedChild('titlebar');
10423         this._ensureTitleBar(this._titleNode);
10424         this._closeNode = this._ensureNamedChild('closebtn');
10425         this._ensureCloseButton(this._closeNode);
10426     }
10427     private _ensureNamedChild(name: string): Node {
10428         let child = this.node.getChildByName(name);
10429         if (!child) {
10430             child = new Node(name);
10431             child.setParent(this.node);
10432         }
10433         return child;
10434     }
10435     private _applyDefaultNodeLayout(
10436         node: Node,
10437         width: number,
10438         height: number,
10439         x: number,
10440         y: number,

```

```
10441         anchorX = 0.5,
10442         anchorY = 0.5,
10443     ): void {
10444         const hadTransform = !!node.getComponent(UITransform);
10445         const ut = node.getComponent(UITransform) ?? node.addComponent(UITransform);
10446         if (hadTransform) {
10447             return;
10448         }
10449         ut.setContentSize(width, height);
10450         ut.setAnchorPoint(anchorX, anchorY);
10451         node.setPosition(x, y, 0);
10452     }
10453     private _ensureBackdrop(node: Node): void {
10454         if (!node.getComponent(UITransform)) {
10455             node.addComponent(UITransform);
10456         }
10457         if (!node.getComponent(BlockInputEvents)) {
10458             node.addComponent(BlockInputEvents);
10459         }
10460         this._removeGfxChild(node);
10461         node.off(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
10462         node.on(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
10463     }
10464     private _ensureTouchBlocker(node: Node): void {
10465         if (!node.getComponent(UITransform)) {
10466             node.addComponent(UITransform);
10467         }
10468         if (!node.getComponent(BlockInputEvents)) {
10469             node.addComponent(BlockInputEvents);
10470         }
10471     }
10472     private _removeGfxChild(host: Node): void {
10473         const gfxNode = host.getChildByName(GFX_CHILD_NAME);
10474         if (gfxNode?.isValid) {
10475             gfxNode.destroy();
10476         }
10477     }
10478     private _disableHostSprite(host: Node): void {
10479         const sprite = host.getComponent(Sprite);
10480         if (sprite) {
10481             sprite.enabled = false;
10482         }
10483     }
10484     private _ensurePanelBody(node: Node): void {
10485         this._applyDefaultNodeLayout(node, PANEL_WIDTH, PANEL_HEIGHT, 0, PANEL_OFFSET_Y);
10486         this._ensureTouchBlocker(node);
10487         this._removeGfxChild(node);
10488         this._disableHostSprite(node);
10489         this._panelGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
10490     }
10491     private _ensureTitleBar(node: Node): void {
10492         this._applyDefaultNodeLayout(node, TITLE_WIDTH, TITLE_HEIGHT, 0, TITLE_OFFSET_Y);
10493         this._ensureTouchBlocker(node);
10494         this._removeGfxChild(node);
10495         this._disableHostSprite(node);
10496         this._titleGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
10497     }
10498     private _ensureCloseButton(node: Node): void {
```

```
10499     this._applyDefaultNodeLayout(node, CLOSE_SIZE, CLOSE_SIZE, CLOSE_OFFSET_X, CLOSE_OFFSET_Y);
10500     this._removeGfxChild(node);
10501     this._disableHostSprite(node);
10502     let btn = node.getComponent(Button);
10503     const createdBtn = !btn;
10504     if (!btn) {
10505         btn = node.addComponent(Button);
10506     }
10507     if (createdBtn) {
10508         btn.transition = Button.Transition.SCALE;
10509         btn.zoomScale = 0.95;
10510     }
10511     this._closeGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
10512     this._closePressCtrl = new HudButtonPressController(
10513         node,
10514         (pressed) => {
10515             this._closePressed = pressed;
10516             this._redrawCloseButton();
10517         },
10518         true,
10519     );
10520 }
10521 private _bindCloseButton(): void {
10522     if (!this._closeNode?.isValid) {
10523         return;
10524     }
10525     this._closeNode.off(Button.EventType.CLICK, this._onCloseClick, this);
10526     this._closeNode.on(Button.EventType.CLICK, this._onCloseClick, this);
10527 }
10528 private _hidePlaceholderSplashes(): void {
10529     const walk = (node: Node): void => {
10530         if (node.name === 'SpriteSplash') {
10531             node.active = false;
10532         }
10533         for (const child of node.children) {
10534             walk(child);
10535         }
10536     };
10537     for (const child of this.node.children) {
10538         walk(child);
10539     }
10540 }
10541 private _redrawStaticChrome(): void {
10542     this._redrawPanelBody();
10543     this._redrawTitleBar();
10544     this._redrawCloseButton();
10545 }
10546 private _redrawPanelBody(): void {
10547     const g = this._panelGraphics;
10548     const ut = this._panelNode?.GetComponent(UITransform);
10549     if (!g?.isValid || !ut) {
10550         return;
10551     }
10552     g.clear();
10553     const w = ut.width;
10554     const h = ut.height;
10555     drawLevelSelectPanelChrome(g, -w * 0.5, -h * 0.5, w, h);
10556 }
```

```
10557     private _redrawTitleBar(): void {
10558         const g = this._titleGraphics;
10559         const ut = this._titleNode?.getComponent(UITransform);
10560         if (!g?.isValid || !ut) {
10561             return;
10562         }
10563         g.clear();
10564         const w = ut.width;
10565         const h = ut.height;
10566         drawLevelSelectTitleBar(g, -w * 0.5, -h * 0.5, w, h);
10567     }
10568     private _redrawCloseButton(): void {
10569         const g = this._closeGraphics;
10570         const ut = this._closeNode?.getComponent(UITransform);
10571         if (!g?.isValid || !ut) {
10572             return;
10573         }
10574         g.clear();
10575         const w = ut.width;
10576         const h = ut.height;
10577         drawLevelSelectCloseGlyph(g, -w * 0.5, -h * 0.5, w, h, this._closePressed);
10578     }
10579     private _onCloseClick(): void {
10580         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
10581         this.close();
10582     }
10583     /** 仅点击 bg 空白遮罩时关闭 */
10584     private _onBackdropTouchEnd(event: EventTouch): void {
10585         if (event.target !== this._backdropNode) {
10586             return;
10587         }
10588         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
10589         this.close();
10590     }
10591 }
10592 /** 为 aboutMePanel 根节点挂上绘制与关闭逻辑 */
10593 export function ensureAboutMePanel(panelNode: Node | null): void {
10594     if (!panelNode?.isValid) {
10595         return;
10596     }
10597     if (!panelNode.getComponent(OpticalPuzzleAboutMePanel)) {
10598         panelNode.addComponent(OpticalPuzzleAboutMePanel);
10599     }
10600 }
10601 /** 自 MenuRoot 解析 layerOverlay 下关于面板 */
10602 export function resolveAboutMePanelNode(root: Node | null): Node | null {
10603     const overlay = root?.getChildByName('layerOverlay') ?? null;
10604     if (!overlay?.isValid) {
10605         return null;
10606     }
10607     return (
10608         overlay.getChildByName('aboutMePanel') ??
10609         overlay.getChildByName('AboutMePanel') ??
10610         overlay.getChildByName('aboutWindow') ??
10611         overlay.getChildByName('aboutPanel') ??
10612         null
10613     );
10614 }
```

```

10615 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleActionButtonGlyph.ts -----
10616 import { Color, Graphics } from 'cc';
10617 import {
10618     drawHudButtonChrome,
10619     HUD_BUTTON_DESIGN_SIZE,
10620     HUD_ICON_BORDER_DESIGN,
10621     HUD_KEY_FILL,
10622     fillGlowLayersMatchingStroke,
10623     pressedHudIconColor,
10624     scaleHudDesign,
10625     strokeGlowLayers,
10626     PRESSED_GLOW_LAYERS,
10627     unlitHudIconColor,
10628 } from './OpticalPuzzleHudButtonCommon';
10629 import { traceHudSvgDirectionArrow } from './OpticalPuzzleHudArrowGlyph';
10630 import { Direction } from '../Core/OpticalPuzzleTypes';
10631 import { UNDO_ICON_FILL_STAGES } from '../Application/OpticalPuzzleSession';
10632 export const ACTION_BUTTON_DESIGN_SIZE = HUD_BUTTON_DESIGN_SIZE;
10633 /** 操作键种类 */
10634 export enum ActionButtonKind {
10635     Undo = 'undo',
10636     Reset = 'reset',
10637     Answer = 'answer',
10638 }
10639 /** 图标占键帽比例（与四向键 ARROW_SIZE_RATIO 一致） */
10640 const ICON_SIZE_RATIO = 0.42;
10641 /** 路径归一化半径（viewBox 1102×1024 → 80 键帽约 17px 半宽） */
10642 const UNDO_PATH_UNIT = 17;
10643 /** 重开图标路径半宽（viewBox 1024×1024, iconR=10） */
10644 const RESET_PATH_UNIT = 10;
10645 /** 重开图标相对默认占格放大系数 */
10646 const RESET_ICON_SIZE_RATIO = 1.22;
10647 /** 路径归一化最大半宽（iconR=10, 用于光晕扩散换算） */
10648 const RESET_ICON_NORM_HALF = 10.323;
10649 /** 答案/提示图标路径半宽（viewBox 1024×1024, iconR=10） */
10650 const ANSWER_PATH_UNIT = 10;
10651 /** 钥匙路径归一化最大半宽（光晕扩散） */
10652 const ANSWER_KEY_NORM_HALF = 8.965;
10653 /** 与 Reset 视觉半宽对齐（Reset 1.22 × 10.323 / 钥匙 8.965） */
10654 const ANSWER_ICON_SIZE_RATIO =
10655     RESET_ICON_SIZE_RATIO * (RESET_ICON_NORM_HALF / ANSWER_KEY_NORM_HALF);
10656 /** 用尽角标占键帽比例（贴右上角略溢出） */
10657 const UNDO_DISABLED_BADGE_SIZE_RATIO = 0.6;
10658 /** 角标外轮廓描边（设计 px, 与 HUD_ICON_BORDER_DESIGN 一致） */
10659 const UNDO_DISABLED_BADGE_BORDER_DESIGN = 3;
10660 /** 角标路径归一化半径（viewBox 1024×1024, iconR=8） */
10661 const UNDO_DISABLED_BADGE_PATH_UNIT = 8;
10662 /** 角标相对键帽右上角的向内偏移（设计 px） */
10663 const UNDO_DISABLED_BADGE_INSET_DESIGN = 6;
10664 /** 角标相对键帽右上角的向外溢出（设计 px） */
10665 const UNDO_DISABLED_BADGE_OVERFLOW_DESIGN = 2;
10666 /** 「+」笔画粗细（设计 px, 纯白） */
10667 const BADGE_PLUS_STROKE_DESIGN = 2.25;
10668 /** 「+」双矩形圆角（设计 px） */
10669 const BADGE_PLUS_CORNER_DESIGN = 1;
10670 /** 「+」臂长（设计 px, 相对 80 键帽） */
10671 const BADGE_PLUS_ARM_HEIGHT_DESIGN = 11;
10672 /** 数字 3 高度（设计 px, 相对 80 键帽） */

```



```

10673 const BADGE_DIGIT3_HEIGHT_DESIGN = 15;
10674 /** 数字 3 加粗倍率（路径填充，与「+」3px 笔画视觉对齐） */
10675 const BADGE_DIGIT3_BOLD_SCALE = 1.25;
10676 /** 「+」与「3」间距（设计 px） */
10677 const BADGE_PLUS3_GAP_DESIGN = 1.5;
10678 /** 角标内框中心 X（badge 归一化坐标，iconR=8） */
10679 const BADGE_INNER_CENTER_NX = (-5.595 + 2.976) * 0.5;
10680 /** 参考解角标内右向三角占键帽比例（较四向键略小） */
10681 const ANSWER_VIDEO_BADGE_ARROW_RATIO = 0.15;
10682 /** 激励视频角标布局（与键帽本地坐标一致：锚点 0.5 时 left=-w/2） */
10683 export interface VideoRewardBadgeLayout {
10684     centerX: number;
10685     centerY: number;
10686     diameter: number;
10687 }
10688 /** 右上角胶片角标中心与热区直径（含溢出区） */
10689 export function computeVideoRewardBadgeLayout(
10690     left: number,
10691     bottom: number,
10692     width: number,
10693     height: number,
10694 ): VideoRewardBadgeLayout {
10695     const size = Math.min(width, height);
10696     const badgeHalf = size * UNDO_DISABLED_BADGE_SIZE_RATIO * 0.5;
10697     const inset = scaleHudDesign(size, UNDO_DISABLED_BADGE_INSET_DESIGN);
10698     const overflow = scaleHudDesign(size, UNDO_DISABLED_BADGE_OVERFLOW_DESIGN);
10699     const cx = left + width - inset + overflow;
10700     const cy = bottom + height - inset + overflow;
10701     const diameter = badgeHalf * 2 + overflow * 2;
10702     return { centerX: cx, centerY: cy, diameter };
10703 }
10704 /** 数字 3 路径归一化半高（viewBox 1024, iconR=10） */
10705 const BADGE_DIGIT3_PATH_HALF_H = 6.322;
10706 /** 数字 3 路径归一化宽（iconR=10） */
10707 const BADGE_DIGIT3_PATH_W = 8.037;
10708 const ICON_STROKE = new Color(255, 255, 255, 255);
10709 type UndoSeg =
10710     | { t: 'M'; x: number; y: number }
10711     | { t: 'L'; x: number; y: number }
10712     | { t: 'C'; x1: number; y1: number; x2: number; y2: number; x: number; y: number }
10713     | { t: 'Z' };
10714 /** 外轮廓（用户 SVG）：实心填充，与四向键一致不做内孔 */
10715 const UNDO_ICON_SEGS: ReadonlyArray<UndoSeg> = [
10716     { t: 'M', x: 14.46, y: -15.078 },
10717     { t: 'C', x1: 14.076, y1: -13.213, x2: 12.148, y2: -7.064, x: 2.129, y: -6.316 },
10718     { t: 'L', x: 2.129, y: -13.027 },
10719     { t: 'C', x1: 2.129, y1: -14.144, x2: 1.553, y2: -15.078, x: 0.779, y: -15.449 },
10720     { t: 'C', x1: -0.183, y1: -16.009, x2: -1.339, y2: -16.009, x: -2.302, y: -15.263 },
10721     { t: 'L', x: -2.495, y: -15.078 },
10722     { t: 'L', x: -17.139, y: -1.661 },
10723     { t: 'C', x1: -17.717, y1: -1.098, x2: -18.295, y2: -0.353, x: -18.295, y: 0.575 },
10724     { t: 'C', x1: -18.295, y1: 1.509, x2: -17.91, y2: 2.254, x: -17.139, y: 2.812 },
10725     { t: 'L', x: -2.302, y: 16.414 },
10726     { t: 'C', x1: -1.339, y1: 17.16, x2: -0.183, y2: 17.16, x: 0.779, y: 16.602 },
10727     { t: 'C', x1: 1.744, y1: 16.043, x2: 2.129, y2: 15.112, x: 2.129, y: 14.178 },
10728     { t: 'L', x: 2.129, y: 8.403 },
10729     { t: 'C', x1: 7.524, y1: 7.098, x2: 18.315, y2: 3.371, x: 18.315, y: -14.889 },
10730     { t: 'C', x1: 18.315, y1: -14.889, x2: 18.506, y2: -16.38, x: 16.772, y: -16.754 },

```

```

10731      { t: 'C', x1: 15.038, y1: -17.126, x2: 14.46, y2: -15.078, x: 14.46, y: -15.078 },
10732      { t: 'Z' },
10733  ];
10734  /** 重开：双弧循环箭头（用户 SVG，iconR=10；弧段为三次贝塞尔） */
10735  const RESET_ICON_SEGS: ReadonlyArray<UndoSeg> = [
10736      { t: 'M', x: -1.234, y: -6.26 },
10737      { t: 'L', x: -2.262, y: -6.26 },
10738      { t: 'C', x1: -5.228, y1: -5.775, x2: -7.498, y2: -3.192, x: -7.498, y: -0.089 },
10739      { t: 'C', x1: -7.498, y1: 0.95, x2: -7.231, y2: 1.923, x: -6.78, y: 2.786 },
10740      { t: 'L', x: -6.013, y: 1.723 },
10741      { t: 'C', x1: -5.455, y1: 0.949, x2: -4.251, y2: 1.165, x: -3.995, y: 2.084 },
10742      { t: 'L', x: -3.384, y: 4.281 },
10743      { t: 'L', x: -2.542, y: 7.311 },
10744      { t: 'C', x1: -2.34, y1: 8.037, x2: -2.886, y2: 8.756, x: -3.639, y: 8.754 },
10745      { t: 'L', x: -6.61, y: 8.75 },
10746      { t: 'L', x: -8.859, y: 8.747 },
10747      { t: 'C', x1: -9.786, y1: 8.746, x2: -10.323, y2: 7.695, x: -9.78, y: 6.943 },
10748      { t: 'L', x: -8.368, y: 4.986 },
10749      { t: 'C', x1: -9.391, y1: 3.555, x2: -9.997, y2: 1.805, x: -9.997, y: -0.089 },
10750      { t: 'C', x1: -9.997, y1: -4.466, x2: -6.788, y2: -8.092, x: -2.596, y: -8.74 },
10751      { t: 'C', x1: -2.481, y1: -8.758, x2: -2.37, y2: -8.753, x: -2.262, y: -8.739 },
10752      { t: 'L', x: -2.262, y: -8.754 },
10753      { t: 'L', x: -1.234, y: -8.754 },
10754      { t: 'C', x1: -0.552, y1: -8.754, x2: 0, y2: -8.202, x: 0, y: -7.52 },
10755      { t: 'L', x: 0, y: -7.495 },
10756      { t: 'C', x1: 0, y1: -6.813, x2: -0.552, y2: -6.26, x: -1.234, y: -6.26 },
10757      { t: 'Z' },
10758      { t: 'M', x: 9.78, y: -6.943 },
10759      { t: 'L', x: 8.368, y: -4.987 },
10760      { t: 'C', x1: 9.391, y1: -3.555, x2: 9.997, y2: -1.805, x: 9.997, y: 0.089 },
10761      { t: 'C', x1: 9.997, y1: 4.466, x2: 6.788, y2: 8.092, x: 2.596, y: 8.74 },
10762      { t: 'C', x1: 2.481, y1: 8.758, x2: 2.37, y2: 8.753, x: 2.262, y: 8.739 },
10763      { t: 'L', x: 2.262, y: 8.754 },
10764      { t: 'L', x: 1.234, y: 8.754 },
10765      { t: 'C', x1: 0.552, y1: 8.754, x2: 0, y2: 8.202, x: 0, y: 7.52 },
10766      { t: 'L', x: 0, y: 7.495 },
10767      { t: 'C', x1: 0, y1: 6.813, x2: 0.552, y2: 6.26, x: 1.234, y: 6.26 },
10768      { t: 'L', x: 2.262, y: 6.26 },
10769      { t: 'C', x1: 5.228, y1: 5.775, x2: 7.498, y2: 3.192, x: 7.498, y: 0.089 },
10770      { t: 'C', x1: 7.498, y1: -0.95, x2: 7.231, y2: -1.923, x: 6.78, y: -2.786 },
10771      { t: 'L', x: 6.013, y: -1.723 },
10772      { t: 'C', x1: 5.455, y1: -0.949, x2: 4.251, y2: -1.165, x: 3.995, y: -2.084 },
10773      { t: 'L', x: 3.385, y: -4.281 },
10774      { t: 'L', x: 2.542, y: -7.311 },
10775      { t: 'C', x1: 2.34, y1: -8.037, x2: 2.886, y2: -8.756, x: 3.639, y: -8.754 },
10776      { t: 'L', x: 6.61, y: -8.75 },
10777      { t: 'L', x: 8.86, y: -8.747 },
10778      { t: 'C', x1: 9.786, y1: -8.746, x2: 10.323, y2: -7.695, x: 9.78, y: -6.943 },
10779      { t: 'Z' },
10780  ];
10781  /** 撤回用尽角标（用户 SVG：胶片框 + 播放三角 + 左上标记块） */
10782  const UNDO_DISABLED_BADGE_FILL_SEGS: ReadonlyArray<UndoSeg> = [
10783      { t: 'M', x: 5.953, y: 3.098 },
10784      { t: 'L', x: 4.047, y: 2.009 },
10785      { t: 'L', x: 4.047, y: 4.255 },
10786      { t: 'C', x1: 4.047, y1: 4.776, x2: 3.62, y2: 5.2, x: 3.095, y: 5.2 },
10787      { t: 'L', x: -5.714, y: 5.2 },
10788      { t: 'C', x1: -6.24, y1: 5.2, x2: -6.667, y2: 4.776, x: -6.667, y: 4.255 },

```

```

10789     { t: 'L', x: -6.667, y: -4.255 },
10790     { t: 'C', x1: -6.667, y1: -4.776, x2: -6.24, y2: -5.2, x: -5.714, y: -5.2 },
10791     { t: 'L', x: 3.095, y: -5.2 },
10792     { t: 'C', x1: 3.621, y1: -5.2, x2: 4.047, y2: -4.776, x: 4.047, y: -4.255 },
10793     { t: 'L', x: 4.047, y: -2.009 },
10794     { t: 'L', x: 5.953, y: -3.098 },
10795     { t: 'L', x: 6.667, y: -2.69 },
10796     { t: 'L', x: 6.667, y: 2.689 },
10797     { t: 'L', x: 5.953, y: 3.098 },
10798     { t: 'Z' },
10799     { t: 'M', x: 2.976, y: -4.136 },
10800     { t: 'L', x: -5.595, y: -4.136 },
10801     { t: 'L', x: -5.595, y: 4.137 },
10802     { t: 'L', x: 2.976, y: 4.137 },
10803     { t: 'L', x: 2.976, y: -4.137 },
10804     { t: 'Z' },
10805     { t: 'M', x: 5.595, y: -1.669 },
10806     { t: 'L', x: 4.047, y: -0.786 },
10807     { t: 'L', x: 4.047, y: 0.785 },
10808     { t: 'L', x: 5.595, y: 1.669 },
10809     { t: 'L', x: 5.595, y: -1.669 },
10810     { t: 'Z' },
10811     { t: 'M', x: -4.524, y: 2.245 },
10812     { t: 'L', x: -2.857, y: 2.245 },
10813     { t: 'C', x1: -2.792, y1: 2.245, x2: -2.738, y2: 2.299, x: -2.738, y: 2.364 },
10814     { t: 'L', x: -2.738, y: 3.073 },
10815     { t: 'L', x: -2.857, y: 3.191 },
10816     { t: 'L', x: -4.524, y: 3.191 },
10817     { t: 'L', x: -4.643, y: 3.073 },
10818     { t: 'L', x: -4.643, y: 2.364 },
10819     { t: 'C', x1: -4.643, y1: 2.299, x2: -4.589, y2: 2.245, x: -4.524, y: 2.245 },
10820     { t: 'Z' },
10821 ];
10822 /** 角标描边路径: 仅最外轮廓(内框/播放三角/左上块只填充不描边, 避免薄形状与共用边叠线) */
10823 const UNDO_DISABLED_BADGE_STROKE_SEGS: ReadonlyArray<UndoSeg> = [
10824     { t: 'M', x: 5.953, y: 3.098 },
10825     { t: 'L', x: 4.047, y: 2.009 },
10826     { t: 'L', x: 4.047, y: 4.255 },
10827     { t: 'C', x1: 4.047, y1: 4.776, x2: 3.62, y2: 5.2, x: 3.095, y: 5.2 },
10828     { t: 'L', x: -5.714, y: 5.2 },
10829     { t: 'C', x1: -6.24, y1: 5.2, x2: -6.667, y2: 4.776, x: -6.667, y: 4.255 },
10830     { t: 'L', x: -6.667, y: -4.255 },
10831     { t: 'C', x1: -6.667, y1: -4.776, x2: -6.24, y2: -5.2, x: -5.714, y: -5.2 },
10832     { t: 'L', x: 3.095, y: -5.2 },
10833     { t: 'C', x1: 3.621, y1: -5.2, x2: 4.047, y2: -4.776, x: 4.047, y: -4.255 },
10834     { t: 'L', x: 4.047, y: -2.009 },
10835     { t: 'L', x: 5.953, y: -3.098 },
10836     { t: 'L', x: 6.667, y: -2.69 },
10837     { t: 'L', x: 6.667, y: 2.689 },
10838     { t: 'L', x: 5.953, y: 3.098 },
10839     { t: 'Z' },
10840 ];
10841 /** 角标内数字 3 (用户 SVG, iconR=10) */
10842 const BADGE_DIGIT3_SEGS: ReadonlyArray<UndoSeg> = [
10843     { t: 'M', x: -3.972, y: -2.864 },
10844     { t: 'L', x: -2.465, y: -2.663 },
10845     { t: 'C', x1: -2.292, y1: -3.517, x2: -1.998, y2: -4.132, x: -1.582, y: -4.509 },
10846     { t: 'C', x1: -1.166, y1: -4.886, x2: -0.66, y2: -5.074, x: -0.063, y: -5.074 },

```

```
10847     { t: 'C', x1: 0.646, y1: -5.074, x2: 1.245, y2: -4.829, x: 1.733, y: -4.337 },
10848     { t: 'C', x1: 2.221, y1: -3.846, x2: 2.466, y2: -3.238, x: 2.466, y: -2.513 },
10849     { t: 'C', x1: 2.466, y1: -1.821, x2: 2.24, y2: -1.25, x: 1.788, y: -0.801 },
10850     { t: 'C', x1: 1.336, y1: -0.351, x2: 0.761, y2: -0.127, x: 0.063, y: -0.127 },
10851     { t: 'C', x1: -0.222, y1: -0.127, x2: -0.576, y2: -0.183, x: -1.0, y: -0.294 },
10852     { t: 'L', x: -0.833, y: 1.029 },
10853     { t: 'C', x1: -0.732, y1: 1.017, x2: -0.651, y2: 1.012, x: -0.59, y: 1.012 },
10854     { t: 'C', x1: 0.052, y1: 1.012, x2: 0.629, y2: 1.179, x: 1.143, y: 1.514 },
10855     { t: 'C', x1: 1.656, y1: 1.849, x2: 1.913, y2: 2.365, x: 1.913, y: 3.063 },
10856     { t: 'C', x1: 1.913, y1: 3.615, x2: 1.726, y2: 4.073, x: 1.352, y: 4.436 },
10857     { t: 'C', x1: 0.978, y1: 4.798, x2: 0.495, y2: 4.98, x: -0.096, y: 4.98 },
10858     { t: 'C', x1: -0.682, y1: 4.98, x2: -1.17, y2: 4.796, x: -1.561, y: 4.427 },
10859     { t: 'C', x1: -1.952, y1: 4.059, x2: -2.203, y2: 3.506, x: -2.314, y: 2.77 },
10860     { t: 'L', x: -3.821, y: 3.038 },
10861     { t: 'C', x1: -3.637, y1: 4.048, x2: -3.219, y2: 4.83, x: -2.566, y: 5.386 },
10862     { t: 'C', x1: -1.913, y1: 5.941, x2: -1.101, y2: 6.219, x: -0.129, y: 6.219 },
10863     { t: 'C', x1: 0.54, y1: 6.219, x2: 1.157, y2: 6.075, x: 1.721, y: 5.788 },
10864     { t: 'C', x1: 2.284, y1: 5.5, x2: 2.715, y2: 5.108, x: 3.014, y: 4.611 },
10865     { t: 'C', x1: 3.312, y1: 4.115, x2: 3.462, y2: 3.587, x: 3.462, y: 3.029 },
10866     { t: 'C', x1: 3.462, y1: 2.499, x2: 3.319, y2: 2.016, x: 3.035, y: 1.581 },
10867     { t: 'C', x1: 2.75, y1: 1.146, x2: 2.329, y2: 0.8, x: 1.771, y: 0.543 },
10868     { t: 'C', x1: 2.496, y1: 0.376, x2: 3.06, y2: 0.028, x: 3.462, y: -0.499 },
10869     { t: 'C', x1: 3.864, y1: -1.027, x2: 4.065, y2: -1.687, x: 4.065, y: -2.479 },
10870     { t: 'C', x1: 4.065, y1: -3.551, x2: 3.674, y2: -4.459, x: 2.893, y: -5.204 },
10871     { t: 'C', x1: 2.111, y1: -5.949, x2: 1.123, y2: -6.322, x: -0.071, y: -6.322 },
10872     { t: 'C', x1: -1.148, y1: -6.322, x2: -2.042, y2: -6.001, x: -2.754, y: -5.359 },
10873     { t: 'C', x1: -3.465, y1: -4.717, x2: -3.871, y2: -3.885, x: -3.972, y: -2.864 },
10874     { t: 'Z' },
10875 ];
10876 /** 答案/提示: 钥匙主体 (用户 SVG 子路径 0) */
10877 const ANSWER_KEY_BODY_SEGS: ReadonlyArray<UndoSeg> = [
10878     { t: 'M', x: -1.133, y: 6.708 },
10879     { t: 'C', x1: 0.788, y1: 8.631, x2: 3.82, y2: 8.877, x: 6.027, y: 7.29 },
10880     { t: 'C', x1: 8.233, y1: 5.703, x2: 8.965, y2: 2.75, x: 7.754, y: 0.317 },
10881     { t: 'C', x1: 6.543, y1: -2.117, x2: 3.747, y2: -3.314, x: 1.15, y: -2.511 },
10882     { t: 'L', x: -1.561, y: -5.221 },
10883     { t: 'L', x: -2.344, y: -4.436 },
10884     { t: 'C', x1: -2.748, y1: -4.032, x2: -3.393, y2: -4.001, x: -3.834, y: -4.364 },
10885     { t: 'L', x: -3.913, y: -4.436 },
10886     { t: 'C', x1: -4.317, y1: -4.84, x2: -4.348, y2: -5.484, x: -3.985, y: -5.925 },
10887     { t: 'L', x: -3.913, y: -6.004 },
10888     { t: 'L', x: -3.13, y: -6.789 },
10889     { t: 'L', x: -4.457, y: -8.117 },
10890     { t: 'C', x1: -4.609, y1: -8.269, x2: -4.821, y2: -8.348, x: -5.037, y: -8.331 },
10891     { t: 'L', x: -7.464, y: -8.144 },
10892     { t: 'C', x1: -7.827, y1: -8.116, x2: -8.116, y2: -7.827, x: -8.144, y: -7.464 },
10893     { t: 'L', x: -8.331, y: -5.037 },
10894     { t: 'C', x1: -8.348, y1: -4.821, x2: -8.269, y2: -4.609, x: -8.117, y: -4.457 },
10895     { t: 'L', x: -2.511, y: 1.149 },
10896     { t: 'C', x1: -3.118, y1: 3.114, x2: -2.588, y2: 5.254, x: -1.133, y: 6.708 },
10897     { t: 'Z' },
10898 ];
10899 /** 答案/提示: 匙孔圆 (子路径 1; 未按时用键帽色镂空) */
10900 const ANSWER_KEY_HOLE_SEGS: ReadonlyArray<UndoSeg> = [
10901     { t: 'M', x: 1.481, y: 4.095 },
10902     { t: 'C', x1: 0.759, y1: 3.373, x2: 0.759, y2: 2.203, x: 1.481, y: 1.481 },
10903     { t: 'C', x1: 2.203, y1: 0.759, x2: 3.373, y2: 0.759, x: 4.095, y: 1.481 },
10904     { t: 'C', x1: 4.817, y1: 2.203, x2: 4.817, y2: 3.373, x: 4.095, y: 4.095 },
```

```

10905     { t: 'C', x1: 3.373, y1: 4.817, x2: 2.203, y2: 4.817, x: 1.481, y: 4.095 },
10906     { t: 'Z' },
10907 ];
10908 /** 匙孔圆心（归一化坐标） */
10909 const ANSWER_KEY_HOLE_CENTER_NX = 2.788;
10910 const ANSWER_KEY_HOLE_CENTER_NY = 2.788;
10911 /** 匙孔未放大时的归一化半径 */
10912 const ANSWER_KEY_HOLE_NORM_RADIUS = 2.029;
10913 /** 按下发光时匙孔放大倍率 */
10914 const ANSWER_KEY_HOLE_PRESSED_SCALE = 1.5;
10915 function tracelconSegs(
10916     g: Graphics,
10917     segs: ReadonlyArray<UndoSeg>,
10918     cx: number,
10919     cy: number,
10920     scale: number,
10921 ): void {
10922     for (const seg of segs) {
10923         switch (seg.t) {
10924             case 'M':
10925                 g.moveTo(cx + seg.x * scale, cy + seg.y * scale);
10926                 break;
10927             case 'L':
10928                 g.lineTo(cx + seg.x * scale, cy + seg.y * scale);
10929                 break;
10930             case 'C':
10931                 g.bezierCurveTo(
10932                     cx + seg.x1 * scale,
10933                     cy + seg.y1 * scale,
10934                     cx + seg.x2 * scale,
10935                     cy + seg.y2 * scale,
10936                     cx + seg.x * scale,
10937                     cy + seg.y * scale,
10938                 );
10939                 break;
10940             case 'Z':
10941                 g.close();
10942                 break;
10943             default:
10944                 break;
10945         }
10946     }
10947 }
10948 function tracelconSegsScaledAbout(
10949     g: Graphics,
10950     segs: ReadonlyArray<UndoSeg>,
10951     cx: number,
10952     cy: number,
10953     scale: number,
10954     originNx: number,
10955     originNy: number,
10956     pointScale: number,
10957 ): void {
10958     const map = (x: number, y: number) => ({
10959         x: originNx + (x - originNx) * pointScale,
10960         y: originNy + (y - originNy) * pointScale,
10961     });
10962     for (const seg of segs) {

```

```

10963         switch (seg.t) {
10964             case 'M': {
10965                 const p = map(seg.x, seg.y);
10966                 g.moveTo(cx + p.x * scale, cy + p.y * scale);
10967                 break;
10968             }
10969             case 'L': {
10970                 const p = map(seg.x, seg.y);
10971                 g.lineTo(cx + p.x * scale, cy + p.y * scale);
10972                 break;
10973             }
10974             case 'C': {
10975                 const p1 = map(seg.x1, seg.y1);
10976                 const p2 = map(seg.x2, seg.y2);
10977                 const p = map(seg.x, seg.y);
10978                 g.bezierCurveTo(
10979                     cx + p1.x * scale,
10980                     cy + p1.y * scale,
10981                     cx + p2.x * scale,
10982                     cy + p2.y * scale,
10983                     cx + p.x * scale,
10984                     cy + p.y * scale,
10985                 );
10986                 break;
10987             }
10988             case 'Z':
10989                 g.close();
10990                 break;
10991             default:
10992                 break;
10993         }
10994     }
10995 }
10996 function traceUndoIcon(g: Graphics, cx: number, cy: number, scale: number): void {
10997     traceIconSegs(g, UNDO_ICON_SEGS, cx, cy, scale);
10998 }
10999 function traceResetIcon(g: Graphics, cx: number, cy: number, scale: number): void {
11000     traceIconSegs(g, RESET_ICON_SEGS, cx, cy, scale);
11001 }
11002 function traceAnswerKeyBody(g: Graphics, cx: number, cy: number, scale: number): void {
11003     traceIconSegs(g, ANSWER_KEY_BODY_SEGS, cx, cy, scale);
11004 }
11005 function traceAnswerKeyHole(
11006     g: Graphics,
11007     cx: number,
11008     cy: number,
11009     scale: number,
11010     holeScale = 1,
11011 ): void {
11012     if (holeScale === 1) {
11013         traceIconSegs(g, ANSWER_KEY_HOLE_SEGS, cx, cy, scale);
11014         return;
11015     }
11016     traceIconSegsScaledAbout(
11017         g,
11018         ANSWER_KEY_HOLE_SEGS,
11019         cx,
11020         cy,

```

```

11021         scale,
11022         ANSWER_KEY_HOLE_CENTER_NX,
11023         ANSWER_KEY_HOLE_CENTER_NY,
11024         holeScale,
11025     );
11026 }
11027 function traceAnswerKeyOutline(
11028     g: Graphics,
11029     cx: number,
11030     cy: number,
11031     scale: number,
11032     holeScale = 1,
11033 ): void {
11034     traceAnswerKeyBody(g, cx, cy, scale);
11035     traceAnswerKeyHole(g, cx, cy, scale, holeScale);
11036 }
11037 /** 匙孔向内光晕：与外圈同款 strokeGlowLayers，再内收暗心保留描边内侧半宽 */
11038 function fillHoleInwardGlow(
11039     g: Graphics,
11040     cx: number,
11041     cy: number,
11042     scale: number,
11043     holeScale: number,
11044     size: number,
11045 ): void {
11046     const holeRadiusPx = Math.max(scale * ANSWER_KEY_HOLE_NORM_RADIUS * holeScale, 1);
11047     const traceHole = (): void => {
11048         traceAnswerKeyHole(g, cx, cy, scale, holeScale);
11049     };
11050     g.fillColor = HUD_KEY_FILL;
11051     traceHole();
11052     g.fill();
11053     strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, traceHole, true);
11054     const maxHalfPx = scaleHudDesign(size, PRESSED_GLOW_LAYERS[0].width) * 0.5;
11055     const centerScale = Math.max(0.05, holeScale * (1 - maxHalfPx / holeRadiusPx));
11056     g.fillColor = HUD_KEY_FILL;
11057     traceAnswerKeyHole(g, cx, cy, scale, centerScale);
11058     g.fill();
11059 }
11060 /** 横向填充比例：stage0=100%，每 +1 少 33% 右缘 */
11061 function undoFillRatio(fillStage: number): number {
11062     const stage = Math.max(0, Math.min(UNDO_ICON_FILL_STAGES, fillStage));
11063     return Math.max(0, (UNDO_ICON_FILL_STAGES - stage) / UNDO_ICON_FILL_STAGES);
11064 }
11065 function undolconBounds(cx: number, cy: number, size: number): {
11066     left: number;
11067     bottom: number;
11068     width: number;
11069     height: number;
11070 } {
11071     const half = size * ICON_SIZE_RATIO * 0.5;
11072     return { left: cx - half, bottom: cy - half, width: half * 2, height: half * 2 };
11073 }
11074 /** 用键帽色盖住图标右缘，实现从左向右保留 fillRatio 的填充 */
11075 function coverUndolconRight(
11076     g: Graphics,
11077     cx: number,
11078     cy: number,

```

```

11079         size: number,
11080         fillRatio: number,
11081     ): void {
11082         if (fillRatio >= 1) {
11083             return;
11084         }
11085         const box = undolconBounds(cx, cy, size);
11086         const coverW = box.width * (1 - fillRatio);
11087         g.fillColor = HUD_KEY_FILL;
11088         g.rect(box.left + box.width - coverW, box.bottom, coverW, box.height);
11089         g.fill();
11090     }
11091     function strokeUndolconOutline(
11092         g: Graphics,
11093         cx: number,
11094         cy: number,
11095         scale: number,
11096         lineWidth: number,
11097     ): void {
11098         g.strokeColor = ICON_STROKE;
11099         g.lineWidth = lineWidth;
11100         g.lineJoin = Graphics.LineJoin.ROUND;
11101         g.lineCap = Graphics.LineCap.ROUND;
11102         traceUndolcon(g, cx, cy, scale);
11103         g.stroke();
11104     }
11105     /** 撤回：内部填充可横向递减；外轮廓始终完整描边，按下时光晕覆盖全路径 */
11106     function drawUndolcon(
11107         g: Graphics,
11108         cx: number,
11109         cy: number,
11110         size: number,
11111         pressed: boolean,
11112         fillStage: number,
11113     ): void {
11114         const iconHalf = size * ICON_SIZE_RATIO * 0.5;
11115         const scale = iconHalf / UNDO_PATH_UNIT;
11116         const borderW = Math.max(1, scaleHudDesign(size, HUD_ICON_BORDER_DESIGN));
11117         const fillRatio = undoFillRatio(fillStage);
11118         const tracePath = (): void => {
11119             traceUndolcon(g, cx, cy, scale);
11120         };
11121         if (fillRatio > 0) {
11122             g.fillColor = pressed ? pressedHudIconColor() : unlitHudIconColor();
11123             tracePath();
11124             g.fill();
11125             coverUndolconRight(g, cx, cy, size, fillRatio);
11126         }
11127         if (pressed) {
11128             strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, tracePath, true);
11129         }
11130         strokeUndolconOutline(g, cx, cy, scale, borderW);
11131     }
11132     /** 重开：未按浅白 + 3px 白描边；按下全白 + 光晕（与四向键/面型键一致） */
11133     function drawResetIcon(
11134         g: Graphics,
11135         cx: number,
11136         cy: number,

```



```

11137     size: number,
11138     pressed: boolean,
11139 ): void {
11140     const iconHalf = size * ICON_SIZE_RATIO * 0.5;
11141     const scale = (iconHalf * RESET_ICON_SIZE_RATIO) / RESET_PATH_UNIT;
11142     const iconHalfPx = scale * RESET_ICON_NORM_HALF;
11143     const borderW = Math.max(1, scaleHudDesign(size, HUD_ICON_BORDER_DESIGN));
11144     const tracePath = (): void => {
11145         traceResetIcon(g, cx, cy, scale);
11146     };
11147     if (pressed) {
11148         fillGlowLayersMatchingStroke(
11149             g,
11150             size,
11151             PRESSED_GLOW_LAYERS,
11152             iconHalfPx,
11153             (expand) => traceResetIcon(g, cx, cy, scale * expand),
11154         );
11155         g.fillColor = pressedHudIconColor();
11156         tracePath();
11157         g.fill();
11158         return;
11159     }
11160     g.fillColor = unlitHudIconColor();
11161     tracePath();
11162     g.fill();
11163     g.strokeColor = ICON_STROKE;
11164     g.lineWidth = borderW;
11165     g.lineJoin = Graphics.LineJoin.ROUND;
11166     g.lineCap = Graphics.LineCap.ROUND;
11167     tracePath();
11168     g.stroke();
11169 }
11170 /** 答案/提示: 浅白填充 + 3px 白描边; 按下外轮廓外扩光晕 + 匙孔向内光晕 */
11171 function drawAnswerIcon(
11172     g: Graphics,
11173     cx: number,
11174     cy: number,
11175     size: number,
11176     pressed: boolean,
11177 ): void {
11178     const iconHalf = size * ICON_SIZE_RATIO * 0.5;
11179     const scale = (iconHalf * ANSWER_ICON_SIZE_RATIO) / ANSWER_PATH_UNIT;
11180     const borderW = Math.max(1, scaleHudDesign(size, HUD_ICON_BORDER_DESIGN));
11181     const traceOuterOutline = (): void => {
11182         traceAnswerKeyBody(g, cx, cy, scale);
11183     };
11184     const traceFullOutline = (holeScale = 1): void => {
11185         traceAnswerKeyOutline(g, cx, cy, scale, holeScale);
11186     };
11187     const fillKey = (bodyColor: Color, holeScale = 1): void => {
11188         g.fillColor = bodyColor;
11189         traceAnswerKeyBody(g, cx, cy, scale);
11190         g.fill();
11191         g.fillColor = HUD_KEY_FILL;
11192         traceAnswerKeyHole(g, cx, cy, scale, holeScale);
11193         g.fill();
11194     };

```

```

11195     const strokeOutline = (color: Color, holeScale = 1): void => {
11196         g.strokeColor = color;
11197         g.lineWidth = borderW;
11198         g.lineJoin = Graphics.LineJoin.ROUND;
11199         g.lineCap = Graphics.LineCap.ROUND;
11200         traceFullOutline(holeScale);
11201         g.stroke();
11202     };
11203     if (pressed) {
11204         const holeScale = ANSWER_KEY_HOLE_PRESSED_SCALE;
11205         strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, traceOuterOutline, true);
11206         g.fillColor = pressedHudIconColor();
11207         traceAnswerKeyBody(g, cx, cy, scale);
11208         g.fill();
11209         fillHoleInwardGlow(g, cx, cy, scale, holeScale, size);
11210         strokeOutline(pressedHudIconColor(), holeScale);
11211         return;
11212     }
11213     fillKey(unlitHudIconColor());
11214     strokeOutline(ICON_STROKE);
11215 }
11216 /** 「+」: 两枚圆角矩形交叉, 3px 厚、1px 圆角, 纯白填充 */
11217 function drawBadgePlusCross(
11218     g: Graphics,
11219     cx: number,
11220     cy: number,
11221     keySize: number,
11222     armLength: number,
11223 ): void {
11224     const thick = Math.max(1, scaleHudDesign(keySize, BADGE_PLUS_STROKE_DESIGN));
11225     const corner = Math.max(0.5, scaleHudDesign(keySize, BADGE_PLUS_CORNER_DESIGN));
11226     g.fillColor = ICON_STROKE;
11227     g.roundRect(cx - armLength * 0.5, cy - thick * 0.5, armLength, thick, corner);
11228     g.fill();
11229     g.roundRect(cx - thick * 0.5, cy - armLength * 0.5, thick, armLength, corner);
11230     g.fill();
11231 }
11232 /** 角标内「+3」: 纯白、3px 笔画; 不参与按下光晕 */
11233 function drawUndoDisabledBadgePlus3(
11234     g: Graphics,
11235     badgeCx: number,
11236     badgeCy: number,
11237     badgeScale: number,
11238     keySize: number,
11239 ): void {
11240     const groupCx = badgeCx + BADGE_INNER_CENTER_NX * badgeScale;
11241     const groupCy = badgeCy;
11242     const digitH = scaleHudDesign(keySize, BADGE_DIGIT3_HEIGHT_DESIGN);
11243     const plusArm = scaleHudDesign(keySize, BADGE_PLUS_ARM_HEIGHT_DESIGN);
11244     const digitScale =
11245         (digitH / (BADGE_DIGIT3_PATH_HALF_H * 2)) * BADGE_DIGIT3_BOLD_SCALE;
11246     const gap = scaleHudDesign(keySize, BADGE_PLUS3_GAP_DESIGN);
11247     const digitW = BADGE_DIGIT3_PATH_W * digitScale;
11248     const groupW = plusArm + gap + digitW;
11249     const plusCx = groupCx - groupW * 0.5 + plusArm * 0.5;
11250     const digitCx = groupCx - groupW * 0.5 + plusArm + gap + digitW * 0.5;
11251     drawBadgePlusCross(g, plusCx, groupCy, keySize, plusArm);
11252     g.fillColor = ICON_STROKE;

```

```

11253     traceIconSegs(g, BADGE_DIGIT3_SEGS, digitCx, groupCy, digitScale);
11254     g.fill();
11255 }
11256 /** stage=3 时在键帽右上角叠激励视频角标（胶片框 + 中心内容） */
11257 function drawVideoRewardBadgeFrame(
11258     g: Graphics,
11259     left: number,
11260     bottom: number,
11261     width: number,
11262     height: number,
11263     pressed: boolean,
11264     drawCenter: (badgeCx: number, badgeCy: number, badgeScale: number, keySize: number) => void,
11265 ): void {
11266     const size = Math.min(width, height);
11267     const layout = computeVideoRewardBadgeLayout(left, bottom, width, height);
11268     const cx = layout.centerX;
11269     const cy = layout.centerY;
11270     const badgeHalf = size * UNDO_DISABLED_BADGE_SIZE_RATIO * 0.5;
11271     const scale = badgeHalf / UNDO_DISABLED_BADGE_PATH_UNIT;
11272     const borderW = Math.max(1, scaleHudDesign(size, UNDO_DISABLED_BADGE_BORDER_DESIGN));
11273     const traceFill = (): void => {
11274         traceIconSegs(g, UNDO_DISABLED_BADGE_FILL_SEGS, cx, cy, scale);
11275     };
11276     const traceStroke = (): void => {
11277         traceIconSegs(g, UNDO_DISABLED_BADGE_STROKE_SEGS, cx, cy, scale);
11278     };
11279     g.fillColor = unlitHudIconColor();
11280     traceFill();
11281     g.fill();
11282     if (pressed) {
11283         strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, traceStroke, true);
11284     }
11285     g.strokeColor = ICON_STROKE;
11286     g.lineWidth = borderW;
11287     g.lineJoin = Graphics.LineJoin.ROUND;
11288     g.lineCap = Graphics.LineCap.ROUND;
11289     traceStroke();
11290     g.stroke();
11291     drawCenter(cx, cy, scale, size);
11292 }
11293 /** 角标内右向三角：对齐左侧内矩形居中，纯白；按下时三角光晕 */
11294 function drawAnswerVideoBadgeTriangle(
11295     g: Graphics,
11296     badgeCx: number,
11297     badgeCy: number,
11298     badgeScale: number,
11299     keySize: number,
11300     pressed: boolean,
11301 ): void {
11302     const triCx = badgeCx + BADGE_INNER_CENTER_NX * badgeScale;
11303     const triCy = badgeCy;
11304     const traceTri = (): void => {
11305         traceHudSvgDirectionArrow(
11306             g,
11307             triCx,
11308             triCy,
11309             keySize,
11310             Direction.Right,

```

```
11311         ANSWER_VIDEO_BADGE_ARROW_RATIO,
11312     );
11313 };
11314 if (pressed) {
11315     strokeGlowLayers(g, keySize, PRESSED_GLOW_LAYERS, traceTri, true);
11316 }
11317 g.fillColor = ICON_STROKE;
11318 traceTri();
11319 g.fill();
11320 }
11321 /** stage=3 时在键帽右上角叠「不可用」胶片角标 */
11322 function drawUndoDisabledBadge(
11323     g: Graphics,
11324     left: number,
11325     bottom: number,
11326     width: number,
11327     height: number,
11328     pressed: boolean,
11329 ): void {
11330     drawVideoRewardBadgeFrame(g, left, bottom, width, height, pressed, (cx, cy, scale, size) => {
11331         drawUndoDisabledBadgePlus3(g, cx, cy, scale, size);
11332     });
11333 }
11334 /** Answer 键：右上角「看视频」角标（右向三角替代 +3） */
11335 function drawAnswerVideoBadge(
11336     g: Graphics,
11337     left: number,
11338     bottom: number,
11339     width: number,
11340     height: number,
11341     pressed: boolean,
11342 ): void {
11343     drawVideoRewardBadgeFrame(g, left, bottom, width, height, pressed, (cx, cy, scale, size) => {
11344         drawAnswerVideoBadgeTriangle(g, cx, cy, scale, size, pressed);
11345     });
11346 }
11347 /** 绘制撤回 / 重置键：80×80 圆角底 + 内部符号 */
11348 export function drawActionButtonGlyph(
11349     g: Graphics,
11350     left: number,
11351     bottom: number,
11352     width: number,
11353     height: number,
11354     kind: ActionButtonKind,
11355     pressed = false,
11356     undoFillStage = 0,
11357     showAnswerVideoBadge = false,
11358 ): void {
11359     drawHudButtonChrome(g, left, bottom, width, height, pressed);
11360     const size = Math.min(width, height);
11361     const cx = left + width * 0.5;
11362     const cy = bottom + height * 0.5;
11363     if (kind === ActionButtonKind.Reset) {
11364         drawResetIcon(g, cx, cy, size, pressed);
11365         return;
11366     }
11367     if (kind === ActionButtonKind.Answer) {
11368         drawAnswerIcon(g, cx, cy, size, pressed);
```

```

11369         if (showAnswerVideoBadge) {
11370             drawAnswerVideoBadge(g, left, bottom, width, height, pressed);
11371         }
11372         return;
11373     }
11374     if (kind !== ActionButtonKind.Undo) {
11375         return;
11376     }
11377     drawUndolcon(g, cx, cy, size, pressed, undoFillStage);
11378     if (undoFillStage >= UNDO_ICON_FILL_STAGES) {
11379         drawUndoDisabledBadge(g, left, bottom, width, height, pressed);
11380     }
11381 }
11382 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleActionButtonView.ts -----
11383 import {
11384     _decorator,
11385     Button,
11386     Component,
11387     Graphics,
11388     Node,
11389     UITransform,
11390 } from 'cc';
11391 import { UNDO_ICON_FILL_STAGES } from '../Application/OpticalPuzzleSession';
11392 import {
11393     ActionButtonKind,
11394     computeVideoRewardBadgeLayout,
11395     drawActionButtonGlyph,
11396     ACTION_BUTTON_DESIGN_SIZE,
11397 } from './OpticalPuzzleActionButtonGlyph';
11398 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
11399 const { ccclass, property } = _decorator;
11400 /** 激励视频角标独立热区（Answer / Undo 右上角，略溢出 80×80 键帽） */
11401 export const VIDEO_BADGE_HIT_NAME = 'VideoBadgeHit';
11402 /** @deprecated 与 {@link VIDEO_BADGE_HIT_NAME} 相同，保留兼容 */
11403 export const ANSWER_VIDEO_BADGE_HIT_NAME = VIDEO_BADGE_HIT_NAME;
11404 /** 与 Undo/Reset 等 HUD 键一致：Button 缩放按压 */
11405 const ACTION_BUTTON_ZOOM_SCALE = 0.95;
11406 /** 撤回 / 重置虚拟键：键帽风格与四向键一致，内部符号不同 */
11407 @ccclass('OpticalPuzzleActionButtonView')
11408 export class OpticalPuzzleActionButtonView extends Component {
11409     /** 留空则按节点名 BtnUndo / BtnReset / BtnAnswer 推断 */
11410     @property
11411     kind: ActionButtonKind = ActionButtonKind.Undo;
11412     private _graphics: Graphics | null = null;
11413     private _pressed = false;
11414     private _undoFillStage = 0;
11415     /** 未解锁参考解时显示右上角激励视频角标 */
11416     private _showAnswerVideoBadge = true;
11417     private _pressCtrl: HudButtonPressController | null = null;
11418     private _badgeHitNode: Node | null = null;
11419     private _badgePressCtrl: HudButtonPressController | null = null;
11420     protected onLoad(): void {
11421         this.kind = this._resolveKind(this.kind, this.node.name);
11422         this._hidePlaceholderSplash();
11423         this._ensureTransform();
11424         this._ensurePressButton();
11425         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
11426             this._pressed = pressed;

```

```

11427         this._redraw();
11428     });
11429     if (this._supportsVideoBadgeHit()) {
11430         this._ensureVideoBadgeHitNode();
11431     }
11432     this._redraw();
11433 }
11434 protected onEnable(): void {
11435     this._pressCtrl?.bind();
11436     this._badgePressCtrl?.bind();
11437     this._redraw();
11438 }
11439 protected onDisable(): void {
11440     this._pressCtrl?.unbind();
11441     this._badgePressCtrl?.unbind();
11442     this._pressed = false;
11443 }
11444 protected onDestroy(): void {
11445     this._pressCtrl?.unbind();
11446     this._badgePressCtrl?.unbind();
11447 }
11448 refresh(): void {
11449     this._redraw();
11450 }
11451 /** 键盘等非触摸输入：短暂显示按压发光 */
11452 flashPressed(durationSec = 0.1): void {
11453     if (this._pressCtrl?.touchActive) {
11454         return;
11455     }
11456     this._pressed = true;
11457     this._redraw();
11458     this.unschedule(this._releaseKeyboardFlash);
11459     this.scheduleOnce(this._releaseKeyboardFlash, durationSec);
11460 }
11461 private _releaseKeyboardFlash = (): void => {
11462     if (this._pressCtrl?.touchActive) {
11463         return;
11464     }
11465     this._pressed = false;
11466     this._redraw();
11467 };
11468 /** 撤回键横向填充阶段（0 满填，3 空） */
11469 setUndoFillStage(stage: number): void {
11470     if (this.kind !== ActionButtonKind.Undo) {
11471         return;
11472     }
11473     this._undoFillStage = stage;
11474     this._syncVideoBadgeHit();
11475     this._redraw();
11476 }
11477 /** 参考解未解锁时显示看视频角标 */
11478 setAnswerVideoBadgeVisible(visible: boolean): void {
11479     if (this.kind !== ActionButtonKind.Answer) {
11480         return;
11481     }
11482     if (this._showAnswerVideoBadge === visible) {
11483         return;
11484     }

```

```

11485         this._showAnswerVideoBadge = visible;
11486         this._syncVideoBadgeHit();
11487         this._redraw();
11488     }
11489     /** 角标独立 Button 节点 (InputHud 绑定与主键相同逻辑) */
11490     getVideoBadgeHitButton(): Button | null {
11491         if (!this._supportsVideoBadgeHit()) {
11492             return null;
11493         }
11494         return this._badgeHitNode?.getComponent(Button) ?? null;
11495     }
11496     private _supportsVideoBadgeHit(): boolean {
11497         return this.kind === ActionButtonKind.Answer || this.kind === ActionButtonKind.Undo;
11498     }
11499     private _shouldShowVideoBadge(): boolean {
11500         if (this.kind === ActionButtonKind.Answer) {
11501             return this._showAnswerVideoBadge;
11502         }
11503         if (this.kind === ActionButtonKind.Undo) {
11504             return this._undoFillStage >= UNDO_ICON_FILL_STAGES;
11505         }
11506         return false;
11507     }
11508     private _ensureVideoBadgeHitNode(): void {
11509         let hit = this.node.getChildByName(VIDEO_BADGE_HIT_NAME);
11510         if (!hit) {
11511             hit = new Node(VIDEO_BADGE_HIT_NAME);
11512             hit.setParent(this.node);
11513             hit.addComponent(UITransform);
11514             hit.addComponent(Button);
11515             this._badgeHitNode = hit;
11516             this._badgePressCtrl = new HudButtonPressController(
11517                 hit,
11518                 (pressed) => {
11519                     this._pressed = pressed;
11520                     this._redraw();
11521                 },
11522                 true,
11523             );
11524         } else {
11525             this._badgeHitNode = hit;
11526             if (!this._badgePressCtrl) {
11527                 this._badgePressCtrl = new HudButtonPressController(
11528                     hit,
11529                     (pressed) => {
11530                         this._pressed = pressed;
11531                         this._redraw();
11532                     },
11533                     true,
11534                 );
11535             }
11536         }
11537         const badgeBtn = this._badgeHitNode.getComponent(Button);
11538         if (badgeBtn) {
11539             this._configureVideoBadgeHitButton(badgeBtn);
11540         }
11541         this._syncVideoBadgeHit();
11542     }

```

```

11543  /** 角标热区：缩放作用于整颗操作键（与主 Button 一致 0.95） */
11544  private _configureVideoBadgeHitButton(btn: Button): void {
11545      btn.transition = Button.Transition.SCALE;
11546      btn.zoomScale = ACTION_BUTTON_ZOOM_SCALE;
11547      btn.target = this.node;
11548      btn.interactable = true;
11549  }
11550  private _syncVideoBadgeHit(): void {
11551      if (!this._supportsVideoBadgeHit() || !this._badgeHitNode?.isValid) {
11552          return;
11553      }
11554      this._badgeHitNode.active = this._shouldShowVideoBadge();
11555      if (!this._shouldShowVideoBadge()) {
11556          return;
11557      }
11558      const ut = this.getComponent(UITransform);
11559      const hitUt = this._badgeHitNode.getComponent(UITransform);
11560      if (!ut || !hitUt) {
11561          return;
11562      }
11563      const w = ut.width;
11564      const h = ut.height;
11565      const left = -ut.anchorX * w;
11566      const bottom = -ut.anchorY * h;
11567      const layout = computeVideoRewardBadgeLayout(left, bottom, w, h);
11568      hitUt.setContentSize(layout.diameter, layout.diameter);
11569      hitUt.setAnchorPoint(0.5, 0.5);
11570      this._badgeHitNode.setPosition(layout.centerX, layout.centerY, 0);
11571      this._badgeHitNode.setSiblingIndex(this.node.children.length - 1);
11572      const badgeBtn = this._badgeHitNode.getComponent(Button);
11573      if (badgeBtn) {
11574          this._configureVideoBadgeHitButton(badgeBtn);
11575      }
11576  }
11577  private _ensureTransform(): void {
11578      let ut = this.getComponent(UITransform);
11579      if (!ut) {
11580          ut = this.addComponent(UITransform);
11581          ut.setContentSize(ACTION_BUTTON_DESIGN_SIZE, ACTION_BUTTON_DESIGN_SIZE);
11582      }
11583  }
11584  /** 触摸缩放 0.95（与 InputHud / 四向键 Button 一致） */
11585  private _ensurePressButton(): void {
11586      let btn = this.getComponent(Button);
11587      if (!btn) {
11588          btn = this.addComponent(Button);
11589      }
11590      btn.transition = Button.Transition.SCALE;
11591      btn.zoomScale = ACTION_BUTTON_ZOOM_SCALE;
11592  }
11593  private _ensureGraphics(): Graphics | null {
11594      if (!this._graphics?.isValid) {
11595          this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
11596      }
11597      return this._graphics;
11598  }
11599  private _redraw(): void {
11600      const g = this._ensureGraphics();

```



```
11601         const ut = this.getComponent(UITransform);
11602         if (!g || !ut) {
11603             return;
11604         }
11605         g.clear();
11606         const w = ut.width;
11607         const h = ut.height;
11608         const left = -ut.anchorX * w;
11609         const bottom = -ut.anchorY * h;
11610         drawActionButtonGlyph(
11611             g,
11612             left,
11613             bottom,
11614             w,
11615             h,
11616             this.kind,
11617             this._pressed,
11618             this.kind === ActionButtonKind.Undo ? this._undoFillStage : 0,
11619             this.kind === ActionButtonKind.Answer && this._showAnswerVideoBadge,
11620         );
11621         if (this._supportsVideoBadgeHit()) {
11622             this._syncVideoBadgeHit();
11623         }
11624     }
11625     private _resolveKind(fallback: ActionButtonKind, nodeName: string): ActionButtonKind {
11626         const key = nodeName.toLowerCase();
11627         if (key.includes('reset')) {
11628             return ActionButtonKind.Reset;
11629         }
11630         if (key.includes('answer')) {
11631             return ActionButtonKind.Answer;
11632         }
11633         if (key.includes('undo')) {
11634             return ActionButtonKind.Undo;
11635         }
11636         return fallback;
11637     }
11638     /** 占位 SpriteSplash 会盖住根节点 Graphics 矢量图标 */
11639     private _hidePlaceholderSplash(): void {
11640         const splash = this.node.getChildByName('SpriteSplash');
11641         if (splash?.isValid) {
11642             splash.active = false;
11643         }
11644     }
11645 }
11646 /** 为单个 HUD 键节点挂上键帽绘制（与 BtnReset 等一致） */
11647 export function ensureActionButtonView(buttonNode: Node | null): void {
11648     if (!buttonNode?.isValid) {
11649         return;
11650     }
11651     if (!buttonNode.getComponent(OpticalPuzzleActionButtonView)) {
11652         buttonNode.addComponent(OpticalPuzzleActionButtonView);
11653     }
11654 }
11655 /** 为 ActionPad 下 BtnUndo / BtnReset / BtnAnswer 等批量挂上绘制 */
11656 export function ensureActionButtonViews(actionPad: Node | null): void {
11657     if (!actionPad?.isValid) {
11658         return;
```

```

11659     }
11660     for (const child of actionPad.children) {
11661         ensureActionButtonView(child);
11662     }
11663 }
11664 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleAnswerPanel.ts -----
11665 import {
11666     _decorator,
11667     BlockInputEvents,
11668     Button,
11669     Component,
11670     EventTouch,
11671     Graphics,
11672     Node,
11673     Sprite,
11674     UITransform,
11675 } from 'cc';
11676 import { MenuOverlayWindow } from '../MenuOverlayWindow';
11677 import { AUDIO_EFFECT_ENUM, EVENT_ENUM } from '../Utils/Enum';
11678 import { PLAY_AUDIO } from '../Utils/Event';
11679 import {
11680     drawLevelSelectCloseGlyph,
11681     drawLevelSelectPanelChrome,
11682     drawLevelSelectTitleBar,
11683     LEVEL_SELECT_CLOSE_DESIGN_SIZE,
11684 } from './OpticalPuzzleLevelSelectPanelGlyph';
11685 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
11686 import { ensureActionButtonView } from './OpticalPuzzleActionButtonView';
11687 import {
11688     ensureAnswerReplayView,
11689     OpticalPuzzleAnswerReplayView,
11690 } from './OpticalPuzzleAnswerReplayView';
11691 import { ensureAnswerPanelStepViews } from './OpticalPuzzleAnswerPanelStepView';
11692 const { ccclass } = _decorator;
11693 /** 与 Game.scene answerPanel 占位布局一致（设计 px） */
11694 const PANEL_WIDTH = 650;
11695 const PANEL_HEIGHT = 800;
11696 const PANEL_OFFSET_Y = -25;
11697 const TITLE_WIDTH = 180;
11698 const TITLE_HEIGHT = 80;
11699 const TITLE_OFFSET_Y = 375;
11700 const CLOSE_SIZE = LEVEL_SELECT_CLOSE_DESIGN_SIZE;
11701 const CLOSE_OFFSET_X = 315;
11702 const CLOSE_OFFSET_Y = 370;
11703 const GFX_CHILD_NAME = 'gfx';
11704 /**
11705  * 参考解弹层：panel / titlebar / closebtn 绘制与关闭；answer 子节点自动回放 bestSolution；resetbtn 从
11706  * 头重播。
11707  * 挂到 layerOverlay/answerPanel。
11708  */
11709 @ccclass('OpticalPuzzleAnswerPanel')
11710 export class OpticalPuzzleAnswerPanel extends Component {
11711     private _backdropNode: Node | null = null;
11712     private _panelNode: Node | null = null;
11713     private _titleNode: Node | null = null;
11714     private _closeNode: Node | null = null;
11715     private _resetNode: Node | null = null;
11716     private _panelGraphics: Graphics | null = null;

```

```
11717     private _titleGraphics: Graphics | null = null;
11718     private _closeGraphics: Graphics | null = null;
11719     private _closePressed = false;
11720     private _closePressCtrl: HudButtonPressController | null = null;
11721     protected onLoad(): void {
11722         this._disableLegacyOverlayWindow();
11723         this._hidePlaceholderSplashes();
11724         this._ensureRootLayout();
11725         this._buildUiTree();
11726         this._bindCloseButton();
11727         this._bindResetButton();
11728         this._redrawStaticChrome();
11729     }
11730     protected onEnable(): void {
11731         this._closePressCtrl?.bind();
11732     }
11733     protected onDisable(): void {
11734         this._closePressCtrl?.unbind();
11735         this._closePressed = false;
11736     }
11737     protected onDestroy(): void {
11738         this._closePressCtrl?.unbind();
11739         if (this._closeNode?.isValid) {
11740             this._closeNode.off(Button.EventType.CLICK, this._onCloseClick, this);
11741         }
11742         if (this._resetNode?.isValid) {
11743             this._resetNode.off(Button.EventType.CLICK, this._onResetClick, this);
11744         }
11745         if (this._backdropNode?.isValid) {
11746             this._backdropNode.off(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
11747         }
11748     }
11749     /** 关闭参考解面板 */
11750     close(): void {
11751         if (this.node?.isValid) {
11752             this.node.active = false;
11753         }
11754     }
11755     private _disableLegacyOverlayWindow(): void {
11756         const legacy = this.getComponent(MenuOverlayWindow);
11757         if (legacy) {
11758             legacy.destroy();
11759         }
11760     }
11761     private _ensureRootLayout(): void {
11762         if (!this.getComponent(UITransform)) {
11763             const ut = this.addComponent(UITransform);
11764             ut.setAnchorPoint(0.5, 0.5);
11765         }
11766     }
11767     private _buildUiTree(): void {
11768         this._backdropNode = this._ensureNamedChild('bg');
11769         this._ensureBackdrop(this._backdropNode);
11770         this._panelNode = this._ensureNamedChild('panel');
11771         this._ensurePanelBody(this._panelNode);
11772         this._titleNode = this._ensureNamedChild('titlebar');
11773         this._ensureTitleBar(this._titleNode);
11774         this._closeNode = this._ensureNamedChild('closebtn');
```

```

11775         this._ensureCloseButton(this._closeNode);
11776         ensureAnswerReplayView(this._ensureNamedChild('answer'));
11777         this._ensureResetButton(this._ensureNamedChild('resetbtn'));
11778         ensureAnswerPanelStepViews(this.node);
11779     }
11780     private _ensureNamedChild(name: string): Node {
11781         let child = this.node.getChildByName(name);
11782         if (!child) {
11783             child = new Node(name);
11784             child.setParent(this.node);
11785         }
11786         return child;
11787     }
11788     private _applyDefaultNodeLayout(
11789         node: Node,
11790         width: number,
11791         height: number,
11792         x: number,
11793         y: number,
11794         anchorX = 0.5,
11795         anchorY = 0.5,
11796     ): void {
11797         const hadTransform = !!node.getComponent(UITransform);
11798         const ut = node.getComponent(UITransform) ?? node.addComponent(UITransform);
11799         if (hadTransform) {
11800             return;
11801         }
11802         ut.setContentSize(width, height);
11803         ut.setAnchorPoint(anchorX, anchorY);
11804         node.setPosition(x, y, 0);
11805     }
11806     private _ensureBackdrop(node: Node): void {
11807         if (!node.getComponent(UITransform)) {
11808             node.addComponent(UITransform);
11809         }
11810         if (!node.getComponent(BlockInputEvents)) {
11811             node.addComponent(BlockInputEvents);
11812         }
11813         this._removeGfxChild(node);
11814         node.off(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
11815         node.on(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
11816     }
11817     private _ensureTouchBlocker(node: Node): void {
11818         if (!node.getComponent(UITransform)) {
11819             node.addComponent(UITransform);
11820         }
11821         if (!node.getComponent(BlockInputEvents)) {
11822             node.addComponent(BlockInputEvents);
11823         }
11824     }
11825     private _removeGfxChild(host: Node): void {
11826         const gfxNode = host.getChildByName(GFX_CHILD_NAME);
11827         if (gfxNode?.isValid) {
11828             gfxNode.destroy();
11829         }
11830     }
11831     private _disableHostSprite(host: Node): void {
11832         const sprite = host.getComponent(Sprite);

```

```

11833         if (sprite) {
11834             sprite.enabled = false;
11835         }
11836     }
11837     private _ensurePanelBody(node: Node): void {
11838         this._applyDefaultNodeLayout(node, PANEL_WIDTH, PANEL_HEIGHT, 0, PANEL_OFFSET_Y);
11839         this._ensureTouchBlocker(node);
11840         this._removeGfxChild(node);
11841         this._disableHostSprite(node);
11842         this._panelGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
11843     }
11844     private _ensureTitleBar(node: Node): void {
11845         this._applyDefaultNodeLayout(node, TITLE_WIDTH, TITLE_HEIGHT, 0, TITLE_OFFSET_Y);
11846         this._ensureTouchBlocker(node);
11847         this._removeGfxChild(node);
11848         this._disableHostSprite(node);
11849         this._titleGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
11850     }
11851     private _ensureCloseButton(node: Node): void {
11852         this._applyDefaultNodeLayout(node, CLOSE_SIZE, CLOSE_SIZE, CLOSE_OFFSET_X, CLOSE_OFFSET_Y);
11853         this._removeGfxChild(node);
11854         this._disableHostSprite(node);
11855         let btn = node.getComponent(Button);
11856         if (!btn) {
11857             btn = node.addComponent(Button);
11858         }
11859         btn.transition = Button.Transition.SCALE;
11860         btn.zoomScale = 0.95;
11861         this._closeGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
11862         this._closePressCtrl = new HudButtonPressController(
11863             node,
11864             (pressed) => {
11865                 this._closePressed = pressed;
11866                 this._redrawCloseButton();
11867             },
11868             true,
11869         );
11870     }
11871     /** resetbtn: 样式与 ActionPad BtnReset 一致 (OpticalPuzzleActionButtonView) */
11872     private _ensureResetButton(node: Node): void {
11873         this._resetNode = node;
11874         ensureActionButtonView(node);
11875     }
11876     private _bindResetButton(): void {
11877         if (!this._resetNode?.isValid) {
11878             return;
11879         }
11880         this._resetNode.off(Button.EventType.CLICK, this._onResetClick, this);
11881         this._resetNode.on(Button.EventType.CLICK, this._onResetClick, this);
11882     }
11883     private _resolveAnswerReplayView(): OpticalPuzzleAnswerReplayView | null {
11884         const answer = this.node.getChildByName('answer');
11885         return answer?.getComponent(OpticalPuzzleAnswerReplayView) ?? null;
11886     }
11887     private _bindCloseButton(): void {
11888         if (!this._closeNode?.isValid) {
11889             return;
11890         }

```

```

11891         this._closeNode.off(Button.EventType.CLICK, this._onCloseClick, this);
11892         this._closeNode.on(Button.EventType.CLICK, this._onCloseClick, this);
11893     }
11894     /** 隐藏占位 Splash; 保留 answer 子树占位 */
11895     private _hidePlaceholderSplashes(): void {
11896         const walk = (node: Node, skipSubtree: boolean): void => {
11897             if (node.name === 'SpriteSplash' && !skipSubtree) {
11898                 node.active = false;
11899             }
11900             const childSkip = skipSubtree || node.name === 'answer';
11901             for (const child of node.children) {
11902                 walk(child, childSkip);
11903             }
11904         };
11905         for (const child of this.node.children) {
11906             walk(child, false);
11907         }
11908     }
11909     private _redrawStaticChrome(): void {
11910         this._redrawPanelBody();
11911         this._redrawTitleBar();
11912         this._redrawCloseButton();
11913     }
11914     private _redrawPanelBody(): void {
11915         const g = this._panelGraphics;
11916         const ut = this._panelNode?.getComponent(UITransform);
11917         if (!g?.isValid || !ut) {
11918             return;
11919         }
11920         g.clear();
11921         const w = ut.width;
11922         const h = ut.height;
11923         drawLevelSelectPanelChrome(g, -w * 0.5, -h * 0.5, w, h);
11924     }
11925     private _redrawTitleBar(): void {
11926         const g = this._titleGraphics;
11927         const ut = this._titleNode?.getComponent(UITransform);
11928         if (!g?.isValid || !ut) {
11929             return;
11930         }
11931         g.clear();
11932         const w = ut.width;
11933         const h = ut.height;
11934         drawLevelSelectTitleBar(g, -w * 0.5, -h * 0.5, w, h);
11935     }
11936     private _redrawCloseButton(): void {
11937         const g = this._closeGraphics;
11938         const ut = this._closeNode?.getComponent(UITransform);
11939         if (!g?.isValid || !ut) {
11940             return;
11941         }
11942         g.clear();
11943         const w = ut.width;
11944         const h = ut.height;
11945         drawLevelSelectCloseGlyph(g, -w * 0.5, -h * 0.5, w, h, this._closePressed);
11946     }
11947     private _onCloseClick(): void {
11948         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);

```

```

11949         this.close();
11950     }
11951     private _onResetClick(): void {
11952         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
11953         this._resolveAnswerReplayView()?.restartFromBeginning();
11954     }
11955     /** 仅点击 bg 空白遮罩时关闭 */
11956     private _onBackdropTouchEnd(event: EventTouch): void {
11957         if (event.target !== this._backdropNode) {
11958             return;
11959         }
11960         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
11961         this.close();
11962     }
11963 }
11964 /** 为 answerPanel 根节点挂上绘制与关闭逻辑 */
11965 export function ensureAnswerPanel(panelNode: Node | null): void {
11966     if (!panelNode?.isValid) {
11967         return;
11968     }
11969     const legacy = panelNode.getComponent(MenuOverlayWindow);
11970     if (legacy) {
11971         legacy.destroy();
11972     }
11973     if (!panelNode.getComponent(OpticalPuzzleAnswerPanel)) {
11974         panelNode.addComponent(OpticalPuzzleAnswerPanel);
11975     }
11976     ensureAnswerPanelStepViews(panelNode);
11977 }
11978 /** 打开参考解面板 */
11979 export function openAnswerPanel(panelNode: Node | null): void {
11980     if (!panelNode?.isValid) {
11981         return;
11982     }
11983     ensureAnswerPanel(panelNode);
11984     panelNode.active = true;
11985 }
11986 /** 自 GameRoot 解析 layerOverlay 下参考解面板 */
11987 export function resolveAnswerPanelNode(root: Node | null): Node | null {
11988     const overlay = root?.getChildByName('layerOverlay') ?? null;
11989     if (!overlay?.isValid) {
11990         return null;
11991     }
11992     return (
11993         overlay.getChildByName('answerPanel') ??
11994         overlay.getChildByName('AnswerPanel') ??
11995         null
11996     );
11997 }
11998 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleAnswerPanelStepView.ts -----
11999 import { Graphics, Node } from 'cc';
12000 import {
12001     ensureStepCountView,
12002     OpticalPuzzleStepCountView,
12003     resolveStepCountNode,
12004 } from './OpticalPuzzleStepCountView';
12005 import { ensureStepIconView } from './OpticalPuzzleStepIconView';
12006 import { OpticalPuzzleStepView } from './OpticalPuzzleStepView';

```

```

12007 function _hideSpriteSplash(node: Node | null): void {
12008     const splash = node?.getChildByName('SpriteSplash');
12009     if (splash?.isValid) {
12010         splash.active = false;
12011     }
12012 }
12013 /** 自 answerPanel 子树解析 Step 节点 */
12014 export function resolveAnswerPanelStepNode(panel: Node | null): Node | null {
12015     return panel?.getChildByName('Step') ?? null;
12016 }
12017 /** 解析 answerPanel 下 StepCount 的步数视图 */
12018 export function resolveAnswerPanelStepCountView(panel: Node | null): OpticalPuzzleStepCountView | null {
12019     const stepCount = resolveStepCountNode(panel);
12020     return stepCount?.getComponent(OpticalPuzzleStepCountView) ?? null;
12021 }
12022 function _removeStepChrome(step: Node): void {
12023     const chrome = step.getComponent(OpticalPuzzleStepView);
12024     if (chrome) {
12025         chrome.destroy();
12026     }
12027     const g = step.getComponent(Graphics);
12028     if (g) {
12029         g.clear();
12030         g.destroy();
12031     }
12032 }
12033 /**
12034  * answerPanel/Step: 不绘制外框, 仅挂 StepIcon 爪印与 StepCount 步数。
12035  * 步数由参考解回放手动刷新, 不跟随局内快照。
12036  */
12037 export function ensureAnswerPanelStepViews(panel: Node | null): void {
12038     if (!panel?.isValid) {
12039         return;
12040     }
12041     const step = resolveAnswerPanelStepNode(panel);
12042     if (!step?.isValid) {
12043         return;
12044     }
12045     _hideSpriteSplash(step);
12046     _removeStepChrome(step);
12047     const stepIcon = step.getChildByName('StepIcon');
12048     _hideSpriteSplash(stepIcon);
12049     ensureStepIconView(stepIcon);
12050     const stepCount = resolveStepCountNode(panel);
12051     _hideSpriteSplash(stepCount);
12052     ensureStepCountView(stepCount, { manualOnly: true });
12053 }
12054 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleAnswerReplayView.ts -----
12055 import { _decorator, Component, Node, Sprite, UITransform } from 'cc';
12056 import { DataManager } from '../Manager/DataManager';
12057 import { getOpticalLevelById } from '../Config/OpticalPuzzleLevels';
12058 import type { IOpticalLevelConfig } from '../Config/OpticalPuzzleLevelSchema';
12059 import { DEV_LEVEL_MINIMAL } from '../Config/OpticalPuzzleLevelSchema';
12060 import { OpticalPuzzleCore } from '../Core/OpticalPuzzleCore';
12061 import {
12062     Direction,
12063     MoveAttemptResult,
12064     normalizeDirection,

```



```

12065 } from '../Core/OpticalPuzzleTypes';
12066 import type { OpticalSessionNotifyReason } from '../Application/OpticalPuzzleSession';
12067 import { OpticalPuzzleBeamView } from './OpticalPuzzleBeamView';
12068 import { OpticalPuzzleBoardView } from './OpticalPuzzleBoardView';
12069 import { resolveAnswerPanelStepCountView } from './OpticalPuzzleAnswerPanelStepView';
12070 import {
12071     boardPixelHeight,
12072     boardPixelWidth,
12073     computePlayLayerScale,
12074     OPTICAL_CELL_SIZE,
12075 } from './OpticalPuzzleLayout';
12076 const { ccclass } = _decorator;
12077 const REPLAY_ROOT_NAME = 'replayRoot';
12078 const BOARD_LAYER_NAME = 'BoardLayer';
12079 const BEAM_LAYER_NAME = 'BeamLayer';
12080 const ANSWER_BOARD_PADDING = 16;
12081 const STEP_INTERVAL_SEC = 0.5;
12082 /** 参考解回放：按 bestSolution 逐步演示，结束后停留在通关解法 */
12083 @ccclass('OpticalPuzzleAnswerReplayView')
12084 export class OpticalPuzzleAnswerReplayView extends Component {
12085     private readonly _core = new OpticalPuzzleCore();
12086     private _replayRoot: Node | null = null;
12087     private _boardView: OpticalPuzzleBoardView | null = null;
12088     private _beamView: OpticalPuzzleBeamView | null = null;
12089     private _level: IOpticalLevelConfig | null = null;
12090     private _solution: readonly string[] = [];
12091     private _stepIndex = 0;
12092     private _elapsed = 0;
12093     private _running = false;
12094     /** 参考解回放已演示的有效移动步数 */
12095     private _replayMoveCount = 0;
12096     /** 从初始局面重新播放参考解（answerPanel resetbtn） */
12097     restartFromBeginning(): void {
12098         if (!this._level) {
12099             this._startReplay();
12100             return;
12101         }
12102         this._restartFromInitial();
12103         this._running = this._solution.length > 0;
12104     }
12105     protected onLoad(): void {
12106         this._hidePlaceholderSplash();
12107         this._ensureReplayLayers();
12108     }
12109     protected onEnable(): void {
12110         this._startReplay();
12111     }
12112     protected onDisable(): void {
12113         this._stopReplay();
12114     }
12115     protected update(dt: number): void {
12116         if (!this._running || !this._level) {
12117             return;
12118         }
12119         this._elapsed += dt;
12120         if (this._elapsed < STEP_INTERVAL_SEC) {
12121             return;
12122         }

```

```

12123         this._elapsed = 0;
12124         if (this._stepIndex >= this._solution.length) {
12125             this._running = false;
12126             return;
12127         }
12128         const result = this._applySolutionStep(this._solution[this._stepIndex]);
12129         this._stepIndex += 1;
12130         if (result === MoveAttemptResult.PlayerMoved || result === MoveAttemptResult.PiecePushed) {
12131             this._replayMoveCount += 1;
12132             this._syncReplayStepCount();
12133         }
12134         this._renderSnapshot(this._notifyReasonForMove(result));
12135         if (this._stepIndex >= this._solution.length) {
12136             this._running = false;
12137         }
12138     }
12139     private _hidePlaceholderSplash(): void {
12140         const splash = this.node.getChildByName('SpriteSplash');
12141         if (splash?.isValid) {
12142             splash.active = false;
12143         }
12144         const sprite = this.node.getComponent(Sprite);
12145         if (sprite) {
12146             sprite.enabled = false;
12147         }
12148     }
12149     private _ensureReplayLayers(): void {
12150         let root = this.node.getChildByName(REPLAY_ROOT_NAME);
12151         if (!root) {
12152             root = new Node(REPLAY_ROOT_NAME);
12153             root.setParent(this.node);
12154         }
12155         this._replayRoot = root;
12156         let boardNode = root.getChildByName(BOARD_LAYER_NAME);
12157         if (!boardNode) {
12158             boardNode = new Node(BOARD_LAYER_NAME);
12159             boardNode.setParent(root);
12160             boardNode.addComponent(UITransform);
12161             boardNode.addComponent(OpticalPuzzleBoardView);
12162         }
12163         this._boardView =
12164             boardNode.getComponent(OpticalPuzzleBoardView) ??
12165             boardNode.addComponent(OpticalPuzzleBoardView);
12166         let beamNode = root.getChildByName(BEAM_LAYER_NAME);
12167         if (!beamNode) {
12168             beamNode = new Node(BEAM_LAYER_NAME);
12169             beamNode.setParent(root);
12170             beamNode.addComponent(UITransform);
12171             beamNode.addComponent(OpticalPuzzleBeamView);
12172         }
12173         this._beamView =
12174             beamNode.getComponent(OpticalPuzzleBeamView) ??
12175             beamNode.addComponent(OpticalPuzzleBeamView);
12176     }
12177     private _resolveCurrentLevel(): IOpticalLevelConfig {
12178         const levelId = DataManager.instance.opticalCurrentLevelId;
12179         return getOpticalLevelById(levelId) ?? DEV_LEVEL_MINIMAL;
12180     }

```

```

12181     private _applyBoardFit(level: IOpticalLevelConfig): void {
12182         const hostUt = this.node.getComponent(UITransform);
12183         const root = this._replayRoot;
12184         if (!hostUt || !root?.isValid) {
12185             return;
12186         }
12187         const boardW = boardPixelWidth(level.width, OPTICAL_CELL_SIZE);
12188         const boardH = boardPixelHeight(level.height, OPTICAL_CELL_SIZE);
12189         const availW = Math.max(1, hostUt.width - ANSWER_BOARD_PADDING * 2);
12190         const scale = computePlayLayerScale(level.width, level.height, availW);
12191         root.setScale(scale, scale, 1);
12192         root.setPosition(0, 0, 0);
12193         const boardUt =
12194             root.getChildByName(BOARD_LAYER_NAME)?.getComponent(UITransform) ??
12195             null;
12196         boardUt?.setContentSize(boardW, boardH);
12197         const beamUt = root.getChildByName(BEAM_LAYER_NAME)?.getComponent(UITransform) ?? null;
12198         beamUt?.setContentSize(boardW, boardH);
12199     }
12200     private _startReplay(): void {
12201         this._level = this._resolveCurrentLevel();
12202         this._solution = this._level.bestSolution?.length ? this._level.bestSolution : [];
12203         this._applyBoardFit(this._level);
12204         this._restartFromInitial();
12205         this._running = this._solution.length > 0;
12206     }
12207     private _stopReplay(): void {
12208         this._running = false;
12209         this._elapsed = 0;
12210         this._stepIndex = 0;
12211         this._replayMoveCount = 0;
12212     }
12213     private _syncReplayStepCount(): void {
12214         resolveAnswerPanelStepCountView(this.node.parent)?.setMoveCount(this._replayMoveCount);
12215     }
12216     private _restartFromInitial(): void {
12217         if (!this._level) {
12218             return;
12219         }
12220         this._core.reset(this._level);
12221         this._boardView?.resetPlayerEyeIdle();
12222         this._stepIndex = 0;
12223         this._elapsed = 0;
12224         this._replayMoveCount = 0;
12225         this._syncReplayStepCount();
12226         this._renderSnapshot('reset');
12227     }
12228     private _applySolutionStep(step: string): MoveAttemptResult {
12229         const dir = normalizeDirection(step, Direction.Down);
12230         this._core.setPlayerFacing(dir);
12231         return this._core.tryMove(dir);
12232     }
12233     private _notifyReasonForMove(result: MoveAttemptResult): OpticalSessionNotifyReason {
12234         if (result === MoveAttemptResult.PiecePushed) {
12235             return 'push';
12236         }
12237         if (result === MoveAttemptResult.PlayerMoved) {
12238             return 'move';

```

```

12239     }
12240     return 'load';
12241 }
12242 private _renderSnapshot(reason: OpticalSessionNotifyReason = 'load'): void {
12243     const snap = this._core.getSnapshot();
12244     const beam = this._core.getBeamSnapshot();
12245     if (reason === 'move' || reason === 'push') {
12246         this._boardView?.notifyPlayerDirectionInput();
12247     }
12248     this._boardView?.render(snap);
12249     this._beamView?.render(beam);
12250     this._boardView?.renderTargetsOverlay(snap);
12251     this._boardView?.syncPlaySnapshot(snap, reason);
12252 }
12253 }
12254 /** 为 answer 节点挂上参考解回放 */
12255 export function ensureAnswerReplayView(answerNode: Node | null): void {
12256     if (!answerNode?.isValid) {
12257         return;
12258     }
12259     if (!answerNode.getComponent(OpticalPuzzleAnswerReplayView)) {
12260         answerNode.addComponent(OpticalPuzzleAnswerReplayView);
12261     }
12262 }
12263 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleBackButtonGlyph.ts -----
12264 import { Graphics } from 'cc';
12265 import {
12266     drawHudShapelcon,
12267     HUD_BUTTON_DESIGN_SIZE,
12268 } from './OpticalPuzzleHudButtonCommon';
12269 /** 图标占按钮比例（四向键 0.42；返回键无键帽，略放大） */
12270 const ICON_SIZE_RATIO = 0.58;
12271 /** 返回箭头路径半宽（viewBox 1024×1024，iconR=10） */
12272 const BACK_PATH_UNIT = 10.198;
12273 type PathSeg =
12274     | { t: 'M'; x: number; y: number }
12275     | { t: 'L'; x: number; y: number }
12276     | { t: 'C'; x1: number; y1: number; x2: number; y2: number; x: number; y: number }
12277     | { t: 'Z' };
12278 /** 返回箭头（用户 SVG，iconR=10；无键帽，仅面型图标） */
12279 const BACK_ICON_SEGS: ReadonlyArray<PathSeg> = [
12280     { t: 'M', x: 5.497, y: -6.585 },
12281     { t: 'L', x: -1.166, y: 0.036 },
12282     { t: 'L', x: 5.455, y: 6.575 },
12283     { t: 'C', x1: 6.247, y1: 7.366, x2: 6.247, y2: 8.615, x: 5.455, y: 9.407 },
12284     { t: 'C', x1: 4.664, y1: 10.198, x2: 3.373, y2: 10.198, x: 2.582, y: 9.407 },
12285     { t: 'L', x: -5.289, y: 1.661 },
12286     { t: 'C', x1: -5.372, y1: 1.619, x2: -5.455, y2: 1.536, x: -5.539, y: 1.494 },
12287     { t: 'C', x1: -5.955, y1: 1.119, x2: -6.122, y2: 0.578, x: -6.122, y: 0.078 },
12288     { t: 'C', x1: -6.122, y1: -0.422, x2: -5.914, y2: -0.963, x: -5.539, y: -1.338 },
12289     { t: 'C', x1: -5.455, y1: -1.421, x2: -5.372, y2: -1.463, x: -5.289, y: -1.504 },
12290     { t: 'L', x: 2.624, y: -9.334 },
12291     { t: 'C', x1: 3.415, y1: -10.125, x2: 4.706, y2: -10.125, x: 5.497, y: -9.334 },
12292     { t: 'C', x1: 6.33, y1: -8.626, x2: 6.33, y2: -7.376, x: 5.497, y: -6.585 },
12293     { t: 'Z' },
12294 ];
12295 function tracelconSegs(
12296     g: Graphics,

```

```

12297     segs: ReadonlyArray<PathSeg>,
12298     cx: number,
12299     cy: number,
12300     scale: number,
12301 ): void {
12302     for (const seg of segs) {
12303         switch (seg.t) {
12304             case 'M':
12305                 g.moveTo(cx + seg.x * scale, cy + seg.y * scale);
12306                 break;
12307             case 'L':
12308                 g.lineTo(cx + seg.x * scale, cy + seg.y * scale);
12309                 break;
12310             case 'C':
12311                 g.bezierCurveTo(
12312                     cx + seg.x1 * scale,
12313                     cy + seg.y1 * scale,
12314                     cx + seg.x2 * scale,
12315                     cy + seg.y2 * scale,
12316                     cx + seg.x * scale,
12317                     cy + seg.y * scale,
12318                 );
12319                 break;
12320             case 'Z':
12321                 g.close();
12322                 break;
12323             default:
12324                 break;
12325         }
12326     }
12327 }
12328 function traceBackIcon(g: Graphics, cx: number, cy: number, scale: number): void {
12329     traceIconSegs(g, BACK_ICON_SEGS, cx, cy, scale);
12330 }
12331 /**
12332  * 绘制返回键：无键帽外框，仅箭头图标（描边/填色/按下光晕与四向键内芯一致）。
12333  */
12334 export function drawBackButtonGlyph(
12335     g: Graphics,
12336     left: number,
12337     bottom: number,
12338     width: number,
12339     height: number,
12340     pressed = false,
12341 ): void {
12342     const size = Math.min(width, height);
12343     const cx = left + width * 0.5;
12344     const cy = bottom + height * 0.5;
12345     const iconHalf = size * ICON_SIZE_RATIO * 0.5;
12346     const scale = iconHalf / BACK_PATH_UNIT;
12347     const refSize = size > 0 ? size : HUD_BUTTON_DESIGN_SIZE;
12348     drawHudShapelcon(g, refSize, pressed, () => {
12349         traceBackIcon(g, cx, cy, scale);
12350     });
12351 }
12352 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleBackButtonView.ts -----
12353 import {
12354     _decorator,

```

```

12355     Button,
12356     Component,
12357     Graphics,
12358     Node,
12359     UITransform,
12360 } from 'cc';
12361 import { drawBackButtonGlyph } from './OpticalPuzzleBackButtonGlyph';
12362 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
12363 const { ccclass } = _decorator;
12364 /** 与四向键 Button.zoomScale 一致 */
12365 const BACK_BUTTON_ZOOM_SCALE = 0.95;
12366 /** TopBar 返回键：仅绘制图标，按下缩放与四向键一致 */
12367 @ccclass('OpticalPuzzleBackButtonView')
12368 export class OpticalPuzzleBackButtonView extends Component {
12369     private _graphics: Graphics | null = null;
12370     private _pressed = false;
12371     private _pressCtrl: HudButtonPressController | null = null;
12372     protected onLoad(): void {
12373         this._ensureButton();
12374         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
12375             this._pressed = pressed;
12376             this._redraw();
12377         });
12378         this._redraw();
12379     }
12380     protected onEnable(): void {
12381         this._pressCtrl?.bind();
12382         this._redraw();
12383     }
12384     protected onDisable(): void {
12385         this._pressCtrl?.unbind();
12386         this._pressed = false;
12387     }
12388     protected onDestroy(): void {
12389         this._pressCtrl?.unbind();
12390     }
12391     refresh(): void {
12392         this._redraw();
12393     }
12394     private _ensureButton(): void {
12395         let btn = this.getComponent(Button);
12396         if (!btn) {
12397             btn = this.addComponent(Button);
12398             btn.transition = Button.Transition.SCALE;
12399         }
12400         btn.zoomScale = BACK_BUTTON_ZOOM_SCALE;
12401     }
12402     private _ensureGraphics(): Graphics | null {
12403         if (!this._graphics?.isValid) {
12404             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
12405         }
12406         return this._graphics;
12407     }
12408     private _redraw(): void {
12409         const g = this._ensureGraphics();
12410         const ut = this.getComponent(UITransform);
12411         if (!g || !ut) {
12412             return;

```

```

12413     }
12414     g.clear();
12415     const w = ut.width;
12416     const h = ut.height;
12417     const left = -ut.anchorX * w;
12418     const bottom = -ut.anchorY * h;
12419     drawBackButtonGlyph(g, left, bottom, w, h, this._pressed);
12420 }
12421 }
12422 /** 为 TopBar/BtnBackMenu 挂上绘制与按压缩放 */
12423 export function ensureBackButtonView(btnBackMenu: Node | null): void {
12424     if (!btnBackMenu?.isValid) {
12425         return;
12426     }
12427     if (!btnBackMenu.getComponent(OpticalPuzzleBackButtonView)) {
12428         btnBackMenu.addComponent(OpticalPuzzleBackButtonView);
12429     }
12430 }
12431 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleBeamGradient.ts -----
12432 import { Color, Graphics } from 'cc';
12433 import {
12434     BEAM_CORE_WIDTH_RATIO,
12435     BEAM_GRADIENT_STEPS,
12436     BEAM_LINE_WIDTH,
12437     OPTICAL_CELL_SIZE,
12438 } from './OpticalPuzzleLayout';
12439 export function lerpBeamColor(from: Color, to: Color, t: number): Color {
12440     return new Color(
12441         from.r + (to.r - from.r) * t,
12442         from.g + (to.g - from.g) * t,
12443         from.b + (to.b - from.b) * t,
12444         from.a + (to.a - from.a) * t,
12445     );
12446 }
12447 export function coreWhiteFor(beamColor: Color): Color {
12448     return new Color(255, 255, 255, Math.min(255, beamColor.a + 15));
12449 }
12450 /** 与 strokeBeamSegment 第 t 层对应的截面半径 */
12451 export function beamCrossSectionRadius(t: number, cellSize: number = OPTICAL_CELL_SIZE): number {
12452     const scale = cellSize / OPTICAL_CELL_SIZE;
12453     return (BEAM_LINE_WIDTH / 2) * scale * (BEAM_CORE_WIDTH_RATIO + (1 - BEAM_CORE_WIDTH_RATIO) *
12454 t);
12455 }
12456 /**
12457  * 炮口圆盘渐变：由外向内白芯→本色，层数/宽度与光路 strokeBeamSegment 一致。
12458  */
12459 export function fillBeamCrossSection(
12460     g: Graphics,
12461     x: number,
12462     y: number,
12463     beamColor: Color,
12464     cellSize: number = OPTICAL_CELL_SIZE,
12465 ): void {
12466     const coreWhite = coreWhiteFor(beamColor);
12467     const steps = BEAM_GRADIENT_STEPS;
12468     for (let i = steps - 1; i >= 0; i--) {
12469         const t = steps <= 1 ? 1 : i / (steps - 1);
12470         g.fillColor = lerpBeamColor(coreWhite, beamColor, t);

```

```

12471         g.circle(x, y, beamCrossSectionRadius(t, cellSize));
12472         g.fill();
12473     }
12474 }
12475 /** 光路线段渐变描边（与 fillBeamCrossSection 共用同一套 t / 色插值） */
12476 export function strokeBeamSegmentGradient(
12477     g: Graphics,
12478     x0: number,
12479     y0: number,
12480     x1: number,
12481     y1: number,
12482     beamColor: Color,
12483 ): void {
12484     const coreWhite = coreWhiteFor(beamColor);
12485     const steps = BEAM_GRADIENT_STEPS;
12486     const coreRatio = BEAM_CORE_WIDTH_RATIO;
12487     g.lineCap = Graphics.LineCap.BUTT;
12488     g.lineJoin = Graphics.LineJoin.ROUND;
12489     for (let i = steps - 1; i >= 0; i--) {
12490         const t = steps <= 1 ? 1 : i / (steps - 1);
12491         g.lineWidth = BEAM_LINE_WIDTH * (coreRatio + (1 - coreRatio) * t);
12492         g.strokeColor = lerpBeamColor(coreWhite, beamColor, t);
12493         g.moveTo(x0, y0);
12494         g.lineTo(x1, y1);
12495         g.stroke();
12496     }
12497 }
12498 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleBeamImpactView.ts -----
12499 import {
12500     _decorator,
12501     assetManager,
12502     Color,
12503     Component,
12504     ImageAsset,
12505     Node,
12506     ParticleSystem2D,
12507     Rect,
12508     resources,
12509     Size,
12510     SpriteFrame,
12511     Texture2D,
12512     UITransform,
12513     Vec2,
12514     sys,
12515 } from 'cc';
12516 import type { OpticalBeamBlockContact } from '../Core/OpticalBeamTypes';
12517 import { resolveBlockContactColorFromSegments } from '../Core/OpticalBeamContactResolve';
12518 import { coerceBeamColorKey } from '../Core/OpticalColorMix';
12519 import type { OpticalBeamSnapshot } from '../Core/OpticalPuzzleCore';
12520 import { resolveBeamColorKey } from '../Core/OpticalLightColor';
12521 import { Direction, normalizeDirection } from '../Core/OpticalPuzzleTypes';
12522 import { BEAM_SPARK_RESOURCE_PATHS, BEAM_SPARK_SPRITE_UUIDS } from
12523 './OpticalPuzzleBeamSparkSpriteUuids';
12524 import { beamColorFromKey } from './OpticalPuzzleColorUtil';
12525 import { OPTICAL_CELL_SIZE } from './OpticalPuzzleLayout';
12526 const { ccclass, property } = _decorator;
12527 const CELL = OPTICAL_CELL_SIZE;
12528 const MAX_IMPACT_EMITTERS = 48;

```



```

12529 const CONTACT_BACK_OFFSET = 0.06;
12530 const GRID_DX: ReadonlyArray<number> = [1, 0, -1, 0];
12531 const GRID_DY: ReadonlyArray<number> = [0, -1, 0, 1];
12532 let _whiteDotSprite: SpriteFrame | null = null;
12533 const _sparkSpriteCache = new Map<string, SpriteFrame>();
12534 let _sparkSpritesLoading = false;
12535 let _sparkSpritesReady = false;
12536 const _sparkLoadWaiters: Array<() => void> = [];
12537 function normalizeImpactColorKey(raw: unknown): string {
12538     return coerceBeamColorKey(raw);
12539 }
12540 function parseSparkColorKeyFromSpriteFrame(sf: SpriteFrame): string | null {
12541     const candidates = [sf.name, sf.texture?.name];
12542     for (const raw of candidates) {
12543         if (!raw) {
12544             continue;
12545         }
12546         const match = /beam_spark_(\w+)/i.exec(String(raw));
12547         if (match?.[1]) {
12548             return resolveBeamColorKey(match[1]);
12549         }
12550     }
12551     return null;
12552 }
12553 function registerSparkSpriteFrames(frames: ReadonlyArray<SpriteFrame | null | undefined>): boolean {
12554     let added = false;
12555     for (const sf of frames) {
12556         if (!sf?.isValid) {
12557             continue;
12558         }
12559         const colorKey = parseSparkColorKeyFromSpriteFrame(sf);
12560         if (!colorKey) {
12561             continue;
12562         }
12563         _sparkSpriteCache.set(colorKey, sf);
12564         added = true;
12565     }
12566     return added;
12567 }
12568 function isValidSpriteFrame(asset: unknown): asset is SpriteFrame {
12569     return !!asset && typeof asset === 'object' && (asset as SpriteFrame).isValid === true;
12570 }
12571 function tryGetSparkSpriteByUuid(uuid: string): SpriteFrame | null {
12572     const cached = assetManager.assets.get(uuid);
12573     if (isValidSpriteFrame(cached)) {
12574         return cached;
12575     }
12576     const mainBundle = assetManager.bundles.get('main');
12577     const fromMain = mainBundle?.get(uuid) ?? null;
12578     return isValidSpriteFrame(fromMain) ? fromMain : null;
12579 }
12580 function loadSparkSpriteAsync(colorKey: string, onDone: () => void): void {
12581     const resourcePath = BEAM_SPARK_RESOURCE_PATHS[colorKey];
12582     if (resourcePath) {
12583         resources.load(resourcePath, (_err, asset) => {
12584             if (isValidSpriteFrame(asset)) {
12585                 _sparkSpriteCache.set(colorKey, asset);
12586                 onDone();

```

```

12587         return;
12588     }
12589     loadSparkSpriteByUuidAsync(colorKey, onDone);
12590 });
12591     return;
12592 }
12593     loadSparkSpriteByUuidAsync(colorKey, onDone);
12594 }
12595 function loadSparkSpriteByUuidAsync(colorKey: string, onDone: () => void): void {
12596     const uuid = BEAM_SPARK_SPRITE_UUIDS[colorKey];
12597     if (!uuid) {
12598         onDone();
12599         return;
12600     }
12601     assetManager.loadAny({ uuid }, (_err, asset) => {
12602         if (isValidSpriteFrame(asset)) {
12603             _sparkSpriteCache.set(colorKey, asset);
12604         }
12605         onDone();
12606     });
12607 }
12608 function ensureSparkSpriteFramesLoaded(onReady?: () => void): void {
12609     if (_sparkSpritesReady) {
12610         onReady?.();
12611         return;
12612     }
12613     if (onReady) {
12614         _sparkLoadWaiters.push(onReady);
12615     }
12616     if (_sparkSpritesLoading) {
12617         return;
12618     }
12619     _sparkSpritesLoading = true;
12620     const colorKeys = Object.keys(BEAM_SPARK_SPRITE_UUIDS);
12621     let pending = 0;
12622     for (const colorKey of colorKeys) {
12623         if (_sparkSpriteCache.has(colorKey)) {
12624             continue;
12625         }
12626         const uuid = BEAM_SPARK_SPRITE_UUIDS[colorKey];
12627         const sync = tryGetSparkSpriteByUuid(uuid);
12628         if (sync) {
12629             _sparkSpriteCache.set(colorKey, sync);
12630             continue;
12631         }
12632         pending += 1;
12633         loadSparkSpriteAsync(colorKey, () => {
12634             pending -= 1;
12635             if (pending <= 0) {
12636                 _finishSparkSpriteLoad();
12637             }
12638         });
12639     }
12640     if (pending === 0) {
12641         _finishSparkSpriteLoad();
12642     }
12643 }
12644 function _finishSparkSpriteLoad(): void {

```

```

12645     _sparkSpritesReady = true;
12646     _sparkSpritesLoading = false;
12647     const waiters = _sparkLoadWaiters.splice(0, _sparkLoadWaiters.length);
12648     for (const cb of waiters) {
12649         cb();
12650     }
12651 }
12652 function getRuntimeWhiteDotSpriteFrame(): SpriteFrame {
12653     if (_whiteDotSprite?.isValid) {
12654         return _whiteDotSprite;
12655     }
12656     const imageAsset = new ImageAsset();
12657     imageAsset.reset({
12658         _data: new Uint8Array([255, 255, 255, 255]),
12659         width: 1,
12660         height: 1,
12661         format: Texture2D.PixelFormat.RGBA8888,
12662         _compressed: false,
12663     });
12664     const texture = new Texture2D();
12665     texture.image = imageAsset;
12666     const spriteFrame = new SpriteFrame();
12667     spriteFrame.texture = texture;
12668     spriteFrame.rect = new Rect(0, 0, 1, 1);
12669     spriteFrame.originalSize = new Size(1, 1);
12670     _whiteDotSprite = spriteFrame;
12671     return spriteFrame;
12672 }
12673 function getSparkSpriteFrame(colorKey: unknown): SpriteFrame | null {
12674     const key = normalizeImpactColorKey(colorKey);
12675     const cached = _sparkSpriteCache.get(key);
12676     return cached?.isValid ? cached : null;
12677 }
12678 function getParticleSpriteFrame(colorKey: unknown): SpriteFrame {
12679     return getSparkSpriteFrame(colorKey) ?? getRuntimeWhiteDotSpriteFrame();
12680 }
12681 function hasBakedSparkSprite(colorKey: unknown): boolean {
12682     return !!getSparkSpriteFrame(colorKey);
12683 }
12684 function contactPoolKey(contact: OpticalBeamBlockContact, colorKey: string): string {
12685     return `${Math.round(contact.x * 2000)},${Math.round(contact.y * 2000)},${contact.dir},${colorKey}`;
12686 }
12687 function isWeChatMiniGame(): boolean {
12688     return sys.platform === sys.Platform.WECHAT_GAME;
12689 }
12690 function beamScreenAngle(dir: Direction): number {
12691     const dx = GRID_DX[dir] ?? 0;
12692     const dy = -(GRID_DY[dir] ?? 0);
12693     return (Math.atan2(dy, dx) * 180) / Math.PI;
12694 }
12695 function beamImpactEmissionAngle(dir: Direction): number {
12696     return beamScreenAngle(dir) + 180;
12697 }
12698 function contactPosVar(dir: Direction): Vec2 {
12699     switch (dir) {
12700         case Direction.Right:
12701         case Direction.Left:
12702             return new Vec2(2, 10);

```

```

12703         case Direction.Up:
12704         case Direction.Down:
12705             return new Vec2(10, 2);
12706         default:
12707             return new Vec2(6, 6);
12708     }
12709 }
12710 function contactScreenPosition(
12711     contact: OpticalBeamBlockContact,
12712     ox: number,
12713     oy: number,
12714 ): { px: number; py: number } {
12715     const dir = normalizeDirection(contact.dir, Direction.Right);
12716     const gx = contact.x - GRID_DX[dir] * CONTACT_BACK_OFFSET;
12717     const gy = contact.y - GRID_DY[dir] * CONTACT_BACK_OFFSET;
12718     return {
12719         px: ox + gx * CELL,
12720         py: oy - gy * CELL,
12721     };
12722 }
12723 function applyBeamColorToParticle(ps: ParticleSystem2D, colorKey: unknown): void {
12724     const resolvedKey = normalizeImpactColorKey(colorKey);
12725     const beam = beamColorFromKey(resolvedKey);
12726     const baked = hasBakedSparkSprite(colorKey);
12727     ps.spriteFrame = getParticleSpriteFrame(colorKey);
12728     if (isWeChatMiniGame()) {
12729         ps.startColor = new Color(beam.r, beam.g, beam.b, Math.min(220, beam.a));
12730         ps.endColor = new Color(beam.r, beam.g, beam.b, 0);
12731         ps.color = new Color(beam.r, beam.g, beam.b, 255);
12732     } else if (baked) {
12733         ps.startColor = new Color(255, 255, 255, 220);
12734         ps.endColor = new Color(255, 255, 255, 0);
12735         ps.color = new Color(255, 255, 255, 255);
12736     } else {
12737         ps.startColor = new Color(beam.r, beam.g, beam.b, Math.min(220, beam.a));
12738         ps.endColor = new Color(beam.r, beam.g, beam.b, 0);
12739         ps.color = new Color(255, 255, 255, 255);
12740     }
12741     ps.startColorVar = new Color(0, 0, 0, 0);
12742     ps.endColorVar = new Color(0, 0, 0, 0);
12743 }
12744 function configureImpactParticle(ps: ParticleSystem2D, dir: Direction, colorKey: string): void {
12745     ps.custom = true;
12746     ps.playOnLoad = false;
12747     ps.autoRemoveOnFinish = false;
12748     ps.duration = -1;
12749     ps.life = 0.3;
12750     ps.lifeVar = 0.1;
12751     ps.emissionRate = 60;
12752     ps.totalParticles = 100;
12753     ps.startSize = 3;
12754     ps.startSizeVar = 1;
12755     ps.endSize = 1;
12756     ps.endSizeVar = 0.5;
12757     ps.angle = beamImpactEmissionAngle(dir);
12758     ps.angleVar = 100;
12759     ps.speed = 50;
12760     ps.speedVar = 20;

```

```

12761     ps.radialAccel = 20;
12762     ps.radialAccelVar = 5;
12763     ps.tangentialAccel = 0;
12764     ps.tangentialAccelVar = 12;
12765     ps.gravity = new Vec2(0, 0);
12766     ps.posVar = contactPosVar(dir);
12767     ps.startSpin = 0;
12768     ps.startSpinVar = 90;
12769     ps.endSpin = 0;
12770     ps.endSpinVar = 0;
12771     ps.emitterMode = ParticleSystem2D.EmitterMode.GRAVITY;
12772     ps.positionType = ParticleSystem2D.PositionType.FREE;
12773     applyBeamColorToParticle(ps, colorKey);
12774 }
12775 function applyImpactDirection(ps: ParticleSystem2D, dir: Direction): void {
12776     ps.angle = beamImpactEmissionAngle(dir);
12777     ps.angleVar = 85;
12778     ps.posVar = contactPosVar(dir);
12779 }
12780 interface ImpactEmitterEntry {
12781     node: Node;
12782     ps: ParticleSystem2D;
12783     colorKey: string;
12784     dir: Direction;
12785 }
12786 function isParticleAlive(ps: ParticleSystem2D | null | undefined): ps is ParticleSystem2D {
12787     return !!ps?.isValid && ps.enabledInHierarchy;
12788 }
12789 function disposeImpactEmitter(entry: ImpactEmitterEntry): void {
12790     if (entry.node?.isValid) {
12791         entry.node.removeFromParent();
12792         entry.node.destroy();
12793     }
12794 }
12795 /** 光线阻挡点粒子：主包贴图（Sprites/OpticalFX） */
12796 @ccclass('OpticalPuzzleBeamImpactView')
12797 export class OpticalPuzzleBeamImpactView extends Component {
12798     /** 可选：拖入 Sprites/OpticalFX 下 beam_spark_*, 编辑器预览与构建保活 */
12799     @property({ type: [SpriteFrame], tooltip: '可选，七色 beam_spark SpriteFrame' })
12800     sparkSpriteFrames: SpriteFrame[] = [];
12801     private _emitters = new Map<string, ImpactEmitterEntry>();
12802     private _lastSnapshot: OpticalBeamSnapshot | null = null;
12803     protected onLoad(): void {
12804         let ut = this.getComponent(UITransform);
12805         if (!ut) {
12806             ut = this.addComponent(UITransform);
12807             ut.setContentSize(700, 700);
12808         }
12809         if (registerSparkSpriteFrames(this.sparkSpriteFrames)) {
12810             _sparkSpritesReady = true;
12811             _sparkSpritesLoading = false;
12812         }
12813         ensureSparkSpriteFramesLoaded(() => {
12814             if (!this.isValid || !this._lastSnapshot) {
12815                 return;
12816             }
12817             this._clearEmitters();
12818             this.render(this._lastSnapshot);

```

```

12819         });
12820     }
12821     protected onDestroy(): void {
12822         this._clearEmitters();
12823     }
12824     /** 由 OpticalPuzzleRoot.beamSparkKeepAlive 注入，保证微信构建包含贴图 */
12825     applyExternalSparkSpriteFrames(frames: ReadonlyArray<SpriteFrame>): void {
12826         if (!registerSparkSpriteFrames(frames)) {
12827             return;
12828         }
12829         _sparkSpritesReady = true;
12830         _sparkSpritesLoading = false;
12831         if (this._lastSnapshot) {
12832             this._clearEmitters();
12833             this.render(this._lastSnapshot);
12834         }
12835     }
12836     render(snapshot: OpticalBeamSnapshot): void {
12837         this._lastSnapshot = snapshot;
12838         if (!_sparkSpritesReady) {
12839             ensureSparkSpriteFramesLoaded(() => {
12840                 if (this.isValid) {
12841                     this.render(snapshot);
12842                 }
12843             });
12844             return;
12845         }
12846         const ox = (-snapshot.width * CELL) / 2;
12847         const oy = (snapshot.height * CELL) / 2;
12848         const contacts = snapshot.blockContacts ?? [];
12849         const segments = snapshot.segments ?? [];
12850         const capped = contacts.length > MAX_IMPACT_EMITTERS
12851             ? contacts.slice(0, MAX_IMPACT_EMITTERS)
12852             : contacts;
12853         const desired = new Map<string, { px: number; py: number; colorKey: string; dir: Direction }>();
12854         for (const contact of capped) {
12855             const dir = normalizeDirection(contact.dir, Direction.Right);
12856             const colorKey = resolveBlockContactColorFromSegments(contact, segments);
12857             const key = contactPoolKey(contact, colorKey);
12858             const { px, py } = contactScreenPosition(contact, ox, oy);
12859             desired.set(key, { px, py, colorKey, dir });
12860         }
12861         for (const [key, entry] of this._emitters) {
12862             if (!desired.has(key)) {
12863                 disposeImpactEmitter(entry);
12864                 this._emitters.delete(key);
12865             }
12866         }
12867         for (const [key, target] of desired) {
12868             let entry = this._emitters.get(key);
12869             if (entry && (!entry.node?.isValid || !entry.ps?.isValid)) {
12870                 this._emitters.delete(key);
12871                 entry = undefined;
12872             }
12873             if (!entry) {
12874                 entry = this._createEmitter(target.colorKey, target.dir);
12875                 this._emitters.set(key, entry);
12876             } else if (entry.colorKey !== target.colorKey) {

```

```

12877         disposeImpactEmitter(entry);
12878         entry = this._createEmitter(target.colorKey, target.dir);
12879         this._emitters.set(key, entry);
12880     }
12881     if (!entry.node?.isValid) {
12882         continue;
12883     }
12884     entry.node.setPosition(target.px, target.py, 0);
12885     if (isParticleAlive(entry.ps) && entry.dir !== target.dir) {
12886         applyImpactDirection(entry.ps, target.dir);
12887         entry.dir = target.dir;
12888     }
12889 }
12890 }
12891 private _createEmitter(colorKey: string, dir: Direction): ImpactEmitterEntry {
12892     const node = new Node('BeamImpact');
12893     const ps = node.addComponent(ParticleSystem2D);
12894     configureImpactParticle(ps, dir, colorKey);
12895     node.parent = this.node;
12896     this.scheduleOnce(() => {
12897         if (!this.isValid || !ps.isValid) {
12898             return;
12899         }
12900         if (isParticleAlive(ps)) {
12901             ps.resetSystem();
12902         }
12903     }, 0);
12904     return { node, ps, colorKey, dir };
12905 }
12906 private _clearEmitters(): void {
12907     for (const entry of this._emitters.values()) {
12908         disposeImpactEmitter(entry);
12909     }
12910     this._emitters.clear();
12911 }
12912 }
12913 /** 菜单等场景复用 beam_spark 粒子配置 */
12914 export function ensureBeamSparkSpritesLoaded(onReady?: () => void): void {
12915     ensureSparkSpriteFramesLoaded(onReady);
12916 }
12917 export function registerBeamSparkSpriteFrames(
12918     frames: ReadonlyArray<SpriteFrame | null | undefined>,
12919 ): boolean {
12920     return registerSparkSpriteFrames(frames);
12921 }
12922 export function configureBeamSparkParticle(
12923     ps: ParticleSystem2D,
12924     dir: Direction,
12925     colorKey: string,
12926 ): void {
12927     configureImpactParticle(ps, dir, colorKey);
12928 }
12929 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleBeamSparkSpriteUuids.ts -----
12930 /** 主包内粒子火花 SpriteFrame UUID（与 assets/Sprites/OpticalFX 下 png.meta 中 f9941 一致） */
12931 export const BEAM_SPARK_SPRITE_UUIDS: Readonly<Record<string, string>> = {
12932     red: '2752aa51-5e0c-476b-ab35-e9b6485d7056@f9941',
12933     green: '5d69d4f2-6998-4b22-8dfa-5331abffd135@f9941',
12934     blue: '7fb70a4a-c46c-45cd-acf6-6454a5680f2f@f9941',

```

```

12935     yellow: '4a39292d-6c36-4b85-84df-a6579a82f60e@f9941',
12936     cyan: '7208d0b3-e155-4c8c-8c23-5054c3574f5b@f9941',
12937     purple: '6accf15d-4ddf-4801-bce7-0a234dbccbd6@f9941',
12938     white: '4b9aae35-0034-4b5c-b7f9-1e61ea071125@f9941',
12939 };
12940 /** resources 目录路径（主包内置 bundle，非远程分包；微信构建必进包） */
12941 export const BEAM_SPARK_RESOURCE_PATHS: Readonly<Record<string, string>> = {
12942     red: 'OpticalFX/beam_spark_red/spriteFrame',
12943     green: 'OpticalFX/beam_spark_green/spriteFrame',
12944     blue: 'OpticalFX/beam_spark_blue/spriteFrame',
12945     yellow: 'OpticalFX/beam_spark_yellow/spriteFrame',
12946     cyan: 'OpticalFX/beam_spark_cyan/spriteFrame',
12947     purple: 'OpticalFX/beam_spark_purple/spriteFrame',
12948     white: 'OpticalFX/beam_spark_white/spriteFrame',
12949 };
12950 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleBeamView.ts -----
12951 import {
12952     _decorator,
12953     Color,
12954     Component,
12955     Graphics,
12956     UITransform,
12957 } from 'cc';
12958 import { computeCrossBeamOverlays, type OpticalCrossBeamOverlay } from '../Core/OpticalBeamOverlapMerge';
12959 import type { OpticalBeamSnapshot } from '../Core/OpticalPuzzleCore';
12960 import { mixLightColors } from '../Core/OpticalColorMix';
12961 import type { OpticalBeamSegment } from '../Core/OpticalBeamTypes';
12962 const { ccclass } = _decorator;
12963 import { beamColorFromKey } from './OpticalPuzzleColorUtil';
12964 import {
12965     coreWhiteFor,
12966     lerpBeamColor,
12967     strokeBeamSegmentGradient,
12968 } from './OpticalPuzzleBeamGradient';
12969 import { BEAM_CORE_WIDTH_RATIO, BEAM_GRADIENT_STEPS, BEAM_LINE_WIDTH, OPTICAL_CELL_SIZE } from
12970 './OpticalPuzzleLayout';
12971 const CELL = OPTICAL_CELL_SIZE;
12972 const EPS = 1e-3;
12973 /**
12974  * 十字叠色单层：各光路先各自做「白芯→本色」渐变，再按该深度混色（中心仍偏白，外圈才混成紫等）。
12975  */
12976 function crossLayerStrokeColor(colorKeys: readonly string[], t: number): Color {
12977     const unique = [...new Set(colorKeys)];
12978     if (unique.length === 0) {
12979         return new Color(255, 255, 255, 235);
12980     }
12981     if (unique.length === 1) {
12982         const beam = beamColorFromKey(unique[0]);
12983         return lerpBeamColor(coreWhiteFor(beam), beam, t);
12984     }
12985     const layerColors = unique.map((key) => {
12986         const beam = beamColorFromKey(key);
12987         return lerpBeamColor(coreWhiteFor(beam), beam, t);
12988     });
12989     const mixedKey = mixLightColors(unique);
12990     const mixedBeam = beamColorFromKey(mixedKey);
12991     const mixedLayer = lerpBeamColor(coreWhiteFor(mixedBeam), mixedBeam, t);
12992     let r = 0;

```



```

12993     let g = 0;
12994     let b = 0;
12995     let a = 0;
12996     for (const c of layerColors) {
12997         r += c.r;
12998         g += c.g;
12999         b += c.b;
13000         a += c.a;
13001     }
13002     const n = layerColors.length;
13003     const avg = new Color(r / n, g / n, b / n, a / n);
13004     // t=0 外圈贴近玩法混色; t=1 内圈贴近多束白芯叠合
13005     return lerpBeamColor(mixedLayer, avg, t);
13006 }
13007 /** 十字交点: 每层同步横竖窄条, 避免先画两束再盖一层导致中心过曝/边缘发脏 */
13008 function strokeCrossOverlay(
13009     g: Graphics,
13010     px: number,
13011     py: number,
13012     halfPx: number,
13013     colorKeys: readonly string[],
13014 ): void {
13015     const steps = BEAM_GRADIENT_STEPS;
13016     const coreRatio = BEAM_CORE_WIDTH_RATIO;
13017     g.lineCap = Graphics.LineCap.BUTT;
13018     g.lineJoin = Graphics.LineJoin.ROUND;
13019     for (let i = steps - 1; i >= 0; i--) {
13020         const t = steps <= 1 ? 1 : i / (steps - 1);
13021         g.lineWidth = BEAM_LINE_WIDTH * (coreRatio + (1 - coreRatio) * t);
13022         g.strokeColor = crossLayerStrokeColor(colorKeys, t);
13023         g.moveTo(px - halfPx, py);
13024         g.lineTo(px + halfPx, py);
13025         g.stroke();
13026         g.moveTo(px, py - halfPx);
13027         g.lineTo(px, py + halfPx);
13028         g.stroke();
13029     }
13030 }
13031 function subtractInterval(lo: number, hi: number, gapLo: number, gapHi: number): Array<[number, number]> {
13032     if (gapHi <= lo + EPS || gapLo >= hi - EPS) {
13033         return [[lo, hi]];
13034     }
13035     const out: Array<[number, number]> = [];
13036     if (gapLo > lo + EPS) {
13037         out.push([lo, gapLo]);
13038     }
13039     if (gapHi < hi - EPS) {
13040         out.push([gapHi, hi]);
13041     }
13042     return out;
13043 }
13044 function crossesOnHorizontal(seg: OpticalBeamSegment, crosses: readonly OpticalCrossBeamOverlay[]):
13045     OpticalCrossBeamOverlay[] {
13046     const lo = Math.min(seg.x0, seg.x1);
13047     const hi = Math.max(seg.x0, seg.x1);
13048     return crosses.filter(
13049         (c) => Math.abs(c.y - seg.y0) < EPS && c.x >= lo - EPS && c.x <= hi + EPS,
13050     );

```

```

13051 }
13052 function crossesOnVertical(seg: OpticalBeamSegment, crosses: readonly OpticalCrossBeamOverlay[]):
13053 OpticalCrossBeamOverlay[] {
13054     const lo = Math.min(seg.y0, seg.y1);
13055     const hi = Math.max(seg.y0, seg.y1);
13056     return crosses.filter(
13057         (c) => Math.abs(c.x - seg.x0) < EPS && c.y >= lo - EPS && c.y <= hi + EPS,
13058     );
13059 }
13060 /** 在十字交点处挖空，避免与叠色窄条重复绘制 */
13061 function splitSegmentAtCrosses(
13062     seg: OpticalBeamSegment,
13063     crosses: readonly OpticalCrossBeamOverlay[],
13064     halfGap: number,
13065 ): OpticalBeamSegment[] {
13066     const dx = seg.x1 - seg.x0;
13067     const dy = seg.y1 - seg.y0;
13068     if (Math.abs(dy) < EPS && Math.abs(dx) > EPS) {
13069         let intervals: Array<[number, number]> = [[Math.min(seg.x0, seg.x1), Math.max(seg.x0, seg.x1)]];
13070         for (const cross of crossesOnHorizontal(seg, crosses)) {
13071             const next: Array<[number, number]> = [];
13072             for (const [lo, hi] of intervals) {
13073                 next.push(...subtractInterval(lo, hi, cross.x - halfGap, cross.x + halfGap));
13074             }
13075             intervals = next;
13076         }
13077         return intervals
13078             .filter(([lo, hi]) => hi - lo > EPS)
13079             .map([x0, x1]) => ({
13080                 x0,
13081                 y0: seg.y0,
13082                 x1,
13083                 y1: seg.y1,
13084                 colorKey: seg.colorKey,
13085             });
13086     }
13087     if (Math.abs(dx) < EPS && Math.abs(dy) > EPS) {
13088         let intervals: Array<[number, number]> = [[Math.min(seg.y0, seg.y1), Math.max(seg.y0, seg.y1)]];
13089         for (const cross of crossesOnVertical(seg, crosses)) {
13090             const next: Array<[number, number]> = [];
13091             for (const [lo, hi] of intervals) {
13092                 next.push(...subtractInterval(lo, hi, cross.y - halfGap, cross.y + halfGap));
13093             }
13094             intervals = next;
13095         }
13096         return intervals
13097             .filter(([lo, hi]) => hi - lo > EPS)
13098             .map([y0, y1]) => ({
13099                 x0: seg.x0,
13100                 y0,
13101                 x1: seg.x0,
13102                 y1,
13103                 colorKey: seg.colorKey,
13104             });
13105     }
13106     return [{ ...seg }];
13107 }
13108 /** 光路占位：直线段；镜格内由 tracer 拆成 L 形两段（后续换美术素材） */

```

```

13109 @cclass('OpticalPuzzleBeamView')
13110 export class OpticalPuzzleBeamView extends Component {
13111     private _graphics: Graphics | null = null;
13112     protected onLoad(): void {
13113         this._ensureGraphics();
13114         let ut = this.getComponent(UITransform);
13115         if (!ut) {
13116             ut = this.addComponent(UITransform);
13117             ut.setContentSize(700, 700);
13118         }
13119     }
13120     private _ensureGraphics(): Graphics | null {
13121         if (!this._graphics?.isValid) {
13122             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
13123         }
13124         return this._graphics;
13125     }
13126     render(snapshot: OpticalBeamSnapshot): void {
13127         const g = this._ensureGraphics();
13128         if (!g) {
13129             return;
13130         }
13131         g.clear();
13132         const ox = (-snapshot.width * CELL) / 2;
13133         const oy = (snapshot.height * CELL) / 2;
13134         const crosses = computeCrossBeamOverlays(snapshot.segments);
13135         const overlapHalf = BEAM_LINE_WIDTH / (2 * CELL);
13136         const halfPx = overlapHalf * CELL;
13137         for (const seg of snapshot.segments) {
13138             const pieces = crosses.length > 0
13139                 ? splitSegmentAtCrosses(seg, crosses, overlapHalf)
13140                 : [{ ...seg }];
13141             for (const piece of pieces) {
13142                 strokeBeamSegmentGradient(
13143                     g,
13144                     ox + piece.x0 * CELL,
13145                     oy - piece.y0 * CELL,
13146                     ox + piece.x1 * CELL,
13147                     oy - piece.y1 * CELL,
13148                     beamColorFromKey(piece.colorKey),
13149                 );
13150             }
13151         }
13152         for (const cross of crosses) {
13153             strokeCrossOverlay(
13154                 g,
13155                 ox + cross.x * CELL,
13156                 oy - cross.y * CELL,
13157                 halfPx,
13158                 cross.colorKeys,
13159             );
13160         }
13161     }
13162 }
13163 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleBoardView.ts -----
13164 import {
13165     _decorator,
13166     Component,

```

```

13167     Graphics,
13168     Node,
13169     UITransform,
13170 } from 'cc';
13171 import type { OpticalSessionNotifyReason } from '../Application/OpticalPuzzleSession';
13172 import type { OpticalBoardSnapshot } from '../Core/OpticalPuzzleTypes';
13173 import { Direction, TerrainKind } from '../Core/OpticalPuzzleTypes';
13174 import { drawConnectivityGlyph } from './OpticalPuzzlePieceGlyph';
13175 import {
13176     animatedSquashedCellRect,
13177     buildMoveAnimEntities,
13178     buildFailedMovePlayerEntity,
13179     MOVE_ANIM_DURATION,
13180     moveAnimProgress,
13181     type MoveAnimEntity,
13182     type MoveAnimState,
13183     type SquashedCellRect,
13184 } from './OpticalPuzzleMoveAnim';
13185 import { drawPlayerEyes } from './OpticalPuzzlePlayerGlyph';
13186 import { drawSourceEmitter } from './OpticalPuzzleSourceGlyph';
13187 import { drawTargetLamp } from './OpticalPuzzleTargetGlyph';
13188 import { OPTICAL_CELL_SIZE } from './OpticalPuzzleLayout';
13189 import { cellScreenRect, fillBoardFloorBase, fillWallCell } from './OpticalPuzzleWallDraw';
13190 const { ccclass } = _decorator;
13191 /** 无操作多久后触发一次眨眼（秒） */
13192 const PLAYER_IDLE_BLINK_AT = 4;
13193 /** 无操作多久后闭眼（秒） */
13194 const PLAYER_IDLE_CLOSE_AT = 10;
13195 /** 眨眼 / 闭眼动画时长（秒） */
13196 const PLAYER_BLINK_DURATION = 0.25;
13197 /** 移动被阻拦时 >< 眼形保持时长（秒） */
13198 const PLAYER_BLOCKED_EYES_DURATION = 0.35;
13199 /** 主角眼部闲置状态 */
13200 enum PlayerEyeIdleState {
13201     /** 正常睁开，累计闲置时间 */
13202     Active,
13203     /** 3s 闲置触发的单次眨眼 */
13204     IdleBlinking,
13205     /** 3s 眨眼结束，等待 10s 闭眼（此阶段不再眨眼） */
13206     IdleAfterBlink,
13207     /** 10s 闲置，闭眼动画中 */
13208     Closing,
13209     /** 已闭眼静止 */
13210     Closed,
13211     /** 按方向键后睁眼（半时长） */
13212     Opening,
13213 }
13214 /** 棋盘占位绘制：墙/地板/光源；目标/元件/主角在光路之上单独层绘制 */
13215 @ccclass('OpticalPuzzleBoardView')
13216 export class OpticalPuzzleBoardView extends Component {
13217     private _graphics: Graphics | null = null;
13218     private _targetGraphics: Graphics | null = null;
13219     private _pieceGraphics: Graphics | null = null;
13220     /** 最近一次元件层快照，供眨眼动画刷新主角 */
13221     private _lastPiecesSnapshot: OpticalBoardSnapshot | null = null;
13222     private _eyeState = PlayerEyeIdleState.Active;
13223     /** 自上次方向输入起的闲置时长（秒） */
13224     private _idleTime = 0;

```

```

13225  /** 本段闲置是否已在 3s 触发过眨眼 */
13226  private _idleBlinkDone = false;
13227  private _animElapsed = 0;
13228  /** 移动被阻拦: >< 眼形 */
13229  private _blockedEyes = false;
13230  private _blockedElapsed = 0;
13231  /** 动画结束后的棋盘快照（用于下一帧 move/push 的起点） */
13232  private _settledSnapshot: OpticalBoardSnapshot | null = null;
13233  /** 主角 / 元件滑格 + 挤压 */
13234  private _moveAnim: MoveAnimState | null = null;
13235  protected onLoad(): void {
13236      this._ensureGraphics();
13237      let ut = this.getComponent(UITransform);
13238      if (!ut) {
13239          ut = this.addComponent(UITransform);
13240          ut.setContentSize(700, 700);
13241      }
13242  }
13243  private _ensureGraphics(): Graphics | null {
13244      if (!this._graphics?.isValid) {
13245          this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
13246      }
13247      return this._graphics;
13248  }
13249  /** 光路层之上的目标层（指示灯须在光束之上才可见） */
13250  private _ensureTargetGraphics(): Graphics | null {
13251      if (this._targetGraphics?.isValid) {
13252          return this._targetGraphics;
13253      }
13254      const root = this.node.parent ?? this.node;
13255      let layer = root.getChildByName('TargetLayer');
13256      if (!layer) {
13257          layer = new Node('TargetLayer');
13258          root.addChild(layer);
13259          const ut = layer.addComponent(UITransform);
13260          const boardUt = this.node.getComponent(UITransform);
13261          if (boardUt) {
13262              ut.setContentSize(boardUt.contentSize);
13263          } else {
13264              ut.setContentSize(700, 700);
13265          }
13266          const beam = root.getChildByName('BeamLayer');
13267          if (beam) {
13268              layer.setSiblingIndex(beam.getSiblingIndex() + 1);
13269          }
13270      }
13271      this._targetGraphics = layer.getComponent(Graphics) ?? layer.addComponent(Graphics);
13272      return this._targetGraphics;
13273  }
13274  /** 光路层之上的元件层（避免窄光线完全盖住元件本色） */
13275  private _ensurePieceGraphics(): Graphics | null {
13276      if (this._pieceGraphics?.isValid) {
13277          return this._pieceGraphics;
13278      }
13279      const root = this.node.parent ?? this.node;
13280      let layer = root.getChildByName('PieceLayer');
13281      if (!layer) {
13282          layer = new Node('PieceLayer');

```

```

13283         root.addChild(layer);
13284         const ut = layer.addComponent(UITransform);
13285         const boardUt = this.node.getComponent(UITransform);
13286         if (boardUt) {
13287             ut.setContentSize(boardUt.contentSize);
13288         } else {
13289             ut.setContentSize(700, 700);
13290         }
13291         const beam = root.getChildByName('BeamLayer');
13292         if (beam) {
13293             const target = root.getChildByName('TargetLayer');
13294             const insertAfter = target ?? beam;
13295             layer.setSiblingIndex(insertAfter.getSiblingIndex() + 1);
13296         }
13297     }
13298     this._pieceGraphics = layer.getComponent(Graphics) ?? layer.addComponent(Graphics);
13299     return this._pieceGraphics;
13300 }
13301 private _cellLayout(snapshot: OpticalBoardSnapshot): { cell: number; ox: number; oy: number } {
13302     const cell = OPTICAL_CELL_SIZE;
13303     return {
13304         cell,
13305         ox: (-snapshot.width * cell) / 2,
13306         oy: (snapshot.height * cell) / 2,
13307     };
13308 }
13309 /** 绘制目标指示灯（须在 BeamView.render 之后调用） */
13310 renderTargetsOverlay(snapshot: OpticalBoardSnapshot): void {
13311     const g = this._ensureTargetGraphics();
13312     if (!g) {
13313         return;
13314     }
13315     g.clear();
13316     const { cell, ox, oy } = this._cellLayout(snapshot);
13317     for (const tgt of snapshot.targets) {
13318         const { left, bottom, size } = cellScreenRect(ox, oy, tgt.x, tgt.y, cell);
13319         drawTargetLamp(g, left, bottom, size, tgt.colorKey, tgt.lit);
13320     }
13321 }
13322 /** 当前眨眼强度：0 睁开，1 闭眼峰值 */
13323 private _currentBlinkAmount(): number {
13324     switch (this._eyeState) {
13325         case PlayerEyeIdleState.Active:
13326         case PlayerEyeIdleState.IdleAfterBlink:
13327             return 0;
13328         case PlayerEyeIdleState.IdleBlinking: {
13329             const p = Math.min(1, this._animElapsed / PLAYER_BLINK_DURATION);
13330             return Math.sin(p * Math.PI);
13331         }
13332         case PlayerEyeIdleState.Closing: {
13333             const p = Math.min(1, this._animElapsed / PLAYER_BLINK_DURATION);
13334             return Math.sin(p * Math.PI * 0.5);
13335         }
13336         case PlayerEyeIdleState.Closed:
13337             return 1;
13338         case PlayerEyeIdleState.Opening: {
13339             const openDur = PLAYER_BLINK_DURATION * 0.5;
13340             const p = Math.min(1, this._animElapsed / openDur);

```

```

13341         return 1 - Math.sin(p * Math.PI * 0.5);
13342     }
13343     default:
13344         return 0;
13345 }
13346 }
13347 /** 任意方向操作：打破睡眠 / 打断闲置眨眼，回到 Active 以重新开始 4s→10s 周期 */
13348 private _wakeFromDirectionInput(): void {
13349     if (
13350         this._eyeState === PlayerEyeIdleState.Closed ||
13351         this._eyeState === PlayerEyeIdleState.Closing ||
13352         this._eyeState === PlayerEyeIdleState.Opening ||
13353         this._eyeState === PlayerEyeIdleState.IdleBlinking ||
13354         this._eyeState === PlayerEyeIdleState.IdleAfterBlink
13355     ) {
13356         this._eyeState = PlayerEyeIdleState.Active;
13357         this._animElapsed = 0;
13358     }
13359 }
13360 /** 方向键 / 四向按钮（移动成功）：重置闲置；若已闭眼则半时长睁眼；取消 >< 阻拦眼 */
13361 notifyPlayerDirectionInput(): void {
13362     if (this._blockedEyes) {
13363         this._clearBlockedEyes();
13364     }
13365     const wasSleeping =
13366         this._eyeState === PlayerEyeIdleState.Closed ||
13367         this._eyeState === PlayerEyeIdleState.Closing;
13368     if (wasSleeping) {
13369         this._eyeState = PlayerEyeIdleState.Opening;
13370         this._animElapsed = 0;
13371         this._idleTime = 0;
13372         this._idleBlinkDone = false;
13373         if (this._lastPiecesSnapshot) {
13374             this.renderPiecesOverlay(this._lastPiecesSnapshot);
13375         }
13376         return;
13377     }
13378     this._wakeFromDirectionInput();
13379     this._idleTime = 0;
13380     this._idleBlinkDone = false;
13381 }
13382 /** 移动被阻拦：>< 眼形 0.5s；期间再次阻拦则重新计时 0.5s，并打破睡眠 */
13383 notifyPlayerMoveBlocked(): void {
13384     this._wakeFromDirectionInput();
13385     this._idleTime = 0;
13386     this._idleBlinkDone = false;
13387     if (this._blockedEyes) {
13388         this._blockedElapsed = 0;
13389     } else {
13390         this._blockedEyes = true;
13391         this._blockedElapsed = 0;
13392     }
13393     if (this._lastPiecesSnapshot) {
13394         this.renderPiecesOverlay(this._lastPiecesSnapshot);
13395     }
13396 }
13397 private _clearBlockedEyes(): void {
13398     this._blockedEyes = false;

```

```

13399         this._blockedElapsed = 0;
13400     }
13401     private _endBlockedEyes(): void {
13402         this._clearBlockedEyes();
13403     }
13404     /** 关卡加载 / 重置时恢复眼部闲置逻辑 */
13405     resetPlayerEyeIdle(): void {
13406         this._clearBlockedEyes();
13407         this._eyeState = PlayerEyeIdleState.Active;
13408         this._idleTime = 0;
13409         this._idleBlinkDone = false;
13410         this._animElapsed = 0;
13411     }
13412     /** 滑格动画进行中（Presentation 输入锁） */
13413     isMoveAnimating(): boolean {
13414         return this._moveAnim !== null;
13415     }
13416     /** 根据 Session 通知更新元件层：move/push 播滑格，load/undo/reset 瞬变 */
13417     syncPlaySnapshot(snapshot: OpticalBoardSnapshot, reason: OpticalSessionNotifyReason): void {
13418         if (reason === 'load' || reason === 'reset' || reason === 'undo') {
13419             this._cancelMoveAnim();
13420             this._settledSnapshot = snapshot;
13421         } else if (reason === 'move' || reason === 'push' || reason === 'complete') {
13422             this._beginMoveAnim(snapshot);
13423         } else if (reason === 'face') {
13424             this._beginFailedMoveAnim(snapshot);
13425         }
13426         this.renderPiecesOverlay(snapshot);
13427     }
13428     private _cancelMoveAnim(): void {
13429         this._moveAnim = null;
13430     }
13431     private _beginMoveAnim(snapshot: OpticalBoardSnapshot): void {
13432         if (!this._settledSnapshot) {
13433             this._settledSnapshot = snapshot;
13434             return;
13435         }
13436         const entities = buildMoveAnimEntities(this._settledSnapshot, snapshot);
13437         if (entities.length === 0) {
13438             this._settledSnapshot = snapshot;
13439             return;
13440         }
13441         this._moveAnim = { elapsed: 0, snapshot, entities };
13442     }
13443     /** 推墙 / 推不动：仅主角原地挤压，元件不参与 */
13444     private _beginFailedMoveAnim(snapshot: OpticalBoardSnapshot): void {
13445         if (!this._settledSnapshot) {
13446             this._settledSnapshot = snapshot;
13447         }
13448         this._moveAnim = {
13449             elapsed: 0,
13450             snapshot,
13451             entities: [buildFailedMovePlayerEntity(snapshot)],
13452         };
13453     }
13454     private _finishMoveAnim(): void {
13455         if (this._moveAnim) {
13456             this._settledSnapshot = this._moveAnim.snapshot;

```



```

13457         this._moveAnim = null;
13458     }
13459 }
13460 private _animRectForEntity(
13461     entity: MoveAnimEntity,
13462     ox: number,
13463     oy: number,
13464     cell: number,
13465 ): SquashedCellRect {
13466     const progress = moveAnimProgress(this._moveAnim!.elapsed);
13467     return animatedSquashedCellRect(
13468         ox,
13469         oy,
13470         cell,
13471         entity.fromX,
13472         entity.fromY,
13473         entity.toX,
13474         entity.toY,
13475         progress,
13476         entity.direction,
13477     );
13478 }
13479 private _playerAnimEntity(): MoveAnimEntity | null {
13480     return this._moveAnim?.entities.find((e) => e.kind === 'player') ?? null;
13481 }
13482 private _pieceAnimEntity(index: number): MoveAnimEntity | null {
13483     return (
13484         this._moveAnim?.entities.find(
13485             (e) => e.kind === 'piece' && e.piecesIndex === index,
13486         ) ?? null
13487     );
13488 }
13489 private _tickIdleTime(dt: number): void {
13490     if (
13491         this._eyeState === PlayerEyeIdleState.Active ||
13492         this._eyeState === PlayerEyeIdleState.IdleAfterBlink ||
13493         this._eyeState === PlayerEyeIdleState.IdleBlinking ||
13494         this._eyeState === PlayerEyeIdleState.Closing
13495     ) {
13496         this._idleTime += dt;
13497     }
13498 }
13499 protected update(dt: number): void {
13500     if (!this._lastPiecesSnapshot) {
13501         return;
13502     }
13503     let needRender = false;
13504     if (this._blockedEyes) {
13505         this._blockedElapsed += dt;
13506         needRender = true;
13507         if (this._blockedElapsed >= PLAYER_BLOCKED_EYES_DURATION) {
13508             this._endBlockedEyes();
13509         }
13510     }
13511     this._tickIdleTime(dt);
13512     switch (this._eyeState) {
13513         case PlayerEyeIdleState.Active:
13514             if (this._idleTime >= PLAYER_IDLE_BLINK_AT && !this._idleBlinkDone) {

```

```

13515         this._eyeState = PlayerEyeIdleState.IdleBlinking;
13516         this._animElapsed = 0;
13517         needRender = true;
13518     } else if (this._idleTime >= PLAYER_IDLE_CLOSE_AT) {
13519         this._eyeState = PlayerEyeIdleState.Closing;
13520         this._animElapsed = 0;
13521         needRender = true;
13522     }
13523     break;
13524 case PlayerEyeIdleState.IdleBlinking:
13525     this._animElapsed += dt;
13526     needRender = true;
13527     if (this._animElapsed >= PLAYER_BLINK_DURATION) {
13528         this._eyeState = PlayerEyeIdleState.IdleAfterBlink;
13529         this._idleBlinkDone = true;
13530         this._animElapsed = 0;
13531     }
13532     break;
13533 case PlayerEyeIdleState.IdleAfterBlink:
13534     if (this._idleTime >= PLAYER_IDLE_CLOSE_AT) {
13535         this._eyeState = PlayerEyeIdleState.Closing;
13536         this._animElapsed = 0;
13537         needRender = true;
13538     }
13539     break;
13540 case PlayerEyeIdleState.Closing:
13541     this._animElapsed += dt;
13542     needRender = true;
13543     if (this._animElapsed >= PLAYER_BLINK_DURATION) {
13544         this._eyeState = PlayerEyeIdleState.Closed;
13545         this._animElapsed = 0;
13546     }
13547     break;
13548 case PlayerEyeIdleState.Closed:
13549     break;
13550 case PlayerEyeIdleState.Opening:
13551     this._animElapsed += dt;
13552     needRender = true;
13553     if (this._animElapsed >= PLAYER_BLINK_DURATION * 0.5) {
13554         this._eyeState = PlayerEyeIdleState.Active;
13555         this._idleTime = 0;
13556         this._idleBlinkDone = false;
13557         this._animElapsed = 0;
13558     }
13559     break;
13560 default:
13561     break;
13562 }
13563 if (this._moveAnim) {
13564     this._moveAnim.elapsed += dt;
13565     needRender = true;
13566     if (this._moveAnim.elapsed >= MOVE_ANIM_DURATION) {
13567         this._finishMoveAnim();
13568     }
13569 }
13570 if (needRender && this._lastPiecesSnapshot) {
13571     this.renderPiecesOverlay(this._lastPiecesSnapshot);
13572 }

```

```

13573     }
13574     /** 绘制元件与主角（须在 BeamView.render 之后调用） */
13575     renderPiecesOverlay(snapshot: OpticalBoardSnapshot): void {
13576         this._lastPiecesSnapshot = snapshot;
13577         const g = this._ensurePieceGraphics();
13578         if (!g) {
13579             return;
13580         }
13581         g.clear();
13582         const { cell, ox, oy } = this._cellLayout(snapshot);
13583         snapshot.pieces.forEach((piece, index) => {
13584             const anim = this._pieceAnimEntity(index);
13585             if (anim) {
13586                 const rect = this._animRectForEntity(anim, ox, oy, cell);
13587                 drawConnectivityGlyph(
13588                     g,
13589                     rect.left,
13590                     rect.bottom + rect.height,
13591                     rect.width,
13592                     piece.connectivity,
13593                     piece.direction,
13594                     piece.colorKey,
13595                     rect.height,
13596                 );
13597                 return;
13598             }
13599             const { left, bottom, size } = cellScreenRect(ox, oy, piece.x, piece.y, cell);
13600             drawConnectivityGlyph(
13601                 g,
13602                 left,
13603                 bottom + size,
13604                 size,
13605                 piece.connectivity,
13606                 piece.direction,
13607                 piece.colorKey,
13608             );
13609         });
13610         const playerAnim = this._playerAnimEntity();
13611         if (playerAnim) {
13612             const rect = this._animRectForEntity(playerAnim, ox, oy, cell);
13613             drawPlayerEyes(
13614                 g,
13615                 rect.left,
13616                 rect.bottom,
13617                 rect.width,
13618                 snapshot.playerFacing ?? Direction.Left,
13619                 this._blockedEyes ? 0 : this._currentBlinkAmount(),
13620                 this._blockedEyes,
13621                 rect.height,
13622             );
13623         } else {
13624             const { x: px, y: py } = snapshot.player;
13625             const playerRect = cellScreenRect(ox, oy, px, py, cell);
13626             drawPlayerEyes(
13627                 g,
13628                 playerRect.left,
13629                 playerRect.bottom,
13630                 playerRect.size,

```

```

13631         snapshot.playerFacing ?? Direction.Left,
13632         this._blockedEyes ? 0 : this._currentBlinkAmount(),
13633         this._blockedEyes,
13634     );
13635     }
13636 }
13637 render(snapshot: OpticalBoardSnapshot): void {
13638     const g = this._ensureGraphics();
13639     if (!g) {
13640         return;
13641     }
13642     g.clear();
13643     const { cell, ox, oy } = this._cellLayout(snapshot);
13644     fillBoardFloorBase(g, snapshot.width, snapshot.height, ox, oy, cell);
13645     const sourceAt = new Map<string, { colorKey: string; direction: Direction }>();
13646     for (const s of snapshot.sources) {
13647         sourceAt.set(`${s.x},${s.y}`, { colorKey: s.colorKey, direction: s.direction });
13648     }
13649     for (let y = 0; y < snapshot.height; y++) {
13650         for (let x = 0; x < snapshot.width; x++) {
13651             const t = snapshot.terrain[y * snapshot.width + x];
13652             const { left, bottom, size } = cellScreenRect(ox, oy, x, y, cell);
13653             if (t === TerrainKind.Wall) {
13654                 fillWallCell(g, snapshot, x, y, left, bottom, size);
13655                 continue;
13656             }
13657             if (t === TerrainKind.Source) {
13658                 const src = sourceAt.get(`${x},${y}`);
13659                 drawSourceEmitter(
13660                     g,
13661                     left,
13662                     bottom,
13663                     size,
13664                     src?.colorKey,
13665                     src?.direction ?? Direction.Up,
13666                 );
13667             }
13668         }
13669     }
13670 }
13671 }
13672 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleColorUtil.ts -----
13673 import { Color } from 'cc';
13674 import type { MixedLightColorKey } from '../Core/OpticalColorMix';
13675 import { normalizeLightColorKey, resolveBeamColorKey } from '../Core/OpticalLightColor';
13676 /** 与 BeamView 光路描边一致的本色（Presentation 统一入口） */
13677 export function beamColorFromKey(colorKey?: string): Color {
13678     const k = resolveBeamColorKey(colorKey) as MixedLightColorKey;
13679     switch (k) {
13680         case 'red':
13681             return new Color(255, 72, 72, 230);
13682         case 'green':
13683             return new Color(72, 220, 110, 230);
13684         case 'blue':
13685             return new Color(72, 140, 255, 230);
13686         case 'yellow':
13687             return new Color(255, 220, 72, 230);
13688         case 'cyan':

```

```

13689         return new Color(72, 220, 230, 230);
13690     case 'purple':
13691         return new Color(180, 90, 255, 230);
13692     default:
13693         return new Color(255, 255, 255, 235);
13694     }
13695 }
13696 /** 光源方块填充色（含 S 层 3 的 V/C/P） */
13697 export function sourceFillColor(colorKey?: string): Color {
13698     const beam = beamColorFromKey(colorKey);
13699     return new Color(beam.r, beam.g, beam.b, 255);
13700 }
13701 /** 激光发射器绘制色（与光路同色阶） */
13702 export function sourceEmitterColors(colorKey?: string): {
13703     beam: Color;
13704     glow: Color;
13705     core: Color;
13706     accent: Color;
13707     chassis: Color;
13708     chassisEdge: Color;
13709 } {
13710     const beam = beamColorFromKey(colorKey);
13711     return {
13712         beam,
13713         glow: new Color(beam.r, beam.g, beam.b, 90),
13714         core: new Color(255, 255, 255, Math.min(255, beam.a + 15)),
13715         accent: new Color(
13716             Math.floor(beam.r * 0.72),
13717             Math.floor(beam.g * 0.72),
13718             Math.floor(beam.b * 0.72),
13719             255,
13720         ),
13721         chassis: new Color(
13722             Math.floor(beam.r * 0.12 + 16),
13723             Math.floor(beam.g * 0.12 + 20),
13724             Math.floor(beam.b * 0.12 + 28),
13725             255,
13726         ),
13727         chassisEdge: new Color(
13728             Math.floor(beam.r * 0.35 + 36),
13729             Math.floor(beam.g * 0.35 + 44),
13730             Math.floor(beam.b * 0.35 + 56),
13731             255,
13732         ),
13733     };
13734 }
13735 /** 光接收靶绘制色（未点亮 / 已点亮） */
13736 export function targetReceptorColors(
13737     colorKey?: string,
13738     lit: boolean = false,
13739 ): {
13740     beam: Color;
13741     glow: Color;
13742     base: Color;
13743     panel: Color;
13744     outerRing: Color;
13745     midRing: Color;
13746     dish: Color;

```

```

13747     dishEdge: Color;
13748     tick: Color;
13749     bracket: Color;
13750     sensorDim: Color;
13751     sensorEdge: Color;
13752     ledOn: Color;
13753     ledOff: Color;
13754 }{
13755     const beam = beamColorFromKey(colorKey);
13756     const accent = new Color(
13757         Math.floor(beam.r * 0.72),
13758         Math.floor(beam.g * 0.72),
13759         Math.floor(beam.b * 0.72),
13760         255,
13761     );
13762     const chassis = new Color(
13763         Math.floor(beam.r * 0.1 + 14),
13764         Math.floor(beam.g * 0.1 + 18),
13765         Math.floor(beam.b * 0.1 + 26),
13766         255,
13767     );
13768     const chassisEdge = new Color(
13769         Math.floor(beam.r * 0.32 + 34),
13770         Math.floor(beam.g * 0.32 + 42),
13771         Math.floor(beam.b * 0.32 + 54),
13772         255,
13773     );
13774     if (lit) {
13775         return {
13776             beam,
13777             glow: new Color(beam.r, beam.g, beam.b, 100),
13778             base: chassis,
13779             panel: new Color(
13780                 Math.floor(beam.r * 0.16 + 20),
13781                 Math.floor(beam.g * 0.16 + 24),
13782                 Math.floor(beam.b * 0.16 + 32),
13783                 255,
13784             ),
13785             outerRing: new Color(beam.r, beam.g, beam.b, 220),
13786             midRing: accent,
13787             dish: new Color(
13788                 Math.floor(beam.r * 0.22 + 18),
13789                 Math.floor(beam.g * 0.22 + 22),
13790                 Math.floor(beam.b * 0.22 + 30),
13791                 255,
13792             ),
13793             dishEdge: chassisEdge,
13794             tick: new Color(255, 255, 255, 180),
13795             bracket: new Color(beam.r, beam.g, beam.b, 200),
13796             sensorDim: new Color(0, 0, 0, 0),
13797             sensorEdge: new Color(0, 0, 0, 0),
13798             ledOn: sourceFillColor(colorKey),
13799             ledOff: new Color(0, 0, 0, 0),
13800         };
13801     }
13802     const dim = targetDimFillColor(colorKey);
13803     return {
13804         beam,

```

```

13805     glow: new Color(beam.r, beam.g, beam.b, 40),
13806     base: chassis,
13807     panel: new Color(
13808         Math.floor(dim.r * 0.35 + 28),
13809         Math.floor(dim.g * 0.35 + 32),
13810         Math.floor(dim.b * 0.35 + 38),
13811         255,
13812     ),
13813     outerRing: new Color(
13814         Math.floor(dim.r * 0.85),
13815         Math.floor(dim.g * 0.85),
13816         Math.floor(dim.b * 0.85),
13817         200,
13818     ),
13819     midRing: new Color(
13820         Math.floor(dim.r * 0.65),
13821         Math.floor(dim.g * 0.65),
13822         Math.floor(dim.b * 0.65),
13823         180,
13824     ),
13825     dish: new Color(
13826         Math.floor(dim.r * 0.45 + 24),
13827         Math.floor(dim.g * 0.45 + 28),
13828         Math.floor(dim.b * 0.45 + 34),
13829         255,
13830     ),
13831     dishEdge: chassisEdge,
13832     tick: new Color(
13833         Math.floor(dim.r * 0.55),
13834         Math.floor(dim.g * 0.55),
13835         Math.floor(dim.b * 0.55),
13836         140,
13837     ),
13838     bracket: new Color(
13839         Math.floor(dim.r * 0.7),
13840         Math.floor(dim.g * 0.7),
13841         Math.floor(dim.b * 0.7),
13842         160,
13843     ),
13844     sensorDim: new Color(
13845         Math.floor(dim.r * 0.55),
13846         Math.floor(dim.g * 0.55),
13847         Math.floor(dim.b * 0.55),
13848         200,
13849     ),
13850     sensorEdge: accent,
13851     ledOn: new Color(0, 0, 0, 0),
13852     ledOff: new Color(
13853         Math.floor(dim.r * 0.4 + 40),
13854         Math.floor(dim.g * 0.4 + 44),
13855         Math.floor(dim.b * 0.4 + 50),
13856         120,
13857     ),
13858 };
13859 }
13860 /** 目标未点亮：浅色弱化版，与地板对比可见，点亮后会更亮更饱和 */
13861 export function targetDimFillColor(colorKey?: string): Color {
13862     switch (resolveBeamColorKey(colorKey)) {

```

```

13863         case 'red':
13864             return new Color(200, 158, 158, 255);
13865         case 'green':
13866             return new Color(158, 198, 168, 255);
13867         case 'blue':
13868             return new Color(158, 178, 210, 255);
13869         case 'yellow':
13870             return new Color(210, 198, 148, 255);
13871         case 'cyan':
13872             return new Color(148, 198, 205, 255);
13873         case 'purple':
13874             return new Color(188, 168, 210, 255);
13875         default:
13876             return new Color(198, 202, 214, 255);
13877     }
13878 }
13879 /** 目标未点亮中心灯体色（与 TargetGlyph 一致） */
13880 export function targetUnlitBulbFillColor(colorKey?: string): Color {
13881     const dim = targetDimFillColor(colorKey);
13882     return new Color(
13883         Math.floor(dim.r * 0.76),
13884         Math.floor(dim.g * 0.76),
13885         Math.floor(dim.b * 0.76),
13886         225,
13887     );
13888 }
13889 /** 通道元件臂/描边色（层 3 `W` 为中性灰，RGB 为对应染色） */
13890 export function pieceChannelColors(colorKey?: string): { stroke: Color; fill: Color } {
13891     switch (normalizeLightColorKey(colorKey)) {
13892         case 'red':
13893             return {
13894                 stroke: new Color(255, 110, 110, 255),
13895                 fill: new Color(255, 110, 110, 120),
13896             };
13897         case 'green':
13898             return {
13899                 stroke: new Color(100, 230, 130, 255),
13900                 fill: new Color(100, 230, 130, 120),
13901             };
13902         case 'blue':
13903             return {
13904                 stroke: new Color(100, 160, 255, 255),
13905                 fill: new Color(100, 160, 255, 120),
13906             };
13907         default:
13908             return {
13909                 stroke: new Color(200, 206, 220, 255),
13910                 fill: new Color(200, 206, 220, 120),
13911             };
13912     }
13913 }
13914 /** 通道元件格底：同色系更深、高透明，光路可透出 */
13915 export function pieceBaseFillColor(colorKey?: string): Color {
13916     const { stroke } = pieceChannelColors(colorKey);
13917     return new Color(
13918         Math.floor(stroke.r * 0.68 + 50),
13919         Math.floor(stroke.g * 0.68 + 50),
13920         Math.floor(stroke.b * 0.68 + 50),

```



```

13921         80,
13922     );
13923 }
13924 /** 主角格底：橘红色纯色 */
13925 export function playerBaseFillColor(): Color {
13926     return new Color(255, 92, 58, 255);
13927 }
13928 /** 主角 bezel 外环（与 Target 外缘结构同色，纯黑） */
13929 export function playerBaseBorderColor(): Color {
13930     return new Color(0, 0, 0, 255);
13931 }
13932 /** 主角眼镜片：深橘色纯色 */
13933 export function playerEyeFillColor(): Color {
13934     return new Color(178, 58, 18, 255);
13935 }
13936 /** 目标已点亮（与期望色一致，含 Y/C/P） */
13937 export function targetLitFillColor(colorKey?: string): Color {
13938     switch (resolveBeamColorKey(colorKey)) {
13939         case 'red':
13940             return new Color(255, 100, 100, 255);
13941         case 'green':
13942             return new Color(100, 235, 140, 255);
13943         case 'blue':
13944             return new Color(100, 165, 255, 255);
13945         case 'yellow':
13946             return new Color(255, 220, 72, 255);
13947         case 'cyan':
13948             return new Color(72, 220, 230, 255);
13949         case 'purple':
13950             return new Color(180, 90, 255, 255);
13951         default:
13952             return new Color(250, 252, 255, 255);
13953     }
13954 }
13955 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleDirButtonGlyph.ts -----
13956 import { Color, Graphics } from 'cc';
13957 import { Direction } from '../Core/OpticalPuzzleTypes';
13958 import {
13959     drawHudButtonChrome,
13960     drawHudButtonChromeTutorialHint,
13961     drawHudShapelcon,
13962     HUD_BUTTON_DESIGN_SIZE,
13963     HUD_ICON_BORDER_DESIGN,
13964     scaleHudDesign,
13965     unlitHudIconColor,
13966 } from './OpticalPuzzleHudButtonCommon';
13967 import {
13968     HUD_DIR_ARROW_SIZE_RATIO,
13969     traceHudSvgDirectionArrow,
13970     drawHudSvgArrowIcon,
13971 } from './OpticalPuzzleHudArrowGlyph';
13972 /** 键帽设计尺寸（与场景 UITransform 一致） */
13973 export const DIR_BUTTON_DESIGN_SIZE = HUD_BUTTON_DESIGN_SIZE;
13974 const ARROW_STROKE = new Color(255, 255, 255, 255);
13975 /** 绘制单枚方向键：80×80 圆角底 + SVG 圆角三角箭头 */
13976 export function drawDirButtonGlyph(
13977     g: Graphics,
13978     left: number,

```

```

13979         bottom: number,
13980         width: number,
13981         height: number,
13982         direction: Direction,
13983         pressed = false,
13984         tutorialBreathT: number | null = null,
13985     ): void {
13986         const size = Math.min(width, height);
13987         const arrowBorderW = Math.max(1, scaleHudDesign(size, HUD_ICON_BORDER_DESIGN));
13988         if (pressed) {
13989             drawHudButtonChrome(g, left, bottom, width, height, true);
13990         } else if (tutorialBreathT != null) {
13991             drawHudButtonChromeTutorialHint(g, left, bottom, width, height, tutorialBreathT);
13992         } else {
13993             drawHudButtonChrome(g, left, bottom, width, height, false);
13994         }
13995         const cx = left + width * 0.5;
13996         const cy = bottom + height * 0.5;
13997         if (pressed) {
13998             drawHudShapelcon(g, size, true, () =>
13999                 traceHudSvgDirectionArrow(g, cx, cy, size, direction, HUD_DIR_ARROW_SIZE_RATIO),
14000             );
14001             return;
14002         }
14003         drawHudSvgArrowlcon(
14004             g,
14005             cx,
14006             cy,
14007             size,
14008             direction,
14009             unlitHudlconColor(),
14010             ARROW_STROKE,
14011             arrowBorderW,
14012             HUD_DIR_ARROW_SIZE_RATIO,
14013         );
14014     }
14015 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleDirButtonView.ts -----
14016 import {
14017     _decorator,
14018     Component,
14019     Graphics,
14020     Node,
14021     UITransform,
14022 } from 'cc';
14023 import { Direction } from '../Core/OpticalPuzzleTypes';
14024 import { drawDirButtonGlyph, DIR_BUTTON_DESIGN_SIZE } from './OpticalPuzzleDirButtonGlyph';
14025 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
14026 const { ccclass, property } = _decorator;
14027 /** 教程黄呼吸光晕周期（秒） */
14028 const TUTORIAL_BREATH_PERIOD_SEC = 1.8;
14029 /** 四向虚拟键：按节点 UITransform 尺寸绘制键帽（位置由场景节点坐标决定） */
14030 @ccclass('OpticalPuzzleDirButtonView')
14031 export class OpticalPuzzleDirButtonView extends Component {
14032     /** 留空则按节点名 BtnUp / BtnDown / BtnLeft / BtnRight 推断 */
14033     @property
14034     direction: Direction = Direction.Up;
14035     private _graphics: Graphics | null = null;
14036     private _pressed = false;

```

```

14037     private _tutorialHint = false;
14038     private _breathPhase = 0;
14039     private _pressCtrl: HudButtonPressController | null = null;
14040     protected onLoad(): void {
14041         this.direction = this._resolveDirection(this.direction, this.node.name);
14042         this._ensureTransform();
14043         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
14044             this._pressed = pressed;
14045             this._redraw();
14046         });
14047         this._redraw();
14048     }
14049     protected onEnable(): void {
14050         this._pressCtrl?.bind();
14051         this._redraw();
14052     }
14053     protected onDisable(): void {
14054         this._pressCtrl?.unbind();
14055         this._pressed = false;
14056     }
14057     protected update(dt: number): void {
14058         if (!this._tutorialHint || this._pressed) {
14059             return;
14060         }
14061         this._breathPhase += dt;
14062         this._redraw();
14063     }
14064     protected onDestroy(): void {
14065         this._pressCtrl?.unbind();
14066     }
14067     /** 改 UITransform 尺寸后可在编辑器调用来刷新 */
14068     refresh(): void {
14069         this._redraw();
14070     }
14071     /** 第 1 关步数 0 等教程态：黄框黄呼吸光晕；按下时仍走默认白描边/白光晕 */
14072     setTutorialHint(active: boolean): void {
14073         if (this._tutorialHint === active) {
14074             return;
14075         }
14076         this._tutorialHint = active;
14077         this._breathPhase = 0;
14078         this._redraw();
14079     }
14080     /** 键盘等非触摸输入：短暂显示按压发光（触摸中则已由 HudButtonPressController 驱动） */
14081     flashPressed(durationSec = 0.1): void {
14082         if (this._pressCtrl?.touchActive) {
14083             return;
14084         }
14085         this._pressed = true;
14086         this._redraw();
14087         this.unschedule(this._releaseKeyboardFlash);
14088         this.scheduleOnce(this._releaseKeyboardFlash, durationSec);
14089     }
14090     private _releaseKeyboardFlash = (): void => {
14091         if (this._pressCtrl?.touchActive) {
14092             return;
14093         }
14094         this._pressed = false;

```

```

14095         this._redraw();
14096     };
14097     private _ensureTransform(): void {
14098         let ut = this.getComponent(UITransform);
14099         if (!ut) {
14100             ut = this.addComponent(UITransform);
14101             ut.setContentSize(DIR_BUTTON_DESIGN_SIZE, DIR_BUTTON_DESIGN_SIZE);
14102         }
14103     }
14104     private _ensureGraphics(): Graphics | null {
14105         if (!this._graphics?.isValid) {
14106             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
14107         }
14108         return this._graphics;
14109     }
14110     private _redraw(): void {
14111         const g = this._ensureGraphics();
14112         const ut = this.getComponent(UITransform);
14113         if (!g || !ut) {
14114             return;
14115         }
14116         g.clear();
14117         const w = ut.width;
14118         const h = ut.height;
14119         const left = -ut.anchorX * w;
14120         const bottom = -ut.anchorY * h;
14121         const tutorialBreathT =
14122             !this._pressed && this._tutorialHint ? this._sampleTutorialBreathT() : null;
14123         drawDirButtonGlyph(g, left, bottom, w, h, this.direction, this._pressed, tutorialBreathT);
14124     }
14125     private _sampleTutorialBreathT(): number {
14126         const phase = (this._breathPhase / TUTORIAL_BREATH_PERIOD_SEC) * Math.PI * 2;
14127         return 0.5 + 0.5 * Math.sin(phase);
14128     }
14129     private _resolveDirection(fallback: Direction, nodeName: string): Direction {
14130         const key = nodeName.toLowerCase();
14131         if (key.includes('up')) {
14132             return Direction.Up;
14133         }
14134         if (key.includes('down')) {
14135             return Direction.Down;
14136         }
14137         if (key.includes('left')) {
14138             return Direction.Left;
14139         }
14140         if (key.includes('right')) {
14141             return Direction.Right;
14142         }
14143         return fallback;
14144     }
14145 }
14146 /** 为 DirPad 下四节点批量挂上绘制（InputHud 自动绑定前调用） */
14147 export function ensureDirButtonViews(dirPad: Node | null): void {
14148     if (!dirPad?.isValid) {
14149         return;
14150     }
14151     for (const child of dirPad.children) {
14152         if (!child.getComponent(OpticalPuzzleDirButtonView)) {

```

```

14153         child.addComponent(OpticalPuzzleDirButtonView);
14154     }
14155 }
14156 }
14157 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleHudArrowGlyph.ts -----
14158 import { Color, Graphics } from 'cc';
14159 import { Direction } from '../Core/OpticalPuzzleTypes';
14160 import { HUD_ARROW_SVG_SEGS } from './OpticalPuzzleHudArrowSegs.generated';
14161 /** 四向键箭头占键帽边长比例 */
14162 export const HUD_DIR_ARROW_SIZE_RATIO = 0.45;
14163 /** 菜单开始键箭头占键帽边长比例 */
14164 export const HUD_MENU_START_ARROW_SIZE_RATIO = 0.42;
14165 function rotationRad(direction: Direction): number {
14166     switch (direction) {
14167         case Direction.Up:
14168             return Math.PI * 0.5;
14169         case Direction.Down:
14170             return -Math.PI * 0.5;
14171         case Direction.Left:
14172             return Math.PI;
14173         default:
14174             return 0;
14175     }
14176 }
14177 /** 右向 SVG 外轮廓 → 按方向旋转后描路径（不含内孔 subpath） */
14178 export function traceHudSvgDirectionArrow(
14179     g: Graphics,
14180     cx: number,
14181     cy: number,
14182     size: number,
14183     direction: Direction,
14184     sizeRatio: number = HUD_DIR_ARROW_SIZE_RATIO,
14185     scaleMul = 1,
14186 ): void {
14187     const height = size * sizeRatio * scaleMul;
14188     const rot = rotationRad(direction);
14189     const cos = Math.cos(rot);
14190     const sin = Math.sin(rot);
14191     for (let i = 0; i < HUD_ARROW_SVG_SEGS.length; i++) {
14192         const p = HUD_ARROW_SVG_SEGS[i];
14193         const lx = p.x * height;
14194         const ly = p.y * height;
14195         const x = cx + lx * cos - ly * sin;
14196         const y = cy + lx * sin + ly * cos;
14197         if (i === 0) {
14198             g.moveTo(x, y);
14199         } else {
14200             g.lineTo(x, y);
14201         }
14202     }
14203     g.close();
14204 }
14205 /**
14206  * 外轮廓 fill + stroke: 描边沿路径法线等宽; 用 MITER/BUTT 避免 ROUND join 在顶点叠圆球。
14207  */
14208 export function drawHudSvgArrowIcon(
14209     g: Graphics,
14210     cx: number,

```

```

14211         cy: number,
14212         size: number,
14213         direction: Direction,
14214         fillColor: Color,
14215         borderColor: Color,
14216         borderPx: number,
14217         sizeRatio: number = HUD_DIR_ARROW_SIZE_RATIO,
14218     ): void {
14219         const trace = (): void => {
14220             traceHudSvgDirectionArrow(g, cx, cy, size, direction, sizeRatio);
14221         };
14222         g.fillColor = fillColor;
14223         trace();
14224         g.fill();
14225         g.strokeColor = borderColor;
14226         g.lineWidth = Math.max(1, borderPx);
14227         g.lineJoin = Graphics.LineJoin.MITER;
14228         g.lineCap = Graphics.LineCap.BUTT;
14229         trace();
14230         g.stroke();
14231     }
14232 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleHudArrowSegs.generated.ts -----
14233 // AUTO-GENERATED by tools/gen-hud-arrow-segs.py — do not edit by hand
14234 /** SVG 外轮廓采样 (svg.path): y 已翻转, 按高度归一化, 原点为面积重心 */
14235 export const HUD_ARROW_SVG_SEGS: ReadonlyArray<Readonly<{ x: number; y: number }>> = [
14236     { x: -0.21209399, y: -0.50008827 },
14237     { x: -0.22398399, y: -0.49948399 },
14238     { x: -0.23577252, y: -0.49782028 },
14239     { x: -0.24736536, y: -0.49511045 },
14240     { x: -0.25866988, y: -0.49137614 },
14241     { x: -0.26959574, y: -0.48664720 },
14242     { x: -0.28005562, y: -0.48096142 },
14243     { x: -0.28996594, y: -0.47436423 },
14244     { x: -0.29924749, y: -0.46690836 },
14245     { x: -0.30782611, y: -0.45865339 },
14246     { x: -0.31563324, y: -0.44966530 },
14247     { x: -0.32260650, y: -0.44001589 },
14248     { x: -0.32869014, y: -0.42978230 },
14249     { x: -0.33383557, y: -0.41904629 },
14250     { x: -0.33800166, y: -0.40789367 },
14251     { x: -0.34115511, y: -0.39641356 },
14252     { x: -0.34327072, y: -0.38469770 },
14253     { x: -0.34433160, y: -0.37283971 },
14254     { x: -0.34446108, y: -0.36093235 },
14255     { x: -0.34446108, y: -0.34902304 },
14256     { x: -0.34446108, y: -0.33711373 },
14257     { x: -0.34446108, y: -0.32520442 },
14258     { x: -0.34446108, y: -0.31329510 },
14259     { x: -0.34446108, y: -0.30138579 },
14260     { x: -0.34446108, y: -0.28947648 },
14261     { x: -0.34446108, y: -0.27756717 },
14262     { x: -0.34446108, y: -0.26565785 },
14263     { x: -0.34446108, y: -0.25374854 },
14264     { x: -0.34446108, y: -0.24183923 },
14265     { x: -0.34446108, y: -0.22992992 },
14266     { x: -0.34446108, y: -0.21802060 },
14267     { x: -0.34446108, y: -0.20611129 },
14268     { x: -0.34446108, y: -0.19420198 },

```

14269 { x: -0.34446108, y: -0.18229267 },
14270 { x: -0.34446108, y: -0.17038335 },
14271 { x: -0.34446108, y: -0.15847404 },
14272 { x: -0.34446108, y: -0.14656473 },
14273 { x: -0.34446108, y: -0.13465542 },
14274 { x: -0.34446108, y: -0.12274610 },
14275 { x: -0.34446108, y: -0.11083679 },
14276 { x: -0.34446108, y: -0.09892748 },
14277 { x: -0.34446108, y: -0.08701817 },
14278 { x: -0.34446108, y: -0.07510885 },
14279 { x: -0.34446108, y: -0.06319954 },
14280 { x: -0.34446108, y: -0.05129023 },
14281 { x: -0.34446108, y: -0.03938092 },
14282 { x: -0.34446108, y: -0.02747160 },
14283 { x: -0.34446108, y: -0.01556229 },
14284 { x: -0.34446108, y: -0.00365298 },
14285 { x: -0.34446108, y: 0.00825633 },
14286 { x: -0.34446108, y: 0.02016565 },
14287 { x: -0.34446108, y: 0.03207496 },
14288 { x: -0.34446108, y: 0.04398427 },
14289 { x: -0.34446108, y: 0.05589358 },
14290 { x: -0.34446108, y: 0.06780290 },
14291 { x: -0.34446108, y: 0.07971221 },
14292 { x: -0.34446108, y: 0.09162152 },
14293 { x: -0.34446108, y: 0.10353083 },
14294 { x: -0.34446108, y: 0.11544015 },
14295 { x: -0.34446108, y: 0.12734946 },
14296 { x: -0.34446108, y: 0.13925877 },
14297 { x: -0.34446108, y: 0.15116808 },
14298 { x: -0.34446108, y: 0.16307740 },
14299 { x: -0.34446108, y: 0.17498671 },
14300 { x: -0.34446108, y: 0.18689602 },
14301 { x: -0.34446108, y: 0.19880533 },
14302 { x: -0.34446108, y: 0.21071465 },
14303 { x: -0.34446108, y: 0.22262396 },
14304 { x: -0.34446108, y: 0.23453327 },
14305 { x: -0.34446108, y: 0.24644258 },
14306 { x: -0.34446108, y: 0.25835190 },
14307 { x: -0.34446108, y: 0.27026121 },
14308 { x: -0.34446108, y: 0.28217052 },
14309 { x: -0.34446108, y: 0.29407983 },
14310 { x: -0.34446108, y: 0.30598915 },
14311 { x: -0.34446108, y: 0.31789846 },
14312 { x: -0.34446108, y: 0.32980777 },
14313 { x: -0.34446108, y: 0.34171708 },
14314 { x: -0.34446108, y: 0.35362640 },
14315 { x: -0.34446108, y: 0.36553571 },
14316 { x: -0.34413746, y: 0.37743736 },
14317 { x: -0.34276489, y: 0.38926323 },
14318 { x: -0.34033102, y: 0.40091706 },
14319 { x: -0.33685569, y: 0.41230377 },
14320 { x: -0.33236724, y: 0.42333052 },
14321 { x: -0.32690230, y: 0.43390737 },
14322 { x: -0.32050542, y: 0.44394804 },
14323 { x: -0.31322877, y: 0.45337066 },
14324 { x: -0.30513172, y: 0.46209837 },
14325 { x: -0.29628028, y: 0.47006000 },
14326 { x: -0.28674667, y: 0.47719060 },

14327 { x: -0.27660862, y: 0.48343202 },
14328 { x: -0.26594883, y: 0.48873335 },
14329 { x: -0.25485424, y: 0.49305137 },
14330 { x: -0.24341532, y: 0.49635085 },
14331 { x: -0.23172539, y: 0.49860487 },
14332 { x: -0.21987977, y: 0.49979507 },
14333 { x: -0.20797507, y: 0.49991173 },
14334 { x: -0.19610841, y: 0.49895390 },
14335 { x: -0.18437654, y: 0.49692939 },
14336 { x: -0.17287516, y: 0.49385471 },
14337 { x: -0.16169808, y: 0.48975494 },
14338 { x: -0.15093645, y: 0.48466352 },
14339 { x: -0.14058110, y: 0.47878304 },
14340 { x: -0.13026688, y: 0.47282917 },
14341 { x: -0.11995267, y: 0.46687529 },
14342 { x: -0.10963845, y: 0.46092141 },
14343 { x: -0.09932423, y: 0.45496753 },
14344 { x: -0.08901002, y: 0.44901366 },
14345 { x: -0.07869580, y: 0.44305978 },
14346 { x: -0.06838158, y: 0.43710590 },
14347 { x: -0.05806737, y: 0.43115203 },
14348 { x: -0.04775315, y: 0.42519815 },
14349 { x: -0.03743893, y: 0.41924427 },
14350 { x: -0.02712472, y: 0.41329039 },
14351 { x: -0.01681050, y: 0.40733652 },
14352 { x: -0.00649628, y: 0.40138264 },
14353 { x: 0.00381793, y: 0.39542876 },
14354 { x: 0.01413215, y: 0.38947488 },
14355 { x: 0.02444637, y: 0.38352101 },
14356 { x: 0.03476059, y: 0.37756713 },
14357 { x: 0.04507480, y: 0.37161325 },
14358 { x: 0.05538902, y: 0.36565938 },
14359 { x: 0.06570324, y: 0.35970550 },
14360 { x: 0.07601745, y: 0.35375162 },
14361 { x: 0.08633167, y: 0.34779774 },
14362 { x: 0.09664589, y: 0.34184387 },
14363 { x: 0.10696010, y: 0.33588999 },
14364 { x: 0.11727432, y: 0.32993611 },
14365 { x: 0.12758854, y: 0.32398223 },
14366 { x: 0.13790275, y: 0.31802836 },
14367 { x: 0.14821697, y: 0.31207448 },
14368 { x: 0.15853119, y: 0.30612060 },
14369 { x: 0.16884540, y: 0.30016672 },
14370 { x: 0.17915962, y: 0.29421285 },
14371 { x: 0.18947384, y: 0.28825897 },
14372 { x: 0.19978805, y: 0.28230509 },
14373 { x: 0.21010227, y: 0.27635122 },
14374 { x: 0.22041649, y: 0.27039734 },
14375 { x: 0.23073071, y: 0.26444346 },
14376 { x: 0.24104492, y: 0.25848958 },
14377 { x: 0.25135914, y: 0.25253571 },
14378 { x: 0.26167336, y: 0.24658183 },
14379 { x: 0.27198757, y: 0.24062795 },
14380 { x: 0.28230179, y: 0.23467407 },
14381 { x: 0.29261601, y: 0.22872020 },
14382 { x: 0.30293022, y: 0.22276632 },
14383 { x: 0.31324444, y: 0.21681244 },
14384 { x: 0.32355866, y: 0.21085857 },

14385 { x: 0.33387287, y: 0.20490469 },
14386 { x: 0.34418709, y: 0.19895081 },
14387 { x: 0.35450131, y: 0.19299693 },
14388 { x: 0.36481552, y: 0.18704306 },
14389 { x: 0.37512974, y: 0.18108918 },
14390 { x: 0.38544396, y: 0.17513530 },
14391 { x: 0.39575817, y: 0.16918142 },
14392 { x: 0.40607239, y: 0.16322755 },
14393 { x: 0.41638661, y: 0.15727367 },
14394 { x: 0.42670083, y: 0.15131979 },
14395 { x: 0.43701504, y: 0.14536592 },
14396 { x: 0.44732926, y: 0.13941204 },
14397 { x: 0.45764348, y: 0.13345816 },
14398 { x: 0.46795769, y: 0.12750428 },
14399 { x: 0.47827191, y: 0.12155041 },
14400 { x: 0.48858613, y: 0.11559653 },
14401 { x: 0.49872669, y: 0.10935766 },
14402 { x: 0.50827786, y: 0.10225057 },
14403 { x: 0.51715037, y: 0.09431241 },
14404 { x: 0.52527214, y: 0.08560767 },
14405 { x: 0.53257721, y: 0.07620705 },
14406 { x: 0.53900624, y: 0.06618691 },
14407 { x: 0.54450700, y: 0.05562863 },
14408 { x: 0.54903482, y: 0.04461797 },
14409 { x: 0.55255293, y: 0.03324438 },
14410 { x: 0.55503273, y: 0.02160023 },
14411 { x: 0.55645410, y: 0.00978010 },
14412 { x: 0.55680549, y: -0.00211999 },
14413 { x: 0.55608404, y: -0.01400339 },
14414 { x: 0.55429561, y: -0.02577357 },
14415 { x: 0.55145473, y: -0.03733494 },
14416 { x: 0.54758447, y: -0.04859357 },
14417 { x: 0.54271628, y: -0.05945802 },
14418 { x: 0.53688969, y: -0.06984005 },
14419 { x: 0.53015203, y: -0.07965533 },
14420 { x: 0.52255803, y: -0.08882412 },
14421 { x: 0.51416937, y: -0.09727196 },
14422 { x: 0.50505418, y: -0.10493023 },
14423 { x: 0.49528652, y: -0.11173673 },
14424 { x: 0.48502621, y: -0.11778118 },
14425 { x: 0.47471199, y: -0.12373506 },
14426 { x: 0.46439778, y: -0.12968893 },
14427 { x: 0.45408356, y: -0.13564281 },
14428 { x: 0.44376934, y: -0.14159669 },
14429 { x: 0.43345513, y: -0.14755056 },
14430 { x: 0.42314091, y: -0.15350444 },
14431 { x: 0.41282669, y: -0.15945832 },
14432 { x: 0.40251247, y: -0.16541220 },
14433 { x: 0.39219826, y: -0.17136607 },
14434 { x: 0.38188404, y: -0.17731995 },
14435 { x: 0.37156982, y: -0.18327383 },
14436 { x: 0.36125561, y: -0.18922771 },
14437 { x: 0.35094139, y: -0.19518158 },
14438 { x: 0.34062717, y: -0.20113546 },
14439 { x: 0.33031296, y: -0.20708934 },
14440 { x: 0.31999874, y: -0.21304321 },
14441 { x: 0.30968452, y: -0.21899709 },
14442 { x: 0.29937031, y: -0.22495097 },

```
14443      { x: 0.28905609, y: -0.23090485 },
14444      { x: 0.27874187, y: -0.23685872 },
14445      { x: 0.26842766, y: -0.24281260 },
14446      { x: 0.25811344, y: -0.24876648 },
14447      { x: 0.24779922, y: -0.25472036 },
14448      { x: 0.23748501, y: -0.26067423 },
14449      { x: 0.22717079, y: -0.26662811 },
14450      { x: 0.21685657, y: -0.27258199 },
14451      { x: 0.20654235, y: -0.27853586 },
14452      { x: 0.19622814, y: -0.28448974 },
14453      { x: 0.18591392, y: -0.29044362 },
14454      { x: 0.17559970, y: -0.29639750 },
14455      { x: 0.16528549, y: -0.30235137 },
14456      { x: 0.15497127, y: -0.30830525 },
14457      { x: 0.14465705, y: -0.31425913 },
14458      { x: 0.13434284, y: -0.32021301 },
14459      { x: 0.12402862, y: -0.32616688 },
14460      { x: 0.11371440, y: -0.33212076 },
14461      { x: 0.10340019, y: -0.33807464 },
14462      { x: 0.09308597, y: -0.34402852 },
14463      { x: 0.08277175, y: -0.34998239 },
14464      { x: 0.07245754, y: -0.35593627 },
14465      { x: 0.06214332, y: -0.36189015 },
14466      { x: 0.05182910, y: -0.36784402 },
14467      { x: 0.04151489, y: -0.37379790 },
14468      { x: 0.03120067, y: -0.37975178 },
14469      { x: 0.02088645, y: -0.38570566 },
14470      { x: 0.01057223, y: -0.39165953 },
14471      { x: 0.00025802, y: -0.39761341 },
14472      { x: -0.01005620, y: -0.40356729 },
14473      { x: -0.02037042, y: -0.40952117 },
14474      { x: -0.03068463, y: -0.41547504 },
14475      { x: -0.04099885, y: -0.42142892 },
14476      { x: -0.05131307, y: -0.42738280 },
14477      { x: -0.06162728, y: -0.43333667 },
14478      { x: -0.07194150, y: -0.43929055 },
14479      { x: -0.08225572, y: -0.44524443 },
14480      { x: -0.09256993, y: -0.45119831 },
14481      { x: -0.10288415, y: -0.45715218 },
14482      { x: -0.11319837, y: -0.46310606 },
14483      { x: -0.12351258, y: -0.46905994 },
14484      { x: -0.13382680, y: -0.47501382 },
14485      { x: -0.14414102, y: -0.48096769 },
14486      { x: -0.15460072, y: -0.48665562 },
14487      { x: -0.16551832, y: -0.49140338 },
14488      { x: -0.17681974, y: -0.49514682 },
14489      { x: -0.18841279, y: -0.49785540 },
14490      { x: -0.20020293, y: -0.49950705 },
14491 ];
14492 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonCommon.ts -----
14493 import { Color, Graphics } from 'cc';
14494 import { targetDimFillColor } from './OpticalPuzzleColorUtil';
14495 export { HudButtonPressController } from './OpticalPuzzleHudButtonPressController';
14496 /** HUD 虚拟键设计尺寸（与场景 UITransform 一致） */
14497 export const HUD_BUTTON_DESIGN_SIZE = 80;
14498 /** 局内四向键场景边长（100×100）；非正方区域外框线宽/圆角与之对齐时用 */
14499 export const HUD_DIR_BUTTON_SCENE_SIZE = 100;
14500 /** 外框圆角（设计 px） */
```

```

14501 export const HUD_KEY_CORNER_DESIGN = 15;
14502 /** 外框线宽（设计 px） */
14503 export const HUD_KEY_BORDER_DESIGN = 2;
14504 /** 内部面型图标描边（设计 px，三角箭头等） */
14505 export const HUD_ICON_BORDER_DESIGN = 3;
14506 /** 内部线型图标白描边（设计 px，撤回/重置圆弧） */
14507 export const HUD_LINE_ICON_BORDER_DESIGN = 2;
14508 /** 内部线型图标体宽（设计 px） */
14509 export const HUD_ICON_BODY_DESIGN = 5;
14510 /** 与未点亮目标内面板一致 #161c26（浅黑底） */
14511 export const HUD_KEY_FILL = new Color(22, 28, 38, 255);
14512 const KEY_FILL = HUD_KEY_FILL;
14513 const KEY_BORDER = new Color(255, 255, 255, 255);
14514 /** 按下时白光层（设计 px 线宽 → alpha，外框与图标共用） */
14515 export const PRESSED_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
14516     { width: 9, alpha: 48 },
14517     { width: 6, alpha: 120 },
14518     { width: 3, alpha: 255 },
14519 ];
14520 /** 第 1 关步数 0 时上方向键教程提示：黄框 + 黄呼吸光晕 */
14521 export const TUTORIAL_HINT_GLOW_RGB = { r: 255, g: 220, b: 80 } as const;
14522 export const TUTORIAL_HINT_BORDER = new Color(255, 220, 80, 255);
14523 const TUTORIAL_BREATH_GLOW_TEMPLATE: ReadonlyArray<{
14524     width: number;
14525     alphaMin: number;
14526     alphaMax: number;
14527     widthBreath: number;
14528 }> = [
14529     { width: 2.5, alphaMin: 36, alphaMax: 120, widthBreath: 3.5 },
14530     { width: 4.5, alphaMin: 22, alphaMax: 80, widthBreath: 5.5 },
14531     { width: 7.5, alphaMin: 10, alphaMax: 40, widthBreath: 7.5 },
14532 ];
14533 /** breathT ∈ [0,1]：光晕线宽与 alpha 随呼吸扩散 */
14534 export function buildTutorialBreathGlowLayers(
14535     breathT: number,
14536 ): ReadonlyArray<{ width: number; alpha: number }> {
14537     const t = Math.max(0, Math.min(1, breathT));
14538     return TUTORIAL_BREATH_GLOW_TEMPLATE.map((layer) => ({
14539         width: layer.width + layer.widthBreath * t,
14540         alpha: Math.round(layer.alphaMin + (layer.alphaMax - layer.alphaMin) * t),
14541     }));
14542 }
14543 export function scaleHudDesign(size: number, design: number): number {
14544     return design * (size / HUD_BUTTON_DESIGN_SIZE);
14545 }
14546 /** 未点亮白灯中心浅白（与 TargetGlyph 灯体 fill 一致） */
14547 export function unlitHudIconColor(): Color {
14548     const dim = targetDimFillColor(undefined);
14549     return new Color(
14550         Math.floor(dim.r * 0.76),
14551         Math.floor(dim.g * 0.76),
14552         Math.floor(dim.b * 0.76),
14553         255,
14554     );
14555 }
14556 export function pressedHudIconColor(): Color {
14557     return new Color(255, 255, 255, 255);
14558 }

```

```

14559  /** 线型图标外轮廓设计线宽（体 + 两侧描边） */
14560  export function hudLineIconTotalDesign(): number {
14561      return HUD_ICON_BODY_DESIGN + HUD_LINE_ICON_BORDER_DESIGN * 2;
14562  }
14563  /** 线型实心笔划半宽（像素，用于外缘光晕路径外移） */
14564  export function hudLineHalfStrokeWidth(size: number): number {
14565      return scaleHudDesign(size, hudLineIconTotalDesign()) * 0.5;
14566  }
14567  export function strokeGlowLayers(
14568      g: Graphics,
14569      size: number,
14570      layers: ReadonlyArray<{ width: number; alpha: number }>,
14571      tracePath: () => void,
14572      roundCaps = false,
14573      glowRgb: Readonly<{ r: number; g: number; b: number }> = { r: 255, g: 255, b: 255 },
14574  ): void {
14575      if (roundCaps) {
14576          g.lineJoin = Graphics.LineJoin.ROUND;
14577          g.lineCap = Graphics.LineCap.ROUND;
14578      }
14579      for (const layer of layers) {
14580          g.strokeColor = new Color(glowRgb.r, glowRgb.g, glowRgb.b, layer.alpha);
14581          g.lineWidth = Math.max(1, scaleHudDesign(size, layer.width));
14582          tracePath();
14583          g.stroke();
14584      }
14585  }
14586  /**
14587   * 实心面型图标按下光晕：与 strokeGlowLayers 共用 width/alpha（扩散与颜色一致），
14588   * 外扩填充画在图标下方，避免尖角处描边叠线过曝。
14589   */
14590  export function fillGlowLayersMatchingStroke(
14591      g: Graphics,
14592      size: number,
14593      layers: ReadonlyArray<{ width: number; alpha: number }>,
14594      iconHalfPx: number,
14595      traceAtScale: (scaleMul: number) => void,
14596      glowRgb: Readonly<{ r: number; g: number; b: number }> = { r: 255, g: 255, b: 255 },
14597  ): void {
14598      const half = Math.max(iconHalfPx, 1);
14599      for (const layer of layers) {
14600          const halfGlow = scaleHudDesign(size, layer.width) * 0.5;
14601          const expand = 1 + halfGlow / half;
14602          g.fillColor = new Color(glowRgb.r, glowRgb.g, glowRgb.b, layer.alpha);
14603          traceAtScale(expand);
14604          g.fill();
14605      }
14606  }
14607  /**
14608   * 绘制键帽底 + 外框（与四向键一致）
14609   * @param chromeScaleSize 线宽/圆角缩放基准边长；省略时用 min(width,height)。Step 等传
14610   HUD_DIR_BUTTON_SCENE_SIZE 与 100×100 四向键对齐。
14611   */
14612  export function drawHudButtonChrome(
14613      g: Graphics,
14614      left: number,
14615      bottom: number,
14616      width: number,

```

```

14617     height: number,
14618     pressed: boolean,
14619     chromeScaleSize?: number,
14620 ): void {
14621     const size = chromeScaleSize ?? Math.min(width, height);
14622     const corner = scaleHudDesign(size, HUD_KEY_CORNER_DESIGN);
14623     const borderW = Math.max(1, scaleHudDesign(size, HUD_KEY_BORDER_DESIGN));
14624     g.fillColor = KEY_FILL;
14625     g.roundRect(left, bottom, width, height, corner);
14626     g.fill();
14627     if (pressed) {
14628         strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, () => {
14629             g.roundRect(left, bottom, width, height, corner);
14630         });
14631     } else {
14632         g.strokeColor = KEY_BORDER;
14633         g.lineWidth = borderW;
14634         g.roundRect(left, bottom, width, height, corner);
14635         g.stroke();
14636     }
14637 }
14638 /** 教程提示态键帽：浅黑底 + 黄色呼吸扩散光晕 + 黄描边（非按下） */
14639 export function drawHudButtonChromeTutorialHint(
14640     g: Graphics,
14641     left: number,
14642     bottom: number,
14643     width: number,
14644     height: number,
14645     breathT: number,
14646     chromeScaleSize?: number,
14647 ): void {
14648     const size = chromeScaleSize ?? Math.min(width, height);
14649     const corner = scaleHudDesign(size, HUD_KEY_CORNER_DESIGN);
14650     const borderW = Math.max(1, scaleHudDesign(size, HUD_KEY_BORDER_DESIGN));
14651     g.fillColor = KEY_FILL;
14652     g.roundRect(left, bottom, width, height, corner);
14653     g.fill();
14654     strokeGlowLayers(
14655         g,
14656         size,
14657         buildTutorialBreathGlowLayers(breathT),
14658         () => {
14659             g.roundRect(left, bottom, width, height, corner);
14660         },
14661         false,
14662         TUTORIAL_HINT_GLOW_RGB,
14663     );
14664     g.strokeColor = TUTORIAL_HINT_BORDER;
14665     g.lineWidth = borderW;
14666     g.roundRect(left, bottom, width, height, corner);
14667     g.stroke();
14668 }
14669 /** 线型图标：未按浅白体 + 白描边；按下全白 + 内外缘光晕（各缘半内半外扩散） */
14670 export function drawHudLineIcon(
14671     g: Graphics,
14672     size: number,
14673     pressed: boolean,
14674     tracePath: () => void,

```

```

14675         traceEdgeGlowPaths?: ReadonlyArray<() => void>,
14676     ): void {
14677         const bodyW = Math.max(1, scaleHudDesign(size, HUD_ICON_BODY_DESIGN));
14678         const borderW = Math.max(1, scaleHudDesign(size, HUD_LINE_ICON_BORDER_DESIGN));
14679         const totalW = bodyW + borderW * 2;
14680         g.lineJoin = Graphics.LineJoin.ROUND;
14681         g.lineCap = Graphics.LineCap.ROUND;
14682         if (pressed) {
14683             g.strokeColor = pressedHudIconColor();
14684             g.lineWidth = totalW;
14685             tracePath();
14686             g.stroke();
14687             if (traceEdgeGlowPaths) {
14688                 for (const traceGlow of traceEdgeGlowPaths) {
14689                     strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, traceGlow, true);
14690                 }
14691             }
14692             return;
14693         }
14694         g.strokeColor = KEY_BORDER;
14695         g.lineWidth = totalW;
14696         tracePath();
14697         g.stroke();
14698         g.strokeColor = unlitHudIconColor();
14699         g.lineWidth = bodyW;
14700         tracePath();
14701         g.stroke();
14702     }
14703     /** 面型图标：未按浅白填充 + 白描边；按下全白 + 光晕 */
14704     export function drawHudShapelcon(
14705         g: Graphics,
14706         size: number,
14707         pressed: boolean,
14708         tracePath: () => void,
14709     ): void {
14710         const borderW = Math.max(1, scaleHudDesign(size, HUD_ICON_BORDER_DESIGN));
14711         const fill = pressed ? pressedHudIconColor() : unlitHudIconColor();
14712         g.fillColor = fill;
14713         tracePath();
14714         g.fill();
14715         if (pressed) {
14716             strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, tracePath, true);
14717             return;
14718         }
14719         g.strokeColor = KEY_BORDER;
14720         g.lineWidth = borderW;
14721         g.lineJoin = Graphics.LineJoin.ROUND;
14722         g.lineCap = Graphics.LineCap.ROUND;
14723         tracePath();
14724         g.stroke();
14725     }
14726     // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController.ts -----
14727     import { EventTouch, Node, NodeEventType, UITransform } from 'cc';
14728     /** 触摸按下态：移出按钮区域时与 Button 缩放同步取消发光 */
14729     export class HudButtonPressController {
14730         private _pressed = false;
14731         private _touchActive = false;
14732         constructor(

```

```
14733     private readonly _node: Node,
14734     private readonly _onChange: (pressed: boolean) => void,
14735     /** 与 Button 同节点时须 true, 否则 Button 抢先消费触摸导致不发光 */
14736     private readonly _useCapture = false,
14737   ) {}
14738   bind(): void {
14739     if (!this._node?.isValid) {
14740       return;
14741     }
14742     this._unbindListeners();
14743     this._node.on(NodeEventType.TOUCH_START, this._onPressStart, this, this._useCapture);
14744     this._node.on(NodeEventType.TOUCH_MOVE, this._onTouchMove, this, this._useCapture);
14745     this._node.on(NodeEventType.TOUCH_END, this._onPressEnd, this, this._useCapture);
14746     this._node.on(NodeEventType.TOUCH_CANCEL, this._onPressEnd, this, this._useCapture);
14747   }
14748   /** 是否处于触摸按压中（键盘脉冲发光时勿覆盖） */
14749   get touchActive(): boolean {
14750     return this._touchActive;
14751   }
14752   unbind(): void {
14753     this._unbindListeners();
14754     this._touchActive = false;
14755     if (this._pressed) {
14756       this._pressed = false;
14757     }
14758   }
14759   private _unbindListeners(): void {
14760     if (!this._node?.isValid) {
14761       return;
14762     }
14763     this._node.off(NodeEventType.TOUCH_START, this._onPressStart, this, this._useCapture);
14764     this._node.off(NodeEventType.TOUCH_MOVE, this._onTouchMove, this, this._useCapture);
14765     this._node.off(NodeEventType.TOUCH_END, this._onPressEnd, this, this._useCapture);
14766     this._node.off(NodeEventType.TOUCH_CANCEL, this._onPressEnd, this, this._useCapture);
14767   }
14768   private _onPressStart(): void {
14769     if (!this._node?.isValid) {
14770       return;
14771     }
14772     this._touchActive = true;
14773     this._setPressed(true);
14774   }
14775   private _onTouchMove(event: EventTouch): void {
14776     if (!this._touchActive || !this._node?.isValid) {
14777       return;
14778     }
14779     this._setPressed(this._isTouchInside(event));
14780   }
14781   private _onPressEnd(): void {
14782     this._touchActive = false;
14783     if (!this._node?.isValid) {
14784       this._pressed = false;
14785       return;
14786     }
14787     this._setPressed(false);
14788   }
14789   private _isTouchInside(event: EventTouch): boolean {
14790     const ut = this._node.getComponent(UITransform);
```

```

14791         if (!ut) {
14792             return false;
14793         }
14794         return ut.getBoundingBoxToWorld().contains(event.getUILocation());
14795     }
14796     private _setPressed(pressed: boolean): void {
14797         if (this._pressed === pressed) {
14798             return;
14799         }
14800         this._pressed = pressed;
14801         this._onChange(pressed);
14802     }
14803 }
14804 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleInputHud.ts -----
14805 import {
14806     Button,
14807     Component,
14808     EventKeyboard,
14809     Input,
14810     KeyCode,
14811     Node,
14812     _decorator,
14813     input,
14814     sys,
14815 } from 'cc';
14816 import { OpticalPuzzleSession } from '../Application/OpticalPuzzleSession';
14817 import { OpticalGameFlowState } from '../Application/OpticalPuzzleStateMachine';
14818 import { Direction } from '../Core/OpticalPuzzleTypes';
14819 import { AUDIO_EFFECT_ENUM, EVENT_ENUM } from '../Utils/Enum';
14820 import { PLAY_AUDIO } from '../Utils/Event';
14821 import { showWeChatRewardedVideo } from '../Utils/WeChatRewardedVideoAd';
14822 import { openAnswerPanel, resolveAnswerPanelNode } from './OpticalPuzzleAnswerPanel';
14823 import { ensureActionButtonViews, OpticalPuzzleActionButtonView } from './OpticalPuzzleActionButtonView';
14824 import type { OpticalPuzzleBoardView } from './OpticalPuzzleBoardView';
14825 import { ensureDirButtonViews, OpticalPuzzleDirButtonView } from './OpticalPuzzleDirButtonView';
14826 const { ccclass, property } = _decorator;
14827 /**
14828  * 四向、撤回、重置。键盘：方向键 / WASD 移动，Z 撤回 1 步，R 重置（与 UI 按钮可同时使用）。
14829  */
14830 @ccclass('OpticalPuzzleInputHud')
14831 export class OpticalPuzzleInputHud extends Component {
14832     @property(Button)
14833     btnUp: Button | null = null;
14834     @property(Button)
14835     btnDown: Button | null = null;
14836     @property(Button)
14837     btnLeft: Button | null = null;
14838     @property(Button)
14839     btnRight: Button | null = null;
14840     @property(Button)
14841     btnUndo: Button | null = null;
14842     @property(Button)
14843     btnReset: Button | null = null;
14844     @property(Button)
14845     btnAnswer: Button | null = null;
14846     private _session: OpticalPuzzleSession | null = null;
14847     private _boardView: OpticalPuzzleBoardView | null = null;
14848     /** 激励广告拉起中，避免重复点击 */

```



```

14849     private _undoRewardAdPending = false;
14850     private _answerRewardAdPending = false;
14851     private _btnUndoBadge: Button | null = null;
14852     private _btnAnswerBadge: Button | null = null;
14853     protected onLoad(): void {
14854         this._autoBindButtons();
14855     }
14856     /** 未在检查器拖引用时，按 DirPad / ActionPad 子节点名自动绑定并补 Button */
14857     private _autoBindButtons(): void {
14858         const dirPad = this.node.getChildByName('DirPad');
14859         ensureDirButtonViews(dirPad);
14860         this.btnUp = this._resolveButton(dirPad, 'BtnUp', this.btnUp);
14861         this.btnDown = this._resolveButton(dirPad, 'BtnDown', this.btnDown);
14862         this.btnLeft = this._resolveButton(dirPad, 'BtnLeft', this.btnLeft);
14863         this.btnRight = this._resolveButton(dirPad, 'BtnRight', this.btnRight);
14864         const actionPad = this.node.getChildByName('ActionPad');
14865         ensureActionButtonViews(actionPad);
14866         this.btnUndo = this._resolveButton(actionPad, 'BtnUndo', this.btnUndo);
14867         this.btnReset = this._resolveButton(actionPad, 'BtnReset', this.btnReset);
14868         this.btnAnswer = this._resolveButton(actionPad, 'BtnAnswer', this.btnAnswer);
14869     }
14870     private _resolveButton(
14871         parent: Node | null,
14872         childName: string,
14873         existing: Button | null,
14874     ): Button | null {
14875         if (existing?.isValid) {
14876             return existing;
14877         }
14878         const node = parent?.getChildByName(childName) ?? null;
14879         if (!node) {
14880             return null;
14881         }
14882         let btn = node.getComponent(Button);
14883         if (!btn) {
14884             btn = node.addComponent(Button);
14885             btn.transition = Button.Transition.SCALE;
14886         }
14887         btn.zoomScale = 0.95;
14888         return btn;
14889     }
14890     setup(session: OpticalPuzzleSession, boardView?: OpticalPuzzleBoardView): void {
14891         this.teardown();
14892         this._autoBindButtons();
14893         this._session = session;
14894         this._boardView = boardView ?? null;
14895         this.btnUp?.node.on(Button.EventType.CLICK, this._onUp, this);
14896         this.btnDown?.node.on(Button.EventType.CLICK, this._onDown, this);
14897         this.btnLeft?.node.on(Button.EventType.CLICK, this._onLeft, this);
14898         this.btnRight?.node.on(Button.EventType.CLICK, this._onRight, this);
14899         this.btnUndo?.node.on(Button.EventType.CLICK, this._onUndo, this);
14900         this.btnReset?.node.on(Button.EventType.CLICK, this._onReset, this);
14901         this.btnAnswer?.node.on(Button.EventType.CLICK, this._onAnswer, this);
14902         this._bindUndoVideoBadgeHit();
14903         this._bindAnswerVideoBadgeHit();
14904         input.on(Input.EventType.KEY_DOWN, this._onKeyDown, this);
14905         this.refreshActionButtons();
14906     }

```

```

14907     private _bindUndoVideoBadgeHit(): void {
14908         this._btnUndoBadge = this._resolveUndoVideoBadgeButton();
14909         if (!this._btnUndoBadge?.node.isValid) {
14910             return;
14911         }
14912         this._btnUndoBadge.node.on(Button.EventType.CLICK, this._onUndoBadgeClick, this);
14913     }
14914     /** 撤回 +3 角标：与主键相同逻辑；CLICK 时必播 UI 点击音 */
14915     private _onUndoBadgeClick(): void {
14916         this._onUndo();
14917     }
14918     private _resolveUndoVideoBadgeButton(): Button | null {
14919         const view = this.btnUndo?.node.getComponent(OpticalPuzzleActionButtonView);
14920         return view?.getVideoBadgeHitButton() ?? null;
14921     }
14922     private _bindAnswerVideoBadgeHit(): void {
14923         this._btnAnswerBadge = this._resolveAnswerVideoBadgeButton();
14924         if (!this._btnAnswerBadge?.node.isValid) {
14925             return;
14926         }
14927         this._btnAnswerBadge.node.on(Button.EventType.CLICK, this._onAnswerBadgeClick, this);
14928     }
14929     /** 角标热区：与主键相同逻辑；CLICK 时必播 UI 点击音 */
14930     private _onAnswerBadgeClick(): void {
14931         this._onAnswer();
14932     }
14933     private _resolveAnswerVideoBadgeButton(): Button | null {
14934         const view = this.btnAnswer?.node.getComponent(OpticalPuzzleActionButtonView);
14935         return view?.getVideoBadgeHitButton() ?? null;
14936     }
14937     teardown(): void {
14938         input.off(Input.EventType.KEY_DOWN, this._onKeyDown, this);
14939         this._unbindBtnClick(this.btnUp, this._onUp);
14940         this._unbindBtnClick(this.btnDown, this._onDown);
14941         this._unbindBtnClick(this.btnLeft, this._onLeft);
14942         this._unbindBtnClick(this.btnRight, this._onRight);
14943         this._unbindBtnClick(this.btnUndo, this._onUndo);
14944         this._unbindBtnClick(this.btnReset, this._onReset);
14945         this._unbindBtnClick(this.btnAnswer, this._onAnswer);
14946         this._unbindUndoBadgeClick();
14947         this._unbindAnswerBadgeClick();
14948         this._btnUndoBadge = null;
14949         this._btnAnswerBadge = null;
14950         this._session = null;
14951         this._boardView = null;
14952         this._undoRewardAdPending = false;
14953         this._answerRewardAdPending = false;
14954     }
14955     private _unbindBtnClick(btn: Button | null, handler: () => void): void {
14956         const node = btn?.node;
14957         if (!node?.isValid) {
14958             return;
14959         }
14960         node.off(Button.EventType.CLICK, handler, this);
14961     }
14962     private _unbindUndoBadgeClick(): void {
14963         const node = this._btnUndoBadge?.node;
14964         if (!node?.isValid) {

```

```

14965         return;
14966     }
14967     node.off(Button.EventType.CLICK, this._onUndoBadgeClick, this);
14968 }
14969 private _unbindAnswerBadgeClick(): void {
14970     const node = this._btnAnswerBadge?.node;
14971     if (!node?.isValid) {
14972         return;
14973     }
14974     node.off(Button.EventType.CLICK, this._onAnswerBadgeClick, this);
14975 }
14976 /** 移动动画中或已进入结算等非 RUNNING 状态时，与遮罩挡按钮一致禁止局内输入 */
14977 private _isPlayInputLocked(): boolean {
14978     if (this._boardView?.isMoveAnimating()) {
14979         return true;
14980     }
14981     return !this._session || this._session.flowState !== OpticalGameFlowState.RUNNING;
14982 }
14983 private _applyDirection(dir: Direction, playClickOnButton = false): void {
14984     if (this._isPlayInputLocked()) {
14985         return;
14986     }
14987     const result = this._session.applyDirection(dir);
14988     if (!playClickOnButton || result === null) {
14989         return;
14990     }
14991     // 玩法音在 Session 内播；按钮仍叠加通用点击音
14992     this._playUiClick();
14993 }
14994 protected onDestroy(): void {
14995     this.teardown();
14996 }
14997 private _playUiClick(): void {
14998     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
14999 }
15000 private _buttonForDirection(dir: Direction): Button | null {
15001     switch (dir) {
15002         case Direction.Up:
15003             return this.btnUp;
15004         case Direction.Down:
15005             return this.btnDown;
15006         case Direction.Left:
15007             return this.btnLeft;
15008         case Direction.Right:
15009             return this.btnRight;
15010         default:
15011             return null;
15012     }
15013 }
15014 /** 键盘输入时同步按钮缩放 + 发光（与触摸按压一致） */
15015 private _pulseButtonFeedback(btn: Button | null, flashView?: { flashPressed(): void } | null): void {
15016     flashView?.flashPressed();
15017     const node = btn?.node;
15018     if (!node?.isValid) {
15019         return;
15020     }
15021     const ox = node.scale.x;
15022     const oy = node.scale.y;

```

```

15023         const oz = node.scale.z;
15024         const zoom = btn.zoomScale > 0 ? btn.zoomScale : 0.95;
15025         node.setScale(ox * zoom, oy * zoom, oz);
15026         this.scheduleOnce(() => {
15027             if (node.isValid) {
15028                 node.setScale(ox, oy, oz);
15029             }
15030         }, 0.1);
15031     }
15032     private _pulseDirButton(dir: Direction): void {
15033         const btn = this._buttonForDirection(dir);
15034         this._pulseButtonFeedback(btn, btn?.node.getComponent(OpticalPuzzleDirButtonView) ?? null);
15035     }
15036     private _pulseActionButton(btn: Button | null): void {
15037         this._pulseButtonFeedback(btn, btn?.node.getComponent(OpticalPuzzleActionButtonView) ?? null);
15038     }
15039     private _onDirectionClick(dir: Direction): void {
15040         this._applyDirection(dir, true);
15041     }
15042     private _onUp(): void {
15043         this._onDirectionClick(Direction.Up);
15044     }
15045     private _onDown(): void {
15046         this._onDirectionClick(Direction.Down);
15047     }
15048     private _onLeft(): void {
15049         this._onDirectionClick(Direction.Left);
15050     }
15051     private _onRight(): void {
15052         this._onDirectionClick(Direction.Right);
15053     }
15054     private _onUndo(): void {
15055         if (this._isPlayInputLocked()) {
15056             return;
15057         }
15058         this._playUiClick();
15059         this._handleUndoRequest();
15060     }
15061     /**
15062      * 撤回：无可撤回步时不变化、不扣次数；填充未耗尽时正常撤回；
15063      * 耗尽后拉起激励广告，完整观看则恢复满填（不执行撤回）。
15064      */
15065     private _handleUndoRequest(): void {
15066         const session = this._session;
15067         if (!session) {
15068             return;
15069         }
15070         if (session.isUndoIconFillExhausted() && session.canUndo()) {
15071             this._requestUndoRewardAd(session);
15072             return;
15073         }
15074         if (!session.canUndo()) {
15075             session.undoBatch();
15076             return;
15077         }
15078         session.registerUndoButtonPress();
15079         session.undoBatch();
15080         this.refreshActionButtons();

```

```
15081     }
15082     private _requestUndoRewardAd(session: OpticalPuzzleSession): void {
15083         if (this._undoRewardAdPending) {
15084             return;
15085         }
15086         this._undoRewardAdPending = true;
15087         showWeChatRewardedVideo()
15088             .then((watched) => {
15089                 if (!this.isValid) {
15090                     return;
15091                 }
15092                 if (watched) {
15093                     session.restoreUndoFillFromRewardedAd();
15094                     this.refreshActionButtons();
15095                 }
15096             })
15097             .then(
15098                 () => {
15099                     if (this.isValid) {
15100                         this._undoRewardAdPending = false;
15101                     }
15102                 },
15103                 () => {
15104                     if (this.isValid) {
15105                         this._undoRewardAdPending = false;
15106                     }
15107                 },
15108             );
15109     }
15110     private _onReset(): void {
15111         if (this._isPlayInputLocked()) {
15112             return;
15113         }
15114         this._playUiClick();
15115         this._session?.resetLevel();
15116     }
15117     private _onAnswer(): void {
15118         if (this._isPlayInputLocked()) {
15119             return;
15120         }
15121         this._playUiClick();
15122         const session = this._session;
15123         if (!session) {
15124             return;
15125         }
15126         if (session.isAnswerUnlocked()) {
15127             openAnswerPanel(this._resolveAnswerPanelNode());
15128             return;
15129         }
15130         this._requestAnswerRewardAd();
15131     }
15132     private _requestAnswerRewardAd(): void {
15133         if (this._answerRewardAdPending) {
15134             return;
15135         }
15136         this._answerRewardAdPending = true;
15137         showWeChatRewardedVideo()
15138             .then((watched) => {
```

```

15139         if (!this.isValid) {
15140             return;
15141         }
15142         if (!watched || !this._session) {
15143             return;
15144         }
15145         this._session.unlockAnswerForCurrentLevel();
15146         this.refreshActionButtons();
15147         openAnswerPanel(this._resolveAnswerPanelNode());
15148     })
15149     .then(
15150         () => {
15151             if (this.isValid) {
15152                 this._answerRewardAdPending = false;
15153             }
15154         },
15155         () => {
15156             if (this.isValid) {
15157                 this._answerRewardAdPending = false;
15158             }
15159         },
15160     );
15161 }
15162 private _resolveAnswerPanelNode(): Node | null {
15163     let cursor: Node | null = this.node;
15164     while (cursor) {
15165         if (cursor.name === 'GameRoot') {
15166             return resolveAnswerPanelNode(cursor);
15167         }
15168         cursor = cursor.parent;
15169     }
15170     return null;
15171 }
15172 /** 同步操作键图标（撤回填充阶段、参考解看视频角标、方向键教程提示等） */
15173 refreshActionButtons(): void {
15174     const stage = this._session?.undoFillStage ?? 0;
15175     const answerUnlocked = this._session?.isAnswerUnlocked() ?? false;
15176     const actionPad = this.node.getChildByName('ActionPad');
15177     if (actionPad?.isValid) {
15178         for (const child of actionPad.children) {
15179             const view = child.getComponent(OpticalPuzzleActionButtonView);
15180             if (!view) {
15181                 continue;
15182             }
15183             view.setUndoFillStage(stage);
15184             view.setAnswerVideoBadgeVisible(!answerUnlocked);
15185         }
15186     }
15187     this._refreshDirTutorialHints();
15188     this._rebindUndoVideoBadgeHit();
15189     this._rebindAnswerVideoBadgeHit();
15190 }
15191 /** 第 1 关且步数为 0：上方向键黄呼吸提示 */
15192 private _refreshDirTutorialHints(): void {
15193     const levelId = this._session?.getSnapshot().levelId ?? 0;
15194     const moveCount = this._session?.moveCount ?? 0;
15195     const showUpHint = levelId === 1 && moveCount === 0;
15196     const dirPad = this.node.getChildByName('DirPad');

```

```

15197         if (!dirPad?.isValid) {
15198             return;
15199         }
15200         for (const child of dirPad.children) {
15201             const view = child.getComponent(OpticalPuzzleDirButtonView);
15202             if (!view) {
15203                 continue;
15204             }
15205             const isUp = view.direction === Direction.Up;
15206             view.setTutorialHint(isUp && showUpHint);
15207         }
15208     }
15209     /** 角标热区在 refresh 后可能才显示, 需补绑 CLICK */
15210     private _rebindUndoVideoBadgeHit(): void {
15211         const badge = this._resolveUndoVideoBadgeButton();
15212         if (badge === this._btnUndoBadge) {
15213             return;
15214         }
15215         this._unbindUndoBadgeClick();
15216         this._btnUndoBadge = badge;
15217         if (this._btnUndoBadge?.node.isValid) {
15218             this._btnUndoBadge.node.on(Button.EventType.CLICK, this._onUndoBadgeClick, this);
15219         }
15220     }
15221     /** 角标热区在 refresh 后可能才创建, 需补绑 CLICK */
15222     private _rebindAnswerVideoBadgeHit(): void {
15223         const badge = this._resolveAnswerVideoBadgeButton();
15224         if (badge === this._btnAnswerBadge) {
15225             return;
15226         }
15227         this._unbindAnswerBadgeClick();
15228         this._btnAnswerBadge = badge;
15229         if (this._btnAnswerBadge?.node.isValid) {
15230             this._btnAnswerBadge.node.on(Button.EventType.CLICK, this._onAnswerBadgeClick, this);
15231         }
15232     }
15233     private _directionFromKey(keyCode: number): Direction | null {
15234         switch (keyCode) {
15235             case KeyCode.ARROW_UP:
15236             case KeyCode.KEY_W:
15237             case 38:
15238                 return Direction.Up;
15239             case KeyCode.ARROW_DOWN:
15240             case KeyCode.KEY_S:
15241             case 40:
15242                 return Direction.Down;
15243             case KeyCode.ARROW_LEFT:
15244             case KeyCode.KEY_A:
15245             case 37:
15246                 return Direction.Left;
15247             case KeyCode.ARROW_RIGHT:
15248             case KeyCode.KEY_D:
15249             case 39:
15250                 return Direction.Right;
15251             default:
15252                 return null;
15253         }
15254     }

```

```

15255  /** 浏览器预览时避免方向键触发页面滚动 */
15256  private _suppressBrowserKeyDefault(e: EventKeyboard, keyCode: number): void {
15257      if (!sys.isBrowser) {
15258          return;
15259      }
15260      const isArrow =
15261          keyCode === KeyCode.ARROW_UP ||
15262          keyCode === KeyCode.ARROW_DOWN ||
15263          keyCode === KeyCode.ARROW_LEFT ||
15264          keyCode === KeyCode.ARROW_RIGHT ||
15265          keyCode === 37 ||
15266          keyCode === 38 ||
15267          keyCode === 39 ||
15268          keyCode === 40;
15269      if (!isArrow) {
15270          return;
15271      }
15272      const raw = (e as unknown as { rawEvent?: KeyboardEvent }).rawEvent;
15273      raw?.preventDefault?.();
15274  }
15275  private _onKeyDown(e: EventKeyboard): void {
15276      if (this._isPlayInputLocked()) {
15277          return;
15278      }
15279      const keyCode = e.keyCode as number;
15280      const dir = this._directionFromKey(keyCode);
15281      if (dir !== null) {
15282          this._suppressBrowserKeyDefault(e, keyCode);
15283          this._pulseDirButton(dir);
15284          this._applyDirection(dir, true);
15285          return;
15286      }
15287      switch (keyCode) {
15288          case KeyCode.KEY_Z:
15289              this._pulseActionButton(this.btnUndo);
15290              this._playUiClick();
15291              this._handleUndoRequest();
15292              break;
15293          case KeyCode.KEY_R:
15294              this._pulseActionButton(this.btnReset);
15295              this._playUiClick();
15296              this._session.resetLevel();
15297              break;
15298          default:
15299              break;
15300      }
15301  }
15302  }
15303  // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLayout.ts -----
15304  import { Color } from 'cc';
15305  /** 棋盘逻辑格边长（像素） */
15306  export const OPTICAL_CELL_SIZE = 56;
15307  /** 墙心 / 默认外扩 / 左上补块 #40c3f9 */
15308  export const WALL_LIGHT_FILL = new Color(0x40, 0xc3, 0xf9, 255);
15309  /** 默认外扩（同 WALL_LIGHT_FILL）；贴地板侧外扩改用 WALL_DARK_FILL */
15310  export const WALL_ARM_FILL = WALL_LIGHT_FILL;
15311  /** 贴地板侧深臂 / 右下补块 #226c8f */
15312  export const WALL_DARK_FILL = new Color(0x22, 0x6c, 0x8f, 255);

```



```

15313  /** 墙心正方形边长 */
15314  export const WALL_CORE_SIZE = 28;
15315  /** 四向臂厚度（与墙角缺口一致） */
15316  export const WALL_ARM_THICK = 14;
15317  /** 墙角补块边长 / 圆角半径 */
15318  export const WALL_CORNER_PATCH = 14;
15319  /** 2×2 墙块内角强制补丁色 #3093bc */
15320  export const WALL_BLOCK_PATCH_FILL = new Color(0x30, 0x93, 0xbc, 255);
15321  /** 叠层统一色（墙心 + 外臂 + 角补） #3093bc */
15322  export const WALL_OVERLAY_FILL = new Color(0x30, 0x93, 0xbc, 255);
15323  /** 叠层墙心正方形边长 */
15324  export const WALL_OVERLAY_CORE_SIZE = 20;
15325  /** 叠层四向臂厚度 / 角补边长（自墙心面向外，8×8） */
15326  export const WALL_OVERLAY_ARM = 8;
15327  /** 叠层邻墙连通扩展额外重叠（消除接缝，用于上下） */
15328  export const WALL_OVERLAY_CONNECT_PAD = 1;
15329  /** 叠层左右连通扩展额外重叠（跨格缝再压 1px） */
15330  export const WALL_OVERLAY_CONNECT_PAD_H = 1;
15331  /** 地板填充 #123749（与选关缩略图 THUMB_FLOOR 一致；不透明以免背景方框透出） */
15332  export const FLOOR_FILL = new Color(0x12, 0x37, 0x49, 255);
15333  /** 光路总宽度（与 BeamView 一致） */
15334  export const BEAM_LINE_WIDTH = 9;
15335  /** 光路最内层白芯占全宽比例（与 BeamView 一致） */
15336  export const BEAM_CORE_WIDTH_RATIO = 0.125;
15337  /** 光路渐变叠层数（与 BeamView 一致） */
15338  export const BEAM_GRADIENT_STEPS = 8;
15339  /** 光路半宽（随格宽等比缩放） */
15340  export function beamHalfRadius(cellSize: number = OPTICAL_CELL_SIZE): number {
15341      return (BEAM_LINE_WIDTH / 2) * (cellSize / OPTICAL_CELL_SIZE);
15342  }
15343  /** 光路内白芯半径（= 最内层描边线宽的一半，随格宽等比缩放） */
15344  export function beamCoreRadius(cellSize: number = OPTICAL_CELL_SIZE): number {
15345      const scale = cellSize / OPTICAL_CELL_SIZE;
15346      return BEAM_LINE_WIDTH * BEAM_CORE_WIDTH_RATIO * 0.5 * scale;
15347  }
15348  /** 缩放后棋盘距 Canvas 顶部的设计边距（px） */
15349  export const PLAY_LAYER_TOP_MARGIN = 125;
15350  /** 关卡棋盘逻辑高度（像素，未缩放） */
15351  export function boardPixelHeight(levelHeight: number, cellSize: number = OPTICAL_CELL_SIZE): number {
15352      return levelHeight * cellSize;
15353  }
15354  /** 关卡棋盘逻辑宽度（像素，未缩放） */
15355  export function boardPixelWidth(levelWidth: number, cellSize: number = OPTICAL_CELL_SIZE): number {
15356      return levelWidth * cellSize;
15357  }
15358  /**
15359   * 计算 layerPlay 等比缩放：棋盘宽撑满 targetWidth（屏宽）；
15360   * 若缩放后棋盘高仍超过 targetWidth，再按 targetWidth / 棋盘高 收紧（高不超过屏宽）。
15361   */
15362  export function computePlayLayerScale(
15363      levelWidth: number,
15364      levelHeight: number,
15365      targetWidth: number,
15366      cellSize: number = OPTICAL_CELL_SIZE,
15367  ): number {
15368      const boardWidth = boardPixelWidth(levelWidth, cellSize);
15369      const boardHeight = boardPixelHeight(levelHeight, cellSize);
15370      if (boardWidth <= 0 || boardHeight <= 0 || targetWidth <= 0) {

```

```

15371         return 1;
15372     }
15373     const scaleByWidth = targetWidth / boardWidth;
15374     const scaleByHeight = targetWidth / boardHeight;
15375     return Math.min(scaleByWidth, scaleByHeight);
15376 }
15377 /**
15378  * 缩放后使棋盘顶边距 Canvas 顶边 topMargin (设计 px)。
15379  * Canvas 锚点居中时, 顶边 y = canvasHeight / 2。
15380  */
15381 export function computePlayLayerPosition(
15382     levelHeight: number,
15383     scale: number,
15384     canvasHeight: number,
15385     topMargin: number = PLAY_LAYER_TOP_MARGIN,
15386     cellSize: number = OPTICAL_CELL_SIZE,
15387 ): { x: number; y: number } {
15388     if (canvasHeight <= 0 || levelHeight <= 0) {
15389         return { x: 0, y: 0 };
15390     }
15391     const boardTopLocal = boardPixelHeight(levelHeight, cellSize) * 0.5;
15392     const canvasTop = canvasHeight * 0.5;
15393     const targetBoardTopY = canvasTop - Math.max(0, topMargin);
15394     return {
15395         x: 0,
15396         y: targetBoardTopY - boardTopLocal * scale,
15397     };
15398 }
15399 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectButtonGlyph.ts -----
15400 import { Graphics } from 'cc';
15401 import {
15402     drawHudShapelcon,
15403     HUD_BUTTON_DESIGN_SIZE,
15404 } from './OpticalPuzzleHudButtonCommon';
15405 /** 局内 TopBar 选关键: 图标占按钮比例 (与返回键一致) */
15406 export const HUD_LEVEL_SELECT_ICON_SIZE_RATIO = 0.58;
15407 /** 返回箭头外接框 (与 OpticalPuzzleBackButtonGlyph 一致, 用于与选关四宫格顶底对齐) */
15408 const BACK_ICON_TOP_NORM = 10.198;
15409 const BACK_ICON_BOTTOM_NORM = -10.125;
15410 const BACK_PATH_UNIT = 10.198;
15411 /**
15412  * 四宫格几何 (用户 SVG viewBox 1024, iconR=10):
15413  * 每格外框 192×192, 圆角约 52.8; 格心见 CELL_CENTERS。
15414  * CELL_SIZE_SCALE: 在保持 2×2 间距前提下放大单格。
15415  */
15416 const CELL_SIZE_SCALE = 1.7;
15417 const CELL_HALF = ((96 * 10) / 512) * CELL_SIZE_SCALE;
15418 const CELL_CORNER = ((52.8 * 10) / 512) * CELL_SIZE_SCALE;
15419 const CELL_CENTERS: ReadonlyArray<readonly [number, number]> = [
15420     [-4.375, 3.438],
15421     [4.375, 3.438],
15422     [-4.375, -5.312],
15423     [4.375, -5.312],
15424 ];
15425 const GRID_TOP_NORM = Math.max(...CELL_CENTERS.map(([, ny]) => ny + CELL_HALF));
15426 const GRID_BOTTOM_NORM = Math.min(...CELL_CENTERS.map(([, ny]) => ny - CELL_HALF));
15427 function traceLevelSelectIcon(g: Graphics, cx: number, cy: number, scale: number): void {
15428     const half = CELL_HALF * scale;

```

```

15429     const corner = CELL_CORNER * scale;
15430     for (const [nx, ny] of CELL_CENTERS) {
15431         const cellCx = cx + nx * scale;
15432         const cellCy = cy + ny * scale;
15433         g.roundRect(cellCx - half, cellCy - half, half * 2, half * 2, corner);
15434     }
15435 }
15436 /** 四宫格 icon（与 Game TopBar BtnLevelSelect 同款几何） */
15437 export function traceLevelSelectGridIconForButton(
15438     g: Graphics,
15439     centerX: number,
15440     centerY: number,
15441     buttonSize: number,
15442     iconSizeRatio: number = HUD_LEVEL_SELECT_ICON_SIZE_RATIO,
15443 ): void {
15444     const iconHalf = buttonSize * iconSizeRatio * 0.5;
15445     const backScale = iconHalf / BACK_PATH_UNIT;
15446     const backHeightNorm = BACK_ICON_TOP_NORM - BACK_ICON_BOTTOM_NORM;
15447     const gridHeightNorm = GRID_TOP_NORM - GRID_BOTTOM_NORM;
15448     const scale = (backHeightNorm * backScale) / gridHeightNorm;
15449     const alignOffsetY = BACK_ICON_TOP_NORM * backScale - GRID_TOP_NORM * scale;
15450     traceLevelSelectIcon(g, centerX, centerY + alignOffsetY, scale);
15451 }
15452 /**
15453  * 绘制选关键：无键帽外框，四枚圆角正方（描边/填色/按下光晕与四向键内芯一致）。
15454  */
15455 export function drawLevelSelectButtonGlyph(
15456     g: Graphics,
15457     left: number,
15458     bottom: number,
15459     width: number,
15460     height: number,
15461     pressed = false,
15462 ): void {
15463     const size = Math.min(width, height);
15464     const cx = left + width * 0.5;
15465     const cy = bottom + height * 0.5;
15466     const refSize = size > 0 ? size : HUD_BUTTON_DESIGN_SIZE;
15467     drawHudShapelcon(g, refSize, pressed, () => {
15468         traceLevelSelectGridIconForButton(g, cx, cy, size);
15469     });
15470 }
15471 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectButtonView.ts -----
15472 import {
15473     _decorator,
15474     Button,
15475     Component,
15476     Graphics,
15477     Node,
15478     UITransform,
15479 } from 'cc';
15480 import { drawLevelSelectButtonGlyph } from './OpticalPuzzleLevelSelectButtonGlyph';
15481 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
15482 const { ccclass } = _decorator;
15483 /** 与四向键 Button.zoomScale 一致 */
15484 const LEVEL_SELECT_BUTTON_ZOOM_SCALE = 0.95;
15485 /** TopBar 选关键：仅绘制图标，按下缩放与四向键一致 */
15486 @ccclass('OpticalPuzzleLevelSelectButtonView')

```

```

15487 export class OpticalPuzzleLevelSelectButtonView extends Component {
15488     private _graphics: Graphics | null = null;
15489     private _pressed = false;
15490     private _pressCtrl: HudButtonPressController | null = null;
15491     protected onLoad(): void {
15492         this._ensureButton();
15493         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
15494             this._pressed = pressed;
15495             this._redraw();
15496         });
15497         this._redraw();
15498     }
15499     protected onEnable(): void {
15500         this._pressCtrl?.bind();
15501         this._redraw();
15502     }
15503     protected onDisable(): void {
15504         this._pressCtrl?.unbind();
15505         this._pressed = false;
15506     }
15507     protected onDestroy(): void {
15508         this._pressCtrl?.unbind();
15509     }
15510     refresh(): void {
15511         this._redraw();
15512     }
15513     private _ensureButton(): void {
15514         let btn = this.getComponent(Button);
15515         if (!btn) {
15516             btn = this.addComponent(Button);
15517             btn.transition = Button.Transition.SCALE;
15518         }
15519         btn.zoomScale = LEVEL_SELECT_BUTTON_ZOOM_SCALE;
15520     }
15521     private _ensureGraphics(): Graphics | null {
15522         if (!this._graphics?.isValid) {
15523             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
15524         }
15525         return this._graphics;
15526     }
15527     private _redraw(): void {
15528         const g = this._ensureGraphics();
15529         const ut = this.getComponent(UITransform);
15530         if (!g || !ut) {
15531             return;
15532         }
15533         g.clear();
15534         const w = ut.width;
15535         const h = ut.height;
15536         const left = -ut.anchorX * w;
15537         const bottom = -ut.anchorY * h;
15538         drawLevelSelectButtonGlyph(g, left, bottom, w, h, this._pressed);
15539     }
15540 }
15541 /** 为 TopBar/BtnLevelSelect 挂上绘制与按压缩放 */
15542 export function ensureLevelSelectButtonView(btnLevelSelect: Node | null): void {
15543     if (!btnLevelSelect?.isValid) {
15544         return;

```

```

15545     }
15546     if (!btnLevelSelect.getComponent(OpticalPuzzleLevelSelectButtonView)) {
15547         btnLevelSelect.addComponent(OpticalPuzzleLevelSelectButtonView);
15548     }
15549 }
15550 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectItemView.ts -----
15551 import {
15552     _decorator,
15553     Button,
15554     Color,
15555     Component,
15556     Graphics,
15557     Label,
15558     Node,
15559     UITransform,
15560     Vec3,
15561 } from 'cc';
15562 import { getOpticalLevelById } from '../Config/OpticalPuzzleLevels';
15563 import {
15564     defaultDimStarStates,
15565     resolveStarSlotStates,
15566     type OpticalStarSlotStates,
15567 } from '../Core/OpticalPuzzleStarRating';
15568 import { DataManager } from '../Manager/DataManager';
15569 import {
15570     applyLevelSelectItemLabelFontStyle,
15571     drawLevelSelectItemChrome,
15572     drawLevelSelectItemStars,
15573     drawLevelSelectLockIcon,
15574     LEVEL_SELECT_ITEM_DESIGN_SIZE,
15575     LEVEL_SELECT_ITEM_LABEL_FONT,
15576     LEVEL_SELECT_ITEM_LABEL_FONT_SCALE,
15577     LEVEL_SELECT_ITEM_LABEL_MARGIN,
15578     LEVEL_SELECT_ITEM_LABEL_NODE_SCALE,
15579     LEVEL_SELECT_ITEM_THUMB_PADDING,
15580     LevelSelectItemVisualState,
15581 } from './OpticalPuzzleLevelSelectPanelGlyph';
15582 import { drawLevelSelectLevelThumbnail } from './OpticalPuzzleLevelSelectLevelThumbnail';
15583 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
15584 const { ccclass } = _decorator;
15585 const THUMB_CHILD_NAME = 'thumbnail';
15586 const LABEL_CHILD_NAME = 'label';
15587 /** 单个关卡项：键帽 + 居中缩略图 + 左上角关卡号 */
15588 @ccclass('OpticalPuzzleLevelSelectItemView')
15589 export class OpticalPuzzleLevelSelectItemView extends Component {
15590     private _graphics: Graphics | null = null;
15591     private _thumbGraphics: Graphics | null = null;
15592     private _label: Label | null = null;
15593     private _levelId = 0;
15594     private _visualState = LevelSelectItemVisualState.Normal;
15595     private _pressed = false;
15596     private _pressCtrl: HudButtonPressController | null = null;
15597     private _onSelect: ((levelId: number) => void) | null = null;
15598     protected onLoad(): void {
15599         this._ensureTransform();
15600         this._ensureThumbnail();
15601         this._ensureLabel();
15602         this._ensureButton();

```

```

15603         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
15604             if (this._visualState === LevelSelectItemVisualState.Locked) {
15605                 return;
15606             }
15607             this._pressed = pressed;
15608             this._redraw();
15609         }, true);
15610     }
15611     protected onEnable(): void {
15612         this._pressCtrl?.bind();
15613         this.node.on(Button.EventType.CLICK, this._onClick, this);
15614         if (this._label?.isValid) {
15615             applyLevelSelectItemLabelFontStyle(this._label);
15616         }
15617     }
15618     protected onDisable(): void {
15619         this._pressCtrl?.unbind();
15620         this.node.off(Button.EventType.CLICK, this._onClick, this);
15621         this._pressed = false;
15622     }
15623     protected onDestroy(): void {
15624         this._pressCtrl?.unbind();
15625         this.node.off(Button.EventType.CLICK, this._onClick, this);
15626     }
15627     /** 绑定关卡数据与点击回调（首次创建项） */
15628     setup(levelId: number, visualState: LevelSelectItemVisualState, onSelect: (levelId: number) => void): void {
15629         this._levelId = levelId;
15630         this._onSelect = onSelect;
15631         this._applyVisualState(visualState, true);
15632     }
15633     /** 更新解锁/当前关状态；状态未变时不重绘缩略图 */
15634     applyVisualState(visualState: LevelSelectItemVisualState): void {
15635         this._applyVisualState(visualState, false);
15636     }
15637     getLevelId(): number {
15638         return this._levelId;
15639     }
15640     private _applyVisualState(visualState: LevelSelectItemVisualState, forceRedraw: boolean): void {
15641         const stateChanged = this._visualState !== visualState;
15642         this._visualState = visualState;
15643         if (this._label?.isValid) {
15644             this._label.string = `${this._levelId}`;
15645             this._label.color = visualState === LevelSelectItemVisualState.Locked
15646                 ? new Color(130, 138, 152, 255)
15647                 : Color.WHITE;
15648         }
15649         const btn = this.getComponent(Button);
15650         if (btn) {
15651             btn.interactable = visualState !== LevelSelectItemVisualState.Locked;
15652         }
15653         if (forceRedraw || stateChanged || this._pressed) {
15654             this._pressed = false;
15655         }
15656         this._redraw();
15657     }
15658     refresh(): void {
15659         this._redraw();
15660     }

```

```

15661     private _ensureTransform(): void {
15662         const ut = this.getComponent(UITransform) ?? this.addComponent(UITransform);
15663         ut.setContentSize(LEVEL_SELECT_ITEM_DESIGN_SIZE, LEVEL_SELECT_ITEM_DESIGN_SIZE);
15664         ut.setAnchorPoint(0.5, 0.5);
15665     }
15666     private _ensureThumbnail(): void {
15667         let thumbNode = this.node.getChildByName(THUMB_CHILD_NAME);
15668         if (!thumbNode) {
15669             thumbNode = new Node(THUMB_CHILD_NAME);
15670             thumbNode.setParent(this.node);
15671         }
15672         thumbNode.setPosition(0, 0, 0);
15673         const ut = thumbNode.getComponent(UITransform) ?? thumbNode.addComponent(UITransform);
15674         ut.setAnchorPoint(0.5, 0.5);
15675         ut.setContentSize(LEVEL_SELECT_ITEM_DESIGN_SIZE, LEVEL_SELECT_ITEM_DESIGN_SIZE);
15676         this._thumbGraphics = thumbNode.getComponent(Graphics) ?? thumbNode.addComponent(Graphics);
15677     }
15678     private _ensureLabel(): void {
15679         let labelNode = this.node.getChildByName(LABEL_CHILD_NAME);
15680         if (!labelNode) {
15681             labelNode = new Node(LABEL_CHILD_NAME);
15682             labelNode.setParent(this.node);
15683         }
15684         labelNode.setScale(
15685             new Vec3(LEVEL_SELECT_ITEM_LABEL_NODE_SCALE, LEVEL_SELECT_ITEM_LABEL_NODE_SCALE,
15686 1),
15687         );
15688         this._label = labelNode.getComponent(Label) ?? labelNode.addComponent(Label);
15689         const displayFont = LEVEL_SELECT_ITEM_LABEL_FONT * LEVEL_SELECT_ITEM_LABEL_FONT_SCALE;
15690         this._label.fontSize = displayFont;
15691         this._label.lineHeight = displayFont + 4;
15692         this._label.color = Color.WHITE;
15693         this._label.horizontalAlign = Label.HorizontalAlign.LEFT;
15694         this._label.verticalAlign = Label.VerticalAlign.TOP;
15695         this._label.overflow = Label.Overflow.NONE;
15696         applyLevelSelectItemLabelFontStyle(this._label);
15697         const lut = labelNode.getComponent(UITransform) ?? labelNode.addComponent(UITransform);
15698         lut.setAnchorPoint(0, 1);
15699         const labelBox = displayFont * 2;
15700         lut.setContentSize(labelBox, labelBox);
15701         this._layoutLabel(labelNode);
15702     }
15703     /** 关卡号贴左上角（相对键帽内缘） */
15704     private _layoutLabel(labelNode: Node): void {
15705         const ut = this.getComponent(UITransform);
15706         if (!ut) {
15707             return;
15708         }
15709         const w = ut.width;
15710         const h = ut.height;
15711         const m = LEVEL_SELECT_ITEM_LABEL_MARGIN;
15712         labelNode.setPosition(-w * 0.5 + m, h * 0.5 - m, 0);
15713     }
15714     private _ensureButton(): void {
15715         let btn = this.getComponent(Button);
15716         if (!btn) {
15717             btn = this.addComponent(Button);
15718             btn.transition = Button.Transition.SCALE;

```

```
15719     }
15720     btn.zoomScale = 0.95;
15721 }
15722 private _ensureGraphics(): Graphics | null {
15723     if (!this._graphics?.isValid) {
15724         this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
15725     }
15726     return this._graphics;
15727 }
15728 private _onClick(): void {
15729     if (this._visualState === LevelSelectItemVisualState.Locked || this._levelId <= 0) {
15730         return;
15731     }
15732     this._onSelect?.(this._levelId);
15733 }
15734 private _redraw(): void {
15735     const ut = this.getComponent(UITransform);
15736     if (!ut) {
15737         return;
15738     }
15739     const w = ut.width;
15740     const h = ut.height;
15741     const left = -ut.anchorX * w;
15742     const bottom = -ut.anchorY * h;
15743     const locked = this._visualState === LevelSelectItemVisualState.Locked;
15744     const g = this._ensureGraphics();
15745     if (g) {
15746         g.clear();
15747         drawLevelSelectItemChrome(g, left, bottom, w, h, this._visualState, this._pressed);
15748     }
15749     const level = getOpticalLevelById(this._levelId);
15750     const thumbG = this._thumbGraphics;
15751     if (thumbG?.isValid && level) {
15752         thumbG.clear();
15753         const pad = LEVEL_SELECT_ITEM_THUMB_PADDING;
15754         drawLevelSelectLevelThumbnail(
15755             thumbG,
15756             level,
15757             left + pad,
15758             bottom + pad,
15759             w - pad * 2,
15760             h - pad * 2,
15761             locked,
15762         );
15763         if (locked) {
15764             drawLevelSelectLockIcon(thumbG, 0, 0, w);
15765         } else {
15766             drawLevelSelectItemStars(
15767                 thumbG,
15768                 left,
15769                 bottom,
15770                 w,
15771                 this._resolveStarSlotStates(level),
15772             );
15773         }
15774     }
15775     const labelNode = this.node.getChildByName(LABEL_CHILD_NAME);
15776     if (labelNode?.isValid) {
```



```

15777         this._layoutLabel(labelNode);
15778     }
15779 }
15780 /** 未通关全暗; 已通关按 bestSteps 与关卡阈值 (同通关页 resolveStarSlotStates) */
15781 private _resolveStarSlotStates(level: Nullable<ReturnType<typeof getOpticalLevelById>>):
15782 OpticalStarSlotStates {
15783     if (!level.starThresholds) {
15784         return defaultDimStarStates();
15785     }
15786     const bestSteps = DataManager.instance.getOpticalLevelBestSteps(this._levelId);
15787     if (bestSteps == null) {
15788         return defaultDimStarStates();
15789     }
15790     return resolveStarSlotStates(bestSteps, level.starThresholds);
15791 }
15792 }
15793 /** 创建关卡项节点 */
15794 export function createLevelSelectItemNode(name: string): Node {
15795     const node = new Node(name);
15796     node.addComponent(OpticalPuzzleLevelSelectItemView);
15797     return node;
15798 }
15799 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectLevelThumbnail.ts -----
15800 import { Color, Graphics } from 'cc';
15801 import type { IOpticalLevelConfig, IOpticalPiece } from '../Config/OpticalPuzzleLevelSchema';
15802 import { colorModeToKey } from '../Core/OpticalLightColor';
15803 import { TerrainKind } from '../Core/OpticalPuzzleTypes';
15804 import {
15805     beamColorFromKey,
15806     pieceChannelColors,
15807     playerBaseFillColor,
15808     sourceFillColor,
15809     targetUnlitBulbFillColor,
15810 } from './OpticalPuzzleColorUtil';
15811 import { LEVEL_SELECT_ITEM_THUMB_BOARD_SCALE } from './OpticalPuzzleLevelSelectPanelGlyph';
15812 const THUMB_FLOOR = new Color(18, 55, 73, 255);
15813 const THUMB_WALL = new Color(48, 147, 188, 255);
15814 /** 缩略图元件填充色 (与局内 `OpticalPuzzlePieceGlyph` 同源) */
15815 function pieceThumbFillColor(piece: IOpticalPiece): Color {
15816     const colorKey = colorModeToKey(piece.colorMode);
15817     if (piece.connectivity === 0) {
15818         return targetUnlitBulbFillColor(colorKey);
15819     }
15820     return pieceChannelColors(colorKey).stroke;
15821 }
15822 /** 缩略图目标填充色 (按 colorKey, 与光路同色阶) */
15823 function targetThumbFillColor(colorKey?: string): Color {
15824     const beam = beamColorFromKey(colorKey);
15825     return new Color(beam.r, beam.g, beam.b, 255);
15826 }
15827 /** 在关卡项中心区域绘制关卡布局缩略图 */
15828 export function drawLevelSelectLevelThumbnail(
15829     g: Graphics,
15830     level: IOpticalLevelConfig,
15831     left: number,
15832     bottom: number,
15833     width: number,
15834     height: number,

```

```

15835         locked: boolean,
15836     ): void {
15837         const lw = Math.max(1, level.width);
15838         const lh = Math.max(1, level.height);
15839         const areaW = width * LEVEL_SELECT_ITEM_THUMB_BOARD_SCALE;
15840         const areaH = height * LEVEL_SELECT_ITEM_THUMB_BOARD_SCALE;
15841         const cell = Math.min(areaW / lw, areaH / lh);
15842         const boardW = lw * cell;
15843         const boardH = lh * cell;
15844         const ox = left + (width - boardW) * 0.5;
15845         const oy = bottom + (height - boardH) * 0.5;
15846         g.fillColor = THUMB_FLOOR;
15847         g.rect(ox, oy, boardW, boardH);
15848         g.fill();
15849         for (let y = 0; y < lh; y++) {
15850             for (let x = 0; x < lw; x++) {
15851                 const kind = level.terrain[y * lw + x];
15852                 const cx = ox + x * cell;
15853                 const cy = oy + (lh - 1 - y) * cell;
15854                 if (kind === TerrainKind.Wall) {
15855                     g.fillColor = THUMB_WALL;
15856                     g.rect(cx, cy, cell, cell);
15857                     g.fill();
15858                 }
15859             }
15860         }
15861         const dot = Math.max(1.5, cell * 0.28);
15862         const inset = cell * 0.22;
15863         for (const src of level.sources) {
15864             g.fillColor = sourceFillColor(src.colorKey);
15865             g.circle(ox + (src.x + 0.5) * cell, oy + (lh - 1 - src.y + 0.5) * cell, dot);
15866             g.fill();
15867         }
15868         for (const tgt of level.targets) {
15869             g.fillColor = targetThumbFillColor(tgt.colorKey);
15870             g.circle(ox + (tgt.x + 0.5) * cell, oy + (lh - 1 - tgt.y + 0.5) * cell, dot * 0.85);
15871             g.fill();
15872         }
15873         for (const piece of level.pieces) {
15874             g.fillColor = pieceThumbFillColor(piece);
15875             g.rect(
15876                 ox + piece.x * cell + inset,
15877                 oy + (lh - 1 - piece.y) * cell + inset,
15878                 cell - inset * 2,
15879                 cell - inset * 2,
15880             );
15881             g.fill();
15882         }
15883         const px = level.player.x;
15884         const py = level.player.y;
15885         g.fillColor = playerBaseFillColor();
15886         g.circle(ox + (px + 0.5) * cell, oy + (lh - 1 - py + 0.5) * cell, dot * 0.75);
15887         g.fill();
15888         if (locked) {
15889             g.fillColor = new Color(0, 0, 0, 72);
15890             g.rect(ox, oy, boardW, boardH);
15891             g.fill();
15892         }

```

```

15893 }
15894 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectPanel.ts -----
15895 import {
15896     _decorator,
15897     BlockInputEvents,
15898     Button,
15899     Component,
15900     director,
15901     EventTouch,
15902     Graphics,
15903     Layout,
15904     Mask,
15905     Node,
15906     ScrollView,
15907     Sprite,
15908     sys,
15909     UITransform,
15910     Vec2,
15911 } from 'cc';
15912 import { DataManager } from '../../Manager/DataManager';
15913 import { AUDIO_EFFECT_ENUM, EVENT_ENUM, SCENE_ENUM } from '../../Utils/Enum';
15914 import { PLAY_AUDIO, SHOW_TOAST } from '../../Utils/Event';
15915 import { OPTICAL_LEVELS } from '../Config/OpticalPuzzleLevels';
15916 import type { IOpticalLevelConfig } from '../Config/OpticalPuzzleLevelSchema';
15917 import {
15918     drawLevelSelectCloseGlyph,
15919     drawLevelSelectPanelChrome,
15920     drawLevelSelectScrollBottomFade,
15921     drawLevelSelectScrollTopFade,
15922     drawLevelSelectTitleBar,
15923     LEVEL_SELECT_CLOSE_DESIGN_SIZE,
15924     LEVEL_SELECT_ITEM_DESIGN_SIZE,
15925     LEVEL_SELECT_SCROLL_BOTTOM_FADE_HEIGHT,
15926     LEVEL_SELECT_SCROLL_BOTTOM_FADE_WIDTH,
15927     LEVEL_SELECT_SCROLL_TOP_FADE_HEIGHT,
15928     LEVEL_SELECT_SCROLL_TOP_FADE_WIDTH,
15929     LevelSelectItemVisualState,
15930 } from './OpticalPuzzleLevelSelectPanelGlyph';
15931 import {
15932     createLevelSelectItemNode,
15933     OpticalPuzzleLevelSelectItemView,
15934 } from './OpticalPuzzleLevelSelectItemView';
15935 import { OpticalPuzzleLevelSelectScrollFadePassThrough } from
15936 './OpticalPuzzleLevelSelectScrollFadePassThrough';
15937 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
15938 const { ccclass } = _decorator;
15939 /** 与 Game 场景 levelSelectPanel 占位布局一致（设计 px） */
15940 const PANEL_WIDTH = 650;
15941 const PANEL_HEIGHT = 1200;
15942 const PANEL_OFFSET_Y = -25;
15943 const TITLE_WIDTH = 250;
15944 const TITLE_HEIGHT = 80;
15945 const TITLE_OFFSET_Y = 575;
15946 const CLOSE_SIZE = LEVEL_SELECT_CLOSE_DESIGN_SIZE;
15947 const CLOSE_OFFSET_X = 315;
15948 const CLOSE_OFFSET_Y = 570;
15949 const SCROLL_WIDTH = 650;
15950 const SCROLL_HEIGHT = 1100;

```

```

15951 const SCROLL_OFFSET_Y = -40;
15952 const GRID_COLUMNS = 4;
15953 /** 面板打开时分帧创建，避免点击瞬间卡顿 */
15954 const LEVEL_LIST_BUILD_BATCH = 8;
15955 /** 面板隐藏时预构建，每帧多画一些 */
15956 const LEVEL_LIST_PREWARM_BATCH = 26;
15957 const GFX_CHILD_NAME = 'gfx';
15958 const SCROLL_FADE_TOP_CHILD_NAME = 'fadetop';
15959 const SCROLL_FADE_BOTTOM_CHILD_NAME = 'fadebottom';
15960 /** 四列网格等间距：左右边距与列间距相同，gap = (容器宽 - 4 × cell) / 5 */
15961 function computeLevelSelectGridGap(scrollWidth: number, columns: number, cellSize: number): number {
15962     return Math.max(0, (scrollWidth - columns * cellSize) / (columns + 1));
15963 }
15964 /** 第 col 列关卡项中心 x（与满行同列对齐；末行不足 4 个也靠左） */
15965 function levelSelectItemCenterX(
15966     scrollWidth: number,
15967     cellSize: number,
15968     gap: number,
15969     col: number,
15970 ): number {
15971     return -scrollWidth * 0.5 + gap + col * (cellSize + gap) + cellSize * 0.5;
15972 }
15973 /** 局内入口能力（避免与 OpticalPuzzleRoot 循环引用） */
15974 interface IOpticalPuzzleRootApi {
15975     getCurrentLevelId?(): number;
15976     loadLevelById(levelId: number): void;
15977 }
15978 /**
15979  * 选关弹层：绘制 + 列表 + 选关逻辑一体。
15980  * 挂到 levelSelectPanel / LevelSelectPanel 根节点，Menu 与 Game 场景可直接复制节点复用。
15981  * 子节点 bg / panel / titlebar / closebtn / scrollpanel 的尺寸与位置由场景控制；
15982  * panel / titlebar / closebtn 的键帽底在各自根节点 Graphics 绘制（勿与 Sprite 同节点）。
15983  * 脚本仅在节点缺少 UITransform 时写入占位默认值。
15984  * 打开时由 MenuManager / GameUIManager 设 active=true；关闭由本脚本处理。
15985  */
15986 @ccclass('OpticalPuzzleLevelSelectPanel')
15987 export class OpticalPuzzleLevelSelectPanel extends Component {
15988     private _backdropNode: Node | null = null;
15989     private _panelNode: Node | null = null;
15990     private _titleNode: Node | null = null;
15991     private _closeNode: Node | null = null;
15992     private _scrollNode: Node | null = null;
15993     private _contentNode: Node | null = null;
15994     private _panelGraphics: Graphics | null = null;
15995     private _titleGraphics: Graphics | null = null;
15996     private _closeGraphics: Graphics | null = null;
15997     private _scrollFadeGraphics: Graphics | null = null;
15998     private _scrollBottomFadeGraphics: Graphics | null = null;
15999     private _closePressed = false;
16000     private _closePressCtrl: HudButtonPressController | null = null;
16001     private _itemNodes: Node[] = [];
16002     private readonly _onLevelItemSelectHandler = (levelId: number): void => {
16003         this._onLevelItemSelected(levelId);
16004     };
16005     private _sortedLevels: IOpticalLevelConfig[] = [];
16006     private _buildCursor = 0;
16007     private _listBuilding = false;
16008     protected onLoad(): void {

```

```

16009         this._hidePlaceholderSplashes();
16010         this._ensureRootLayout();
16011         this._buildUITree();
16012         this._bindCloseButton();
16013         this._redrawStaticChrome();
16014         this.scheduleOnce(() => this.prewarmLevelList(), 0);
16015     }
16016     protected onEnable(): void {
16017         this._closePressCtrl?.bind();
16018         if (this.isLevelListReady()) {
16019             this.syncLevelListVisuals();
16020             this._scrollToCurrentLevelRowCenter();
16021         } else if (!this._listBuilding) {
16022             this.prewarmLevelList();
16023         }
16024     }
16025     /** 进入场景后分帧预构建（面板隐藏时加速，打开时不阻塞主线程） */
16026     prewarmLevelList(): void {
16027         this._startAsyncLevelListBuild(false);
16028     }
16029     /** 当前关 / 解锁进度变化后仅更新高亮，不重建节点 */
16030     syncLevelListVisuals(): void {
16031         if (!this.isLevelListReady()) {
16032             return;
16033         }
16034         this._applyLevelItemVisualStates();
16035     }
16036     /** 列表是否已全部创建并绘制完成 */
16037     isLevelListReady(): boolean {
16038         return !this._listBuilding && this._itemNodes.length === OPTICAL_LEVELS.length;
16039     }
16040     protected onDisable(): void {
16041         this._closePressCtrl?.unbind();
16042         this._closePressed = false;
16043     }
16044     protected onDestroy(): void {
16045         this.unschedule(this._processLevelListBuildBatch);
16046         this._closePressCtrl?.unbind();
16047         if (this._closeNode?.isValid) {
16048             this._closeNode.off(Button.EventType.CLICK, this._onCloseClick, this);
16049         }
16050         if (this._backdropNode?.isValid) {
16051             this._backdropNode.off(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
16052         }
16053     }
16054     /** 打开面板前/时调用：保证列表已构建并刷新解锁态（避免首次打开空白） */
16055     ensureLevelListReadyForOpen(): void {
16056         if (!this.isLevelListReady()) {
16057             if (!this._listBuilding) {
16058                 this._startAsyncLevelListBuild(false);
16059             }
16060             while (this._listBuilding) {
16061                 this._processLevelListBuildBatch();
16062             }
16063         }
16064         this.syncLevelListVisuals();
16065         this._scrollToCurrentLevelRowCenter();
16066         if (sys.platform === sys.Platform.WECHAT_GAME) {

```

```

16067         console.log('[LevelSelect] open', {
16068             clearPairs: DataManager.instance.getOpticalLevelClearPairCount(),
16069             currentId: this._resolveCurrentPlayingLevelId(),
16070             level1Steps: DataManager.instance.getOpticalLevelBestSteps(1),
16071         });
16072     }
16073 }
16074 /** 关闭选关面板 */
16075 close(): void {
16076     if (this.node?.isValid) {
16077         this.node.active = false;
16078     }
16079 }
16080 /** 刷新关卡项（进度变化后调用） */
16081 refresh(): void {
16082     if (this.isLevelListReady()) {
16083         this.syncLevelListVisuals();
16084         return;
16085     }
16086     this.prewarmLevelList();
16087 }
16088 // #region UI 构建
16089 private _ensureRootLayout(): void {
16090     // 弹层根节点尺寸/Widget 由场景控制，脚本不覆盖
16091     if (!this.getComponent(UITransform)) {
16092         const ut = this.addComponent(UITransform);
16093         ut.setAnchorPoint(0.5, 0.5);
16094     }
16095 }
16096 private _buildUiTree(): void {
16097     this._backdropNode = this._ensureNamedChild('bg');
16098     this._ensureBackdrop(this._backdropNode);
16099     this._panelNode = this._ensureNamedChild('panel');
16100     this._ensurePanelBody(this._panelNode);
16101     this._titleNode = this._ensureNamedChild('titlebar');
16102     this._ensureTitleBar(this._titleNode);
16103     this._closeNode = this._ensureNamedChild('closebtn');
16104     this._ensureCloseButton(this._closeNode);
16105     this._scrollNode = this._ensureNamedChild('scrollpanel');
16106     this._ensureScrollArea(this._scrollNode);
16107 }
16108 private _ensureNamedChild(name: string): Node {
16109     let child = this.node.getChildByName(name);
16110     if (!child) {
16111         child = new Node(name);
16112         child.setParent(this.node);
16113     }
16114     return child;
16115 }
16116 /**
16117  * 场景已挂 UITransform 时不改尺寸/位置；仅脚本新建占位节点时写入默认值。
16118  */
16119 private _applyDefaultNodeLayout(
16120     node: Node,
16121     width: number,
16122     height: number,
16123     x: number,
16124     y: number,

```

```

16125         anchorX = 0.5,
16126         anchorY = 0.5,
16127     ): void {
16128         const hadTransform = !!node.getComponent(UITransform);
16129         const ut = node.getComponent(UITransform) ?? node.addComponent(UITransform);
16130         if (hadTransform) {
16131             return;
16132         }
16133         ut.setContentSize(width, height);
16134         ut.setAnchorPoint(anchorX, anchorY);
16135         node.setPosition(x, y, 0);
16136     }
16137     /** bg 全屏遮罩由场景自行表现；脚本只补点击拦截与点空白关闭 */
16138     private _ensureBackdrop(node: Node): void {
16139         if (!node.getComponent(UITransform)) {
16140             node.addComponent(UITransform);
16141         }
16142         if (!node.getComponent(BlockInputEvents)) {
16143             node.addComponent(BlockInputEvents);
16144         }
16145         this._removeGfxChild(node);
16146         node.off(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
16147         node.on(Node.EventType.TOUCH_END, this._onBackdropTouchEnd, this);
16148     }
16149     /** 面板内容区拦截触摸，避免穿透到 bg 触发关闭 */
16150     private _ensureTouchBlocker(node: Node): void {
16151         if (!node.getComponent(UITransform)) {
16152             node.addComponent(UITransform);
16153         }
16154         if (!node.getComponent(BlockInputEvents)) {
16155             node.addComponent(BlockInputEvents);
16156         }
16157     }
16158     /** 移除历史 gfx 绘图层（已改在 host 根节点 Graphics 绘制） */
16159     private _removeGfxChild(host: Node): void {
16160         const gfxNode = host.getChildByName(GFX_CHILD_NAME);
16161         if (gfxNode?.isValid) {
16162             gfxNode.destroy();
16163         }
16164     }
16165     /** 占位 Sprite 由脚本绘制接管，关闭以免与根节点 Graphics 叠显 */
16166     private _disableHostSprite(host: Node): void {
16167         const sprite = host.getComponent(Sprite);
16168         if (sprite) {
16169             sprite.enabled = false;
16170         }
16171     }
16172     private _ensurePanelBody(node: Node): void {
16173         this._applyDefaultNodeLayout(node, PANEL_WIDTH, PANEL_HEIGHT, 0, PANEL_OFFSET_Y);
16174         this._ensureTouchBlocker(node);
16175         this._removeGfxChild(node);
16176         this._disableHostSprite(node);
16177         this._panelGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
16178     }
16179     private _ensureTitleBar(node: Node): void {
16180         this._applyDefaultNodeLayout(node, TITLE_WIDTH, TITLE_HEIGHT, 0, TITLE_OFFSET_Y);
16181         this._ensureTouchBlocker(node);
16182         // 与 closebtn 一致：Graphics 挂 titlebar 根节点；勿与 Sprite 同节点（UIRenderer 互斥）

```

```

16183         this._removeGfxChild(node);
16184         this._disableHostSprite(node);
16185         this._titleGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
16186     }
16187     private _ensureCloseButton(node: Node): void {
16188         this._applyDefaultNodeLayout(node, CLOSE_SIZE, CLOSE_SIZE, CLOSE_OFFSET_X, CLOSE_OFFSET_Y);
16189         // 与 TopBar 返回键一致: Graphics 挂按钮根节点, 避免 gfx 子节点抢触摸
16190         this._removeGfxChild(node);
16191         this._disableHostSprite(node);
16192         let btn = node.getComponent(Button);
16193         const createdBtn = !btn;
16194         if (!btn) {
16195             btn = node.addComponent(Button);
16196         }
16197         if (createdBtn) {
16198             btn.transition = Button.Transition.SCALE;
16199             btn.zoomScale = 0.95;
16200         }
16201         this._closeGraphics = node.getComponent(Graphics) ?? node.addComponent(Graphics);
16202         this._closePressCtrl = new HudButtonPressController(
16203             node,
16204             (pressed) => {
16205                 this._closePressed = pressed;
16206                 this._redrawCloseButton();
16207             },
16208             true,
16209         );
16210     }
16211     private _ensureScrollArea(node: Node): void {
16212         this._applyDefaultNodeLayout(node, SCROLL_WIDTH, SCROLL_HEIGHT, 0, SCROLL_OFFSET_Y);
16213         this._ensureTouchBlocker(node);
16214         this._disableHostSprite(node);
16215         node.getComponent(Mask) ?? node.addComponent(Mask);
16216         let scrollView = node.getComponent(ScrollView);
16217         const createdScrollView = !scrollView;
16218         if (!scrollView) {
16219             scrollView = node.addComponent(ScrollView);
16220         }
16221         if (createdScrollView) {
16222             scrollView.horizontal = false;
16223             scrollView.vertical = true;
16224             scrollView.inertia = true;
16225             scrollView.brake = 0.75;
16226             scrollView.elastic = true;
16227         }
16228         const scrollUt = node.getComponent(UITransform);
16229         const scrollW = scrollUt?.width ?? SCROLL_WIDTH;
16230         const scrollH = scrollUt?.height ?? SCROLL_HEIGHT;
16231         // CC 3.8: view 为只读 UITransform, Mask 挂在 ScrollView 节点; 仅 content 可赋值
16232         let contentNode = this._resolveScrollContentNode(node);
16233         this._contentNode = contentNode;
16234         const hadContentTransform = !contentNode.getComponent(UITransform);
16235         const contentUt = contentNode.getComponent(UITransform) ??
16236         contentNode.addComponent(UITransform);
16237         if (!hadContentTransform) {
16238             contentUt.setAnchorPoint(0.5, 1);
16239             contentUt.setContentSize(scrollW, scrollH);
16240             contentNode.setPosition(0, scrollH * 0.5, 0);

```



```

16241     }
16242     scrollView.content = contentNode;
16243     this._scrollNode = node;
16244     const legacyView = node.getChildByName('view');
16245     if (legacyView?.isValid) {
16246         legacyView.active = false;
16247     }
16248     this._ensureScrollTopFade(node);
16249     this._ensureScrollBottomFade(node);
16250 }
16251 /** 滚动视口顶部渐隐蒙层（固定不随 content 滚动） */
16252 private _ensureScrollTopFade(scrollNode: Node): void {
16253     let fadeNode = scrollNode.getChildByName(SCROLL_FADE_TOP_CHILD_NAME);
16254     if (!fadeNode) {
16255         fadeNode = new Node(SCROLL_FADE_TOP_CHILD_NAME);
16256         fadeNode.setParent(scrollNode);
16257         fadeNode.addComponent(OpticalPuzzleLevelSelectScrollFadePassThrough);
16258     }
16259     fadeNode.setSiblingIndex(scrollNode.children.length - 1);
16260     const scrollUt = scrollNode.getComponent(UITransform);
16261     if (!scrollUt) {
16262         return;
16263     }
16264     const scrollH = scrollUt.height;
16265     const fadeW = LEVEL_SELECT_SCROLL_TOP_FADE_WIDTH;
16266     const fadeH = LEVEL_SELECT_SCROLL_TOP_FADE_HEIGHT;
16267     const ut = fadeNode.getComponent(UITransform) ?? fadeNode.addComponent(UITransform);
16268     ut.setAnchorPoint(0.5, 1);
16269     ut.setContentSize(fadeW, fadeH);
16270     fadeNode.setPosition(0, scrollH * 0.5, 0);
16271     this._scrollFadeGraphics = fadeNode.getComponent(Graphics) ?? fadeNode.addComponent(Graphics);
16272     this._redrawScrollTopFade();
16273 }
16274 /** 滚动视口底部渐隐蒙层（固定不随 content 滚动） */
16275 private _ensureScrollBottomFade(scrollNode: Node): void {
16276     let fadeNode = scrollNode.getChildByName(SCROLL_FADE_BOTTOM_CHILD_NAME);
16277     if (!fadeNode) {
16278         fadeNode = new Node(SCROLL_FADE_BOTTOM_CHILD_NAME);
16279         fadeNode.setParent(scrollNode);
16280         fadeNode.addComponent(OpticalPuzzleLevelSelectScrollFadePassThrough);
16281     }
16282     fadeNode.setSiblingIndex(scrollNode.children.length - 1);
16283     const scrollUt = scrollNode.getComponent(UITransform);
16284     if (!scrollUt) {
16285         return;
16286     }
16287     const scrollH = scrollUt.height;
16288     const fadeW = LEVEL_SELECT_SCROLL_BOTTOM_FADE_WIDTH;
16289     const fadeH = LEVEL_SELECT_SCROLL_BOTTOM_FADE_HEIGHT;
16290     const ut = fadeNode.getComponent(UITransform) ?? fadeNode.addComponent(UITransform);
16291     ut.setAnchorPoint(0.5, 0);
16292     ut.setContentSize(fadeW, fadeH);
16293     fadeNode.setPosition(0, -scrollH * 0.5, 0);
16294     this._scrollBottomFadeGraphics = fadeNode.getComponent(Graphics) ??
16295     fadeNode.addComponent(Graphics);
16296     this._redrawScrollBottomFade();
16297 }
16298 private _redrawScrollTopFade(): void {

```

```

16299         const g = this._scrollFadeGraphics;
16300         const ut = g?.node.getComponent(UITransform);
16301         if (!g?.isValid || !ut) {
16302             return;
16303         }
16304         g.clear();
16305         const w = ut.width;
16306         const h = ut.height;
16307         drawLevelSelectScrollTopFade(g, -w * 0.5, -h, w, h);
16308     }
16309     private _redrawScrollBottomFade(): void {
16310         const g = this._scrollBottomFadeGraphics;
16311         const ut = g?.node.getComponent(UITransform);
16312         if (!g?.isValid || !ut) {
16313             return;
16314         }
16315         g.clear();
16316         const w = ut.width;
16317         const h = ut.height;
16318         drawLevelSelectScrollBottomFade(g, -w * 0.5, 0, w, h);
16319     }
16320     /** 解析 content 节点: 优先 scrollpanel/content, 兼容旧结构 scrollpanel/view/content */
16321     private _resolveScrollContentNode(scrollNode: Node): Node {
16322         let contentNode = scrollNode.getChildByName('content');
16323         if (contentNode?.isValid) {
16324             if (contentNode.parent !== scrollNode) {
16325                 contentNode.setParent(scrollNode);
16326             }
16327             return contentNode;
16328         }
16329         const legacyView = scrollNode.getChildByName('view');
16330         contentNode = legacyView?.getChildByName('content') ?? null;
16331         if (contentNode?.isValid) {
16332             contentNode.setParent(scrollNode);
16333             return contentNode;
16334         }
16335         contentNode = new Node('content');
16336         contentNode.setParent(scrollNode);
16337         return contentNode;
16338     }
16339     private _bindCloseButton(): void {
16340         if (!this._closeNode?.isValid) {
16341             return;
16342         }
16343         this._closeNode.off(Button.EventType.CLICK, this._onCloseClick, this);
16344         this._closeNode.on(Button.EventType.CLICK, this._onCloseClick, this);
16345     }
16346     private _hidePlaceholderSplashes(): void {
16347         const walk = (node: Node): void => {
16348             if (node.name === 'SpriteSplash') {
16349                 node.active = false;
16350             }
16351             for (const child of node.children) {
16352                 walk(child);
16353             }
16354         };
16355         for (const child of this.node.children) {
16356             walk(child);

```

```
16357     }
16358 }
16359 //endregion
16360 //region 绘制
16361 private _redrawStaticChrome(): void {
16362     this._redrawPanelBody();
16363     this._redrawTitleBar();
16364     this._redrawCloseButton();
16365     this._redrawScrollTopFade();
16366     this._redrawScrollBottomFade();
16367 }
16368 private _redrawPanelBody(): void {
16369     const g = this._panelGraphics;
16370     const ut = this._panelNode?.getComponent(UITransform);
16371     if (!g?.isValid || !ut) {
16372         return;
16373     }
16374     g.clear();
16375     const w = ut.width;
16376     const h = ut.height;
16377     drawLevelSelectPanelChrome(g, -w * 0.5, -h * 0.5, w, h);
16378 }
16379 private _redrawTitleBar(): void {
16380     const g = this._titleGraphics;
16381     const ut = this._titleNode?.getComponent(UITransform);
16382     if (!g?.isValid || !ut) {
16383         return;
16384     }
16385     g.clear();
16386     const w = ut.width;
16387     const h = ut.height;
16388     drawLevelSelectTitleBar(g, -w * 0.5, -h * 0.5, w, h);
16389 }
16390 private _redrawCloseButton(): void {
16391     const g = this._closeGraphics;
16392     const ut = this._closeNode?.getComponent(UITransform);
16393     if (!g?.isValid || !ut) {
16394         return;
16395     }
16396     g.clear();
16397     const w = ut.width;
16398     const h = ut.height;
16399     drawLevelSelectCloseGlyph(g, -w * 0.5, -h * 0.5, w, h, this._closePressed);
16400 }
16401 //endregion
16402 //region 关卡列表
16403 /** 进度或关卡表变化后全量重建 */
16404 private _refreshLevelList(): void {
16405     this._startAsyncLevelListBuild(true);
16406 }
16407 private _startAsyncLevelListBuild(force = false): void {
16408     const content = this._contentNode;
16409     if (!content?.isValid) {
16410         return;
16411     }
16412     if (this._listBuilding) {
16413         return;
16414     }
```

```

16415         if (!force && this.isLevelListReady()) {
16416             return;
16417         }
16418         for (const item of this._itemNodes) {
16419             if (item?.isValid) {
16420                 item.destroy();
16421             }
16422         }
16423         this._itemNodes = [];
16424         this._sortedLevels = [...OPTICAL_LEVELS].sort((a, b) => a.levelId - b.levelId);
16425         this._buildCursor = 0;
16426         this._listBuilding = true;
16427         this._updateContentHeight(this._sortedLevels.length);
16428         this.unschedule(this._processLevelListBuildBatch);
16429         this.schedule(this._processLevelListBuildBatch, 0);
16430     }
16431     private _processLevelListBuildBatch = (): void => {
16432         if (!this._listBuilding || !this.isValid) {
16433             this.unschedule(this._processLevelListBuildBatch);
16434             return;
16435         }
16436         const content = this._contentNode;
16437         if (!content?.isValid) {
16438             this._listBuilding = false;
16439             this.unschedule(this._processLevelListBuildBatch);
16440             return;
16441         }
16442         const currentId = this._resolveCurrentPlayingLevelId();
16443         const batch = this.node.activeInHierarchy
16444             ? LEVEL_LIST_BUILD_BATCH
16445             : LEVEL_LIST_PREWARM_BATCH;
16446         const end = Math.min(this._buildCursor + batch, this._sortedLevels.length);
16447         for (let i = this._buildCursor; i < end; i++) {
16448             const level = this._sortedLevels[i];
16449             const itemNode = createLevelSelectItemNode(`LevelItem_${level.levelId}`);
16450             itemNode.setParent(content);
16451             this._layoutSingleLevelItem(itemNode, i);
16452             let visualState = LevelSelectItemVisualState.Normal;
16453             if (!this._isLevelUnlocked(level.levelId)) {
16454                 visualState = LevelSelectItemVisualState.Locked;
16455             } else if (level.levelId === currentId) {
16456                 visualState = LevelSelectItemVisualState.Current;
16457             }
16458             itemNode.getComponent(OpticalPuzzleLevelSelectItemView)?.setup(
16459                 level.levelId,
16460                 visualState,
16461                 this._onLevelItemSelectHandler,
16462             );
16463             this._itemNodes.push(itemNode);
16464         }
16465         this._buildCursor = end;
16466         if (this._buildCursor >= this._sortedLevels.length) {
16467             this._listBuilding = false;
16468             this.unschedule(this._processLevelListBuildBatch);
16469             this.syncLevelListVisuals();
16470             if (this.node.activeInHierarchy) {
16471                 this._scrollToCurrentLevelRowCenter();
16472             }

```

```

16473     }
16474 };
16475 /** 当前关所在行滚到视口垂直中心; 不够居中则滚到顶/底 */
16476 private _scrollToCurrentLevelRowCenter(): void {
16477     const scrollView = this._scrollNode?.getComponent(ScrollView);
16478     if (!scrollView?.isValid || !this._contentNode?.isValid) {
16479         return;
16480     }
16481     const scrollUt = this._scrollNode!.getComponent(UITransform);
16482     if (!scrollUt) {
16483         return;
16484     }
16485     const currentId = this._resolveCurrentPlayingLevelId();
16486     const levels = this._sortedLevels.length > 0
16487         ? this._sortedLevels
16488         : [...OPTICAL_LEVELS].sort((a, b) => a.levelId - b.levelId);
16489     const index = levels.findIndex((lv) => lv.levelId === currentId);
16490     if (index < 0) {
16491         scrollView.scrollToTop(0);
16492         return;
16493     }
16494     const scrollH = scrollUt.height;
16495     const cellSize = LEVEL_SELECT_ITEM_DESIGN_SIZE;
16496     const gap = computeLevelSelectGridGap(scrollUt.width, GRID_COLUMNS, cellSize);
16497     const row = Math.floor(index / GRID_COLUMNS);
16498     const rowCenterFromTop = gap + row * (cellSize + gap) + cellSize * 0.5;
16499     const targetOffsetY = rowCenterFromTop - scrollH * 0.5;
16500     const contentH = this._contentNode.getComponent(UITransform)?.height ?? scrollH;
16501     const maxOffsetY = Math.max(0, contentH - scrollH);
16502     if (targetOffsetY <= 0 || maxOffsetY <= 0) {
16503         scrollView.scrollToTop(0);
16504         return;
16505     }
16506     if (targetOffsetY >= maxOffsetY) {
16507         scrollView.scrollToBottom(0);
16508         return;
16509     }
16510     scrollView.scrollToOffset(new Vec2(0, targetOffsetY), 0);
16511 }
16512 private _applyLevelItemVisualStates(): void {
16513     const currentId = this._resolveCurrentPlayingLevelId();
16514     for (const itemNode of this._itemNodes) {
16515         if (!itemNode?.isValid) {
16516             continue;
16517         }
16518         const view = itemNode.getComponent(OpticalPuzzleLevelSelectItemView);
16519         const levelId = view?.getLevelId() ?? 0;
16520         if (levelId <= 0) {
16521             continue;
16522         }
16523         let visualState = LevelSelectItemVisualState.Normal;
16524         if (!this._isLevelUnlocked(levelId)) {
16525             visualState = LevelSelectItemVisualState.Locked;
16526         } else if (levelId === currentId) {
16527             visualState = LevelSelectItemVisualState.Current;
16528         }
16529         view?.applyVisualState(visualState);
16530         view?.refresh();
16531     }

```

```

16531     }
16532 }
16533 private _layoutSingleLevelItem(node: Node, index: number): void {
16534     const scrollUt = this._scrollNode?.getComponent(UITransform);
16535     const scrollW = scrollUt?.width ?? SCROLL_WIDTH;
16536     const cellSize = LEVEL_SELECT_ITEM_DESIGN_SIZE;
16537     const cols = GRID_COLUMNS;
16538     const gap = computeLevelSelectGridGap(scrollW, cols, cellSize);
16539     const row = Math.floor(index / cols);
16540     const col = index % cols;
16541     const cx = levelSelectItemCenterX(scrollW, cellSize, gap, col);
16542     const cy = -gap - row * (cellSize + gap) - cellSize * 0.5;
16543     node.setPosition(cx, cy, 0);
16544 }
16545 private _updateContentHeight(itemCount: number): void {
16546     const content = this._contentNode;
16547     if (!content?.isValid) {
16548         return;
16549     }
16550     const legacyLayout = content.getComponent(Layout);
16551     if (legacyLayout) {
16552         legacyLayout.enabled = false;
16553     }
16554     const scrollUt = this._scrollNode?.getComponent(UITransform);
16555     const scrollW = scrollUt?.width ?? SCROLL_WIDTH;
16556     const scrollH = scrollUt?.height ?? SCROLL_HEIGHT;
16557     const cellSize = LEVEL_SELECT_ITEM_DESIGN_SIZE;
16558     const gap = computeLevelSelectGridGap(scrollW, GRID_COLUMNS, cellSize);
16559     const rowCount = Math.max(1, Math.ceil(itemCount / GRID_COLUMNS));
16560     const contentUt = content.getComponent(UITransform);
16561     if (contentUt) {
16562         contentUt.setAnchorPoint(0.5, 1);
16563         contentUt.width = scrollW;
16564         const contentH = gap + rowCount * cellSize + Math.max(0, rowCount - 1) * gap + gap;
16565         contentUt.height = Math.max(scrollH, contentH);
16566     }
16567 }
16568 /** Menu 场景用存档进度；Game 场景优先用局内当前关卡 */
16569 private _resolveCurrentPlayingLevelId(): number {
16570     if (director.getScene()?.name === SCENE_ENUM.GAME) {
16571         const root = this._resolveOpticalPuzzleRoot();
16572         const inGameId = root?.getCurrentLevelId?.() ?? 0;
16573         if (inGameId > 0) {
16574             return inGameId;
16575         }
16576     }
16577     return DataManager.instance.opticalCurrentLevelId;
16578 }
16579 /** 是否在 opticalLevelClears 中有记录（含 999999 仅解锁占位） */
16580 private _isLevelUnlocked(levelId: number): boolean {
16581     return DataManager.instance.isOpticalLevelUnlocked(levelId);
16582 }
16583 //endregion
16584 //region 交互
16585 /** 仅点击 bg 空白遮罩时关闭（不响应 panel/titlebar 等穿透或冒泡） */
16586 private _onBackdropTouchEnd(event: EventTouch): void {
16587     if (event.target !== this._backdropNode) {
16588         return;

```

```

16589     }
16590     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
16591     this.close();
16592 }
16593 private _onCloseClick(): void {
16594     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
16595     this.close();
16596 }
16597 private _onLevelItemSelected(levelId: number): void {
16598     if (!this._isLevelUnlocked(levelId)) {
16599         SHOW_TOAST.emit(EVENT_ENUM.SHOW_TOAST, {
16600             message: '关卡尚未解锁',
16601             bgWidth: 320,
16602         });
16603         return;
16604     }
16605     PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
16606     DataManager.instance.opticalCurrentLevelId = levelId;
16607     if (director.getScene()?.name === SCENE_ENUM.GAME) {
16608         this.close();
16609         this._resolveOpticalPuzzleRoot()?.loadLevelById(levelId);
16610         return;
16611     }
16612     // Menu → Game: 不切走选关层, 避免先露出 Menu 再 loadScene 闪一下
16613     director.loadScene(SCENE_ENUM.GAME);
16614 }
16615 private _resolveGameRoot(): Node | null {
16616     let node: Node | null = this.node;
16617     while (node?.parent) {
16618         if (node.name === 'GameRoot') {
16619             return node;
16620         }
16621         node = node.parent;
16622     }
16623     return null;
16624 }
16625 private _resolveOpticalPuzzleRoot(): IOpticalPuzzleRootApi | null {
16626     const gameRoot = this._resolveGameRoot();
16627     if (!gameRoot?.isValid) {
16628         return null;
16629     }
16630     const comp =
16631         gameRoot.getComponent('OpticalPuzzleRoot') ??
16632         gameRoot.getComponentInChildren('OpticalPuzzleRoot');
16633     return comp as unknown as IOpticalPuzzleRootApi | null;
16634 }
16635 //endregion
16636 }
16637 /** 为选关面板根节点挂上完整逻辑（复制空节点时可由 Manager 调用） */
16638 export function ensureLevelSelectPanel(panelNode: Node | null): void {
16639     if (!panelNode?.isValid) {
16640         return;
16641     }
16642     if (!panelNode.getComponent(OpticalPuzzleLevelSelectPanel)) {
16643         panelNode.addComponent(OpticalPuzzleLevelSelectPanel);
16644     }
16645 }
16646 /** 进入场景后预构建选关列表（Game / Menu 的 Manager onLoad 调用） */

```

```

16647 export function prewarmLevelSelectPanel(panelNode: Node | null): void {
16648     panelNode?.getComponent(OpticalPuzzleLevelSelectPanel)?.prewarmLevelList();
16649 }
16650 /** 打开选关面板并刷新解锁状态 */
16651 export function openLevelSelectPanel(panelNode: Node | null): void {
16652     if (!panelNode?.isValid) {
16653         return;
16654     }
16655     // 用户打开选关时再次预加载 Game，降低选关后进局等待（Menu onLoad 已预载一次）
16656     director.preloadScene(SCENE_ENUM.GAME);
16657     panelNode.active = true;
16658     panelNode.getComponent(OpticalPuzzleLevelSelectPanel)?.ensureLevelListReadyForOpen();
16659 }
16660 /** 同步当前关 / 解锁高亮（换关、通关后调用） */
16661 export function syncLevelSelectPanelVisuals(panelNode: Node | null): void {
16662     panelNode?.getComponent(OpticalPuzzleLevelSelectPanel)?.syncLevelListVisuals();
16663 }
16664 /** 自场景根解析 layerOverlay 下选关面板（兼容 levelSelectPanel / LevelSelectPanel 命名） */
16665 export function resolveLevelSelectPanelNode(root: Node | null): Node | null {
16666     const overlay = root?.getChildByName('layerOverlay') ?? null;
16667     if (!overlay?.isValid) {
16668         return null;
16669     }
16670     return (
16671         overlay.getChildByName('levelSelectPanel') ??
16672         overlay.getChildByName('LevelSelectPanel') ??
16673         null
16674     );
16675 }
16676 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectPanelGlyph.ts -----
16677 import { Color, Graphics, Label } from 'cc';
16678 import {
16679     HUD_DIR_BUTTON_SCENE_SIZE,
16680     HUD_KEY_FILL,
16681     PRESSED_GLOW_LAYERS,
16682     pressedHudIconColor,
16683     scaleHudDesign,
16684     strokeGlowLayers,
16685     unlitHudIconColor,
16686 } from './OpticalPuzzleHudButtonCommon';
16687 import { drawWinPanelWindsChrome } from './OpticalPuzzleWinPanelWindsView';
16688 import { OpticalStarVisualState, type OpticalStarSlotStates } from './Core/OpticalPuzzleStarRating';
16689 import { drawWinPanelStarGlyph, WIN_STAR_BORDER_PX } from './OpticalPuzzleWinPanelStarGlyph';
16690 import { WIN_STAR_DESIGN_SIZE } from './OpticalPuzzleWinPanelStarsView';
16691 /** 选关面板内容区圆角（设计 px） */
16692 export const LEVEL_SELECT_PANEL_CORNER_PX = 20;
16693 /** 标题条圆角（设计 px） */
16694 export const LEVEL_SELECT_TITLE_CORNER_PX = 15;
16695 /** 标题条白描边（设计 px） */
16696 export const LEVEL_SELECT_TITLE_BORDER_PX = 5;
16697 /** 关卡项设计边长（4 列网格 cell 尺寸） */
16698 export const LEVEL_SELECT_ITEM_DESIGN_SIZE = 110;
16699 /** 关卡项外框白描边（固定 px，不随格尺寸缩放） */
16700 export const LEVEL_SELECT_ITEM_BORDER_PX = 5;
16701 /** 选关项底部单颗星边长（设计 px） */
16702 export const LEVEL_SELECT_ITEM_STAR_SIZE = 18;
16703 /** 底部三颗星间距（设计 px） */
16704 export const LEVEL_SELECT_ITEM_STAR_GAP = 4;

```



```

16705  /** 星形底边距关卡项底边（设计 px，贴下边缘白框） */
16706  export const LEVEL_SELECT_ITEM_STAR_BOTTOM_INSET = 4;
16707  /** 未点亮星描边相对点亮星加粗倍数 */
16708  const LEVEL_SELECT_ITEM_STAR_DIM_BORDER_MUL = 1.85;
16709  /** 关卡号 Label 字号（与 ITEM_DESIGN_SIZE 对齐） */
16710  export const LEVEL_SELECT_ITEM_LABEL_FONT = 30;
16711  /** 关卡号清晰渲染：字号 ×10 后节点 scale 0.1 还原视觉大小 */
16712  export const LEVEL_SELECT_ITEM_LABEL_FONT_SCALE = 10;
16713  export const LEVEL_SELECT_ITEM_LABEL_NODE_SCALE = 0.1;
16714  /** 关卡号 Font Style 字体族（useSystemFont 时生效） */
16715  export const LEVEL_SELECT_ITEM_LABEL_FONT_FAMILY = 'Arial';
16716  /** 关卡号距项左上角边距（设计 px） */
16717  export const LEVEL_SELECT_ITEM_LABEL_MARGIN = 5;
16718  /** 缩略图区内边距（设计 px） */
16719  export const LEVEL_SELECT_ITEM_THUMB_PADDING = 6;
16720  /** 缩略图内关卡沙盘缩放（相对内容区，略小于 1 留出边距） */
16721  export const LEVEL_SELECT_ITEM_THUMB_BOARD_SCALE = 0.9;
16722  /** 滚动区顶部渐隐蒙层宽度（设计 px，略小于 scrollpanel 宽，避免盖住左右白框） */
16723  export const LEVEL_SELECT_SCROLL_TOP_FADE_WIDTH = 645;
16724  /** 滚动区顶部渐隐蒙层高度（设计 px，与面板底色 HUD_KEY_FILL 衔接） */
16725  export const LEVEL_SELECT_SCROLL_TOP_FADE_HEIGHT = 40;
16726  /** 滚动区底部渐隐蒙层宽度（设计 px，与顶部一致，避免盖住左右白框） */
16727  export const LEVEL_SELECT_SCROLL_BOTTOM_FADE_WIDTH = 645;
16728  /** 滚动区底部渐隐蒙层高度（设计 px） */
16729  export const LEVEL_SELECT_SCROLL_BOTTOM_FADE_HEIGHT = 40;
16730  /** 关闭键设计边长 */
16731  export const LEVEL_SELECT_CLOSE_DESIGN_SIZE = 80;
16732  /** 关闭键圆形外框白描边（设计 px） */
16733  export const LEVEL_SELECT_CLOSE_BORDER_PX = 5;
16734  /** 关闭键 X 外轮廓 / 内芯 / 半臂长（设计 px，80×80 基准） */
16735  export const LEVEL_SELECT_CLOSE_X_OUTLINE_PX = 3;
16736  export const LEVEL_SELECT_CLOSE_X_BODY_PX = 6;
16737  export const LEVEL_SELECT_CLOSE_X_HALF_PX = 14;
16738  /** 关卡项视觉状态 */
16739  export enum LevelSelectItemVisualState {
16740      Locked = 1,
16741      Normal = 2,
16742      Current = 3,
16743  }
16744  /** 锁定关键帽底色（整格圆角，不再叠缩略图矩形蒙层） */
16745  export const LEVEL_SELECT_LOCKED_KEY_FILL = new Color(52, 56, 66, 255);
16746  const CURRENT_GLOW = new Color(255, 220, 80, 255);
16747  /** strokeGlowLayers 用 plain RGB（勿传 Color 实例） */
16748  const CURRENT_GLOW_RGB = { r: 255, g: 220, b: 80 } as const;
16749  const WHITE_GLOW_RGB = { r: 255, g: 255, b: 255 } as const;
16750  /** 当前关未按下：两层、光晕略小 */
16751  const CURRENT_IDLE_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
16752      { width: 6, alpha: 48 },
16753      { width: 3, alpha: 140 },
16754  ];
16755  /** 当前关按下：原三层光晕样式（与 HUD_PRESSED 同结构） */
16756  const CURRENT_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
16757      { width: 10, alpha: 48 },
16758      { width: 6, alpha: 120 },
16759      { width: 3, alpha: 255 },
16760  ];
16761  /** 全屏半透明遮罩 */
16762  export function drawLevelSelectBackdrop(

```

```
16763     g: Graphics,
16764     left: number,
16765     bottom: number,
16766     width: number,
16767     height: number,
16768 ): void {
16769     g.fillColor = new Color(0, 0, 0, 140);
16770     g.rect(left, bottom, width, height);
16771     g.fill();
16772 }
16773 /** 选关主面板底（与 winPanel/winds 同风格） */
16774 export function drawLevelSelectPanelChrome(
16775     g: Graphics,
16776     left: number,
16777     bottom: number,
16778     width: number,
16779     height: number,
16780 ): void {
16781     drawWinPanelWindsChrome(g, left, bottom, width, height);
16782 }
16783 /** 滚动区顶部渐隐：自上而下由面板底色过渡到透明（关卡滚出时软消失） */
16784 export function drawLevelSelectScrollTopFade(
16785     g: Graphics,
16786     left: number,
16787     bottom: number,
16788     width: number,
16789     fadeHeight: number,
16790     solid: Color = HUD_KEY_FILL,
16791 ): void {
16792     const steps = 18;
16793     const sliceH = fadeHeight / steps;
16794     for (let i = 0; i < steps; i++) {
16795         const tMid = (i + 0.5) / steps;
16796         const alpha = Math.round(solid.a * tMid);
16797         g.fillColor = new Color(solid.r, solid.g, solid.b, alpha);
16798         g.rect(left, bottom + i * sliceH, width, sliceH + 0.5);
16799         g.fill();
16800     }
16801 }
16802 /** 滚动区底部渐隐：自下而上由面板底色过渡到透明（关卡滚出时软消失） */
16803 export function drawLevelSelectScrollBottomFade(
16804     g: Graphics,
16805     left: number,
16806     bottom: number,
16807     width: number,
16808     fadeHeight: number,
16809     solid: Color = HUD_KEY_FILL,
16810 ): void {
16811     const steps = 18;
16812     const sliceH = fadeHeight / steps;
16813     for (let i = 0; i < steps; i++) {
16814         const tMid = (i + 0.5) / steps;
16815         const alpha = Math.round(solid.a * (1 - tMid));
16816         g.fillColor = new Color(solid.r, solid.g, solid.b, alpha);
16817         g.rect(left, bottom + i * sliceH, width, sliceH + 0.5);
16818         g.fill();
16819     }
16820 }
```

```
16821 const TITLE_BORDER_COLOR = new Color(255, 255, 255, 255);
16822 const CLOSE_BORDER_COLOR = new Color(255, 255, 255, 255);
16823 function traceCloseX(g: Graphics, cx: number, cy: number, half: number): void {
16824     g.moveTo(cx - half, cy - half);
16825     g.lineTo(cx + half, cy + half);
16826     g.moveTo(cx + half, cy - half);
16827     g.lineTo(cx - half, cy + half);
16828 }
16829 /** 关闭键 80×80, 按下光晕按 100×100 四向键线宽缩放 (+25%) */
16830 const CLOSE_PRESSED_GLOW_SCALE_SIZE = HUD_DIR_BUTTON_SCENE_SIZE;
16831 /** 按下时: PRESSED_GLOW_LAYERS 在 xBody 上逐层加粗 (光晕画在实芯之上, 线宽 +25%) */
16832 function strokeCloseXStackedGlow(
16833     g: Graphics,
16834     size: number,
16835     traceX: () => void,
16836     bodyDesignWidth: number,
16837 ): void {
16838     const baseW = Math.max(1, scaleHudDesign(size, bodyDesignWidth));
16839     g.lineJoin = Graphics.LineJoin.ROUND;
16840     g.lineCap = Graphics.LineCap.ROUND;
16841     for (const layer of PRESSED_GLOW_LAYERS) {
16842         g.strokeColor = new Color(255, 255, 255, layer.alpha);
16843         g.lineWidth =
16844             baseW + Math.max(1, scaleHudDesign(CLOSE_PRESSED_GLOW_SCALE_SIZE, layer.width));
16845         traceX();
16846         g.stroke();
16847     }
16848 }
16849 /** 标题条: 窗口同色底 + 圆角白描边 (5px) */
16850 export function drawLevelSelectTitleBar(
16851     g: Graphics,
16852     left: number,
16853     bottom: number,
16854     width: number,
16855     height: number,
16856 ): void {
16857     g.fillColor = HUD_KEY_FILL;
16858     g.roundRect(left, bottom, width, height, LEVEL_SELECT_TITLE_CORNER_PX);
16859     g.fill();
16860     g.strokeColor = TITLE_BORDER_COLOR;
16861     g.lineWidth = LEVEL_SELECT_TITLE_BORDER_PX;
16862     g.roundRect(left, bottom, width, height, LEVEL_SELECT_TITLE_CORNER_PX);
16863     g.stroke();
16864 }
16865 /** 关闭键: 圆形外框 + 加粗 x (外轮廓 5px) */
16866 export function drawLevelSelectCloseGlyph(
16867     g: Graphics,
16868     left: number,
16869     bottom: number,
16870     width: number,
16871     height: number,
16872     pressed: boolean,
16873 ): void {
16874     const size = Math.min(width, height);
16875     const cx = left + width * 0.5;
16876     const cy = bottom + height * 0.5;
16877     const radius = size * 0.5 - LEVEL_SELECT_CLOSE_BORDER_PX * 0.5;
16878     const traceCircle = (): void => {
```

```

16879         g.circle(cx, cy, radius);
16880     };
16881     g.fillColor = HUD_KEY_FILL;
16882     traceCircle();
16883     g.fill();
16884     g.lineJoin = Graphics.LineJoin.ROUND;
16885     g.lineCap = Graphics.LineCap.ROUND;
16886     if (pressed) {
16887         strokeGlowLayers(g, CLOSE_PRESSED_GLOW_SCALE_SIZE, PRESSED_GLOW_LAYERS, traceCircle, false);
16888     } else {
16889         g.strokeColor = CLOSE_BORDER_COLOR;
16890         g.lineWidth = LEVEL_SELECT_CLOSE_BORDER_PX;
16891         traceCircle();
16892         g.stroke();
16893     }
16894     const xHalf = scaleHudDesign(size, LEVEL_SELECT_CLOSE_X_HALF_PX);
16895     const xBody = LEVEL_SELECT_CLOSE_X_BODY_PX;
16896     const xOutline = LEVEL_SELECT_CLOSE_X_OUTLINE_PX;
16897     const xTotal = xBody + xOutline * 2;
16898     const traceX = (): void => traceCloseX(g, cx, cy, xHalf);
16899     if (pressed) {
16900         g.strokeColor = pressedHudIconColor();
16901         g.lineWidth = xBody;
16902         traceX();
16903         g.stroke();
16904         strokeCloseXStackedGlow(g, size, traceX, xBody);
16905         return;
16906     }
16907     g.strokeColor = CLOSE_BORDER_COLOR;
16908     g.lineWidth = xTotal;
16909     traceX();
16910     g.stroke();
16911     g.strokeColor = unlitHudIconColor();
16912     g.lineWidth = xBody;
16913     traceX();
16914     g.stroke();
16915 }
16916 /** 关卡项键帽：锁定 / 可选 / 当前进行中（黄框 + 黄光晕） */
16917 export function drawLevelSelectItemChrome(
16918     g: Graphics,
16919     left: number,
16920     bottom: number,
16921     width: number,
16922     height: number,
16923     state: LevelSelectItemVisualState,
16924     pressed: boolean,
16925 ): void {
16926     const size = Math.min(width, height);
16927     const corner = scaleHudDesign(size, 15);
16928     if (state === LevelSelectItemVisualState.Locked) {
16929         g.fillColor = LEVEL_SELECT_LOCKED_KEY_FILL;
16930     } else {
16931         g.fillColor = HUD_KEY_FILL;
16932     }
16933     g.roundRect(left, bottom, width, height, corner);
16934     g.fill();
16935     const traceFrame = (): void => {
16936         g.roundRect(left, bottom, width, height, corner);

```

```

16937     };
16938     // 未按下：当前关两层小光晕
16939     if (state === LevelSelectItemVisualState.Current && !pressed) {
16940         strokeGlowLayers(g, size, CURRENT_IDLE_GLOW_LAYERS, traceFrame, false, CURRENT_GLOW_RGB);
16941     }
16942     // 按下：当前关三层光晕；其余可选关白色 HUD 光晕
16943     if (pressed && state !== LevelSelectItemVisualState.Locked) {
16944         if (state === LevelSelectItemVisualState.Current) {
16945             strokeGlowLayers(g, size, CURRENT_GLOW_LAYERS, traceFrame, false, CURRENT_GLOW_RGB);
16946         } else {
16947             strokeGlowLayers(g, size, PRESSED_GLOW_LAYERS, traceFrame, false, WHITE_GLOW_RGB);
16948         }
16949     }
16950     // 可选关按下时仅光晕、不描边（与 drawHudButtonChrome 一致）；当前关/锁定/未按可选关仍描边
16951     const drawBorder = state === LevelSelectItemVisualState.Locked
16952         || state === LevelSelectItemVisualState.Current
16953         || !pressed;
16954     if (drawBorder) {
16955         const borderW = LEVEL_SELECT_ITEM_BORDER_PX;
16956         g.strokeStyle = state === LevelSelectItemVisualState.Locked
16957             ? new Color(160, 168, 180, 160)
16958             : state === LevelSelectItemVisualState.Current
16959                 ? CURRENT_GLOW
16960                 : new Color(255, 255, 255, 255);
16961         g.lineWidth = borderW;
16962         traceFrame();
16963         g.stroke();
16964     }
16965 }
16966 /** 锁定态小挂锁（画在项中央偏上，给关卡号留空） */
16967 export function drawLevelSelectLockIcon(
16968     g: Graphics,
16969     cx: number,
16970     cy: number,
16971     size: number,
16972 ): void {
16973     const bodyW = scaleHudDesign(size, 24);
16974     const bodyH = scaleHudDesign(size, 17);
16975     const shackleR = scaleHudDesign(size, 9);
16976     const lineW = Math.max(2.5, scaleHudDesign(size, 3.5));
16977     g.lineWidth = lineW;
16978     g.lineJoin = Graphics.LineJoin.ROUND;
16979     g.lineCap = Graphics.LineCap.ROUND;
16980     g.strokeStyle = unlitHudIconColor();
16981     const bodyBottom = cy - scaleHudDesign(size, 4);
16982     const bodyTop = bodyBottom + bodyH;
16983     g.rect(cx - bodyW * 0.5, bodyBottom, bodyW, bodyH);
16984     g.stroke();
16985     g.arc(cx, bodyTop, shackleR, Math.PI, 0, false);
16986     g.stroke();
16987 }
16988 /** 关卡项底边三颗星（左→右对应 1~3 星；复用通关页 glyph） */
16989 export function drawLevelSelectItemStars(
16990     g: Graphics,
16991     left: number,
16992     bottom: number,
16993     width: number,
16994     states: OpticalStarSlotStates,

```

```

16995 ): void {
16996     const starSize = LEVEL_SELECT_ITEM_STAR_SIZE;
16997     const gap = LEVEL_SELECT_ITEM_STAR_GAP;
17000     const totalW = starSize * 3 + gap * 2;
17001     const rowLeft = left + (width - totalW) * 0.5;
17002     const starBottom = bottom + LEVEL_SELECT_ITEM_STAR_BOTTOM_INSET;
17003     const baseBorderPx = Math.max(
17004         1,
17005         (starSize / WIN_STAR_DESIGN_SIZE) * WIN_STAR_BORDER_PX,
17006     );
17007     const dimBorderPx = Math.max(2, baseBorderPx * LEVEL_SELECT_ITEM_STAR_DIM_BORDER_MUL);
17008     for (let i = 0; i < 3; i++) {
17009         const borderPx = states[i] === OpticalStarVisualState.Dim ? dimBorderPx : baseBorderPx;
17010         drawWinPanelStarGlyph(
17011             g,
17012             rowLeft + i * (starSize + gap),
17013             starBottom,
17014             starSize,
17015             starSize,
17016             states[i],
17017             0,
17018             borderPx,
17019         );
17020     }
17021 }
17022 /** 3.7+ Label 内部 TextStyle (引擎未公开导出, Font Style 粗体走 isBold) */
17023 interface ILevelSelectLabelTextStyle {
17024     isBold: boolean;
17025 }
17026 /** 应用关卡号 Label Font Style 粗体 (系统字 + textStyle.isBold, 禁用 CHAR 缓存) */
17027 export function applyLevelSelectItemLabelFontStyle(label: Label): void {
17028     label.useSystemFont = true;
17029     label.fontFamily = LEVEL_SELECT_ITEM_LABEL_FONT_FAMILY;
17030     label.cacheMode = Label.CacheMode.NONE;
17031     label.isBold = true;
17032     const textStyle = (label as Label & { textStyle?: ILevelSelectLabelTextStyle }).textStyle;
17033     if (textStyle) {
17034         textStyle.isBold = true;
17035     }
17036     label.updateRenderData(true);
17037 }
17038 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectScrollFadePassThrough.ts
17039 -----
17040 import { _decorator, Component, EventTouch, NodeEventType } from 'cc';
17041 const { ccclass } = _decorator;
17042 const PASS_THROUGH_TOUCH_TYPES = [
17043     NodeEventType.TOUCH_START,
17044     NodeEventType.TOUCH_MOVE,
17045     NodeEventType.TOUCH_END,
17046     NodeEventType.TOUCH_CANCEL,
17047 ] as const;
17048 /** 渐隐蒙层不参与触摸吞噬, ScrollView 仍可拖动 */
17049 @ccclass('OpticalPuzzleLevelSelectScrollFadePassThrough')
17050 export class OpticalPuzzleLevelSelectScrollFadePassThrough extends Component {
17051     protected onLoad(): void {
17052         for (const type of PASS_THROUGH_TOUCH_TYPES) {
17053             this.node.on(type, this._passTouch, this, true);
17054         }
17055     }

```

```

17053     }
17054     protected onDestroy(): void {
17055         for (const type of PASS_THROUGH_TOUCH_TYPES) {
17056             this.node.off(type, this._passTouch, this, true);
17057         }
17058     }
17059     private _passTouch(event: EventTouch): void {
17060         event.preventSwallow = true;
17061     }
17062 }
17063 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleMoveAnim.ts -----
17064 import type { OpticalBoardSnapshot } from '../Core/OpticalPuzzleTypes';
17065 import { Direction } from '../Core/OpticalPuzzleTypes';
17066 import { cellScreenRect } from './OpticalPuzzleWallDraw';
17067 /** 主角 / 元件滑格时长（秒） */
17068 export const MOVE_ANIM_DURATION = 0.1;
17069 /** 挤压峰值：运动轴 1.06，垂直轴 0.8（中点达到） */
17070 const SQUASH_AXIS_PEAK = 1.06;
17071 const SQUASH_AXIS_DIP = 0.8;
17072 export interface SquashedCellRect {
17073     left: number;
17074     bottom: number;
17075     width: number;
17076     height: number;
17077 }
17078 export interface MoveAnimEntity {
17079     kind: 'player' | 'piece';
17080     pieceIndex?: number;
17081     fromX: number;
17082     fromY: number;
17083     toX: number;
17084     toY: number;
17085     direction: Direction;
17086 }
17087 export interface MoveAnimState {
17088     elapsed: number;
17089     snapshot: OpticalBoardSnapshot;
17090     entities: MoveAnimEntity[];
17091 }
17092 export function moveAnimProgress(elapsed: number): number {
17093     return Math.min(1, Math.max(0, elapsed / MOVE_ANIM_DURATION));
17094 }
17095 /** 0→1→0, t=0.5 时为 1 */
17096 function squashWave(progress: number): number {
17097     return Math.sin(progress * Math.PI);
17098 }
17099 export function squashScales(dir: Direction, progress: number): { sx: number; sy: number } {
17100     const a = squashWave(progress);
17101     const horizontal = dir === Direction.Left || dir === Direction.Right;
17102     if (horizontal) {
17103         return {
17104             sx: 1 + (SQUASH_AXIS_PEAK - 1) * a,
17105             sy: 1 - (1 - SQUASH_AXIS_DIP) * a,
17106         };
17107     }
17108     return {
17109         sx: 1 - (1 - SQUASH_AXIS_DIP) * a,
17110         sy: 1 + (SQUASH_AXIS_PEAK - 1) * a,

```

```

17111     };
17112 }
17113 export function directionFromGridDelta(dx: number, dy: number): Direction {
17114     if (dx > 0) {
17115         return Direction.Right;
17116     }
17117     if (dx < 0) {
17118         return Direction.Left;
17119     }
17120     if (dy > 0) {
17121         return Direction.Up;
17122     }
17123     if (dy < 0) {
17124         return Direction.Down;
17125     }
17126     return Direction.Left;
17127 }
17128 /** 格子坐标插值 + 以格心为锚的挤压缩放 */
17129 export function animatedSquashedCellRect(
17130     ox: number,
17131     oy: number,
17132     cell: number,
17133     fromX: number,
17134     fromY: number,
17135     toX: number,
17136     toY: number,
17137     progress: number,
17138     dir: Direction,
17139 ): SquashedCellRect {
17140     const gx = fromX + (toX - fromX) * progress;
17141     const gy = fromY + (toY - fromY) * progress;
17142     const base = cellScreenRect(ox, oy, gx, gy, cell);
17143     const { sx, sy } = squashScales(dir, progress);
17144     const cx = base.left + base.size * 0.5;
17145     const cy = base.bottom + base.size * 0.5;
17146     const w = base.size * sx;
17147     const h = base.size * sy;
17148     return {
17149         left: cx - w * 0.5,
17150         bottom: cy - h * 0.5,
17151         width: w,
17152         height: h,
17153     };
17154 }
17155 export function buildMoveAnimEntities(
17156     fromSnap: OpticalBoardSnapshot,
17157     toSnap: OpticalBoardSnapshot,
17158 ): MoveAnimEntity[] {
17159     const entities: MoveAnimEntity[] = [];
17160     const pdx = toSnap.player.x - fromSnap.player.x;
17161     const pdy = toSnap.player.y - fromSnap.player.y;
17162     if (pdx !== 0 || pdy !== 0) {
17163         entities.push({
17164             kind: 'player',
17165             fromX: fromSnap.player.x,
17166             fromY: fromSnap.player.y,
17167             toX: toSnap.player.x,
17168             toY: toSnap.player.y,

```



```

17169         direction: directionFromGridDelta(pdx, pdy),
17170     });
17171 }
17172 const count = Math.min(fromSnap.pieces.length, toSnap.pieces.length);
17173 for (let i = 0; i < count; i++) {
17174     const from = fromSnap.pieces[i];
17175     const to = toSnap.pieces[i];
17176     const dx = to.x - from.x;
17177     const dy = to.y - from.y;
17178     if (dx === 0 && dy === 0) {
17179         continue;
17180     }
17181     entities.push({
17182         kind: 'piece',
17183         pieceIndex: i,
17184         fromX: from.x,
17185         fromY: from.y,
17186         toX: to.x,
17187         toY: to.y,
17188         direction: directionFromGridDelta(dx, dy),
17189     });
17190 }
17191 return entities;
17192 }
17193 /** 移动失败：主角原地挤压形变，元件不动 */
17194 export function buildFailedMovePlayerEntity(snapshot: OpticalBoardSnapshot): MoveAnimEntity {
17195     const { x, y } = snapshot.player;
17196     const direction = snapshot.playerFacing ?? Direction.Left;
17197     return {
17198         kind: 'player',
17199         fromX: x,
17200         fromY: y,
17201         toX: x,
17202         toY: y,
17203         direction,
17204     };
17205 }
17206 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzlePieceGlyph.ts -----
17207 import { Color, Graphics } from 'cc';
17208 import {
17209     openDirectionsForPiece,
17210     type PieceConnectivity,
17211 } from '../Core/OpticalPieceConnectivity';
17212 import { Direction } from '../Core/OpticalPuzzleTypes';
17213 import {
17214     pieceBaseFillColor,
17215     pieceChannelColors,
17216     playerBaseBorderColor,
17217     targetUnlitBulbFillColor,
17218 } from './OpticalPuzzleColorUtil';
17219 import { OPTICAL_CELL_SIZE } from './OpticalPuzzleLayout';
17220 const ARM_WIDTH_RATIO = 0.14;
17221 const HOLE_RATIO = 0.3;
17222 /** 臂端距格边留白（0 = 延至格缘） */
17223 const ARM_EDGE_INSET = 1;
17224 /** 元件格底圆角（设计像素，随格宽等比缩放） */
17225 const PIECE_CORNER_RADIUS = 4;
17226 /** 与 Target / Player 外缘一致：bezel 环宽、内面板圆角系数 */

```

```

17227 const BEZEL_RATIO = 0.017;
17228 const INNER_CORNER_FACTOR = 0.7;
17229 /** 内面板描边线宽比例（与 TargetGlyph 点亮内框一致） */
17230 const INNER_FRAME_STROKE_RATIO = 0.008;
17231 function scaledPieceCornerRadius(size: number): number {
17232     return PIECE_CORNER_RADIUS * (size / OPTICAL_CELL_SIZE);
17233 }
17234 const SCREEN_DIR_DX: ReadonlyArray<number> = [1, 0, -1, 0];
17235 const SCREEN_DIR_DY: ReadonlyArray<number> = [0, 1, 0, -1];
17236 function drawThickArm(
17237     g: Graphics,
17238     cx: number,
17239     cy: number,
17240     dir: Direction,
17241     holeR: number,
17242     armLen: number,
17243     halfW: number,
17244 ): void {
17245     const dx = SCREEN_DIR_DX[dir];
17246     const dy = SCREEN_DIR_DY[dir];
17247     const px = -dy * halfW;
17248     const py = dx * halfW;
17249     const x0 = cx + dx * holeR;
17250     const y0 = cy + dy * holeR;
17251     const x1 = cx + dx * (holeR + armLen);
17252     const y1 = cy + dy * (holeR + armLen);
17253     g.moveTo(x0 + px, y0 + py);
17254     g.lineTo(x1 + px, y1 + py);
17255     g.lineTo(x1 - px, y1 - py);
17256     g.lineTo(x0 - px, y0 - py);
17257     g.close();
17258     g.fill();
17259     g.stroke();
17260 }
17261 /** 挡光元件（connectivity 0）：Target 式黑边 bezel + 纯色内面板 */
17262 function drawBlockingPieceFrame(
17263     g: Graphics,
17264     left: number,
17265     bottom: number,
17266     width: number,
17267     height: number,
17268     refSize: number,
17269     innerFill: Color,
17270 ): void {
17271     const corner = scaledPieceCornerRadius(refSize);
17272     const bezel = refSize * BEZEL_RATIO;
17273     const innerLeft = left + bezel;
17274     const innerBottom = bottom + bezel;
17275     const innerWidth = width - bezel * 2;
17276     const innerHeight = height - bezel * 2;
17277     const innerCorner = corner * INNER_CORNER_FACTOR;
17278     const border = playerBaseBorderColor();
17279     g.fillStyle = border;
17280     g.roundRect(left, bottom, width, height, corner);
17281     g.fill();
17282     g.fillStyle = innerFill;
17283     g.roundRect(innerLeft, innerBottom, innerWidth, innerHeight, innerCorner);
17284     g.fill();

```

```

17285     g.strokeColor = border;
17286     g.lineWidth = Math.max(1, refSize * INNER_FRAME_STROKE_RATIO);
17287     g.roundRect(innerLeft, innerBottom, innerWidth, innerHeight, innerCorner);
17288     g.stroke();
17289 }
17290 function blockingPieceInnerFill(colorKey?: string): Color {
17291     return targetUnlitBulbFillColor(colorKey);
17292 }
17293 /**
17294  * 绘制统一通道元件占位：格心留空（后期换图），开口方向画直角通道臂。
17295  * @param width 格宽（挤压动画时可非正方形）
17296  * @param height 格高，默认与 width 相同
17297  */
17298 export function drawConnectivityGlyph(
17299     g: Graphics,
17300     left: number,
17301     top: number,
17302     width: number,
17303     connectivity: PieceConnectivity,
17304     pieceDirection: Direction,
17305     colorKey?: string,
17306     height?: number,
17307 ): void {
17308     const h = height ?? width;
17309     const refSize = Math.min(width, h);
17310     const cx = left + width * 0.5;
17311     const cy = top - h * 0.5;
17312     const bottom = top - h;
17313     const cornerR = scaledPieceCornerRadius(refSize);
17314     if (connectivity === 0) {
17315         drawBlockingPieceFrame(
17316             g,
17317             left,
17318             bottom,
17319             width,
17320             h,
17321             refSize,
17322             blockingPieceInnerFill(colorKey),
17323         );
17324         return;
17325     }
17326     g.fillColor = pieceBaseFillColor(colorKey);
17327     g.roundRect(left, bottom, width, h, cornerR);
17328     g.fill();
17329     const holeR = refSize * HOLE_RATIO * 0.5;
17330     const armLen = refSize * 0.5 - holeR - ARM_EDGE_INSET;
17331     const halfW = refSize * ARM_WIDTH_RATIO * 0.5;
17332     const channelColors = pieceChannelColors(colorKey);
17333     g.fillColor = channelColors.fill;
17334     g.strokeColor = channelColors.stroke;
17335     g.lineWidth = 2;
17336     const dirs = openDirectionsForPiece(connectivity, pieceDirection);
17337     for (const d of dirs) {
17338         drawThickArm(g, cx, cy, d, holeR, armLen, halfW);
17339     }
17340     g.fillColor = channelColors.stroke;
17341     g.circle(cx, cy, holeR);
17342     g.fill();

```

```

17343     g.strokeColor = channelColors.stroke;
17344     g.lineWidth = 2;
17345     g.circle(cx, cy, holeR);
17346     g.stroke();
17347 }
17348 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzlePlayerGlyph.ts -----
17349 import { Color, Graphics } from 'cc';
17350 import { Direction } from '../Core/OpticalPuzzleTypes';
17351 import {
17352     playerBaseBorderColor,
17353     playerBaseFillColor,
17354     playerEyeFillColor,
17355 } from './OpticalPuzzleColorUtil';
17356 import { OPTICAL_CELL_SIZE } from './OpticalPuzzleLayout';
17357 /** 单眼宽 / 高（设计像素） */
17358 const EYE_WIDTH = 6;
17359 const EYE_HEIGHT = 18;
17360 /** 眨眼压扁后单眼宽 / 高（设计像素） */
17361 const BLINK_EYE_WIDTH = 12;
17362 const BLINK_EYE_HEIGHT = 4;
17363 /** 眨眼时双眼整体下移（设计像素） */
17364 const BLINK_DOWN_OFFSET = 2;
17365 /** 两眼中心间距（设计像素，眨眼时不变） */
17366 const EYE_CENTER_GAP = 20;
17367 /** 阻拦 >< 眼两眼中心间距（设计像素） */
17368 const BLOCKED_EYE_CENTER_GAP = 22;
17369 /** 与 TargetGlyph 外缘一致：bezel 环宽比例、内面板圆角系数 */
17370 const BEZEL_RATIO = 0.017;
17371 const INNER_CORNER_FACTOR = 0.7;
17372 const INNER_FRAME_STROKE_RATIO = 0.008;
17373 /** 格底圆角（设计像素，与 Target / 发射器一致） */
17374 const CELL_CORNER_RADIUS = 4;
17375 /** 眼镜片圆角（略小于格底） */
17376 const EYE_CORNER_RADIUS = 2;
17377 /** 双眼整体中心相对格心的偏移（设计像素，y 向上为正） */
17378 const PAIR_CENTER_BY_DIR: Readonly<Record<Direction, { x: number; y: number }>> = {
17379     [Direction.Left]: { x: -4, y: 3 },
17380     [Direction.Up]: { x: 0, y: 3 },
17381     [Direction.Right]: { x: 4, y: 3 },
17382     [Direction.Down]: { x: 0, y: 0 },
17383 };
17384 function scaleDesign(size: number, design: number): number {
17385     return design * (size / OPTICAL_CELL_SIZE);
17386 }
17387 function scaledCellCornerRadius(size: number): number {
17388     return CELL_CORNER_RADIUS * (size / OPTICAL_CELL_SIZE);
17389 }
17390 function lerp(a: number, b: number, t: number): number {
17391     return a + (b - a) * t;
17392 }
17393 /** 眨眼时眼色为原色的比例（峰值） */
17394 const BLINK_EYE_COLOR_SCALE = 0.8;
17395 function eyeColorForBlink(base: Color, blinkAmount: number): Color {
17396     const t = Math.max(0, Math.min(1, blinkAmount));
17397     const scale = lerp(1, BLINK_EYE_COLOR_SCALE, t);
17398     return new Color(
17399         Math.floor(base.r * scale),
17400         Math.floor(base.g * scale),

```

```

17401         Math.floor(base.b * scale),
17402         base.a,
17403     );
17404 }
17405 /** 阻拦 >< 眼：单条矩形长 / 厚 / 夹角 / 圆角（设计像素） */
17406 const BLOCKED_BAR_LENGTH = 12;
17407 const BLOCKED_BAR_THICKNESS = 4;
17408 const BLOCKED_BAR_ANGLE_DEG = 60;
17409 const BLOCKED_BAR_CORNER = 2;
17410 const BLOCKED_HALF_ANGLE_RAD = (BLOCKED_BAR_ANGLE_DEG * 0.5 * Math.PI) / 180;
17411 /** 长边沿 local +x；四角均为 2px 圆角（含交叠内端） */
17412 function fillRotatedRoundRect(
17413     g: Graphics,
17414     cx: number,
17415     cy: number,
17416     length: number,
17417     thickness: number,
17418     angleRad: number,
17419     corner: number,
17420 ): void {
17421     const hl = length * 0.5;
17422     const ht = thickness * 0.5;
17423     const r = Math.min(corner, hl, ht);
17424     const cos = Math.cos(angleRad);
17425     const sin = Math.sin(angleRad);
17426     const toWorld = (lx: number, ly: number): { x: number; y: number } => ({
17427         x: cx + lx * cos - ly * sin,
17428         y: cy + lx * sin + ly * cos,
17429     });
17430     const move = (lx: number, ly: number): void => {
17431         const w = toWorld(lx, ly);
17432         g.moveTo(w.x, w.y);
17433     };
17434     const line = (lx: number, ly: number): void => {
17435         const w = toWorld(lx, ly);
17436         g.lineTo(w.x, w.y);
17437     };
17438     const arcCorner = (
17439         cornerCx: number,
17440         cornerCy: number,
17441         startA: number,
17442         endA: number,
17443     ): void => {
17444         const segs = 4;
17445         for (let i = 0; i <= segs; i++) {
17446             const a = startA + ((endA - startA) * i) / segs;
17447             line(cornerCx + r * Math.cos(a), cornerCy + r * Math.sin(a));
17448         }
17449     };
17450     if (r <= 0.001) {
17451         move(-hl, -ht);
17452         line(-hl, ht);
17453         line(hl, ht);
17454         line(hl, -ht);
17455         g.close();
17456         g.fill();
17457         return;
17458     }

```

```

17459 // 四角 2px 圆角
17460 move(-hl + r, -ht);
17461 line(hl - r, -ht);
17462 arcCorner(hl - r, -ht + r, -Math.PI * 0.5, 0);
17463 line(hl, ht - r);
17464 arcCorner(hl - r, ht - r, 0, Math.PI * 0.5);
17465 line(-hl + r, ht);
17466 arcCorner(-hl + r, ht - r, Math.PI * 0.5, Math.PI);
17467 line(-hl, -ht + r);
17468 arcCorner(-hl + r, -ht + r, Math.PI, Math.PI * 1.5);
17469 g.close();
17470 g.fill();
17471 }
17472 /** 内端向内微量延伸（设计像素），外端长度不变 */
17473 const BLOCKED_BAR_INNER_EXTEND = 2;
17474 /** 单眼 ><: 两枚圆角矩形拼接（pointingRight: 左眼泪 >, 右眼泪 <） */
17475 function drawBlockedBarEye(
17476     g: Graphics,
17477     eyeCx: number,
17478     eyeCy: number,
17479     size: number,
17480     pointingRight: boolean,
17481 ): void {
17482     const barLen = scaleDesign(size, BLOCKED_BAR_LENGTH);
17483     const barThick = scaleDesign(size, BLOCKED_BAR_THICKNESS);
17484     const corner = scaleDesign(size, BLOCKED_BAR_CORNER);
17485     const cosH = Math.cos(BLOCKED_HALF_ANGLE_RAD);
17486     const sinH = Math.sin(BLOCKED_HALF_ANGLE_RAD);
17487     /** 内端微量延伸，外端长度不变，交叠侧同样 2px 圆角 */
17488     const innerExtend = scaleDesign(size, BLOCKED_BAR_INNER_EXTEND);
17489     const drawLen = barLen + innerExtend;
17490     const centerAlong = (barLen - innerExtend) * 0.5;
17491     const apexOffset = scaleDesign(
17492         size,
17493         (BLOCKED_BAR_LENGTH * 0.5) * cosH,
17494     );
17495     const apexX = eyeCx + (pointingRight ? apexOffset : -apexOffset);
17496     const apexY = eyeCy;
17497     const outwardSign = pointingRight ? -1 : 1;
17498     const topDirX = outwardSign * cosH;
17499     const topDirY = sinH;
17500     const botDirX = outwardSign * cosH;
17501     const botDirY = -sinH;
17502     fillRotatedRoundRect(
17503         g,
17504         apexX + topDirX * centerAlong,
17505         apexY + topDirY * centerAlong,
17506         drawLen,
17507         barThick,
17508         Math.atan2(topDirY, topDirX),
17509         corner,
17510     );
17511     fillRotatedRoundRect(
17512         g,
17513         apexX + botDirX * centerAlong,
17514         apexY + botDirY * centerAlong,
17515         drawLen,
17516         barThick,

```

```

17517         Math.atan2(botDirY, botDirX),
17518         corner,
17519     );
17520 }
17521 function drawBlockedChevronEyes(
17522     g: Graphics,
17523     pairCx: number,
17524     pairCy: number,
17525     size: number,
17526 ): void {
17527     const halfGap = scaleDesign(size, BLOCKED_EYE_CENTER_GAP) * 0.5;
17528     g.fillColor = eyeColorForBlink(playerEyeFillColor(), 1);
17529     drawBlockedBarEye(g, pairCx - halfGap, pairCy, size, true);
17530     drawBlockedBarEye(g, pairCx + halfGap, pairCy, size, false);
17531 }
17532 /**
17533  * 外缘 bezier 填充环 + 内嵌圆角面板（与 Target 未点亮外框同结构，颜色为主角黑边/橘底）。
17534  * 描边仅画内面板路径，不外溢格缘。
17535  */
17536 function drawPlayerCellFrame(
17537     g: Graphics,
17538     left: number,
17539     bottom: number,
17540     width: number,
17541     height: number,
17542     refSize: number,
17543 ): void {
17544     const corner = scaledCellCornerRadius(refSize);
17545     const bezel = refSize * BEZEL_RATIO;
17546     const innerLeft = left + bezel;
17547     const innerBottom = bottom + bezel;
17548     const innerWidth = width - bezel * 2;
17549     const innerHeight = height - bezel * 2;
17550     const innerCorner = corner * INNER_CORNER_FACTOR;
17551     const border = playerBaseBorderColor();
17552     g.fillColor = border;
17553     g.roundRect(left, bottom, width, height, corner);
17554     g.fill();
17555     g.fillColor = playerBaseFillColor();
17556     g.roundRect(innerLeft, innerBottom, innerWidth, innerHeight, innerCorner);
17557     g.fill();
17558     g.strokeColor = border;
17559     g.lineWidth = Math.max(1, refSize * INNER_FRAME_STROKE_RATIO);
17560     g.roundRect(innerLeft, innerBottom, innerWidth, innerHeight, innerCorner);
17561     g.stroke();
17562 }
17563 /**
17564  * 主角：Target 式 bezier 外缘 + 橘色内面板 + 两枚深橘眼镜片。
17565  * @param width 格宽（挤压动画时可非正方形）
17566  * @param height 格高，默认与 width 相同
17567  */
17568 export function drawPlayerEyes(
17569     g: Graphics,
17570     left: number,
17571     bottom: number,
17572     width: number,
17573     facing: Direction,
17574     blinkAmount = 0,

```

```

17575         blockedEyes = false,
17576         height?: number,
17577     ): void {
17578         const h = height ?? width;
17579         const refSize = Math.min(width, h);
17580         drawPlayerCellFrame(g, left, bottom, width, h, refSize);
17581         const cx = left + width * 0.5;
17582         const cy = bottom + h * 0.5;
17583         const offset = PAIR_CENTER_BY_DIR[facing] ?? PAIR_CENTER_BY_DIR[Direction.Left];
17584         const pairCx = cx + scaleDesign(refSize, offset.x);
17585         const pairCy = cy + scaleDesign(refSize, offset.y);
17586         if (blockedEyes) {
17587             drawBlockedChevronEyes(g, pairCx, pairCy, refSize);
17588             return;
17589         }
17590         const pairCyBlink =
17591             pairCy - scaleDesign(refSize, BLINK_DOWN_OFFSET * blinkAmount);
17592         const t = Math.max(0, Math.min(1, blinkAmount));
17593         const eyeW = scaleDesign(refSize, lerp(EYE_WIDTH, BLINK_EYE_WIDTH, t));
17594         const eyeH = scaleDesign(refSize, lerp(EYE_HEIGHT, BLINK_EYE_HEIGHT, t));
17595         const halfGap = scaleDesign(refSize, EYE_CENTER_GAP) * 0.5;
17596         const eyeCorner = scaleDesign(
17597             refSize,
17598             lerp(EYE_CORNER_RADIUS, Math.min(3, BLINK_EYE_HEIGHT * 0.5), t),
17599         );
17600         const eyeColor = eyeColorForBlink(playerEyeFillColor(), t);
17601         g.fillColor = eyeColor;
17602         for (const ex of [pairCx - halfGap, pairCx + halfGap]) {
17603             g.roundRect(ex - eyeW * 0.5, pairCyBlink - eyeH * 0.5, eyeW, eyeH, eyeCorner);
17604             g.fill();
17605         }
17606     }
17607     // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzlePlayerOutline.ts -----
17608     /** 主角外轮廓（用户 SVG viewBox 1024, iconR=10; 高度由 Presentation 层纵向三切片扩展） */
17609     export const PLAYER_FRAME_BOX_W = 20.000 as const;
17610     export const PLAYER_FRAME_NORM_CX = 0.000 as const;
17611     /** 外轮廓 bbox 纵向范围（归一化坐标） */
17612     export const PLAYER_FRAME_MIN_Y = -6.182 as const;
17613     export const PLAYER_FRAME_MAX_Y = 6.364 as const;
17614     /** 上盖下缘：内轮廓背带凹槽最低点，其上是耳 / 背带（不纵向拉伸） */
17615     export const PLAYER_FRAME_TOP_SLICE_Y = 1.577 as const;
17616     /** 下盖下缘：底圆角起点，其下是底边（不纵向拉伸） */
17617     export const PLAYER_FRAME_BOTTOM_SLICE_Y = -3.182 as const;
17618     export type PlayerSvgSeg =
17619         | { t: 'M'; x: number; y: number }
17620         | { t: 'L'; x: number; y: number }
17621         | { t: 'C'; x1: number; y1: number; x2: number; y2: number; x: number; y: number }
17622         | { t: 'Z' };
17623     export const PLAYER_FRAME_OUTER_SEGS: ReadonlyArray<PlayerSvgSeg> = [
17624         { t: 'M', x: 9.091, y: 5.545 },
17625         { t: 'C', x1: 8.091, y1: 6.364, x2: 6.545, y2: 6.273, x: 5.636, y: 5.273 },
17626         { t: 'L', x: 4.182, y: 3.545 },
17627         { t: 'C', x1: 3.909, y1: 3.182, x2: 3.455, y2: 3, x: 3, y: 3 },
17628         { t: 'L', x: -3, y: 3 },
17629         { t: 'C', x1: -3.455, y1: 3, x2: -3.909, y2: 3.182, x: -4.182, y: 3.545 },
17630         { t: 'L', x: -5.727, y: 5.273 },
17631         { t: 'C', x1: -6.545, y1: 6.273, x2: -8.091, y2: 6.364, x: -9.091, y: 5.545 },
17632         { t: 'C', x1: -9.727, y1: 5.091, x2: -10, y2: 4.364, x: -10, y: 3.636 },

```



```

17633     { t: 'L', x: -10, y: -3.182 },
17634     { t: 'C', x1: -10, y1: -4.818, x2: -8.636, y2: -6.182, x: -7.091, y: -6.182 },
17635     { t: 'L', x: 7, y: -6.182 },
17636     { t: 'C', x1: 8.636, y1: -6.182, x2: 10, y2: -4.818, x: 10, y: -3.182 },
17637     { t: 'L', x: 10, y: 3.636 },
17638     { t: 'C', x1: 10, y1: 4.364, x2: 9.727, y2: 5.091, x: 9.091, y: 5.545 },
17639     { t: 'Z' },
17640 ];
17641 export const PLAYER_FRAME_INNER_SEGS: ReadonlyArray<PlayerSvgSeg> = [
17642     { t: 'M', x: 8.727, y: -3.182 },
17643     { t: 'C', x1: 8.727, y1: -4.091, x2: 8, y2: -4.818, x: 7.091, y: -4.818 },
17644     { t: 'L', x: -7.091, y: -4.818 },
17645     { t: 'C', x1: -8, y1: -4.818, x2: -8.727, y2: -4.091, x: -8.727, y: -3.182 },
17646     { t: 'L', x: -8.727, y: 3.636 },
17647     { t: 'C', x1: -8.727, y1: 4, x2: -8.545, y2: 4.273, x: -8.273, y: 4.545 },
17648     { t: 'C', x1: -8.091, y1: 4.727, x2: -7.818, y2: 4.818, x: -7.545, y: 4.818 },
17649     { t: 'C', x1: -7.182, y1: 4.818, x2: -6.909, y2: 4.636, x: -6.636, y: 4.455 },
17650     { t: 'L', x: -5.273, y: 2.727 },
17651     { t: 'C', x1: -4.727, y1: 2.091, x2: -3.909, y2: 1.727, x: -3.091, y: 1.727 },
17652     { t: 'L', x: 3, y: 1.727 },
17653     { t: 'C', x1: 3.818, y1: 1.727, x2: 4.636, y2: 2.091, x: 5.182, y: 2.727 },
17654     { t: 'L', x: 6.636, y: 4.455 },
17655     { t: 'C', x1: 7.091, y1: 4.909, x2: 7.818, y2: 5, x: 8.273, y: 4.636 },
17656     { t: 'C', x1: 8.545, y1: 4.455, x2: 8.727, y2: 4.091, x: 8.727, y: 3.727 },
17657     { t: 'L', x: 8.727, y: -3.182 },
17658     { t: 'Z' },
17659 ];
17660 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleRoot.ts -----
17661 import { _decorator, Component, director, find, Node, SpriteFrame, UITransform, Vec3 } from 'cc';
17662 import { DataManager } from '../Manager/DataManager';
17663 import { ToastManager } from '../Manager/ToastManager';
17664 import { AUDIO_EFFECT_ENUM, EVENT_ENUM } from '../Utils/Enum';
17665 import { OPTICAL_PUZZLE, PLAY_AUDIO, SHOW_TOAST } from '../Utils/Event';
17666 import { OpticalGameFlowState } from '../Application/OpticalPuzzleStateMachine';
17667 import {
17668     OpticalPuzzleSession,
17669     type OpticalSessionNotifyReason,
17670     type OpticalSnapshotNotify,
17671 } from '../Application/OpticalPuzzleSession';
17672 import { getOpticalLevelById } from '../Config/OpticalPuzzleLevels';
17673 import { DEV_LEVEL_MINIMAL } from '../Config/OpticalPuzzleLevelSchema';
17674 import type { OpticalBeamSnapshot } from '../Core/OpticalPuzzleCore';
17675 import { OpticalPuzzleBeamView } from './OpticalPuzzleBeamView';
17676 import { OpticalPuzzleBoardView } from './OpticalPuzzleBoardView';
17677 import { OpticalPuzzleInputHud } from './OpticalPuzzleInputHud';
17678 import { computePlayLayerPosition, computePlayLayerScale } from './OpticalPuzzleLayout';
17679 import {
17680     OpticalPuzzleWinPanelNextLevelButtonView,
17681     resolveWinPanelNextLevelNode,
17682 } from './OpticalPuzzleWinPanelNextLevelButtonView';
17683 import {
17684     resolvePreWinPanelNode,
17685     resolveWinPanelNode,
17686     resolveWinPanelWindsNode,
17687 } from './OpticalPuzzleWinPanelWindsView';
17688 const { ccclass, property } = _decorator;
17689 /** 通关后先挡输入，再展示 winPanel 的等待时长（秒） */
17690 const PRE_WIN_PANEL_DELAY_SEC = 0.75;

```

```

17691  /** 阻挡点粒子视图（按 ccclass 名运行时解析，避免与 Root 强耦合） */
17692  interface IBeamImpactView {
17693      render(snapshot: OpticalBeamSnapshot): void;
17694      applyExternalSparkSpriteFrames?(frames: ReadonlyArray<SpriteFrame>): void;
17695  }
17696  const BEAM_IMPACT_VIEW_CLASS = 'OpticalPuzzleBeamImpactView';
17697  /** SHOW_TOAST 事件载荷（与 ToastManager.show 入参一致） */
17698  interface ToastShowPayload {
17699      message: string;
17700      bgWidth?: number;
17701      localY?: number;
17702  }
17703  /**
17704   * 光学解谜局内入口：创建 Session、绑定视图与输入、广播快照事件。
17705   * 挂到带 Canvas 的节点上，并为同节点或子节点挂上 BoardView / InputHud。
17706   */
17707  @ccclass('OpticalPuzzleRoot')
17708  export class OpticalPuzzleRoot extends Component {
17709      @property(OpticalPuzzleBoardView)
17710      boardView: OpticalPuzzleBoardView | null = null;
17711      @property(OpticalPuzzleBeamView)
17712      beamView: OpticalPuzzleBeamView | null = null;
17713      @property(OpticalPuzzleInputHud)
17714      inputHud: OpticalPuzzleInputHud | null = null;
17715      /** 玩法区容器（默认向上查找名为 layerPlay 的父节点） */
17716      @property(Node)
17717      layerPlay: Node | null = null;
17718      /** 棋盘左右留白（设计像素，从 Canvas 可用宽中扣除） */
17719      @property
17720      playAreaWidthPadding = 0;
17721      /** 缩放后棋盘顶边距 Canvas 顶边（设计像素） */
17722      @property
17723      playAreaTopMargin = 125;
17724      /** 构建保活：七色 beam_spark SpriteFrame，微信端依赖贴图烘焙色（非顶点色） */
17725      @property({ type: [SpriteFrame], tooltip: 'Sprites/OpticalFX 下 beam_spark 七色' })
17726      beamSparkKeepAlive: SpriteFrame[] = [];
17727      private readonly _session = new OpticalPuzzleSession();
17728      private _beamImpactView: IBeamImpactView | null = null;
17729      protected onLoad(): void {
17730          DataManager.instance.init();
17731          if (!this.boardView) {
17732              this.boardView =
17733                  this.getComponent(OpticalPuzzleBoardView) ??
17734                  this.getComponentInChildren(OpticalPuzzleBoardView);
17735          }
17736          if (!this.beamView) {
17737              this.beamView =
17738                  this.getComponent(OpticalPuzzleBeamView) ??
17739                  this.getComponentInChildren(OpticalPuzzleBeamView);
17740          }
17741          if (!this.beamView) {
17742              const beamNode = this.node.getChildByName('BeamLayer');
17743              if (beamNode?.isValid) {
17744                  this.beamView =
17745                      beamNode.getComponent(OpticalPuzzleBeamView) ??
17746                      beamNode.addComponent(OpticalPuzzleBeamView);
17747              }
17748          }
17749      }

```

```
17749     this._resolveBeamImpactView();
17750     if (!this.inputHud) {
17751         this.inputHud =
17752             this.getComponent(OpticalPuzzleInputHud) ??
17753             this.getComponentInChildren(OpticalPuzzleInputHud);
17754     }
17755     this._session.onStateChanged = (reason) => this._onSessionChanged(reason);
17756     SHOW_TOAST.on(EVENT_ENUM.SHOW_TOAST, this._onShowToast, this);
17757     if (this.inputHud) {
17758         this.inputHud.setup(this._session, this.boardView ?? undefined);
17759     } else {
17760         console.warn('[OpticalPuzzleRoot] 未绑定 OpticalPuzzleInputHud');
17761     }
17762     this.reloadCurrentLevel();
17763 }
17764 /** 本局当前关卡 id（来自 Session 快照，非存档） */
17765 getCurrentLevelId(): number {
17766     return this._session.getSnapshot().levelId;
17767 }
17768 /** 按 DataManager.opticalCurrentLevelId 重载关卡（菜单进局 / 读档） */
17769 reloadCurrentLevel(): void {
17770     this.loadLevelById(DataManager.instance.opticalCurrentLevelId);
17771 }
17772 /** 按关卡 id 加载本局（局内下一关等，不写存档） */
17773 loadLevelById(levelId: number): void {
17774     const resolved = getOpticalLevelById(levelId);
17775     const level = resolved ?? DEV_LEVEL_MINIMAL;
17776     if (!resolved) {
17777         console.warn('[OpticalPuzzleRoot] 未知关卡 id=${levelId}，使用 DEV_LEVEL_MINIMAL');
17778     }
17779     this._applyPlayLayerLayout(level.width, level.height);
17780     this._session.loadLevel(level);
17781     this._cancelWinRevealSchedule();
17782     this._setPreWinPanelVisible(false);
17783     this._setWinPanelVisible(false);
17784 }
17785 protected onDestroy(): void {
17786     SHOW_TOAST.off(EVENT_ENUM.SHOW_TOAST, this._onShowToast, this);
17787     this._cancelWinRevealSchedule();
17788     this._session.onStateChanged = null;
17789     this.inputHud?.teardown();
17790 }
17791 private _resolveGameRoot(): Node | null {
17792     let node: Node | null = this.node;
17793     while (node?.parent) {
17794         if (node.name === 'GameRoot') {
17795             return node;
17796         }
17797         node = node.parent;
17798     }
17799     return null;
17800 }
17801 private _setWinPanelVisible(visible: boolean): void {
17802     const panel = resolveWinPanelNode(this._resolveGameRoot());
17803     if (panel?.isValid) {
17804         panel.active = visible;
17805     }
17806 }
```

```

17807     private _setPreWinPanelVisible(visible: boolean): void {
17808         const panel = resolvePreWinPanelNode(this._resolveGameRoot());
17809         if (!panel?.isValid) {
17810             if (visible) {
17811                 console.warn('[OpticalPuzzleRoot] layerOverlay/preWinPanel 未找到，无法挡通关前输入');
17812             }
17813             return;
17814         }
17815         if (visible) {
17816             const overlay = panel.parent;
17817             if (overlay?.isValid) {
17818                 panel.setSiblingIndex(overlay.children.length - 1);
17819             }
17820         }
17821         panel.active = visible;
17822     }
17823     private _cancelWinRevealSchedule(): void {
17824         this.unschedule(this._revealWinPanelAfterPreWin);
17825     }
17826     /** 通关：先 preWinPanel 挡输入，延迟后再展示 winPanel 并播通关音 */
17827     private _beginWinRevealSequence(): void {
17828         this._setWinPanelVisible(false);
17829         this._setPreWinPanelVisible(true);
17830         this._cancelWinRevealSchedule();
17831         this.scheduleOnce(this._revealWinPanelAfterPreWin, PRE_WIN_PANEL_DELAY_SEC);
17832     }
17833     private _revealWinPanelAfterPreWin = (): void => {
17834         this._setPreWinPanelVisible(false);
17835         this._showWinPanel();
17836     };
17837     private _showWinPanel(): void {
17838         const gameRoot = this._resolveGameRoot();
17839         this._setWinPanelVisible(true);
17840         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.OPTICAL_LEVEL_COMPLETE);
17841         // winPanel 默认 inactive，子组件 onLoad 晚于 complete 事件；展示后补发一次同步步数/星级
17842         this._replayWinPanelSnapshotNotify();
17843         const nextLevelBtn = resolveWinPanelNextLevelNode(gameRoot)?.getComponent(
17844             OpticalPuzzleWinPanelNextLevelButtonView,
17845         );
17846         nextLevelBtn?.setLabelForLevel(this.getCurrentLevelId());
17847         nextLevelBtn?.refresh();
17848     }
17849     /** winPanel 首次激活时其监听尚未注册，需用最近一次通关快照刷新 winds 内 UI */
17850     private _replayWinPanelSnapshotNotify(): void {
17851         if (this._session.flowState !== OpticalGameFlowState.SETTLEMENT) {
17852             return;
17853         }
17854         const notify: OpticalSnapshotNotify = {
17855             snapshot: this._session.getSnapshot(),
17856             moveCount: this._session.moveCount,
17857             notifyReason: 'complete',
17858         };
17859         OPTICAL_PUZZLE.emit(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, notify);
17860     }
17861     /** 转发到场景内 ToastManager，不修改 ToastManager 本身 */
17862     private _onShowToast(payload: ToastShowPayload): void {
17863         if (!payload?.message) {
17864             return;

```

```

17865     }
17866     const scene = director.getScene();
17867     const mgr = scene
17868         ?.getComponentsInChildren(ToastManager)
17869         .find((m) => m?.isValid && m.enabled);
17870     mgr?.show(payload.message, payload.bgWidth, payload.localY ?? 0);
17871 }
17872 private _onSessionChanged(reason: OpticalSessionNotifyReason): void {
17873     const snap = this._session.getSnapshot();
17874     const beam = this._session.getBeamSnapshot();
17875     if (reason === 'complete') {
17876         DataManager.instance.recordOpticalLevelClear(
17877             this.getCurrentLevelId(),
17878             this._session.moveCount,
17879         );
17880         this._beginWinRevealSequence();
17881     }
17882     if (reason === 'face') {
17883         this.boardView?.notifyPlayerMoveBlocked();
17884     } else if (reason === 'move' || reason === 'push' || reason === 'complete') {
17885         this.boardView?.notifyPlayerDirectionInput();
17886     }
17887     if (reason === 'load' || reason === 'reset') {
17888         this.boardView?.resetPlayerEyedle();
17889     }
17890     this.boardView?.render(snap);
17891     this.beamView?.render(beam);
17892     this._beamImpactView?.render(beam);
17893     this.boardView?.renderTargetsOverlay(snap);
17894     this.boardView?.syncPlaySnapshot(snap, reason);
17895     this.inputHud?.refreshActionButtons();
17896     const notify: OpticalSnapshotNotify = {
17897         snapshot: snap,
17898         moveCount: this._session.moveCount,
17899         notifyReason: reason,
17900     };
17901     OPTICAL_PUZZLE.emit(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, notify);
17902 }
17903 private _resolveBeamImpactView(): void {
17904     if (this._beamImpactView) {
17905         this._syncBeamSparkKeepAlive(this._beamImpactView);
17906         return;
17907     }
17908     const fromSelf =
17909         this.getComponent(BEAM_IMPACT_VIEW_CLASS) ??
17910         this.getComponentInChildren(BEAM_IMPACT_VIEW_CLASS);
17911     if (fromSelf) {
17912         this._beamImpactView = fromSelf as unknown as IBeamImpactView;
17913         this._syncBeamSparkKeepAlive(this._beamImpactView);
17914         return;
17915     }
17916     const beamNode = this.node.getChildByName('BeamLayer');
17917     if (!beamNode?.isValid) {
17918         return;
17919     }
17920     const onBeam =
17921         beamNode.getComponent(BEAM_IMPACT_VIEW_CLASS) ??
17922         beamNode.addComponent(BEAM_IMPACT_VIEW_CLASS);

```

```

17923         this._beamImpactView = onBeam as unknown as IBeamImpactView;
17924         this._syncBeamSparkKeepAlive(this._beamImpactView);
17925     }
17926     private _syncBeamSparkKeepAlive(view: IBeamImpactView | null): void {
17927         if (!view?.applyExternalSparkSpriteFrames) {
17928             return;
17929         }
17930         if (this.beamSparkKeepAlive.length === 0) {
17931             console.warn('[OpticalPuzzleRoot] beamSparkKeepAlive 未绑定，微信包可能缺少 beam_spark
17932 贴图');
17933             return;
17934         }
17935         view.applyExternalSparkSpriteFrames(this.beamSparkKeepAlive);
17936     }
17937     /** 按关卡尺寸缩放并定位 layerPlay：宽撑满屏宽；高超过屏宽时同步压缩 */
17938     private _applyPlayLayerLayout(levelWidth: number, levelHeight: number): void {
17939         const playLayer = this._resolveLayerPlay();
17940         if (!playLayer?.isValid) {
17941             return;
17942         }
17943         const targetWidth = this._resolvePlayAreaWidth();
17944         const scale = computePlayLayerScale(levelWidth, levelHeight, targetWidth);
17945         playLayer.setScale(new Vec3(scale, scale, 1));
17946         const canvasHeight = this._resolveCanvasHeight();
17947         const pos = computePlayLayerPosition(
17948             levelHeight,
17949             scale,
17950             canvasHeight,
17951             this.playAreaTopMargin,
17952         );
17953         const current = playLayer.position;
17954         playLayer.setPosition(new Vec3(pos.x, pos.y, current.z));
17955     }
17956     private _resolveLayerPlay(): Node | null {
17957         if (this.layerPlay?.isValid) {
17958             return this.layerPlay;
17959         }
17960         let node: Node | null = this.node;
17961         while (node) {
17962             if (node.name === 'layerPlay') {
17963                 return node;
17964             }
17965             node = node.parent;
17966         }
17967         return this.node.parent;
17968     }
17969     /** Canvas 设计宽（Widget 拉伸后即当前屏宽坐标） */
17970     private _resolvePlayAreaWidth(): number {
17971         const canvasWidth = this._resolveCanvasSize().width;
17972         const padding = Math.max(0, this.playAreaWidthPadding);
17973         if (canvasWidth > padding) {
17974             return canvasWidth - padding;
17975         }
17976         return canvasWidth > 0 ? canvasWidth : 750;
17977     }
17978     /** Canvas 设计高（与宽度同为 UI 坐标系） */
17979     private _resolveCanvasHeight(): number {
17980         const h = this._resolveCanvasSize().height;

```

```

17981         return h > 0 ? h : 1334;
17982     }
17983     private _resolveCanvasSize(): { width: number; height: number } {
17984         const canvas = find('Canvas');
17985         const size = canvas?.getComponent(UITransform)?.contentSize;
17986         return {
17987             width: size?.width ?? 0,
17988             height: size?.height ?? 0,
17989         };
17990     }
17991 }
17992 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleSourceGlyph.ts -----
17993 import { Color, Graphics } from 'cc';
17994 import { Direction } from '../Core/OpticalPuzzleTypes';
17995 import { fillBeamCrossSection } from './OpticalPuzzleBeamGradient';
17996 import { OPTICAL_CELL_SIZE } from './OpticalPuzzleLayout';
17997 import { sourceEmitterColors, sourceFillColor } from './OpticalPuzzleColorUtil';
17998 /** 发射器外轮廓圆角（设计像素，随格宽等比缩放） */
17999 const EMITTER_CORNER_RADIUS = 4;
18000 /** 六边形框路径半宽（viewBox 1024, iconR=10） */
18001 const SOURCE_HEX_FRAME_HALF_W = 7.646;
18002 type SvgSeg =
18003     | { t: 'M'; x: number; y: number }
18004     | { t: 'L'; x: number; y: number }
18005     | { t: 'C'; x1: number; y1: number; x2: number; y2: number; x: number; y: number }
18006     | { t: 'Z' };
18007 /** 发射器四角框：用户 SVG 外六边形 + 内孔（iconR=10） */
18008 const SOURCE_HEX_FRAME_FILL_SEGS: ReadonlyArray<SvgSeg> = [
18009     { t: 'M', x: 0, y: -8.738 },
18010     { t: 'C', x1: -0.102, y1: -8.738, x2: -0.203, y2: -8.713, x: -0.293, y: -8.66 },
18011     { t: 'L', x: -7.354, y: -4.584 },
18012     { t: 'C', x1: -7.535, y1: -4.479, x2: -7.646, y2: -4.285, x: -7.646, y: -4.076 },
18013     { t: 'L', x: -7.646, y: 4.076 },
18014     { t: 'C', x1: -7.646, y1: 4.285, x2: -7.535, y2: 4.479, x: -7.354, y: 4.584 },
18015     { t: 'L', x: -0.293, y: 8.66 },
18016     { t: 'C', x1: -0.111, y1: 8.766, x2: 0.111, y2: 8.766, x: 0.293, y: 8.66 },
18017     { t: 'L', x: 7.354, y: 4.584 },
18018     { t: 'C', x1: 7.535, y1: 4.479, x2: 7.646, y2: 4.285, x: 7.646, y: 4.076 },
18019     { t: 'L', x: 7.646, y: -4.076 },
18020     { t: 'C', x1: 7.646, y1: -4.285, x2: 7.535, y2: -4.479, x: 7.354, y: -4.584 },
18021     { t: 'L', x: 0.293, y: -8.66 },
18022     { t: 'C', x1: 0.203, y1: -8.713, x2: 0.102, y2: -8.738, x: 0, y: -8.738 },
18023     { t: 'Z' },
18024     { t: 'M', x: -6.475, y: -3.738 },
18025     { t: 'L', x: 0, y: -7.477 },
18026     { t: 'L', x: 6.475, y: -3.738 },
18027     { t: 'L', x: 6.475, y: 3.738 },
18028     { t: 'L', x: 0, y: 7.477 },
18029     { t: 'L', x: -6.475, y: 3.738 },
18030     { t: 'L', x: -6.475, y: -3.738 },
18031     { t: 'Z' },
18032 ];
18033 /** 仅外轮廓描边，内孔只参与填充 */
18034 const SOURCE_HEX_FRAME_STROKE_SEGS: ReadonlyArray<SvgSeg> = SOURCE_HEX_FRAME_FILL_SEGS.slice(0,
18035 16);
18036 /** 内六边形（挖孔） */
18037 const SOURCE_HEX_FRAME_INNER_SEGS: ReadonlyArray<SvgSeg> = SOURCE_HEX_FRAME_FILL_SEGS.slice(16);
18038 const SCREEN_DIR_DX: ReadonlyArray<number> = [1, 0, -1, 0];

```

```
18039 const SCREEN_DIR_DY: ReadonlyArray<number> = [0, 1, 0, -1];
18040 type LocalPoint = { lx: number; ly: number };
18041 function emitLocalToWorld(
18042     cx: number,
18043     cy: number,
18044     lx: number,
18045     ly: number,
18046     dx: number,
18047     dy: number,
18048     px: number,
18049     py: number,
18050 ): { x: number; y: number } {
18051     return {
18052         x: cx + lx * px + ly * dx,
18053         y: cy + lx * py + ly * dy,
18054     };
18055 }
18056 function fillLocalPoly(
18057     g: Graphics,
18058     cx: number,
18059     cy: number,
18060     dx: number,
18061     dy: number,
18062     px: number,
18063     py: number,
18064     points: readonly LocalPoint[],
18065     color: Color,
18066 ): void {
18067     if (points.length < 3) {
18068         return;
18069     }
18070     const first = emitLocalToWorld(cx, cy, points[0].lx, points[0].ly, dx, dy, px, py);
18071     g.fillColor = color;
18072     g.moveTo(first.x, first.y);
18073     for (let i = 1; i < points.length; i++) {
18074         const p = emitLocalToWorld(cx, cy, points[i].lx, points[i].ly, dx, dy, px, py);
18075         g.lineTo(p.x, p.y);
18076     }
18077     g.close();
18078     g.fill();
18079 }
18080 function strokeLocalPoly(
18081     g: Graphics,
18082     cx: number,
18083     cy: number,
18084     dx: number,
18085     dy: number,
18086     px: number,
18087     py: number,
18088     points: readonly LocalPoint[],
18089     color: Color,
18090     lineWidth: number,
18091 ): void {
18092     if (points.length < 2) {
18093         return;
18094     }
18095     const first = emitLocalToWorld(cx, cy, points[0].lx, points[0].ly, dx, dy, px, py);
18096     g.strokeColor = color;
```



```

18097     g.lineWidth = lineWidth;
18098     g.moveTo(first.x, first.y);
18099     for (let i = 1; i < points.length; i++) {
18100         const p = emitLocalToWorld(cx, cy, points[i].lx, points[i].ly, dx, dy, px, py);
18101         g.lineTo(p.x, p.y);
18102     }
18103     g.close();
18104     g.stroke();
18105 }
18106 function traceScreenSegs(
18107     g: Graphics,
18108     segs: ReadonlyArray<SvgSeg>,
18109     cx: number,
18110     cy: number,
18111     scale: number,
18112 ): void {
18113     for (const seg of segs) {
18114         switch (seg.t) {
18115             case 'M':
18116                 g.moveTo(cx + seg.x * scale, cy + seg.y * scale);
18117                 break;
18118             case 'L':
18119                 g.lineTo(cx + seg.x * scale, cy + seg.y * scale);
18120                 break;
18121             case 'C':
18122                 g.bezierCurveTo(
18123                     cx + seg.x1 * scale,
18124                     cy + seg.y1 * scale,
18125                     cx + seg.x2 * scale,
18126                     cy + seg.y2 * scale,
18127                     cx + seg.x * scale,
18128                     cy + seg.y * scale,
18129                 );
18130                 break;
18131             case 'Z':
18132                 g.close();
18133                 break;
18134             default:
18135                 break;
18136         }
18137     }
18138 }
18139 /** 格心对称六边形框：外环填充 + 内孔 */
18140 function fillSourceHexFrame(
18141     g: Graphics,
18142     cx: number,
18143     cy: number,
18144     size: number,
18145     halfW: number,
18146     fillColor: Color,
18147     holeColor: Color,
18148 ): void {
18149     const frameHalf = halfW + size * 0.12;
18150     const scale = frameHalf / SOURCE_HEX_FRAME_HALF_W;
18151     g.fillColor = fillColor;
18152     traceScreenSegs(g, SOURCE_HEX_FRAME_STROKE_SEGS, cx, cy, scale);
18153     g.fill();
18154     g.fillColor = holeColor;

```

```

18155     traceScreenSegs(g, SOURCE_HEX_FRAME_INNER_SEGS, cx, cy, scale);
18156     g.fill();
18157 }
18158 /** 六边形外轮廓描边 */
18159 function strokeSourceHexFrame(
18160     g: Graphics,
18161     cx: number,
18162     cy: number,
18163     size: number,
18164     halfW: number,
18165     strokeColor: Color,
18166     lineWidth: number,
18167 ): void {
18168     const frameHalf = halfW + size * 0.12;
18169     const scale = frameHalf / SOURCE_HEX_FRAME_HALF_W;
18170     g.strokeColor = strokeColor;
18171     g.lineWidth = lineWidth;
18172     g.lineJoin = Graphics.LineJoin.ROUND;
18173     g.lineCap = Graphics.LineCap.ROUND;
18174     traceScreenSegs(g, SOURCE_HEX_FRAME_STROKE_SEGS, cx, cy, scale);
18175     g.stroke();
18176 }
18177 /** 六边形框四角指示点（本色调实心圆，同 Target 四角小圆） */
18178 function drawSourceCornerDots(
18179     g: Graphics,
18180     cx: number,
18181     cy: number,
18182     size: number,
18183     halfW: number,
18184     fillColor: Color,
18185 ): void {
18186     const corner = halfW + size * 0.12;
18187     const dotR = size * 0.035;
18188     const corners = [
18189         { x: -corner, y: -corner },
18190         { x: corner, y: -corner },
18191         { x: corner, y: corner },
18192         { x: -corner, y: corner },
18193     ];
18194     g.fillColor = fillColor;
18195     for (const c of corners) {
18196         g.circle(cx + c.x, cy + c.y, dotR);
18197         g.fill();
18198     }
18199 }
18200 function scaledCornerRadius(size: number): number {
18201     return EMITTER_CORNER_RADIUS * (size / OPTICAL_CELL_SIZE);
18202 }
18203 /** 局部坐标系圆角矩形（lx 横向，ly 沿发射方向） */
18204 function buildLocalRoundRectOutline(
18205     halfW: number,
18206     minLy: number,
18207     maxLy: number,
18208     radius: number,
18209     segsPerCorner = 5,
18210 ): LocalPoint[] {
18211     const r = Math.min(radius, halfW - 0.5, (maxLy - minLy) * 0.5 - 0.5);
18212     if (r <= 0.5) {

```

```

18213         return [
18214             { lx: -halfW, ly: minLy },
18215             { lx: halfW, ly: minLy },
18216             { lx: halfW, ly: maxLy },
18217             { lx: -halfW, ly: maxLy },
18218         ];
18219     }
18220     const pts: LocalPoint[] = [];
18221     const pushArc = (cx: number, cy: number, a0: number, a1: number): void => {
18222         for (let i = 1; i <= segsPerCorner; i++) {
18223             const a = a0 + ((a1 - a0) * i) / segsPerCorner;
18224             pts.push({ lx: cx + Math.cos(a) * r, ly: cy + Math.sin(a) * r });
18225         }
18226     };
18227     pts.push({ lx: -halfW + r, ly: minLy });
18228     pts.push({ lx: halfW - r, ly: minLy });
18229     pushArc(halfW - r, minLy + r, -Math.PI / 2, 0);
18230     pts.push({ lx: halfW, ly: maxLy - r });
18231     pushArc(halfW - r, maxLy - r, 0, Math.PI / 2);
18232     pts.push({ lx: -halfW + r, ly: maxLy });
18233     pushArc(-halfW + r, maxLy - r, Math.PI / 2, Math.PI);
18234     pts.push({ lx: -halfW, ly: minLy + r });
18235     pushArc(-halfW + r, minLy + r, Math.PI, (Math.PI * 3) / 2);
18236     return pts;
18237 }
18238 /**
18239  * 激光发射器：圆角金属机身 + 定向炮筒；炮口对齐格边。
18240  */
18241 export function drawSourceEmitter(
18242     g: Graphics,
18243     left: number,
18244     bottom: number,
18245     size: number,
18246     colorKey: string | undefined,
18247     direction: Direction,
18248 ): void {
18249     const cx = left + size * 0.5;
18250     const cy = bottom + size * 0.5;
18251     const dx = SCREEN_DIR_DX[direction];
18252     const dy = SCREEN_DIR_DY[direction];
18253     const px = -dy;
18254     const py = dx;
18255     const colors = sourceEmitterColors(colorKey);
18256     const half = size * 0.5;
18257     /** 炮口略伸出格边，与光路衔接更自然 */
18258     const muzzleLy = half + size * 0.045;
18259     const cornerR = scaledCornerRadius(size);
18260     g.fillColor = new Color(8, 12, 20, 255);
18261     g.roundRect(left, bottom, size, size, cornerR);
18262     g.fill();
18263     /** 机身半宽 (lx 横向，垂直于炮口方向)；整宽 = halfW × 2 */
18264     const halfW = size * 0.24;
18265     const hexFill = new Color(
18266         Math.floor(colors.accent.r * 0.55),
18267         Math.floor(colors.accent.g * 0.55),
18268         Math.floor(colors.accent.b * 0.55),
18269         200,
18270     );

```

```

18271     const hexLineW = Math.max(2.2, size * 0.05);
18272     // 格心对称六边形框（用户 SVG，先铺底再画机身）
18273     fillSourceHexFrame(g, cx, cy, size, halfW, hexFill, new Color(8, 12, 20, 255));
18274     // 光点锚点（不随机身/梯形调整而移动）
18275     const bodyCx = cx - dx * size * 0.05;
18276     const bodyCy = cy - dy * size * 0.05;
18277     /** 机身后缘 / 前缘（ly 沿发射方向，backY 为负=炮口反侧） */
18278     const backY = -size * 0.26;
18279     const frontY = size * 0.04;
18280     const lw = Math.max(1.5, size * 0.024);
18281     // 仅机身前移；梯形窄端固定、宽端跟随机身 → 梯形长度缩短
18282     const chassisForward = size * 0.075;
18283     const emitCx = bodyCx + dx * chassisForward;
18284     const emitCy = bodyCy + dy * chassisForward;
18285     const barrelFrontY = muzzleLy - size * 0.018;
18286     const barrelBackY = chassisForward + frontY;
18287     const chassis = buildLocalRoundRectOutline(halfW, backY, frontY, cornerR);
18288     fillLocalPoly(g, emitCx, emitCy, dx, dy, px, py, chassis, colors.chassis);
18289     strokeLocalPoly(g, emitCx, emitCy, dx, dy, px, py, chassis, colors.chassisEdge, lw);
18290     const innerHalfW = halfW - size * 0.045;
18291     const innerBackY = backY + size * 0.025;
18292     const innerFrontY = frontY - size * 0.01;
18293     const innerR = Math.max(1, cornerR - size * 0.012);
18294     const innerChassis = buildLocalRoundRectOutline(innerHalfW, innerBackY, innerFrontY, innerR);
18295     fillLocalPoly(
18296         g,
18297         emitCx,
18298         emitCy,
18299         dx,
18300         dy,
18301         px,
18302         py,
18303         innerChassis,
18304         new Color(
18305             Math.floor(colors.chassis.r * 0.85 + 8),
18306             Math.floor(colors.chassis.g * 0.85 + 10),
18307             Math.floor(colors.chassis.b * 0.85 + 14),
18308             255,
18309         ),
18310     );
18311     strokeSourceHexFrame(g, cx, cy, size, halfW, colors.accent, hexLineW);
18312     drawSourceCornerDots(g, cx, cy, size, halfW, sourceFillColor(colorKey));
18313     const railLen = size * 0.14;
18314     for (const side of [-1, 1]) {
18315         const lx = side * (halfW - size * 0.07);
18316         const r0 = emitLocalToWorld(emitCx, emitCy, lx, backY + size * 0.07, dx, dy, px, py);
18317         const r1 = emitLocalToWorld(emitCx, emitCy, lx, backY + size * 0.07 + railLen, dx, dy, px, py);
18318         g.strokeColor = colors.chassisEdge;
18319         g.lineWidth = Math.max(1, size * 0.016);
18320         g.moveTo(r0.x, r0.y);
18321         g.lineTo(r1.x, r1.y);
18322         g.stroke();
18323     }
18324     // 梯形窄端固定（随 body 系），宽端 = 机身前缘；机身越靠前梯形越短
18325     const barrelBackHalf = size * 0.115;
18326     const barrelFrontHalf = size * 0.07;
18327     const barrel: LocalPoint[] = [
18328         { lx: -barrelBackHalf, ly: barrelBackY },

```

```

18329         { lx: barrelBackHalf, ly: barrelBackY },
18330         { lx: barrelFrontHalf, ly: barrelFrontY },
18331         { lx: -barrelFrontHalf, ly: barrelFrontY },
18332     ];
18333     fillLocalPoly(g, bodyCx, bodyCy, dx, dy, px, py, barrel, colors.chassisEdge);
18334     strokeLocalPoly(g, bodyCx, bodyCy, dx, dy, px, py, barrel, colors.accent, lw * 0.85);
18335     const muzzle = emitLocalToWorld(bodyCx, bodyCy, 0, muzzleLy, dx, dy, px, py);
18336     // 外层扩散光晕（先画，位于渐变截面之下）
18337     const glowRadii = [size * 0.22, size * 0.17, size * 0.13];
18338     const glowAlphas = [42, 78, 115];
18339     for (let i = 0; i < glowRadii.length; i++) {
18340         g.fillColor = new Color(colors.beam.r, colors.beam.g, colors.beam.b, glowAlphas[i]);
18341         g.circle(muzzle.x, muzzle.y, glowRadii[i]);
18342         g.fill();
18343     }
18344     // 与光路相同的白芯→本色渐变截面（叠在光晕之上，与光束衔接）
18345     fillBeamCrossSection(g, muzzle.x, muzzle.y, colors.beam, size);
18346 }
18347 /** 炮口中心（屏幕坐标，与 drawSourceEmitter 内计算一致） */
18348 export function sourceMuzzleScreenPoint(
18349     left: number,
18350     bottom: number,
18351     size: number,
18352     direction: Direction,
18353 ): { x: number; y: number } {
18354     const cx = left + size * 0.5;
18355     const cy = bottom + size * 0.5;
18356     const dx = SCREEN_DIR_DX[direction];
18357     const dy = SCREEN_DIR_DY[direction];
18358     const px = -dy;
18359     const py = dx;
18360     const half = size * 0.5;
18361     const muzzleLy = half + size * 0.045;
18362     const bodyCx = cx - dx * size * 0.05;
18363     const bodyCy = cy - dy * size * 0.05;
18364     return emitLocalToWorld(bodyCx, bodyCy, 0, muzzleLy, dx, dy, px, py);
18365 }
18366 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleStepCountView.ts -----
18367 import { _decorator, Component, Label, Node } from 'cc';
18368 import { EVENT_ENUM } from '../Utils/Enum';
18369 import { OPTICAL_PUZZLE } from '../Utils/Event';
18370 import type { OpticalSnapshotNotify } from '../Application/OpticalPuzzleSession';
18371 const { ccclass } = _decorator;
18372 /** 最近一次快照事件（供 inactive 节点 onEnable 时补刷） */
18373 let _lastSnapshotNotify: OpticalSnapshotNotify | null = null;
18374 export function getLastOpticalSnapshotNotify(): OpticalSnapshotNotify | null {
18375     return _lastSnapshotNotify;
18376 }
18377 export interface EnsureStepCountViewOptions {
18378     /** 为 true 时不订阅快照事件，仅由 setMoveCount 驱动（参考解回放） */
18379     manualOnly?: boolean;
18380 }
18381 /** Step/StepCount: 显示本局移动步数（TopBar 与 winPanel/winds/step 共用） */
18382 @ccclass('OpticalPuzzleStepCountView')
18383 export class OpticalPuzzleStepCountView extends Component {
18384     /** 参考解回放等场景：不监听局内快照，避免与 TopBar 步数混淆 */
18385     manualOnly = false;
18386     private _label: Label | null = null;

```

```

18387     protected onLoad(): void {
18388         this._label = this.getComponent(Label) ?? this.getComponentInChildren(Label);
18389     }
18390     protected onEnable(): void {
18391         if (!this.manualOnly) {
18392             OPTICAL_PUZZLE.on(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, this._onSnapshotChanged,
18393 this);
18394             const cached = _lastSnapshotNotify?.moveCount;
18395             if (typeof cached === 'number') {
18396                 this._refreshLabel(cached);
18397             } else {
18398                 this._refreshLabel(0);
18399             }
18400             return;
18401         }
18402         this._refreshLabel(0);
18403     }
18404     protected onDisable(): void {
18405         if (!this.manualOnly) {
18406             OPTICAL_PUZZLE.off(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, this._onSnapshotChanged,
18407 this);
18408         }
18409     }
18410     protected onDestroy(): void {
18411         if (!this.manualOnly) {
18412             OPTICAL_PUZZLE.off(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, this._onSnapshotChanged,
18413 this);
18414         }
18415     }
18416     /** 直接刷新步数（开局或事件未到前） */
18417     setMoveCount(count: number): void {
18418         this._refreshLabel(count);
18419     }
18420     private _onSnapshotChanged(payload: OpticalSnapshotNotify): void {
18421         if (!payload || typeof payload.moveCount !== 'number') {
18422             return;
18423         }
18424         _lastSnapshotNotify = payload;
18425         this._refreshLabel(payload.moveCount);
18426     }
18427     private _refreshLabel(count: number): void {
18428         if (!this._label?.isValid) {
18429             return;
18430         }
18431         this._label.string = String(Math.max(0, Math.floor(count)));
18432     }
18433 }
18434 /** 自 TopBar 子树解析 StepCount（兼容 TopBar/StepCount 与 TopBar/Step/StepCount） */
18435 export function resolveStepCountNode(topBar: Node | null): Node | null {
18436     if (!topBar?.isValid) {
18437         return null;
18438     }
18439     const direct = topBar.getChildByName('StepCount');
18440     if (direct?.isValid) {
18441         return direct;
18442     }
18443     const underStep = topBar.getChildByName('Step')?.getChildByName('StepCount') ?? null;
18444     if (underStep?.isValid) {

```

```

18445         return underStep;
18446     }
18447     return _findDescendantByName(topBar, 'StepCount');
18448 }
18449 function _findDescendantByName(root: Node, name: string): Node | null {
18450     for (const child of root.children) {
18451         if (!child.isValid) {
18452             continue;
18453         }
18454         if (child.name === name) {
18455             return child;
18456         }
18457         const found = _findDescendantByName(child, name);
18458         if (found?.isValid) {
18459             return found;
18460         }
18461     }
18462     return null;
18463 }
18464 /** 为 StepCount 挂上步数 Label 同步 */
18465 export function ensureStepCountView(
18466     stepCount: Node | null,
18467     options?: EnsureStepCountViewOptions,
18468 ): void {
18469     if (!stepCount?.isValid) {
18470         return;
18471     }
18472     let view = stepCount.getComponent(OpticalPuzzleStepCountView);
18473     if (!view) {
18474         view = stepCount.addComponent(OpticalPuzzleStepCountView);
18475     }
18476     if (options?.manualOnly) {
18477         view.manualOnly = true;
18478     }
18479 }
18480 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleStepIconGlyph.ts -----
18481 import { Color, Graphics } from 'cc';
18482 /** 图标占 StepIcon 节点短边比例（非按钮，铺满 UITransform） */
18483 const ICON_SIZE_RATIO = 1;
18484 /** 掌垫+四趾豆外接半宽（iconR=10，由 STEP_ICON_PAD_SEGS 控制点估算） */
18485 const STEP_ICON_PATH_UNIT = 7.67;
18486 const STEP_ICON_WHITE = new Color(255, 255, 255, 255);
18487 type PathSeg =
18488     | { t: 'M'; x: number; y: number }
18489     | { t: 'L'; x: number; y: number }
18490     | { t: 'C'; x1: number; y1: number; x2: number; y2: number; x: number; y: number }
18491     | { t: 'Z' };
18492 /** 掌垫 + 四趾豆（不含 SVG 最外层整圈轮廓） */
18493 const STEP_ICON_PAD_SEGS: ReadonlyArray<PathSeg> = [
18494     { t: 'M', x: 2.892, y: 7.365 },
18495     { t: 'C', x1: 3.707, y1: 7.365, x2: 4.368, y2: 6.483, x: 4.368, y: 5.395 },
18496     { t: 'C', x1: 4.368, y1: 4.308, x2: 3.707, y2: 3.426, x: 2.892, y: 3.426 },
18497     { t: 'C', x1: 2.077, y1: 3.426, x2: 1.416, y2: 4.308, x: 1.416, y: 5.395 },
18498     { t: 'C', x1: 1.416, y1: 6.483, x2: 2.077, y2: 7.364, x: 2.892, y: 7.364 },
18499     { t: 'Z' },
18500     { t: 'M', x: -2.144, y: 7.365 },
18501     { t: 'C', x1: -1.329, y1: 7.365, x2: -0.669, y2: 6.483, x: -0.669, y: 5.395 },
18502     { t: 'C', x1: -0.669, y1: 4.308, x2: -1.329, y2: 3.426, x: -2.144, y: 3.426 },

```

```

18503     { t: 'C', x1: -2.959, y1: 3.426, x2: -3.62, y2: 4.308, x: -3.62, y: 5.395 },
18504     { t: 'C', x1: -3.62, y1: 6.483, x2: -2.959, y2: 7.364, x: -2.144, y: 7.364 },
18505     { t: 'Z' },
18506     { t: 'M', x: -5.73, y: 0.225 },
18507     { t: 'C', x1: -6.545, y1: 0.225, x2: -7.206, y2: 1.106, x: -7.206, y: 2.194 },
18508     { t: 'C', x1: -7.206, y1: 3.282, x2: -6.545, y2: 4.163, x: -5.73, y: 4.163 },
18509     { t: 'C', x1: -4.915, y1: 4.163, x2: -4.254, y2: 3.282, x: -4.254, y: 2.194 },
18510     { t: 'C', x1: -4.254, y1: 1.106, x2: -4.915, y2: 0.225, x: -5.73, y: 0.225 },
18511     { t: 'Z' },
18512     { t: 'M', x: 4.302, y: -6.224 },
18513     { t: 'C', x1: 3.524, y1: -6.696, x2: 2.483, y2: -6.539, x: 1.511, y: -5.912 },
18514     { t: 'C', x1: 0.905, y1: -5.647, x2: -0.1, y2: -5.377, x: -1.139, y: -5.838 },
18515     { t: 'C', x1: -1.331, y1: -5.969, x2: -1.525, y2: -6.082, x: -1.719, y: -6.176 },
18516     { t: 'C', x1: -2.548, y1: -6.573, x2: -3.387, y2: -6.62, x: -4.041, y: -6.224 },
18517     { t: 'C', x1: -5.353, y1: -5.428, x2: -5.426, y2: -3.15, x: -4.204, y: -1.135 },
18518     { t: 'C', x1: -3.883, y1: -0.606, x2: -3.505, y2: -0.148, x: -3.098, y: 0.226 },
18519     { t: 'L', x: -3.094, y: 0.229 },
18520     { t: 'L', x: -3.093, y: 0.231 },
18521     { t: 'L', x: -3.092, y: 0.232 },
18522     { t: 'L', x: -3.091, y: 0.233 },
18523     { t: 'C', x1: -2.481, y1: 0.791, x2: -1.806, y2: 1.16, x: -1.163, y: 1.294 },
18524     { t: 'C', x1: -0.354, y1: 1.542, x2: 0.652, y2: 1.627, x: 1.746, y: 1.208 },
18525     { t: 'C', x1: 2.726, y1: 0.892, x2: 3.741, y2: 0.058, x: 4.465, y: -1.135 },
18526     { t: 'C', x1: 5.687, y1: -3.15, x2: 5.614, y2: -5.428, x: 4.302, y: -6.224 },
18527     { t: 'Z' },
18528     { t: 'M', x: 6.194, y: 0.226 },
18529     { t: 'C', x1: 5.379, y1: 0.226, x2: 4.718, y2: 1.107, x: 4.718, y: 2.195 },
18530     { t: 'C', x1: 4.718, y1: 3.283, x2: 5.379, y2: 4.164, x: 6.194, y: 4.164 },
18531     { t: 'C', x1: 7.009, y1: 4.164, x2: 7.67, y2: 3.283, x: 7.67, y: 2.195 },
18532     { t: 'C', x1: 7.67, y1: 1.107, x2: 7.009, y2: 0.226, x: 6.194, y: 0.226 },
18533     { t: 'Z' },
18534 ];
18535 function traceStepIconSegs(
18536     g: Graphics,
18537     segs: ReadonlyArray<PathSeg>,
18538     cx: number,
18539     cy: number,
18540     scale: number,
18541 ): void {
18542     for (const seg of segs) {
18543         switch (seg.t) {
18544             case 'M':
18545                 g.moveTo(cx + seg.x * scale, cy + seg.y * scale);
18546                 break;
18547             case 'L':
18548                 g.lineTo(cx + seg.x * scale, cy + seg.y * scale);
18549                 break;
18550             case 'C':
18551                 g.bezierCurveTo(
18552                     cx + seg.x1 * scale,
18553                     cy + seg.y1 * scale,
18554                     cx + seg.x2 * scale,
18555                     cy + seg.y2 * scale,
18556                     cx + seg.x * scale,
18557                     cy + seg.y * scale,
18558                 );
18559                 break;
18560             case 'Z':

```



```

18561             g.close();
18562             break;
18563         default:
18564             break;
18565     }
18566 }
18567 }
18568 /**
18569  * 绘制步数爪印：非按钮；掌垫与四趾豆全部纯白填充。
18570  * 缩放按节点 UITransform 短边适配（非 TopBar 按钮的 0.58 内缩）。
18571  */
18572 export function drawStepIconGlyph(
18573     g: Graphics,
18574     left: number,
18575     bottom: number,
18576     width: number,
18577     height: number,
18578 ): void {
18579     const size = Math.min(width, height);
18580     const cx = left + width * 0.5;
18581     const cy = bottom + height * 0.5;
18582     const iconHalf = (size * ICON_SIZE_RATIO) * 0.5;
18583     const scale = iconHalf / STEP_ICON_PATH_UNIT;
18584     g.fillColor = STEP_ICON_WHITE;
18585     traceStepIconSegs(g, STEP_ICON_PAD_SEGS, cx, cy, scale);
18586     g.fill();
18587 }
18588 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleStepIconView.ts -----
18589 import { _decorator, Component, Graphics, Node, UITransform } from 'cc';
18590 import { drawStepIconGlyph } from './OpticalPuzzleStepIconGlyph';
18591 const { ccclass } = _decorator;
18592 /** TopBar 步数爪印：仅绘制，无按钮交互 */
18593 @ccclass('OpticalPuzzleStepIconView')
18594 export class OpticalPuzzleStepIconView extends Component {
18595     private _graphics: Graphics | null = null;
18596     protected onLoad(): void {
18597         this._redraw();
18598     }
18599     protected onEnable(): void {
18600         this._redraw();
18601     }
18602     refresh(): void {
18603         this._redraw();
18604     }
18605     private _ensureGraphics(): Graphics | null {
18606         if (!this._graphics?.isValid) {
18607             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
18608         }
18609         return this._graphics;
18610     }
18611     private _redraw(): void {
18612         const g = this._ensureGraphics();
18613         const ut = this.getComponent(UITransform);
18614         if (!g || !ut) {
18615             return;
18616         }
18617         g.clear();
18618         const w = ut.width;

```

```

18619         const h = ut.height;
18620         const left = -ut.anchorX * w;
18621         const bottom = -ut.anchorY * h;
18622         drawStepIconGlyph(g, left, bottom, w, h);
18623     }
18624 }
18625 /** 为 TopBar/StepIcon 挂上爪印绘制 */
18626 export function ensureStepIconView(stepIcon: Node | null): void {
18627     if (!stepIcon?.isValid) {
18628         return;
18629     }
18630     if (!stepIcon.getComponent(OpticalPuzzleStepIconView)) {
18631         stepIcon.addComponent(OpticalPuzzleStepIconView);
18632     }
18633 }
18634 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleStepView.ts -----
18635 import { _decorator, Component, Graphics, Node, UITransform } from 'cc';
18636 import { drawHudButtonChrome, HUD_DIR_BUTTON_SCENE_SIZE } from './OpticalPuzzleHudButtonCommon';
18637 import { ensureStepCountView, resolveStepCountNode } from './OpticalPuzzleStepCountView';
18638 import { ensureStepIconView } from './OpticalPuzzleStepIconView';
18639 const { ccclass } = _decorator;
18640 /** TopBar 步数区外框：圆角 + 白边 + 键帽底色；非按钮，仅绘制底图 */
18641 @ccclass('OpticalPuzzleStepView')
18642 export class OpticalPuzzleStepView extends Component {
18643     private _graphics: Graphics | null = null;
18644     protected onLoad(): void {
18645         this._redraw();
18646     }
18647     protected onEnable(): void {
18648         this._redraw();
18649     }
18650     refresh(): void {
18651         this._redraw();
18652     }
18653     private _ensureGraphics(): Graphics | null {
18654         if (!this._graphics?.isValid) {
18655             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
18656         }
18657         return this._graphics;
18658     }
18659     private _redraw(): void {
18660         const g = this._ensureGraphics();
18661         const ut = this.getComponent(UITransform);
18662         if (!g || !ut) {
18663             return;
18664         }
18665         g.clear();
18666         const w = ut.width;
18667         const h = ut.height;
18668         const left = -ut.anchorX * w;
18669         const bottom = -ut.anchorY * h;
18670         drawHudButtonChrome(g, left, bottom, w, h, false, HUD_DIR_BUTTON_SCENE_SIZE);
18671     }
18672 }
18673 export interface EnsureStepViewsOptions {
18674     /** 步数仅由外部 setMoveCount 驱动，不订阅 OPTICAL_SNAPSHOT_CHANGED */
18675     manualStepCount?: boolean;
18676 }

```

```

18677 /** 为 TopBar/Step 挂上外框; StepIcon 仍在子节点上单独绘制爪印 */
18678 export function ensureStepViews(topBar: Node | null, options?: EnsureStepViewsOptions): void {
18679     const step = topBar?.getChildByName('Step') ?? null;
18680     if (step?.isValid && !step.getComponent(OpticalPuzzleStepView)) {
18681         step.addComponent(OpticalPuzzleStepView);
18682     }
18683     const stepIcon = step?.getChildByName('StepIcon') ?? topBar?.getChildByName('StepIcon') ?? null;
18684     ensureStepIconView(stepIcon);
18685     ensureStepCountView(resolveStepCountNode(topBar), {
18686         manualOnly: options?.manualStepCount ?? false,
18687     });
18688 }
18689 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleTargetGlyph.ts -----
18690 import { Color, Graphics } from 'cc';
18691 import { lerpBeamColor } from './OpticalPuzzleBeamGradient';
18692 import {
18693     targetDimFillColor,
18694     targetLitFillColor,
18695     targetUnlitBulbFillColor,
18696 } from './OpticalPuzzleColorUtil';
18697 import { OPTICAL_CELL_SIZE } from './OpticalPuzzleLayout';
18698 /** 与发射器一致的外轮廓圆角 */
18699 const TARGET_CORNER_RADIUS = 4;
18700 function scaledCornerRadius(size: number): number {
18701     return TARGET_CORNER_RADIUS * (size / OPTICAL_CELL_SIZE);
18702 }
18703 /** 内嵌面板（点亮/未点亮共用，仅边框染色） */
18704 const INNER_PANEL = new Color(22, 28, 38, 255);
18705 const OUTER_BASE = new Color(8, 12, 20, 255);
18706 function litBorderColor(litFill: Color): Color {
18707     return new Color(litFill.r, litFill.g, litFill.b, 255);
18708 }
18709 /** 灯体径向渐变：外圈柔光 → 内圈 100% 纯色小芯 */
18710 function fillTargetBulbGradient(
18711     g: Graphics,
18712     cx: number,
18713     cy: number,
18714     size: number,
18715     litFill: Color,
18716 ): void {
18717     const outerR = size * 0.34;
18718     const coreR = size * 0.062;
18719     const steps = 12;
18720     const outerColor = new Color(litFill.r, litFill.g, litFill.b, 88);
18721     for (let i = 0; i < steps; i++) {
18722         const u = steps <= 1 ? 1 : i / (steps - 1);
18723         const r = outerR + (coreR - outerR) * u;
18724         g.fillColor = lerpBeamColor(outerColor, litFill, u);
18725         g.circle(cx, cy, r);
18726         g.fill();
18727     }
18728     g.fillColor = litFill;
18729     g.circle(cx, cy, coreR);
18730     g.fill();
18731 }
18732 /** 灯珠右上 Specular: 多层白芯向外渐隐 */
18733 function drawBulbSpecularHighlight(
18734     g: Graphics,

```

```

18735     hx: number,
18736     hy: number,
18737     size: number,
18738 ): void {
18739     const outerR = size * 0.058;
18740     const coreR = size * 0.014;
18741     const steps = 6;
18742     for (let i = 0; i < steps; i++) {
18743         const u = steps <= 1 ? 1 : i / (steps - 1);
18744         const r = outerR + (coreR - outerR) * u;
18745         const alpha = Math.floor(6 + 204 * u * u);
18746         g.fillColor = new Color(255, 255, 255, alpha);
18747         g.circle(hx, hy, r);
18748         g.fill();
18749     }
18750 }
18751 /** 内面板四角指示点（格心对称，靠近四角） */
18752 function drawCornerIndicatorDots(
18753     g: Graphics,
18754     cx: number,
18755     cy: number,
18756     innerHalf: number,
18757     size: number,
18758     color: Color,
18759 ): void {
18760     const inset = size * 0.1;
18761     const dotR = size * 0.035;
18762     const offsets = [
18763         { x: -innerHalf + inset, y: -innerHalf + inset },
18764         { x: innerHalf - inset, y: -innerHalf + inset },
18765         { x: -innerHalf + inset, y: innerHalf - inset },
18766         { x: innerHalf - inset, y: innerHalf - inset },
18767     ];
18768     g.fillColor = color;
18769     for (const o of offsets) {
18770         g.circle(cx + o.x, cy + o.y, dotR);
18771         g.fill();
18772     }
18773 }
18774 /**
18775  * 简约指示灯：未点亮为磨砂暗灯，点亮后与期望色同色发光。
18776  * 须在光路层之上绘制，避免被光束盖住。
18777  */
18778 export function drawTargetLamp(
18779     g: Graphics,
18780     left: number,
18781     bottom: number,
18782     size: number,
18783     colorKey: string | undefined,
18784     lit: boolean,
18785 ): void {
18786     const cx = left + size * 0.5;
18787     const cy = bottom + size * 0.5;
18788     const corner = scaledCornerRadius(size);
18789     const lw = Math.max(1, size * 0.022);
18790     const litFill = targetLitFillColor(colorKey);
18791     const bezel = size * 0.017;
18792     const innerLeft = left + bezel;

```

```

18793     const innerBottom = bottom + bezel;
18794     const innerSize = size - bezel * 2;
18795     const innerCorner = corner * 0.7;
18796     const innerHalf = innerSize * 0.5;
18797     if (lit) {
18798         const border = litBorderColor(litFill);
18799         // 边框环: 外圈 100% 灯光色
18800         g.fillColor = border;
18801         g.roundRect(left, bottom, size, size, corner);
18802         g.fill();
18803         g.strokeColor = border;
18804         g.lineWidth = Math.max(1, size * 0.011);
18805         g.roundRect(left, bottom, size, size, corner);
18806         g.stroke();
18807         // 内部仍用暗色面板, 灯体由下方光晕/灯芯绘制
18808         g.fillColor = INNER_PANEL;
18809         g.roundRect(innerLeft, innerBottom, innerSize, innerSize, innerCorner);
18810         g.fill();
18811         g.strokeColor = border;
18812         g.lineWidth = Math.max(1, size * 0.008);
18813         g.roundRect(innerLeft, innerBottom, innerSize, innerSize, innerCorner);
18814         g.stroke();
18815     } else {
18816         g.fillColor = OUTER_BASE;
18817         g.roundRect(left, bottom, size, size, corner);
18818         g.fill();
18819         g.fillColor = INNER_PANEL;
18820         g.roundRect(innerLeft, innerBottom, innerSize, innerSize, innerCorner);
18821         g.fill();
18822     }
18823     const bulbR = size * 0.34;
18824     if (lit) {
18825         const halos = [
18826             { r: size * 0.48, a: 24 },
18827             { r: size * 0.44, a: 38 },
18828             { r: size * 0.4, a: 52 },
18829             { r: size * 0.36, a: 68 },
18830             { r: size * 0.32, a: 86 },
18831         ];
18832         for (const h of halos) {
18833             g.fillColor = new Color(litFill.r, litFill.g, litFill.b, h.a);
18834             g.circle(cx, cy, h.r);
18835             g.fill();
18836         }
18837         fillTargetBulbGradient(g, cx, cy, size, litFill);
18838         drawBulbSpecularHighlight(
18839             g,
18840             cx + bulbR * 0.34,
18841             cy + bulbR * 0.34,
18842             size,
18843         );
18844         drawCornerIndicatorDots(g, cx, cy, innerHalf, size, new Color(255, 255, 255, 255));
18845         return;
18846     }
18847     const dim = targetDimFillColor(colorKey);
18848     const bulbFill = targetUnlitBulbFillColor(colorKey);
18849     g.fillColor = bulbFill;
18850     g.circle(cx, cy, bulbR);

```

```

18851     g.fill();
18852     g.strokeColor = new Color(
18853         Math.floor(dim.r * 0.92),
18854         Math.floor(dim.g * 0.92),
18855         Math.floor(dim.b * 0.92),
18856         170,
18857     );
18858     g.lineWidth = lw * 0.65;
18859     g.circle(cx, cy, bulbR);
18860     g.stroke();
18861     drawCornerIndicatorDots(g, cx, cy, innerHalf, size, litBorderColor(litFill));
18862 }
18863 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWallDraw.ts -----
18864 import { Color, Graphics } from 'cc';
18865 import type { OpticalBoardSnapshot } from '../Core/OpticalPuzzleTypes';
18866 import { TerrainKind } from '../Core/OpticalPuzzleTypes';
18867 import {
18868     FLOOR_FILL,
18869     OPTICAL_CELL_SIZE,
18870     WALL_ARM_FILL,
18871     WALL_ARM_THICK,
18872     WALL_BLOCK_PATCH_FILL,
18873     WALL_CORE_SIZE,
18874     WALL_CORNER_PATCH,
18875     WALL_DARK_FILL,
18876     WALL_LIGHT_FILL,
18877     WALL_OVERLAY_ARM,
18878     WALL_OVERLAY_CONNECT_PAD,
18879     WALL_OVERLAY_CONNECT_PAD_H,
18880     WALL_OVERLAY_CORE_SIZE,
18881     WALL_OVERLAY_FILL,
18882 } from './OpticalPuzzleLayout';
18883 export function cellScreenRect(
18884     ox: number,
18885     oy: number,
18886     x: number,
18887     y: number,
18888     cell: number = OPTICAL_CELL_SIZE,
18889 ): { left: number; bottom: number; size: number } {
18890     return {
18891         left: ox + x * cell,
18892         bottom: oy - (y + 1) * cell,
18893         size: cell,
18894     };
18895 }
18896 /** 棋盘全域铺地板（先于墙/光源；圆角元件四角透明时不透出背景布朗方块） */
18897 export function fillBoardFloorBase(
18898     g: Graphics,
18899     width: number,
18900     height: number,
18901     ox: number,
18902     oy: number,
18903     cell: number = OPTICAL_CELL_SIZE,
18904     fill: Color = FLOOR_FILL,
18905 ): void {
18906     if (fill.a <= 0) {
18907         return;
18908     }

```

```

18909     const boardW = width * cell;
18910     const boardH = height * cell;
18911     g.fillColor = fill;
18912     g.rect(ox, oy - boardH, boardW, boardH);
18913     g.fill();
18914 }
18915 /** 单格地板垫底（用于元件/目标等上层圆角透明区域） */
18916 export function fillFloorCell(
18917     g: Graphics,
18918     ox: number,
18919     oy: number,
18920     x: number,
18921     y: number,
18922     cell: number = OPTICAL_CELL_SIZE,
18923     fill: Color = FLOOR_FILL,
18924 ): void {
18925     if (fill.a <= 0) {
18926         return;
18927     }
18928     const { left, bottom, size } = cellScreenRect(ox, oy, x, y, cell);
18929     g.fillColor = fill;
18930     g.rect(left, bottom, size, size);
18931     g.fill();
18932 }
18933 function isFloor(snapshot: OpticalBoardSnapshot, x: number, y: number): boolean {
18934     if (x < 0 || y < 0 || x >= snapshot.width || y >= snapshot.height) {
18935         return false;
18936     }
18937     return snapshot.terrain[y * snapshot.width + x] !== TerrainKind.Wall;
18938 }
18939 function isWall(snapshot: OpticalBoardSnapshot, x: number, y: number): boolean {
18940     if (x < 0 || y < 0 || x >= snapshot.width || y >= snapshot.height) {
18941         return false;
18942     }
18943     return snapshot.terrain[y * snapshot.width + x] === TerrainKind.Wall;
18944 }
18945 /** 外角圆角：两邻格均为地板（不判对角） */
18946 function isWallCornerRound(
18947     snapshot: OpticalBoardSnapshot,
18948     gx: number,
18949     gy: number,
18950     corner: 'tl' | 'tr' | 'bl' | 'br',
18951 ): boolean {
18952     switch (corner) {
18953         case 'tl':
18954             return isFloor(snapshot, gx, gy - 1) && isFloor(snapshot, gx - 1, gy);
18955         case 'tr':
18956             return isFloor(snapshot, gx, gy - 1) && isFloor(snapshot, gx + 1, gy);
18957         case 'bl':
18958             return isFloor(snapshot, gx - 1, gy) && isFloor(snapshot, gx, gy + 1);
18959         case 'br':
18960             return isFloor(snapshot, gx + 1, gy) && isFloor(snapshot, gx, gy + 1);
18961     }
18962 }
18963 function isOutOfBounds(snapshot: OpticalBoardSnapshot, x: number, y: number): boolean {
18964     return x < 0 || y < 0 || x >= snapshot.width || y >= snapshot.height;
18965 }
18966 /** 外扩实际是否为 #226c8f（与 fillOutwardRect 规则唯一同步，角块只读此结果） */

```

```

18967 function isOutwardExtDark(
18968     snapshot: OpticalBoardSnapshot,
18969     gx: number,
18970     gy: number,
18971     side: 'right' | 'bottom' | 'left' | 'top',
18972 ): boolean {
18973     switch (side) {
18974         case 'right':
18975             return isFloor(snapshot, gx + 1, gy);
18976         case 'bottom':
18977             return isFloor(snapshot, gx, gy + 1);
18978         case 'left':
18979         case 'top':
18980             default:
18981                 return false;
18982     }
18983 }
18984 function fillRect(g: Graphics, color: Color, x: number, y: number, w: number, h: number): void {
18985     g.fillColor = color;
18986     g.rect(x, y, w, h);
18987     g.fill();
18988 }
18989 function fillTriangle(
18990     g: Graphics,
18991     color: Color,
18992     x1: number,
18993     y1: number,
18994     x2: number,
18995     y2: number,
18996     x3: number,
18997     y3: number,
18998 ): void {
18999     g.fillColor = color;
19000     g.moveTo(x1, y1);
19001     g.lineTo(x2, y2);
19002     g.lineTo(x3, y3);
19003     g.close();
19004     g.fill();
19005 }
19006 /** 圆心 + 两径向边 + 弧段（折线近似，不依赖 arc 方向） */
19007 function fillPieWedge(
19008     g: Graphics,
19009     cx: number,
19010     cy: number,
19011     r: number,
19012     startRad: number,
19013     endRad: number,
19014     color: Color,
19015     segments: number = 10,
19016 ): void {
19017     g.fillColor = color;
19018     g.moveTo(cx, cy);
19019     g.lineTo(cx + Math.cos(startRad) * r, cy + Math.sin(startRad) * r);
19020     for (let i = 1; i <= segments; i++) {
19021         const t = startRad + ((endRad - startRad) * i) / segments;
19022         g.lineTo(cx + Math.cos(t) * r, cy + Math.sin(t) * r);
19023     }
19024     g.close();

```



```

19025     g.fill();
19026 }
19027 /** 外角圆角补块：圆心在内角，仅画补块内可见的 90° 四分之一弧 */
19028 function fillCornerQuarter(
19029     g: Graphics,
19030     cx: number,
19031     cy: number,
19032     r: number,
19033     corner: 'tl' | 'tr' | 'bl' | 'br',
19034     color: Color,
19035 ): void {
19036     switch (corner) {
19037         case 'tl':
19038             fillPieWedge(g, cx, cy, r, Math.PI / 2, Math.PI, color);
19039             break;
19040         case 'tr':
19041             fillPieWedge(g, cx, cy, r, 0, Math.PI / 2, color);
19042             break;
19043         case 'bl':
19044             fillPieWedge(g, cx, cy, r, Math.PI, (3 * Math.PI) / 2, color);
19045             break;
19046         case 'br':
19047             fillPieWedge(g, cx, cy, r, (3 * Math.PI) / 2, 2 * Math.PI, color);
19048             break;
19049     }
19050 }
19051 function fillCornerSquare(
19052     g: Graphics,
19053     left: number,
19054     bottom: number,
19055     right: number,
19056     top: number,
19057     p: number,
19058     corner: 'tl' | 'tr' | 'bl' | 'br',
19059     color: Color,
19060 ): void {
19061     switch (corner) {
19062         case 'tl':
19063             fillRect(g, color, left, top - p, p, p);
19064             break;
19065         case 'tr':
19066             fillRect(g, color, right - p, top - p, p, p);
19067             break;
19068         case 'bl':
19069             fillRect(g, color, left, bottom, p, p);
19070             break;
19071         case 'br':
19072             fillRect(g, color, right - p, bottom, p, p);
19073             break;
19074     }
19075 }
19076 /**
19077  * 仅一种 L 内凹地形（地板在 2×2 右下，墙占其余三格）：
19078  * 地板在 (fx,fy)，三墙分别为 (fx-1,fy-1)(fx,fy-1)(fx-1,fy)：
19079  *   ##      左上 (fx-1,fy-1) → BR 整块深色
19080  *   #.      右上 (fx,fy-1)   → BL 整块深色
19081  *   .#      左下 (fx-1,fy)   → TR 整块深色
19082  * 其余拐角仍走原有外扩 / 分半规则。

```

```

19083     */
19084     function isLConcaveFloor(snapshot: OpticalBoardSnapshot, fx: number, fy: number): boolean {
19085         return (
19086             isFloor(snapshot, fx, fy) &&
19087             isWall(snapshot, fx - 1, fy - 1) &&
19088             isWall(snapshot, fx, fy - 1) &&
19089             isWall(snapshot, fx - 1, fy)
19090         );
19091     }
19092     /**
19093      * ## / #. 内凹 L（地板在 2×2 右下）：按格坐标唯一映射补块，作强制补丁。
19094      * 不依赖外扩分半；在 fillWallCell 末尾覆盖绘制。
19095      */
19096     function getLConcavePatchCorner(
19097         snapshot: OpticalBoardSnapshot,
19098         gx: number,
19099         gy: number,
19100     ): 'tl' | 'tr' | 'bl' | 'br' | null {
19101         if (isLConcaveFloor(snapshot, gx + 1, gy + 1)) {
19102             return 'br';
19103         }
19104         if (isLConcaveFloor(snapshot, gx, gy + 1)) {
19105             return 'bl';
19106         }
19107         if (isLConcaveFloor(snapshot, gx + 1, gy)) {
19108             return 'tr';
19109         }
19110         return null;
19111     }
19112     /** 内层 L 内凹强制补丁：整角 14×14 #226c8f（须在叠层之前绘制） */
19113     export function applyLConcaveForcedPatch(
19114         g: Graphics,
19115         snapshot: OpticalBoardSnapshot,
19116         gx: number,
19117         gy: number,
19118         left: number,
19119         bottom: number,
19120         right: number,
19121         top: number,
19122         p: number,
19123     ): void {
19124         const corner = getLConcavePatchCorner(snapshot, gx, gy);
19125         if (!corner) {
19126             return;
19127         }
19128         fillCornerSquare(g, left, bottom, right, top, p, corner, WALL_DARK_FILL);
19129     }
19130     /** 右上角圆角：四分之一圆在 45° 处分两段（左下角的镜像） */
19131     function fillTrRoundSplit(
19132         g: Graphics,
19133         cx: number,
19134         cy: number,
19135         r: number,
19136         upperColor: Color,
19137         lowerColor: Color,
19138     ): void {
19139         const a45 = Math.PI / 4;
19140         const a90 = Math.PI / 2;

```

```

19141         fillPieWedge(g, cx, cy, r, 0, a45, lowerColor);
19142         fillPieWedge(g, cx, cy, r, a45, a90, upperColor);
19143     }
19144     /** 左下角圆角：四分之一圆在 225° 处分两段（与方角三角对角线一致） */
19145     function fillBlRoundSplit(
19146         g: Graphics,
19147         cx: number,
19148         cy: number,
19149         r: number,
19150         upperColor: Color,
19151         lowerColor: Color,
19152     ): void {
19153         const a180 = Math.PI;
19154         const a225 = (5 * Math.PI) / 4;
19155         const a270 = (3 * Math.PI) / 2;
19156         fillPieWedge(g, cx, cy, r, a225, a270, lowerColor);
19157         fillPieWedge(g, cx, cy, r, a180, a225, upperColor);
19158     }
19159     /** 外扩矩形：邻格为地板时按方向加深（仅右、下两向） */
19160     function fillOutwardRect(
19161         g: Graphics,
19162         snapshot: OpticalBoardSnapshot,
19163         gx: number,
19164         gy: number,
19165         x: number,
19166         y: number,
19167         w: number,
19168         h: number,
19169         side: 'right' | 'bottom' | 'left' | 'top',
19170     ): void {
19171         const color = isOutwardExtDark(snapshot, gx, gy, side) ? WALL_DARK_FILL : WALL_ARM_FILL;
19172         fillRect(g, color, x, y, w, h);
19173     }
19174     /** 邻格墙的外扩色（非墙格无外扩，不参与角块着色） */
19175     function wallOutwardExtDark(
19176         snapshot: OpticalBoardSnapshot,
19177         gx: number,
19178         gy: number,
19179         side: 'right' | 'bottom' | 'left' | 'top',
19180     ): boolean {
19181         if (!isWall(snapshot, gx, gy)) {
19182             return false;
19183         }
19184         return isOutwardExtDark(snapshot, gx, gy, side);
19185     }
19186     /**
19187      * 左下角补块：沿外角对角线（225°）拆成两半着色。
19188      * 圆角：可见四分之一弧 180° - 270° 在 225° 处分段；方角：沿对角线切三角。
19189      * 上半：本色左扩或左邻墙下扩为深色；下半：本色下扩或下邻墙左扩为深色。
19190      */
19191     function fillBottomLeftCorner(
19192         g: Graphics,
19193         snapshot: OpticalBoardSnapshot,
19194         gx: number,
19195         gy: number,
19196         left: number,
19197         bottom: number,
19198         right: number,

```

```

19199     top: number,
19200     cx: number,
19201     cy: number,
19202     p: number,
19203     blRound: boolean,
19204 ): void {
19205     const upperDark =
19206         isOutwardExtDark(snapshot, gx, gy, 'left') ||
19207         wallOutwardExtDark(snapshot, gx - 1, gy, 'bottom');
19208     const lowerDark =
19209         isOutwardExtDark(snapshot, gx, gy, 'bottom') ||
19210         wallOutwardExtDark(snapshot, gx, gy + 1, 'left');
19211     const upperColor = upperDark ? WALL_DARK_FILL : WALL_ARM_FILL;
19212     const lowerColor = lowerDark ? WALL_DARK_FILL : WALL_ARM_FILL;
19213     if (blRound) {
19214         fillBlRoundSplit(g, cx, cy, p, upperColor, lowerColor);
19215         return;
19216     }
19217     const outerX = left;
19218     const outerY = bottom;
19219     const innerX = left + p;
19220     const innerY = bottom + p;
19221     fillTriangle(g, upperColor, outerX, innerY, outerX, outerY, innerX, innerY);
19222     fillTriangle(g, lowerColor, outerX, outerY, innerX, outerY, innerX, innerY);
19223 }
19224 /**
19225  * 外层右上角 upper/lower 着色。
19226  * upper 深色：正上方为墙，或本格上外扩贴地板，或上邻墙右外扩贴地板。
19227  */
19228 function resolveTopRightCornerColors(
19229     snapshot: OpticalBoardSnapshot,
19230     gx: number,
19231     gy: number,
19232 ): { upperColor: Color; lowerColor: Color } {
19233     const upperDark =
19234         isWall(snapshot, gx, gy - 1) ||
19235         isOutwardExtDark(snapshot, gx, gy, 'top') ||
19236         wallOutwardExtDark(snapshot, gx, gy - 1, 'right');
19237     const lowerDark =
19238         isOutwardExtDark(snapshot, gx, gy, 'right') ||
19239         wallOutwardExtDark(snapshot, gx + 1, gy, 'top');
19240     return {
19241         upperColor: upperDark ? WALL_DARK_FILL : WALL_ARM_FILL,
19242         lowerColor: lowerDark ? WALL_DARK_FILL : WALL_ARM_FILL,
19243     };
19244 }
19245 /**
19246  * 右上角补块：沿外角对角线（45°）拆成两半着色（左下角镜像）。
19247  * 圆角：可见四分之一弧 0° - 90° 在 45° 处分段；方角：沿对角线切三角。
19248  * 上半：上邻为墙 / 本色上扩 / 上邻墙右扩为深色；下半：本色右扩或右邻墙上扩为深色。
19249  */
19250 function fillTopRightCorner(
19251     g: Graphics,
19252     snapshot: OpticalBoardSnapshot,
19253     gx: number,
19254     gy: number,
19255     left: number,
19256     bottom: number,

```

```

19257     right: number,
19258     top: number,
19259     cx: number,
19260     cy: number,
19261     p: number,
19262     trRound: boolean,
19263 ): void {
19264     const { upperColor, lowerColor } = resolveTopRightCornerColors(snapshot, gx, gy);
19265     if (trRound) {
19266         fillTrRoundSplit(g, cx, cy, p, upperColor, lowerColor);
19267         return;
19268     }
19269     const outerX = right;
19270     const outerY = top;
19271     const innerX = right - p;
19272     const innerY = top - p;
19273     fillTriangle(g, upperColor, innerX, outerY, outerX, outerY, innerX, innerY);
19274     fillTriangle(g, lowerColor, outerX, outerY, outerX, innerY, innerX, innerY);
19275 }
19276 function fillOverlayCornerSquare(
19277     g: Graphics,
19278     coreX: number,
19279     coreY: number,
19280     arm: number,
19281     corner: 'tl' | 'tr' | 'bl' | 'br',
19282     color: Color,
19283 ): void {
19284     switch (corner) {
19285         case 'bl':
19286             fillRect(g, color, coreX - arm, coreY - arm, arm, arm);
19287             break;
19288         case 'br':
19289             fillRect(g, color, coreX + WALL_OVERLAY_CORE_SIZE, coreY - arm, arm, arm);
19290             break;
19291         case 'tl':
19292             fillRect(g, color, coreX - arm, coreY + WALL_OVERLAY_CORE_SIZE, arm, arm);
19293             break;
19294         case 'tr':
19295             fillRect(g, color, coreX + WALL_OVERLAY_CORE_SIZE, coreY + WALL_OVERLAY_CORE_SIZE, arm,
19296 arm);
19297             break;
19298     }
19299 }
19300 /**
19301  * 叠层: 20×20 墙心 + 四向 8px 臂 + 四角 8×8 补块; 邻墙连通时先上下、后左右补至格边界。
19302  */
19303 function fillWallOverlayLayer(
19304     g: Graphics,
19305     snapshot: OpticalBoardSnapshot,
19306     gx: number,
19307     gy: number,
19308     left: number,
19309     bottom: number,
19310     size: number,
19311 ): void {
19312     const arm = WALL_OVERLAY_ARM;
19313     const coreSize = WALL_OVERLAY_CORE_SIZE;
19314     const overlayInset = (size - coreSize) * 0.5;

```

```

19315     const coreX = left + overlayInset;
19316     const coreY = bottom + overlayInset;
19317     fillRect(g, WALL_OVERLAY_FILL, coreX, coreY, coreSize, coreSize);
19318     fillRect(g, WALL_OVERLAY_FILL, coreX, coreY - arm, coreSize, arm);
19319     fillRect(g, WALL_OVERLAY_FILL, coreX, coreY + coreSize, coreSize, arm);
19320     fillRect(g, WALL_OVERLAY_FILL, coreX - arm, coreY, arm, coreSize);
19321     fillRect(g, WALL_OVERLAY_FILL, coreX + coreSize, coreY, arm, coreSize);
19322     const tlRound = isWallCornerRound(snapshot, gx, gy, 'tl');
19323     const blRound = isWallCornerRound(snapshot, gx, gy, 'bl');
19324     const trRound = isWallCornerRound(snapshot, gx, gy, 'tr');
19325     const brRound = isWallCornerRound(snapshot, gx, gy, 'br');
19326     const blCx = coreX;
19327     const blCy = coreY;
19328     const brCx = coreX + coreSize;
19329     const brCy = coreY;
19330     const tlCx = coreX;
19331     const tlCy = coreY + coreSize;
19332     const trCx = coreX + coreSize;
19333     const trCy = coreY + coreSize;
19334     if (tlRound) {
19335         fillCornerQuarter(g, tlCx, tlCy, arm, 'tl', WALL_OVERLAY_FILL);
19336     } else {
19337         fillOverlayCornerSquare(g, coreX, coreY, arm, 'tl', WALL_OVERLAY_FILL);
19338     }
19339     if (trRound) {
19340         fillCornerQuarter(g, trCx, trCy, arm, 'tr', WALL_OVERLAY_FILL);
19341     } else {
19342         fillOverlayCornerSquare(g, coreX, coreY, arm, 'tr', WALL_OVERLAY_FILL);
19343     }
19344     if (blRound) {
19345         fillCornerQuarter(g, blCx, blCy, arm, 'bl', WALL_OVERLAY_FILL);
19346     } else {
19347         fillOverlayCornerSquare(g, coreX, coreY, arm, 'bl', WALL_OVERLAY_FILL);
19348     }
19349     if (brRound) {
19350         fillCornerQuarter(g, brCx, brCy, arm, 'br', WALL_OVERLAY_FILL);
19351     } else {
19352         fillOverlayCornerSquare(g, coreX, coreY, arm, 'br', WALL_OVERLAY_FILL);
19353     }
19354     fillWallOverlayConnectExtensions(g, snapshot, gx, gy, left, bottom, size, overlayInset);
19355 }
19356 /**
19357  * 2×2 墙块内角补丁（内层 14×14）：叠层之前用 #3093bc 补块内对角。
19358  * 本格为块左上 → 补 BR；块右上 → BL；块左下 → TR；块右下 → TL。
19359  */
19360 function applyWallBlockInnerCornerPatch(
19361     g: Graphics,
19362     snapshot: OpticalBoardSnapshot,
19363     gx: number,
19364     gy: number,
19365     left: number,
19366     bottom: number,
19367     right: number,
19368     top: number,
19369     p: number,
19370 ): void {
19371     const wallAt = (dx: number, dy: number) => isWall(snapshot, gx + dx, gy + dy);
19372     if (wallAt(1, 0) && wallAt(0, 1) && wallAt(1, 1)) {

```

```

19373         fillCornerSquare(g, left, bottom, right, top, p, 'br', WALL_BLOCK_PATCH_FILL);
19374     }
19375     if (wallAt(-1, 0) && wallAt(0, 1) && wallAt(-1, 1)) {
19376         fillCornerSquare(g, left, bottom, right, top, p, 'bl', WALL_BLOCK_PATCH_FILL);
19377     }
19378     if (wallAt(1, 0) && wallAt(0, -1) && wallAt(1, -1)) {
19379         fillCornerSquare(g, left, bottom, right, top, p, 'tr', WALL_BLOCK_PATCH_FILL);
19380     }
19381     if (wallAt(-1, 0) && wallAt(0, -1) && wallAt(-1, -1)) {
19382         fillCornerSquare(g, left, bottom, right, top, p, 'tl', WALL_BLOCK_PATCH_FILL);
19383     }
19384 }
19385 /**
19386  * 叠层连通扩展：先上下、后左右。
19387  * 邻格为墙 → 沿十字臂带宽补缝；越界 → 整侧 overlayInset 带补满。
19388  */
19389 function fillWallOverlayConnectExtensions(
19390     g: Graphics,
19391     snapshot: OpticalBoardSnapshot,
19392     gx: number,
19393     gy: number,
19394     left: number,
19395     bottom: number,
19396     size: number,
19397     overlayInset: number,
19398 ): void {
19399     const arm = WALL_OVERLAY_ARM;
19400     const coreSize = WALL_OVERLAY_CORE_SIZE;
19401     const padV = WALL_OVERLAY_CONNECT_PAD;
19402     const padH = WALL_OVERLAY_CONNECT_PAD_H;
19403     const gapV = overlayInset - arm + padV;
19404     const gapH = overlayInset - arm + padV;
19405     const coreX = left + overlayInset;
19406     const coreY = bottom + overlayInset;
19407     const top = bottom + size;
19408     const right = left + size;
19409     const crossSpan = coreSize + arm * 2;
19410     if (isOutOfBounds(snapshot, gx, gy + 1)) {
19411         fillRect(g, WALL_OVERLAY_FILL, left, bottom, size, overlayInset);
19412     } else if (isWall(snapshot, gx, gy + 1)) {
19413         fillRect(g, WALL_OVERLAY_FILL, coreX - arm, bottom, crossSpan, gapV);
19414     }
19415     if (isOutOfBounds(snapshot, gx, gy - 1)) {
19416         fillRect(g, WALL_OVERLAY_FILL, left, top - overlayInset, size, overlayInset);
19417     } else if (isWall(snapshot, gx, gy - 1)) {
19418         fillRect(
19419             g,
19420             WALL_OVERLAY_FILL,
19421             coreX - arm,
19422             coreY + coreSize + arm - padV,
19423             crossSpan,
19424             gapV,
19425         );
19426     }
19427     if (isOutOfBounds(snapshot, gx - 1, gy)) {
19428         fillRect(g, WALL_OVERLAY_FILL, left, bottom, overlayInset, size);
19429     } else if (isWall(snapshot, gx - 1, gy)) {
19430         fillRect(g, WALL_OVERLAY_FILL, left - padH, coreY - arm, gapH + padH, crossSpan);

```

```

19431     }
19432     if (isOutOfBounds(snapshot, gx + 1, gy)) {
19433         fillRect(g, WALL_OVERLAY_FILL, right - overlayInset, bottom, overlayInset, size);
19434     } else if (isWall(snapshot, gx + 1, gy)) {
19435         fillRect(g, WALL_OVERLAY_FILL, right - gapH, coreY - arm, gapH + padH, crossSpan);
19436     }
19437 }
19438 /**
19439  * 墙格: 外层 28×28 + 臂 14 + 角补; 叠层 #3093bc 墙心 20×20 + 臂 8 + 角补 8×8。
19440  */
19441 export function fillWallCell(
19442     g: Graphics,
19443     snapshot: OpticalBoardSnapshot,
19444     gx: number,
19445     gy: number,
19446     left: number,
19447     bottom: number,
19448     size: number = OPTICAL_CELL_SIZE,
19449 ): void {
19450     const inset = (size - WALL_CORE_SIZE) * 0.5;
19451     const top = bottom + size;
19452     const right = left + size;
19453     const p = WALL_CORNER_PATCH;
19454     const tlCx = left + inset;
19455     const tlCy = top - inset;
19456     const trCx = right - inset;
19457     const trCy = top - inset;
19458     const blCx = left + inset;
19459     const blCy = bottom + inset;
19460     const brCx = right - inset;
19461     const brCy = bottom + inset;
19462     fillRect(g, WALL_LIGHT_FILL, left + inset, bottom + inset, WALL_CORE_SIZE, WALL_CORE_SIZE);
19463     fillOutwardRect(g, snapshot, gx, gy, left + inset, bottom, WALL_CORE_SIZE, WALL_ARM_THICK, 'bottom');
19464     fillOutwardRect(g, snapshot, gx, gy, left + inset, top - WALL_ARM_THICK, WALL_CORE_SIZE,
19465 WALL_ARM_THICK, 'top');
19466     fillOutwardRect(g, snapshot, gx, gy, left, bottom + inset, WALL_ARM_THICK, WALL_CORE_SIZE, 'left');
19467     fillOutwardRect(g, snapshot, gx, gy, right - WALL_ARM_THICK, bottom + inset, WALL_ARM_THICK,
19468 WALL_CORE_SIZE, 'right');
19469     const tlRound = isWallCornerRound(snapshot, gx, gy, 'tl');
19470     const blRound = isWallCornerRound(snapshot, gx, gy, 'bl');
19471     const trRound = isWallCornerRound(snapshot, gx, gy, 'tr');
19472     const brRound = isWallCornerRound(snapshot, gx, gy, 'br');
19473     if (tlRound) {
19474         fillCornerQuarter(g, tlCx, tlCy, p, 'tl', WALL_ARM_FILL);
19475     } else {
19476         fillCornerSquare(g, left, bottom, right, top, p, 'tl', WALL_ARM_FILL);
19477     }
19478     fillTopRightCorner(g, snapshot, gx, gy, left, bottom, right, top, trCx, trCy, p, trRound);
19479     fillBottomLeftCorner(g, snapshot, gx, gy, left, bottom, right, top, blCx, blCy, p, blRound);
19480     const brColor =
19481         isOutwardExtDark(snapshot, gx, gy, 'right') || isOutwardExtDark(snapshot, gx, gy, 'bottom')
19482         ? WALL_DARK_FILL
19483         : WALL_ARM_FILL;
19484     if (brRound) {
19485         fillCornerQuarter(g, brCx, brCy, p, 'br', brColor);
19486     } else {
19487         fillCornerSquare(g, left, bottom, right, top, p, 'br', brColor);
19488     }

```



```

19489 // 内层 L 内凹补丁（叠层之前）
19490 applyLConcaveForcedPatch(g, snapshot, gx, gy, left, bottom, right, top, p);
19491 applyWallBlockInnerCornerPatch(g, snapshot, gx, gy, left, bottom, right, top, p);
19492 fillWallOverlayLayer(g, snapshot, gx, gy, left, bottom, size);
19493 }
19494 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelNextLevelButtonGlyph.ts
19495 -----
19496 import { Color, Graphics } from 'cc';
19497 import {
19498     HUD_DIR_BUTTON_SCENE_SIZE,
19499     HUD_KEY_CORNER_DESIGN,
19500     scaleHudDesign,
19501     strokeGlowLayers,
19502     unlitHudIconColor,
19503 } from './OpticalPuzzleHudButtonCommon';
19504 /** nextlevel 外框线宽（设计 px，不随节点缩放） */
19505 export const WIN_NEXT_LEVEL_BORDER_PX = 4;
19506 const NEXT_LEVEL_BORDER = new Color(255, 255, 255, 255);
19507 /** nextlevel 按下光晕（略大于四向键 PRESSED_GLOW_LAYERS，缩放基准与 100×100 四向键对齐） */
19508 const WIN_NEXT_LEVEL_PRESSED_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
19509     { width: 14, alpha: 48 },
19510     { width: 10, alpha: 120 },
19511     { width: 5, alpha: 255 },
19512 ];
19513 /** 绘制 winds/nextlevel: 四向键 icon 未按填充色 + 4px 圆角白框（无内芯图标） */
19514 export function drawWinPanelNextLevelButtonGlyph(
19515     g: Graphics,
19516     left: number,
19517     bottom: number,
19518     width: number,
19519     height: number,
19520     pressed = false,
19521     borderPx: number = WIN_NEXT_LEVEL_BORDER_PX,
19522 ): void {
19523     const size = Math.min(width, height);
19524     const corner = scaleHudDesign(size, HUD_KEY_CORNER_DESIGN);
19525     const traceChrome = (): void => {
19526         g.roundRect(left, bottom, width, height, corner);
19527     };
19528     g.fillColor = unlitHudIconColor();
19529     traceChrome();
19530     g.fill();
19531     if (pressed) {
19532         strokeGlowLayers(
19533             g,
19534             HUD_DIR_BUTTON_SCENE_SIZE,
19535             WIN_NEXT_LEVEL_PRESSED_GLOW_LAYERS,
19536             traceChrome,
19537             true,
19538         );
19539         return;
19540     }
19541     g.strokeColor = NEXT_LEVEL_BORDER;
19542     g.lineWidth = borderPx;
19543     traceChrome();
19544     g.stroke();
19545 }
19546 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelNextLevelButtonView.ts -----

```

```

19547 import {
19548     _decorator,
19549     Button,
19550     Component,
19551     director,
19552     Graphics,
19553     Label,
19554     Node,
19555     UITransform,
19556 } from 'cc';
19557 import { AUDIO_EFFECT_ENUM, EVENT_ENUM, SCENE_ENUM } from '../Utils/Enum';
19558 import { PLAY_AUDIO } from '../Utils/Event';
19559 import { getNextOpticalLevelId } from './Config/OpticalPuzzleLevels';
19560 import { HudButtonPressController } from './OpticalPuzzleHudButtonCommon';
19561 import { drawWinPanelNextLevelButtonGlyph } from './OpticalPuzzleWinPanelNextLevelButtonGlyph';
19562 import {
19563     resolveWinPanelNode,
19564     resolveWinPanelWindsNode,
19565 } from './OpticalPuzzleWinPanelWindsView';
19566 const { ccclass } = _decorator;
19567 /** 与四向键 Button.zoomScale 一致 */
19568 const NEXT_LEVEL_BUTTON_ZOOM_SCALE = 0.95;
19569 /** nextlevel/label 文案 */
19570 const WIN_NEXT_LEVEL_TEXT = '下一关';
19571 const WIN_RETURN_TEXT = '返回';
19572 /** 局内入口能力（避免与 OpticalPuzzleRoot 循环引用） */
19573 interface IOpticalPuzzleRootApi {
19574     getCurrentLevelId(): number;
19575     reloadCurrentLevel(): void;
19576 }
19577 /** winPanel/winds/nextlevel: 下一关 / 返回主菜单 */
19578 @ccclass('OpticalPuzzleWinPanelNextLevelButtonView')
19579 export class OpticalPuzzleWinPanelNextLevelButtonView extends Component {
19580     private _graphics: Graphics | null = null;
19581     private _label: Label | null = null;
19582     private _pressed = false;
19583     private _pressCtrl: HudButtonPressController | null = null;
19584     protected onLoad(): void {
19585         this._hidePlaceholderSplash();
19586         this._label = this.node.getChildByName('label')?.getComponent(Label) ?? null;
19587         this._ensureButton();
19588         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
19589             this._pressed = pressed;
19590             this._redraw();
19591         });
19592         this._redraw();
19593     }
19594     protected onEnable(): void {
19595         this._pressCtrl?.bind();
19596         this.node.on(Button.EventType.CLICK, this._onClick, this);
19597         this._refreshLabelText();
19598         this._redraw();
19599     }
19600     protected onDisable(): void {
19601         this._pressCtrl?.unbind();
19602         this.node.off(Button.EventType.CLICK, this._onClick, this);
19603         this._pressed = false;
19604     }

```

```

19605     protected onDestroy(): void {
19606         this._pressCtrl?.unbind();
19607         this.node.off(Button.EventType.CLICK, this._onClick, this);
19608     }
19609     /** 按关卡 id 设置 label 文案（由通关面板弹出方传入，避免 onEnable 时机不准） */
19610     setLabelForLevel(levelId: number): void {
19611         if (!this._label?.isValid) {
19612             return;
19613         }
19614         const nextId = getNextOpticalLevelId(levelId);
19615         this._label.string = nextId != null ? WIN_NEXT_LEVEL_TEXT : WIN_RETURN_TEXT;
19616     }
19617     refresh(): void {
19618         this._redraw();
19619     }
19620     private _refreshLabelText(): void {
19621         const levelId = this._resolveOpticalPuzzleRoot()?.getCurrentLevelId() ?? 0;
19622         this.setLabelForLevel(levelId);
19623     }
19624     private _hidePlaceholderSplash(): void {
19625         const splash = this.node.getChildByName('SpriteSplash');
19626         if (splash?.isValid) {
19627             splash.active = false;
19628         }
19629     }
19630     private _ensureButton(): void {
19631         let btn = this.getComponent(Button);
19632         if (!btn) {
19633             btn = this.addComponent(Button);
19634             btn.transition = Button.Transition.SCALE;
19635         }
19636         btn.zoomScale = NEXT_LEVEL_BUTTON_ZOOM_SCALE;
19637     }
19638     private _onClick(): void {
19639         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
19640         const gameRoot = this._resolveGameRoot();
19641         const winPanel = resolveWinPanelNode(gameRoot);
19642         if (winPanel?.isValid) {
19643             winPanel.active = false;
19644         }
19645         const puzzleRoot = this._resolveOpticalPuzzleRoot(gameRoot);
19646         const currentId = puzzleRoot?.getCurrentLevelId() ?? 0;
19647         const nextId = getNextOpticalLevelId(currentId);
19648         if (nextId != null) {
19649             // opticalCurrentLevelId 已在通关瞬间写入下一关 id
19650             puzzleRoot?.reloadCurrentLevel();
19651             return;
19652         }
19653         // 最后一关: opticalCurrentLevelId 已在通关瞬间写回首关 id
19654         director.loadScene(SCENE_ENUM.MENU);
19655     }
19656     private _resolveGameRoot(): Node | null {
19657         let node: Node | null = this.node;
19658         while (node?.parent) {
19659             if (node.name === 'GameRoot') {
19660                 return node;
19661             }
19662             node = node.parent;

```

```

19663     }
19664     return null;
19665 }
19666 private _resolveOpticalPuzzleRoot(gameRoot?: Node | null): IOpticalPuzzleRootApi | null {
19667     const root = gameRoot ?? this._resolveGameRoot();
19668     if (!root?.isValid) {
19669         return null;
19670     }
19671     const comp =
19672         root.getComponent('OpticalPuzzleRoot') ??
19673         root.getComponentInChildren('OpticalPuzzleRoot');
19674     return comp as unknown as IOpticalPuzzleRootApi | null;
19675 }
19676 private _ensureGraphics(): Graphics | null {
19677     if (!this._graphics?.isValid) {
19678         this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
19679     }
19680     return this._graphics;
19681 }
19682 private _redraw(): void {
19683     const g = this._ensureGraphics();
19684     const ut = this.getComponent(UITransform);
19685     if (!g || !ut) {
19686         return;
19687     }
19688     g.clear();
19689     const w = ut.width;
19690     const h = ut.height;
19691     const left = -ut.anchorX * w;
19692     const bottom = -ut.anchorY * h;
19693     drawWinPanelNextLevelButtonGlyph(g, left, bottom, w, h, this._pressed);
19694 }
19695 }
19696 /** 自 GameRoot 子树解析 layerOverlay/winPanel/winds/nextlevel */
19697 export function resolveWinPanelNextLevelNode(root: Node | null): Node | null {
19698     return resolveWinPanelWindsNode(root)?.getChildByName('nextlevel') ?? null;
19699 }
19700 /** 为 winds/nextlevel 挂上键帽绘制与点击逻辑 */
19701 export function ensureWinPanelNextLevelButtonView(root: Node | null): void {
19702     const nextLevel = resolveWinPanelNextLevelNode(root);
19703     if (!nextLevel?.isValid) {
19704         return;
19705     }
19706     if (!nextLevel.getComponent(OpticalPuzzleWinPanelNextLevelButtonView)) {
19707         nextLevel.addComponent(OpticalPuzzleWinPanelNextLevelButtonView);
19708     }
19709 }
19710 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelStarGlyph.ts -----
19711 import { Color, Graphics } from 'cc';
19712 import { OpticalStarVisualState } from '../Core/OpticalPuzzleStarRating';
19713 import {
19714     fillGlowLayersMatchingStroke,
19715     strokeGlowLayers,
19716 } from './OpticalPuzzleHudButtonCommon';
19717 import { beamColorFromKey } from './OpticalPuzzleColorUtil';
19718 import {
19719     WIN_STAR_SEGS,
19720     WIN_STAR_VIEW_CX,

```

```

19721     WIN_STAR_VIEW_CY,
19722     WIN_STAR_VIEW_H,
19723     WIN_STAR_VIEW_W,
19724     WinStarPathSeg,
19725 } from './OpticalPuzzleWinPanelStarSegs.generated';
19726 /** 单颗星外轮廓线宽（设计 px，不随节点缩放） */
19727 export const WIN_STAR_BORDER_PX = 4;
19728 /** 左右槽位星相对中间的旋转角（度）：左逆时针、右顺时针 */
19729 export const WIN_STAR_SIDE_ROTATION_DEG = 15;
19730 /** 发光态外缘光晕（比 HUD 按钮更厚） */
19731 const WIN_STAR_GLOW_LAYERS: ReadonlyArray<{ width: number; alpha: number }> = [
19732     { width: 16, alpha: 36 },
19733     { width: 11, alpha: 96 },
19734     { width: 7, alpha: 180 },
19735     { width: 4, alpha: 255 },
19736 ];
19737 const STAR_BORDER = new Color(255, 255, 255, 255);
19738 const STAR_WHITE = new Color(255, 255, 255, 255);
19739 /** 点亮态：与光路 yellow 同色填充 */
19740 const _STAR_YELLOW_BEAM = beamColorFromKey('yellow');
19741 const STAR_YELLOW = new Color(_STAR_YELLOW_BEAM.r, _STAR_YELLOW_BEAM.g, _STAR_YELLOW_BEAM.b,
19742 255);
19743 const STAR_GLOW_RGB = {
19744     r: _STAR_YELLOW_BEAM.r,
19745     g: _STAR_YELLOW_BEAM.g,
19746     b: _STAR_YELLOW_BEAM.b,
19747 };
19748 function rotateAround(
19749     x: number,
19750     y: number,
19751     cx: number,
19752     cy: number,
19753     rad: number,
19754 ): { x: number; y: number } {
19755     if (rad === 0) {
19756         return { x, y };
19757     }
19758     const dx = x - cx;
19759     const dy = y - cy;
19760     const cos = Math.cos(rad);
19761     const sin = Math.sin(rad);
19762     return {
19763         x: cx + dx * cos - dy * sin,
19764         y: cy + dx * sin + dy * cos,
19765     };
19766 }
19767 function mapStarSvgPoint(
19768     svgX: number,
19769     svgY: number,
19770     cx: number,
19771     cy: number,
19772     scale: number,
19773     scaleMul = 1,
19774     rotationRad = 0,
19775 ): { x: number; y: number } {
19776     const s = scale * scaleMul;
19777     const mapped = {
19778         x: cx + (svgX - WIN_STAR_VIEW_CX) * s,

```

```

19779         y: cy - (svgY - WIN_STAR_VIEW_CY) * s,
19780     };
19781     return rotateAround(mapped.x, mapped.y, cx, cy, rotationRad);
19782 }
19783 function traceStarSegs(
19784     g: Graphics,
19785     segs: ReadonlyArray<WinStarPathSeg>,
19786     cx: number,
19787     cy: number,
19788     scale: number,
19789     scaleMul = 1,
19790     rotationRad = 0,
19791 ): void {
19792     for (const seg of segs) {
19793         switch (seg.t) {
19794             case 'M': {
19795                 const p = mapStarSvgPoint(seg.x, seg.y, cx, cy, scale, scaleMul, rotationRad);
19796                 g.moveTo(p.x, p.y);
19797                 break;
19798             }
19799             case 'L': {
19800                 const p = mapStarSvgPoint(seg.x, seg.y, cx, cy, scale, scaleMul, rotationRad);
19801                 g.lineTo(p.x, p.y);
19802                 break;
19803             }
19804             case 'C': {
19805                 const p1 = mapStarSvgPoint(seg.x1, seg.y1, cx, cy, scale, scaleMul, rotationRad);
19806                 const p2 = mapStarSvgPoint(seg.x2, seg.y2, cx, cy, scale, scaleMul, rotationRad);
19807                 const p = mapStarSvgPoint(seg.x, seg.y, cx, cy, scale, scaleMul, rotationRad);
19808                 g.bezierCurveTo(p1.x, p1.y, p2.x, p2.y, p.x, p.y);
19809                 break;
19810             }
19811             case 'Z':
19812                 g.close();
19813                 break;
19814             default:
19815                 break;
19816         }
19817     }
19818 }
19819 function traceStarPath(
19820     g: Graphics,
19821     cx: number,
19822     cy: number,
19823     scale: number,
19824     scaleMul = 1,
19825     rotationRad = 0,
19826 ): void {
19827     traceStarSegs(g, WIN_STAR_SEGS, cx, cy, scale, scaleMul, rotationRad);
19828 }
19829 /** 在矩形区域内绘制单颗五角星（用户 SVG 路径，含圆角） */
19830 export function drawWinPanelStarGlyph(
19831     g: Graphics,
19832     left: number,
19833     bottom: number,
19834     width: number,
19835     height: number,
19836     state: OpticalStarVisualState,

```

```

19837     rotationDeg = 0,
19838     borderPx = WIN_STAR_BORDER_PX,
19839 ): void {
19840     const size = Math.min(width, height);
19841     const cx = left + width * 0.5;
19842     const cy = bottom + height * 0.5;
19843     const scale = (size * 0.5) / Math.max(WIN_STAR_VIEW_W * 0.5, WIN_STAR_VIEW_H * 0.5);
19844     const iconHalf = size * 0.45;
19845     const rotationRad = (rotationDeg * Math.PI) / 180;
19846     const trace = (): void => {
19847         traceStarPath(g, cx, cy, scale, 1, rotationRad);
19848     };
19849     const traceScaled = (mul: number): void => {
19850         traceStarPath(g, cx, cy, scale, mul, rotationRad);
19851     };
19852     if (state === OpticalStarVisualState.Glow) {
19853         fillGlowLayersMatchingStroke(g, size, WIN_STAR_GLOW_LAYERS, iconHalf, traceScaled,
19854 STAR_GLOW_RGB);
19855         g.fillColor = STAR_YELLOW;
19856         trace();
19857         g.fill();
19858         strokeGlowLayers(g, size, WIN_STAR_GLOW_LAYERS, trace, true, STAR_GLOW_RGB);
19859         return;
19860     }
19861     g.fillColor = state === OpticalStarVisualState.Filled ? STAR_WHITE : new Color(0, 0, 0, 0);
19862     trace();
19863     g.fill();
19864     g.strokeColor = STAR_BORDER;
19865     g.lineWidth = borderPx;
19866     g.lineJoin = Graphics.LineJoin.ROUND;
19867     g.lineCap = Graphics.LineCap.ROUND;
19868     trace();
19869     g.stroke();
19870 }
19871 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelStarSegs.generated.ts -----
19872 // AUTO-GENERATED by tools/gen-win-star-segs.js — do not edit by hand
19873 export const WIN_STAR_VIEW_MIN_X = 11.52;
19874 export const WIN_STAR_VIEW_MAX_X = 1013.76;
19875 export const WIN_STAR_VIEW_MIN_Y = 35.84;
19876 export const WIN_STAR_VIEW_MAX_Y = 992;
19877 export const WIN_STAR_VIEW_W = 1002.24;
19878 export const WIN_STAR_VIEW_H = 956.16;
19879 export const WIN_STAR_VIEW_CX = 512.64;
19880 export const WIN_STAR_VIEW_CY = 513.92;
19881 export type WinStarPathSeg =
19882     | { t: 'M'; x: number; y: number }
19883     | { t: 'L'; x: number; y: number }
19884     | { t: 'C'; x1: number; y1: number; x2: number; y2: number; x: number; y: number }
19885     | { t: 'Z' };
19886 /** 用户 SVG 五角星路径（含圆角贝塞尔与弧段） */
19887 export const WIN_STAR_SEGS: ReadonlyArray<WinStarPathSeg> = [
19888     { t: 'M', x: 1004.16, y: 393.6 },
19889     { t: 'C', x1: 994.56, y1: 364.16, x2: 969.6, y2: 343.04, x: 939.52, y: 338.56 },
19890     { t: 'L', x: 694.4, y: 302.72 },
19891     { t: 'L', x: 584.96, y: 80.64 },
19892     { t: 'C', x1: 571.52, y1: 53.12, x2: 544, y2: 35.84, x: 512.64, y: 35.84 },
19893     { t: 'C', x1: 481.28, y1: 35.84, x2: 454.4, y2: 53.12, x: 440.32, y: 80.64 },
19894     { t: 'L', x: 330.88, y: 302.72 },

```

```

19895     { t: 'L', x: 85.76, y: 338.56 },
19896     { t: 'C', x1: 55.04, y1: 343.04, x2: 30.72, y2: 364.16, x: 21.12, y: 393.6 },
19897     { t: 'C', x1: 11.52, y1: 423.04, x2: 19.2, y2: 454.4, x: 41.6, y: 476.16 },
19898     { t: 'L', x: 218.88, y: 648.96 },
19899     { t: 'L', x: 176.64, y: 893.44 },
19900     { t: 'C', x1: 172.548, y1: 916.841, x2: 179.033, y2: 940.853, x: 194.351, y: 959.012 },
19901     { t: 'C', x1: 209.668, y1: 977.17, x2: 232.244, y2: 987.61, x: 256, y: 987.52 },
19902     { t: 'C', x1: 268.8, y1: 987.52, x2: 281.6, y2: 984.32, x: 293.12, y: 977.92 },
19903     { t: 'L', x: 512.64, y: 862.72 },
19904     { t: 'L', x: 732.16, y: 977.92 },
19905     { t: 'C', x1: 759.68, y1: 992, x2: 791.68, y2: 990.08, x: 816.64, y: 972.16 },
19906     { t: 'C', x1: 841.6, y1: 954.24, x2: 853.76, y2: 924.16, x: 848.64, y: 893.44 },
19907     { t: 'L', x: 806.4, y: 648.96 },
19908     { t: 'L', x: 983.68, y: 476.16 },
19909     { t: 'C', x1: 1005.44, y1: 454.4, x2: 1013.76, y2: 423.04, x: 1004.16, y: 393.6 },
19910     { t: 'Z' },
19911 ];
19912 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelStarsView.ts -----
19913 import { _decorator, Component, Graphics, Node, UITransform } from 'cc';
19914 import { getOpticalLevelById } from '../Config/OpticalPuzzleLevels';
19915 import {
19916     defaultDimStarStates,
19917     type OpticalStarSlotStates,
19918     resolveStarSlotStates,
19919 } from '../Core/OpticalPuzzleStarRating';
19920 import type { OpticalSnapshotNotify } from '../Application/OpticalPuzzleSession';
19921 import { EVENT_ENUM } from '../Utils/Enum';
19922 import { OPTICAL_PUZZLE } from '../Utils/Event';
19923 import {
19924     drawWinPanelStarGlyph,
19925     WIN_STAR_SIDE_ROTATION_DEG,
19926 } from './OpticalPuzzleWinPanelStarGlyph';
19927 import { resolveWinPanelWindsNode } from './OpticalPuzzleWinPanelWindsView';
19928 const { ccclass } = _decorator;
19929 /** 单颗星设计边长 */
19930 export const WIN_STAR_DESIGN_SIZE = 80;
19931 @ccclass('OpticalPuzzleWinPanelStarsView')
19932 export class OpticalPuzzleWinPanelStarsView extends Component {
19933     private _graphics: Graphics | null = null;
19934     private _slotStates: OpticalStarSlotStates = defaultDimStarStates();
19935     protected onLoad(): void {
19936         this._hidePlaceholderSplash();
19937         OPTICAL_PUZZLE.on(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, this._onSnapshotChanged, this);
19938         this._redraw();
19939     }
19940     protected onEnable(): void {
19941         this._redraw();
19942     }
19943     protected onDestroy(): void {
19944         OPTICAL_PUZZLE.off(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, this._onSnapshotChanged, this);
19945     }
19946     /** 直接设置三颗星状态（调试用） */
19947     setSlotStates(states: OpticalStarSlotStates): void {
19948         this._slotStates = states;
19949         this._redraw();
19950     }
19951     refresh(): void {
19952         this._redraw();

```



```

19953     }
19954     private _hidePlaceholderSplash(): void {
19955         const splash = this.node.getChildByName('SpriteSplash');
19956         if (splash?.isValid) {
19957             splash.active = false;
19958         }
19959     }
19960     private _onSnapshotChanged(payload: OpticalSnapshotNotify): void {
19961         const reason = payload?.notifyReason;
19962         if (reason === 'load' || reason === 'reset') {
19963             this._slotStates = defaultDimStarStates();
19964             this._redraw();
19965             return;
19966         }
19967         if (reason !== 'complete') {
19968             return;
19969         }
19970         const levelId = payload.snapshot?.levelId ?? 0;
19971         const level = getOpticalLevelById(levelId);
19972         if (!level?.starThresholds) {
19973             this._slotStates = defaultDimStarStates();
19974             this._redraw();
19975             return;
19976         }
19977         this._slotStates = resolveStarSlotStates(payload.moveCount, level.starThresholds);
19978         this._redraw();
19979     }
19980     private _ensureGraphics(): Graphics | null {
19981         if (!this._graphics?.isValid) {
19982             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
19983         }
19984         return this._graphics;
19985     }
19986     private _redraw(): void {
19987         const g = this._ensureGraphics();
19988         const ut = this.getComponent(UITransform);
19989         if (!g || !ut) {
19990             return;
19991         }
19992         g.clear();
19993         const w = ut.width;
19994         const h = ut.height;
19995         const left = -ut.anchorX * w;
19996         const bottom = -ut.anchorY * h;
19997         const half = WIN_STAR_DESIGN_SIZE * 0.5;
19998         const slots: ReadonlyArray<{ cx: number; cy: number; rotationDeg: number }> = [
19999             { cx: left + half, cy: bottom + h * 0.5, rotationDeg: WIN_STAR_SIDE_ROTATION_DEG },
20000             { cx: left + w * 0.5, cy: bottom + h - half, rotationDeg: 0 },
20001             { cx: left + w - half, cy: bottom + h * 0.5, rotationDeg: -WIN_STAR_SIDE_ROTATION_DEG },
20002         ];
20003         for (let i = 0; i < slots.length; i++) {
20004             const { cx, cy, rotationDeg } = slots[i];
20005             drawWinPanelStarGlyph(
20006                 g,
20007                 cx - half,
20008                 cy - half,
20009                 WIN_STAR_DESIGN_SIZE,
20010                 WIN_STAR_DESIGN_SIZE,

```

```

20011         this._slotStates[i],
20012         rotationDeg,
20013     );
20014     }
20015 }
20016 }
20017 export function resolveWinPanelStarsNode(root: Node | null): Node | null {
20018     return resolveWinPanelWindsNode(root)?.getChildByName('stars') ?? null;
20019 }
20020 export function ensureWinPanelStarsView(root: Node | null): void {
20021     const stars = resolveWinPanelStarsNode(root);
20022     if (!stars?.isValid) {
20023         return;
20024     }
20025     if (!stars.getComponent(OpticalPuzzleWinPanelStarsView)) {
20026         stars.addComponent(OpticalPuzzleWinPanelStarsView);
20027     }
20028 }
20029 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelStepView.ts -----
20030 import { Node } from 'cc';
20031 import { ensureStepCountView } from './OpticalPuzzleStepCountView';
20032 import { ensureStepIconView } from './OpticalPuzzleStepIconView';
20033 import { resolveWinPanelWindsNode } from './OpticalPuzzleWinPanelWindsView';
20034 /** 自 GameRoot 子树解析 layerOverlay/winPanel/winds/step */
20035 export function resolveWinPanelStepNode(root: Node | null): Node | null {
20036     return resolveWinPanelWindsNode(root)?.getChildByName('step') ?? null;
20037 }
20038 function _hideSpriteSplash(node: Node | null): void {
20039     const splash = node?.getChildByName('SpriteSplash');
20040     if (splash?.isValid) {
20041         splash.active = false;
20042     }
20043 }
20044 /**
20045  * winPanel/winds/step: 不绘制外框, 仅挂 StepIcon 爪印与 StepCount 步数同步 (逻辑同 TopBar/Step)。
20046  */
20047 export function ensureWinPanelStepViews(root: Node | null): void {
20048     const step = resolveWinPanelStepNode(root);
20049     if (!step?.isValid) {
20050         return;
20051     }
20052     _hideSpriteSplash(step);
20053     const stepIcon = step.getChildByName('StepIcon');
20054     _hideSpriteSplash(stepIcon);
20055     ensureStepIconView(stepIcon);
20056     const stepCount = step.getChildByName('StepCount');
20057     _hideSpriteSplash(stepCount);
20058     ensureStepCountView(stepCount);
20059 }
20060 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelTitleGlyph.ts -----
20061 import { Color, Graphics } from 'cc';
20062 import { unlitHudIconColor } from './OpticalPuzzleHudButtonCommon';
20063 /** 标题横条描边 (设计 px, 不随节点缩放) */
20064 export const WIN_TITLE_BORDER_PX = 4;
20065 const WIN_TITLE_BORDER = new Color(255, 255, 255, 255);
20066 /** SVG viewBox 1024; 仅取上半部横条 (中条 + 左右缎带, 忽略绿圆与勾) */
20067 const WIN_TITLE_VIEW_MIN_X = 94.208;
20068 const WIN_TITLE_VIEW_MAX_X = 896;

```

```

20069 const WIN_TITLE_VIEW_MIN_Y = 1.536;
20070 const WIN_TITLE_VIEW_MAX_Y = 218.624;
20071 const WIN_TITLE_VIEW_W = WIN_TITLE_VIEW_MAX_X - WIN_TITLE_VIEW_MIN_X;
20072 const WIN_TITLE_VIEW_H = WIN_TITLE_VIEW_MAX_Y - WIN_TITLE_VIEW_MIN_Y;
20073 const WIN_TITLE_VIEW_CX = (WIN_TITLE_VIEW_MIN_X + WIN_TITLE_VIEW_MAX_X) * 0.5;
20074 const WIN_TITLE_VIEW_CY = (WIN_TITLE_VIEW_MIN_Y + WIN_TITLE_VIEW_MAX_Y) * 0.5;
20075 type PathSeg =
20076     | { t: 'M'; x: number; y: number }
20077     | { t: 'L'; x: number; y: number }
20078     | { t: 'Z' };
20079 /** 中条: M216.064 1.536h552.96v205.312h-552.96V1.536z */
20080 const WIN_TITLE_CENTER_SEGS: ReadonlyArray<PathSeg> = [
20081     { t: 'M', x: 216.064, y: 1.536 },
20082     { t: 'L', x: 769.024, y: 1.536 },
20083     { t: 'L', x: 769.024, y: 206.848 },
20084     { t: 'L', x: 216.064, y: 206.848 },
20085     { t: 'Z' },
20086 ];
20087 /** 左缎带: M94.208 218.624h190.464V13.312H94.208l52.736 102.4-52.736 102.912z */
20088 const WIN_TITLE_LEFT_SEGS: ReadonlyArray<PathSeg> = [
20089     { t: 'M', x: 94.208, y: 218.624 },
20090     { t: 'L', x: 284.672, y: 218.624 },
20091     { t: 'L', x: 284.672, y: 13.312 },
20092     { t: 'L', x: 94.208, y: 13.312 },
20093     { t: 'L', x: 146.944, y: 115.712 },
20094     { t: 'Z' },
20095 ];
20096 /** 右缎带: M896 218.624h-198.656V13.312H896l-55.296 102.4L896 218.624z */
20097 const WIN_TITLE_RIGHT_SEGS: ReadonlyArray<PathSeg> = [
20098     { t: 'M', x: 896, y: 218.624 },
20099     { t: 'L', x: 697.344, y: 218.624 },
20100     { t: 'L', x: 697.344, y: 13.312 },
20101     { t: 'L', x: 896, y: 13.312 },
20102     { t: 'L', x: 840.704, y: 115.712 },
20103     { t: 'Z' },
20104 ];
20105 const WIN_TITLE_PARTS: ReadonlyArray<ReadonlyArray<PathSeg>> = [
20106     WIN_TITLE_CENTER_SEGS,
20107     WIN_TITLE_LEFT_SEGS,
20108     WIN_TITLE_RIGHT_SEGS,
20109 ];
20110 /** 三块并集的外轮廓 (顺时针); 描边仅用此路径, 避免交叠内线 */
20111 const WIN_TITLE_OUTLINE_SEGS: ReadonlyArray<PathSeg> = [
20112     { t: 'M', x: 94.208, y: 218.624 },
20113     { t: 'L', x: 146.944, y: 115.712 },
20114     { t: 'L', x: 94.208, y: 13.312 },
20115     { t: 'L', x: 216.064, y: 13.312 },
20116     { t: 'L', x: 216.064, y: 1.536 },
20117     { t: 'L', x: 769.024, y: 1.536 },
20118     { t: 'L', x: 769.024, y: 13.312 },
20119     { t: 'L', x: 896, y: 13.312 },
20120     { t: 'L', x: 840.704, y: 115.712 },
20121     { t: 'L', x: 896, y: 218.624 },
20122     { t: 'L', x: 697.344, y: 218.624 },
20123     { t: 'L', x: 697.344, y: 206.848 },
20124     { t: 'L', x: 284.672, y: 206.848 },
20125     { t: 'L', x: 284.672, y: 218.624 },
20126     { t: 'L', x: 94.208, y: 218.624 },

```

```

20127     { t: 'Z' },
20128 ];
20129 function mapTitleSvgPoint(svgX: number, svgY: number, cx: number, cy: number, scale: number): { x: number; y:
20130 number } {
20131     return {
20132         x: cx + (svgX - WIN_TITLE_VIEW_CX) * scale,
20133         y: cy - (svgY - WIN_TITLE_VIEW_CY) * scale,
20134     };
20135 }
20136 function traceTitleSegs(
20137     g: Graphics,
20138     segs: ReadonlyArray<PathSeg>,
20139     cx: number,
20140     cy: number,
20141     scale: number,
20142 ): void {
20143     for (const seg of segs) {
20144         switch (seg.t) {
20145             case 'M': {
20146                 const p = mapTitleSvgPoint(seg.x, seg.y, cx, cy, scale);
20147                 g.moveTo(p.x, p.y);
20148                 break;
20149             }
20150             case 'L': {
20151                 const p = mapTitleSvgPoint(seg.x, seg.y, cx, cy, scale);
20152                 g.lineTo(p.x, p.y);
20153                 break;
20154             }
20155             case 'Z':
20156                 g.close();
20157                 break;
20158             default:
20159                 break;
20160         }
20161     }
20162 }
20163 function traceTitleBanner(g: Graphics, cx: number, cy: number, scale: number): void {
20164     for (const part of WIN_TITLE_PARTS) {
20165         traceTitleSegs(g, part, cx, cy, scale);
20166     }
20167 }
20168 /** 绘制 winds/title 通关标题横条：与四向键 icon 未按填充色一致 + 3px 纯白描边 */
20169 export function drawWinPanelTitleGlyph(
20170     g: Graphics,
20171     left: number,
20172     bottom: number,
20173     width: number,
20174     height: number,
20175 ): void {
20176     const cx = left + width * 0.5;
20177     const cy = bottom + height * 0.5;
20178     const scale = Math.min(width / WIN_TITLE_VIEW_W, height / WIN_TITLE_VIEW_H);
20179     g.fillColor = unlitHudIconColor();
20180     traceTitleBanner(g, cx, cy, scale);
20181     g.fill();
20182     g.strokeColor = WIN_TITLE_BORDER;
20183     g.lineWidth = WIN_TITLE_BORDER_PX;
20184     g.lineJoin = Graphics.LineJoin.ROUND;

```

```

20185         g.lineCap = Graphics.LineCap.ROUND;
20186         traceTitleSegs(g, WIN_TITLE_OUTLINE_SEGS, cx, cy, scale);
20187         g.stroke();
20188     }
20189     // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelTitleView.ts -----
20190     import { _decorator, Component, Graphics, Label, Node, UITransform } from 'cc';
20191     import { getOpticalLevelById } from '../Config/OpticalPuzzleLevels';
20192     import type { OpticalSnapshotNotify } from '../Application/OpticalPuzzleSession';
20193     import { isPerfectClear } from '../Core/OpticalPuzzleStarRating';
20194     import { EVENT_ENUM } from '../Utils/Enum';
20195     import { OPTICAL_PUZZLE } from '../Utils/Event';
20196     import { drawWinPanelTitleGlyph } from './OpticalPuzzleWinPanelTitleGlyph';
20197     import { resolveWinPanelWindsNode } from './OpticalPuzzleWinPanelWindsView';
20198     const { ccclass } = _decorator;
20199     /** 通关标题文案（winds/title/label） */
20200     const WIN_TITLE_TEXT_SUCCESS = '成功点亮';
20201     const WIN_TITLE_TEXT_PERFECT = '完美点亮';
20202     /** winPanel/winds/title: 通关标题横条（SVG 上半部造型）+ 标题文案 */
20203     @ccclass('OpticalPuzzleWinPanelTitleView')
20204     export class OpticalPuzzleWinPanelTitleView extends Component {
20205         private _graphics: Graphics | null = null;
20206         private _label: Label | null = null;
20207         protected onLoad(): void {
20208             this._hidePlaceholderSplash();
20209             this._label = this.node.getChildByName('label')?.getComponent(Label) ?? null;
20210             OPTICAL_PUZZLE.on(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, this._onSnapshotChanged, this);
20211             this._setTitleText(WIN_TITLE_TEXT_SUCCESS);
20212             this._redraw();
20213         }
20214         protected onEnable(): void {
20215             this._redraw();
20216         }
20217         protected onDestroy(): void {
20218             OPTICAL_PUZZLE.off(EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED, this._onSnapshotChanged, this);
20219         }
20220         refresh(): void {
20221             this._redraw();
20222         }
20223         private _hidePlaceholderSplash(): void {
20224             const splash = this.node.getChildByName('SpriteSplash');
20225             if (splash?.isValid) {
20226                 splash.active = false;
20227             }
20228         }
20229         private _onSnapshotChanged(payload: OpticalSnapshotNotify): void {
20230             const reason = payload?.notifyReason;
20231             if (reason === 'load' || reason === 'reset') {
20232                 this._setTitleText(WIN_TITLE_TEXT_SUCCESS);
20233                 return;
20234             }
20235             if (reason !== 'complete') {
20236                 return;
20237             }
20238             const levelId = payload.snapshot?.levelId ?? 0;
20239             const level = getOpticalLevelById(levelId);
20240             const thresholds = level?.starThresholds;
20241             if (!thresholds) {
20242                 this._setTitleText(WIN_TITLE_TEXT_SUCCESS);

```

```

20243         return;
20244     }
20245     const text = isPerfectClear(payload.moveCount, thresholds)
20246         ? WIN_TITLE_TEXT_PERFECT
20247         : WIN_TITLE_TEXT_SUCCESS;
20248     this._setTitleText(text);
20249 }
20250 private _setTitleText(text: string): void {
20251     if (!this._label?.isValid) {
20252         return;
20253     }
20254     this._label.string = text;
20255 }
20256 private _ensureGraphics(): Graphics | null {
20257     if (!this._graphics?.isValid) {
20258         this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
20259     }
20260     return this._graphics;
20261 }
20262 private _redraw(): void {
20263     const g = this._ensureGraphics();
20264     const ut = this.getComponent(UITransform);
20265     if (!g || !ut) {
20266         return;
20267     }
20268     g.clear();
20269     const w = ut.width;
20270     const h = ut.height;
20271     const left = -ut.anchorX * w;
20272     const bottom = -ut.anchorY * h;
20273     drawWinPanelTitleGlyph(g, left, bottom, w, h);
20274 }
20275 }
20276 /** 自 GameRoot 子树解析 layerOverlay/winPanel/winds/title */
20277 export function resolveWinPanelTitleNode(root: Node | null): Node | null {
20278     return resolveWinPanelWindsNode(root)?.getChildByName('title') ?? null;
20279 }
20280 /** 为 winds/title 挂上标题横条绘制 */
20281 export function ensureWinPanelTitleView(root: Node | null): void {
20282     const title = resolveWinPanelTitleNode(root);
20283     if (!title?.isValid) {
20284         return;
20285     }
20286     if (!title.getComponent(OpticalPuzzleWinPanelTitleView)) {
20287         title.addComponent(OpticalPuzzleWinPanelTitleView);
20288     }
20289 }
20290 // ----- assets/Scripts/Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelWindsView.ts -----
20291 import { _decorator, Color, Component, Graphics, Node, UITransform } from 'cc';
20292 import { HUD_KEY_FILL } from './OpticalPuzzleHudButtonCommon';
20293 const { ccclass } = _decorator;
20294 /** 胜利面板内容区圆角（设计 px，不随节点缩放） */
20295 const WIN_WINDS_CORNER_PX = 20;
20296 /** 胜利面板内容区白边线宽（设计 px） */
20297 const WIN_WINDS_BORDER_PX = 5;
20298 const WIN_WINDS_BORDER_COLOR = new Color(255, 255, 255, 255);
20299 /** 绘制 winPanel/winds: 键帽底色 + 纯白圆角描边 */
20300 export function drawWinPanelWindsChrome{

```

```

20301         g: Graphics,
20302         left: number,
20303         bottom: number,
20304         width: number,
20305         height: number,
20306     ): void {
20307         g.fillColor = HUD_KEY_FILL;
20308         g.roundRect(left, bottom, width, height, WIN_WINDS_CORNER_PX);
20309         g.fill();
20310         g.strokeColor = WIN_WINDS_BORDER_COLOR;
20311         g.lineWidth = WIN_WINDS_BORDER_PX;
20312         g.roundRect(left, bottom, width, height, WIN_WINDS_CORNER_PX);
20313         g.stroke();
20314     }
20315     /** winPanel/winds: 圆角面板底图（非按钮） */
20316     @ccclass('OpticalPuzzleWinPanelWindsView')
20317     export class OpticalPuzzleWinPanelWindsView extends Component {
20318         private _graphics: Graphics | null = null;
20319         protected onLoad(): void {
20320             this._hidePlaceholderSplash();
20321             this._redraw();
20322         }
20323         protected onEnable(): void {
20324             this._redraw();
20325         }
20326         refresh(): void {
20327             this._redraw();
20328         }
20329         private _hidePlaceholderSplash(): void {
20330             const splash = this.node.getChildByName('SpriteSplash');
20331             if (splash?.isValid) {
20332                 splash.active = false;
20333             }
20334         }
20335         private _ensureGraphics(): Graphics | null {
20336             if (!this._graphics?.isValid) {
20337                 this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
20338             }
20339             return this._graphics;
20340         }
20341         private _redraw(): void {
20342             const g = this._ensureGraphics();
20343             const ut = this.getComponent(UITransform);
20344             if (!g || !ut) {
20345                 return;
20346             }
20347             g.clear();
20348             const w = ut.width;
20349             const h = ut.height;
20350             const left = -ut.anchorX * w;
20351             const bottom = -ut.anchorY * h;
20352             drawWinPanelWindsChrome(g, left, bottom, w, h);
20353         }
20354     }
20355     /** 自 GameRoot 子树解析 layerOverlay/winPanel */
20356     export function resolveWinPanelNode(root: Node | null): Node | null {
20357         return root?.getChildByName('layerOverlay')?.getChildByName('winPanel') ?? null;
20358     }

```

```

20359 /** 自 GameRoot 子树解析 layerOverlay/preWinPanel（通关前全屏 BlockInputEvents 挡操作） */
20360 export function resolvePreWinPanelNode(root: Node | null): Node | null {
20361     return root?.getChildByName('layerOverlay')?.getChildByName('preWinPanel') ?? null;
20362 }
20363 /** 自 GameRoot 子树解析 layerOverlay/winPanel/winds */
20364 export function resolveWinPanelWindsNode(root: Node | null): Node | null {
20365     return resolveWinPanelNode(root)?.getChildByName('winds') ?? null;
20366 }
20367 /** 为 winPanel/winds 挂上面板底图绘制 */
20368 export function ensureWinPanelWindsView(root: Node | null): void {
20369     const winds = resolveWinPanelWindsNode(root);
20370     if (!winds?.isValid) {
20371         return;
20372     }
20373     if (!winds.getComponent(OpticalPuzzleWinPanelWindsView)) {
20374         winds.addComponent(OpticalPuzzleWinPanelWindsView);
20375     }
20376 }
20377 // ----- assets/Scripts/Manager/DataManager.ts -----
20378 import { sys } from 'cc';
20379 import { MOCK_PLAYER_DATA, USE_DEBUG MOCK_SAVE } from '../Config/DebugMockSave';
20380 import {
20381     getFirstOpticalLevelId,
20382     getNextOpticalLevelId,
20383 } from '../Games/OpticalPuzzle/Config/OpticalPuzzleLevels';
20384 import {
20385     ensureDefaultFirstLevelUnlock,
20386     getOpticalLevelBestStepsFromFlat,
20387     isOpticalLevelClearedInFlat,
20388     isOpticalLevelUnlockedInFlat,
20389     markNextOpticalLevelUnlockedIfAbsent,
20390     mergeOpticalLevelClears,
20391     mergeOpticalLevelClearsFromPlayerJsonText,
20392     OPTICAL_LEVEL_UNLOCK_PLACEHOLDER_STEPS,
20393     OpticalLevelClearsFlat,
20394     resolveOpticalMaxUnlockedLevelId,
20395     upsertOpticalLevelBestSteps,
20396 } from '../Games/OpticalPuzzle/Infrastructure/OpticalPlayerPersist';
20397 import { GAME_STATE_ENUM } from '../Utils/Enum';
20398 export type { OpticalLevelClearsFlat } from '../Games/OpticalPuzzle/Infrastructure/OpticalPlayerPersist';
20399 const PLAYER_DATA_STORAGE_KEY = 'LightPuzzle_player_v1';
20400 /** 旧版字符串关卡 id → 现用整数 id（读档兼容） */
20401 const LEGACY_OPTICAL_LEVEL_ID: Readonly<Record<string, number>> = {
20402     dev_minimal: 0,
20403     lvl_01_tutorial: 1,
20404     lvl_02_cross: 2,
20405     lvl_03_corridor: 3,
20406     lvl_04_gate: 4,
20407     lvl_05_source_target: 5,
20408     lvl_06_dual_targets: 6,
20409 };
20410 /** 持久化数据结构（可按玩法扩展字段） */
20411 export interface IPlayerPersistData {
20412     /**
20413      * 继续游戏 / 上次进度关卡 id。
20414      * 写入时机：选关选中、关卡通关瞬间（有下一关→下一关 id，最后一关→首关 id）。
20415      * 与解锁无关；解锁见 opticalLevelClears 中是否出现该 levelId。
20416      */

```



```

20417     opticalCurrentLevelId: number;
20418     /**
20419      * 关卡记录: `[levelId, steps, ...]` 扁平 number 数组。
20420      * steps 为真实最少步数; 仅解锁未通时为 OPTICAL_LEVEL_UNLOCK_PLACEHOLDER_STEPS。
20421      */
20422     opticalLevelClears: OpticalLevelClearsFlat;
20423     bgmOn: boolean;
20424     sfxOn: boolean;
20425 }
20426 /** 新玩家默认存档（无本地档时使用） */
20427 function cloneDefaultData(): IPlayerPersistData {
20428     return {
20429         opticalCurrentLevelId: DEFAULT_DATA.opticalCurrentLevelId,
20430         opticalLevelClears: [...DEFAULT_DATA.opticalLevelClears],
20431         bgmOn: DEFAULT_DATA.bgmOn,
20432         sfxOn: DEFAULT_DATA.sfxOn,
20433     };
20434 }
20435 /** 微信端或手动开启时打印存档诊断（Console 搜 [DataManager]） */
20436 function shouldLogSaveDebug(): boolean {
20437     return sys.platform === sys.Platform.WECHAT_GAME
20438         || !(globalThis as { LIGHT_PUZZLE_SAVE_DEBUG?: boolean }).LIGHT_PUZZLE_SAVE_DEBUG;
20439 }
20440 function logSaveDebug(message: string, detail?: unknown): void {
20441     if (!shouldLogSaveDebug()) {
20442         return;
20443     }
20444     if (detail === undefined) {
20445         console.log(`[DataManager] ${message}`);
20446         return;
20447     }
20448     console.log(`[DataManager] ${message}`, detail);
20449 }
20450 function summarizeClears(clears: OpticalLevelClearsFlat): string {
20451     const normalized = mergeOpticalLevelClears(clears);
20452     const pairs: string[] = [];
20453     for (let i = 0; i + 1 < normalized.length; i += 2) {
20454         pairs.push(`${normalized[i]}:${normalized[i + 1]}`);
20455     }
20456     return pairs.length > 0 ? pairs.join(',') : '(empty)';
20457 }
20458 /** 微信小游戏仅用 sys.localStorage（wx 存储）; 浏览器预览才双读 window */
20459 function isWeChatMiniGameStorage(): boolean {
20460     return sys.platform === sys.Platform.WECHAT_GAME;
20461 }
20462 function readRawStorageValue(key: string): unknown {
20463     if (isWeChatMiniGameStorage()) {
20464         try {
20465             return sys.localStorage.getItem(key);
20466         } catch (e) {
20467             logSaveDebug('wechat getItem failed', e);
20468             return null;
20469         }
20470     }
20471     let fromSys: unknown = null;
20472     let fromGlobal: unknown = null;
20473     try {
20474         fromSys = sys.localStorage.getItem(key);

```

```
20475         if (fromSys === '' || fromSys === undefined) {
20476             fromSys = null;
20477         }
20478     } catch {
20479         /* ignore */
20480     }
20481     try {
20482         const gl = (globalThis as { localStorage?: Storage }).localStorage;
20483         if (gl && typeof gl.getItem === 'function') {
20484             fromGlobal = gl.getItem(key);
20485             if (fromGlobal === '' || fromGlobal === undefined) {
20486                 fromGlobal = null;
20487             }
20488         }
20489     } catch {
20490         /* ignore */
20491     }
20492     if (fromSys != null && fromGlobal != null && fromSys !== fromGlobal) {
20493         console.warn('[DataManager] sys 与 window.localStorage 同一 key 不一致, 使用 window 中的值');
20494     };
20495     return fromGlobal;
20496 }
20497 return fromGlobal ?? fromSys;
20498 }
20499 /**
20500  * 解析本地档: 支持 JSON 字符串、微信已反序列化的 object、BOM、URI 编码字符串。
20501  */
20502 function parseStoragePayload(raw: unknown): Record<string, unknown> | null {
20503     if (raw == null || raw === '') {
20504         return null;
20505     }
20506     if (typeof raw === 'object' && !Array.isArray(raw)) {
20507         return raw as Record<string, unknown>;
20508     }
20509     if (typeof raw !== 'string') {
20510         logSaveDebug('storage raw 类型异常', typeof raw);
20511         return null;
20512     }
20513     let text = raw.trim().replace(/\uFEFF/, '');
20514     if (!text) {
20515         return null;
20516     }
20517     const tryParse = (s: string): Record<string, unknown> | null => {
20518         try {
20519             const parsed = JSON.parse(s);
20520             if (parsed && typeof parsed === 'object' && !Array.isArray(parsed)) {
20521                 return parsed as Record<string, unknown>;
20522             }
20523             logSaveDebug('JSON 根节点不是 object', parsed);
20524         } catch (e) {
20525             logSaveDebug('JSON.parse 失败', { head: s.slice(0, 160), error: e });
20526         }
20527         return null;
20528     };
20529     const direct = tryParse(text);
20530     if (direct) {
20531         return direct;
20532     }
```

```
20533     if (/^%7B|^%22/i.test(text)) {
20534         try {
20535             const decoded = decodeURIComponent(text);
20536             const fromUri = tryParse(decoded);
20537             if (fromUri) {
20538                 logSaveDebug('storage 经 URI 解码后读档成功');
20539                 return fromUri;
20540             }
20541         } catch (e) {
20542             logSaveDebug('URI 解码失败', e);
20543         }
20544     }
20545     return null;
20546 }
20547 function readStoragePayload(key: string): Record<string, unknown> | null {
20548     const raw = readRawStorageValue(key);
20549     logSaveDebug('read raw', {
20550         type: typeof raw,
20551         preview: typeof raw === 'string' ? raw.slice(0, 160) : raw,
20552     });
20553     return parseStoragePayload(raw);
20554 }
20555 /** 新玩家默认存档字段（无本地档时使用） */
20556 const DEFAULT_DATA: IPlayerPersistData = {
20557     opticalCurrentLevelId: 1,
20558     opticalLevelClears: [1, OPTICAL_LEVEL_UNLOCK_PLACEHOLDER_STEPS],
20559     bgmOn: true,
20560     sfxOn: true,
20561 };
20562 function setStorageItem(key: string, value: string): void {
20563     if (isWeChatMiniGameStorage()) {
20564         try {
20565             sys.localStorage.setItem(key, value);
20566             logSaveDebug('save wechat ok', {
20567                 bytes: value.length,
20568                 clears: value.includes('opticalLevelClears') ? 'present' : 'missing',
20569             });
20570         } catch (e) {
20571             console.warn('[DataManager] save failed (wechat)', e);
20572         }
20573         return;
20574     }
20575     try {
20576         sys.localStorage.setItem(key, value);
20577     } catch {
20578         /* ignore */
20579     }
20580     try {
20581         const gl = (globalThis as { localStorage?: Storage }).localStorage;
20582         if (gl && typeof gl.setItem === 'function') {
20583             gl.setItem(key, value);
20584         }
20585     } catch {
20586         /* ignore */
20587     }
20588 }
20589 function mergeOpticalCurrentLevelId(raw: unknown): number {
20590     if (typeof raw === 'number' && Number.isFinite(raw)) {
```

```

20591         return Math.trunc(raw);
20592     }
20593     if (typeof raw === 'string') {
20594         const trimmed = raw.trim();
20595         const legacy = LEGACY_OPTICAL_LEVEL_ID[trimmed];
20596         if (legacy !== undefined) {
20597             return legacy;
20598         }
20599         const n = Number.parseInt(trimmed, 10);
20600         if (Number.isFinite(n)) {
20601             return n;
20602         }
20603     }
20604     return DEFAULT_DATA.opticalCurrentLevelId;
20605 }
20606 function mergePlayerData(raw: unknown, storageJsonText?: string | null): IPlayerPersistData {
20607     if (!raw || typeof raw !== 'object') {
20608         return cloneDefaultData();
20609     }
20610     const o = raw as Record<string, unknown>;
20611     const opticalCurrentLevelId = mergeOpticalCurrentLevelId(o.opticalCurrentLevelId);
20612     let opticalLevelClears = mergeOpticalLevelClears(o.opticalLevelClears);
20613     if (opticalLevelClears.length === 0 && storageJsonText) {
20614         const fromText = mergeOpticalLevelClearsFromPlayerJsonText(storageJsonText);
20615         if (fromText.length > 0) {
20616             opticalLevelClears = fromText;
20617             logSaveDebug('clears recovered from storage JSON text', summarizeClears(fromText));
20618         }
20619     }
20620     const merged: IPlayerPersistData = {
20621         opticalCurrentLevelId,
20622         opticalLevelClears: ensureDefaultFirstLevelUnlock(opticalLevelClears),
20623         bgmOn: typeof o.bgmOn === 'boolean' ? o.bgmOn : DEFAULT_DATA.bgmOn,
20624         sfxOn: typeof o.sfxOn === 'boolean' ? o.sfxOn : DEFAULT_DATA.sfxOn,
20625     };
20626     logSaveDebug('mergePlayerData', {
20627         currentLevelId: merged.opticalCurrentLevelId,
20628         clears: summarizeClears(merged.opticalLevelClears),
20629         rawClearsType: typeof o.opticalLevelClears,
20630         rawClearsPreview: typeof o.opticalLevelClears === 'string'
20631             ? (o.opticalLevelClears as string).slice(0, 120)
20632             : o.opticalLevelClears,
20633         rawClearsLength: Array.isArray(o.opticalLevelClears)
20634             ? o.opticalLevelClears.length
20635             : (o.opticalLevelClears as { length?: number } | null)?.length,
20636     });
20637     return merged;
20638 }
20639 /**
20640  * 全局数据（单例）。与 project-rules 一致：重要字段经 setter 可触发 save。
20641  */
20642 export class DataManager {
20643     private static _instance: DataManager | null = null;
20644     /** USE_DEBUG MOCK_SAVE 时仅第一次 init 写入 mock */
20645     private static _debugMockSeededThisRun = false;
20646     /** 是否已成功从本地读过至少一次（避免微信端二次 restore 读空覆盖内存） */
20647     private _everRestoredFromDisk = false;
20648     static get instance(): DataManager {

```

```

20649         if (this._instance === null) {
20650             this._instance = new DataManager();
20651         }
20652         return this._instance;
20653     }
20654     /** 与规则示例一致：全局游戏状态 */
20655     gameStatus: GAME_STATE_ENUM = GAME_STATE_ENUM.INIT;
20656     private _data: IPlayerPersistData = cloneDefaultData();
20657     /** 诊断：当前内存中通关记录条数 */
20658     getOpticalLevelClearPairCount(): number {
20659         return Math.floor(mergeOpticalLevelClears(this._data.opticalLevelClears).length / 2);
20660     }
20661     get opticalCurrentLevelId(): number {
20662         return this._data.opticalCurrentLevelId;
20663     }
20664     set opticalCurrentLevelId(id: number) {
20665         if (this._data.opticalCurrentLevelId === id) {
20666             return;
20667         }
20668         this._data.opticalCurrentLevelId = id;
20669         this.save();
20670     }
20671     get bgmOn(): boolean {
20672         return this._data.bgmOn;
20673     }
20674     set bgmOn(v: boolean) {
20675         this._data.bgmOn = v;
20676         this.save();
20677     }
20678     get sfxOn(): boolean {
20679         return this._data.sfxOn;
20680     }
20681     set sfxOn(v: boolean) {
20682         this._data.sfxOn = v;
20683         this.save();
20684     }
20685     reset(): void {
20686         this._data = cloneDefaultData();
20687         this.gameStatus = GAME_STATE_ENUM.INIT;
20688         this.save();
20689     }
20690     /** @deprecated 诊断用：clears 中最大 levelId；选关请用 isOpticalLevelUnlocked */
20691     getOpticalMaxUnlockedLevelId(): number {
20692         return resolveOpticalMaxUnlockedLevelId(
20693             this._data.opticalCurrentLevelId,
20694             this._data.opticalLevelClears,
20695         );
20696     }
20697     /** 该关是否在 clears 中有记录（含仅解锁占位 999999） */
20698     isOpticalLevelUnlocked(levelId: number): boolean {
20699         return isOpticalLevelUnlockedInFlat(this._data.opticalLevelClears, levelId);
20700     }
20701     isOpticalLevelCleared(levelId: number): boolean {
20702         return isOpticalLevelClearedInFlat(this._data.opticalLevelClears, levelId);
20703     }
20704     getOpticalLevelBestSteps(levelId: number): number | null {
20705         return getOpticalLevelBestStepsFromFlat(this._data.opticalLevelClears, levelId);
20706     }

```

```

20707  /** 关卡通关：写入/刷新最少步数、解锁下一关占位、立即推进 opticalCurrentLevelId */
20708  recordOpticalLevelClear(levelId: number, steps: number): void {
20709      const id = Math.trunc(levelId);
20710      if (id <= 0) {
20711          return;
20712      }
20713      let { changed, clears } = upsertOpticalLevelBestSteps(this._data.opticalLevelClears, id, steps);
20714      const unlock = markNextOpticalLevelUnlockedIfAbsent(clears, id);
20715      if (unlock.changed) {
20716          changed = true;
20717          clears = unlock.clears;
20718      }
20719      this._data.opticalLevelClears = clears;
20720      const continueLevelId = getNextOpticalLevelId(id) ?? getFirstOpticalLevelId();
20721      const currentLevelChanged = this._data.opticalCurrentLevelId !== continueLevelId;
20722      if (currentLevelChanged) {
20723          this._data.opticalCurrentLevelId = continueLevelId;
20724          changed = true;
20725      }
20726      logSaveDebug('recordOpticalLevelClear', {
20727          levelId: id,
20728          steps,
20729          changed,
20730          opticalCurrentLevelId: this._data.opticalCurrentLevelId,
20731          clears: summarizeClears(this._data.opticalLevelClears),
20732      });
20733      this.save();
20734  }
20735  save(): void {
20736      try {
20737          const before = this._data.opticalLevelClears;
20738          let sanitized = mergeOpticalLevelClears(before);
20739          if (sanitized.length === 0 && before.length > 0) {
20740              sanitized = mergeOpticalLevelClears(JSON.stringify(before));
20741          }
20742          if (sanitized.length > 0 || before.length === 0) {
20743              this._data.opticalLevelClears = sanitized;
20744          } else {
20745              logSaveDebug('save: merge 未得到 clears, 保留内存原数组', before);
20746          }
20747          const json = JSON.stringify(this._data);
20748          setStorageItem(PLAYER_DATA_STORAGE_KEY, json);
20749          if (isWeChatMiniGameStorage()) {
20750              const verifyRaw = readRawStorageValue(PLAYER_DATA_STORAGE_KEY);
20751              const verify = parseStoragePayload(verifyRaw);
20752              let verifyClears = verify
20753                  ? mergeOpticalLevelClears(verify.opticalLevelClears)
20754                  : [];
20755              if (verifyClears.length === 0 && typeof verifyRaw === 'string') {
20756                  verifyClears = mergeOpticalLevelClearsFromPlayerJsonText(verifyRaw);
20757              }
20758              logSaveDebug('save verify readback', {
20759                  ok: verify != null,
20760                  clears: summarizeClears(verifyClears),
20761                  rawClears: verify?.opticalLevelClears,
20762              });
20763          }
20764      } catch (e) {

```

```

20765         console.warn('[DataManager] save failed', e);
20766     }
20767 }
20768 /**
20769  * 初始化：可选首次写入 mock 到本地，再 restore 合并默认存档。
20770  */
20771 init(): void {
20772     if (USE_DEBUG MOCK_SAVE && !DataManager._debugMockSeededThisRun) {
20773         try {
20774             setStorageItem(PLAYER_DATA_STORAGE_KEY, JSON.stringify(MOCK_PLAYER_DATA));
20775             DataManager._debugMockSeededThisRun = true;
20776         } catch (e) {
20777             console.warn('[DataManager] 调试写入本地存档失败', e);
20778         }
20779     }
20780     this.restore();
20781 }
20782 restore(): void {
20783     try {
20784         const rawStorage = readRawStorageValue(PLAYER_DATA_STORAGE_KEY);
20785         const storageText = typeof rawStorage === 'string' ? rawStorage : null;
20786         const parsed = parseStoragePayload(rawStorage);
20787         if (!parsed) {
20788             if (!this._everRestoredFromDisk) {
20789                 this._data = cloneDefaultData();
20790                 logSaveDebug('restore: 无本地档，使用默认');
20791             } else {
20792                 logSaveDebug('restore: 读档为空，保留内存', {
20793                     currentLevelId: this._data.opticalCurrentLevelId,
20794                     clears: summarizeClears(this._data.opticalLevelClears),
20795                 });
20796             }
20797             this._everRestoredFromDisk = true;
20798             return;
20799         }
20800         this._data = mergePlayerData(parsed, storageText);
20801         this._everRestoredFromDisk = true;
20802     } catch (e) {
20803         console.warn('[DataManager] restore failed', e);
20804         if (!this._everRestoredFromDisk) {
20805             this._data = cloneDefaultData();
20806         }
20807         this._everRestoredFromDisk = true;
20808     }
20809 }
20810 }
20811 // ----- assets/Scripts/Manager/GameManager.ts -----
20812 import { _decorator, Component } from 'cc';
20813 const { ccclass } = _decorator;
20814 /**
20815  * 全局局内编排入口占位。
20816  * 光学解谜当前由 `OpticalPuzzleRoot` + `OpticalPuzzleSession` 承担；后续多玩法时可在此分发。
20817  */
20818 @ccclass('GameManager')
20819 export class GameManager extends Component {
20820     protected onLoad(): void {
20821         // 预留：读档、切关、与 Menu 场景衔接等
20822     }

```

```

20823 }
20824 // ----- assets/Scripts/Manager/GameUIManager.ts -----
20825 import { _decorator, Button, Component, director, Label, Node } from 'cc';
20826 import type { OpticalSnapshotNotify } from '../Games/OpticalPuzzle/Application/OpticalPuzzleSession';
20827 import { ensureBackButtonView } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleBackButtonView';
20828 import { ensureLevelSelectButtonView } from
20829 '../Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectButtonView';
20830 import {
20831     ensureAnswerPanel,
20832     openAnswerPanel,
20833     resolveAnswerPanelNode,
20834 } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleAnswerPanel';
20835 import {
20836     ensureLevelSelectPanel,
20837     openLevelSelectPanel,
20838     prewarmLevelSelectPanel,
20839     syncLevelSelectPanelVisuals,
20840 } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectPanel';
20841 import { ensureStepViews } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleStepView';
20842 import { ensureWinPanelNextLevelButtonView } from
20843 '../Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelNextLevelButtonView';
20844 import { ensureWinPanelStarsView } from
20845 '../Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelStarsView';
20846 import { ensureWinPanelStepViews } from
20847 '../Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelStepView';
20848 import { ensureWinPanelTitleView } from
20849 '../Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelTitleView';
20850 import { ensureWinPanelWindsView, resolvePreWinPanelNode } from
20851 '../Games/OpticalPuzzle/Presentation/OpticalPuzzleWinPanelWindsView';
20852 import { DataManager } from '../Data/Manager';
20853 import { AUDIO_EFFECT_ENUM, EVENT_ENUM, SCENE_ENUM } from '../Utils/Enum';
20854 import { OPTICAL_PUZZLE, PLAY_AUDIO } from '../Utils/Event';
20855 const { ccclass, property } = _decorator;
20856 /**
20857  * Game 场景局内 UI 编排：返回菜单、局内选关弹层等。
20858  * 挂在 GameRoot；按钮与面板拖入对应属性，勿在 Button Click Events 里重复绑逻辑。
20859  */
20860 @ccclass('GameUIManager')
20861 export class GameUIManager extends Component {
20862     /** 返回菜单（TopBar/BtnBackMenu） */
20863     @property(Node)
20864     btnBackMenu: Node = null;
20865     /** 打开局内选关面板（TopBar/BtnLevelSelect） */
20866     @property(Node)
20867     btnLevelSelect: Node = null;
20868     /** 选关弹层根节点（layerOverlay/LevelSelectPanel） */
20869     @property(Node)
20870     levelSelectPanel: Node = null;
20871     /** 参考解弹层（layerOverlay/answerPanel） */
20872     @property(Node)
20873     answerPanel: Node = null;
20874     /** 当前关卡标题（TopBar/LabelLevel 的 cc.Label） */
20875     @property(Label)
20876     labelLevel: Label | null = null;
20877     private _displayedLevelId = -1;
20878     protected onLoad(): void {
20879         director.preloadScene(SCENE_ENUM.MENU);
20880         this._resolveLabelLevel();

```



```

20881         this._ensureTopBarIconButtons();
20882         ensureWinPanelWindsView(this.node);
20883         ensureWinPanelTitleView(this.node);
20884         ensureWinPanelStarsView(this.node);
20885         ensureWinPanelStepViews(this.node);
20886         ensureWinPanelNextLevelButtonView(this.node);
20887         this._resolveAnswerPanel();
20888         this._hidePreWinPanel();
20889         ensureLevelSelectPanel(this.levelSelectPanel);
20890         ensureAnswerPanel(this.answerPanel);
20891         this.closeLevelSelect();
20892         this.closeAnswerPanel();
20893         prewarmLevelSelectPanel(this.levelSelectPanel);
20894         this.bindBackMenuButton();
20895         this.bindLevelSelectButton();
20896         OPTICAL_PUZZLE.on(
20897             EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED,
20898             this._onOpticalSnapshotChanged,
20899             this,
20900         );
20901         this.refreshLevelLabel(DataManager.instance.opticalCurrentLevelId);
20902     }
20903     protected onDestroy(): void {
20904         OPTICAL_PUZZLE.off(
20905             EVENT_ENUM.OPTICAL_SNAPSHOT_CHANGED,
20906             this._onOpticalSnapshotChanged,
20907             this,
20908         );
20909         this.unbindBackMenuButton();
20910         this.unbindLevelSelectButton();
20911     }
20912     /** 将 TopBar 关卡标题设为「第 x 关」 */
20913     refreshLevelLabel(levelId: number): void {
20914         if (!this.labelLevel?.isValid || levelId <= 0) {
20915             return;
20916         }
20917         this.labelLevel.string = `第 ${levelId} 关`;
20918         this._displayedLevelId = levelId;
20919     }
20920     /** 返回 Menu 场景 */
20921     backToMenu(): void {
20922         director.loadScene(SCENE_ENUM.MENU);
20923     }
20924     /** 显示局内选关面板（列表已在进入场景时预构建，打开前刷新解锁） */
20925     openLevelSelect(): void {
20926         openLevelSelectPanel(this.levelSelectPanel);
20927     }
20928     /** 隐藏局内选关面板 */
20929     closeLevelSelect(): void {
20930         if (!this.levelSelectPanel?.isValid) {
20931             return;
20932         }
20933         this.levelSelectPanel.active = false;
20934     }
20935     /** 显示参考解面板 */
20936     openAnswerPanel(): void {
20937         openAnswerPanel(this.answerPanel);
20938     }

```

```

20939  /** 隐藏参考解面板 */
20940  closeAnswerPanel(): void {
20941      if (!this.answerPanel?.isValid) {
20942          return;
20943      }
20944      this.answerPanel.active = false;
20945  }
20946  private _hidePreWinPanel(): void {
20947      const panel = resolvePreWinPanelNode(this.node);
20948      if (panel?.isValid) {
20949          panel.active = false;
20950      }
20951  }
20952  private bindBackMenuButton(): void {
20953      this.bindNodeClick(this.btnBackMenu, this.onBackMenuClick);
20954  }
20955  private unbindBackMenuButton(): void {
20956      this.unbindNodeClick(this.btnBackMenu, this.onBackMenuClick);
20957  }
20958  private bindLevelSelectButton(): void {
20959      this.bindNodeClick(this.btnLevelSelect, this.onLevelSelectClick);
20960  }
20961  private unbindLevelSelectButton(): void {
20962      this.unbindNodeClick(this.btnLevelSelect, this.onLevelSelectClick);
20963  }
20964  private bindNodeClick(node: Node | null, handler: () => void): void {
20965      if (!node?.isValid) {
20966          return;
20967      }
20968      const button = node.getComponent(Button);
20969      if (button) {
20970          button.node.on(Button.EventType.CLICK, handler, this);
20971          return;
20972      }
20973      node.on(Node.EventType.TOUCH_END, handler, this);
20974  }
20975  private unbindNodeClick(node: Node | null, handler: () => void): void {
20976      if (!node?.isValid) {
20977          return;
20978      }
20979      const button = node.getComponent(Button);
20980      const clickNode = button?.node;
20981      if (clickNode?.isValid) {
20982          clickNode.off(Button.EventType.CLICK, handler, this);
20983          return;
20984      }
20985      node.off(Node.EventType.TOUCH_END, handler, this);
20986  }
20987  private onBackMenuClick(): void {
20988      PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
20989      // 下一帧切场景，避免与 Button 缩放/绘制抢同一帧主线程
20990      this.scheduleOnce(() => this.backToMenu(), 0);
20991  }
20992  private onLevelSelectClick(): void {
20993      PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
20994      this.openLevelSelect();
20995  }
20996  /** TopBar 图标键：Graphics 绘制 + 与四向键一致的按压缩放 */

```

```

20997     private _ensureTopBarIconButton(): void {
20998         const topBar = this.node.getChildByName('layHub')?.getChildByName('TopBar') ?? null;
20999         if (!this.btnBackMenu?.isValid) {
21000             this.btnBackMenu = topBar?.getChildByName('BtnBackMenu') ?? null;
21001         }
21002         if (!this.btnLevelSelect?.isValid) {
21003             this.btnLevelSelect = topBar?.getChildByName('BtnLevelSelect') ?? null;
21004         }
21005         ensureBackButtonView(this.btnBackMenu);
21006         ensureLevelSelectButtonView(this.btnLevelSelect);
21007         ensureStepViews(topBar);
21008     }
21009     /** 未在检查器绑定时，按 layerOverlay/answerPanel 解析 */
21010     private _resolveAnswerPanel(): void {
21011         if (this.answerPanel?.isValid) {
21012             return;
21013         }
21014         this.answerPanel = resolveAnswerPanelNode(this.node);
21015     }
21016     /** 未在检查器绑定时，按 GameRoot/layHub/TopBar/LabelLevel 解析 */
21017     private _resolveLabelLevel(): void {
21018         if (this.labelLevel?.isValid) {
21019             return;
21020         }
21021         const labelNode =
21022             this.node.getChildByName('layHub')?.getChildByName('TopBar')?.getChildByName('LabelLevel') ??
21023             null;
21024         if (!labelNode?.isValid) {
21025             return;
21026         }
21027         this.labelLevel = labelNode.getComponent(Label);
21028     }
21029     private _onOpticalSnapshotChanged(payload: OpticalSnapshotNotify): void {
21030         const snap = payload?.snapshot;
21031         if (!snap) {
21032             return;
21033         }
21034         if (snap.levelId !== this._displayedLevelId) {
21035             this.refreshLevelLabel(snap.levelId);
21036         }
21037         syncLevelSelectPanelVisuals(this.levelSelectPanel);
21038     }
21039 }
21040 // ----- assets/Scripts/Manager/LevelManager.ts -----
21041 import { _decorator, Component } from 'cc';
21042 const { ccclass } = _decorator;
21043 /** 关卡列表与切换占位；光学解谜关卡整数 id 与 DataManager.opticalCurrentLevelId 对齐 */
21044 @ccclass('LevelManager')
21045 export class LevelManager extends Component {}
21046 // ----- assets/Scripts/Manager/MenuManager.ts -----
21047 import { _decorator, Button, Component, director, Node } from 'cc';
21048 import {
21049     ensureAboutMePanel,
21050     resolveAboutMePanelNode,
21051 } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleAboutMePanel';
21052 import {
21053     ensureLevelSelectPanel,
21054     openLevelSelectPanel,

```

```

21055     prewarmLevelSelectPanel,
21056     resolveLevelSelectPanelNode,
21057 } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectPanel';
21058 import { ensureMenuGameTitleView } from '../MenuGameTitleView';
21059 import { ensureMenuGameTitle2View } from '../MenuGameTitle2View';
21060 import { ensureMenuGameTitleIconView } from '../MenuGameTitleIconView';
21061 import { ensureMenuGameTitleIcon2View } from '../MenuGameTitleIcon2View';
21062 import { ensureMenuAboutButtonView } from '../MenuAboutButtonView';
21063 import { ensureMenuLevelSelectButtonView } from '../MenuLevelSelectButtonView';
21064 import { ensureMenuShareButtonView } from '../MenuShareButtonView';
21065 import { ensureMenuStartButtonView } from '../MenuStartButtonView';
21066 import { DataManager } from './DataManager';
21067 import { AUDIO_EFFECT_ENUM, EVENT_ENUM, SCENE_ENUM } from './Utils/Enum';
21068 import { PLAY_AUDIO } from './Utils/Event';
21069 import { invokeWeChatFriendShare } from './Utils/WeChatShare';
21070 const { ccclass, property } = _decorator;
21071 /**
21072  * 菜单场景编排：读档、开始游戏、选关弹层等。
21073  * 挂在 MenuRoot；按钮拖入对应属性，勿在 Button Click Events 里重复绑定逻辑。
21074  */
21075 @ccclass('MenuManager')
21076 export class MenuManager extends Component {
21077     /** 开始游戏按钮（MainPanel/BtnStart） */
21078     @property(Node)
21079     btnStart: Node = null;
21080     /** 打开选关面板（MainPanel/BtnLevelSelect） */
21081     @property(Node)
21082     btnLevelSelect: Node = null;
21083     /** 选关弹层根节点（layerOverlay/LevelSelectPanel） */
21084     @property(Node)
21085     levelSelectPanel: Node = null;
21086     /** 微信分享（MainPanel/BtnShare） */
21087     @property(Node)
21088     btnShare: Node = null;
21089     /** 关于（MainPanel/BtnAbout） */
21090     @property(Node)
21091     btnAbout: Node = null;
21092     /** 关于弹层（layerOverlay/aboutMePanel） */
21093     @property(Node)
21094     aboutMePanel: Node = null;
21095     /** 游戏标题（MainPanel/GameTitle，矢量描边） */
21096     @property(Node)
21097     gameTitle: Node = null;
21098     /** 副标题（MainPanel/GameTitle2，矢量描边） */
21099     @property(Node)
21100     gameTitle2: Node = null;
21101     /** 标题主角图标（MainPanel/GameTitleIcon，睁眼朝左看） */
21102     @property(Node)
21103     gameTitleIcon: Node = null;
21104     /** 标题白色光源（MainPanel/GameTitleIcon2，朝右 1000px 光束） */
21105     @property(Node)
21106     gameTitleIcon2: Node = null;
21107     protected onLoad(): void {
21108         DataManager.instance.init();
21109         director.preloadScene(SCENE_ENUM.GAME);
21110         this._resolveMenuNodes();
21111         ensureMenuStartButtonView(this.btnStart);
21112         ensureMenuLevelSelectButtonView(this.btnLevelSelect);

```

```

21113         ensureMenuShareButtonView(this.btnShare);
21114         ensureMenuAboutButtonView(this.btnAbout);
21115         ensureLevelSelectPanel(this.levelSelectPanel);
21116         ensureAboutMePanel(this.aboutMePanel);
21117         ensureMenuGameTitleView(this.gameTitle);
21118         ensureMenuGameTitle2View(this.gameTitle2);
21119         ensureMenuGameTitleIconView(this.gameTitleIcon);
21120         ensureMenuGameTitleIcon2View(this.gameTitleIcon2);
21121         this.closeLevelSelect();
21122         this.closeAbout();
21123         prewarmLevelSelectPanel(this.levelSelectPanel);
21124         this.bindStartButton();
21125         this.bindLevelSelectButton();
21126         this.bindShareButton();
21127         this.bindAboutButton();
21128     }
21129     protected onDestroy(): void {
21130         this.unbindStartButton();
21131         this.unbindLevelSelectButton();
21132         this.unbindShareButton();
21133         this.unbindAboutButton();
21134     }
21135     /** 进入局内场景（当前进度关卡由 DataManager.opticalCurrentLevelId 决定） */
21136     startGame(): void {
21137         director.loadScene(SCENE_ENUM.GAME);
21138     }
21139     /** 显示选关面板 */
21140     openLevelSelect(): void {
21141         openLevelSelectPanel(this.levelSelectPanel);
21142     }
21143     /** 隐藏选关面板（MenuOverlayWindow 关闭时也可调） */
21144     closeLevelSelect(): void {
21145         if (!this.levelSelectPanel?.isValid) {
21146             return;
21147         }
21148         this.levelSelectPanel.active = false;
21149     }
21150     /** 显示关于弹层 */
21151     openAbout(): void {
21152         if (!this.aboutMePanel?.isValid) {
21153             return;
21154         }
21155         this.closeLevelSelect();
21156         this.aboutMePanel.active = true;
21157     }
21158     /** 隐藏关于弹层 */
21159     closeAbout(): void {
21160         if (!this.aboutMePanel?.isValid) {
21161             return;
21162         }
21163         this.aboutMePanel.active = false;
21164     }
21165     /** 未在检查器绑定时，按 MenuRoot 子节点名解析 */
21166     private _resolveMenuNodes(): void {
21167         if (!this.btnStart?.isValid) {
21168             this.btnStart = this._findMainPanel()?.getChildByName('BtnStart') ?? null;
21169         }
21170         if (!this.btnLevelSelect?.isValid) {

```

```

21171         this.btnLevelSelect = this._findMainPanel()?.getChildByName('BtnLevelSelect') ?? null;
21172     }
21173     if (!this.btnShare?.isValid) {
21174         this.btnShare = this._findMainPanel()?.getChildByName('BtnShare') ?? null;
21175     }
21176     if (!this.btnAbout?.isValid) {
21177         this.btnAbout = this._findMainPanel()?.getChildByName('BtnAbout') ?? null;
21178     }
21179     if (!this.levelSelectPanel?.isValid) {
21180         this.levelSelectPanel = resolveLevelSelectPanelNode(this.node);
21181     }
21182     if (!this.aboutMePanel?.isValid) {
21183         this.aboutMePanel = resolveAboutMePanelNode(this.node);
21184     }
21185     if (!this.gameTitle?.isValid) {
21186         this.gameTitle = this._findMainPanel()?.getChildByName('GameTitle') ?? null;
21187     }
21188     if (!this.gameTitle2?.isValid) {
21189         this.gameTitle2 = this._findMainPanel()?.getChildByName('GameTitle2') ?? null;
21190     }
21191     if (!this.gameTitleIcon?.isValid) {
21192         this.gameTitleIcon = this._findMainPanel()?.getChildByName('GameTitleIcon') ?? null;
21193     }
21194     if (!this.gameTitleIcon2?.isValid) {
21195         this.gameTitleIcon2 = this._findMainPanel()?.getChildByName('GameTitleIcon2') ?? null;
21196     }
21197 }
21198 private _findMainPanel(): Node | null {
21199     return this.node.getChildByName('layerMain')?.getChildByName('MainPanel') ?? null;
21200 }
21201 private bindStartButton(): void {
21202     this.bindNodeClick(this.btnStart, this.onStartClick);
21203 }
21204 private unbindStartButton(): void {
21205     this.unbindNodeClick(this.btnStart, this.onStartClick);
21206 }
21207 private bindLevelSelectButton(): void {
21208     this.bindNodeClick(this.btnLevelSelect, this.onLevelSelectClick);
21209 }
21210 private unbindLevelSelectButton(): void {
21211     this.unbindNodeClick(this.btnLevelSelect, this.onLevelSelectClick);
21212 }
21213 private bindShareButton(): void {
21214     this.bindNodeClick(this.btnShare, this.onShareClick);
21215 }
21216 private unbindShareButton(): void {
21217     this.unbindNodeClick(this.btnShare, this.onShareClick);
21218 }
21219 private bindAboutButton(): void {
21220     this.bindNodeClick(this.btnAbout, this.onAboutClick);
21221 }
21222 private unbindAboutButton(): void {
21223     this.unbindNodeClick(this.btnAbout, this.onAboutClick);
21224 }
21225 private bindNodeClick(node: Node | null, handler: () => void): void {
21226     if (!node?.isValid) {
21227         return;
21228     }

```

```

21229         const button = node.getComponent(Button);
21230         if (button) {
21231             button.node.on(Button.EventType.CLICK, handler, this);
21232             return;
21233         }
21234         node.on(Node.EventType.TOUCH_END, handler, this);
21235     }
21236     private unbindNodeClick(node: Node | null, handler: () => void): void {
21237         if (!node?.isValid) {
21238             return;
21239         }
21240         const button = node.getComponent(Button);
21241         if (button) {
21242             button.node.off(Button.EventType.CLICK, handler, this);
21243             return;
21244         }
21245         node.off(Node.EventType.TOUCH_END, handler, this);
21246     }
21247     private onStartClick(): void {
21248         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21249         this.startGame();
21250     }
21251     private onLevelSelectClick(): void {
21252         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21253         this.openLevelSelect();
21254     }
21255     /** 微信好友分享（非微信环境 onFail） */
21256     private onShareClick(): void {
21257         invokeWeChatFriendShare({
21258             onSuccess: () => {
21259                 /* 可按需加分享奖励 */
21260             },
21261             onFail: () => {
21262                 /* 浏览器预览等环境无 wx.shareAppMessage */
21263             },
21264         });
21265         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21266     }
21267     private onAboutClick(): void {
21268         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21269         this.openAbout();
21270     }
21271 }
21272 // ----- assets/Scripts/Manager/MusicManager.ts -----
21273 import {
21274     _decorator,
21275     AudioClip,
21276     AudioSource,
21277     Component,
21278     director,
21279     Director,
21280     Node,
21281 } from 'cc';
21282 import { DataManager } from './DataManager';
21283 import { AUDIO_EFFECT_ENUM, EVENT_ENUM, SCENE_ENUM } from '../Utils/Enum';
21284 import { PLAY_AUDIO, PLAY_BGM } from '../Utils/Event';
21285 import {
21286     cancelWeChatRewardedVideoPreloadSchedule,

```

```

21287         scheduleWeChatRewardedVideoPreloadForGame,
21288     } from './Utils/WeChatRewardedVideoAd';
21289     import { initWeChatMiniGameShare, trySettlePendingShareAfterWxOnShow } from './Utils/WeChatShare';
21290     const { ccclass, property } = _decorator;
21291     /**
21292      * 音频管理: PersistRoot + 事件驱动 (project-rules)。
21293      * 全局 BGM (如 Bg.mp3) 在 Menu / Game 场景循环播放; 音效经 PLAY_AUDIO 播放。
21294      * 未绑定 AudioClip 或 AudioSource 时跳过, 避免空引用。
21295      */
21296     @ccclass('MusicManager')
21297     export class MusicManager extends Component {
21298         // #region 编辑器绑定
21299         /** 全局背景音乐 (如 assets/Audio/Bg.mp3, Menu / Game 共用) */
21300         @property({ type: AudioClip, tooltip: '全局 BGM (Bg.mp3)' })
21301         bgm: AudioClip | null = null;
21302         /** UI 通用按钮点击 (建议 assets/Audio/Click.mp3) */
21303         @property({ type: AudioClip, tooltip: '通用按钮点击 (Click.mp3)' })
21304         clickButton: AudioClip | null = null;
21305         /** 四向键移动成功: 主角走一格或推动元件 (建议 Right.mp3) */
21306         @property({ type: AudioClip, tooltip: '四向键移动成功 (Right.mp3)' })
21307         opticalMoveSuccess: AudioClip | null = null;
21308         /** 四向键移动失败: 与主角阻拦脸同级 (建议 Failed.mp3) */
21309         @property({ type: AudioClip, tooltip: '四向键移动失败 (Failed.mp3)' })
21310         opticalMoveFail: AudioClip | null = null;
21311         /** 单个目标灯被点亮 (待加资源, 如 Light.mp3) */
21312         @property({ type: AudioClip, tooltip: '单个目标灯点亮 (待加 Light.mp3 等)' })
21313         opticalTargetLit: AudioClip | null = null;
21314         /** 关卡通关成功 (建议 Pass.mp3; winPanel 展示时播放, 前有 PRE_WIN_PANEL_DELAY_SEC) */
21315         @property({ type: AudioClip, tooltip: '关卡通关 (Pass.mp3; 结算 winPanel 弹出时播放)' })
21316         opticalLevelComplete: AudioClip | null = null;
21317         /** 用于循环 BGM; 音效使用 playOneShot 播在同一 AudioSource 上 */
21318         @property(AudioSource)
21319         audioSource: AudioSource | null = null;
21320         // #endregion
21321         private static _persistRegistered = false;
21322         /** 当前正在播放的 BGM Clip, 避免同曲重复 stop/play */
21323         private _currentBgmClip: AudioClip | null = null;
21324         private readonly _wxOnShowHandler = (): void => {
21325             trySettlePendingShareAfterWxOnShow();
21326         };
21327         protected onLoad(): void {
21328             this.registerPersistRoot();
21329             director.on(Director.EVENT_AFTER_SCENE_LAUNCH, this.onAfterSceneLaunch, this);
21330             PLAY_AUDIO.on(EVENT_ENUM.PLAY_AUDIO, this.onAudioPlay, this);
21331             PLAY_BGM.on(EVENT_ENUM.PLAY_BGM, this.onPlayBgmEvent, this);
21332             initWeChatMiniGameShare();
21333             this._bindWeChatOnShow();
21334         }
21335         protected onDestroy(): void {
21336             this._unbindWeChatOnShow();
21337             cancelWeChatRewardedVideoPreloadSchedule();
21338             director.off(Director.EVENT_AFTER_SCENE_LAUNCH, this.onAfterSceneLaunch, this);
21339             PLAY_AUDIO.off(EVENT_ENUM.PLAY_AUDIO, this.onAudioPlay, this);
21340             PLAY_BGM.off(EVENT_ENUM.PLAY_BGM, this.onPlayBgmEvent, this);
21341         }
21342         // #region BGM
21343         onAfterSceneLaunch(): void {
21344             const sceneName = director.getScene()?.name ?? '';

```



```

21345         this.switchBgmBySceneName(sceneName);
21346         cancelWeChatRewardedVideoPreloadSchedule();
21347         if (sceneName === SCENE_ENUM.GAME) {
21348             scheduleWeChatRewardedVideoPreloadForGame();
21349         }
21350     }
21351     /** 设置页切换 BGM 开关后可 emit PLAY_BGM 刷新当前 BGM */
21352     onPlayBgmEvent(): void {
21353         const sceneName = director.getScene()?.name ?? '';
21354         this.switchBgmBySceneName(sceneName);
21355     }
21356     switchBgmBySceneName(sceneName: string): void {
21357         if (sceneName === SCENE_ENUM.MENU || sceneName === SCENE_ENUM.GAME) {
21358             this.playGlobalBgm();
21359         } else {
21360             this.stopBgm();
21361         }
21362     }
21363     playGlobalBgm(): void {
21364         if (!DataManager.instance.bgmOn) {
21365             this.stopBgm();
21366             return;
21367         }
21368         this.playBgmClip(this.bgm);
21369     }
21370     private playBgmClip(clip: AudioClip | null): void {
21371         const source = this.audioSource;
21372         if (!source) {
21373             return;
21374         }
21375         if (!clip) {
21376             this.stopBgm();
21377             return;
21378         }
21379         if (this._currentBgmClip === clip && source.playing) {
21380             return;
21381         }
21382         source.stop();
21383         source.clip = clip;
21384         source.loop = true;
21385         source.play();
21386         this._currentBgmClip = clip;
21387     }
21388     private stopBgm(): void {
21389         if (this.audioSource) {
21390             this.audioSource.stop();
21391         }
21392         this._currentBgmClip = null;
21393     }
21394     //endregion
21395     //region 音效
21396     onAudioPlay(type: AUDIO_EFFECT_ENUM | AUDIO_EFFECT_ENUM[]): void {
21397         if (!DataManager.instance.sfxOn || !this.audioSource) {
21398             return;
21399         }
21400         const types = Array.isArray(type) ? type : [type];
21401         for (const sfxType of types) {
21402             const clip = this.resolveSfxClip(sfxType);

```

```

21403         if (clip) {
21404             this.audioSource.playOneShot(clip);
21405         }
21406     }
21407 }
21408 private resolveSfxClip(type: AUDIO_EFFECT_ENUM): AudioClip | null {
21409     switch (type) {
21410         case AUDIO_EFFECT_ENUM.CLICK_BUTTON:
21411             return this.clickButton;
21412         case AUDIO_EFFECT_ENUM.OPTICAL_MOVE_SUCCESS:
21413             return this.opticalMoveSuccess;
21414         case AUDIO_EFFECT_ENUM.OPTICAL_MOVE_FAIL:
21415             return this.opticalMoveFail;
21416         case AUDIO_EFFECT_ENUM.OPTICAL_TARGET_LIT:
21417             return this.opticalTargetLit;
21418         case AUDIO_EFFECT_ENUM.OPTICAL_LEVEL_COMPLETE:
21419             return this.opticalLevelComplete;
21420         default:
21421             return null;
21422     }
21423 }
21424 //endregion
21425 /** 持久化 PersistRoot (含 AudioService / ToastService 兄弟节点) */
21426 private registerPersistRoot(): void {
21427     if (MusicManager._persistRegistered) {
21428         return;
21429     }
21430     const root = this.findPersistRootNode();
21431     director.addPersistRootNode(root);
21432     MusicManager._persistRegistered = true;
21433 }
21434 private findPersistRootNode(): Node {
21435     let node: Node | null = this.node;
21436     while (node) {
21437         if (node.name === 'PersistRoot') {
21438             return node;
21439         }
21440         node = node.parent;
21441     }
21442     return this.node;
21443 }
21444 private _bindWeChatOnShow(): void {
21445     const wxApi = (globalThis as { wx?: { onShow?(fn: () => void): void; offShow?(fn: () => void): void } }).wx;
21446     wxApi?.onShow?.(this._wxOnShowHandler);
21447 }
21448 private _unbindWeChatOnShow(): void {
21449     const wxApi = (globalThis as { wx?: { offShow?(fn: () => void): void } }).wx;
21450     wxApi?.offShow?.(this._wxOnShowHandler);
21451 }
21452 }
21453 // ----- assets/Scripts/Manager/ToastManager.ts -----
21454 import { _decorator, Component, Prefab, instantiate, Node, Label, UITransform, UIOpacity, tween, Tween } from
21455 'cc';
21456 import { RoundBox } from '../RoundBox';
21457 const { cclass, property } = _decorator;
21458 /**
21459  * Toast: 展示文案 + 背景宽度; 显示 DISPLAY_SEC 秒后, FADE_SEC 秒内透明度变为 0。
21460  * 新一次 show 会立刻打断上一次动画并展示新内容。

```

```

21461 * 预制体根节点下需有子节点: text (挂 Label)、bg (挂 UITransform 可调宽度)。
21462 */
21463 @ccclass('ToastManager')
21464 export class ToastManager extends Component {
21465     // Toast 预制体 (根节点下子节点: text、bg)
21466     @property(Prefab)
21467     toastPrefab: Prefab = null;
21468     private static readonly DISPLAY_SEC = 1;
21469     private static readonly FADE_SEC = 1;
21470     /** 未传或非法 bgWidth 时使用 (避免 undefined → NaN 导致 UITransform.width 无效) */
21471     private static readonly DEFAULT_BG_WIDTH = 320;
21472     private _toastRoot: Node | null = null;
21473     private static resolveBgWidth(bgWidth: number | undefined): number {
21474         if (typeof bgWidth !== 'number' || !Number.isFinite(bgWidth)) {
21475             return ToastManager.DEFAULT_BG_WIDTH;
21476         }
21477         return Math.max(0, bgWidth);
21478     }
21479     private static resolveLocalY(localY: number | undefined): number {
21480         if (typeof localY !== 'number' || !Number.isFinite(localY)) {
21481             return 0;
21482         }
21483         return localY;
21484     }
21485     /**
21486      * @param message 提示文本
21487      * @param bgWidth 背景节点 `bg` 的 **宽度** (像素)。注意第二参是宽度不是 Y; 若误传
21488      * `undefined` (如可选链未绑定到值) 会走默认宽 320。
21489      * @param localY 弹窗根节点本地坐标 **Y**；省略或非法时为 0。要只调纵向位置请写 `show(msg,
21490      * undefined, 450)` 或先传宽度再传 Y。
21491      */
21492     show(message: string, bgWidth?: number, localY: number = 0): void {
21493         if (!this.toastPrefab) {
21494             console.warn('[ToastManager] 未绑定 toastPrefab');
21495             return;
21496         }
21497         if (!this._toastRoot || !this._toastRoot.isValid) {
21498             this._toastRoot = instantiate(this.toastPrefab);
21499             this._toastRoot.setParent(this.node);
21500         }
21501         const root = this._toastRoot;
21502         const py = ToastManager.resolveLocalY(localY);
21503         root.setPosition(0, py, 0);
21504         const uiOp = root.getComponent(UIOpacity) ?? root.addComponent(UIOpacity);
21505         Tween.stopAllByTarget(uiOp);
21506         /** 先定 bg 宽并立刻刷新 RoundBox: 仅改 UITransform 时 onSizeChanged 不会重算 UV, 若只
21507         schedule 下一帧强刷, 首帧仍会按旧网格绘制 → 背景宽度闪一下 */
21508         const bgNode = root.getChildByName('bg');
21509         const bgUt = bgNode?.getComponent(UITransform);
21510         const wResolved = ToastManager.resolveBgWidth(bgWidth);
21511         if (bgUt) {
21512             bgUt.width = wResolved;
21513             const rb = bgNode?.getComponent(RoundBox);
21514             if (rb?.isValid) {
21515                 rb.forceRefreshRender();
21516                 /** 首帧 chunk/texture 未就绪时补一次 (与 MenuOverlayWindow 等场景一致) */
21517                 this.scheduleOnce(() => {
21518                     if (rb.isValid && bgUt.isValid) {

```

```

21519         rb.forceRefreshRender();
21520     }
21521     }, 0);
21522 }
21523 } else {
21524     console.warn('[ToastManager] 预制体根下未找到子节点 bg 或未挂 UITransform');
21525 }
21526 const textNode = root.getChildByName('text');
21527 const label = textNode?.getComponent(Label);
21528 if (label) {
21529     label.string = message;
21530 } else {
21531     console.warn('[ToastManager] 预制体根下未找到子节点 text 或未挂 Label');
21532 }
21533 root.active = true;
21534 uiOp.opacity = 255;
21535 tween(uiOp)
21536     .delay(ToastManager.DISPLAY_SEC)
21537     .to(ToastManager.FADE_SEC, { opacity: 0 })
21538     .call(() => {
21539         // 下一帧再关节点，避免与最后一帧 UI 提交叠在同一时刻，圆角 VERTEX 底图容易「到
21540 0 突然没」
21541         this.scheduleOnce(() => {
21542             if (root?.isValid) {
21543                 root.active = false;
21544             }
21545             }, 0);
21546         })
21547         .start();
21548     }
21549 }
21550 // ----- assets/Scripts/MenuAboutButtonView.ts -----
21551 import {
21552     _decorator,
21553     Button,
21554     Component,
21555     Graphics,
21556     Node,
21557     UITransform,
21558 } from 'cc';
21559 import { HudButtonPressController } from
21560 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
21561 import { drawMenuAboutButtonGlyph } from './Games/OpticalPuzzle/Presentation/MenuAboutButtonGlyph';
21562 const { ccclass } = _decorator;
21563 /** 与开始键 Button.zoomScale 一致 */
21564 const ABOUT_MENU_BUTTON_ZOOM_SCALE = 0.95;
21565 /** Menu/MainPanel/BtnAbout: 圆形键帽 + 用户 icon (点击由 MenuManager 绑定) */
21566 @ccclass('MenuAboutButtonView')
21567 export class MenuAboutButtonView extends Component {
21568     private _graphics: Graphics | null = null;
21569     private _pressed = false;
21570     private _pressCtrl: HudButtonPressController | null = null;
21571     protected onLoad(): void {
21572         this._hidePlaceholderSplash();
21573         this._ensureButton();
21574         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
21575             this._pressed = pressed;
21576             this._redraw();

```

```

21577         });
21578         this._redraw();
21579     }
21580     protected onEnable(): void {
21581         this._pressCtrl?.bind();
21582         this._redraw();
21583     }
21584     protected onDisable(): void {
21585         this._pressCtrl?.unbind();
21586         this._pressed = false;
21587     }
21588     protected onDestroy(): void {
21589         this._pressCtrl?.unbind();
21590     }
21591     private _hidePlaceholderSplash(): void {
21592         const splash = this.node.getChildByName('SpriteSplash') ?? this.node.getChildByName('bg');
21593         if (splash?.isValid) {
21594             splash.active = false;
21595         }
21596     }
21597     private _ensureButton(): void {
21598         let btn = this.getComponent(Button);
21599         if (!btn) {
21600             btn = this.addComponent(Button);
21601             btn.transition = Button.Transition.SCALE;
21602         }
21603         btn.zoomScale = ABOUT_MENU_BUTTON_ZOOM_SCALE;
21604         btn.transition = Button.Transition.SCALE;
21605     }
21606     private _ensureGraphics(): Graphics | null {
21607         if (!this._graphics?.isValid) {
21608             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21609         }
21610         return this._graphics;
21611     }
21612     private _redraw(): void {
21613         const g = this._ensureGraphics();
21614         const ut = this.getComponent(UITransform);
21615         if (!g || !ut) {
21616             return;
21617         }
21618         g.clear();
21619         const w = ut.width;
21620         const h = ut.height;
21621         const left = -ut.anchorX * w;
21622         const bottom = -ut.anchorY * h;
21623         drawMenuAboutButtonGlyph(g, left, bottom, w, h, this._pressed);
21624     }
21625 }
21626 /** 为 MainPanel/BtnAbout 挂上键帽绘制与按压缩放 */
21627 export function ensureMenuAboutButtonView(btnAbout: Node | null): void {
21628     if (!btnAbout?.isValid) {
21629         return;
21630     }
21631     if (!btnAbout.getComponent(MenuAboutButtonView)) {
21632         btnAbout.addComponent(MenuAboutButtonView);
21633     }
21634 }

```

```
21635 // ----- assets/Scripts/MenuGameTitle2View.ts -----
21636 import { _decorator, Component, Graphics, Node, Sprite, UITransform } from 'cc';
21637 import { drawGameSubtitleOutline } from './Games/OpticalPuzzle/Presentation/MenuGameSubtitleGlyph';
21638 const { ccclass } = _decorator;
21639 /** Menu/MainPanel/GameTitle2: 副标题矢量描边（替代 PNG Sprite） */
21640 @ccclass('MenuGameTitle2View')
21641 export class MenuGameTitle2View extends Component {
21642     private _graphics: Graphics | null = null;
21643     protected onLoad(): void {
21644         this._disableRasterTitle();
21645         this._ensureGraphics();
21646         this._redraw();
21647     }
21648     protected onEnable(): void {
21649         this._redraw();
21650     }
21651     private _disableRasterTitle(): void {
21652         const splash = this.node.getChildByName('SpriteSplash');
21653         if (splash?.isValid) {
21654             splash.active = false;
21655         }
21656         const sprite = this.getComponent(Sprite);
21657         if (sprite) {
21658             sprite.enabled = false;
21659         }
21660     }
21661     private _ensureGraphics(): Graphics | null {
21662         if (!this._graphics?.isValid) {
21663             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21664         }
21665         return this._graphics;
21666     }
21667     private _redraw(): void {
21668         const g = this._ensureGraphics();
21669         const ut = this.getComponent(UITransform);
21670         if (!g || !ut) {
21671             return;
21672         }
21673         g.clear();
21674         const w = ut.width;
21675         const h = ut.height;
21676         const left = -ut.anchorX * w;
21677         const bottom = -ut.anchorY * h;
21678         drawGameSubtitleOutline(g, left, bottom, w, h);
21679     }
21680 }
21681 /** 为 GameTitle2 节点挂上矢量副标题绘制 */
21682 export function ensureMenuGameTitle2View(titleNode: Node | null): void {
21683     if (!titleNode?.isValid) {
21684         return;
21685     }
21686     if (!titleNode.getComponent(MenuGameTitle2View)) {
21687         titleNode.addComponent(MenuGameTitle2View);
21688     }
21689 }
21690 // ----- assets/Scripts/MenuGameTitleIcon2View.ts -----
21691 import {
21692     _decorator,
```

```

21693     Component,
21694     Graphics,
21695     Node,
21696     ParticleSystem2D,
21697     Sprite,
21698     SpriteFrame,
21699     UITransform,
21700 } from 'cc';
21701 import {
21702     ensureBeamSparkSpritesLoaded,
21703     registerBeamSparkSpriteFrames,
21704 } from './Games/OpticalPuzzle/Presentation/OpticalPuzzleBeamImpactView';
21705 import {
21706     computeMenuTitleRightBeamLayout,
21707     configureMenuTitleBeamSparkParticle,
21708     drawMenuTitleWhiteSourceRightBeam,
21709 } from './Games/OpticalPuzzle/Presentation/MenuGameTitleIcon2Glyph';
21710 const { ccclass, property } = _decorator;
21711 /** Menu/MainPanel/GameTitleIcon2: 白色光源朝右，碰屏右缘 beam_spark */
21712 @ccclass('MenuGameTitleIcon2View')
21713 export class MenuGameTitleIcon2View extends Component {
21714     /** 光束右边界参考（默认向上找 Canvas） */
21715     @property({ type: Node, tooltip: '光束右边界参考节点，默认 Canvas' })
21716     beamBoundsNode: Node | null = null;
21717     /** 可选：拖入 beam_spark 贴图保活（微信包） */
21718     @property({ type: [SpriteFrame], tooltip: '可选，Sprites/OpticalFX/beam_spark_' })
21719     sparkSpriteFrames: SpriteFrame[] = [];
21720     private _graphics: Graphics | null = null;
21721     private _sparkNode: Node | null = null;
21722     private _sparkPs: ParticleSystem2D | null = null;
21723     private _sparkReady = false;
21724     protected onLoad(): void {
21725         this._disableRasterPlaceholder();
21726         this._ensureGraphics();
21727         if (registerBeamSparkSpriteFrames(this.sparkSpriteFrames)) {
21728             this._sparkReady = true;
21729         }
21730         ensureBeamSparkSpritesLoaded(() => {
21731             if (!this.node?.isValid) {
21732                 return;
21733             }
21734             this._sparkReady = true;
21735             this._ensureSparkEmitter();
21736             this._redraw();
21737         });
21738         this._redraw();
21739     }
21740     protected onEnable(): void {
21741         this._redraw();
21742     }
21743     private _disableRasterPlaceholder(): void {
21744         const splash = this.node.getChildByName('SpriteSplash');
21745         if (splash?.isValid) {
21746             splash.active = false;
21747         }
21748         const sprite = this.getComponent(Sprite);
21749         if (sprite) {
21750             sprite.enabled = false;

```

```

21751     }
21752 }
21753 private _ensureGraphics(): Graphics | null {
21754     if (!this._graphics?.isValid) {
21755         this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21756     }
21757     return this._graphics;
21758 }
21759 private _resolveBeamBoundsNode(): Node | null {
21760     if (this.beamBoundsNode?.isValid) {
21761         return this.beamBoundsNode;
21762     }
21763     let node: Node | null = this.node;
21764     while (node) {
21765         if (node.name === 'Canvas') {
21766             return node;
21767         }
21768         node = node.parent;
21769     }
21770     return null;
21771 }
21772 private _ensureSparkEmitter(): void {
21773     if (!this._sparkReady || this._sparkPs?.isValid) {
21774         return;
21775     }
21776     const node = new Node('BeamSpark');
21777     node.setParent(this.node);
21778     const ps = node.addComponent(ParticleSystem2D);
21779     configureMenuTitleBeamSparkParticle(ps);
21780     this._sparkNode = node;
21781     this._sparkPs = ps;
21782     this.scheduleOnce(() => {
21783         if (ps?.isValid) {
21784             ps.resetSystem();
21785         }
21786     }, 0);
21787 }
21788 private _syncSpark(layout: { endX: number; endY: number; hitsScreenEdge: boolean }): void {
21789     if (!this._sparkReady) {
21790         return;
21791     }
21792     this._ensureSparkEmitter();
21793     const sparkNode = this._sparkNode;
21794     const ps = this._sparkPs;
21795     if (!sparkNode?.isValid || !ps?.isValid) {
21796         return;
21797     }
21798     if (!layout.hitsScreenEdge) {
21799         sparkNode.active = false;
21800         ps.stopSystem();
21801         return;
21802     }
21803     sparkNode.active = true;
21804     sparkNode.setPosition(layout.endX, layout.endY, 0);
21805 }
21806 private _redraw(): void {
21807     const g = this._ensureGraphics();
21808     const ut = this.getComponent(UITransform);

```



```

21809         if (!g || !ut) {
21810             return;
21811         }
21812         g.clear();
21813         const w = ut.width;
21814         const h = ut.height;
21815         const left = -ut.anchorX * w;
21816         const bottom = -ut.anchorY * h;
21817         const layout = computeMenuTitleRightBeamLayout(
21818             ut,
21819             left,
21820             bottom,
21821             w,
21822             h,
21823             this._resolveBeamBoundsNode(),
21824         );
21825         drawMenuTitleWhiteSourceRightBeam(g, left, bottom, w, h, layout);
21826         this._syncSpark(layout);
21827     }
21828 }
21829 /** 为 GameTitleIcon2 节点挂上光源图标绘制 */
21830 export function ensureMenuGameTitleIcon2View(iconNode: Node | null): void {
21831     if (!iconNode?.isValid) {
21832         return;
21833     }
21834     if (!iconNode.getComponent(MenuGameTitleIcon2View)) {
21835         iconNode.addComponent(MenuGameTitleIcon2View);
21836     }
21837 }
21838 // ----- assets/Scripts/MenuGameTitleIconView.ts -----
21839 import { _decorator, Component, Graphics, Node, Sprite, UITransform } from 'cc';
21840 import { Direction } from './Games/OpticalPuzzle/Core/OpticalPuzzleTypes';
21841 import { drawPlayerEyes } from './Games/OpticalPuzzle/Presentation/OpticalPuzzlePlayerGlyph';
21842 const { ccclass } = _decorator;
21843 /** 与 OpticalPuzzleBoardView 主角闲置眨眼一致 */
21844 const MENU_PLAYER_BLINK_INTERVAL = 4;
21845 const MENU_PLAYER_BLINK_DURATION = 0.25;
21846 /** Menu/MainPanel/GameTitleIcon: 主角图标（睁眼、朝左看，每 4s 眨一次） */
21847 @ccclass('MenuGameTitleIconView')
21848 export class MenuGameTitleIconView extends Component {
21849     private _graphics: Graphics | null = null;
21850     /** 眨眼周期计时（秒） */
21851     private _blinkClock = 0;
21852     protected onLoad(): void {
21853         this._disableRasterPlaceholder();
21854         this._ensureGraphics();
21855         this._redraw();
21856     }
21857     protected onEnable(): void {
21858         this._blinkClock = 0;
21859         this._redraw();
21860     }
21861     protected update(dt: number): void {
21862         this._blinkClock += dt;
21863         this._redraw();
21864     }
21865     private _currentBlinkAmount(): number {
21866         const cycle = MENU_PLAYER_BLINK_INTERVAL + MENU_PLAYER_BLINK_DURATION;

```

```

21867         const t = this._blinkClock % cycle;
21868         if (t < MENU_PLAYER_BLINK_INTERVAL) {
21869             return 0;
21870         }
21871         const p = (t - MENU_PLAYER_BLINK_INTERVAL) / MENU_PLAYER_BLINK_DURATION;
21872         return Math.sin(Math.min(1, p) * Math.PI);
21873     }
21874     private _disableRasterPlaceholder(): void {
21875         const splash = this.node.getChildByName('SpriteSplash');
21876         if (splash?.isValid) {
21877             splash.active = false;
21878         }
21879         const sprite = this.getComponent(Sprite);
21880         if (sprite) {
21881             sprite.enabled = false;
21882         }
21883     }
21884     private _ensureGraphics(): Graphics | null {
21885         if (!this._graphics?.isValid) {
21886             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21887         }
21888         return this._graphics;
21889     }
21890     private _redraw(): void {
21891         const g = this._ensureGraphics();
21892         const ut = this.getComponent(UITransform);
21893         if (!g || !ut) {
21894             return;
21895         }
21896         g.clear();
21897         const w = ut.width;
21898         const h = ut.height;
21899         const size = Math.min(w, h);
21900         const left = -ut.anchorX * w + (w - size) * 0.5;
21901         const bottom = -ut.anchorY * h + (h - size) * 0.5;
21902         drawPlayerEyes(
21903             g,
21904             left,
21905             bottom,
21906             size,
21907             Direction.Left,
21908             this._currentBlinkAmount(),
21909             false,
21910         );
21911     }
21912 }
21913 /** 为 GameTitleIcon 节点挂上主角图标绘制 */
21914 export function ensureMenuGameTitleIconView(iconNode: Node | null): void {
21915     if (!iconNode?.isValid) {
21916         return;
21917     }
21918     if (!iconNode.getComponent(MenuGameTitleIconView)) {
21919         iconNode.addComponent(MenuGameTitleIconView);
21920     }
21921 }
21922 // ----- assets/Scripts/MenuGameTitleView.ts -----
21923 import { _decorator, Component, Graphics, Node, Sprite, UITransform } from 'cc';
21924 import { drawGameTitleOutline } from './Games/OpticalPuzzle/Presentation/MenuGameTitleGlyph';

```

```

21925 const { ccclass } = _decorator;
21926 /** Menu/MainPanel/GameTitle: 矢量轮廓标题（替代 PNG Sprite） */
21927 @ccclass('MenuGameTitleView')
21928 export class MenuGameTitleView extends Component {
21929     private _graphics: Graphics | null = null;
21930     protected onLoad(): void {
21931         this._disableRasterTitle();
21932         this._ensureGraphics();
21933         this._redraw();
21934     }
21935     protected onEnable(): void {
21936         this._redraw();
21937     }
21938     private _disableRasterTitle(): void {
21939         const splash = this.node.getChildByName('SpriteSplash');
21940         if (splash?.isValid) {
21941             splash.active = false;
21942         }
21943         const sprite = this.getComponent(Sprite);
21944         if (sprite) {
21945             sprite.enabled = false;
21946         }
21947     }
21948     private _ensureGraphics(): Graphics | null {
21949         if (!this._graphics?.isValid) {
21950             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21951         }
21952         return this._graphics;
21953     }
21954     private _redraw(): void {
21955         const g = this._ensureGraphics();
21956         const ut = this.getComponent(UITransform);
21957         if (!g || !ut) {
21958             return;
21959         }
21960         g.clear();
21961         const w = ut.width;
21962         const h = ut.height;
21963         const left = -ut.anchorX * w;
21964         const bottom = -ut.anchorY * h;
21965         drawGameTitleOutline(g, left, bottom, w, h);
21966     }
21967 }
21968 /** 为 GameTitle 节点挂上矢量标题绘制 */
21969 export function ensureMenuGameTitleView(titleNode: Node | null): void {
21970     if (!titleNode?.isValid) {
21971         return;
21972     }
21973     if (!titleNode.getComponent(MenuGameTitleView)) {
21974         titleNode.addComponent(MenuGameTitleView);
21975     }
21976 }
21977 // ----- assets/Scripts/MenuLevelSelectButtonView.ts -----
21978 import {
21979     _decorator,
21980     Button,
21981     Component,
21982     Graphics,

```

```

21983     Node,
21984     UITransform,
21985 } from 'cc';
21986 import { HudButtonPressController } from
21987 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
21988 import { drawMenuLevelSelectButtonGlyph } from
21989 './Games/OpticalPuzzle/Presentation/MenuLevelSelectButtonGlyph';
21990 const { ccclass } = _decorator;
21991 /** 与开始键 Button.zoomScale 一致 */
21992 const LEVEL_SELECT_MENU_BUTTON_ZOOM_SCALE = 0.95;
21993 /** Menu/MainPanel/BtnLevelSelect: 圆形键帽 + 四宫格 icon (点击由 MenuManager 绑定) */
21994 @ccclass('MenuLevelSelectButtonView')
21995 export class MenuLevelSelectButtonView extends Component {
21996     private _graphics: Graphics | null = null;
21997     private _pressed = false;
21998     private _pressCtrl: HudButtonPressController | null = null;
21999     protected onLoad(): void {
22000         this._hidePlaceholderSplash();
22001         this._ensureButton();
22002         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
22003             this._pressed = pressed;
22004             this._redraw();
22005         });
22006         this._redraw();
22007     }
22008     protected onEnable(): void {
22009         this._pressCtrl?.bind();
22010         this._redraw();
22011     }
22012     protected onDisable(): void {
22013         this._pressCtrl?.unbind();
22014         this._pressed = false;
22015     }
22016     protected onDestroy(): void {
22017         this._pressCtrl?.unbind();
22018     }
22019     private _hidePlaceholderSplash(): void {
22020         const splash = this.node.getChildByName('SpriteSplash') ?? this.node.getChildByName('bg');
22021         if (splash?.isValid) {
22022             splash.active = false;
22023         }
22024     }
22025     private _ensureButton(): void {
22026         let btn = this.getComponent(Button);
22027         if (!btn) {
22028             btn = this.addComponent(Button);
22029             btn.transition = Button.Transition.SCALE;
22030         }
22031         btn.zoomScale = LEVEL_SELECT_MENU_BUTTON_ZOOM_SCALE;
22032         btn.transition = Button.Transition.SCALE;
22033     }
22034     private _ensureGraphics(): Graphics | null {
22035         if (!this._graphics?.isValid) {
22036             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
22037         }
22038         return this._graphics;
22039     }
22040     private _redraw(): void {

```

```

22041         const g = this._ensureGraphics();
22042         const ut = this.getComponent(UITransform);
22043         if (!g || !ut) {
22044             return;
22045         }
22046         g.clear();
22047         const w = ut.width;
22048         const h = ut.height;
22049         const left = -ut.anchorX * w;
22050         const bottom = -ut.anchorY * h;
22051         drawMenuLevelSelectButtonGlyph(g, left, bottom, w, h, this._pressed);
22052     }
22053 }
22054 /** 为 MainPanel/BtnLevelSelect 挂上键帽绘制与按压缩放 */
22055 export function ensureMenuLevelSelectButtonView(btnLevelSelect: Node | null): void {
22056     if (!btnLevelSelect?.isValid) {
22057         return;
22058     }
22059     if (!btnLevelSelect.getComponent(MenuLevelSelectButtonView)) {
22060         btnLevelSelect.addComponent(MenuLevelSelectButtonView);
22061     }
22062 }
22063 // ----- assets/Scripts/MenuOverlayWindow.ts -----
22064 import { _decorator, Component, Node } from 'cc';
22065 import { RoundBox } from '../RoundBox';
22066 import { EVENT_ENUM, AUDIO_EFFECT_ENUM } from './Utils/Enum';
22067 import { PLAY_AUDIO } from './Utils/Event';
22068 const { ccclass, property } = _decorator;
22069 /**
22070  * 菜单简单弹层：挂在窗口根节点上，关闭键（及可选子节点 `bg` 衬底）点击后隐藏本节点
22071  */
22072 @ccclass('MenuOverlayWindow')
22073 export class MenuOverlayWindow extends Component {
22074     /** 关闭按钮 */
22075     @property(Node)
22076     closeButtonNode: Node = null;
22077     private _backdropBgNode: Node | null = null;
22078     protected onLoad(): void {
22079         this._backdropBgNode = this.node.getChildByName('bg') ?? null;
22080         if (this._backdropBgNode) {
22081             this._backdropBgNode.on(Node.EventType.TOUCH_END, this.onCloseClick, this);
22082         }
22083         if (this.closeButtonNode) {
22084             this.closeButtonNode.on(Node.EventType.TOUCH_END, this.onCloseClick, this);
22085         }
22086     }
22087     protected onEnable(): void {
22088         // 根节点从场景初始 inactive 首次打开时，子节点 RoundBox 可能首帧未拿到 texture/chunk
22089         this.scheduleOnce(this.refreshChildRoundBoxes, 0);
22090     }
22091     private refreshChildRoundBoxes(): void {
22092         if (!this.node?.isValid || !this.node.activeInHierarchy) {
22093             return;
22094         }
22095         const list = this.node.getComponentsInChildren(RoundBox);
22096         for (let i = 0; i < list.length; i++) {
22097             list[i].forceRefreshRender();
22098         }

```

```

22099     }
22100     protected onDestroy(): void {
22101         if (this._backdropBgNode?.isValid) {
22102             this._backdropBgNode.off(Node.EventType.TOUCH_END, this.onCloseClick, this);
22103         }
22104         this._backdropBgNode = null;
22105         if (this.closeButtonNode?.isValid) {
22106             this.closeButtonNode.off(Node.EventType.TOUCH_END, this.onCloseClick, this);
22107         }
22108     }
22109     private onCloseClick(): void {
22110         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
22111         this.node.active = false;
22112     }
22113 }
22114 // ----- assets/Scripts/MenuShareButtonView.ts -----
22115 import {
22116     _decorator,
22117     Button,
22118     Component,
22119     Graphics,
22120     Node,
22121     UITransform,
22122 } from 'cc';
22123 import { HudButtonPressController } from
22124 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
22125 import { drawMenuShareButtonGlyph } from './Games/OpticalPuzzle/Presentation/MenuShareButtonGlyph';
22126 const { ccclass } = _decorator;
22127 /** 与开始键 Button.zoomScale 一致 */
22128 const SHARE_MENU_BUTTON_ZOOM_SCALE = 0.95;
22129 /** Menu/MainPanel/BtnShare: 圆形键帽 + 分享 icon (点击由 MenuManager 绑定) */
22130 @ccclass('MenuShareButtonView')
22131 export class MenuShareButtonView extends Component {
22132     private _graphics: Graphics | null = null;
22133     private _pressed = false;
22134     private _pressCtrl: HudButtonPressController | null = null;
22135     protected onLoad(): void {
22136         this._hidePlaceholderSplash();
22137         this._ensureButton();
22138         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
22139             this._pressed = pressed;
22140             this._redraw();
22141         });
22142         this._redraw();
22143     }
22144     protected onEnable(): void {
22145         this._pressCtrl?.bind();
22146         this._redraw();
22147     }
22148     protected onDisable(): void {
22149         this._pressCtrl?.unbind();
22150         this._pressed = false;
22151     }
22152     protected onDestroy(): void {
22153         this._pressCtrl?.unbind();
22154     }
22155     private _hidePlaceholderSplash(): void {
22156         const splash = this.node.getChildByName('SpriteSplash') ?? this.node.getChildByName('bg');

```

```

22157         if (splash?.isValid) {
22158             splash.active = false;
22159         }
22160     }
22161     private _ensureButton(): void {
22162         let btn = this.getComponent(Button);
22163         if (!btn) {
22164             btn = this.addComponent(Button);
22165             btn.transition = Button.Transition.SCALE;
22166         }
22167         btn.zoomScale = SHARE_MENU_BUTTON_ZOOM_SCALE;
22168         btn.transition = Button.Transition.SCALE;
22169     }
22170     private _ensureGraphics(): Graphics | null {
22171         if (!this._graphics?.isValid) {
22172             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
22173         }
22174         return this._graphics;
22175     }
22176     private _redraw(): void {
22177         const g = this._ensureGraphics();
22178         const ut = this.getComponent(UITransform);
22179         if (!g || !ut) {
22180             return;
22181         }
22182         g.clear();
22183         const w = ut.width;
22184         const h = ut.height;
22185         const left = -ut.anchorX * w;
22186         const bottom = -ut.anchorY * h;
22187         drawMenuShareButtonGlyph(g, left, bottom, w, h, this._pressed);
22188     }
22189 }
22190 /** 为 MainPanel/BtnShare 挂上键帽绘制与按压缩放 */
22191 export function ensureMenuShareButtonView(btnShare: Node | null): void {
22192     if (!btnShare?.isValid) {
22193         return;
22194     }
22195     if (!btnShare.getComponent(MenuShareButtonView)) {
22196         btnShare.addComponent(MenuShareButtonView);
22197     }
22198 }
22199 // ----- assets/Scripts/MenuStartButtonView.ts -----
22200 import {
22201     _decorator,
22202     Button,
22203     Component,
22204     Graphics,
22205     Node,
22206     UITransform,
22207 } from 'cc';
22208 import { HudButtonPressController } from
22209 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
22210 import { drawMenuStartButtonGlyph } from './Games/OpticalPuzzle/Presentation/MenuStartButtonGlyph';
22211 const { ccclass } = _decorator;
22212 /** 与 nextlevel / 四向键 Button.zoomScale 一致 */
22213 const START_BUTTON_ZOOM_SCALE = 0.95;
22214 /** Menu/MainPanel/BtnStart: 圆形键帽（点击逻辑由 MenuManager 绑定） */

```

```

22215 @cclass('MenuStartButtonView')
22216 export class MenuStartButtonView extends Component {
22217     private _graphics: Graphics | null = null;
22218     private _pressed = false;
22219     private _pressCtrl: HudButtonPressController | null = null;
22220     protected onLoad(): void {
22221         this._hidePlaceholderBg();
22222         this._ensureButton();
22223         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
22224             this._pressed = pressed;
22225             this._redraw();
22226         });
22227         this._redraw();
22228     }
22229     protected onEnable(): void {
22230         this._pressCtrl?.bind();
22231         this._redraw();
22232     }
22233     protected onDisable(): void {
22234         this._pressCtrl?.unbind();
22235         this._pressed = false;
22236     }
22237     protected onDestroy(): void {
22238         this._pressCtrl?.unbind();
22239     }
22240     private _hidePlaceholderBg(): void {
22241         const bg = this.node.getChildByName('bg');
22242         if (bg?.isValid) {
22243             bg.active = false;
22244         }
22245     }
22246     private _ensureButton(): void {
22247         let btn = this.getComponent(Button);
22248         if (!btn) {
22249             btn = this.addComponent(Button);
22250             btn.transition = Button.Transition.SCALE;
22251         }
22252         btn.zoomScale = START_BUTTON_ZOOM_SCALE;
22253         btn.transition = Button.Transition.SCALE;
22254     }
22255     private _ensureGraphics(): Graphics | null {
22256         if (!this._graphics?.isValid) {
22257             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
22258         }
22259         return this._graphics;
22260     }
22261     private _redraw(): void {
22262         const g = this._ensureGraphics();
22263         const ut = this.getComponent(UITransform);
22264         if (!g || !ut) {
22265             return;
22266         }
22267         g.clear();
22268         const w = ut.width;
22269         const h = ut.height;
22270         const left = -ut.anchorX * w;
22271         const bottom = -ut.anchorY * h;
22272         drawMenuStartButtonGlyph(g, left, bottom, w, h, this._pressed);

```



```

22273     }
22274 }
22275 /** 为 MainPanel/BtnStart 挂上键帽绘制与按压缩放 */
22276 export function ensureMenuStartButtonView(btnStart: Node | null): void {
22277     if (!btnStart?.isValid) {
22278         return;
22279     }
22280     if (!btnStart.getComponent(MenuStartButtonView)) {
22281         btnStart.addComponent(MenuStartButtonView);
22282     }
22283 }
22284 // ----- assets/Scripts/Utils/Enum.ts -----
22285 /**
22286  * 全局枚举集中定义（与 project-rules 一致）。
22287  * 光学解谜专用事件名使用 OPTICAL_ 前缀，避免与通用事件混淆。
22288  */
22289 export enum EVENT_ENUM {
22290     /** 音效播放（载荷为 AUDIO_EFFECT_ENUM） */
22291     PLAY_AUDIO = 'PLAY_AUDIO',
22292     /** 背景音乐切换 */
22293     PLAY_BGM = 'PLAY_BGM',
22294     /** 全局游戏状态变更 */
22295     GAME_STATE_CHANGE = 'GAME_STATE_CHANGE',
22296     /** 光学解谜棋盘快照更新（载荷为 OpticalBoardSnapshot，由 Presentation 监听） */
22297     OPTICAL_SNAPSHOT_CHANGED = 'OPTICAL_SNAPSHOT_CHANGED',
22298     /** Toast 提示（载荷 { message, bgWidth?, localY? }） */
22299     SHOW_TOAST = 'SHOW_TOAST',
22300 }
22301 /** 音效类型（按需扩展；播放经 PLAY_AUDIO 事件，由 MusicManager 映射 AudioClip） */
22302 export enum AUDIO_EFFECT_ENUM {
22303     /** UI 通用按钮点击 */
22304     CLICK_BUTTON = 'CLICK_BUTTON',
22305     /** 四向键：移动成功（走一格或推动元件） */
22306     OPTICAL_MOVE_SUCCESS = 'OPTICAL_MOVE_SUCCESS',
22307     /** 四向键：移动失败（与主角「><」阻拦脸同一时机） */
22308     OPTICAL_MOVE_FAIL = 'OPTICAL_MOVE_FAIL',
22309     /** 单个目标灯被点亮 */
22310     OPTICAL_TARGET_LIT = 'OPTICAL_TARGET_LIT',
22311     /** 关卡通关成功 */
22312     OPTICAL_LEVEL_COMPLETE = 'OPTICAL_LEVEL_COMPLETE',
22313 }
22314 /** 背景音乐种类（PLAY_BGM 刷新用；全局单曲 Bg.mp3，不再区分 Menu/Game） */
22315 export enum BGM_KIND_ENUM {
22316     GLOBAL = 'GLOBAL',
22317 }
22318 /** 场景名与 build 设置中场景文件名一致 */
22319 export enum SCENE_ENUM {
22320     MENU = 'Menu',
22321     /** 局内场景占位，创建 Game 场景后改为实际名称 */
22322     GAME = 'Game',
22323 }
22324 /** 与 project-rules 示例一致的全局游戏状态（跨玩法可复用） */
22325 export enum GAME_STATE_ENUM {
22326     INIT,
22327     RUNNING,
22328     PAUSED,
22329     LOSED,
22330     COMPLETE,

```

```

22331 }
22332 // ----- assets/Scripts/Utils/Event.ts -----
22333 import { EventTarget } from 'cc';
22334 /** 全局音效 */
22335 export const PLAY_AUDIO = new EventTarget();
22336 /** 背景音乐 */
22337 export const PLAY_BGM = new EventTarget();
22338 /** 全局状态 */
22339 export const GAME_STATE_CHANGE = new EventTarget();
22340 /** 光学解谜: 棋盘 / HUD 刷新等 */
22341 export const OPTICAL_PUZZLE = new EventTarget();
22342 /** 全局 Toast 提示 (载荷 { message, bgWidth?, localY?, 由局内 Presentation 转发到 ToastManager) */
22343 export const SHOW_TOAST = new EventTarget();
22344 // ----- assets/Scripts/Utils/Utils.ts -----
22345 /**
22346  * 通用工具函数 (与 project-rules 目录约定一致)。
22347  */
22348 export function clamp(value: number, min: number, max: number): number {
22349     return Math.max(min, Math.min(max, value));
22350 }
22351 // ----- assets/Scripts/Utils/WeChatMiniGameAds.ts -----
22352 import { director } from 'cc';
22353 import { EVENT_ENUM, SCENE_ENUM } from './Enum';
22354 import { PLAY_BGM } from './Event';
22355 /**
22356  * 微信小游戏流量主 — 激励视频 (参考 MemoMaster)。
22357  * 文档: https://developers.weixin.qq.com/minigame/dev/api/ad/wx.createRewardedVideoAd.html
22358  */
22359 const LOG = '[WeChatMiniGameAds]';
22360 /**
22361  * 开发测试: true 时不拉真实广告, 直接 resolve true。
22362  * 发版前务必改回 false。
22363  */
22364 export let USE_DEBUG MOCK_REWARDED_AD_SUCCESS = true; // mock 广告返回 true
22365 /** 激励式视频广告位 id (流量主后台; 留空则 show 直接 resolve false) */
22366 export let WECHAT_REWARDED_VIDEO_AD_UNIT_ID = '';
22367 interface RewardedVideoCloseResult {
22368     isEnded?: boolean;
22369 }
22370 interface IRewardedVideoAd {
22371     load(): Promise<void>;
22372     show(): Promise<void>;
22373     onClose(listener: (res?: RewardedVideoCloseResult) => void): void;
22374     offClose(listener: (res?: RewardedVideoCloseResult) => void): void;
22375     onError(listener: (err: unknown) => void): void;
22376 }
22377 interface IWxAds {
22378     createRewardedVideoAd?(opt: { adUnitId: string; multiton?: boolean }): IRewardedVideoAd | null;
22379 }
22380 function getWx(): IWxAds | undefined {
22381     const g = globalThis as unknown as { wx?: IWxAds };
22382     return g.wx;
22383 }
22384 let _rewarded: IRewardedVideoAd | null = null;
22385 let _rewardedErrorBound = false;
22386 let _rewardedShowInFlight = false;
22387 let _warnedEmptyRewardedId = false;
22388 let _firstPreloadTimer: ReturnType<typeof setTimeout> | null = null;

```

```
22389 const REWARDED_PRELOAD_DELAY_MS = 2000;
22390 const REWARDED_RESUME_BGM_DELAY_MS = 100;
22391 function scheduleResumeBgmAfterRewardedAd(): void {
22392     setTimeout(() => {
22393         const scene = director.getScene()?.name;
22394         if (scene === SCENE_ENUM.MENU || scene === SCENE_ENUM.GAME) {
22395             PLAY_BGM.emit(EVENT_ENUM.PLAY_BGM, scene);
22396         }
22397     }, REWARDED_RESUME_BGM_DELAY_MS);
22398 }
22399 function bindRewardedErrorOnce(ad: IRewardedVideoAd): void {
22400     if (_rewardedErrorBound) {
22401         return;
22402     }
22403     _rewardedErrorBound = true;
22404     ad.onError((err: unknown) => {
22405         console.warn(LOG, 'Rewarded onError', err);
22406     });
22407 }
22408 function getOrCreateRewarded(adUnitId: string): IRewardedVideoAd | null {
22409     const wxApi = getWx();
22410     if (typeof wxApi?.createRewardedVideoAd !== 'function') {
22411         return null;
22412     }
22413     if (!_rewarded) {
22414         const ad = wxApi.createRewardedVideoAd({ adUnitId });
22415         if (!ad) {
22416             return null;
22417         }
22418         _rewarded = ad;
22419         bindRewardedErrorOnce(ad);
22420     }
22421     return _rewarded;
22422 }
22423 export function isWeChatRewardedVideoAdConfigured(): boolean {
22424     return WECHAT_REWARDED_VIDEO_AD_UNIT_ID.trim().length > 0;
22425 }
22426 export function cancelWeChatRewardedVideoPreloadSchedule(): void {
22427     if (_firstPreloadTimer !== null) {
22428         clearTimeout(_firstPreloadTimer);
22429         _firstPreloadTimer = null;
22430     }
22431 }
22432 export function initWeChatRewardedVideoAd(delayMs: number): void {
22433     cancelWeChatRewardedVideoPreloadSchedule();
22434     const run = (): void => {
22435         _firstPreloadTimer = null;
22436         preloadWeChatRewardedVideo();
22437     };
22438     if (delayMs <= 0) {
22439         run();
22440         return;
22441     }
22442     _firstPreloadTimer = setTimeout(run, delayMs);
22443 }
22444 /** Game 场景：延迟预载，避开开局动画 */
22445 export function scheduleWeChatRewardedVideoPreloadForGame(): void {
22446     initWeChatRewardedVideoAd(REWARDED_PRELOAD_DELAY_MS);
```

```

22447 }
22448 /** 展示激励视频；完整观看返回 true，否则 false */
22449 export function showWeChatRewardedVideo(): Promise<boolean> {
22450   if (USE_DEBUG MOCK_REWARDED_AD_SUCCESS) {
22451     console.warn(LOG, 'USE_DEBUG MOCK_REWARDED_AD_SUCCESS=true，模拟观看成功');
22452     return Promise.resolve(true);
22453   }
22454   const id = WECHAT_REWARDED_VIDEO_AD_UNIT_ID.trim();
22455   if (!id) {
22456     if (!_warnedEmptyRewardedId) {
22457       _warnedEmptyRewardedId = true;
22458       console.warn(LOG, 'WECHAT_REWARDED_VIDEO_AD_UNIT_ID 为空，已跳过激励视频');
22459     }
22460     return Promise.resolve(false);
22461   }
22462   const wxApi = getWx();
22463   if (typeof wxApi?.createRewardedVideoAd !== 'function') {
22464     return Promise.resolve(false);
22465   }
22466   if (_rewardedShowInFlight) {
22467     console.warn(LOG, '激励视频已在展示/加载中，忽略本次');
22468     return Promise.resolve(false);
22469   }
22470   const ad = getOrCreateRewarded(id);
22471   if (!ad) {
22472     return Promise.resolve(false);
22473   }
22474   _rewardedShowInFlight = true;
22475   return new Promise<boolean>((resolve) => {
22476     let settled = false;
22477     const finish = (ok: boolean): void => {
22478       if (settled) {
22479         return;
22480       }
22481       settled = true;
22482       _rewardedShowInFlight = false;
22483       ad.offClose(onClose);
22484       resolve(ok);
22485     };
22486     const onClose = (res?: RewardedVideoCloseResult): void => {
22487       const ok = !res || res.isEnded === true;
22488       finish(ok);
22489       scheduleResumeBgmAfterRewardedAd();
22490       preloadWeChatRewardedVideo();
22491     };
22492     ad.onClose(onClose);
22493     ad.show()
22494       .catch(() => ad.load().then(() => ad.show()))
22495       .catch((err: unknown) => {
22496         console.warn(LOG, '激励视频 广告显示失败', err);
22497         finish(false);
22498       });
22499   });
22500 }
22501 export function preloadWeChatRewardedVideo(): void {
22502   const id = WECHAT_REWARDED_VIDEO_AD_UNIT_ID.trim();
22503   if (!id) {
22504     return;

```

```

22505     }
22506     const ad = getOrCreateRewarded(id);
22507     if (!ad) {
22508         return;
22509     }
22510     ad.load().catch((err: unknown) => {
22511         console.warn(LOG, 'Rewarded preload 失败', err);
22512     });
22513 }
22514 // ----- assets/Scripts/Utils/WeChatRewardedVideoAd.ts -----
22515 /**
22516  * 兼容入口：激励视频实现见 `WeChatMiniGameAds.ts`（与 MemoMaster 一致）。
22517  */
22518 export {
22519     WECHAT_REWARDED_VIDEO_AD_UNIT_ID,
22520     USE_DEBUG MOCK_REWARDED_AD_SUCCESS,
22521     isWeChatRewardedVideoAdConfigured,
22522     initWeChatRewardedVideoAd,
22523     cancelWeChatRewardedVideoPreloadSchedule,
22524     scheduleWeChatRewardedVideoPreloadForGame,
22525     showWeChatRewardedVideo,
22526     preloadWeChatRewardedVideo,
22527 } from './WeChatMiniGameAds';
22528 // ----- assets/Scripts/Utils/WeChatShare.ts -----
22529 /**
22530  * 微信小游戏：右上角「转发 / 朋友圈 / 复制链接」需 showShareMenu + 分享回调，否则常为灰色。
22531  * `ensureWxShareMenuAndPassiveHooks` 可多次调用：重复执行 `showShareMenu`（点击前再补一次可避
22532 免「不调起面板」）。
22533  */
22534 /** 与转发、朋友圈展示文案一致时可改此处 */
22535 const SHARE_TITLE_FRIEND = '光学解谜 — 你能点亮全部接收器吗？';
22536 const SHARE_TITLE_TIMELINE = '光学解谜 — 推光路大挑战';
22537 /**
22538  * 分享预览图（imageUrl）。
22539  * 留空：不传 imageUrl，微信会用当前游戏画面截图（稳定，推荐默认）
22540  */
22541 const SHARE_PREVIEW_IMAGE_HTTPS = '';
22542 const SHARE_FOREGROUND_DEBOUNCE_MS = 320;
22543 const SHARE_RETURN_AFTER_HIDE_MS = 80;
22544 const SHARE_NO_HIDE_FALLBACK_MS = 520;
22545 const SHARE_FAIL_TIMEOUT_MS = 25000;
22546 interface ShareAppMessageOptions {
22547     title?: string;
22548     imageUrl?: string;
22549     query?: string;
22550     imageUrlId?: string;
22551 }
22552 interface IWxMiniGameShare {
22553     showShareMenu(opt: { withShareTicket?: boolean; menus?: string[] }): void;
22554     onShareAppMessage(fn: () => { title?: string; imageUrl?: string; query?: string }): void;
22555     onShareTimeline?(fn: () => { title?: string; imageUrl?: string; query?: string }): void;
22556     shareAppMessage?(opt: ShareAppMessageOptions): void;
22557     onShow?(fn: (res?: unknown) => void): void;
22558     onHide?(fn: () => void): void;
22559     offShow?(fn: (res?: unknown) => void): void;
22560     offHide?(fn: () => void): void;
22561 }
22562 function getWx(): IWxMiniGameShare | undefined {

```

```

22563     const g = globalThis as unknown as { wx?: IWxMiniGameShare };
22564     return g.wx;
22565 }
22566 function buildFriendSharePayload(): { title: string; imageUrl?: string } {
22567     const o: { title: string; imageUrl?: string } = { title: SHARE_TITLE_FRIEND };
22568     if (SHARE_PREVIEW_IMAGE_HTTPS) {
22569         o.imageUrl = SHARE_PREVIEW_IMAGE_HTTPS;
22570     }
22571     return o;
22572 }
22573 let _hooksBound = false;
22574 function ensureWxShareMenuAndPassiveHooks(): void {
22575     const wxApi = getWx();
22576     if (!wxApi) {
22577         return;
22578     }
22579     try {
22580         if (typeof wxApi.showShareMenu === 'function') {
22581             wxApi.showShareMenu({
22582                 withShareTicket: true,
22583                 menus: ['shareAppMessage', 'shareTimeline'],
22584             });
22585         }
22586     } catch {
22587         /* 部分环境首帧不可用，稍后点击再调 */
22588     }
22589     if (_hooksBound) {
22590         return;
22591     }
22592     if (typeof wxApi.onShareAppMessage !== 'function') {
22593         return;
22594     }
22595     wxApi.onShareAppMessage(() => buildFriendSharePayload());
22596     const sharePayloadTimeline = () => {
22597         const o: { title: string; imageUrl?: string } = { title: SHARE_TITLE_TIMELINE };
22598         if (SHARE_PREVIEW_IMAGE_HTTPS) {
22599             o.imageUrl = SHARE_PREVIEW_IMAGE_HTTPS;
22600         }
22601         return o;
22602     };
22603     if (typeof wxApi.onShareTimeline === 'function') {
22604         wxApi.onShareTimeline(sharePayloadTimeline);
22605     }
22606     _hooksBound = true;
22607 }
22608 export function trySettlePendingShareAfterWxOnShow(): void {
22609     handleShareAwaitOnForeground();
22610 }
22611 type ShareAwait = {
22612     onSuccess: () => void;
22613     onFail: (reason?: string) => void;
22614     startAt: number;
22615     hideAt: number | null;
22616     settled: boolean;
22617     hideListener: (() => void) | null;
22618     failTimer: ReturnType<typeof setTimeout> | null;
22619 };
22620 let _shareAwait: ShareAwait | null = null;

```

```
22621 function cancelShareAwaitHooks(s: ShareAwait): void {
22622     const wxApi = getWx();
22623     if (s.hideListener && typeof wxApi?.offHide === 'function') {
22624         wxApi.offHide(s.hideListener);
22625     }
22626     s.hideListener = null;
22627     if (s.failTimer !== null) {
22628         clearTimeout(s.failTimer);
22629         s.failTimer = null;
22630     }
22631 }
22632 function cancelPendingShareAwaitSilent(): void {
22633     const s = _shareAwait;
22634     if (!s || s.settled) {
22635         return;
22636     }
22637     s.settled = true;
22638     cancelShareAwaitHooks(s);
22639     _shareAwait = null;
22640 }
22641 function settleShareAwait(mode: 'success' | 'fail', reason?: string): void {
22642     const s = _shareAwait;
22643     if (!s || s.settled) {
22644         return;
22645     }
22646     s.settled = true;
22647     cancelShareAwaitHooks(s);
22648     _shareAwait = null;
22649     if (mode === 'success') {
22650         s.onSuccess();
22651     } else {
22652         s.onFail(reason);
22653     }
22654 }
22655 function handleShareAwaitOnForeground(): void {
22656     const s = _shareAwait;
22657     if (!s || s.settled) {
22658         return;
22659     }
22660     const now = Date.now();
22661     const sinceStart = now - s.startAt;
22662     if (sinceStart < SHARE_FOREGROUND_DEBOUNCE_MS) {
22663         return;
22664     }
22665     if (s.hideAt !== null && now - s.hideAt >= SHARE_RETURN_AFTER_HIDE_MS) {
22666         settleShareAwait('success');
22667         return;
22668     }
22669     if (s.hideAt === null && sinceStart >= SHARE_NO_HIDE_FALLBACK_MS) {
22670         settleShareAwait('success');
22671     }
22672 }
22673 export function initWeChatMiniGameShare(): void {
22674     ensureWxShareMenuAndPassiveHooks();
22675 }
22676 export function invokeWeChatFriendShare(cb: {
22677     onSuccess?: () => void;
22678     onFail?: (reason?: string) => void;
```

```
22679   }): void {
22680     const wxApi = getWx();
22681     if (!wxApi || typeof wxApi.shareAppMessage !== 'function') {
22682       cb.onFail?.('unsupported');
22683       return;
22684     }
22685     ensureWxShareMenuAndPassiveHooks();
22686     cancelPendingShareAwaitSilent();
22687     const onSuccess = cb.onSuccess ?? (() => undefined);
22688     const onFail = cb.onFail ?? (() => undefined);
22689     const hideListener = (): void => {
22690       const s = _shareAwait;
22691       if (!s || s.settled) {
22692         return;
22693       }
22694       s.hideAt = Date.now();
22695     };
22696     const awaitState: ShareAwait = {
22697       onSuccess,
22698       onFail,
22699       startAt: Date.now(),
22700       hideAt: null,
22701       settled: false,
22702       hideListener: null,
22703       failTimer: null,
22704     };
22705     _shareAwait = awaitState;
22706     if (typeof wxApi.onHide === 'function') {
22707       awaitState.hideListener = hideListener;
22708       wxApi.onHide(hideListener);
22709     }
22710     awaitState.failTimer = setTimeout(() => {
22711       settleShareAwait('fail', 'timeout');
22712     }, SHARE_FAIL_TIMEOUT_MS);
22713     try {
22714       const base = buildFriendSharePayload();
22715       wxApi.shareAppMessage({
22716         title: base.title,
22717         ...(base.imageUrl ? { imageUrl: base.imageUrl } : {}),
22718       });
22719     } catch {
22720       settleShareAwait('fail', 'error');
22721     }
22722   }
```